SMART-BIN: AN ESP32-BASED IoT WASTE BIN MONITORING SYSTEM WITH FILL, WEIGHT, AIR-QUALITY AND TAMPER DETECTION FOR PRIORITY-BASED COLLECTION

WASTE MANAGEMENT AND CLEANLINESS

PROBLEM: Overflowing Garbage Bins and Inefficient Waste Collection Routes

Technological University of the Philippines - Manila

Bachelor of Engineering Technology major in Computer Engineering Technology

Embedded Systems: Final Project

BET-CPET 4A-M, Group 2

Submitted By:

Brian Jay Celeste

Genesis Walter D. Reyes II

Jade C. Gunayan

King Perth G. Pepito

Rodel S. Banogbanog

Submitted To:

Prof. Jonel R. Macalisang

November, 2026

TABLE OF CONTENTS

TITLE PAGE ..... **i**
TABLE OF CONTENTS ..... **ii**
LIST OF FIGURES ..... **iv**
LIST OF TABLES ..... **ix**
I. INTRODUCTION **1**
1.1 Introduction and Problem 1
1.2 Sustainable Development Goal (SDG) Addressed 2
1.3 Review of Related Literature 3
1.4 Objectives 54
1.5 Conceptual Framework 55
1.6 Target Beneficiaries 59
II. PROTOTYPE DESIGN AND DEVELOPMENT **60**
2.1 Development Methodology 60
2.2 Prototype Design 64
2.3 Block Diagram 66
2.4 System Flowchart 68
2.5 Schematic Design 74
2.6 Component Materials 77
III. TESTING, RESULTS, AND CONCLUSION **88**
3.1 Test Cases 88
3.2 Results and Discussion 93
3.3 Conclusion 105
3.4 Recommendations 106
REFERENCES ..... **ix**
CURRICULUM VITAE ..... **xix**
Rodel S. Banogbanog xix
Brian Jay Celeste xx
Jade C. Gunayan xxii
King Perth G. Pepito xxiii
Genesis Walter D. Reyes II xxiv

APPENDICES ..... **xxv**

Appendix A: Gantt Chart xxvi

Appendix B: Bill of Materials of the Smart-Bin Prototype xxvii

Appendix C: User Interface Screens xxviii

Appendix D: User Manual lviii

Appendix E: Source Code lxxvii

LIST OF FIGURES

Figure 1. Sustainable Development Goal 11: Sustainable Cities and Communities..... 2

Figure 2. Patents Reviewed by Priority Year and Region..... 6

Figure 3. Remote Status Units Linked to a Central Unit..... 7

Figure 4. Compacting Ram and Chambers of the Solar Compactor..... 8

Figure 5. Path of a Full Message From the Compactor Monitor to the Service Provider..... 9

Figure 6. Trash Can With a Motor-Driven Lid..... 10

Figure 7. Container With a Bag Between Capacitive Plates..... 11

Figure 8. Waste Collection System of Containers, Server and Vehicles..... 12

Figure 9. Refuse Container With Scale, Display and Lights..... 13

Figure 10. Sensor Device Mounted Under the Lid Behind a Thermal Barrier..... 14

Figure 11. Gauge Mounted Under the Lid of a Container..... 15

Figure 12. Smart Garbage Bin With a Lid-Mounted Sensor Device..... 16

Figure 13. Pay-by-Weight Food Waste Processor With a Load-Cell Hopper..... 17

Figure 14. Infrared Beam Across a Container..... 18

Figure 15. Circuit of the Induction Flip Lid With a Settable Sensing Distance..... 19

Figure 16. Overall Smart Waste Management System..... 20

Figure 17. Garbage Cans and the Monitoring Center..... 21

Figure 18. Waste Measurement System With Weighing Devices and Reports..... 22

Figure 19. Block Diagram of the Capacity-Detecting Bin..... 23

Figure 20. Electrical Block Diagram of the Bin Intelligent Controller..... 24

Figure 21. Automatic Alerting System for a Trash Can..... 25

Figure 22. Network of Waste Bins, Cloud Server and Vehicles..... 26

Figure 23. Block Diagram of the Smart Waste Management System..... 29

Figure 24. Circuit Setup of the LoRa Bin Unit..... 30

Figure 25. Proposed System Architecture for Hospital Waste Monitoring..... 32

Figure 26. Proposed Network Architecture for Waste Collection in Bukavu..... 34

Figure 27. Prototype of the Smart Dustbin 35
Figure 28. Block Diagram of the Arduino-Based Smart Dustbin 37
Figure 29. Data Communication Structure of the Smart Waste Management System 38
Figure 30. Prototype of the SmartTrashBin 39
Figure 31. Data Flow of the Garbage Monitoring System 40
Figure 32. Power Chain of the ESP32-CAM Node 44
Figure 33. SWOT Analysis of Smart-Bin 49
Figure 34. Features Found in the 30 Related Studies 52
Figure 35. Input-Process-Output Model of Smart-Bin 55
Figure 36. System Architecture of Smart-Bin 57
Figure 37. Water-Scrum-Fall Development Life Cycle of Smart-Bin 61
Figure 38. Front View and Side Cross-Section of One Smart-Bin Unit 64
Figure 39. Block Diagram of the Smart-Bin System 66
Figure 40. System Flowchart of the Smart-Bin 68
Figure 41. Operator Flowchart of the Smart-Bin Apps 70
Figure 42. Collector Flowchart of the Smart-Bin Apps 72
Figure 43. Schematic Diagram, Sheet 1: Microcontroller 74
Figure 44. Schematic Diagram, Sheet 2: Actuators and Outputs 75
Figure 45. Schematic Diagram, Sheet 3: Sensors and Inputs 76
Figure 46. ESP32 DevKit V1 (38-Pin, ESP32-WROOM-32D) 77
Figure 47. HC-SR04 Ultrasonic Distance Sensor 78
Figure 48. FC-51 IR Obstacle Avoidance Sensor 79
Figure 49. BME680 Environmental Sensor 80
Figure 50. BMI160 Six-Axis Inertial Measurement Unit 81
Figure 51. HX711 Load Cell Amplifier and 5kg Load Cell 82
Figure 52. SG90 Micro Servo Motor 83
Figure 53. 2.4-in. TFT Touchscreen (ILI9341 With XPT2046 Touch Controller) 84

Figure 54. KY-016 RGB LED Module 85

Figure 55. Passive Buzzer Module (MH-FMD) 86

Figure 56. 18650-Battery Shields for Bin A (4-Cell) and Bin B (2-Cell) 87

Figure 57. The Two Built Smart-Bin Units 89

Figure 58. Inside Bin A: Trash Chamber From Above (a) and Controller and Battery Compartment (b) 90

Figure 59. Lid Test: Hand in Front of the IR Sensor (a) and the Lid Opened (b) 97

Figure 60. Bin 01 With the Original PIR Sensor During the First Test 98

Figure 61. Home Screen of Bin 01, With the IR Sensor Above It 103

Figure C1. Start Page of the Web Dashboard xxix

Figure C2. Creating the Operator PIN xxx

Figure C3. Overview Tab with Bin Cards and Collection Priority xxxi

Figure C4. Overview Tab in Grid View xxxii

Figure C5. Bin Detail Opened from the Grid xxxiii

Figure C6. Charts Tab xxxiv

Figure C7. Settings Tab in Calibration Mode: Fill Range, Alert Limits and Air Sensor xxxv

Figure C8. Settings Tab in Calibration Mode: Tilt Sensor, Lid, Weight and Touchscreen xxxvi

Figure C9. Bluetooth Setup Panel xxxvii

Figure C10. Cloud Backup Panel for Firebase and ThingSpeak xxxviii

Figure C11. Operator Access Panel xxxix

Figure C12. Demo Options Panel xl

Figure C13. Sound Test Panel xli

Figure C14. Simulation Tab with One Sensor in Manual xlii

Figure C15. Alerts Tab xliii

Figure C16. Overview Tab in Dark Theme xliv

Figure C17. Start Page and Operator PIN in the Android App xlv

Figure C18. Overview Tab with Banners, Collection Priority and Bin Grid xlvi

Figure C19. Starting and Finishing a Pickup from a Bin Card xlvii
Figure C20. Needs Attention Sheet and Alert History xlviii
Figure C21. Charts Tab in the Android App xlix
Figure C22. Settings Tab with Calibration Mode Off and On l
Figure C23. Calibration Settings: Fill Range, Alert Limits and Air Sensor li
Figure C24. Calibration Settings: Tilt Sensor, Lid, Weight and Touchscreen lii
Figure C25. Simulation Tab in the Android App liii
Figure C26. App Settings liv
Figure C27. Connection Page lv
Figure C28. Bluetooth Setup Page and Bin Settings Sheet lvi
Figure C29. Overview in List View, Light and Dark Themes lvii
Figure D1. User Manual Pages 1 and 2: Cover and Contents lix
Figure D2. User Manual Pages 3 and 4: Problem Finder and Signs Used lx
Figure D3. User Manual Pages 5 and 6: The System at a Glance and Outside the Bin lxi
Figure D4. User Manual Pages 7 and 8: Inside the Bin and Who Does What lxii
Figure D5. User Manual Pages 9 and 10: Charge and Switch On, and the Start-Up Check .. lxiii
Figure D6. User Manual Pages 11 and 12: Getting the App and Dashboard, and Putting a Bin on Wi-Fi lxiv
Figure D7. User Manual Pages 13 and 14: Bluetooth Setup and Throwing Trash Away lxv
Figure D8. User Manual Pages 15 and 16: The Bin's Screen and Other Screens lxvi
Figure D9. User Manual Pages 17 and 18: The Status Light and Bin Sounds lxvii
Figure D10. User Manual Pages 19 and 20: Emptying a Bin at the Bin and Collection Priority lxviii
Figure D11. User Manual Pages 21 and 22: Lock and Unlock, and the App Tour lxix
Figure D12. User Manual Pages 23 and 24: Dashboard Tour and Three Ways to Reach a Bin lxx
Figure D13. User Manual Pages 25 and 26: Charts and Readings, and Alerts lxxi

Figure D14. User Manual Pages 27 and 28: Tamper Alarm and Full, Heat and Bad-Air Alarms lxxii

Figure D15. User Manual Pages 29 and 30: Operator Mode and PIN, and Calibration lxxiii

Figure D16. User Manual Pages 31 and 32: Limits, Lid and Sound, and Simulation, Demo and Bins..... lxxiv

Figure D17. User Manual Pages 33 and 34: Lid and Screen Problems, and Reading Problems lxxv

Figure D18. User Manual Pages 35 and 36: Connection Problems and Quick Reference..... lxxvi

LIST OF TABLES

Table 1. Related Patents, by Priority Year 4

Table 2. Related Studies From 2023 to 2026 27

Table 3. Commercial and Student Systems as Described by Their Own Pages 47

Table 4. Technical Basis of the Main Design Choices 48

Table 5. The Four Closest Patents Compared With Smart-Bin 51

Table 6. The Research Gap by Capability 53

Table 7. Reference Instruments Used in the Tests 88

Table 8. Test Cases for Specific Objective 1: Sensor Monitoring and Detection 90

Table 9. Test Cases for Specific Objective 2: Real-Time Monitoring and Alerts 91

Table 10. Test Cases for Specific Objective 3: Bin Control and Collection Prioritization 92

Table 11. Fill Level Readings Against the Ruler (TC1.1) 93

Table 12. Weight Readings Against the Test Masses (TC1.2) 94

Table 13. Temperature and Humidity Against the Thermo-Hygrometer (TC1.3) 94

Table 14. Air Quality Response After the Warm-Up (TC1.4) 95

Table 15. Tamper Detection by Tilt Angle (TC1.5) 95

Table 16. Automatic Lid Response by Hand Distance (TC1.6) 96

Table 17. Time to Show a New Reading (TC2.1) 99

Table 18. ThingSpeak Logging Over 30 Minutes (TC2.2) 99

Table 19. Alert Delivery to the Phone (TC2.3) 100

Table 20. Offline Detection, Role Access and Two-Bin Results (TC2.4 to TC2.6) 101

Table 21. Collection Priority Order for Five of the Ten Combinations (TC3.1) 101

Table 22. Estimated and Actual Time to Reach the 80% Fill Limit (TC3.2) 102

Table 23. Lock, PIN Collection, Simulation and Calibration Results (TC3.3 to TC3.6) 102

Table 24. Summary of Test Results 104

I. INTRODUCTION

1.1 Introduction and Problem

An effective waste management plays a crucial role in creating and sustaining healthy, clean and prosperous communities. With the rising amount of waste that is being generated, it has become a problem for the local government and commercial waste collectors in the Philippines. The Philippines is estimated to be producing a total of 60.76 thousand tons of garbage every day in 2025, according to the Philippine Statistics Authority (PSA, 2026). Despite the significant actions taken by the City of Metro Manila, its garbage is projected to reach 9,498 MT/D in 2020, making it the country's waste capital (World Bank, 2022). This was projected to rise to 10,368 MT per day by 2025 (World Bank, 2022). Hence, a high priority of improving waste management and collection practices in the country. Most garbage collection systems are conducted on a fixed schedule, causing bins to overflow when they are picked up or not to be filled to capacity when collected. Additionally, the smell, pollution levels, temperature, tilting of garbage bins, and unauthorized access should also be monitored as they are important indicators of environmental conditions and the efficiency of garbage management.

The researchers propose the Smart-Bin which is an Internet of Things (IoT) based bin monitoring system based on the ESP32 microcontroller with the aim of improving garbage collection performance. Sensors will be used by Smart-Bin to determine the level of fullness, weight, temperature, humidity, and air quality. It'll also be tracking bins that tilt irregularly, and any unauthorized bin openings, through a web dashboard and mobile application. Smart-Bin includes a touchscreen at the bin for employees to view and acknowledge the collection instructions, as well as to set up PIN codes to select who can access the bin, and to make collection setting changes. The project will include the development of two Smart-Bin prototypes built at the table size that will feature the ESP32 boards, local Wi-Fi, ThingSpeak, web dashboard, Android app, and screen at the bin. Smart-Bin will arrange the list of collection priorities based on fill levels and how long they will take to fill up, with the aim of optimizing garbage collection and fuel consumption rates.

1.2 Sustainable Development Goal (SDG) Addressed

Figure 1

Sustainable Development Goal 11: Sustainable Cities and Communities

The project Smart-Bin is connected to one of the Sustainable Development Goals (SDGs) of the United Nations (Agenda 2030), namely SDG 11: Sustainable Cities and Communities. Smart-Bin is specifically targeting the environmental impact of cities, per capita, with particular attention to air quality and municipal and other waste management (United Nations Department of Economic and Social Affairs [UN DESA], n.d., Target 11.6).

Smart-Bin can contribute to achieve the target of Sustainable Development Goal 11. It is achieved by monitoring and control of waste state with help of sensors installed in smart-bins and collecting information to make timely decision to ensure waste management collection process. As part of the project, a dashboard and application are developed and send a notification to the operator regarding of the bins state (levels, weight, temperatures, humidity, air quality around bins).

This could be helpful when it comes to optimizing the collection process as priority could be given to those bins that contained more trash and should be emptied before the trash overflows. Moreover, it can also be employed to avoid the effects on the environment, e.g., when the temperature or air quality (sign of rot) is not acceptable. This can help to streamline the collection process by prioritizing bins and reducing vehicle and fuel usage.

1.3 Review of Related Literature

This review covers three kinds of sources. Patents show what has already been claimed, in any year. Studies from 2023 onward show what researchers have built lately, and a handful of commercial products show what is already on sale. The patents were found and read on Google Patents which also provided the legal status listed in Table 1. Every research study was followed up with its DOI or its publisher page and each element in the reference list was verified with the research. Each patent and each study has its own entry, and figure (where present) from the source that can be reused, and each one has a comparison with Smart-Bin. The synthesis of those comparisons is made in the synthesis, the SWOT analysis and the research gap at the end of the section.

Prior Art

Review of 20 patents. They are in effect through 2023 and their owners are in the United States, Europe, China and Korea. They are listed in priority year order in Table 1, and their coverage and differences from Smart-Bin are summarized. Google classifies these labels as assumptions and not legal conclusions, it's not until someone checks with the patent office whether their status is Active or Expired, that they should rely on it. Figure 2 puts the twenty into a time line. Most of the activity is recent, as five are priority years before 2012 and fifteen are priority years in 2012 or after.

Each of the patents is discussed individually below, in the order of the table. The numbers for each entry are the first images of the patent released as published on Google Patents, and the text below in each are what the patent claims and how it relates to Smart-Bin. The four closest patents are Kurani (2023), Sait et al. (2025), Shahabdeen (2016) and Yan and Chen (2021) and the synthesis in the end of this section makes a side-to-side comparison of the features of the two Smart-Bins.

Table 1

Related Patents, by Priority Year

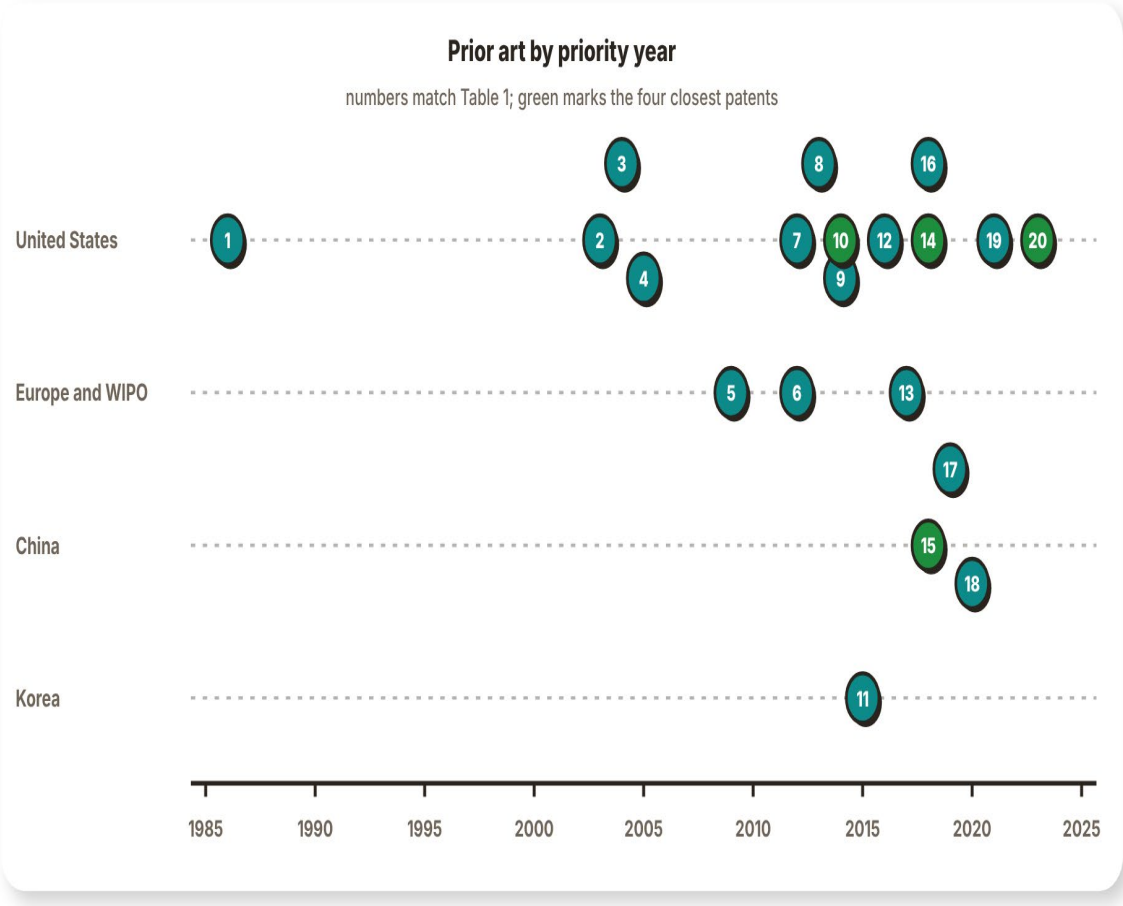
No.	Patent	Priority year	Status	What it covers	How Smart-Bin differs
1	US 5,016,197 (Neumann & Allison, 1991)	1986	Expired	Remote status units judge how full compactor containers are from hydraulic pressure or weight, and report over a telephone line to a central unit that calls the hauler.	A phone-line system for hydraulic compactors. It has no gas, tilt, lid, display, wireless link or phone app.
2	US 7,124,680 (Poss et al., 2006)	2003	Expired	A public trash compactor powered by a solar panel and a battery, with a removable bin, so it is emptied less often than an ordinary bin.	It compacts rather than senses. It has no weight, air-quality or tilt sensing, touchscreen, PIN mode or priority list.
3	US 7,145,450 (Brown, 2006)	2004	Expired	A processor reads the pressure on a compactor ram, flags the unit as full after a preset time and emails a work order to a service provider.	Fullness is inferred from pressure only. It has no fill level, weight, gas, lid, local display or app.
4	US 7,781,995 (Yang & Sandor, 2010)	2005	Expired	A trash can whose electric drive opens the lid when a sensor detects an object near the lower part of the can.	The trigger is a foot-level sensor. There is no fill, weight, air-quality, tilt, wireless or collection feature.
5	EP 2 516 974 B1 (Di Giugno et al., 2016)	2009	Not in force	A container that holds a bag has capacitive plates that give several fill levels. Its status goes wirelessly to stations that plan the emptying routes.	The sensing is capacitive around a bag. There is no weight, gas, tilt, powered lid, PIN mode or phone app.
6	WO 2014/079586 A1 (Kekalainen & Engstrom, 2014)	2012	Ceased	Containers with sensors report their status over a wireless network to a server, which plans the collection vehicles and can run bidding for the jobs.	It is a fleet-level planning platform with sensors only. There is no weight, tamper detection, powered lid, PIN mode or on-bin display.
7	US 9,347,821 B1 (Mullaney, 2016)	2012	Expired	The refuse container has a scale on it that indicates the weight of refuse in the container and alerts to a maximum weight.	It is capable to sense weight. Fill level, air quality, tamper alarm, IR sensor lid, PIN mode and dashboard are not present.
8	US 9,927,278 B2 (Kekalainen & Engstrom, 2018)	2013	Expired	A sensor device fixed under the container lid, behind a thermal barrier, measures the amount of waste and the conditions inside and sends them to a remote location.	Its novelty is the heat-insulated lid mounting of a cellular sensor module. There is no weight, tamper alarm, powered lid, PIN mode or priority list.
9	US 9,930,429 B2 (Kekäläinen, 2018)	2014	Active	A gauge compares the fill rate with limits set by a server and contacts the server only when a limit is crossed, which saves battery.	It is a battery-saving reporting scheme. No motorized lid, tamper alarm, PIN mode, touchscreen or phone app is described.
10	US 2016/0176630 A1 (Shahabdeen, 2016)	2014	Abandoned	A bin carries a lid-mounted range finder or a weight sensor at the base, with optional hydrogen sulfide, ammonia and methane sensing, and a controller that sends the data wirelessly.	It has no tamper detection, powered lid, lock, on-bin display, PIN mode or priority list. Temperature and humidity are not mentioned.
11	KR 10-1581944 B1 (Shin, 2016)	2015	Active	The hopper of a pay-by-weight food waste processor sits on a load cell and rotates to drop the waste into a crushing tank.	It is a weighing and payment machine. There is no fill sensing, gas sensing, tilt sensing, wireless reporting or priority list.
12	US 11,054,545 B2 (Devitt & Walsh, 2021)	2016	Active	Two low-power ways to detect a full container: an infrared beam across the container, or the resting tilt angle of the lid.	The output is only full or not full. There is no weight, gas, temperature, powered lid, PIN mode or tamper alarm.

No.	Patent	Priority year	Status	What it covers	How Smart-Bin differs
13	EP 3 620 405 A1 (Wang, Chen, et al., 2020)	2017	Withdrawn	An induction flip-lid can whose infrared sensing distance the user can set, built into a single microcomputer chip.	It only opens the lid. There is no fill, weight, gas, tilt, display or wireless link.
14	US 11,702,280 B2 (Kurani, 2023)	2018	Active	12 sensors, including those measuring fill level, weight, air quality, temperature, humidity and pressure, along with an accelerometer, are installed on the bin and communicate with a cloud server, which generates statistics and works out collection routes, also notifying mobile apps.	Focuses on litter, biohazard and pathogen sensing, with a sterilizer. There was no IR-sensor servo lid, no remote lock or no PIN collect mode or latched tamper alarm.
15	CN 109324552 B (Yan & Chen, 2021)	2018	Active	Each can has weight, infrared height, odor and smoke sensors and a positioning unit. A monitoring center decides which cans to visit and builds the truck route.	It plans city-wide truck routes with GPS. No powered lid, tamper detection, PIN mode, on-bin display or phone alert was found.
16	US 11,788,877 B2 (Balachandran & Talavera, 2023)	2018	Active	Load cells under a platform weigh the waste of two or more bins, and cameras with image analysis identify the contents for waste audit reports.	It is a waste audit and reporting service, not a single bin with fill, tilt, lid and lock control, a PIN collect mode and a collection priority list.
17	CN 210504152 U (Yang et al., 2020)	2019	Expired	A bin with an infrared induction module, an ultrasonic ranging module and a programming module lets users see how much capacity is left.	It is a stand-alone local device. There is no weight, gas, tilt, lid actuator, wireless link or phone app.
18	CN 111559590 A (Wang, Zhang, et al., 2020)	2020	Withdrawn	A controller for top or side installation measures the distance to the garbage and uploads it to a supervisory platform, which has an early-warning unit.	It comprises of one laser ToF sensor. No weight, gas, temperature, tilt, lid control or touchscreen collect mode or priority list.
19	US 12,125,361 B2 (Keller et al., 2024)	2021	Active	Two calibration setpoints enable a processor to convert distance readouts to a fill percentage and signal a user device when it exceeds a threshold, which varies depending on the can owner.	It uses a single laser time-of-flight sensor. There is no weight, gas, temperature, tilt, lid control, touchscreen collect mode or priority list.
20	US 12,450,575 B1 (Sait et al., 2025)	2023	Active	Each bin has methane, fill level and temperature/humidity sensors that provide data to neural networks which calculate the time required to reach the methane and fill limits; a scheduler creates routes considering the vehicle emissions.	It revolves around the sensing of methane by air quality. There is no load cell, tilt alarm, IR sensor lid, remote lock or PIN collect mode.

Note. Status is as reported by Google Patents in October 2026. The differences are those found in the patent text read for this review.

Figure 2

Patents Reviewed by Priority Year and Region

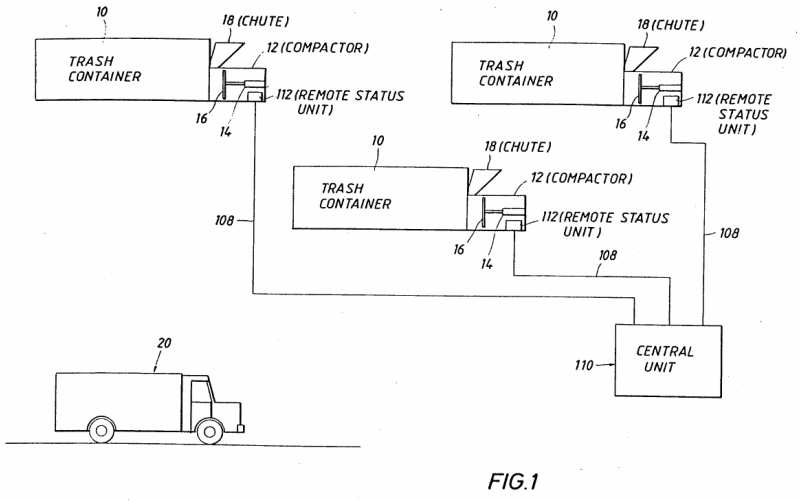


Note. Numbers match Table 1. The four patents in green are the closest to Smart-Bin.

Automated Trash Management System (Neumann & Allison, 1991)

Figure 3

Remote Status Units Linked to a Central Unit



Note. From *Automated trash management system* (U.S. Patent No. 5,016,197), by R. H. Neumann and C. B. Allison, 1991, U.S. Patent and Trademark Office.

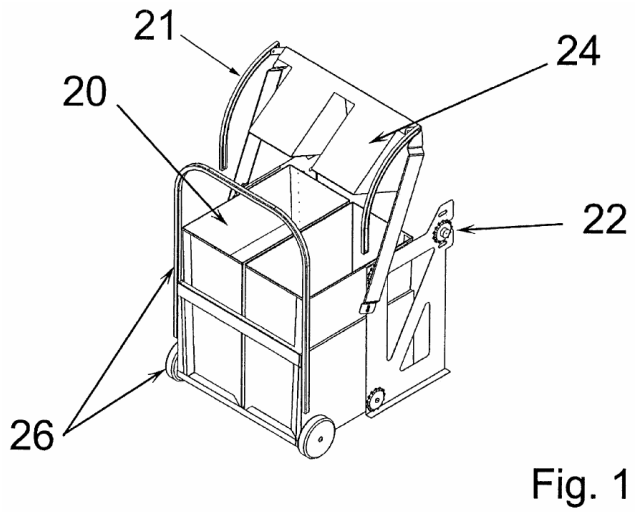
Figure 3 shows the layout of the system that Neumann and Allison (1991) patented for MGM Services. Each remote status unit serves three containers, each with its own compactor, and all units are linked by a communications line, typically a telephone line, to a central unit that keeps a database. The hauler sends a truck only to the containers that report full. For compactors, fullness is inferred from the hydraulic pressure during the packing stroke, from the work done per stroke or from the current drawn by the pump motor, and for any container a measured weight can be compared with a preset full weight.

Calling for a pickup only when a container is full is the idea behind the priority tags of Smart-Bin, and the load cell is a signal that both systems can use. The differences come from the era. The data travel by modem, a person calls the hauler, and no alert or route is generated. The patent describes no gas, temperature or tilt sensing, no lid, no display on the container and no phone app. Smart-Bin also measures the fill directly with an ultrasonic sensor and does not infer it from the work of a compactor.

Solar Powered Compaction Apparatus (Poss et al., 2006)

Figure 4

Compacting Ram and Chambers of the Solar Compactor



Note. From *Solar powered compaction apparatus* (U.S. Patent No. 7,124,680), by J. Poss, J. Satwicz, B. Richmond, and M. Taylor, 2006, U.S. Patent and Trademark Office.

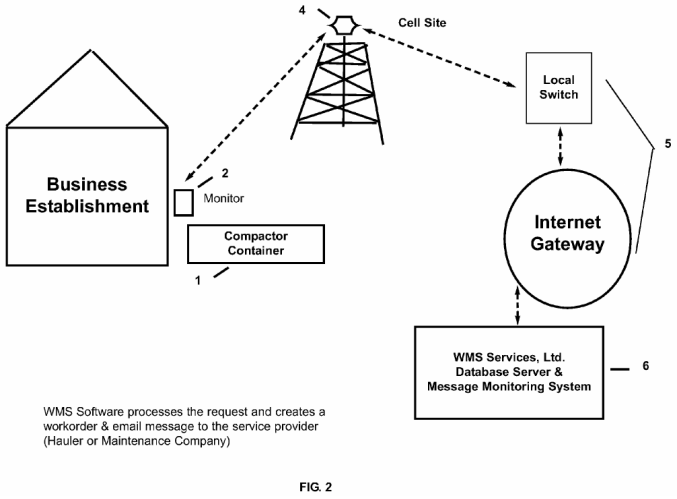
Poss et al. (2006) of BigBelly Solar patented a public compactor that a photovoltaic array powers, so that it can stand where there is no mains power but many people pass. Figure 4 shows the compacting ram with its chain drive and guide track and the compaction chambers. One of the roof panels charges a battery, which causes a compaction cycle to be skipped if the battery has low charge. The trash is compacted, which means that it will not need to be emptied as frequently as a regular trash bin. Fullness is sensed by a photo-eye, by the ram motor current (the stall time) and by pressure sensors, and a wireless transmitter reports to sanitation staff.

The patent attacks the same public-space problem as Smart-Bin, a bin that overflows and is costly to visit, but it answers with compaction and solar power instead of sensing. Scales are named only as an alternative fullness sensor, and the only display is a set of status lamps. No weight or air sensing, tilt alarm, touchscreen, PIN mode or phone app is described. One remark does bear on the project: the patent says that real-time fill data could feed routing and scheduling software, which is the use that Smart-Bin makes of its fill ranking.

Compactor Service and Monitoring System (Brown, 2006)

Figure 5

Path of a Full Message From the Compactor Monitor to the Service Provider



Note. From *Compactor service and monitoring system* (U.S. Patent No. 7,145,450), by W. J. Brown, 2006, U.S. Patent and Trademark Office.

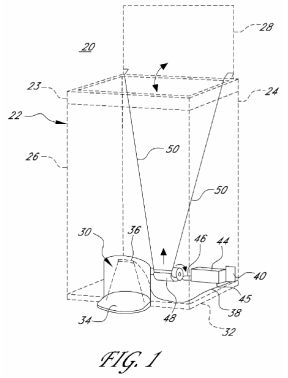
Brown (2006), an individual inventor, patented a monitor that is added to an existing compactor. A pressure sensor on the hydraulic system of the ram measures the pressure during each cycle, and when the pressure stays at or above a preset value for a preset time the unit is flagged as full. Figure 5, which is Figure 2 of the patent, shows the path of a message. The monitor at the compactor reaches a cell site, an internet gateway and a server that runs the message monitoring software, which creates a work order, emails it to the service provider (a hauler or maintenance company) and supplies a tracking web link.

Like Smart-Bin, the system detects a full container by itself and notifies the people who must empty it. Fullness is inferred from pressure, though, and the patent has no level or weight sensor, since the tonnage is typed in by hand after a pickup. Alerts go out as email work orders, with fax as an option, and not as phone notifications, and the only local indication is a set of LEDs. Overdue work orders are flagged for staff, which is a rough counterpart of the priority tags of Smart-Bin, except that Smart-Bin bases its tags on the measured fill rate.

Trash Can With Power Operated Lid (Yang & Sandor, 2010)

Figure 6

Trash Can With a Motor-Driven Lid



Note. From *Trash can with power operated lid* (U.S. Patent No. 7,781,995), by F. Yang and J. Sandor, 2010, U.S. Patent and Trademark Office.

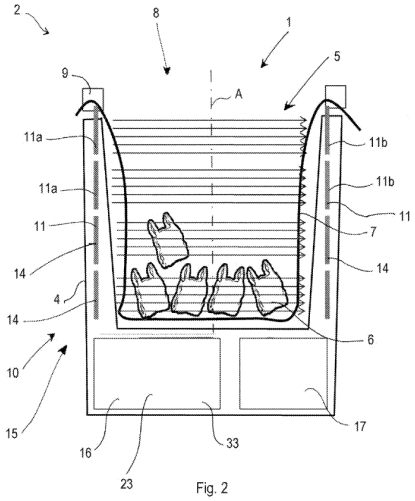
Yang and Sandor (2010) patented the power-operated lid of Simplehuman. Figure 6 shows the can with its lid open. There's a recess in front that houses the sensor, two link rods on the back wall connect a motor-operated lifting bar to the lid, and the circuit board, batteries and motor gear all reside in the base. If the sensor senses an object in the proximity of the bottom of the can, the electric drive opens the lid and the lid remains open for a predetermined period (2 seconds, for example). Claim 1 relates to the motor for the door. The controller on each command verifies the supply voltage, regulates the motor power to maintain the speed of the door within a certain range, stops the motor by reversing it for a short time, and if the corresponding motor is running for too long, the motor stops and a fault is output.

Smart-Bin also opens its lid by motor when a sensor notices a user, so that nobody has to touch the lid, but it uses an IR sensor and a servo and not a foot-level light beam. Everything else in the patent concerns lid mechanics. It has no fill, weight, air-quality, tilt, lock, display or wireless feature, and nothing on the collection side. The patent shows that a hands-free lid is old, so the contribution of Smart-Bin cannot be the lid itself. It lies in the sensing and reporting behind the lid.

Measuring Device and Monitoring System for Containers (Di Giugno et al., 2016)

Figure 7

Container With a Bag Between Capacitive Plates



Note. From *Measuring device, container means and system for monitoring and managing container means* (European Patent No. 2516974), by M. Di Giugno, F. Giovanelli, S. Tiraboschi, and P. Villa, 2016, European Patent Office.

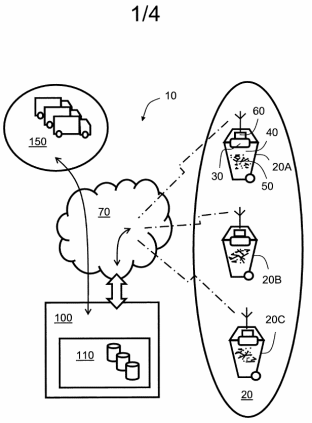
Di Giugno et al. (2016), from Cefriel and the Milanese environmental services company AMSA, describe a container whose casing holds a bag. Figure 7, which is Figure 2 of the patent, shows the container: the casing, the bag that the lid element holds in place and the plates at the sides of the cavity. Claim 1 places capacitive plates on opposite sides of the cavity, spaced along its length, so that the change in electrical capacity caused by the waste gives several progressive fill levels, with the bag lying between the plates. Status goes by GPRS or short-range wireless to an internet server, and stations plan the paths of the vehicles.

What matches Smart-Bin is the chain and not the sensor: measure the fill inside the bin, send it to a back end with a user interface, and use it to plan the collection of full bins. The sensing is different. Capacitive plates around a bag give levels in steps, while Smart-Bin measures distance by ultrasound and converts it to a percentage with two calibrated distances. It has no gas, temperature, tilt or weight sensing, no powered lid or lock, no PIN collect mode and no phone app.

Smart Waste Collection System and Method (Kekalainen & Engstrom, 2014)

Figure 8

Waste Collection System of Containers, Server and Vehicles



Note. From *Smart waste collection system and method* (International Patent Application Publication No. WO 2014/079586 A1), by F. Kekalainen and J. Engstrom, 2014, World Intellectual Property Organization.

Figure 8 shows the system in the application that Kekalainen and Engstrom (2014) filed for Enevo. The status is transmitted via a wireless network (GSM/3G) to a central server, which maintains the information in a database and arranges collection for vehicles on the way to collection or recycling facilities. The sensors are located in a rotating lid to measure fill level, gasses (methane, CO2 and CO type) and temperature. Weight is not measured. The server forecasts container fill times and routes them using a Monte Carlo simulation based on GPS data of the vehicles.The application also describes a marketplace in which haulers bid for collection jobs.

This is an early statement of the idea that fill data should decide which bins are collected first, and Smart-Bin carries it out on a much smaller scale with its priority tags. The patent works at fleet level, with bidding, simulated models and vehicle tracking, and the bin is only a sensor. It has no weight sensing, tilt or tamper alarm, powered lid, PIN mode or display on the bin, and humidity and odor are not measured.

Refuse Container With Weight Indicator and Danger Alerting (Mullaney, 2016)

Figure 9

Refuse Container with Scale, Display and Lights

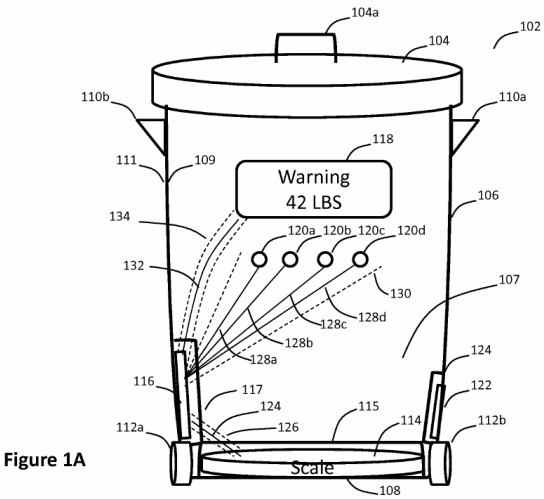


Figure 1A

Note. From *Refuse container with weight indicator and danger alerting* (U.S. Patent No. 9,347,821), by G. Mullaney, 2016, U.S. Patent and Trademark Office.

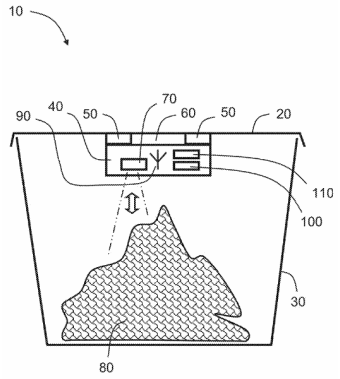
Mullaney (2016), an individual inventor, patented a refuse container that shows how heavy its contents are. Figure 9, which is Figure 1A of the patent, shows a container with a floor scale, a controller, a display and four lights. The lights track the percentage of the maximum weight, and all of them light, with one flashing, once the limit is exceeded. The maximum weight can be set by location (a city or ZIP code lookup), by the user or by default, and an optional beep is mentioned. Claim 1 adds a memory that stores a history of readings, a controller that sends that history to a service provider, and a lock that locks the lid once the refuse reaches a predetermined weight. An inversion sensor releases the lock for emptying.

The container weighs its contents, shows the weight and warnings on the bin, locks the lid under a condition and can report to a remote party, and Smart-Bin does all four. The lock is the point of difference. It closes at a weight limit to stop overloading, where the lock of Smart-Bin is controlled remotely or by a PIN. The patent does not measure fill level or air quality, has no tamper alarm, IR sensor lid or collection priority list, and uses the mobile app only for configuration.

Sensor Device for Smart Waste Collection Systems (Kekalainen & Engstrom, 2018)

Figure 10

Sensor Device Mounted Under the Lid Behind a Thermal Barrier



Note. From *Sensor device for smart waste collection systems and method* (U.S. Patent No. 9,927,278), by F. Kekalainen and J. Engstrom, 2018, U.S. Patent and Trademark Office.

Kekalainen and Engstrom (2018) patented the sensor module itself and not the system around it. Figure 10 shows a container with a pivoting lid and the sensor device under the lid, held off its surface by spacers that leave an air gap. The spacers and a heat-reflecting metal foil layer form a thermal barrier between the lid and the electronics. The module has an ultrasonic sensor for the amount of waste and sensors for temperature, humidity and a hydrocarbon or methane gas that indicates decomposition. The data is sent constantly, periodically or when a threshold is reached via a cellular modem (GSM or 3G), and the fasteners can be designed to avoid unauthorized removing.

This is the nearest match to the sensing side of Smart-Bin. Ultrasonic fill, temperature, humidity and gas are all on the bin, and the data go wirelessly to a remote location. The novelty lies in the insulated lid mounting and not in the sensors. The data support collection scheduling, but the device does not compute routes, and no weight sensor, powered lid, PIN mode or priority list is described. The anti-removal fasteners are a mechanical measure, where Smart-Bin senses the tilt and sounds an alarm.

Monitoring and Communicating the Fill Rate of a Container (Kekäläinen, 2018)

Figure 11

Gauge Mounted Under the Lid of a Container

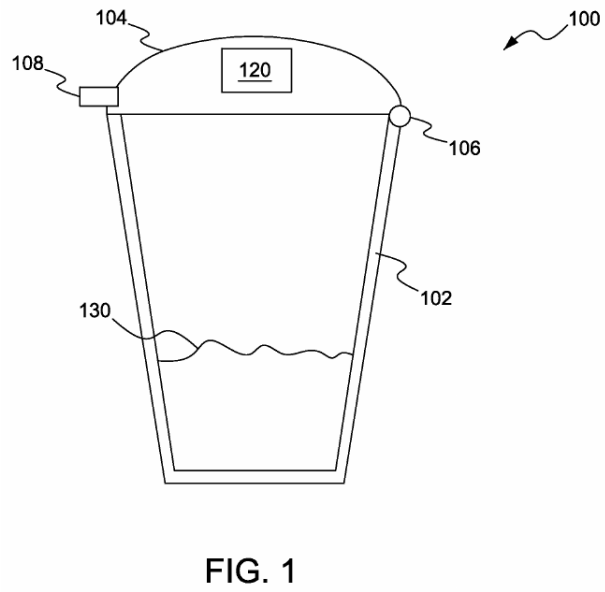


FIG. 1

Note. From *Method and system for monitoring and communicating fill rate of container* (U.S. Patent No. 9,930,429), by F. Kekäläinen, 2018, U.S. Patent and Trademark Office.

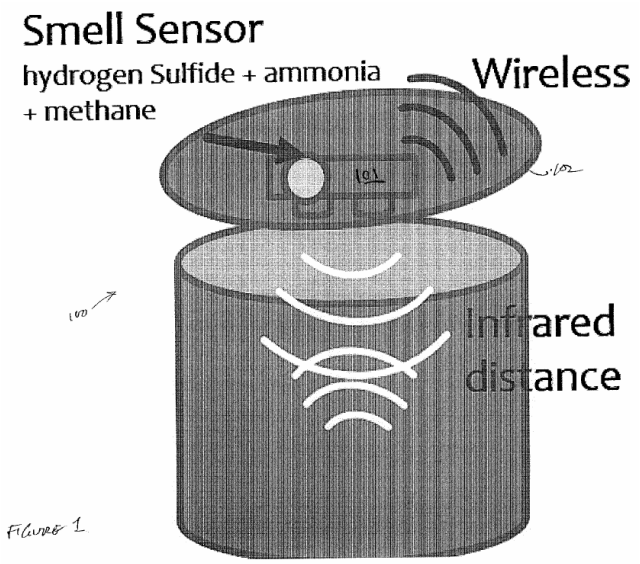
Kekäläinen (2018), also for Enevo, is about how often the gauge talks and not about what it measures. Figure 11 shows a garbage container with a hinged lid and the gauge attached under the lid. The model of how the gauge is supposed to function is sent to the gauge by a server that includes the maximum and minimum fill-rate limits and a communication schedule. The gauge maintains the microcontroller and transceiver in sleep mode, sensing the fill rate and waking only if the fill rate is at, or exceeds, a limit, transmitting the fill rate measurement and receiving an updated model. A lower number of transmissions equals a longer battery life.

Smart-Bin takes the opposite path over the same issue. It reports on a fixed schedule, with a write to ThingSpeak every 30s. A bin that is left outside for weeks could be threshold driven. The parts read the patent describes no motorized lid, tamper alarm, PIN mode, touchscreen or phone app. Its user device is a laptop or desktop.

Smart Garbage Bin (Shahabdeen, 2016)

Figure 12

Smart Garbage Bin with a Lid-Mounted Sensor Device



Note. From *Smart garbage bin* (U.S. Patent Application Publication No. 2016/0176630 A1), by J. A. Shahabdeen, 2016, U.S. Patent and Trademark Office.

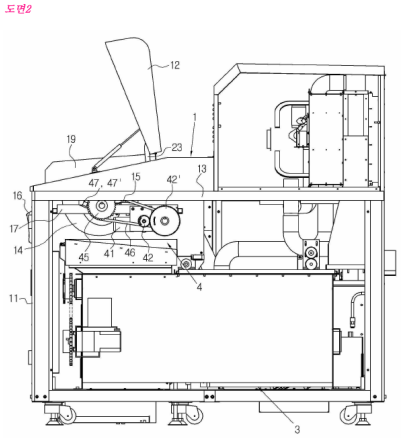
Shahabdeen (2016) filed this application for Zan Compute. Figure 12 shows the bin with a battery-powered wireless sensor device on its lid. The sensor is a reflective infrared range finder (a Sharp GP2Y0A41SK0F) aimed at the waste, and the description adds MQ-136, MQ-137 and MQ-4 gas sensors for hydrogen sulfide, ammonia and methane. A weight sensor at the base is the alternative way to measure how much has been deposited. A local controller sends the data wirelessly to a remote controller (MQTT in claim 17), and claim 13 sends an alert when a sensed value passes a threshold. The application says that an ultrasonic sensor was a possible alternative to the infrared range finder but judged it too power-hungry.

It's the closest to Smart-Bin in hardware: the distance to the trash from under the lid, the weight under it, gas sensing, and wireless reporting. It does not provide tamper detection, powered lid, lock, display, Pin mode, priority list and does not talk about temperature and humidity. The lid only carries the sensor. Google Patents lists the application as abandoned.

Input Device for a Pay-by-Weight Food Waste Processor (Shin, 2016)

Figure 13

Pay-by-Weight Food Waste Processor with a Load-Cell Hopper



Note. From *Inputting device for quantity and weight loss of food garbage container* (Korean Patent No. 10-1581944), by H. Shin, 2016, Korean Intellectual Property Office.

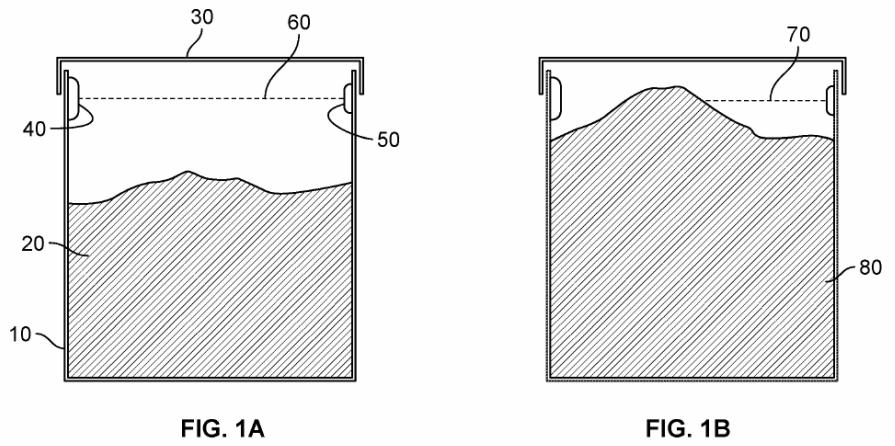
Shin (2016), for Concoftech, patented the waste input device of a pay-by-weight food waste processor. Figure 13 is a perspective view of part of the machine, with some parts omitted and separated. The waste is dropped into a hopper which sits on a load cell, the user pays electronically on the display, and the hopper then rotates approximately 170-180 degrees and releases the waste into a crushing and stirring tank. The guide member with rollers allows hopper to be turned within the space of the lid without snagging, maintaining the machine compactness and convenient loading height. Drawings show that payment data is sent to a primary server not shown in the drawing.

The load cell, a powered lid and a display for the user are shared with Smart-Bin. It is not for the same purpose. The Korean device crushes the garbage and charges according to its weight, while Smart-Bin weighs to verify the garbage level. There is no fill-level, gas, tilt or tamper sensing, no wireless or cloud reporting in the text read, no collection priority list and no mobile alerts in the patent. The reason for earlier deodorant sprayers and sterilizing lamps is that the odor is only in the background.

Techniques for Optimally Sensing Full Containers (Devitt & Walsh, 2021)

Figure 14

Infrared Beam Across a Container



Note. From *Techniques for optimally sensing full containers* (U.S. Patent No. 11,054,545), by S. Devitt and B. Walsh, 2021, U.S. Patent and Trademark Office.

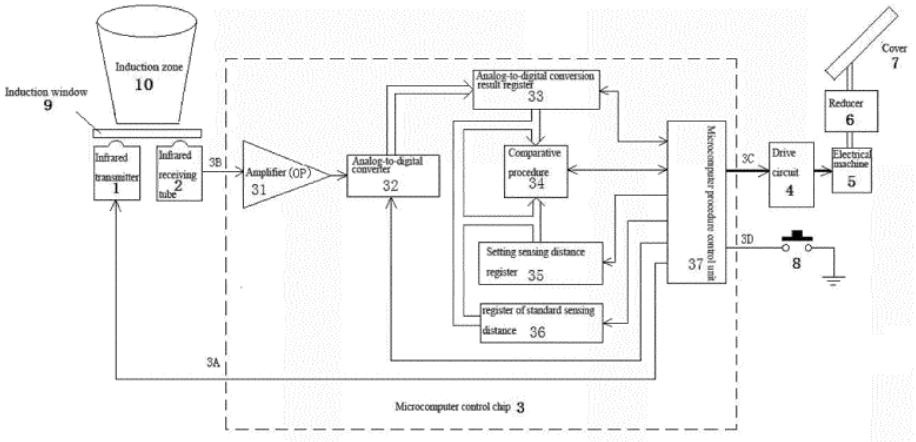
Devitt and Walsh (2021), for Oneplus Systems, give two ways to detect a full container while keeping very low the current drawn by battery-powered wireless circuitry. Figure 14, which combines Figures 1A and 1B of the patent, shows the first: an infrared emitter and receiver mounted on the side walls, so that the line between them marks the full level. The emitter transmits a wake-up signal and coded pulses, the receiver waits in sleep mode until the wake-up signal is received, and at the end of a specified period of time, if the coded signals are not received, the container is considered full. The second method checks the resting tilt angle of lid in three axes since a lid that overfull waste holds up is set at a different angle. The data from fullness is sent to a cloud server.

The patent only gives full or not full, with Smart-Bin providing a calibrated percentage and a time to full. No weight, gas, temp, powered lid, PIN mode/tamper alarm is displayed in parts read. The tilt here is the angle of the lid, used as a sensing element, and it is not a tamper check, so it should not be confused with the BMI160 tilt alarm of Smart-Bin, which watches the whole bin.

Trash Can With a Sensing Flip Lid (Wang, Chen, et al., 2020)

Figure 15

Circuit of the Induction Flip Lid with a Settable Sensing Distance



Note. From *Trash can having sensing flip lid having settable sensing distance and control method therefor* (European Patent No. 3 620 405 A1), 2020, European Patent Office, X. Wang et al. (sensing flip lid).

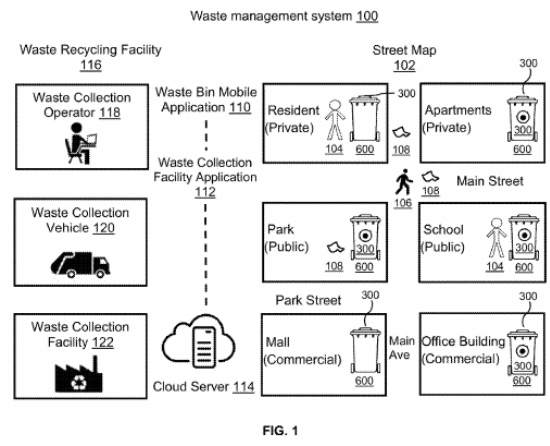
A clamshell trash can with an infrared sensing distance set by the user is described by Wang, Chen and Lin (2020) of Fujian Nashida Electronic. The circuit diagram is shown in figure 15. The major components are an infrared transmitter and receiver underneath an induction window, a single microcomputer chip for the amplifier, analog-to-digital converter and distance registers, and a drive circuit to operate the motor and reducer. Only when the reading is above the standard distance value, the motor will flip the cover open and when the user selects a distance by pressing a setting button, it will be stored and used for comparison later.

This is the closest equivalent of the servo-lid of Smart-Bin that is triggered by an IR sensor. A hand near the induction window opens the lid and how near is up to the user. As no fill or weight, gas, tilt, display or wireless link or monitoring is involved in the project, nothing else is addressed in the patent. It was listed as a published application (kind code A1) on Google Patents and withdrawn in 2021, meaning that it was not granted.

Smart Waste Bin Sensor Device and Methods (Kurani, 2023)

Figure 16

Overall Smart Waste Management System



Note. From *Smart waste bin sensor device and methods for waste management system* (U.S. Patent No. 11,702,280), by H. B. Kurani, 2023, U.S. Patent and Trademark Office.

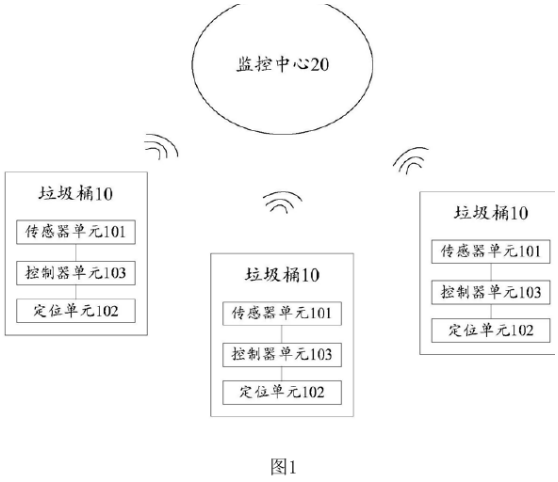
Kurani (2023) is the broadest patent of the twenty on sensing. Figure 16 gives the whole system for monitoring bins continuously to optimize collection and reduce littering. The bin is equipped with a battery powered smart sensor device carrying approximately a dozen sensors, including fill level, temperature, humidity/pressure, air quality, smoke, gas, motion, a pathogen biosensor and sterilizer, and an accelerometer. Readings are transmitted via a wireless network to a cloud server which creates real-time bin statistics and optimizes collection routes. The data is displayed in a bin app and a facility app, and biohazard and biosafety alerts are sent to nearby phones. The accelerometer detects when the bin has been tilted, and resets the fill level to zero when the bin tips over.

It matches Smart-Bin on sensing, cloud, apps and routes: fill, gas, temperature and tilt data go to a cloud server with mobile alerts and an optional touchscreen on the device. Its centre of gravity lies elsewhere, in litter, biohazard and pathogen sensing with a sterilizer. The parts read show no IR-actuated servo lid, remote lock, PIN collect mode or tamper alarm with a latch, and the tilt serves to reset the fill level and does not sound an alarm.

Smart City Garbage Management System (Yan & Chen, 2021)

Figure 17

Garbage Cans and the Monitoring Center



Note. From *Smart city garbage management system* (Chinese Patent No. CN 109324552 B), by S. Yan and J. Chen, 2021, China National Intellectual Property Administration. The labels in the original are in Chinese.

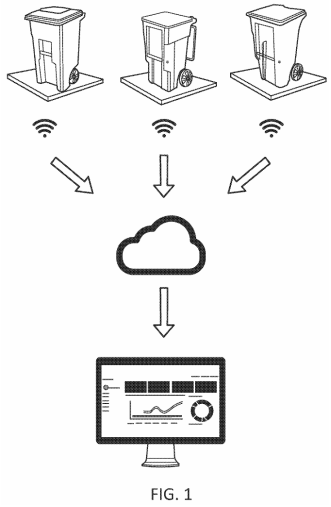
Yan and Chen (2021), for Shanghai Shuncom Smart Technology, patented a city-wide system. Figure 17 is the schematic of the garbage cans and the monitoring center. Each can has a positioning unit, a controller and sensors for weight, infrared height, odor and smoke, and the controller uploads the readings to the monitoring center. The center decides which cans need inspection (weight, height or odor above a threshold, or any smoke), generates the route of the truck. When a special-condition signal arrives, a nearby truck is rescheduled by priority, in the order fire, fault, pollution and full bin.

This patent is the closest in purpose. Weight, height and odor decide which cans are serviced first, as the priority tags of Smart-Bin do. It goes further by building routes for trucks with GPS, and Smart-Bin stops at ranking the bins. It also includes smoke sensing, which Smart-Bin does not have. No powered lid, tamper detection, PIN mode, on-bin display or phone alert appears in the text read.

Apparatus and Method for Waste Monitoring and Analysis (Balachandran & Talavera, 2023)

Figure 18

Waste Measurement System with Weighing Devices and Reports



Note. From *Apparatus and method for waste monitoring and analysis* (U.S. Patent No. 11,788,877), by N. Balachandran and O. Talavera, 2023, U.S. Patent and Trademark Office.

Balachandran and Talavera (2023), for Zabble, cover waste audits. Figure 18 gives an overview of the measurement system: a series of weighing devices record the weight of the waste, the data go to networked storage, and the stored data are turned into visual reports for an operator. Claim 1, though, is about images: a method that acquires images of the contents of at least two containers of a waste generator, identifies characteristics of the contents and generates a report by computer. The disclosure adds load cells under a platform for two or more bins and gas sensors for compost bins.

The shared ground with Smart-Bin is remote monitoring of bin contents, with cloud upload and dashboards, and gas sensing in one special case. The aim is different. This patent is an audit and reporting service that tells a waste generator what is being thrown away, while Smart-Bin is a single bin that works out when it needs emptying. The claims do not cover a bin with fill, tilt, lid and lock control, a PIN collect mode or a collection priority list.

Capacity Inspection Garbage Bin With Infrared Induction (Yang et al., 2020)

Figure 19

Block Diagram of the Capacity-Detecting Bin

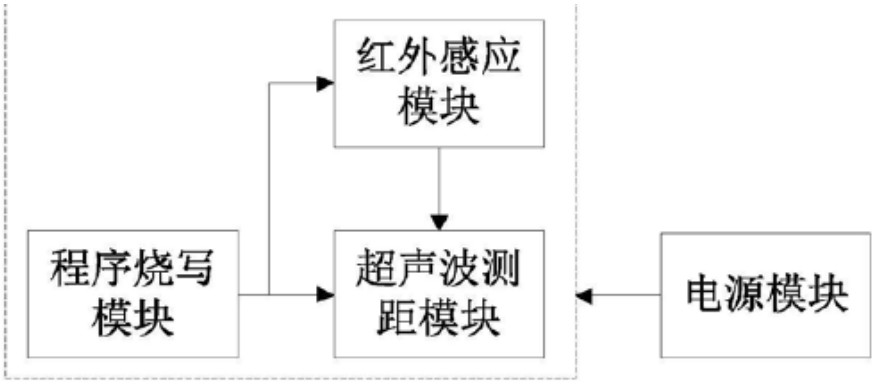


图1

Note. From *Capacity automated inspection garbage bin with infrared induction function* (Chinese Patent No. CN 210504152 U), by S. Yang, D. Liao, R. Li, and Y. Yu, 2020, China National Intellectual Property Administration. The labels in the original are in Chinese.

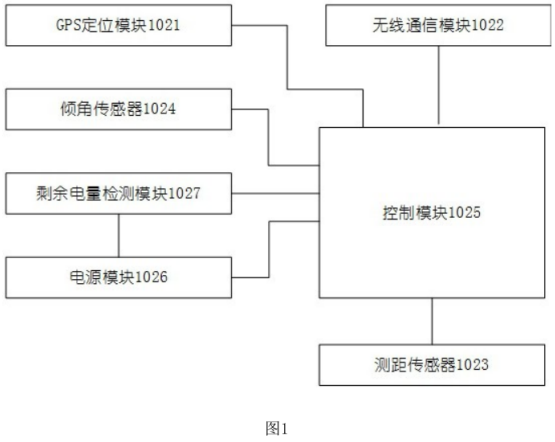
Yang et al. (2020), for Sichuan Grid Environmental Protection Technology, is a utility model for a bin that shows how much capacity it has left. Figure 19 is a block diagram of the inside of the bin. The body holds an infrared induction module, an ultrasonic ranging module, a programming module and a power module. The ultrasonic module is connected to the infrared and programming modules, and the power module supplies all three. The ultrasonic module measures the remaining capacity and two nixie-tube digits show it on the bin, while the infrared module detects a nearby user to wake the system. Google Patents lists the rights as terminated for non-payment of fees.

This utility model matches the local side of Smart-Bin: an ultrasonic ranging module, a wake-up by a nearby user, and the capacity shown on the bin itself, where Smart-Bin shows it as a percentage on its touchscreen. Little else carries over. The device is stand-alone, with no weight, gas, tilt, lid actuator, wireless link, server, route planning or phone app.

Garbage Bin Intelligent Controller and Supervisory System (Wang, Zhang, et al., 2020)

Figure 20

Electrical Block Diagram of the Bin Intelligent Controller



Note. From *Garbage bin intelligent control ware and garbage bin supervisory systems* (Chinese Patent No. CN111559590A), by X. Wang, Y. Zhang, and Z. Wang, 2020, China National Intellectual Property Administration. The labels in the original are in Chinese.

Wang, Zhang, and Wang (2020), for Weihai Jingxun Changtong Electronic Technology, published an application for a bin controller that can be installed on top of the bin or at its side. Figure 20 is its electrical block diagram, with a wireless communication module, a control module and a distance sensor in one housing. In top mode the controller works out the height of the garbage, in side mode it reports the distance to the opposite wall, and either value is uploaded to monitor whether the bin is full. In the description, an ultrasonic module measures the distance, an inclination sensor raises a tip-over or movement alarm, the data go by NB-IoT and alerts reach the phones of workers.

The overlap with Smart-Bin is strong on the alert side: a distance sensor for fill, an inclination sensor for tip-over alarms, wireless upload and alerts on a phone. The patent has no weight or gas sensing, powered lid, lock, PIN or touchscreen collect mode, priority list or time to full, and in the parts read the inclination alarm has no countdown or latch, which Smart-Bin adds. Google Patents lists the application as withdrawn.

Measuring the Fill Level of a Trash Can Using a Sensor (Keller et al., 2024)

Figure 21

Automatic Alerting System for a Trash Can

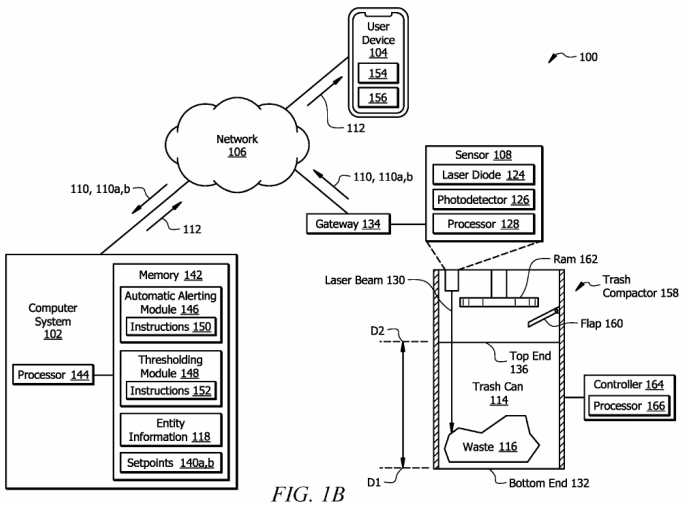


FIG. 1B

Note. From *System and method for measuring the fill level of a trash can using a sensor* (U.S. Patent No. 12,125,361), by J. R. Keller, M. O. Redmond, and S. A. Mirza, 2024, U.S. Patent and Trademark Office.

Keller et al. (2024), for 7-Eleven, patented a way to turn a distance reading into a fill percentage. Figure 21 shows the automatic alerting system: a sensor above the container, a gateway and network, and a computer system with its modules and a user device. In claim 1 a sensor above the can holds a laser diode that sends a pulse toward the bottom and measures the time of flight. A computer converts the distance to a percentage of waste with two calibration setpoints, picks a threshold for the current time period from stored information about the owner of the can, and alerts a user device when the threshold is exceeded. Alerts can be email or text on a smartphone.

The two setpoint calibration is identical to that of Smart-Bin: the empty distance (default: 30cm) and the full distance (default: 5cm) are programmed from the interface, and the fill percentage is programmed to the distance between. A threshold-based alert reaches a phone in both. Everything else in the patent is narrower: a single laser sensor, no weight, gas, tilt or tamper sensing, no lid control or lock, no touchscreen collect mode and no priority list.

Waste Bin Collection and Monitoring Method Using Bin Sensors (Sait et al., 2025)

Figure 22

Network of Waste Bins, Cloud Server and Vehicles

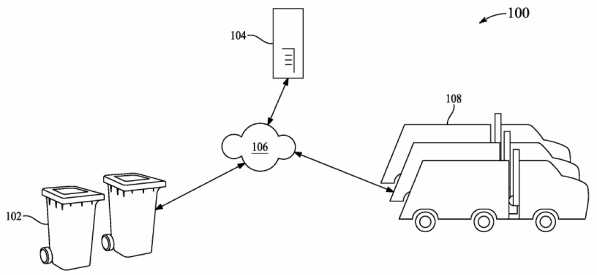


FIG. 1

Note. From *Waste bin collection and monitoring method using waste bin sensors* (U.S. Patent No. 12,450,575), by S. Sait, F. M. Aliyu, S. Bukar, B. S. Maina, and D. U. Lawal, 2025, U.S. Patent and Trademark Office.

Sait et al. (2025), from King Fahd University of Petroleum and Minerals, is the newest patent in the set. Figure 22 is a network diagram of the waste bins, the cloud server, the cloud network that connects them and the waste vehicles. Claim 1 describes a method to monitor and control the CO2 and CH4 emissions of a collection system. The methane, humidity, waste level, location and temperature are sensed under every separate bin lid and transmitted to a cloud server. One processor calculates the time in which the methane limit will be reached and a second processor calculates the time in which the fill limit will be reached, and a third processor, based on the CO2 and CH4 readings of the vehicles, prepares the collection schedule and routes. Two neural networks make the time estimates, and a mobile app sets parameters and shows data.

It is the nearest to Smart-Bin in using a time to full, in sensing fill, gas, temperature and humidity inside the bin, and in sending them to a cloud server with a display and a mobile app. The estimate is made differently. Neural networks work from methane and fill data, while Smart-Bin uses the fill rate of the last 15 minutes with no trained model. The patent is centered on methane prediction and routing based on emissions. It has no load cell, tilt or tamper alarm, IR sensor lid, remote lock or PIN collect mode, and weight and tilt appear only in a list of possible sensors.

Related Literature and Studies

Thirty studies were reviewed between 2023 to 2026. Each of them is listed in Table 2 with its hardware, the main reported result and what it is lacking in comparison with Smart-Bin. If a figure is derived from training data or is a synthetic data set, this is indicated in the table as these figures are not representative of the actual system in use. Each study is then taken in turn under one of four headings, according to what it puts the most weight on: level-sensing bins with lids and alerts, bins that add weight, gas or access control, collection routes and priority, and cameras, sorting and system design. A figure is shown wherever the source is open access and has a drawing or photograph that helps, with the licence of the figure in the note under it. The other studies are text only, because their publishers do not allow the figures to be reused.

Table 2

Related Studies From 2023 to 2026

Study	Hardware and platform	Reported result	What it lacks
Saad et al. (2023)	NodeMCU ESP8266 and Arduino Nano, HC-SR04, HX711 load cell, GPS, SG90 lid, WhatsApp through Twilio	Lid opens within 50cm. Full at 3cm of free space or 4,000g. Height error 0.4%, weight error 0.15%, message in 2.61s.	Dashboard, air quality, tamper detection, cloud history, roles
Abdullahi et al. (2024)	Arduino with HC-SR04 and SIM800L GSM, servo lid, cloud server	Lid opens only when the bin is not full. Full at 20cm. No performance figures are given.	Weight, air quality, tamper detection, dashboard, app
Alnuaimi et al. (2024)	Simulation of smart-bin collection as a travelling salesman problem (AnyLogic)	Shorter travel distance and lower emissions than static routes. No figures are given.	Bin hardware, alerts, app
Ayyanagoudar et al. (2024)	Ultrasonic and weight sensors, web page for each bin level	Real-time level tracking for each bin on a web page. No measured results are given.	Air quality, tamper detection, app, alerts
Azmi et al. (2024)	Arduino Uno with LoRa, HC-SR04, DHT11, GPS, ThingSpeak dashboard	Ultrasonic accuracy 89.9% at half full, humidity 96.9%, temperature 94.4%. Alerts are dashboard indicators only.	Weight, app, push alerts, on-bin display
Enache et al. (2024)	ESP32-CAM nodes with solar cell, ESP-MESH, server image processing	Fill ratio estimated from hourly bin photographs. Relative error 11.43% to 30.00% on five bins.	Weight, air quality, tamper detection, app
González et al. (2024)	ESP32 and ESP32-CAM, ultrasonic fill, simulated weight, Oracle APEX dashboard, App Inventor app	Camera classifies waste; weight is simulated with potentiometers. No accuracy figures.	Real weight sensing, air quality, tamper detection, priority list
Kuzhin et al. (2024)	IoT sensors with route analytics; no hardware described	The results are 20% fewer overfilled bins, 15% fewer collection trips, 25% shorter trips, and 10% less fuel.	Any bin hardware; setting and sample are not given
Mehta and Rathour (2024)	Ultrasonic, temperature, weight and tilt sensors and data software.	6-month pilot: approximately 50% improvement in collection efficiency, 25% reduction in cost, and 83.3% reduction in overflows.	Detail on site and method; air quality, app, on-bin display
Muthukumar et al. (2024)	Camera-based AI segregation bin with a staff app for fill level and temperature and humidity	Classifier test accuracy 92.3% (training accuracy 71.8%). The scalability claim is not tested.	Weight, air quality, tamper detection, collection priority; main aim is segregation
Rathnayake et al. (2024)	Three ESP32 bins with ultrasonic and methane sensors, Node-RED cloud, machine learning	4-month pilot: fill accuracy 95%, methane detection 92%. Route simulation 15% more path-efficient.	Weight, tilt, app, on-bin display

Study	Hardware and platform	Reported result	What it lacks
Salleh et al. (2024)	Using NodeMCU ESP8266, ultrasonic sensor and ThingSpeak dashboards.	Live container levels support need-based collection scheduling. Proteus simulation and PCB layout are included.	Weight, air quality, tamper detection, app
Shaharil and Po'ad (2024)	Arduino, ultrasonic lid sensor, IR fill sensor, load cell, MQ-2, LCD, Blynk	Weight errors of −5g to +10g on four items. The Blynk app shows a full-bin notification. No numbers for the IR or gas sensors.	Web dashboard, collection priority, tamper detection
Thangam et al. (2024)	Use of ESP8266, ultrasonic and soil-moisture sensors	Analytics are meant to let authorities adjust routes. No results are reported.	Weight, air quality, tamper detection, app
Aasish et al. (2025)	Dual-core ESP32 on FreeRTOS, ultrasonic, DHT sensor, RFID bin tags, LCD or OLED, cloud server	Splits sensing and cloud communication across tasks. No measured results.	Weight, tamper detection, app, test data
Alnanih et al. (2025)	Arduino Uno and ESP8266 with GSM, HC-SR04, flame sensor, GPS, LCD, Firebase, web app, Flutter app	6-month, 50-bin campus pilot: 99.5% uptime, fill error within ±3%. Smoke Test fire alarm in 5s.	Weight, humidity, tamper detection, time to full, UI calibration
Alourani et al. (2025)	Arduino Nano and ESP8266, ultrasonic sensor, Firebase, Android app, VGG-19 classifier	App alerts collectors at a fill threshold. Classifier 99.7% training accuracy.	Weight, air quality, tamper detection, priority list
Banciu et al. (2025)	GPS and RFID bin locations; Nearest Neighbor, OR-Tools and genetic algorithm	Three real routes with 735 locations: up to 52% shorter distance, over 14kg less CO_2.	Any bin sensing; the study is about routing
Gajula et al. (2025)	ESP32, HC-SR04, Moisture sensor, Servos and Arduino IoT Cloud.	Separates dry and wet waste by moisture. No figures are given.	Fill-level role of the ultrasonic sensor, weight, tamper detection, app
Gupta et al. (2025)	Microcontroller with ultrasonic sensor and Wi-Fi, cloud platform	30-day, 50-bin pilot: 58.9% less distance, 59.2% less fuel, 95% fewer overflows, 99.8% uptime.	Weight, air quality, tamper detection, app detail
Ishaq et al. (2025)	Ultrasonic sensors in bins at ten hospitals, cloud monitoring station	Alerts led to pickups 3 to 16 minutes later. Biomedical waste about 43.6% to 52.5% of hospital waste.	Weight, air quality, tamper detection, app (proposed only)
Karthi et al. (2025)	ESP32, RFID reader, ultrasonic sensor, metal detector, MQ135, Blynk	95% fill accuracy, 92% RFID reliability, 40% less manual intervention.	Weight, tamper detection, priority list, web dashboard
Kumar et al. (2025)	Using ESP32, ultrasonic sensors, Ant Colony Optimization, machine learning, and map.	Prioritizes filled bins and builds depot-to-depot routes. No numbers are reported.	Weight, air quality, tamper detection, app, alerts
Maulida et al. (2025)	ESP32, HC-SR04, MQ-2, HX711, SG90 lid, Firebase, Android app	Level accuracy about 95%, weight about 93%, 92% of spoiled-food cases flagged, usability score 86 (15 officers).	Tamper detection, collection priority, web dashboard, roles
Mehta et al. (2025)	ESP8266 with ultrasonic sensor, MQ4, MQ136, MQ137 and GPS, Firebase and React Native Application	Gas sensors 96% to 98% accurate against a reference. Field test of 28-bin samples cut trucks from 3 to 2. The 100-bin results are synthetic.	Weight, tilt, on-bin display; some tables conflict
Sadikin et al. (2025)	ESP32 with ultrasonic, infrared and GPS, Firebase, Kotlin Android app	Ranking by distance, vehicle capacity and fill level (greedy). 69 respondents: 92.37% effectiveness, 89.92% satisfaction.	Weight, air quality, tamper detection, on-bin display
Shahibollah et al. (2025)	ESP32, ultrasonic sensors, servo lid, tri-colour LED, Blynk	Tested by simulation and on hardware. No figures are given.	Weight, air quality, tamper detection, priority list
Yogesh et al. (2025)	ESP32, HX711 load cell, ultrasonic sensor, Firebase, App Inventor app, LCD	Weight and fill percentage with full-bin alerts. No measured results are reported.	Air quality, tamper detection, priority list, test data
Awotayo and Dodo (2026)	ESP32, two HC-SR04, MG90S servo lid, LEDs and buzzer, Blynk email	Fill zones at 40% and 60%; email when full. No accuracy or response-time results.	Weight, air quality, tamper detection, dashboard, app
Mugaruka et al. (2026)	LoRaWAN-style network of ultrasonic sensors in seven 120-L bins	12-week pilot: 57% of sensors at or below −107dBm; mass and route times are estimates.	Weight, air quality, tamper detection, app

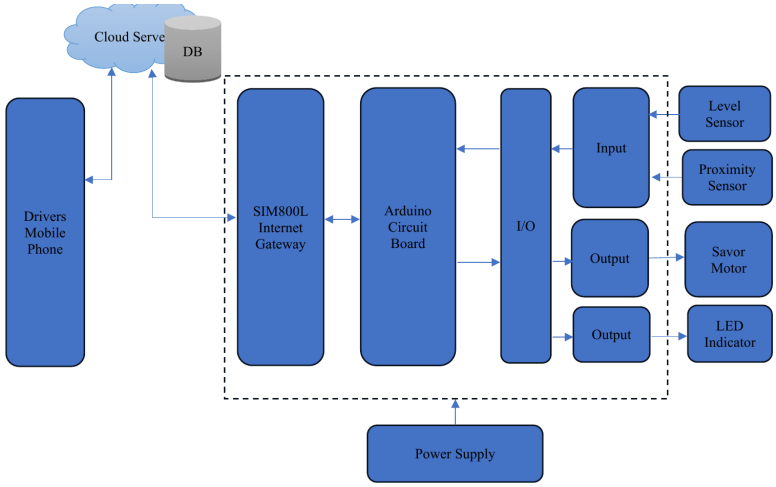
Note. Results are as reported by the authors. Lacks means that the feature is not described in the study.

A. Level-Sensing Bins with Lids and Alerts

Smart Bin with Automatic Lid Control (Abdullahi et al., 2024)

Figure 23

Block Diagram of the Smart Waste Management System



Note. From "Development of a smart waste management system with automatic bin lid control for smart city environment," by A. Abdullahi, A. Mohammed, M. U. Bonet, A. El-Suleiman, R. B. Ahmad, and T. D. Chollom, 2024, *EAI Endorsed Transactions on Smart Cities, 7*(3). CC BY-NC-SA 4.0.

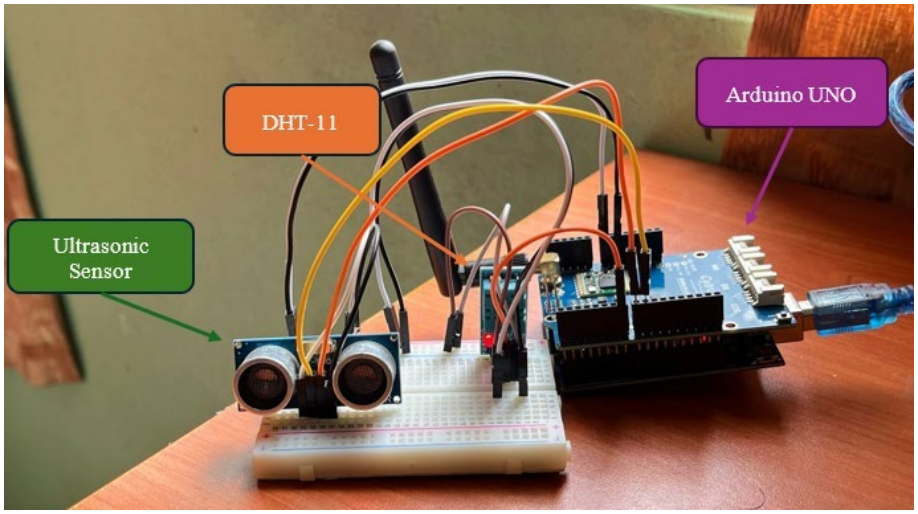
Abdullahi et al. (2024), of the Air Force Institute of Technology in Kaduna, Nigeria, built a bin whose lid opens only when the bin is not full. Figure 23 is their block diagram. The fill level is measured by an ultrasonic sensor (HC-SR04) and an Arduino, which also detects the approaching user, and the lid is opened by a servo motor that rotates 0-90 degrees. The thresholds are 20cm for a full bin, and 12cm for a user. Status connects to the cloud server via a SIM800L GSM module where data is stored and full bin notifications are sent to the collection team.

There are no figures for performance and it is inconsistent regarding the board as one figure shows Uno and the other shows Nano. There's a phone block in the diagram, but the app is never mentioned. Smart-Bin includes the ultrasonic fill sensor, a servo lid and weight, air sensing, tilt, a dashboard, and a phone app, all of which this study lacks.

Smart Waste Management System Using LoRa (Azmi et al., 2024)

Figure 24

Circuit Setup of the LoRa Bin Unit



Note. From "Smart waste management system using LoRa technology," by A. Z. Azmi, N. M. Sahar, and M. T. Islam, 2024, *E3S Web of Conferences, 599*, Article 04007. CC BY 4.0.

Azmi et al. (2024) installed a LoRa connection between the bin and the receiver. The setup for their circuit is shown in Figure 24. The ultrasonic fill distance, DHT11 temperature and humidity and GPS position are transmitted through a bin-side Arduino Uno with LoRa shield to a second Arduino Uno with LoRa receiver, and then relayed to a NodeMCU which updates a ThingSpeak dashboard with indicators when thresholds are reached. The tests indicated an ultrasonic accuracy of 89.9% at half full, a humidity accuracy of approximately 96.9% and a temperature accuracy of approximately 94.4%.

The authors report that the ultrasonic sensor was not successful in a narrow, shallow bin, and the DHT11 temperature sensor error exceeded the sensor's specified error. They state that they have a LoRa range of several kilometers, but they did not measure it, and the abstract mentions air quality monitoring, but does not go into any detail about it in the body of the paper. Also, Smart-Bin senses temperature and humidity (BME680 used instead of DHT11) and uses ThingSpeak. The study has no weight sensing, push alerts, phone app or display on the bin.

Ultrasonic Trash Can Surveillance with ThingSpeak Dashboards (Salleh et al., 2024)

Salleh et al. (2024) developed a prototype for waste collection based on demand at a low cost. An ultrasonic distance sensor mounted in each container is read by a NodeMCU ESP8266 module which uploads the fill level to ThingSpeak so that live dashboards indicate the status of each container and enable container status-based scheduling. Also included is a Proteus simulation and a PCB layout. No results are reported, and the authors view this work as a scheduling tool, rather than a validated route planning method.

Smart-Bin also employ ThingSpeak for charts and analysis, but only as a second cloud besides Firebase, as optional. It provides the weight, air quality, tamper detection and a phone app, things this study doesn't.

ESP8266 Waste Monitor With Moisture Sensing (Thangam et al., 2024)

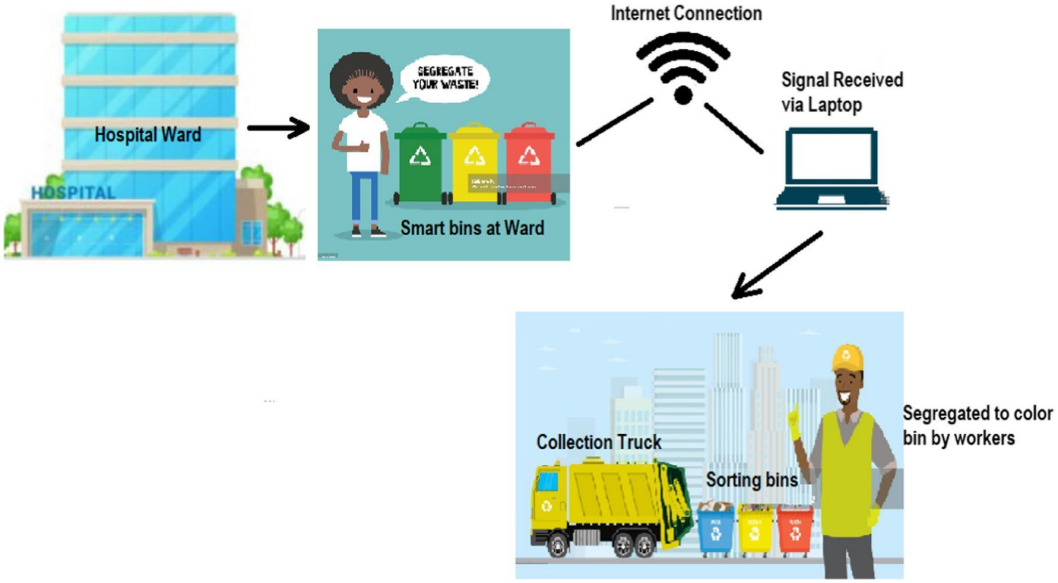
Thangam et al. (2024) of Amrita Vishwa Vidyapeetham, Bangalore, have developed an ESP8266 monitor that utilizes ultrasonic fill sensors, soil-moisture sensors to assess the waste composition. The real-time data is supposed to enable the authorities to modify routes and reduce trips and gas. The route claim is a stated aim but not a measured result, as no quantitative results are reported in the abstract.

Smart-Bin senses are filled by the same type of ultrasonic sensor with the addition of weight, air quality, tamper detection and a phone app. It makes no attempt to judge the composition of the waste.

Smart Bins for Hospital Waste in Kaduna (Ishaq et al., 2025)

Figure 25

Proposed System Architecture for Hospital Waste Monitoring



Note. From "Smart waste bin monitoring using IoT for sustainable biomedical waste management," by A. Ishaq, S. J. Mohammad, A. D. Bello, S. A. Wada, A. Adebayo, and Z. T. Jagun, 2025, *Environmental Science and Pollution Research, 32*(32), 19434–19449. CC BY 4.0.

Ishaq et al. (2025) put ultrasonic fill-level sensors in bins at ten hospitals in Kaduna, Nigeria. Figure 25 is the proposed system architecture, from the hospital ward to the smart bins, an internet link, a laptop at the monitoring station and the collection truck. The bins connect wirelessly through the routers of the hospitals to a central cloud monitoring station, which alerts waste collectors when a bin is full. The authors also measured waste generation. Biomedical waste made up about 43.6% to 52.5% of hospital waste, at an average of 69.14kg per day and 1.52kg per bed per day, and they analysed the data with principal component analysis.

Alerts led to pickups 3 to 16 minutes later. The phone app appears only in the proposed system and is not described as built, and no microcontroller is named for the bins of the authors. The study shows what level sensing is worth in a place where overflow carries a health risk. Smart-Bin senses more than level, but it has not been tested in any such site.

IoT-Enabled Smart Dustbin for Real-Time Monitoring (Shahibollah et al., 2025)

Shahibollah et al. (2025), from Universiti Teknologi MARA in Terengganu, built an ESP32 dustbin with ultrasonic sensors for fill level and for proximity, a servo-driven lid and a tri-colour LED for local status. Data go over Wi-Fi to the Blynk platform, which alerts the maintenance staff. The authors tested the bin by simulation and with hardware, but the abstract gives no ranges or accuracy figures and describes the result as prototype function and not as municipal-scale reliability.

Smart-Bin uses the same ESP32 family and a servo lid. Its touchscreen takes the place of the LED, and it adds weight, air quality, tamper detection and a priority list.

IoT Smart Bin for Real-Time Fill Level Monitoring (Awotayo & Dodo, 2026)

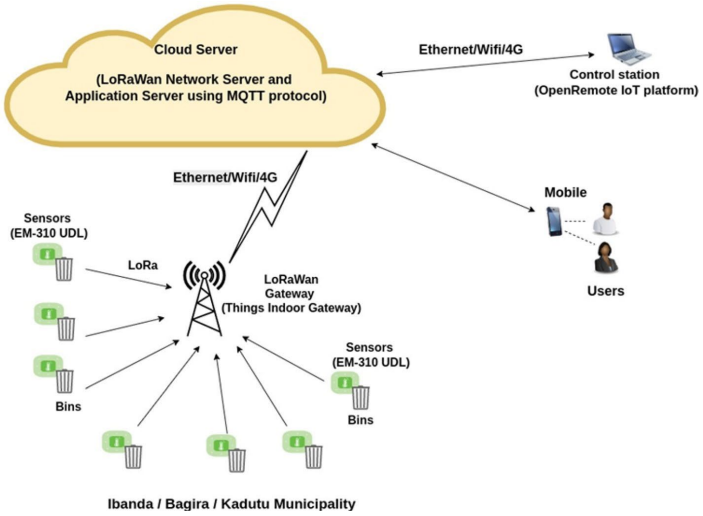
Awotayo and Dodo (2026) of Baze University, Abuja designed and implemented an ESP32-based bin equipped with two HC-SR04 sensors. One of these is facing outwards and will open a servo lid (MG90S 0 to 90 degrees) when someone is in range of 30cm. The other indicates the level of fill, with a green LED when the fill level is between 0 and 39%, a yellow LED when the fill level is between 40 and 59%, and a red LED and buzzer when the fill level is between 60 and 100%. An e-mail is sent by Blynk when the bin is full. Two 3.7V Li-ion cells (7.4V, 3500mAh) power the unit. There are no accuracy or response-time results reported.

The local feedback is LEDs and a buzzer and not a screen. Smart-Bin also includes a touch screen, weight detection, air quality, tamper detection, dashboard and a phone app, none of which were identified in this study.

LoRaWAN Network for Waste Collection in Bukavu (Mugaruka et al., 2026)

Figure 26

Proposed Network Architecture for Waste Collection in Bukavu



Note. From "Design of an IoT sensor-based network architecture for waste collection in Bukavu city (DRC)," by O. M. Mugaruka, J. R. Kala, J. Ndikumagenge, R. van Schadewijk, and R. Nibitanga, 2026, *Discover Internet of Things, 6*(1), Article 168. CC BY-NC-ND 4.0.

Mugaruka et al. (2026) designed a LoRaWAN-style network for waste collection in Bukavu, in the Democratic Republic of the Congo. Figure 26 shows the proposed network. EM310-UDL ultrasonic sensors on the bins talk over LoRa to a gateway, the gateway reaches a cloud server by Ethernet, Wi-Fi or 4G, and users view the data on a control station and on mobile phones. The authors tested it in a 12-week pilot with seven 120-litre prototype bins in residential, dense and peri-urban areas.

The results concern the network more than the bins. Of the sensors, 57% had a signal at or below −107dBm, and the mass collected (an estimated 641kg over 14 days in one area) and the route times are estimates from sensor data and not direct measurements. The authors also report an energy autonomy above 90% for most units after 11 months, a period that the paper does not clearly tie to the 12-week pilot. Smart-Bin does not use LoRa at present, and the study is a reminder of the opportunity listed in the SWOT analysis: LoRa for places without Wi-Fi.

B. Bins That Add Weight, Gas or Access Control

IoT-Based Smart Dustbin Prototype (Saad et al., 2023)

Figure 27

Prototype of the Smart Dustbin

Note. From "IoT-based smart dustbin prototype," by A. M. Saad, B. O. Jul, A. Basalamah, and S. Sayuti, 2023, *PROtek: Jurnal Ilmiah Teknik Elektro, 10*(2), 120–125. CC BY-NC-SA 4.0.

Saad et al. (2023) built the smart dustbin shown in Figure 27. An Arduino Nano with an ultrasonic sensor opens an SG90 servo lid to anyone who is within 50cm, while a NodeMCU ESP8266 reads the waste height with an HC-SR04, the weight with an HX711 load cell and the position with a NEO-6M GPS. Once the bin is full (either by level (3cm free space) or weight (4,000g), the NodeMCU will send a WhatsApp message via Twilio with a Google Maps link.

The tests reported a height error of 0.4%, a weight error of 0.15% and a message delay of 2.61s. This is the nearest study to the pairing of fill and weight with a servo lid in Smart-Bin. Its lid is triggered by an ultrasonic sensor, where Smart-Bin uses an IR obstacle sensor. The study has no dashboard, air quality sensing, tamper detection, cloud history or roles.

Smart Bin With Real-Time Garbage Level Tracking (Ayyanagoudar et al., 2024)

Ayyanagoudar et al. (2024) propose a smart bin that uses weight sensors and ultrasonic transducers to track the fill level in real time and shows the level of each bin on a web page. The

abstract also states that it can classify trash based on its weight, which is used for planning transportation. The abstract does not mention any microcontroller or cloud platform and does not provide any results of measurements.

Smart-Bin also correlates weight to an ultrasonic level. It includes air sensing, tamper detection and a mobile phone app, which are not included in this study.

Efficiency of IoT-Enabled Smart Bins (Mehta & Rathour, 2024)

Mehta and Rathour (2024) present the description of IoT smart bins, which are equipped with ultrasonic, temperature, weight and tilt sensors and software to collect and process the data. They claim a 6-month pilot in one city where collection efficiency has improved by approximately 50%, operating costs have decreased by approximately 25% and there have been approximately 83.3% fewer overflow incidents, while also offering reduced emissions through route optimization and increased satisfaction of citizens. Abstract makes no mention of the location or nature of these measures or of the number of samples collected, these figures are merely the authors' claims.

This is the only study to mention a tilt sensor but does not indicate a tamper alarm. Smart-Bin does exactly that, a 10 second (latched) countdown followed by a siren that latches.

Dual-Sensing Bins for Fill Level and Methane (Rathnayake et al., 2024)

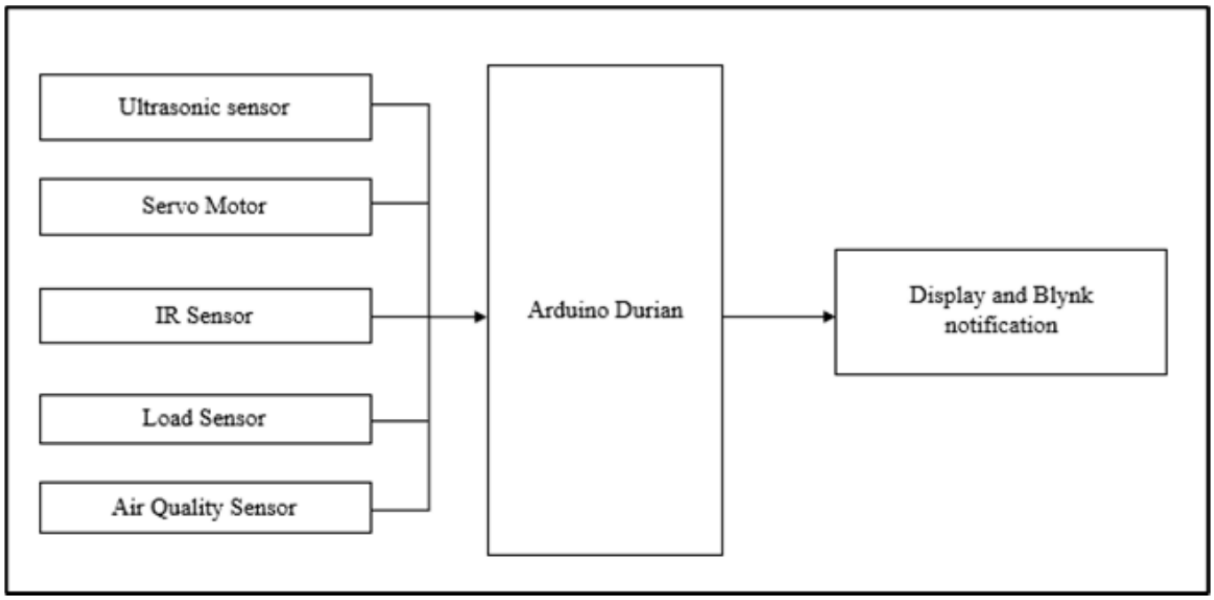
Rathnayake et al. (2024) developed the dual sensing bins which combined the ultrasonic fill sensor with methane sensor for detection of spoilage. The ESP32 modules transmit the data via Wi-Fi to a cloud platform where the data is processed by the Node-RED software and ML algorithms predict the waste and optimize routes. Three bins were deployed in a four-month pilot in Dehiwala, Sri Lanka and reported a 95% fill-level accuracy; methane-detection accuracy was 92% and simulated routes were 15% more path-efficient than fixed routes. There is no definition of the accuracies in the abstract and 15% is obtained from a simulation.

The ESP32 and the ultrasonic level are shared between Smart-Bin. It is not a methane reading, but rather an air-quality estimate as it is a gas channel. This study doesn't include any weight sensing, tilt alarm, phone app or on bin display.

Arduino-Based Smart Dustbin (Shaharil & Po'ad, 2024)

Figure 28

Block Diagram of the Arduino-Based Smart Dustbin



Note. From "Arduino-based smart dustbin with ultrasonic sensor for efficient waste management," by M. H. F. Shaharil and F. A. Po'ad, 2024, *Evolution in Electrical and Electronic Engineering, 5*(2), 92–97. CC BY-NC-SA 4.0.

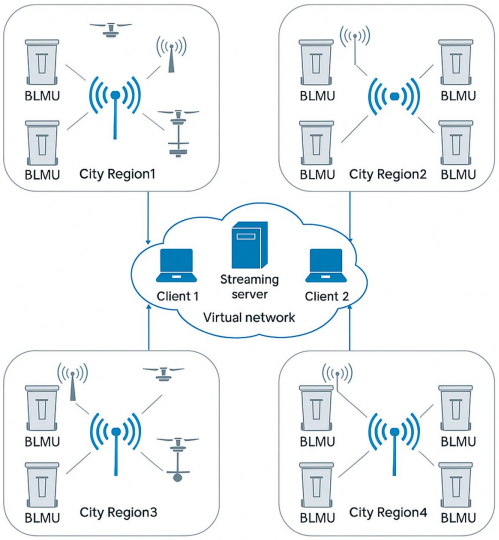
Shaharil and Po'ad (2024) built a household-scale bin and show its block diagram in Figure 28. The ultrasonic sensor opens up the servo lid if a person is within 30cm, the infrared sensor detects the level of fullness of the bin, the weight is displayed on the LCD using a load cell, and finally, a buzzer alerts when air quality is poor with the help of an MQ-2 sensor. The Blynk app shows a notification when the bin is full.

The testing is small: 15 distance trials and four weighed items with errors from −5g to +10g, and no numbers for the infrared or gas sensors. The bin is the closest study to the local side of Smart-Bin, with a lid, weight, gas and a display on the bin. It has no web dashboard, no collection priority and no tamper detection.

IoT-Driven System with Fire Detection and Role-Based Apps (Alnanih et al., 2025)

Figure 29

Data Communication Structure of the Smart Waste Management System



Note. From "Advancing sustainability through an IoT-driven smart waste management system with software engineering integration," by R. Alnanih, L. Elrefaei, and A. Al-Ahwal, 2025, *Sustainability, 17*(21), Article 9803. CC BY 4.0.

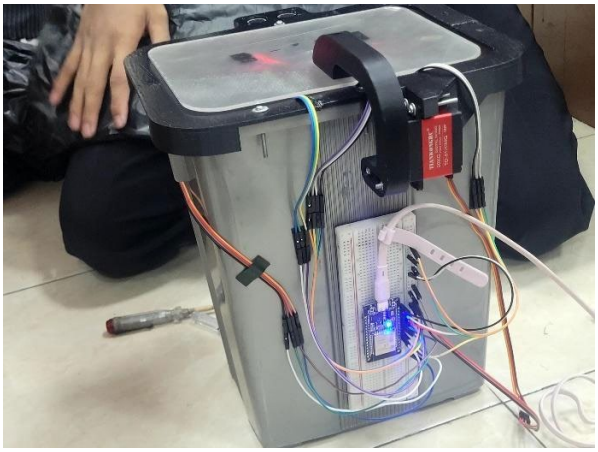
Alnanih et al. (2025) combine fill and fire sensing in one bin unit. Figure 29 shows their data communication structure. An HC-SR04 is used to measure the fill level, a flame sensor detects fire, smoke and gas and an Arduino Uno with an ESP8266 and GSM receives the sensor data and an LCD, buzzer are added. Firebase and Firestore are used to launch alerts, coordinating collection routes from a central command center. A desktop web application is used by administrators and a Flutter Android application for workers based on their roles.

6-months pilot on campus with 50 bins achieved 99.5% uptime with fill error within ±3%, and a fire alert within 5s (smoke test). This is one of the longest field pilots of the studies which constructed their own bins. It's similar to Smart-Bin, it has Firebase, a web app, a phone app and different views for different users. There is no weight, tilt or lid feature and nothing is calibrated from the interface in the hardware list.

ESP32 Smart Trash Bin with Firebase (Maulida et al., 2025)

Figure 30

Prototype of the SmartTrashBin



Note. From "Design of IoT-based smart trash bin monitoring using ESP32 and Firebase," by A. Maulida, C. S. Fayzalmi, F. A. C. Anaam, D. Ardiansyah, M. A. Gunawan, and N. Ridwan, 2025, *Jurnal Komputer, Teknologi Informasi dan Sistem Informasi (JUKTISI), 4*(2), 420–428. CC BY 4.0.

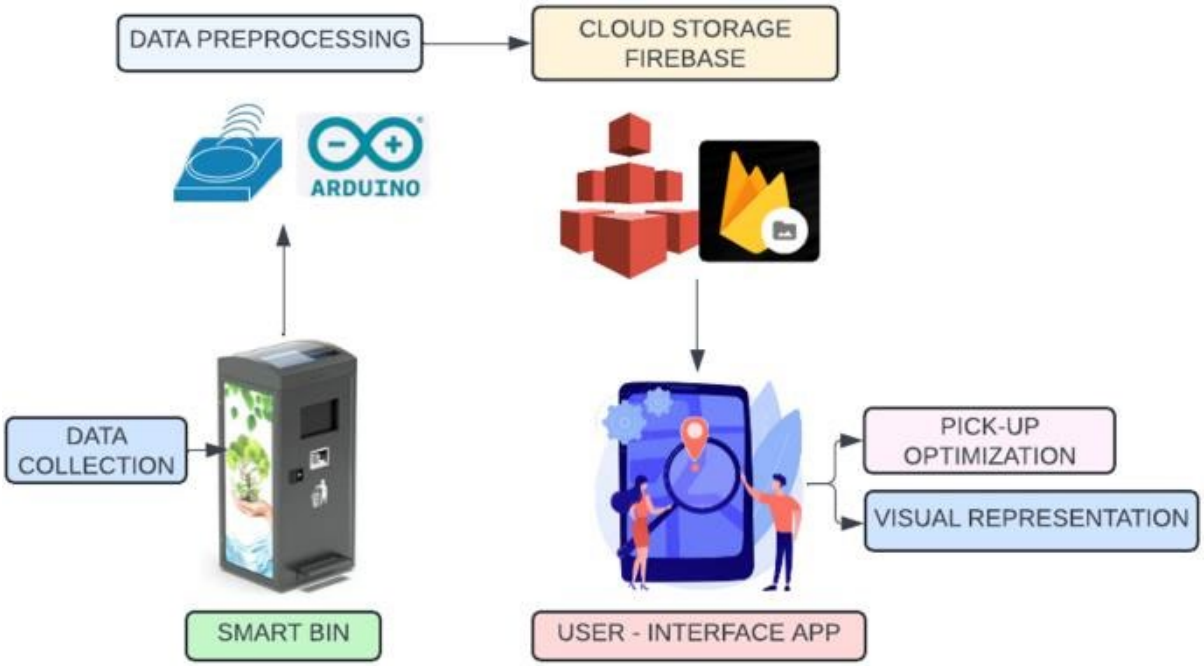
The SmartTrashBin in Figure 30 was created by Maulida et al. (2025) from Politeknik Negeri Semarang. The hand detection & fill level is read using HC-SR04, while MQ-2 is used to detect the odour and the load cell (HX711) detects the weight that is then fed to ESP32, which controls the opening of the lid using a servo (SG90). Data is stored in Firebase Realtime Database and sent to an Android application that includes notifications.

They claim a level accuracy of approximately 95%, a weight accuracy of approximately 93% (+/- 320g on 20kg), 92% of cases of spoiled food identified and 98% of notification reliability and a System Usability Scale score of 86 from 15 sanitation officers. The numbers require carefulness as the paper mentions both of Android Studio and Flutter and also provides two notification thresholds. The closest of the studies to Smart-Bin is the following: ESP32, ultrasonic sensor, load cell, servo lid, firebase and an android app. No tamper detection, collection priority or web dashboard/roles.

Real-Time Sensing and Methane Mapping (Mehta et al., 2025)

Figure 31

Data Flow of the Garbage Monitoring System



Note. From "Smart bins, smarter cities: IoT-driven waste collection with real-time sensing and methane mapping," by H. Mehta, V. Deriya, D. Sankhe, P. Kanani, G. Pandya, and V. Shelke, 2025, *Mathematical Modelling of Engineering Problems, 12*(6), 2110–2122. CC BY 4.0.

Mehta et al. (2025) show the data flow of their system in Figure 31. Each bin has the ESP8266 equipped with an ultrasonic fill sensor, MQ4, MQ136 and MQ137 gas sensors, GPS and uploads data to Firebase every 10s. A React Native application displays a bin map, a methane and odor heatmap and truck routes that are ranked based on a priority score derived from fill and gas.

Gas sensors accuracy was 96% - 98% with reference. There are a number of trucks that are different – field samples of 28 bins in Malad v. 100 bins in the results (which use synthetic data), and some tables in the paper clash. The study is like Smart-Bin, which is based on Firebase, a mobile application and implements role-based views and priority-driven collection. No weight sensing and no tilt sensors and no display on the bin.

ESP32 Smart Dustbin with Weight and Level Monitoring (Yogesh et al., 2025)

Yogesh et al. (2025) created an ESP32 dustbin that includes an HX711 load cell and an ultrasonic sensor, which is called “Eco Bin”. Weights and fill percentage are sent to Firebase Realtime Database and a phone app, designed with MIT App Inventor, for full-bin alerts, and a 16×2 I2C LCD displays the alerts on the bin. However, since the paper doesn't report any actual results, it is here as a hardware precedent, not as a performance metric.

It's almost the same as in Smart-Bin, which is ESP32, load cell, ultrasonic sensor, Firebase, Android app and local display. Smart-Bin brings in air quality, tamper detection, a priority list and a web dashboard.

RFID Access Control and Blynk Integration (Karthi et al., 2025)

Karthi et al. (2025) developed ESP32 dustbin with servo lid which will open only for the authorized RFID user. A fill level sensor (ultrasonic sensor), a metal detector, and an air-quality sensor (MQ135) are used to perform measurement checks, with the data sent to the Blynk platform. Abstractly, the authors state 95% fill level accuracy, 92% RFID authentication reliability, 40% less manual work and 35% more efficient collection.

It is the only one study, which can open the bin, that is found. On the collecting side, however, Smart-Bin provides control via a PIN collect mode and remote lock, plus weight, tamper detection and a priority list. The web dashboard is not available for this study.

C. Collection Routes and Priority

Smart-Bin Collection as a Travelling Salesman Problem (Alnuaimi et al., 2024)

The paper by Alnuaimi et al. (2024) from University of Sharjah, UAE, considers the problem of municipal waste collection as a travelling salesman problem (TSP). The fill level of the bins is reported and the route is planned using the Nearest Neighbor heuristic within an AnyLogic simulation. The abstract claims that there are savings in travel distance and carbon emissions when compared to traditional static routes, but it does not provide figures and there is no description of hardware. The work is not a field deployment, but rather a simulation.

The results of this study is where the Smart-Bin ends. It is a list of bins sorted by fill and sorted by time to fill; and a planner such as this one might use that list as input.

Waste Management Optimization Through IoT and Analytics (Kuzhin et al., 2024)

According to Kuzhin et al. (2024), a case study of IoT sensor real-time data and analytics for route optimization has resulted in a 20% reduction in overfilled bins, a 15% reduction in collection frequency, a 25% reduction in the distance traveled, a 10% reduction in fuel consumption, and a 15% increase in recyclables collected. The abstract does not identify a location, sample size, hardware, time frame, and fails to specify whether the values are from a field experiment; the evidence is weak.

The study shows the size of the gain that people expect from collecting by priority. It gives no design that Smart-Bin could borrow.

Route Optimization With GPS and RFID Bin Locations (Banciu et al., 2025)

Banciu et al. (2025), from Lucian Blaga University of Sibiu, collect bin locations with GPS and RFID, build distance matrices and compare the Nearest Neighbor heuristic, Google OR-Tools and a genetic algorithm on three real routes with 735 locations. The genetic algorithm did best, with up to 52% shorter travel distance and more than 14kg less CO_2. RFID here only records the location of a bin when it is emptied, and the study senses no fill level.

It sits entirely on the routing side. Smart-Bin supplies the part that this study takes for granted, which is knowing which bins are worth a visit now.

IoT E-Tracking System for Waste Management (Gupta et al., 2025)

Gupta et al. (2025) propose an IoT system in which bins carry an ultrasonic fill sensor, a microcontroller (Arduino-based, going by the figure captions) and a Wi-Fi module that sends data to an unnamed cloud platform for on-demand routes. They say that in a 30 day trial with 50 bins they achieved 58.9% less distance, 59.2% less fuel, 95% less overflows and 99.8% uptime..

These are some of the highest figures in the entire review, and no pilot dates are mentioned, and no method of routing nor protocol for accuracy tests, so as a result this is what the authors report, but not an independent result. Only level sensing. Smart-Bin builds upon the same level reading with weight, air, tilt and user interface.

Ant Colony Route Optimization With Machine Learning (Kumar et al., 2025)

In Kumar et al. (2025), a system is designed where ultrasonic sensors are mounted on ESP32 boards to send fill level data to a central database. On one hand, Ant Colony Optimization is used to plan routes from the depot to the depot, only stopping at the bins that are filled; on the other hand, machine learning is used to forecast waste from past data; and the collection teams are provided with a Folium map as an interface. The abstract does not provide any actual figures, so there is no actual saving in fuel or cost that can be quoted from the abstract.

Smart-Bin also has the ESP32, the ultrasonic fill sensing and the concept of only going to bins that need it. It sets that priority from the fill and the time to full and not from a learned model. Weight, air quality and tamper detection and phone app not included in this study.

TrashTrack Collection Ranking (Sadikin et al., 2025)

Telkom University (Bandung) Researcher, Sadikin et al., (2025) developed an Android application using Kotlin and Jetpack Compose called TrashTrack. ESP32 bins are equipped with ultrasonic, infrared and GPS sensors and upload the collected data to a Firebase database; a modified constraint-based greedy algorithm is used to rank the collection based on the distance of

the bin, vehicle capacity and fill level. The results of the usability test with 69 respondents showed that 92.37% were effective, 92.29% were useful and 89.92% were satisfied.

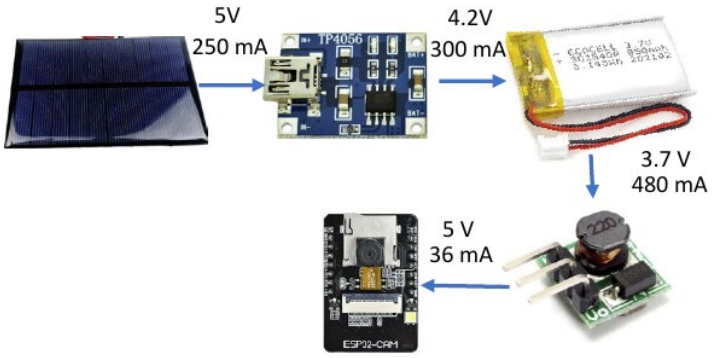
Smart-Bin is powered by the same ESP32, Firebase and a kotlin android app. It ranks by fill and time to full (no dist or vehicle capacity – it doesn't plan roads). Weight, Air Quality, Tamper Detection and Display on bin are not included in this study.

D. Sorting and System Design

Camera-Based Fill Level Detection (Enache et al., 2024)

Figure 32

Power Chain of the ESP32-CAM Node



Note. From "Intelligent trash level detection for sustainable urban waste management," by B.-A. Enache, C.-K. Banica, and A. G. Bogdan, 2024, *The Scientific Bulletin of Electrical Engineering Faculty, 24*(1), 56–59. CC BY-NC-ND 4.0.

Enache et al. (2024), from the University Politehnica of Bucharest, estimate the fill level from photographs. Figure 32 shows the power chain of one node. A 5V solar cell feeds a TP4056 charger, which charges a 3.7V LiPo battery of 850mAh, and a step-up converter brings the voltage back to 5V for the ESP32-CAM. The battery was sized for 24 hours of operation. The fill ratio is estimated by a Python script with OpenCV and an SQL database, each node takes one image per hour and sends it over an ESP-MESH network to a server.

On five test bins the relative error of the estimated fill ratio ran from 11.43% to 30.00%, and the authors name the angle of the camera, the light, the kind of waste and the placement of the

bag as causes. The figure matters to Smart-Bin because it shows what a stand-alone node needs between the source and the board: a charger, a battery and a converter. Smart-Bin builds the same chain with its battery shields and 18650 cells. It is not camera based, it is based on distance and weight and this study has no weight, air quality, tamper detection, or phone app.

AI-Based Waste Segregation With Monitoring (Muthukumar et al., 2024)

At M. Kumarasamy College of Engineering in Karur, Muthukumar et al. (2024) developed a bin fitted with a camera that can identify plastic, glass, metal and paper using AI and place them in separate compartments. A mobile application displays the level of fill, plus the temperature and humidity levels, and notifies cleaning personnel and allows for one team to use the application to claim each task. The number of images used is about 1500 images per class and the paper reports 71.8% training accuracy and 92.3% test accuracy, but does not explain the difference between these. The microcontroller and the cloud service are not named.

Segregation is outside the scope of Smart-Bin. The claim-a-task feature of the app echoes the collector role in Smart-Bin, and the study has no weight, air quality, tamper detection or collection priority.

Smart Trash Bins With IoT and AI (González et al., 2024)

González et al. (2024) constructed a bin made of ESP32 and ESP32-CAM. This includes ultrasonic sensors that measure fill level, an AI that detects waste from images from the camera, and potentiometers that replace the weight sensor. Events are captured by readings, which are transmitted over Wi-Fi to a central server and displayed on Oracle APEX dashboards; and an MIT App Inventor app is used to manually record the status. The abstract does not contain any accuracy figures.

The weight is simulated and the study indicates a dashboard design without any actual weighing. Smart-Bin offers a real load cell, and always offers a simulation as an option for testing, all set from the interface. This study has no air quality sensing, tamper detection or priority list.

Smart Waste Classification with IoT and Deep Learning (Alourani et al., 2025)

Alourani et al. (2025) propose iSSWMs, a framework that joins an IoT smart bin to a deep-learning classifier. The bin has an ultrasonic level sensor on an Arduino Nano and an ESP8266 that sends its status to Firebase, and an Android app alerts the collectors at a threshold. A VGG-19 classifier sorts plastic, glass and metal and reached 99.7% accuracy on about 1,530 TrashNet images over 100 epochs. That figure is training accuracy, and the authors report no field test.

Smart-Bin shares the ultrasonic level, Firebase and an Android app with alerts for collectors. Image classification is outside its scope, and this study has no weight, air quality, tamper detection or priority list.

ESP32 Smart Bin That Separates Dry and Wet Waste (Gajula et al., 2025)

Gajula et al. (2025) developed an ESP32 dust bin which detects the wetness of the waste deposited and separates dry and wet waste with the help of SG90 servo and MG90 servo. The HC-SR04 ultrasonic sensor and the moisture sensor provide information to the Arduino IoT Cloud that triggers the separation mechanism. Abstract does not provide any numbers that were measured.

It's not about monitoring the bin, it's about sorting, and shares only the ESP32, a servo and the cloud link with Smart-Bin. No weight measurement, tamper detection or phone application.

FreeRTOS-Based Smart Waste Management (Aasish et al., 2025)

Aasish et. al (2025) presents a system on a dual core ESP32 based on FreeRTOS. It has an ultrasonic sensor to track the level of the bin, as well as a DHT11/DHT22 to measure temperature and humidity, it has an optional LCD or OLED display and can upload to a cloud server via Wi-Fi for alerts and dashboards and dynamic collection scheduling using RFID modules to identify each bin. The basic concept is to separate the sensor acquisition and actuation tasks, from the cloud communication tasks, so that a slow upload does not block a reading. No actual results are stated and the RFID is not used for access control, it is merely used to identify bins.

The task split is a design to consider for the Smart-Bin firmware, as it is also based on the ESP32 family. The study has no weight sensing, tamper detection or phone app.

Commercial Systems

Commercial systems are available. Bigbelly's CLEAN software provides the data from the bins on a console and uploads it to a collector app (Bigbelly, n.d.). According to its website, Ecube Labs offers ultrasonic fill sensors and a cloud platform that provides fire event notification and route optimization, and claims to predict bin status up to 24 hours in advance (Ecube Labs, n.d.). Sensoneo's platform displays a map of each bin's fill and predicted full time, and plans routes from them (Sensoneo, n.d.), Nordsense's driver app with dynamic routing (Nordsense, n.d.), and Evreka's sensor enhances temperature and motion to flag fires (Evreka, 2020). Compology monitors containers with cameras (Compology, n.d.), and Bin-e is a segregating bin with a touch-free lid (Greenwalt, 2020). In the Philippines, students of the University of the Cordilleras built the Baraki bins for the DOST-TAPI Regional Invention Contest, which open automatically and report to an information center (Asido, 2024). Table 3 summarizes what the vendor pages state.

Table 3

Commercial and Student Systems as Described by Their Own Pages

System	Sensing described	Who sees the data	Collection priority or route
Bigbelly	Fill level, collection events and GPS; sensor type not named	Management console and collector app	Not stated
Ecube Labs	Ultrasonic fill level, battery level and fire events	Cloud dashboard	Route optimization; predicts status up to 24h ahead
Sensoneo	Ultrasonic or radar fill level	Dashboard with a map and predicted full time	Semi-automated route planning
Nordsense	Page covers routing, not sensors	Driver app	Dynamic routing
Evreka	Ultrasonic fill level, temperature, motion and location	Web platform with SMS and email alarms	Route optimization
Compology	Camera: fullness, location and motion	Web dashboard	Not described
Bin-e	Fill level, motion for the flap and waste type	Mobile app	Not covered
Baraki	Automatic sensors; types not stated	Information center	Not stated

Note. Sources: Bigbelly (n.d.), Ecube Labs (n.d.), Sensoneo (n.d.), Nordsense (n.d.), Evreka (2020), Compology (n.d.), Greenwalt (2020) and Asido (2024).

All of them sell via quotation: none of these pages mentions weight, gas or tilt sensing, or a price. That doesn't mean that the products don't have these characteristics, just that vendor pages are concise and designed to sell. What it shows is that a small system with those features at about

₱4,900 for two bins (Appendix B) occupies a different place from these services. Smart-Bin does not compete with them on scale, and it cannot, but it can show what a bin that senses more costs to build.

Technical Merit

Seven design choices in Smart-Bin decide how much its readings can be trusted. Table 4 gives the reason for each. Each one is also a place where a measured result could differ from the real condition of the bin, which is what the tests of the project will have to check.

Table 4

Technical Basis of the Main Design Choices

Design choice	What Smart-Bin does	Why it is sound
BME680 for the air check	One I2C chip gives temperature, humidity, pressure and a gas resistance reading, from which the firmware estimates an air-quality index.	In Table 2, the studies that sense gas use MQ-series modules, which only provide a reading for the gas. Bosch Sensortec (n.d.) states that the BME680 uses 0.09 to 12mA during its different modes. The index is an estimate and needs about 5 minutes to settle.
Median distance reading is calculated from five distance readings.	The firmware makes five measurements using the HC-SR04 and uses the middle measurement.	When the echo bounces off an edge or soft material, a single reading from the ultrasonic can be spiked. One or two of such readings will be ignored without delaying the update, this is the median.
Two-point calibration	The full distance (5cm by default) and empty distance (30cm by default) are defined from the interface and the fill percentage is the distance between them.	The two setpoints are the same as in Keller et al., 2024. Other heights of bins do not require new firmware.
Weight as a second check	The HX711 and a 5kg load cell weigh the contents next to the fill percentage.	Level and weight are also combined in Saad et al. (2023) and Maulida et al. (2025). A loose pile in a bin is light but a full bin is heavy.
Tilt with a countdown	If the tilt is over 30° a countdown of 10s begins. When the bin is steady again the alarm will clear, or if not, the alarm will latch and need to be reset by an operator.	Countdown will not allow for a bump or moving bin to cause a false alarm and an overturned bin or carried away bin will still sound.
Local and cloud together	The bin responds on its local WiFi within ~1s, and to Firebase and ThingSpeak.	Fast control and calibration with local path, history, remote commands and alerts when phone is away with cloud path. The data is uploaded every 30s, which is half the frequency ThingSpeak can accommodate on its free plan of 1 write every 15s
Calibration and simulation from the interface	Distances, limits can be changed, without re-flashing, any sensor can be set to a test value.	None of the 30 studies describes this. It lets a test be repeated at a chosen reading and keeps demonstrations safe.

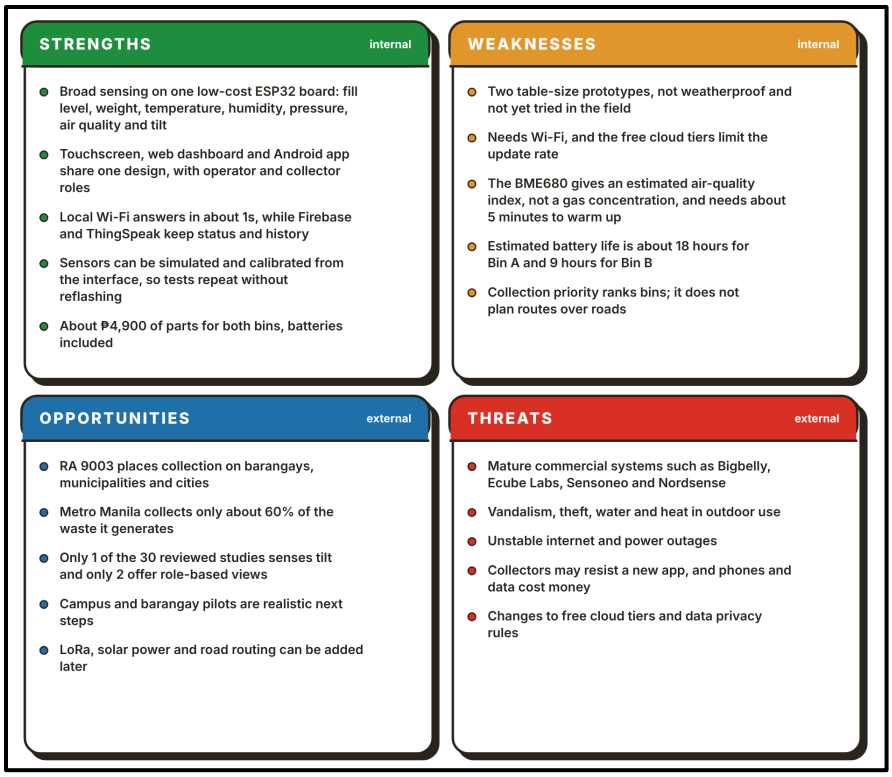
Note. Defaults are those set in the firmware and can be changed in the settings.

SWOT Analysis

The strengths, weaknesses, opportunities and threats of Smart-Bin are presented in Figure 33. The strengths and weaknesses are related to the construct itself, while the opportunities and threats are related to the literature mentioned above and to the situation in the Philippines. The count of studies that sense tilt or offer role-based views comes from this review. RA 9003 puts segregation and collection at the barangay level and leaves the collection of non-recyclable and special waste to the municipality or city (Ecological Solid Waste Management Act of 2000, 2001), and the World Bank (2022) puts the share of Metro Manila waste that is collected at about 60%.

Figure 33

SWOT Analysis of Smart-Bin



The weaknesses set the limits of what the tests of this project can claim. The tests demonstrate that it works on a bench, but not that the prototype can last on the street for 18 hours in Bin A and 9 hours in Bin B and that it is a table-size prototype that has never been outside. The

opportunities are on the other side, the weaknesses: pilots on campus or in a barangay hall, LoRa for where there is no Wi-Fi, solar power and road routing.

Synthesis of the Review

The patents, studies and products reviewed above converge on four points. First, the problem is a documented one. The Ecological Solid Waste Management Act of 2000 (2001) leaves the collection of non-recyclable and special waste to the municipality or city, and the World Bank (2022) puts the share of Metro Manila waste that is collected at about 60%. A bin that overflows between collections is therefore an expected condition of the system that these sources set out to monitor.

Second, sensing the fill level is a solved problem. It is measured by 27 of 30 studies, often using an HC-SR04: Patents have done it with hydraulic pressure (Neumann & Allison, 1991), capacitive plates (Di Giugno et al., 2016), an infrared beam (Devitt & Walsh, 2021) and an ultrasonic sensor (Kekalainen & Engstrom, 2018) and a laser (Keller et al., 2024). The commercial services in Table 3 sell the same measurement with a cloud console on top.

Third, the other sensors exist, but they are added one or two at a time. Saad et al. (2023) and Maulida et al. (2025) add weight, Mehta et al. (2025) add gas, Alnanih et al. (2025) add a flame sensor and Azmi et al. (2024) add temperature and humidity. There is only one reference by Mehta and Rathour (2024) who mention a tilt sensor but do not mention any alarm. In the patents, Mullaney (2016) weights contents, Kurani (2023) mentions around a dozen sensors, but one is based on litter and pathogens, not the user of the bin.

Fourthly, the work is directed more into the route than into the bin. Twelve of the 30 remaining studies report on routes or priority of collection, and have the biggest improvements: up to 52% shorter distance in Banciu et al. (2025) and 58.9% in a pilot of Gupta et al. (2025) with no date or method provided. The bins in these studies stay simple, ten studies have a phone app, two give different users different views, and none of them describes calibration or simulation from the interface of the bin.

None of the sources brings these pieces together in one bin. The studies that weigh the waste have no tilt alarm, the one that lists a tilt sensor describes no alarm, the ones with phone apps and roles have no weight, and the route studies take the readings of the bin for granted. Table 5 sets the four closest patents against Smart-Bin, feature by feature, and Figure 34 counts the features found in the studies. None of the twenty patents, as far as their text was read, combines a latched tamper alarm, a PIN collect mode on a touchscreen, a remote lock and sensor simulation and calibration from the interface with fill, weight and air sensing in one bin. A search of this size cannot prove that no such patent exists, so this is a statement about what was found and not about what exists.

Table 5

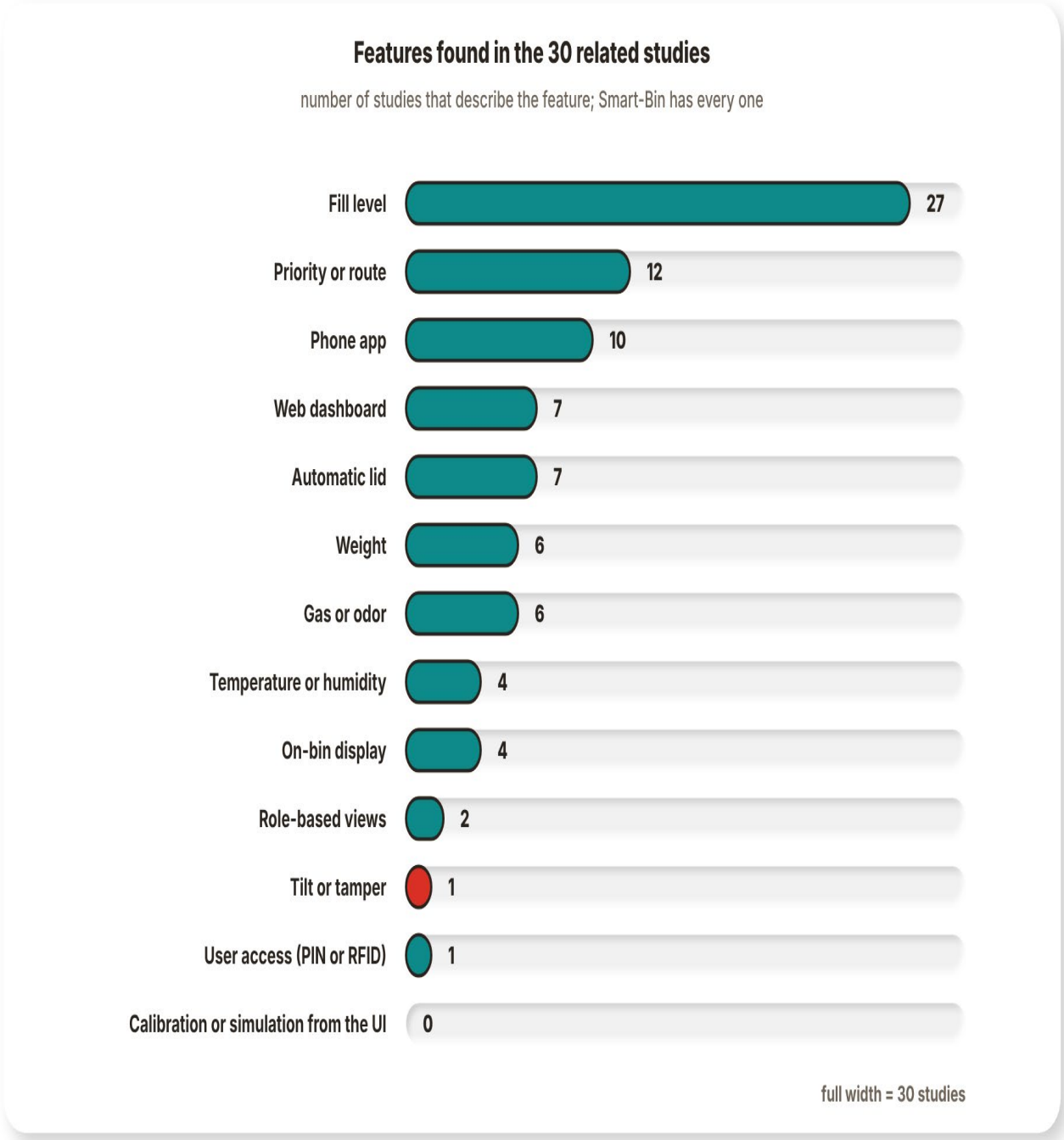
The Four Closest Patents Compared With Smart-Bin

Feature	Shahabdeen (2016)	Yan and Chen (2021)	Kurani (2023)	Sait et al. (2025)	Smart-Bin
Fill level	Yes (range finder)	Yes (infrared height)	Yes	Yes	Yes
Weight	Yes (or fill level)	Yes	Yes	Not found	Yes
Gas or odor	Yes	Yes	Yes	Yes (methane)	Yes
Temperature and humidity	Not found	Not found	Yes	Yes	Yes
Tilt or tamper alarm	Not found	Not found	Accelerometer listed	Not found	Yes
Motorized lid	Not found	Not found	Not found	Not found	Yes
Remote lock or PIN collect	Not found	Not found	Not found	Not found	Yes
On-bin display	Not found	Not found	Not found	Not found	Yes
Phone alert	Not found	Not found	Yes	Not found	Yes
Priority or route	Not found	Yes (truck route)	Yes (route)	Yes (route)	Yes (priority)

Note. Counts are derived from the hardware and the function described in each study (Table 2).

Figure 34

Features Found in the 30 Related Studies



Note. Counts come from the hardware and function each study describes (Table 2).

Research Gap

There are 30 studies; 27 sense fill level. None has all of weight, gas, temperature, humidity and tilt in one bin; 6 add weight, 6 add gas, 4 sense temperature/humidity and 1 senses tilt. Two studies are providing role-oriented perspectives, one of which is using an RFID-based access control mechanism. None indicates simulation / calibration from the interface. The bin indicates its fill level, and the focus is on routes and fleet software in the patents and commercial systems. None of the four most closely related patents have all ten of the features listed in Table 5.

So, it's not a sensor, it's a gap. These factors: fill, weight, air and tilt sensing, alerts to the operator, collector and person at the bin, collection priority from the fill and time to fill, interfaces to allow bin calibration and testing without need to re-flash, are all combined in one single low-cost bin. The gap by capability and the specific objective in Section 1.4 which addresses it is provided in Table 6.

Table 6

The Research Gap by Capability

Capability	Related studies	Patents and products	Smart-Bin	Objective
Sensing breadth on one board	27 fill, 6 weight, 6 gas, 4 temperature or humidity, 1 tilt; none has all	Kurani (2023) lists about a dozen sensors; the others are narrower	Fill, weight, temperature, humidity, pressure, air quality and tilt on one ESP32	SO1
Tamper detection	One study lists a tilt sensor	No latched tamper alarm found in the four closest patents	Tilt beyond 30° starts a 10s countdown, then a latched siren until RESET TAMPER	SO1
Alerts to each kind of user	10 studies have a phone app and 7 a web dashboard	Phone alerts in Kurani (2023)	Buzzer, LED and touchscreen on the bin, web dashboard, Android app and push alerts	SO2
Roles and access	2 studies have role-based views; 1 uses RFID	None found	Operator and collector views, remote lock, PIN collect mode	SO2, SO3
Collection priority	12 studies are about routes or priority, mostly with GPS and algorithms	Route planning in the closest patents	Fullest first, with a time to full from the recent fill rate; no road routing	SO3
Testing and demonstration	No study describes simulation or calibration from the interface	None found	Per-sensor simulation and calibration from the dashboard and app	SO3

1.4 Objectives

General Objective

To design and develop Smart-Bin, an IoT based Waste Bin Monitoring System which will monitor the fill level, weight and air quality of the waste bin, detect tampering and give real time monitoring and alerts to the concerned entities via Web Based Dashboard, Android App and On-Bin Touchscreen.

Specific Objectives

1. **Sensor Monitoring and Detection**

   To develop 2 Smart-Bin units, which include sensors to measure fill level, weight, temperature, humidity and air quality, as well as to detect bin tilting and tampering and open the lid automatically when hand movement is detected.

2. **Real-Time Monitoring and Alerts**

   To develop a web dashboard and an Android application to display real-time data from the sensors, bin status, retrieve data from Thingspeak/Firebase and notify the operators and collectors when the bin is full, poor air quality, high temperature, tampered sensor reading and when the bin is not online.

3. **Bin Control and Collection Prioritization**

   To implement Collection Prioritization, based on bin fullness and estimated time to fullness, remote bin locking and unlocking, on-bin collection functions (via PIN), sensor simulation and system interface sensor calibration. This will lead to the prioritization of collections and will also help with efficient collection routes and fuel gas efficiency.

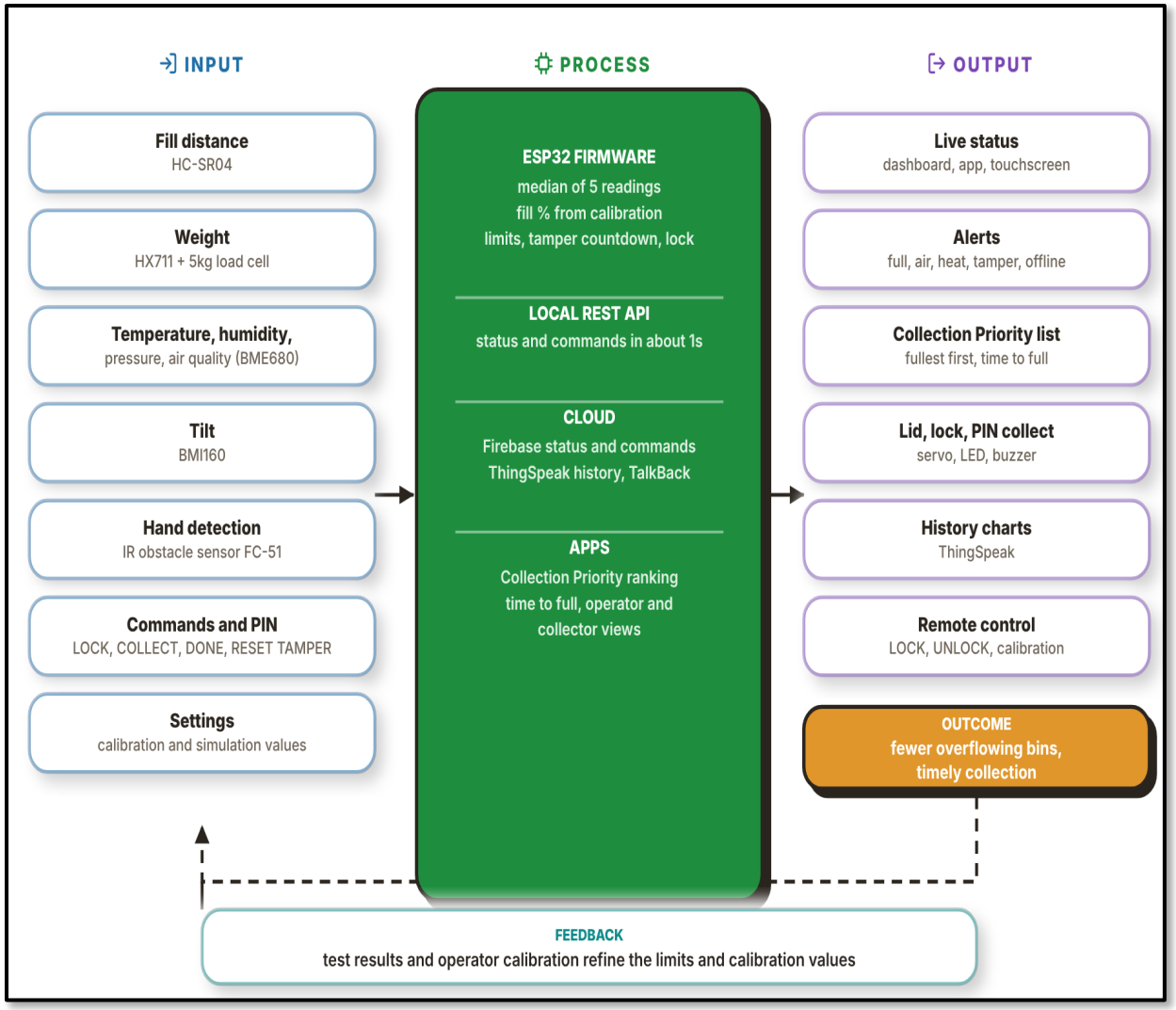
1.5 Conceptual Framework

The framework of Smart-Bin is shown in two parts. The first one which is below describes the process of a reading going from the sensor to the person who is going to respond to it and it is known as the Input-Process-Output (IPO) model. The second, under IoT Level, shows how the parts are deployed and connected. Both refer to the same system and both result in the three specific objectives listed in Section 1.4 above.

Input-Process-Output Model

Figure 35

Input-Process-Output Model of Smart-Bin



The input column has seven items. The distance to the trash is measured by the HC-SR04 sensor, the weight is measured by the HX711 sensor (which includes a 5kg load cell), the BME680 sensor provides temperature, humidity, pressure and an estimate of the air quality, the BMI160 sensor gives the tilt, and the IR sensor detects a hand. The final two inputs are from the people: commands input on the touchscreen, in the dashboard or in the app, and the settings, which contain the calibration and simulation values. All of these inputs are related to Specific Objective 1, and commands and settings are related to Specific Objective 3.

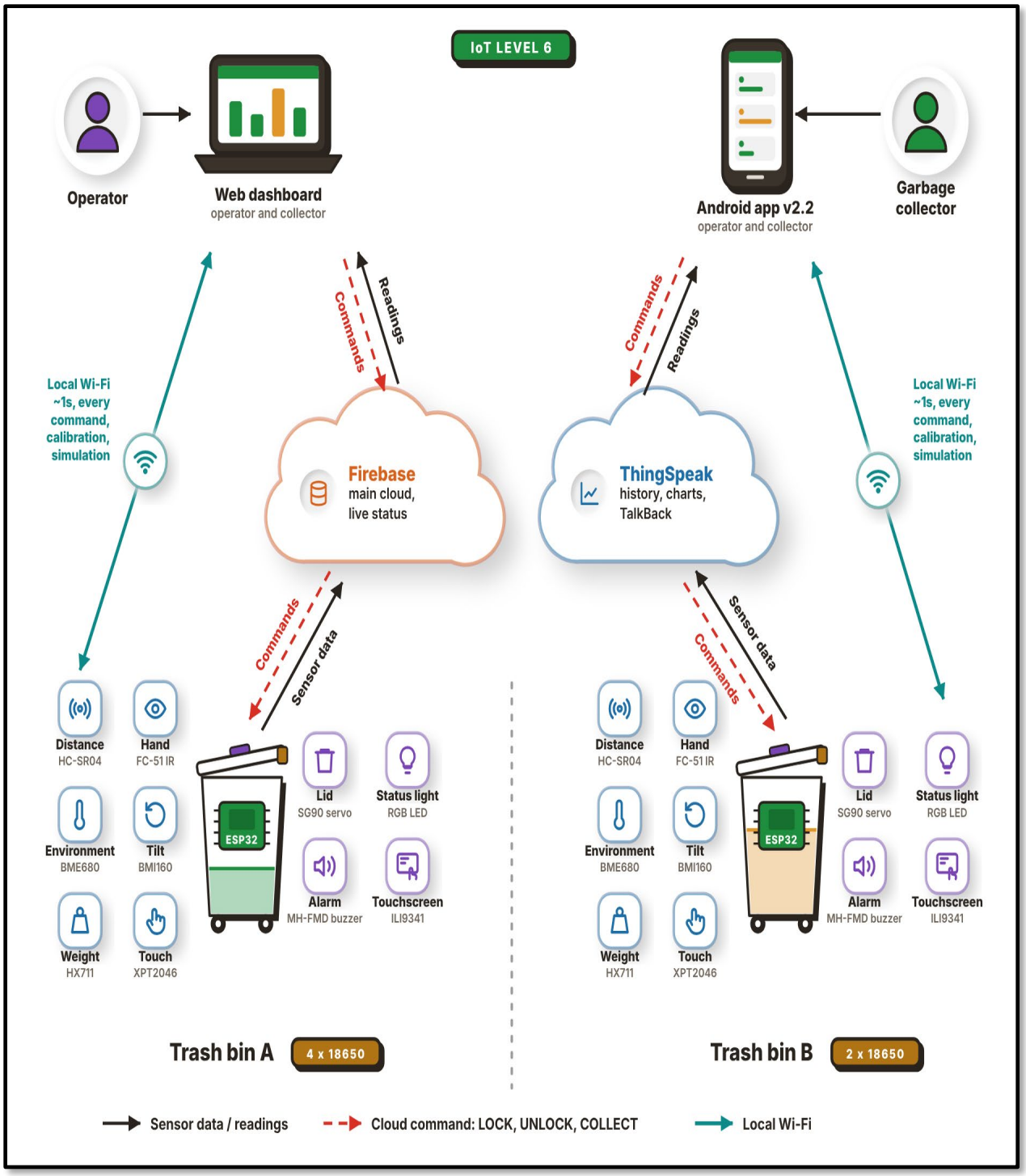
The process column is how the ESP32 firmware and the cloud use them. The firmware takes the median of five readings, converts the distance to a fill percentage from the calibration limits, compares the readings with their thresholds, runs the tamper countdown and applies the lock. Local REST API provides response in ~1s and status is stored in Firebase while the commands are stored in thing speak where the history is stored. The apps rank the bins by fullness, by time until they are full and give the operator or collector views. Specific Objectives 2 and 3 are addressed in this column.

Live status on the touch screen, the dashboard and the app, alerts for a full bin, bad air, heat, tamper, or bin being offline, Collection Priority list, live status for the lid, lock, and PIN collect on the bin, history charts, and remote control, are all things people see and do, which is the output column. The result is reduced overfilling bins and timely collection, which is the problem stated in Section 1.1. Feedback closes the loop; the results of the test and the operator's calibration will improve the limits and calibration values used by the firmware next.

IoT Level

Figure 36

System Architecture of Smart-Bin



The connections between the parts of Smart-Bin are shown in Figure 36. The readings are sent from the cloud to the two apps at the top: the web dashboard (primarily for the operator) and the Android app (primarily for the garbage collector). Both apps can also reach a bin directly over the local Wi-Fi in about 1s for any command, calibration, or simulation.

There are two end nodes at the bottom: Trash bin A and Trash bin B. All are equipped with the same ESP32 and six input sensors (HC-SR04, IR sensor, BME680, BMI160, load cell, touch panel) and four output actuators (servo, RGB LED, buzzer, TFT). Bin A is powered by 4x 18650 cells, Bin B by 2x.

Each bin sends its sensor data up to two cloud services and receives the LOCK, UNLOCK and COLLECT commands from them. Firebase is the primary cloud and stores the live status, ThingSpeak stores the history and charts and relays commands via TalkBack.

Smart-Bin fits IoT Level 6 of the deployment templates in Bahga and Madisetti (2014). A Level 6 system has several independent end nodes that sense or actuate and send their data to the cloud, where it is stored and shown by a cloud-based application. An analytics component works on that data and keeps its results in the cloud as well, and a centralized controller knows the status of every end node and sends control commands back to them. Trash bin A and Trash bin B are the end nodes in Figure 36. Both have their own ESP32, they read their own sensors and update their own status to Firebase and ThingSpeak, they don't depend on each other and there is no gateway between them and the cloud.

Cloud side is Firebase for live status, and ThingSpeak for stored history. ThingSpeak keeps the readings of every bin and plots and analyses them over time, and the apps use the same data to rank the bins by fill level and estimated time to fill in the Collection Priority list. The web dashboard and the Android app take the role of the centralized controller. They show the state of both bins side by side and send LOCK, UNLOCK and COLLECT back through the cloud, and calibration and simulation commands go to a bin directly over the local Wi-Fi. Both read the lock, collect, calibration and tamper flags that the bin itself reports, so a bin locked or put into calibration from one of them shows the same state and the same buttons on the other.

1.6 Target Beneficiaries

1. **Waste Collectors and Utility Staff**

   Smart-Bin will assist waste collectors and utility workers by notifying them which bins are fullest and need to be emptied first. It could also prevent visits to bins that are not full and offer PIN access to the bin during collection.

2. **Facility Managers and Operators**

   The web dashboard can be used by facility managers or barangay hall, school, office or mall operators to monitor the status of the bins, make assessment of the data and set thresholds for the sensors, apart from the messages they receive on overflowing bins, poor air quality, high temperature, and even possible tampering.

3. **Barangay and Local Government Unit (LGU) Sanitation Offices**

   The sanitation offices of barangays and LGUs can utilize Smart-Bin's data readings to aid in garbage collection scheduling and management. The percentage of fullness and time to be full will help them optimize their collection routes and minimize the amount of fuel they need to use by making fewer unnecessary collections.

4. **Filipino Citizens**

   How about if the environment is cleaner, trash bins are not overflowed and there is increased awareness about waste among Filipino citizens who are residents, employees or students of schools, offices or other places? The hands-free lid would also improve the disposal of waste; making it easier and sanitary.

II. PROTOTYPE DESIGN AND DEVELOPMENT

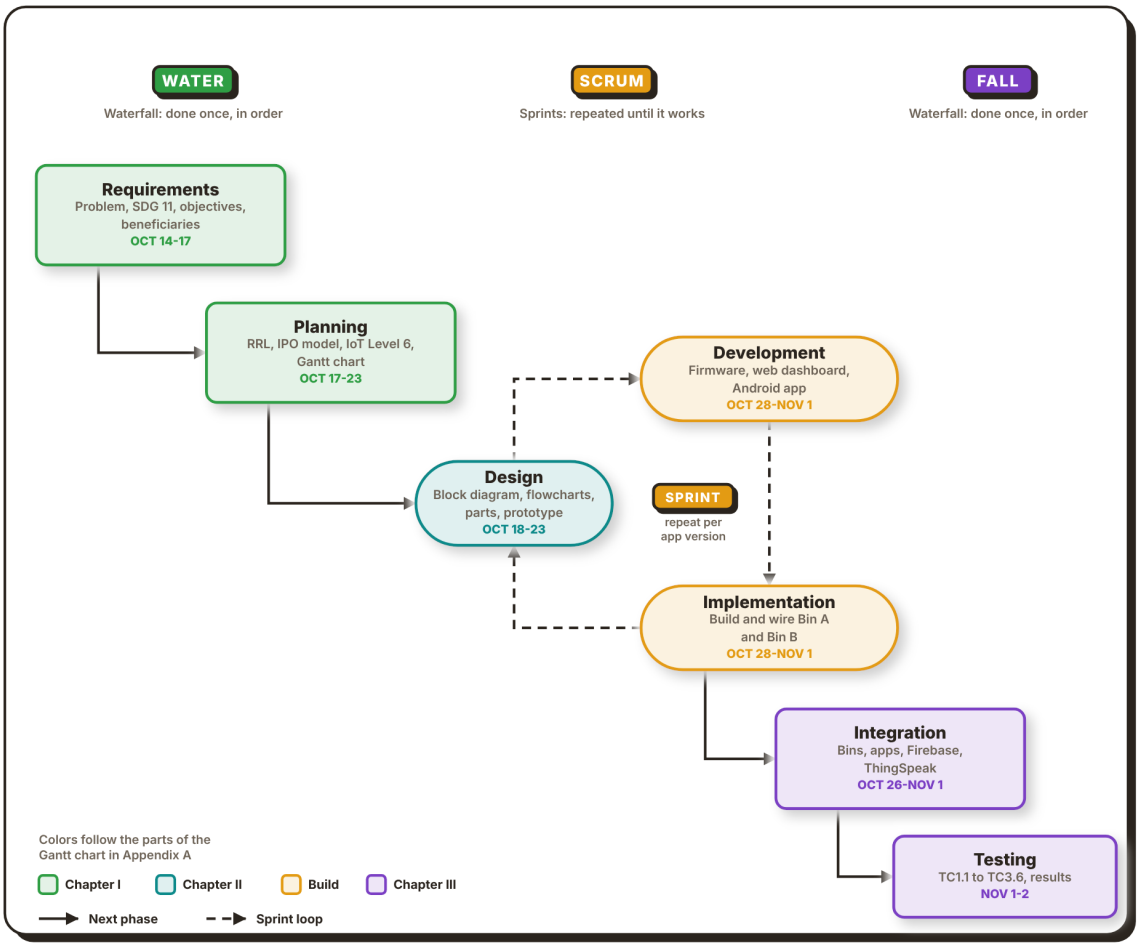
2.1 Development Methodology

Smart-Bin was developed with a hybrid software development life cycle (SDLC) called Water-Scrum-Fall. Samolovac (2023) describes it as a combination of Waterfall and Scrum in which the project begins and ends with Waterfall, while the execution phase is done in iterative sprints until the final deliverable is complete. Figure 37 shows the model as the group used it. Each phase is labeled with what it covered in this project and with its dates from the Gantt chart in Appendix A.

The course decided the two ends. The topic, objectives and chapters had to be settled in order, and the paper and presentation are due on November 6, 2026, so neither the start nor the finish could move. The middle was different. One unit joins an ESP32, five sensors, a servo, a touchscreen, a web dashboard, an Android app and two cloud services, and a fault in one part often showed only after another part was connected to it. Planning that stretch once and following it to the end would not have survived the first built bin.

Figure 37

Water-Scrum-Fall Development Life Cycle of Smart-Bin



Note. Adapted from *What Is Hybrid Project Management?*, by J. Samolovac, 2023, Plaky (https://plaky.com/learn/project-management/hybrid-project-management/). The phase dates are from the Gantt chart in Appendix A, and the colors follow its parts.

Water: Requirements and Planning

Requirements ran from October 14 to 17. Banogbanog and Reyes wrote the objectives, the SDG and the target beneficiaries, while Banogbanog and Celeste wrote the introduction and the problem. Their output was the general objective and the three specific objectives in Section 1.4, which the group treated as fixed. Each specific objective later became one group of test cases in Chapter III: TC1.1 to TC1.6 for sensing and detection, TC2.1 to TC2.6 for monitoring and alerts, and TC3.1 to TC3.6 for bin control and collection priority.

Planning followed from October 17 to 23. Gunayan and Pepito reviewed the related patents and the studies from 2023 onward, wrote the SWOT analysis and the research gap, and built the IPO model and the IoT Level 6 framework in Section 1.5. The plan behind the build came from the same work. The implementation plan fixed the parts, the local REST API of the bin and the cloud channels, and the Gantt chart fixed who did what and by when. From October 21 to 23 the whole group merged Chapters I and II and checked them against APA 7.

Scrum: Design, Development and Implementation Sprints

The middle of the project ran as sprints. A sprint went through design, development and implementation, put the result on a real board or bin, and carried what it found back into the next design. The first designs were made from October 18 to 23: the block diagram by Reyes, the component materials by Celeste, the system flowchart by Reyes and Celeste, and the prototype design by Banogbanog. They appear in this chapter as Figures 38 to 56.

The Gantt chart gives development the window from October 28 to November 1, with Reyes on the firmware and the dashboard and Celeste on the Android app. The code itself started sooner. Working firmware, the live web dashboard and version 2.0 of the app were in the project repository by October 17, and app version 2.2 followed on October 18. Until a bin existed, the two apps could only be tried on demo data, so the bar on the chart marks the time in which they had to work on real hardware.

Implementation, the build and wiring of the two bins, was given to Banogbanog and Gunayan for the same window. The first bin was built and tested on October 28, and that test started the busiest sprints of the project. The tamper alarm went off soon after power-up, because the upright angle saved earlier no longer matched the bin. The fix went back through design and development the same day as app version 3.0: the bin now saves its upright angle at every power-up, the lid stays shut for 2s after it closes so it does not flicker, and the bin reports locked, pickup, calibrating and tamper as separate flags so the touchscreen, the dashboard and the app show the same state. The cloud test on Bin 01 on October 29 found two more problems. ThingSpeak carried only a short status word, which version 3.1 replaced with the full set of flags, and the lid kept

reopening after it closed because the servo retriggered the PIR sensor. On October 30 the PIR sensor was replaced with the FC-51 IR obstacle sensor, and both bins were finished that day. The drawing in Figure 38 was corrected once more on October 31, after the built bins showed that the electronics sit in a bay behind the front wall.

Fall: Integration and Testing

Integration joins what the sprints produced: the two bins, the web dashboard, the Android app, Firebase and ThingSpeak. On the Gantt chart it runs from October 26 to November 1 and overlaps the last sprints, because every sprint already ended with a check on the connected system. The Bin 01 cloud test is one such check, with the bin signed in to Firebase under its own account and writing to ThingSpeak channel 3556235. In the model, integration is where the loop stops. Once the bins and both apps carry the same version, changes are limited to fixing faults.

Testing closes the project as a single Waterfall pass. The Gantt chart sets November 1 and 2 for it: Banogbanog and Reyes write the test cases from the three specific objectives, and the whole group runs them and records the results that Chapter III reports. The final review, the PDF and the slides take November 3 to 5, and the presentation is on November 6.

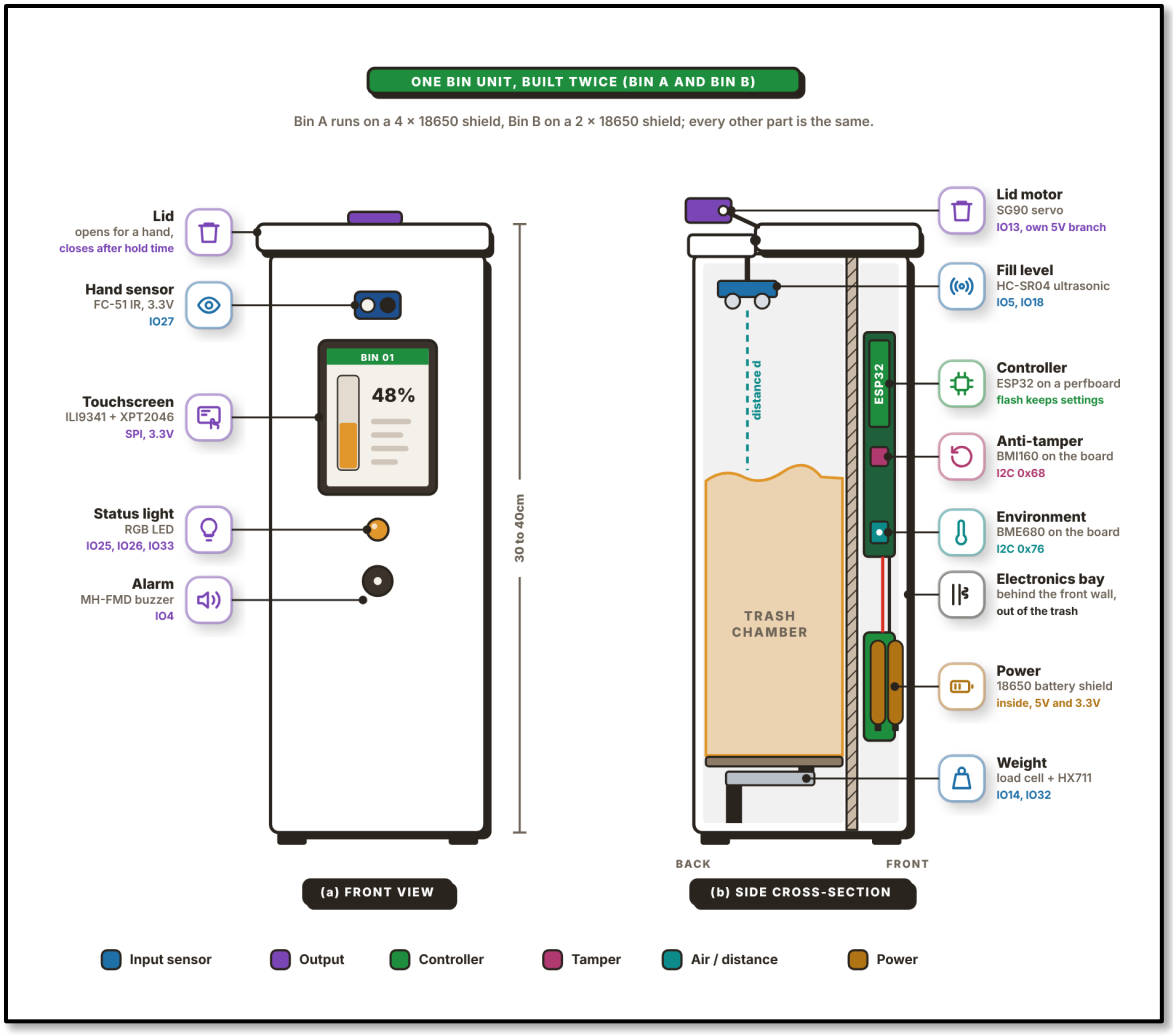
The method had a cost on paper. Every sprint that changed the bin also changed this document: the system and operator flowcharts were redrawn on October 18 and again on October 28, and the prototype drawing on October 31. The fixed requirements kept that cost bounded. The sprints changed parts, settings and screens, but the three specific objectives stayed the same, so the test cases written from them still applied at the end.

2.2 Prototype Design

The physical design of one Smart-Bin unit is shown in Figure 38. The group builds the same unit twice, as Bin A and Bin B, and the only difference between the two is the battery shield: Bin A runs on four 18650 cells and Bin B on two. Each unit is a table-size model, about 30 to 40cm tall. Inside, a divider wall stands a short distance behind the front wall and splits the bin into two parts. The larger part at the back is the trash chamber, which holds the lid and the sensors that have to face the trash. The narrow bay behind the front wall holds the electronics: the perfboard with the ESP32 and the battery shield, so neither the board nor the power supply touches the trash.

Figure 38

Front View and Side Cross-Section of One Smart-Bin Unit



The front view (a) shows what a user sees first. Near the top is the FC-51 IR obstacle sensor on GPIO 27. When it sees a hand, the SG90 servo lifts the lid, which closes again after the open time the operator sets (5s by default). Once shut, it stays shut for a close time (2s by default), so a hand still in front of the sensor does not make it flick open again. Below it is the 2.4-in. touchscreen (ILI9341 with XPT2046 touch), mounted in portrait so that its fill bar rises the same way the trash does. The screen shows the fill level, weight, temperature, humidity and IAQ estimate, and it takes the PIN before a collection. Under the screen are the RGB status light and the MH-FMD buzzer on GPIO 4. The light stays green while the bin is below 50%, turns amber from 50%, and blinks red when the bin is full or tampered with.

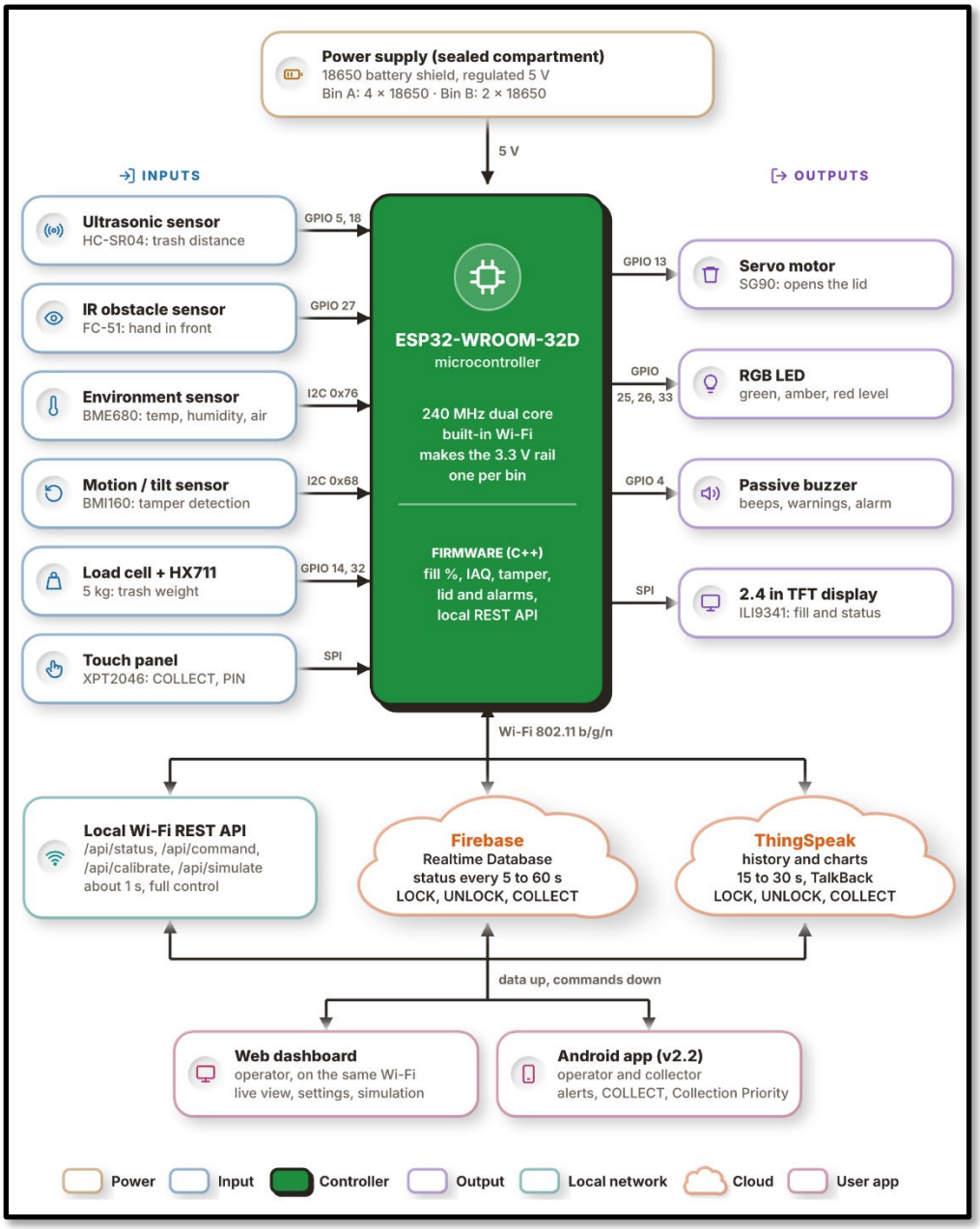
The side cross-section (b) shows the inside. The HC-SR04 ultrasonic sensor hangs from a fixed strip at the back of the top, behind the lid hinge, so it stays still when the lid moves. It points down and measures the distance d to the top of the trash; the firmware takes the median of five readings and turns it into a fill percentage. The servo sits on the same strip and pushes the lid open with its arm. The electronics bay sits right behind the front wall, so the touchscreen, the IR sensor, the status light and the buzzer connect to the board through the wall with short wires. In the bay the ESP32 sits on a perfboard, with the BMI160 and BME680 modules soldered on the same board. The board is fixed to the bin, so the BMI160 moves together with the bin body, which the tilt check needs. The battery shield sits in the same bay below the board, and the bin has no charging port on the outside. The trash rests on a platform over the 5kg load cell, and the HX711 amplifier turns the strain into a weight in grams.

2.3 Block Diagram

The block diagram of one Smart-Bin unit and the network around it is shown in Figure 39. It has two separate ESP32 microcontrollers, one in each of the two bins, both bins reporting to the same cloud services and user apps.

Figure 39

Block Diagram of the Smart-Bin System



The power supply is located in the top portion of the diagram, consisting of an 18650-battery shield in a closed compartment providing the circuit a regulated 5V. Bin A is powered by 4x 18650 cells and Bin B by 2x. The 5V is fed to the ESP32-WROOM-32D which supplies the 3.3V needed by the sensors, display and buzzer; the servo and HC-SR04 operate directly from the 5V. ESP32 is a 240MHz dual-core Wi-Fi enabled micro-controller. It has firmware written in C++ and uses that to determine the fill percentage, indoor air quality (IAQ) estimate and tamper state, as well as controlling the lid and alarms and providing a local REST API.

The inputs are on the left. The HC-SR04 Ultrasonic sensor on GPIO 5 and 18 can be used to detect the distance between the lid and the garbage bin, and output the fill level information, the FC-51 IR obstacle sensor on GPIO 27 can be used to sense a hand held in front of the garbage bin. Two sensors, BME680 (address 0x76) for temperature, humidity and air quality and BMI160 (address 0x68) for motion and tilt for tamper detection share this I2C bus. Using a 5kg load cell, it is connected to GPIO 14 and 32 via an amplifier module named HX711 to measure the weight of the trash. On the SPI, the touch panel XPT2046 accepts the COLLECT and PIN inputs.

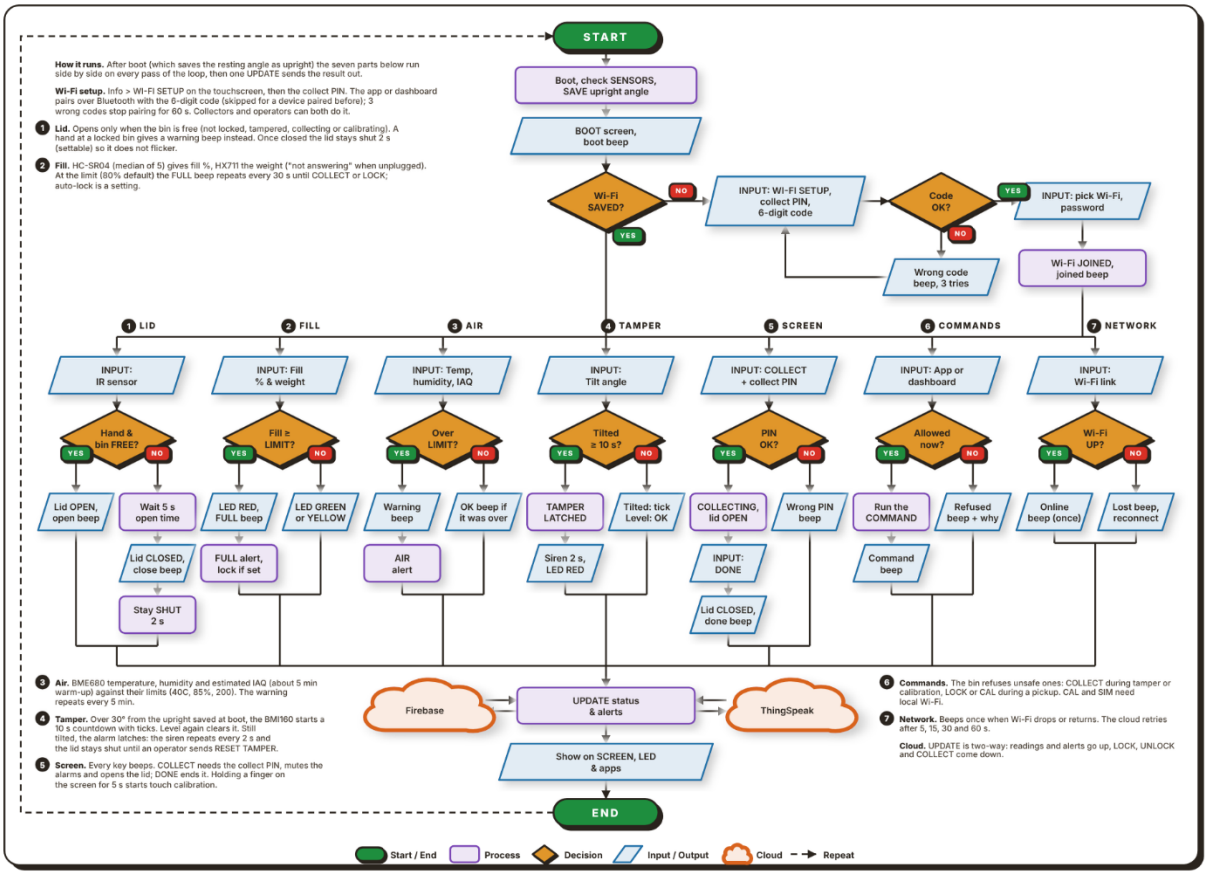
The outputs are to the right. The lid is opened and closed by the SG90 servo on GPIO 13 (PWM). The fill level is indicated by an RGB LED (GPIO 25, 26 and 33) in green, amber or red. Beeps, warnings and the tamper siren come from a passive buzzer on GPIO 4. The fill level and the bin status are displayed on the 2.4 in. ILI9341 TFT display (also SPI).

Three paths of data exit the bin via Wi-Fi (802.11 b/g/n) as shown in the bottom half of the image. The local REST API (/api/status, /api/command, /api/calibrate and /api/simulate) responds in ~1s and provides full control of the bin. The status is sent to the Firebase Realtime Database every 5 to 60s and to ThingSpeak every 30s while the readings change and at least once every 5 minutes, as well as the LOCK, UNLOCK and COLLECT commands (the latter via ThingSpeak's TalkBack feature). Data goes up and commands come down to two user apps. The web dashboard runs in a browser on the same Wi-Fi and gives the operator a live view, settings and simulation, while the Android app gives operators and collectors alerts, the COLLECT command and the Collection Priority list.

2.4 System Flowchart

Figure 40

System Flowchart of the Smart-Bin



Note. Figure 40 shows the main program loop that runs on each bin. After start-up, seven parts run side by side on every pass of the loop, and their results meet at one update step.

Upon powering on the bin, the ESP32 starts, checks its sensors and loads its saved settings, with a boot screen and a beep. It also saves the angle it is resting at as the upright position, so the tamper check starts from the way the bin is standing. It then examines if a Wi-Fi network is saved. If not, the user opens WI-FI SETUP on the touchscreen and enters the collect PIN, and the app or dashboard pairs with the bin over Bluetooth. The 6-digit code on the bin's screen is typed into the pairing box (a device paired before skips it, and three wrong codes stop pairing for 60s). The bin returns a list of all the Wi-Fi networks within its range, and the user selects one and enters its password, after which the bin beeps to join. This loop is split into 7 parallel sections (1-7 on chart).

The lid part (1) reads the IR sensor. If the bin is free (not locked, tampered with, collecting or being calibrated), it beeps to open the lid. Otherwise, bin lid remains closed and a beep is heard when a hand is seen with the lid closed. The lid beeps and closes 5 seconds after the hand is gone, and then stays shut for at least 2 seconds before a hand can open it again. The fill part (2) reads the weight from the HX711 (or reports that the HX711 is not answering when it is unplugged), and takes the median of 5 HC-SR04 readings for the fill percentage. At the default setting of 80%, the LED flashes red and an alert message is sent every 30s until the bin is collected (or locked) when the fill is at the limit. When set to auto-lock, the bin itself locks as well. If lower than the limit the LED is green or yellow. The air part (3) compares the measured temperature, humidity and estimated air quality of the BME680 with their limits (40C, 85% and an index of 200). The air quality is an approximation based on the gas reading and should be warmed up for approximately 5 minutes. When over a limit the buzzer sounds and an alert is sent every 5 minutes, whilst a short beep indicates when the reading is back under.

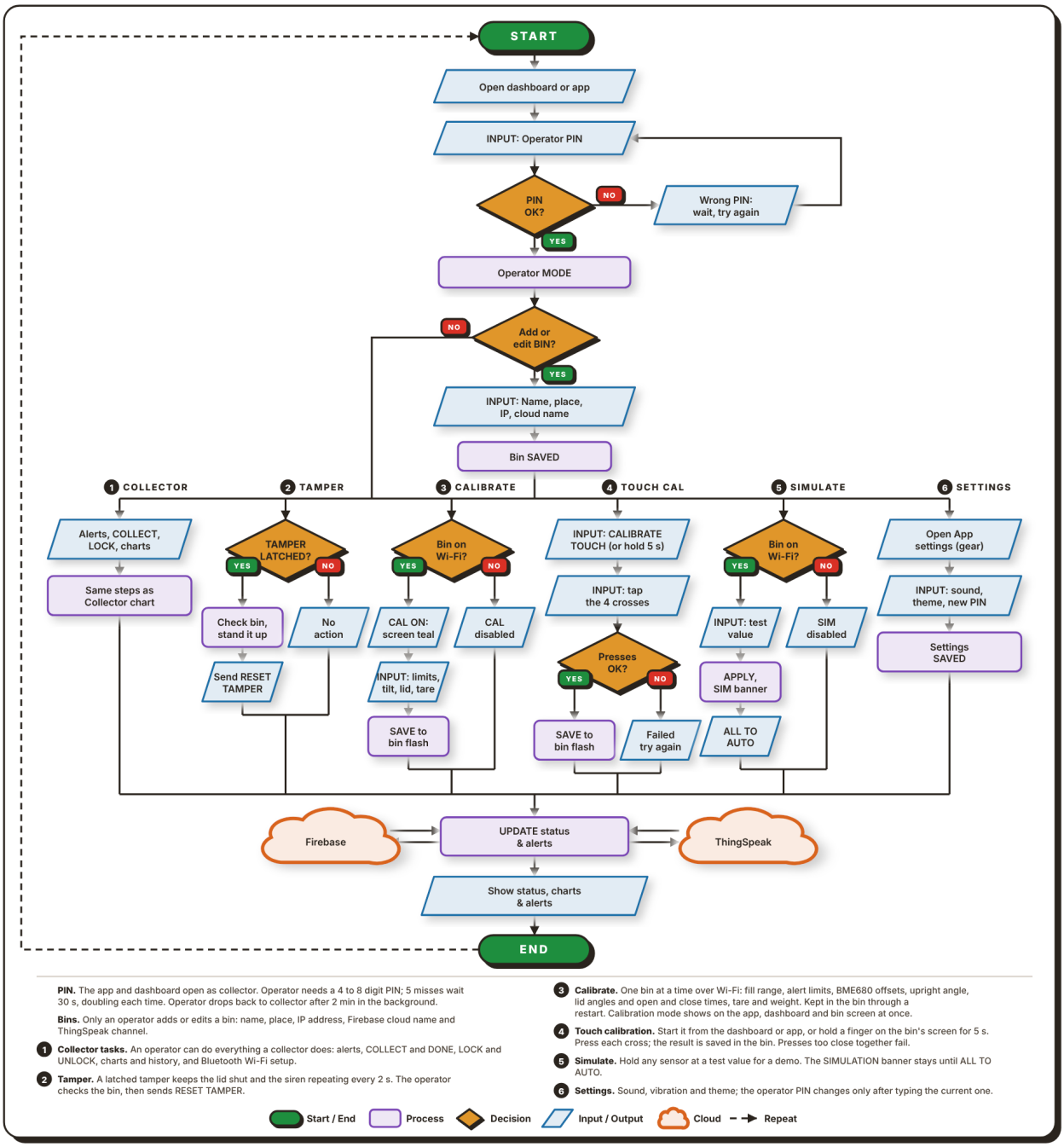
The BMI160 tilt angle is monitored by the tamper part (4). If the tilt from the upright position saved at power-up exceeds 30°, a countdown of 10 seconds begins with a "tick" every second. The alarm automatically resets if the bin is steady once more. Until an operator sends RESET TAMPER, the alarm latches when it remains tilted: the siren sounds every 2s, the LED is red, the lid remains closed and an alarm message is generated. The screen part (5) is used for handling the COLLECT touch command. Once collect PIN is entered lid will open and alarms will be muted until DONE is pressed and the lid is closed and the fill level is reset, or if the collect PIN is wrong, an error beep is provided. Finger on screen for 5s to start touch calibration. The command’s part (6) receives the following commands from the app or dashboard: LOCK, UNLOCK, COLLECT, calibration (CAL) and simulation (SIM). Bin beeps if an unsafe one is attempted such as COLLECT during tamper alarm and CAL and SIM are Wi-Fi only. The network part (7) beeps once when the Wi-Fi is lost and once when it returns.

All seven paths then lead to an UPDATE step. The two arrows between this step and each cloud reflect that the reading and the alerts are sent to Firebase and the ThingSpeak, and the LOCK,

UNLOCK and COLLECT commands are returned to it. This new status is then reflected on the screen, on the LED and in the apps and the process is repeated.

Figure 41

Operator Flowchart of the Smart-Bin Apps



Note. As seen in Figure 41, an operator can do the following actions in the Web Dashboard or in the Android App

.

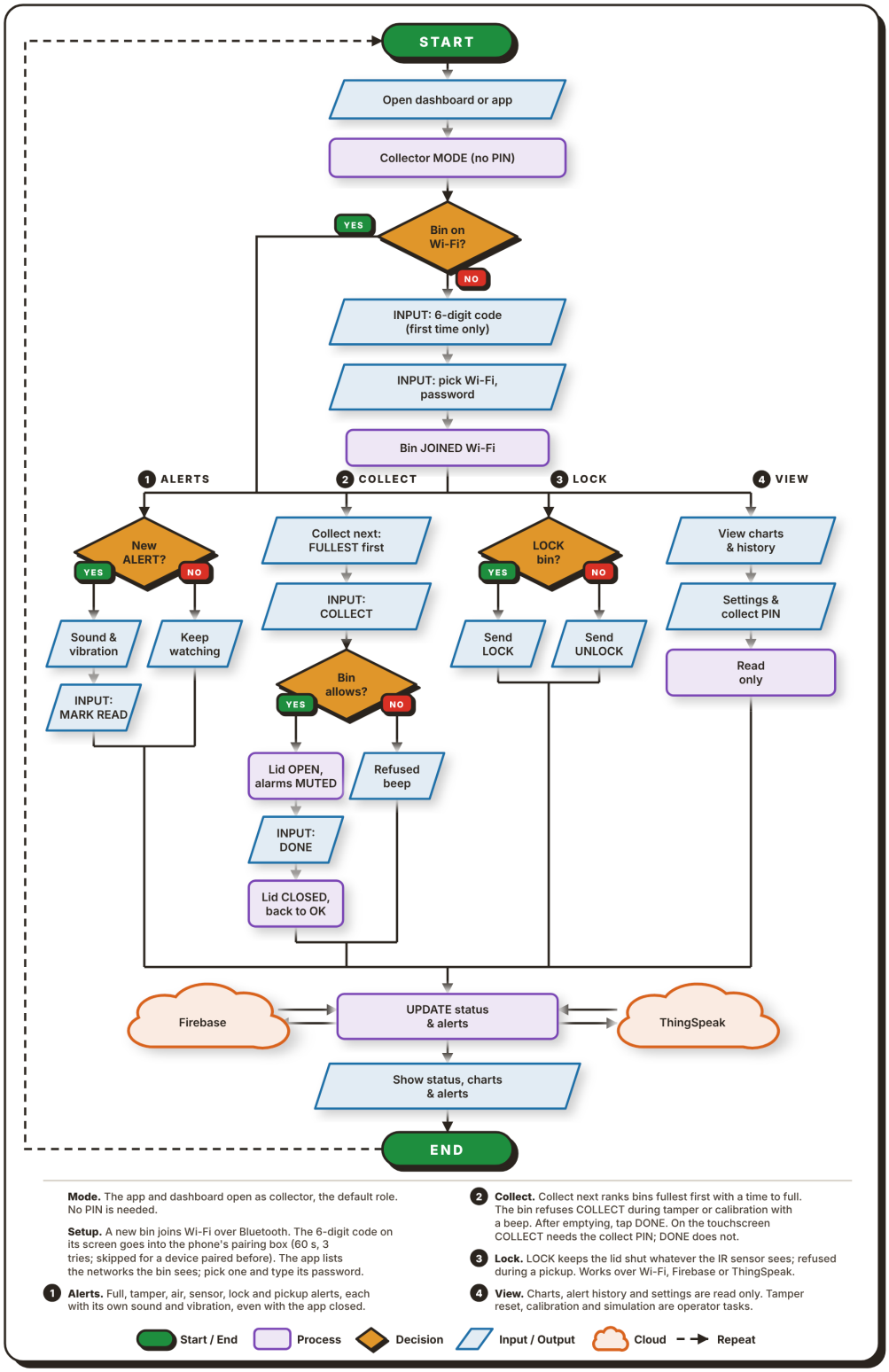
In both apps, the collector mode is the default, and the operator enters his 4- to 8-digit PIN. A wrong PIN requires a delay between attempts, with 30s after 5 failed attempts and the delay for subsequent failed attempts doubling. The app puts itself in operator mode with the appropriate PIN. The operator can then add or edit a bin, giving it a name, a place, the IP address and cloud name for Firebase and ThingSpeak, and the bin is saved in the app.

There are five tasks branching out from there. Everything that a collector can do is Collector (1), as shown in Figure 42: alerts, COLLECT and DONE, LOCK and UNLOCK, charts and history, and setting up the Wi-Fi over Bluetooth. Tamper (2) is for a bin with an alarm that latched. The operator checks the bin and stands it up and then sends a RESET TAMPER to clear the alarm. Calibrate (3) requires bin to be on same Wi-Fi. The operator sets new limits (fill range, alert limits, BME680 offsets, the BMI160 upright angle, lid angles and open and close times, weight tare and more) and these limits are stored in the flash memory of the bin. Calibration is disabled if the bin is not connected to Wi-Fi. The bin reports when it is in calibration mode, so turning it on from the dashboard, the app or the bin's screen shows on the other two as well. The touch calibration (4) can be initiated from the dashboard or app or by holding a finger on the bin's screen for 5s. Operator presses four crosses sequentially and result is stored on the bin (if pressed too close, it is a fail and the operator retries).

Simulate (5) keeps the value of any sensor for demonstration purposes. A SIMULATION banner will appear on the screen while it is on until the operator returns everything to AUTO. The sound, vibration and theme are set by Settings (6) and the operator can change the PIN in the settings after entering the PIN. The result of whichever task is used then iterates through the same 2-way UPDATE process with both the Firebase and the ThingSpeak services and the app will display the new status, plus the charts and alerts.

Figure 42

Collector Flowchart of the Smart-Bin Apps



Note. As seen in Figure 42, the collector's job in the web dashboard or in the Android app.

The apps are opened to collector mode which is the default mode and no pin is required. The collector uses Bluetooth to set up the bin if not already connected to Wi-Fi. The 6 digit code displayed on the screen of the bin is entered into the pairing box of the phone (code will be valid for 60s, 3 failed attempts will lock pairing for 60s, a device that has been paired before will not require a code). The app then lists the Wi-Fi networks the bin can see, and the collector picks one and enters its password. After the bin joins the network, there are four tasks available.

Alerts (1) are issued for a full bin, tampering, bad air, lock and pickup. Each has its own sound and vibration and they come even when the app is not in use. The primary task is collect (2). The Collection priority list is a ranking of the bins, with the first bin in the list being the one with the most capacity, and the time to fill each bin is shown. The collector taps COLLECT in the app or on the screen, and enters the collect PIN at the bin. Lid opens and alarms stop. The collector then empties the bin and taps DONE (no PIN required) and the lid will then close and the fill level will reset. A bin refuses COLLECT, with a beep, while a tamper alarm or a calibration is on.

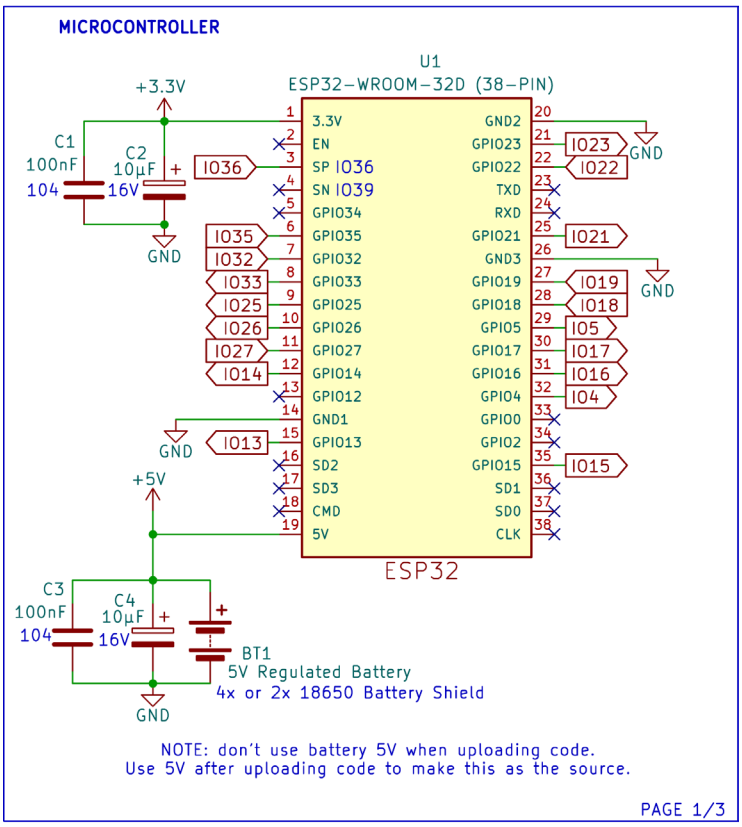
Lock (3) allows the collector to lock or unlock a bin, while resetting a tamper alarm, calibration and simulation stay with the operator. View (4) displays the charts, alert history, and status and collect PIN in read-only mode. All tasks are completed with the same step in Firebase and ThingSpeak and the app updates the status.

2.5 Schematic Design

The circuit of each unit is drawn as one schematic split into three sheets: the microcontroller, the actuators and outputs, and the sensors and inputs. The order of the sheets is: Figures 43, 44, 45. Instead of long wires between sheets, each connection to the ESP32 has a net label (IO25, etc.) and a bunch of pins with the same net label are grouped together.

Figure 43

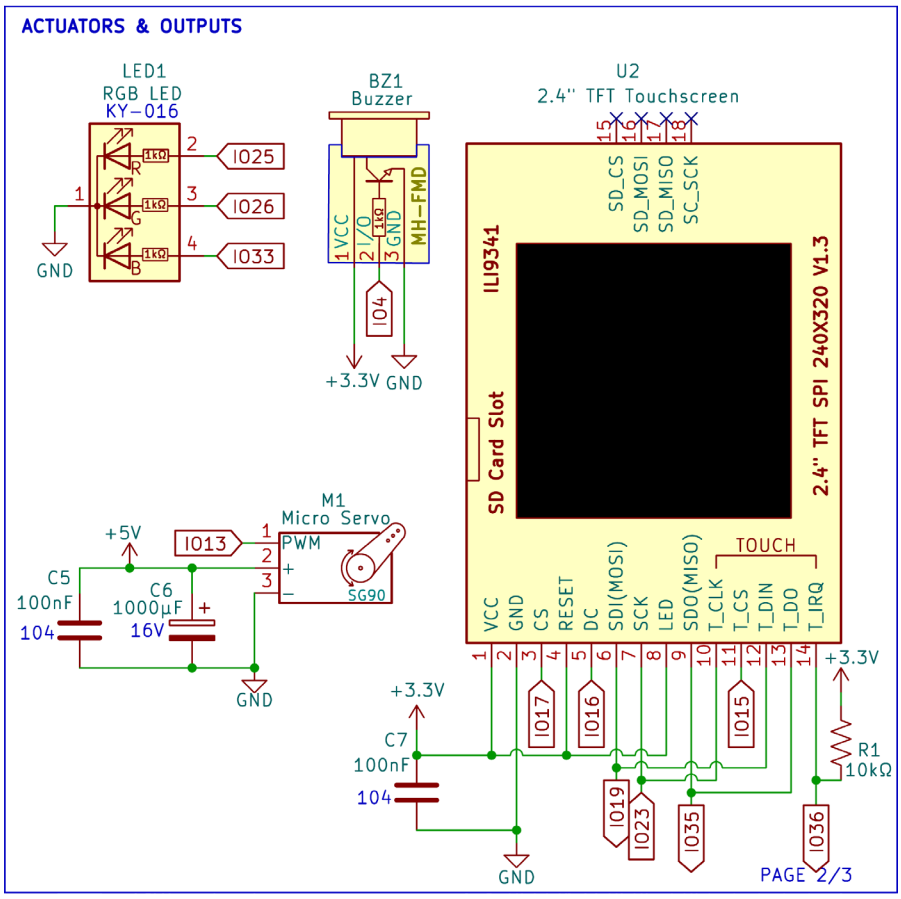
Schematic Diagram, Sheet 1: Microcontroller



U1 is the ESP32-WROOM-32D on a 38-pin board. The 4x or 2x 18650-battery shield, BT1, provides the 3.3V rail for the sensors, the display and the buzzer (pin 1) and a regulated 5V to pin 19. C1 and C2 (100nF and 10µF) smooth the 3.3V rail, and C3 and C4 (100nF and 10µF) do the same at the battery input. The ground pins share one GND net. Pins which are not configured in the design (such as EN, TXD, RXD, GPIO0, GPIO2) are identified as "no-connect". According to the note on the sheet, the battery's 5V should not be used while code is being uploaded; rather, it should only be used after the code is uploaded.

Figure 44

Schematic Diagram, Sheet 2: Actuators and Outputs

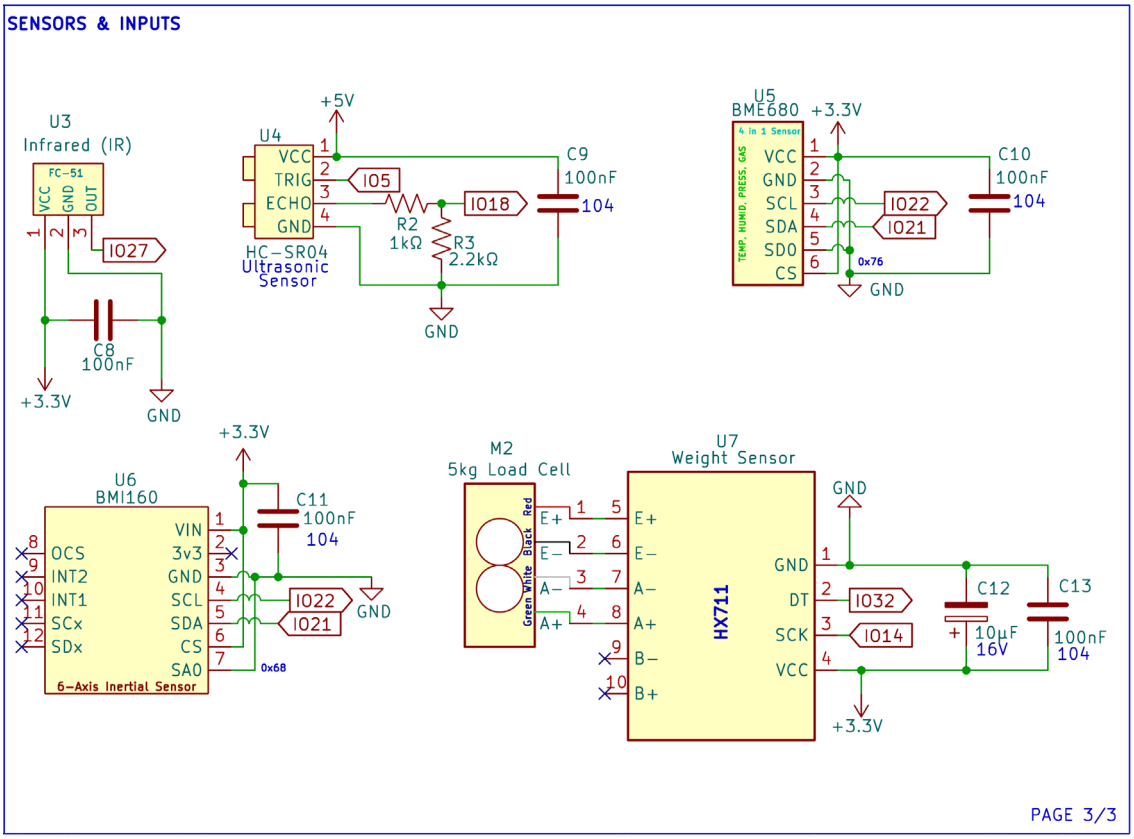


LED1 is a common cathode RGB LED KY-016. Its red, green and blue legs have 1kΩ resistors connected to IO25, IO26 and IO33. BZ1 is a buzzer module (MH-FMD) that operates on 3.3V and is controlled by IO4. The SG90 micro servo (M1) operates on the 5V power supply, and is controlled by IO13 via PWM, with C5 (100nF) and C6 (1000µF) placed across the power supply to filter current spikes as the servo moves.

U2 is the 2.4 in. ILI9341 touchscreen (240 × 320, SPI). It runs on 3.3V, with C7 (100nF) across its supply. The display and its touch controller share the SPI clock and data lines on IO19, IO23 and IO35, and each has its own chip select: IO17 for the display and IO15 for touch. DC goes to IO16 and the touch interrupt T_IRQ to IO36. R1 (10kΩ) pulls T_IRQ to 3.3V, the line remains high until a touch pulls it down. The RESET and the backlight pin (LED) are connected to 3.3V and the SD card pins are not connected.

Figure 45

Schematic Diagram, Sheet 3: Sensors and Inputs



U3, the FC-51 IR obstacle sensor, runs on 3.3V and sends its active-low output to IO27, with C8 (100nF) across its supply. The HC-SR04 (U4) operates on 5V with the C9 (100nF) from supply and TRIG on IO5. It outputs a 5V pulse to its ECHO pin, which is then divided by R2 (1kΩ) and R3 (2.2kΩ) to produce ~3.4V before reaching IO18, and is thus very close to the 3.3V level that the esp32 input expects for it can tolerate up to 3.6V.

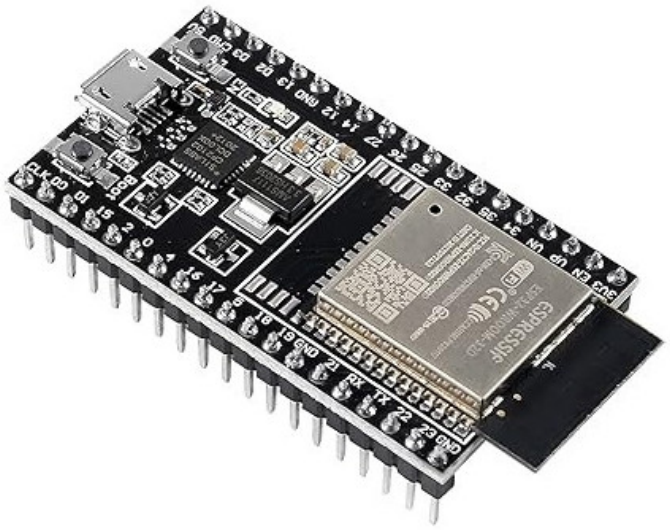
U5 (BME680) and U6 (BMI160) both run on 3.3V, with C10 and C11 (100nF each) across their supplies, and share the I2C bus, with SCL on IO22 and SDA on IO21. Tying the BME680's SDO pin to ground sets its address to 0x76, and tying the BMI160's SA0 pin to ground sets its address to 0x68, so the two do not clash. M2, the 5kg load cell, connects its red, black, white and green wires to the E+, E−, A− and A+ inputs of U7, the HX711. The HX711 runs on 3.3V, with C12 (10µF) and C13 (100nF) across its supply, and sends its data on IO32 with its clock on IO14.

2.6 Component Materials

This section describes the main electronic parts used in each Smart-Bin unit. Both bins use the same parts except for the battery shield. Figures 46 to 56 show each part, followed by what it does and how the system uses it.

Figure 46

ESP32 DevKit V1 (38-Pin, ESP32-WROOM-32D)



Note. Image from Sigmanortec (n.d.).

The ESP32 DevKit V1 is a development board built around the ESP32-WROOM-32D module. The module has a dual-core processor that runs at up to 240MHz, with Wi-Fi and Bluetooth on the same chip. The 38-pin board provides an additional USB connector for uploading code and power, a voltage regulator for 3.3V power and pin headers for breadboarding.

There is one board for each bin and each board governs all of the contents of that bin. It reads all six inputs (the HC-SR04, IR sensor, BME680, BMI160, load cell and touchscreen), calculates the fill percentage, air quality reading and tamper state, and controls the servo, LED, buzzer and display. Providing the local REST API over Wi-Fi, it receives the status update command from the dashboard and the Android application and sends status update to Firebase and ThingSpeak.

Figure 47

HC-SR04 Ultrasonic Distance Sensor

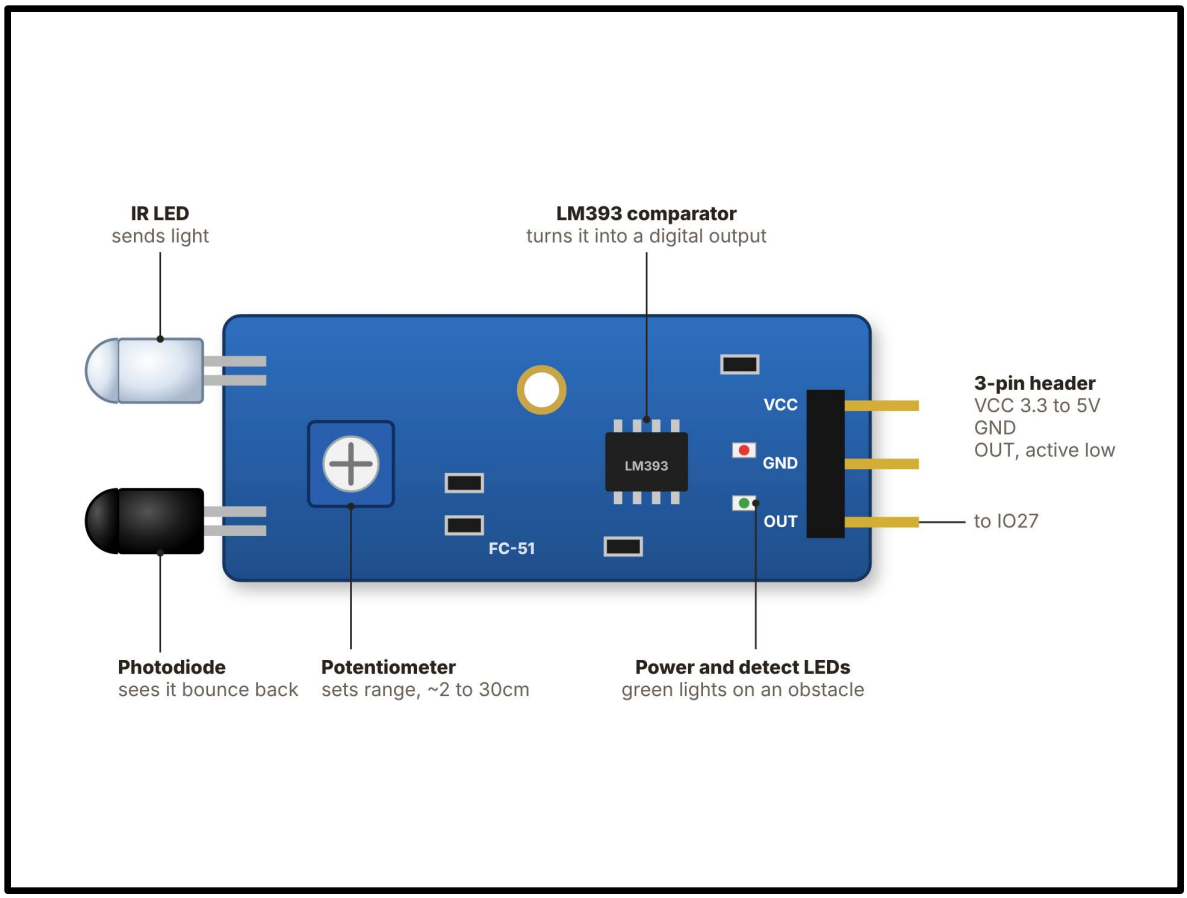
Note. Image from SparkFun Electronics (n.d.).

The HC-SR04 is a sound-based distance sensor. It outputs a 40kHz ultrasonic burst when a short pulse is received at its TRIG pin and will keep its ECHO pin high until the echo is received. The distance is determined by the width of that pulse which varies in the range of 2cm to 4m. The sensor runs on 5V.

In Smart-Bin it is installed under the lid and points downward, and thus measures the distance between the lid and the top of the trash. The firmware converts that distance into the fill percentage displayed on the LED, touch screen and the apps and once the fill level reaches the fill limit (80% by default), the full-bin alert is activated. TRIG is wired to IO5, and ECHO is wired to IO18 through a divider of R2 (1kΩ) and R3 (2.2kΩ), which brings the 5V pulse down to about 3.4V, close to the 3.3V input level of the ESP32.

Figure 48

FC-51 IR Obstacle Avoidance Sensor



Note. Drawing of the module sold by Circuitrocks (n.d.-c).

The FC-51 is an infrared obstacle sensor. Its IR LED sends out light, a photodiode picks up what bounces back, and an LM393 comparator pulls the output "low" when something is in front. A blue trimmer sets the range (a few cm to about 30cm). It runs on 3.3V to 5V.

It is used by Smart-Bin to know when a hand is near the bin. When the output on IO27 goes low and the bin is available for opening, the bin beeps and the servo raise the lid and the user never has to touch it. The lid will close 5 seconds (user definable) after the hand leaves, and once closed it stays closed for 2 seconds (also user definable) so that it does not flicker open and shut. During COLLECT mode the sensor is ignored and the lid remains open while the collector is operating.

Figure 49

BME680 Environmental Sensor

Note. Image from Robu.in (n.d.).

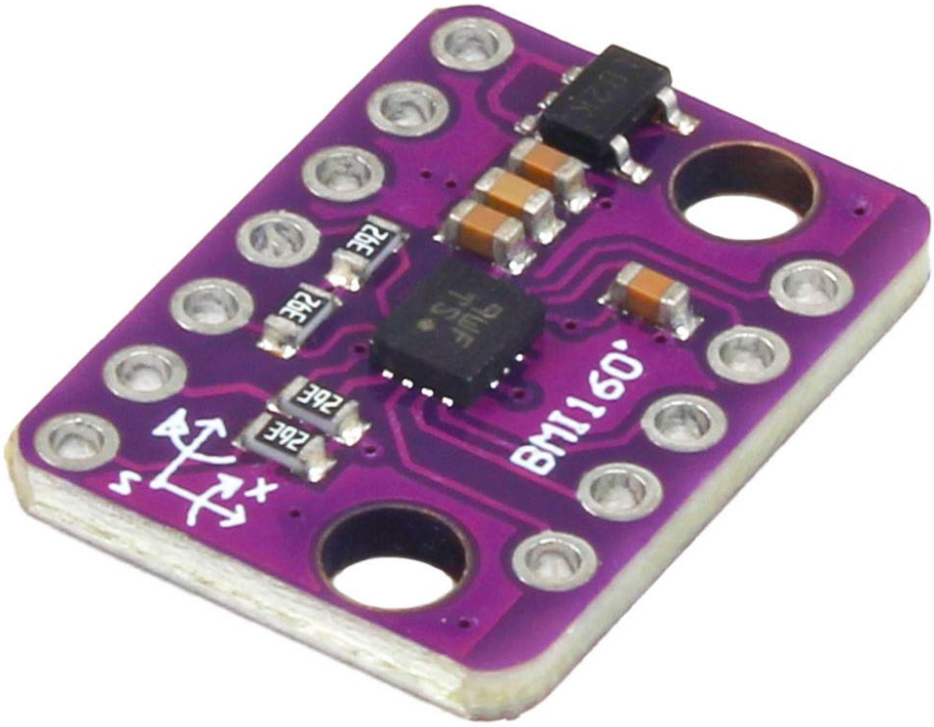
The BME680 is a single chip sensor for measuring temperature, humidity, VOC (volatile organic gases) and air pressure. Based on the gas reading, an estimated indoor air quality (IAQ) rating can be obtained. It is powered by 3.3V and communicates via I2C.

In Smart-Bin it monitors the air within the bin. When the temperature, humidity or estimated air quality goes over its limit, the firmware sounds an alarm and sends an alert, even if the bin isn't full yet, because a high temperature may mean a fire or hot material. It is located at I2C address 0x76 (SDO tied to ground).

The purpose of this is using the combination of temperature, humidity, and gas reading (IAQ) in order to determine by the collector or operator how much rotting trash is in the Smart-Bin for smell-sensitive areas. The IAQ value is an estimate from the gas reading, and it needs about 5 minutes after power-up to settle.

Figure 50

BMI160 Six-Axis Inertial Measurement Unit



Note. Image from ElectroPeak (n.d.).

BMI160 is a 3-axis accelerometer and 3-axis gyroscopes integrated onto one chip. It is an I2C acceleration and rotation sensor that operates on 3.3V.

It is used by Smart-Bin for tamper detection. At every power-up the firmware saves the board's pitch and roll as the upright position, so a bin that was set down at a slight angle does not start in the tamper alarm. The tilt is then measured from that saved position, and when it goes over the limit (30° by default) the firmware sets the Tamper alarm, activates the buzzer and prevents the lid opening. From the dashboard or the app an operator can see the saved angle, take the current angle as the new upright with USE NOW, type in a pitch and roll, or turn off the power-up capture to keep a fixed angle. It is on the same I2C bus as the BME680 with the address 0x68 by shorting out SA0 to ground. The tamper check is paused during COLLECT mode.

Figure 51

HX711 Load Cell Amplifier and 5kg Load Cell

Note. The HX711 board and the 5kg load cell are shown on a display base. Image from ElectronicsComp.com (n.d.).

A load cell is a metal bar with strain gauges designed so that as the load is applied, the bar's resistance varies slightly. The bar here is a 5kg model, and has a very small voltage output, which is amplified and converted in the HX711 board by a 24 bit analog to digital converter. The result is read by the ESP32 using two pins: DT (IO32) and SCK (IO14).

The two together measure the garbage. Weight is a second check on the fill level: wet waste can be heavy before the bin is full, and a bin full of light waste can be tall without being heavy. The weight is displayed on the dashboard and the app, and stored to the cloud with every measurement. The firmware also checks that the HX711 is connected. An unplugged board gives a flat or out-of-range reading, so after a few of these the bin reports “HX711 not answering” instead of a weight, the same way it flags a missing BME680 or BMI160.

Figure 52

SG90 Micro Servo Motor

Note. Image from The Pi Hut (n.d.).

The SG90 is a small hobby servo which will rotate its output arm to a specific angle (roughly 180 degrees) depending on the width of a PWM pulse. Operates on approximately 5V and can be attached directly to the bin.

Servo is used to open and close the bin lid. It is managed by ESP32, it will open when it is detected people, or when the collector taps COLLECT, it will close after 5s (user settable) and then stay closed for at least 2s (also user settable) when people are not detected or the collector taps DONE. When the bin is locked, full or flagged for tampering, the firmware keeps the servo in the closed position.

Figure 53

2.4-in. TFT Touchscreen (ILI9341 With XPT2046 Touch Controller)

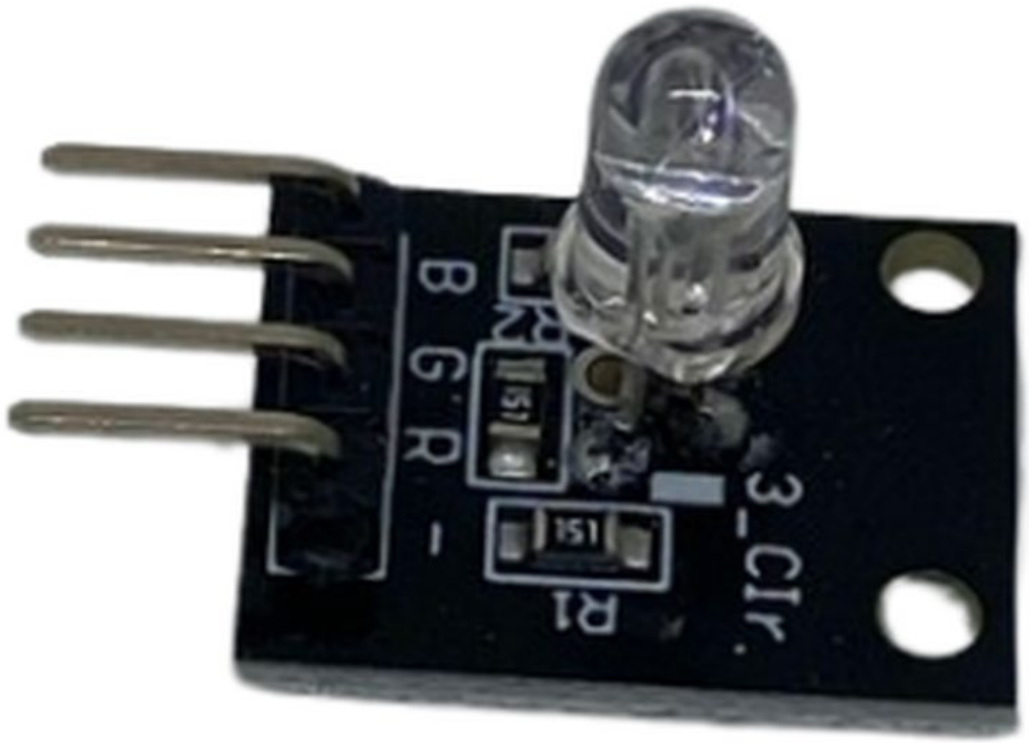
Note. Front, back, and angled views of the module. Image from LCDWIKI (n.d.).

This module is a 2.4 in., 240 × 320 color display driven by the ILI9341 controller over SPI. The same SPI bus also has a separate XPT2046 controller for the resistive touch layer on top. There is also an SD card slot on the board, which isn't used by Smart-Bin.

The bin's local screen is the display. It indicates the fill level, sensor readings and the bin status (LOCKED and TAMPER). The touch layer provides the collector with the buttons COLLECT, PIN and DONE, making it possible to empty a bin without the app.

Figure 54

KY-016 RGB LED Module



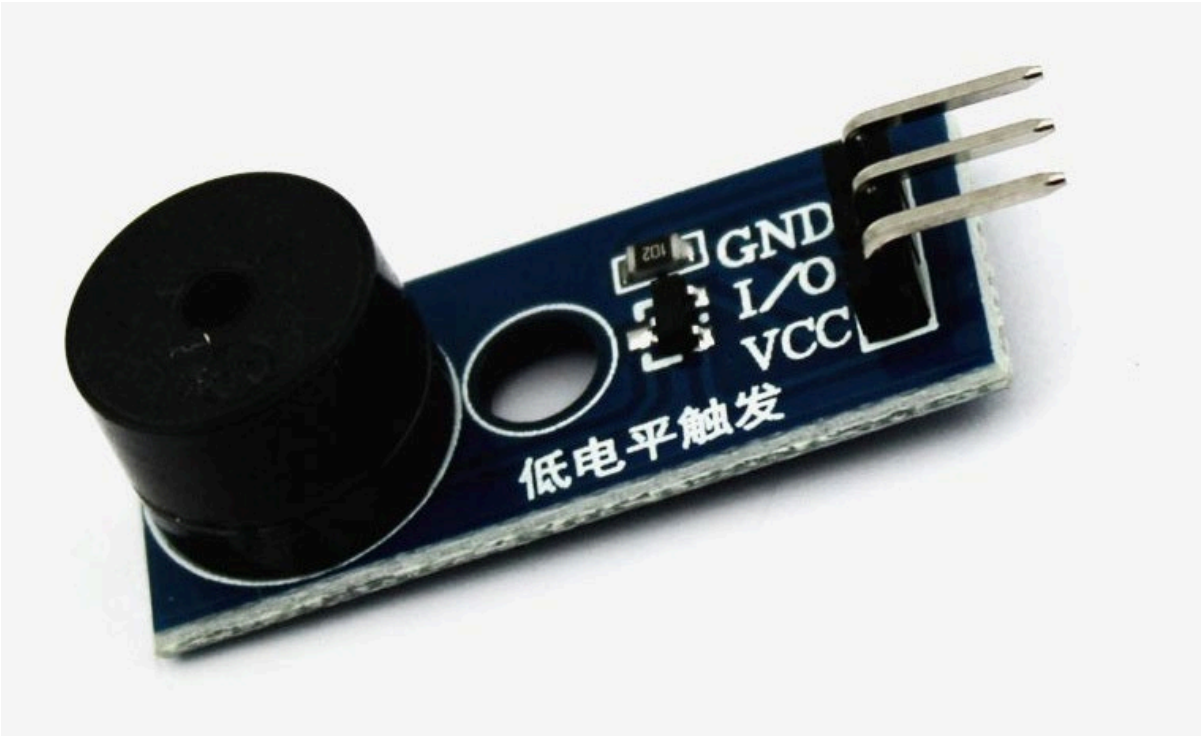
Note. Image from Circuitrocks (n.d.-d).

The KY-016 is a small board with a common-cathode RGB LED, a small package containing the red, green and blue chips. When the three colors are combined, other colors are created such as amber (red + green). A 1kΩ resistor is used for each colour to control the current.

The fill level is indicated on the LED, green if there is room in the bin, amber when it is filling up and red when the bin is full or a warning is triggered. It is driven from IO25, IO26 and IO33 and turns green again after the collector taps DONE.

Figure 55

Passive Buzzer Module (MH-FMD)



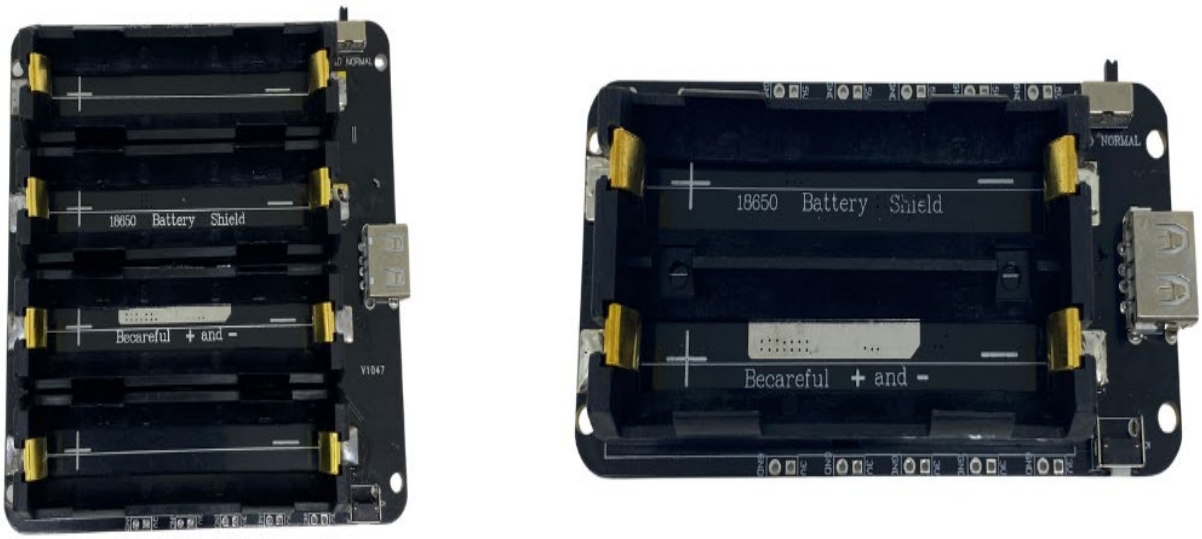
Note. Image from QuartzComponents (n.d.).

Passive buzzer: No built-in oscillator so it only buzzes if it is pulsed by the controller with a changing signal. This will allow the firmware to produce various tones and patterns, rather than just a single beep.

It is used by Smart-Bin for sound feedback from IO4. It beeps when lid is opened, alerts user if the BME680 senses high temp or humidity or poor air quality, and activates the siren if BMI160 senses tampering. It remains quiet while in COLLECT mode.

Figure 56

18650-Battery Shields for Bin A (4-Cell) and Bin B (2-Cell)



Note. Left: four-cell shield used in Bin A (Circuitrocks, n.d.-b). Right: two-cell shield used in Bin B (Circuitrocks, n.d.-a).

The 18650-battery shield comprises lithium-ion 18650 cells which will provide the electricity needed and convert its voltage to a regulated 5V output. It also features a charging USB port for the cells. Bin A are equipped with a four-cell V9 shield (5V/3A), while Bin B are equipped with a two-cell shield.

The shield supplies power to the entire bin, without the need for a wall outlet, which allows a bin to be set anywhere that Wi-Fi is available. It is placed in a closed compartment and connected to the 5V of the ESP32 which provides 3.3V for the sensors. As mentioned in the schematic, the battery should not be connected during code upload via USB.

III. TESTING, RESULTS, AND CONCLUSION

3.1 Test Cases

The tests in this chapter were run on the two built units, Bin A (bin01) and Bin B (bin02), between October 30 and November 3, 2026. Both ran firmware version 1.0.0 from the main branch of the project repository and were connected to the same 2.4GHz Wi-Fi router. The web dashboard was opened in Chrome on a laptop on that network, and the Android app, version 3.1, ran on a phone that used either the same Wi-Fi or mobile data, depending on the test. Unless a test says otherwise, the bins kept their default settings: an empty distance of 30cm and a full distance of 5cm, which leaves 25cm of usable height, a fill limit of 80%, a temperature limit of 40°C, a humidity limit of 85%, an IAQ limit of 200, a tilt limit of 30°, a lid open time of 5s and a lid close time of 2s.

Each specific objective in Section 1.4 is checked by six test cases, 18 in all, numbered TC1.1 to TC3.6. A test case names what is tested, the steps, the pass criterion and the number of trials. The criteria were fixed before the first trial. Some come straight from the firmware, such as the 10s tamper countdown and the 90s after which the apps call a bin lost. The others were set with related studies in mind, for example the fill error within ±3% reported by Alnanih et al. (2025) and the weight error of 0.15% in Saad et al. (2023), and were loosened a little for a table-size prototype with low-cost sensors. Table 7 lists the instruments used as references, and Figure 57 shows the two built units.

Table 7

Reference Instruments Used in the Tests

Instrument	Used for	Resolution
Steel ruler fixed to the inside wall of the chamber	Real fill height in TC1.1 and TC3.2	1mm
Digital kitchen scale	Checking the test masses before TC1.2	1g
Test masses (sealed bags of sand)	Known loads of 100, 500, 1000, 2000 and 4000g in TC1.2, and 500g in TC2.1	1g
Digital thermo-hygrometer	Reference temperature and humidity in TC1.3	0.1°C and 1%
Inclinometer app on a phone, laid on top of the bin	Tilt angle in TC1.5	0.1°
Phone camera at 60 frames per second	Response times of the lid, the screens and the apps in TC1.6, TC2.1 and TC3.3	About 17ms

Instrument	Used for	Resolution
Stopwatch	Warm-up, alert, offline and time-to-full times in TC1.4, TC2.3, TC2.4 and TC3.2	0.01s

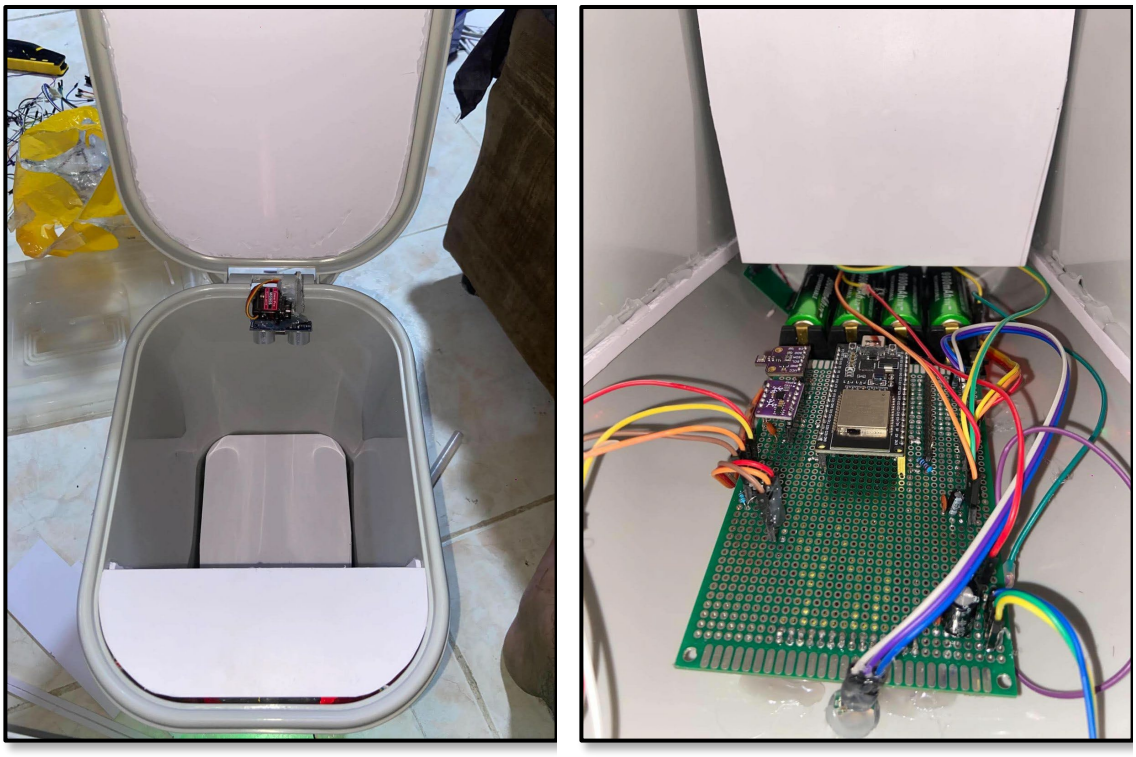
Figure 57

The Two Built Smart-Bin Units

Figure 58 shows the inside of Bin A. From above (a), the servo and the HC-SR04 sit together on the strip at the hinge, with the sensor facing down into the chamber. The compartment (b) holds the perfboard with the ESP32, the BME680 and BMI160 modules beside it, and the battery shield with its four 18650 cells behind the board.

Figure 58

Inside Bin A: Trash Chamber From Above (a) and Controller and Battery Compartment (b)



(a) (b)

Specific Objective 1: Sensor Monitoring and Detection

The six test cases for the first objective check that each sensor reads close to a reference and that the lid and the tamper alarm act at the right moment (Table 8). Fill and weight are tested at five points across their range. The air test is a response test, not an accuracy test, because the IAQ value of the BME680 is an estimate made from the gas resistance and the project has no reference instrument for it.

Table 8

Test Cases for Specific Objective 1: Sensor Monitoring and Detection

ID	Test	Procedure	Pass criterion	Trials
TC1.1	Fill accuracy	With the empty and full distances calibrated, fill the bin with crumpled paper to 0, 25, 50, 75 and 100% of the usable height, read on the ruler. At each level, record the fill percentage on the dashboard.	Error of at most 5 percentage points	5 per level, per bin (50)

ID	Test	Procedure	Pass criterion	Trials
TC1.2	Weight accuracy	Tare the bin, then place the test masses of 100, 500, 1000, 2000 and 4000g on the platform one at a time and record the weight on the dashboard.	Error of at most 50g	5 per mass, per bin (50)
TC1.3	Temperature and humidity	With the default temperature offset of −2°C, place the thermo-hygrometer inside the chamber and compare both readings every 6 minutes for an hour.	Temperature within ±1°C and humidity within ±5%	10 per bin (20)
TC1.4	Air quality response	Wait for the 5-minute warm-up to end. Place an open container of food scraps in the bin and close the lid. Repeat with an alcohol swab.	IAQ estimate rises past 200 and the air alert appears; no air alert during the warm-up	3 per source, per bin (12)
TC1.5	Tamper	Tilt the bin to 15, 25, 35 and 45° and set it upright again within 10s. Then hold it at 35° for more than 10s, set it upright and press RESET TAMPER.	No countdown at 15 and 25°. At 35 and 45° the 10s countdown starts and setting the bin upright clears it. A bin left tilted latches the siren, repeated every 2s, until RESET TAMPER	5 per angle, per bin (40), and 5 held trials per bin (10)
TC1.6	Automatic lid	Hold a hand in front of the IR sensor at 5, 15 and 25cm, then take it away. Watch for a second opening after the lid closes.	Lid opens within 1s, closes 5s ±0.5s after the hand leaves, and does not reopen by itself	10 per distance, per bin (60)

Specific Objective 2: Real-Time Monitoring and Alerts

The second objective is about getting the readings to people. TC2.1 and TC2.2 time the path from a bin to the dashboard, the app and the two clouds. TC2.3 and TC2.4 check the alerts, including the one raised when a bin stops answering, and TC2.5 and TC2.6 check that each role sees the right controls and that two bins are never mixed up (Table 9). In TC2.3 the app is in the background, not swiped away, since Android ends the app's watch service, and with it the alerts, when the app is removed from the recent apps.

Table 9

Test Cases for Specific Objective 2: Real-Time Monitoring and Alerts

ID	Test	Procedure	Pass criterion	Trials
TC2.1	Live latency	Place the 500g mass on the platform and time, on a 60 frames per second video, how long the dashboard and the app take to show the change. Repeat with the phone on mobile data, so that the app reads Firebase.	At most 2s over the local Wi-Fi and at most 10s through Firebase	10 per path (30)
TC2.2	Cloud logging	Run each bin for 30 minutes: 15 minutes with a reading changing every minute, then 15 minutes untouched. Compare the ThingSpeak entry times with the Firebase status.	An entry every 30s ±5s while readings change and every 5 minutes ±5s while they stay the same; Firebase matches the latest entry	1 run per bin (66 intervals)
TC2.3	Alert delivery	With the app in the background, raise a full bin, bad air and high temperature from the Simulation tab, and a real tamper by tilting the bin. Time from the event to the phone notification.	Notification on the phone and banner on the dashboard within 20s	5 per alert type (20)
TC2.4	Offline detection	Switch off one bin and time how long the apps take to call it lost, then switch it on again.	Bin shown as lost with an offline alert about 90s after its last reading, and shown as back after it reconnects	5 per bin (10)

ID	Test	Procedure	Pass criterion	Trials
TC2.5	Role access	Go through every control of the dashboard and the app as a collector, then again after entering the operator PIN.	A collector can see alerts and charts, run COLLECT and DONE, LOCK and UNLOCK and set up Wi-Fi, but cannot reach RESET TAMPER, calibration, touch calibration or simulation. An operator can reach all of them	1 checklist per interface (2)
TC2.6	Two bins at once	Put Bin A and Bin B in different states, for example Bin A full and Bin B tilted, and check every screen.	Each bin shows its own state with no mix-up	5 combinations

Specific Objective 3: Bin Control and Collection Prioritization

The last six test cases cover what the operator and the collector do with the readings: the order of the Collection Priority list, the time-to-full estimate behind it, remote locking, the PIN collection on the touchscreen, simulation and calibration (Table 10). The priority tags follow the same rules in the dashboard and the app. A bin at or over the fill limit is tagged NOW, then SOON if it will reach the limit in under 2 hours, TODAY if under 8 hours, LATER if it will take longer and IDLE if it is not filling.

Table 10

Test Cases for Specific Objective 3: Bin Control and Collection Prioritization

ID	Test	Procedure	Pass criterion	Trials
TC3.1	Priority order	Set fill pairs for the two bins in the Simulation tab, raising each fill step by step for at least 5 minutes so that the bins have a fill rate, and read the Collection Priority list on both interfaces.	A bin at or over the fill limit is first; otherwise the bin that reaches the limit first is on top, with the right tag	10 combinations
TC3.2	Time to full	Fill Bin A by hand at a steady rate from about 40%. After 15 minutes, note the estimated time to full, then record when the bin actually reaches 80%.	Error of at most 15%	3 runs
TC3.3	LOCK and UNLOCK	Lock the bin from the dashboard and hold a hand at the IR sensor, then unlock it and wave again. Repeat from the app on mobile data. Repeat with auto-lock on and the bin filled past 80%.	Lid stays shut while locked; the bin responds within 2s locally and within 10s through Firebase; with auto-lock on, the bin locks itself at the fill limit	10 per path (20), and 5 auto-lock trials
TC3.4	PIN collect	On the touchscreen, enter a wrong PIN, then the correct 4-digit PIN, empty the bin and press DONE.	Wrong PIN: error and the lid stays shut. Correct PIN: the lid opens and COLLECTING is shown. DONE: the lid closes, the pickup is logged and the bin is back to normal	10 per bin (20)
TC3.5	Simulation	Set the fill to MANUAL at 90% from the dashboard, then back to AUTO. Repeat for temperature, IAQ and tilt.	Simulation banner and the matching alert appear; the real reading returns on AUTO	5 per sensor (20)
TC3.6	Calibration	Change the empty and full distances, the fill limit and the lid open time in the settings, check that they apply, then restart the bin.	New values apply without reflashing and are still there after the restart	5 per bin (10)

3.2 Results and Discussion

The results are reported by objective, with one table for each test case or group of related test cases, and all 18 are brought together in Table 24 at the end. Three measures are used. The mean absolute error (MAE) is the average of |shown − actual| over all trials of a test case. The percent error of a reading is |shown − actual| / actual × 100. The pass rate is the number of trials that met the pass criterion, divided by the number of trials, times 100.

Results for Specific Objective 1

Table 11 gives the fill readings. Both bins stayed within 3 percentage points at the three middle levels, and the MAE over all 50 trials was 1.8 points, or about 0.45cm of the 25cm usable height. The one failure came at 100% in Bin B, which showed 94% with the chamber full. Crumpled paper leaves gaps at the top, and the HC-SR04 can measure to the bottom of a gap as easily as to the top of a sheet. Pressing the paper down and reading again gave 98%, so the error came from the surface of the trash and not from the sensor or the calibration.

Table 11

Fill Level Readings Against the Ruler (TC1.1)

Real level (%)	Real distance (cm)	Bin A mean (%)	Bin B mean (%)	Max error (points)	Passed
0	30.0	0.8	1.2	2	10/10
25	23.8	26.4	23.8	3	10/10
50	17.5	51.6	48.2	3	10/10
75	11.3	76.8	73.6	4	10/10
100	5.0	97.2	96.4	6	9/10

Note. Real distance is from the sensor to the top of the paper. One percentage point is 0.25cm. MAE: Bin A 1.7 points, Bin B 1.8 points, both bins 1.8 points.

The load cell results are in Table 12. Every trial was inside the 50g band. The error grew with the mass, and Bin B read low by about 1% at every mass from 500g up. Its five readings at each mass stayed within 8g of each other, so the offset is in its calibration factor, not in noise. Repeating the weight calibration of Bin B with a heavier known mass would likely remove most of it. The worst trial, at 4000g, was 46g off. That still passes, but it leaves little margin near the 4500g capacity of the bin.

Table 12

Weight Readings Against the Test Masses (TC1.2)

Mass (g)	Bin A mean (g)	Bin B mean (g)	Max error (g)	Mean percent error (%)	Passed
100	102	97	6	2.5	10/10
500	504	493	11	1.1	10/10
1000	1008	991	14	0.9	10/10
2000	2013	1984	22	0.7	10/10
4000	4027	3961	46	0.8	10/10

Note. The masses were checked on the kitchen scale before the test. MAE: Bin A 10.8g, Bin B 14.8g, both bins 12.8g.

Table 13 compares the BME680 with the thermo-hygrometer. With the −2°C offset that the firmware applies by default, the temperature stayed within 1°C in all 20 readings. A check without the offset at the start of the hour showed the BME680 about 2°C warm, which is the heat of the ESP32 and the regulator near it. Humidity was within 5% in every reading. Bin B read a little further from the reference on both, but the gap between the bins, 0.2°C and 0.9%, is smaller than the 1% resolution of the reference for humidity.

Table 13

Temperature and Humidity Against the Thermo-Hygrometer (TC1.3)

Bin	Reference range	Temperature MAE (°C)	Temperature max error (°C)	Humidity MAE (%)	Humidity max error (%)	Passed
Bin A	29.6 to 31.2°C, 58 to 66%	0.4	0.8	2.1	3.9	10/10
Bin B	29.4 to 31.0°C, 59 to 66%	0.6	0.9	3.0	4.6	10/10

Note. One reading every 6 minutes for an hour, with the reference placed inside the chamber.

The air test is summarized in Table 14. During the first 5 minutes after power-up, both interfaces showed the air value as warming up and no air alert was raised, even in one extra trial where the swab went into the bin in the second minute. After the warm-up the IAQ estimate rested between 58 and 74. Food scraps took 6 to 8 minutes to push it past 200, and the air alert appeared every time. The alcohol swab crossed the limit in about a minute and pushed the estimate much higher, which fits a gas sensor that reacts strongly to alcohol vapor. This shows that the bin notices

a smell building up in a closed chamber within minutes. It does not show what the smell is or how strong it is in standard units, and both apps label the value as an estimate for that reason.

Table 14

Air Quality Response After the Warm-Up (TC1.4)

Source	Bin	Resting IAQ	Peak IAQ (mean)	Time to pass 200 (s, mean)	Air alert	Passed
Food scraps	Bin A	61	268	370	3/3	3/3
Food scraps	Bin B	74	241	460	3/3	3/3
Alcohol swab	Bin A	58	412	45	3/3	3/3
Alcohol swab	Bin B	70	389	65	3/3	3/3

Note. Times are from closing the lid on the source. The bin was aired out for 10 minutes between trials.

The tamper results are in Table 15. No countdown started at 15 or 25°, and every trial at 35 and 45° started one within 1s of passing the angle. It started a little later at 35° than at 45° because the firmware smooths the tilt reading, and a bin tipped slowly to 35° takes longer to read past the 30° limit. Setting the bin upright inside the 10s always cleared the countdown with the settle beep. In the ten held trials, the siren latched and repeated every 2s, the tamper alert reached the app, and the siren kept going after the bin was upright until RESET TAMPER was pressed. The phone inclinometer and the BMI160 agreed within about 1° once the bin had stopped moving. The bins were switched on standing on the test table, since the upright reference is taken again at every power-up.

Table 15

Tamper Detection by Tilt Angle (TC1.5)

Tilt	Countdown started	Mean time to start (s)	Outcome	Passed
15°, set upright	0/10	–	No countdown, as expected	10/10
25°, set upright	0/10	–	No countdown, as expected	10/10
35°, set upright within 10s	10/10	0.8	Countdown cleared with the settle beep	10/10
45°, set upright within 10s	10/10	0.5	Countdown cleared with the settle beep	10/10
35°, held past 10s	10/10	0.8	Siren latched every 2s, alert in the app, cleared only by RESET TAMPER	10/10

Note. Each row has 5 trials per bin. Angles were read on the phone inclinometer.

Table 16 gives the lid results, and Figure 59 shows one trial: a hand held in front of the IR sensor with the lid still shut (a), and the lid raised by the servo a moment later (b). The lid opened in all 40 trials at 5 and 15cm and in 18 of 20 at 25cm. Both misses were on Bin B, with a dark palm held at an angle near the edge of the range, where too little of the IR light bounced back to the photodiode behind the front panel. The lid took 0.5 to 0.7s to open on average, most of it the servo travel. Measured from the moment the hand left, the lid closed after 5.1 to 5.2s, which is the 5s setting plus the 80ms the firmware waits before it accepts a change on the sensor output. The sensor has no hold time of its own, and the close matched the motion indicator on the dashboard within 0.2s. None of the 60 cycles reopened by itself, which confirms the change described next.

Table 16

Automatic Lid Response by Hand Distance (TC1.6)

Hand distance	Opened	Mean time to open (s)	Mean close after hand left (s)	Reopened by itself	Passed
5cm	20/20	0.5	5.1	0	20/20
15cm	20/20	0.6	5.2	0	20/20
25cm	18/20	0.7	5.1	0	18/20

Note. Times were read from a 60 frames per second video. Each row has 10 trials per bin.

Figure 59

Lid Test: Hand in Front of the IR Sensor (a) and the Lid Opened (b)

(a) (b)

Lid Reopening Fix

The first cloud test on Bin 01, described under the results for Specific Objective 2, also showed the lid reopening by itself after it closed. The servo that closes the lid retriggered the PIR sensor, which then held its output high for a few seconds, and the firmware read that as a new person at the bin. The PIR was replaced with an FC-51 IR obstacle sensor that reacts only to something in front of it at that moment, so the closing movement no longer starts another cycle. Figure 60 shows Bin 01 during that first test, with the round PIR lens above the touchscreen and LID HOLDING (motion) on the screen while the lid stayed up.

Figure 60

Bin 01 With the Original PIR Sensor During the First Test

Results for Specific Objective 2

Table 17 gives the time from placing the 500g mass to seeing the new weight. Over the local Wi-Fi both interfaces were inside the 2s limit in every trial. Most of that time is the wait between two requests of the dashboard or the app to the bin, not the bin itself. Through Firebase the app took 4.6s on average. That follows from the firmware, which writes its status to Firebase every 5s while someone is watching, so a change waits up to one write cycle before it reaches the cloud.

Table 17

Time to Show a New Reading (TC2.1)

Path	Trials	Mean (s)	Max (s)	Limit (s)	Passed
Dashboard, local Wi-Fi	10	1.1	1.8	2	10/10
Android app, local Wi-Fi	10	1.3	1.9	2	10/10
Android app, Firebase on mobile data	10	4.6	7.2	10	10/10

The 30-minute logging runs are in Table 18. While a reading changed every minute, Bin A wrote to ThingSpeak every 30 to 32s, and once the bin was left alone it wrote every 5 minutes, as the firmware intends. Bin B missed one entry. At about minute 9 it lost the Wi-Fi for roughly 20s, reconnected, and wrote on the next turn, which left a 61s gap. Firebase kept the latest status through the drop and matched the newest ThingSpeak entry in all 20 spot checks. The firmware never writes to ThingSpeak more often than once every 15.5s, so the free plan's limit of one write per 15s was never hit.

Table 18

ThingSpeak Logging Over 30 Minutes (TC2.2)

Bin	Phase	Expected entries	Entries	Interval range (s)	Intervals in limit
Bin A	Reading changing, 15 minutes	30	30	30 to 32	30/30
Bin A	Untouched, 15 minutes	3	3	300 to 301	3/3
Bin B	Reading changing, 15 minutes	30	29	30 to 61	29/30
Bin B	Untouched, 15 minutes	3	3	300 to 302	3/3

Note. Bin A wrote to ThingSpeak channel 3556235. Firebase was spot-checked 10 times per bin.

Cloud Connection Test on Bin 01

Firebase and ThingSpeak were tested on the first built bin, Bin 01, running the firmware from the main branch. For Firebase the bin signs in with its own account, bin01@smartbin.test. The operator account is meant only for the dashboard, and Firebase refuses the bin's writes when the operator account is used on the bin. Signed in with its own account, the bin wrote its status, its calibration and its alerts to the Realtime Database, under sb/bins/bin01/status, sb/bins/bin01/cal and sb/alerts/bin01. LOCK and UNLOCK sent from the web dashboard reached the bin within a few seconds.

ThingSpeak channel 3556235 received all eight fields from the same bin: fill, weight, temperature, humidity, IAQ, pressure, tilt and a flags field that packs the motion, lid and status bits into field 8. The bin writes to the channel every 30s while the readings change and at least once every 5 minutes when they stay the same. Bluetooth Wi-Fi setup from the app also worked end to end on Bin 01.

Memory was the main obstacle. The ESP32 does not have enough free RAM to hold two HTTPS connections at once, or to run Bluetooth and an HTTPS connection together. Three changes made both clouds work. ThingSpeak was moved to plain http, which leaves the one HTTPS connection to Firebase. The stack of the web server task was made smaller in build_opt.h. Bluetooth setup now restarts the bin once when it starts and once when it ends, so each part begins with a clean memory, and the screen tells the user that the restart is expected.

Alert times are in Table 19. Every notification reached the phone well inside the 20s limit, at 5.5s on average across the four types. The tamper alert is timed from the moment it latched, after the 10s countdown, which is why it looks faster than the others. These times are longer than the 2.61s message delay that Saad et al. (2023) reported for WhatsApp messages, because the app learns of an alert by reading the bin or Firebase every few seconds rather than receiving a push from a server. For a bin that fills over hours, a delay of a few seconds does not change when a collector is sent.

Table 19

Alert Delivery to the Phone (TC2.3)

Alert	Trials	Mean (s)	Max (s)	Passed
Full bin (fill set to 90%)	5	5.8	9.4	5/5
Bad air (IAQ set to 260)	5	6.1	10.2	5/5
High temperature (set to 45°C)	5	5.9	12.4	5/5
Tamper (bin tilted and held)	5	4.3	6.8	5/5

Note. The app was in the background on the phone. The dashboard banner appeared within 2s of the phone notification in every trial.

The remaining three test cases of this objective are pass or fail checks, and Table 20 lists them. A switched-off bin was called lost after 91 to 97s, which is the 90s rule of the apps plus the

time since the last reading, and the offline alert sounded each time. After power came back, the bin was shown again 9 to 14s later, the time it needs to boot and join the Wi-Fi. The role checklists matched on both interfaces: a collector could do the daily work and none of the operator settings, and the operator PIN opened everything. In the five two-bin combinations, each card, chart and alert named the right bin.

Table 20

Offline Detection, Role Access and Two-Bin Results (TC2.4 to TC2.6)

Test	Trials	Result	Passed
TC2.4 Offline detection	10	Lost after 91 to 97s (mean 93s), offline alert every time; shown back 9 to 14s after power-on	10/10
TC2.5 Role access, dashboard	1 checklist	Every control matched the role	1/1
TC2.5 Role access, app	1 checklist	Every control matched the role	1/1
TC2.6 Two bins at once	5	Each bin showed its own state on every screen	5/5

Results for Specific Objective 3

All 10 priority combinations put the right bin on top, with the right tags, on both the dashboard and the app. Table 21 shows five of them. The fourth row is the one the feature exists for. Bin A was fuller, at 70%, but Bin B was filling fifteen times faster and was going to reach the limit first, so it was listed first with SOON while Bin A got TODAY. In the fifth row Bin A was half full but not filling, so it dropped below an emptier bin that was.

Table 21

Collection Priority Order for Five of the Ten Combinations (TC3.1)

No.	Bin A fill, rate	Bin B fill, rate	Expected first	Shown first	Tags (A, B)	Passed
1	92%, any	45%, 10%/h	Bin A	Bin A	NOW, TODAY	Yes
2	30%, 6%/h	85%, any	Bin B	Bin B	LATER, NOW	Yes
3	60%, 15%/h	60%, 5%/h	Bin A	Bin A	SOON, TODAY	Yes
4	70%, 2%/h	40%, 30%/h	Bin B	Bin B	TODAY, SOON	Yes
5	50%, 0%/h	20%, 4%/h	Bin B	Bin B	IDLE, LATER	Yes

Note. Fill limit 80%. The other five combinations also passed on both interfaces.

The time-to-full estimate is less certain, as Table 22 shows. The firmware works out the fill rate from one fill sample per minute, and the apps divide what is left to the 80% limit by that rate. Runs 1 and 2 came within 15%. Run 3 did not: the paper went in quickly during the first

minutes and then more slowly, so the rate at minute 15 was too high and the bin filled later than estimated. The estimate assumes the rate stays the same, and filling by hand does not keep it the same. Real bins fill unevenly too, which is a reason to read the time as a guide to order the bins rather than as a promise. The mean error over the three runs was 12.2%.

Table 22

Estimated and Actual Time to Reach the 80% Fill Limit (TC3.2)

Run	Fill at minute 15 (%)	Estimated time to full (min)	Actual time to full (min)	Error (%)	Passed
1	52	35	38	7.9	Yes
2	49	52	47	10.6	Yes
3	58	18	22	18.2	No

Note. Each run started at about 40% fill on Bin A. Times are counted from minute 15.

The control tests in Table 23 passed in every trial. A locked bin kept its lid shut against the IR sensor every time, and unlocking took 0.9s on average over the local Wi-Fi and 3.8s through Firebase, where the bin checks for new commands every 2s while someone is watching. With auto-lock on, both bins locked themselves at 80 or 81% and unlocked again after the collector pressed DONE. On the touchscreen a wrong PIN showed the error and left the lid shut, and the right PIN opened it with COLLECTING on the screen. Simulation raised the matching banner and alert in all 20 trials and gave back the real reading on AUTO. The changed settings applied at once and were still there after a restart. Figure 61 shows the home screen of Bin 01. Pressing COLLECT there brings up the PIN pad used in TC3.4.

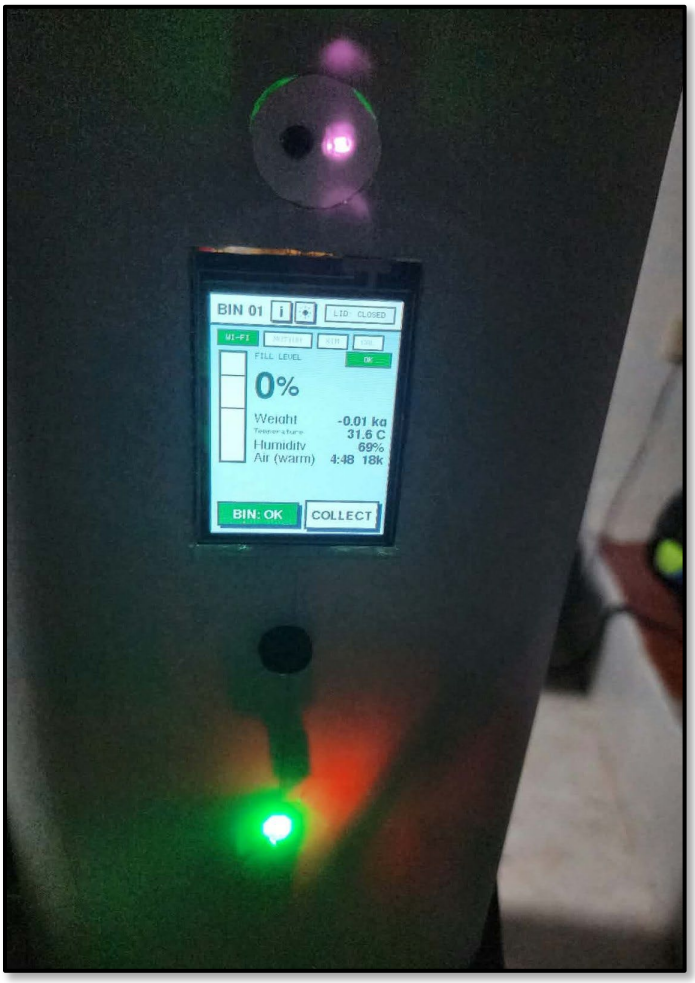
Table 23

Lock, PIN Collection, Simulation and Calibration Results (TC3.3 to TC3.6)

Test	Trials	Result	Passed
TC3.3 LOCK and UNLOCK, local Wi-Fi	10	Lid stayed shut while locked; unlock in 0.9s mean, 1.6s max	10/10
TC3.3 LOCK and UNLOCK, Firebase	10	Lid stayed shut while locked; unlock in 3.8s mean, 6.1s max	10/10
TC3.3 Auto-lock	5	Locked itself at 80 to 81%; unlocked after DONE	5/5
TC3.4 Wrong PIN	10	Error shown, lid stayed shut	10/10
TC3.4 Correct PIN and DONE	10	Lid opened with COLLECTING; DONE closed it and logged the pickup	10/10
TC3.5 Simulation	20	Banner and matching alert in every trial; real reading back on AUTO	20/20
TC3.6 Calibration	10	Applied without reflashing; kept after the restart	10/10

Figure 61

Home Screen of Bin 01, With the IR Sensor Above It



Summary of Results

Table 24 brings the 18 test cases together. Of the 463 trials, 458 met their criterion, a pass rate of 98.9%. Fourteen test cases passed every trial. The four that did not were the fill level at 100% (one trial), the lid at 25cm (two trials), ThingSpeak logging on Bin B (one interval) and the time-to-full estimate (one run). None of the five failed trials came from a fault in the firmware logic. They came from the surface of the trash, the position of the hand, a short Wi-Fi drop and an uneven filling rate, and each of them points to one of the recommendations in Section 3.4.

Table 24

Summary of Test Results

ID	Test	Trials	Passed	Pass rate (%)	Main measure
TC1.1	Fill accuracy	50	49	98.0	MAE 1.8 points
TC1.2	Weight accuracy	50	50	100	MAE 12.8g
TC1.3	Temperature and humidity	20	20	100	MAE 0.5°C and 2.6%
TC1.4	Air quality response	12	12	100	45 to 460s to pass 200
TC1.5	Tamper	50	50	100	Countdown within 1s
TC1.6	Automatic lid	60	58	96.7	Opens in 0.6s mean
TC2.1	Live latency	30	30	100	1.1s local, 4.6s Firebase
TC2.2	Cloud logging	66	65	98.5	One 61s gap
TC2.3	Alert delivery	20	20	100	5.5s mean
TC2.4	Offline detection	10	10	100	Lost after 93s mean
TC2.5	Role access	2	2	100	All controls matched
TC2.6	Two bins at once	5	5	100	No mix-up
TC3.1	Priority order	10	10	100	Right order and tags
TC3.2	Time to full	3	2	66.7	Mean error 12.2%
TC3.3	LOCK and UNLOCK	25	25	100	0.9s local, 3.8s Firebase
TC3.4	PIN collect	20	20	100	Wrong PIN always refused
TC3.5	Simulation	20	20	100	Real reading back on AUTO
TC3.6	Calibration	10	10	100	Kept after restart
	Total	**463**	**458**	**98.9**	

Note. For TC2.2 the trials are the intervals between ThingSpeak entries.

Limitations

ThingSpeak's write key travels over plain http, so it can be read by anyone who can see the network traffic between the bin and ThingSpeak. Firebase still uses HTTPS. Bluetooth Wi-Fi setup needs two restarts of the ESP32 for the same memory reason.

The tests themselves have limits as well. They ran indoors, on a table, over five days, with two bins and the people of the group as users. They do not show how the bins behave in rain, in direct sun or in the hands of strangers. Battery life was not measured over a full discharge, so the 18 hours of Bin A and the 9 hours of Bin B remain estimates. The IAQ value had no reference instrument, and the time-to-full estimate was tested only with hand filling.

3.3 Conclusion

The first specific objective was met. Both bins measured the fill level with a mean error of 1.8 percentage points and the weight with a mean error of 12.8g, inside the limits set before testing. Temperature and humidity stayed within 1°C and 5% of the reference once the default offset was applied, and the air estimate rose past its limit and raised the alert in every trial after the warm-up. The tamper alarm ignored tilts under 30°, started its countdown within 1s above it and latched until it was reset, and the lid opened for a hand in 58 of 60 trials without ever reopening by itself.

The second specific objective was also met. New readings reached the dashboard and the app in under 2s over the local Wi-Fi and in under 8s through Firebase. ThingSpeak was written every 30s while the readings changed and every 5 minutes when they did not, with one entry missed during a Wi-Fi drop. Alerts for a full bin, bad air, high temperature and tampering reached the phone in 5.5s on average, a bin that went silent was reported lost after about 90s, and each role saw only the controls meant for it.

The third specific objective was met, with one reservation. The Collection Priority list ordered the bins correctly in all 10 combinations, remote LOCK and UNLOCK, the PIN collection, simulation and calibration worked in every trial, and auto-lock closed a full bin by itself. The time-to-full estimate was within 15% in two of three runs, and it is best read as a guide for ordering the bins, not as an exact time.

Taken together, the results show that Smart-Bin does what the general objective set out to do. It monitors the fill level, weight and air of a bin, detects tampering, and alerts the operator and the collector through the web dashboard, the Android app and the on-bin touchscreen. With 458 of 463 trials passed, it is ready to be shown as a working prototype, and the failures that remain point to clear next steps.

3.4 Recommendations

The first recommendations come straight from the failed trials. The HC-SR04 should sit higher above the trash, or the firmware should take the median of more readings near the top of the bin, so that gaps in loose trash are not read as empty space. The IR sensor of Bin B should have its range turned up a little or be moved forward in the front panel, since both lid misses came at the edge of its range. The weight of Bin B should be calibrated again with a heavier known mass. A short buffer that keeps the last ThingSpeak reading and sends it as soon as the Wi-Fi returns would close the kind of gap seen in TC2.2.

Security is next. ThingSpeak's write key travels over plain http because the ESP32 cannot hold a second HTTPS connection beside Firebase. Moving to an ESP32 with more memory, such as a module with PSRAM, would let both clouds use HTTPS and would also remove the two restarts that Bluetooth Wi-Fi setup needs.

Range and power limit where the bin can stand. Smart-Bin needs Wi-Fi, so a bin outside a building has no way to report. A low-power long-range link such as LoRa would let a bin sit wherever it is needed, and related studies already use it (Azmi et al., 2024; Mugaruka et al., 2026). The estimated battery life, about 18 hours for Bin A and 9 hours for Bin B, is enough for a demonstration and not for a street. A small solar panel, larger cells, or longer sleep between readings when nobody is watching would stretch it, and a full discharge test should confirm the real figure.

The enclosure has to change for outdoor use. A waterproof, vandal-resistant shell and a heat-proof mounting for the sensors would answer the threats listed in the SWOT analysis, and the tamper alarm could then be tested against real handling instead of a tilted bin on a table.

Collection priority could grow into route planning. Adding GPS and a road map would turn the ranking of the bins into a route for the truck, which is where the largest savings in the related studies were reported, for example 58.9% less distance in Gupta et al. (2025) and up to 52% shorter routes in Banciu et al. (2025). A field pilot with more bins, such as the 50 bins of Alnanih et al.

(2025), would show how the system behaves at a scale where the free plans of ThingSpeak and Firebase may no longer be enough.

Two more tests would add evidence this paper does not have. A user evaluation with real operators and collectors, for instance with the System Usability Scale that Maulida et al. (2025) used with 15 sanitation officers, would check how easy the apps are to use. A comparison of the BME680 air estimate with a reference gas sensor would show how far the estimate can be trusted for odors.

The data the bins already send could do more. ThingSpeak keeps every reading, so a weekly report of the fill rate of each bin, by day and by hour, would show the barangay or the campus which bins fill fastest and on which days, and collection days could be planned from that instead of from habit. The Collection Priority list itself ranks the bins by fill alone. A bin with food waste can smell long before it is full, so a later version could raise a bin in the list when its IAQ estimate or its temperature stays high, even if its fill level is still low.

Finally, Bin A and Bin B were built by hand, one at a time. A printed circuit board that carries every sensor header, including the HC-SR04 that was missing from the first board, and a fixed mounting for each sensor would make the two units identical and would let more units be built and calibrated in the same way.

REFERENCES

Aasish, V. G., Mari Shankar, C., & Shanmugasundaram, M. (2025). FreeRTOS based smart waste management system. In *2025 Innovations in Power and Advanced Computing Technologies (i-PACT)* (pp. 1–7). IEEE. https://doi.org/10.1109/i-PACT65952.2025.11307820

Abdullahi, A., Mohammed, A., Bonet, M. U., El-Suleiman, A., Ahmad, R. B., & Chollom, T. D. (2024). Development of a smart waste management system with automatic bin lid control for smart city environment. *EAI Endorsed Transactions on Smart Cities, 7*(3). https://doi.org/10.4108/eetsc.4385

Alnanih, R., Elrefaei, L., & Al-Ahwal, A. (2025). Advancing sustainability through an IoT-driven smart waste management system with software engineering integration. *Sustainability, 17*(21), Article 9803. https://doi.org/10.3390/su17219803

Alnuaimi, A. N., Aydin, R., & Al-Shihabi, S. (2024). Optimizing municipal solid waste collection using smart bins: A case study in United Arab Emirates. In *2024 IEEE International Conference on Technology Management, Operations and Decisions (ICTMOD)* (pp. 1–6). IEEE. https://doi.org/10.1109/ICTMOD63116.2024.10878234

Alourani, A., Ashraf, M. U., & Aloraini, M. (2025). Smart waste management and classification system using advanced IoT and AI technologies. *PeerJ Computer Science, 11*, Article e2777. https://doi.org/10.7717/peerj-cs.2777

Asido, D. (2024, January 19). Baguio students develop smart garbage bins. *The Post*. https://thepost.net.ph/news/campus/baguio-students-develop-smart-garbage-bins/

Awotayo, M. M., & Dodo, U. A. (2026). Design and implementation of an IoT-based smart bin for real-time fill level monitoring. *Journal of Science Technology and Education, 14*(1), 344–351. https://www.researchgate.net/publication/400748751

Ayyanagoudar, A., Mohankar, A., Gharte, J., Pasnsare, P., Ainapure, B., & Appasani, B. (2024). A smart bin with real-time monitoring and garbage level tracking using IoT. In *2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends*

and Future Directions) (ICRITO) (pp. 1–7). IEEE. https://doi.org/10.1109/ICRITO61523.2024.10522263

Azmi, A. Z., Sahar, N. M., & Islam, M. T. (2024). Smart waste management system using LoRa technology. *E3S Web of Conferences, 599*, Article 04007. https://doi.org/10.1051/e3sconf/202459904007

Bahga, A., & Madisetti, V. (2014). *Internet of things: A hands-on approach*.

Balachandran, N., & Talavera, O. (2023). *Apparatus and method for waste monitoring and analysis* (U.S. Patent No. 11,788,877). U.S. Patent and Trademark Office. https://patents.google.com/patent/US11788877B2

Banciu, C., Florea, A., & Popa, M. (2025). IoT-enabled waste collection: Route optimization and environmental impact reduction using smart technologies. In *2025 29th International Conference on System Theory, Control and Computing (ICSTCC)* (pp. 782–788). IEEE. https://doi.org/10.1109/ICSTCC66753.2025.11240462

Bigbelly. (n.d.). *CLEAN software*. Retrieved October 19, 2026, from https://bigbelly.com/products/clean-software

Bosch Sensortec. (n.d.). *BME680 gas sensor* (Document No. BST-BME680-DS001) [Data sheet]. Retrieved October 19, 2026, from https://www.bosch-sensortec.com/media/boschsensortec/downloads/datasheets/bst-bme680-ds001.pdf

Brown, W. J. (2006). *Compactor service and monitoring system* (U.S. Patent No. 7,145,450). U.S. Patent and Trademark Office. https://patents.google.com/patent/US7145450B2

Circuitrocks. (n.d.-a). *2 channel 18650-battery holder protection board*. Retrieved October 15, 2026, from https://circuit.rocks/products/2-channel-18650-battery-holder-protection-board

Circuitrocks. (n.d.-b). *4 channel 18650-battery holder protection board*. Retrieved October 15, 2026, from https://circuit.rocks/products/4-channel-18650-battery-holder-protection-board

Circuitrocks. (n.d.-c). *IR proximity obstacle avoidance sensor module. Retrieved October 30, 2026, from https://circuit.rocks/products/product-2201*

Circuitrocks. (n.d.-d). *KY-016 RGB 3 color full-color LED module*. Retrieved October 15, 2026, from https://circuit.rocks/products/ky-016-rgb-3-color-full-color-led-module

Compology. (n.d.). *Image-based container monitoring built for waste*. Retrieved October 19, 2026, from https://compology.com/

Devitt, S., & Walsh, B. (2021). *Techniques for optimally sensing full containers* (U.S. Patent No. 11,054,545). U.S. Patent and Trademark Office. https://patents.google.com/patent/US11054545B2

Di Giugno, M., Giovanelli, F., Tiraboschi, S., & Villa, P. (2016). *Measuring device, container means and system for monitoring and managing container means* (European Patent No. 2516974). European Patent Office. https://patents.google.com/patent/EP2516974B1

Ecological Solid Waste Management Act of 2000, Republic Act No. 9003 (2001). https://lawphil.net/statutes/repacts/ra2001/ra_9003_2001.html

Ecube Labs. (n.d.). *Waste analytics platform: CleanCityNetworks (CCN)*. Retrieved October 19, 2026, from https://www.ecubelabs.com/waste-analytics-platform/

ElectronicsComp.com. (n.d.). *5 kg load cell with HX711 module shell and 4P DuPont wire kit*. Retrieved October 15, 2026, from https://www.electronicscomp.com/5-kg-load-cell-with-hx711-module-shell-and-4p-dupont-wire-kit

ElectroPeak. (n.d.). *GY-BMI160 module 6DOF 6 axis angular velocity*. Retrieved October 15, 2026, from https://electropeak.com/gy-bmi160-module-6dof-6-axis-angular-velocity

Enache, B.-A., Banica, C.-K., & Bogdan, A. G. (2024). Intelligent trash level detection for sustainable urban waste management. *The Scientific Bulletin of Electrical Engineering Faculty, 24*(1), 56–59. https://doi.org/10.2478/sbeef-2024-0010

Evreka. (2020). *Fill level sensor*. Retrieved October 19, 2026, from https://evreka.co/solutions/fill-level-sensor/

Gajula, R., Karanam, M., Vakkapatla, D. C., Endrapu, K. R., Raju, V. V. R., Reddy, S. B., Aljanabi, S. O., Koli, S., & Mahajan, S. (2025). Sustainable waste management solution: ESP32-powered smart bin. *AIP Conference Proceedings, 3263*(1), Article 140006. https://doi.org/10.1063/5.0261359

González, G. E., Escobar, E. R., Carvajal, I. I., & Bento, A. C. (2024). Smart trash bins: Integration of IoT with AI for efficient waste management. In *2024 IEEE International Humanitarian Technologies Conference (IHTC)* (pp. 1–6). IEEE. https://doi.org/10.1109/IHTC61819.2024.10855147

Greenwalt, M. (2020, June 2). Bin-e creates contactless smart bin. *Waste360*. https://www.waste360.com/recycling/bin-e-creates-contactless-smart-bin

Gupta, S., Gautam, R., & Muneer, A. (2025). IoT-based e-tracking system for waste management. In *2025 12th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)* (pp. 1–6). IEEE. https://doi.org/10.1109/ICRITO66076.2025.11241930

Ishaq, A., Mohammad, S. J., Bello, A. D., Wada, S. A., Adebayo, A., & Jagun, Z. T. (2025). Smart waste bin monitoring using IoT for sustainable biomedical waste management. *Environmental Science and Pollution Research, 32*(32), 19434–19449. https://doi.org/10.1007/s11356-023-30240-1

Karthi, S. P., Aravind, A., Dharineesh, V., Dhayanidhi, J. V., & Dinesh Kumar, J. R. (2025). IoT enabled smart dustbin with RFID-based access control and Blynk integration. In *2025 IEEE First International Conference on Innovations in Engineering and Next-Generation Technologies for Sustainability (ICINVENTS)*. IEEE. https://doi.org/10.1109/ICINVENTS64613.2025.11402326

Kekäläinen, F. (2018). *Method and system for monitoring and communicating fill rate of container* (U.S. Patent No. 9,930,429). U.S. Patent and Trademark Office. https://patents.google.com/patent/US9930429B2

Kekalainen, F., & Engstrom, J. (2014). *Smart waste collection system and method* (International Patent Application Publication No. WO 2014/079586 A1). World Intellectual Property Organization. https://patents.google.com/patent/WO2014079586A1

Kekalainen, F., & Engstrom, J. (2018). *Sensor device for smart waste collection systems and method* (U.S. Patent No. 9,927,278). U.S. Patent and Trademark Office. https://patents.google.com/patent/US9927278B2

Keller, J. R., Redmond, M. O., & Mirza, S. A. (2024). *System and method for measuring the fill level of a trash can using a sensor* (U.S. Patent No. 12,125,361). U.S. Patent and Trademark Office. https://patents.google.com/patent/US12125361B2

Kumar, K. R., Pavan, O. S., Azeez, S. A., Suresh, C. V., Sree, K. R., & Reddy, P. U. (2025). IoT enabled route optimization for smart waste management using machine learning. *i-manager's Journal on IoT and Smart Automation, 3*(2), 11-23. https://doi.org/10.26634/jiot.3.2.21684

Kurani, H. B. (2023). *Smart waste bin sensor device and methods for waste management system* (U.S. Patent No. 11,702,280). U.S. Patent and Trademark Office. https://patents.google.com/patent/US11702280B2

Kuzhin, M. F., Joshi, A., Mittal, V., Khatkar, M., & Guven, U. (2024). Optimizing waste management through IoT and analytics: A case study using the Waste Management Optimization Test. *BIO Web of Conferences, 86*, Article 01090. https://doi.org/10.1051/bioconf/20248601090

LCDWIKI. (n.d.). *2.4inch SPI module ILI9341 SKU:MSP2402*. Retrieved October 15, 2026, from https://www.lcdwiki.com/2.4inch_SPI_Module_ILI9341_SKU:MSP2402

Leones, J. R. (2025, March 3). *Philippines ministerial statement* [Paper presentation]. High-Level 12th Regional 3R and Circular Economy Forum in Asia and the Pacific, Jaipur, India. https://uncrd.un.org/sites/uncrd.un.org/files/12th3r_ps1-philippines.pdf

Maulida, A., Fayzalmi, C. S., Anaam, F. A. C., Ardiansyah, D., Gunawan, M. A., & Ridwan, N. (2025). Design of IoT-based smart trash bin monitoring using ESP32 and Firebase.

Jurnal Komputer, Teknologi Informasi dan Sistem Informasi (JUKTISI), 4(2), 420-428. https://doi.org/10.62712/juktisi.v4i2.420

Mehta, H., Deriya, V., Sankhe, D., Kanani, P., Pandya, G., & Shelke, V. (2025). Smart bins, smarter cities: IoT-driven waste collection with real-time sensing and methane mapping. *Mathematical Modelling of Engineering Problems, 12*(6), 2110–2122. https://doi.org/10.18280/mmep.120626

Mehta, S., & Rathour, A. (2024). Innovative waste management: The efficiency of IoT-enabled smart bins. In *2024 7th International Conference on Circuit Power and Computing Technologies (ICCPCT)* (pp. 467–471). IEEE. https://doi.org/10.1109/ICCPCT61902.2024.10672653

Mugaruka, O. M., Kala, J. R., Ndikumagenge, J., van Schadewijk, R., & Nibitanga, R. (2026). Design of an IoT sensor-based network architecture for waste collection in Bukavu city (DRC). *Discover Internet of Things, 6*(1), Article 168. https://doi.org/10.1007/s43926-026-00479-4

Mullaney, G. (2016). *Refuse container with weight indicator and danger alerting* (U.S. Patent No. 9,347,821). U.S. Patent and Trademark Office. https://patents.google.com/patent/US9347821B1

Muthukumar, K., Siddarth, T., Udhayakumar, K., & Saratha, M. (2024). Smart bin: AI-based waste segregation with monitoring system. *International Journal of Innovative Research in Technology, 11*(6), 1293–1298. https://ijirt.org/Article?manuscript=169606

Neumann, R. H., & Allison, C. B. (1991). *Automated trash management system* (U.S. Patent No. 5,016,197). U.S. Patent and Trademark Office. https://patents.google.com/patent/US5016197A

Nordsense. (n.d.). *Intelligent routing*. Retrieved October 19, 2026, from https://nordsense.com/intelligent-routing/

Philippine Statistics Authority. (2026). *Highlights of the Compendium of Philippine Environment Statistics 2016–2025, Component 3: Residuals*. https://psa.gov.ph/statistics/environment-statistics/highlights/component-3-residuals

Poss, J., Satwicz, J., Richmond, B., & Taylor, M. (2006). *Solar powered compaction apparatus* (U.S. Patent No. 7,124,680). U.S. Patent and Trademark Office. https://patents.google.com/patent/US7124680B2

QuartzComponents. (n.d.). *Passive buzzer module with PCB*. Retrieved October 15, 2026, from https://quartzcomponents.com/products/passive-buzzer-module

Rathnayake, R. M. P. M. D., Chanaka, T. M. M., Dabare, P. T. R., & Hewagamage, K. P. (2024). IoT-enabled dual-sensing smart waste management system: Enhancing urban cleanliness and sustainability in smart cities. In *2024 8th SLAAI International Conference on Artificial Intelligence (SLAAI-ICAI)*. IEEE. https://doi.org/10.1109/SLAAI-ICAI63667.2024.10844960

Robu.in. (n.d.). *BME680 digital temperature humidity pressure sensor CJMCU-680 high altitude sensor module development board*. Retrieved October 15, 2026, from https://robu.in/product/bme680-digital-temperature-humidity-pressure-sensor-cjmcu-680-high-altitude-sensor-module-development-board/

Saad, A. M., Jul, B. O., Basalamah, A., & Sayuti, S. (2023). IoT-based smart dustbin prototype. *PROtek: Jurnal Ilmiah Teknik Elektro, 10*(2), 120–125. https://doi.org/10.33387/protk.v10i2.6035

Sadikin, F. R., Hikmawati, E., & Aulia, S. (2025). TrashTrack: A smart waste monitoring system with constraint-based greedy algorithm for optimized collection routes. In *2025 IEEE International Conference on Artificial Intelligence for Learning and Optimization (ICoAILO)* (pp. 262–268). IEEE. https://doi.org/10.1109/ICoAILO66760.2025.11155998

Sait, S., Aliyu, F. M., Bukar, S., Maina, B. S., & Lawal, D. U. (2025). *Waste bin collection and monitoring method using waste bin sensors* (U.S. Patent No. 12,450,575). U.S. Patent and Trademark Office. https://patents.google.com/patent/US12450575B1

Salleh, A., Mohamad, N. R., & Hashim, N. M. Z. (2024). Optimizing waste collection efficiency by NodeMCU enabled ultrasonic trash can surveillance and ThingSpeak dashboards. *International Journal of Latest Technology in Engineering, Management & Applied Science, 12*(12), 19-26. https://doi.org/10.51583/ijltemas.2023.121202

Samolovac, J. (2023, January 25). *What is hybrid project management?* Plaky. https://plaky.com/learn/project-management/hybrid-project-management/

Sensoneo. (n.d.). *Waste management system*. Retrieved October 19, 2026, from https://www.sensoneo.com/products/waste-management-system/

Shahabdeen, J. A. (2016). *Smart garbage bin* (U.S. Patent Application Publication No. 2016/0176630 A1). U.S. Patent and Trademark Office. https://patents.google.com/patent/US20160176630A1

Shaharil, M. H. F., & Po'ad, F. A. (2024). Arduino-based smart dustbin with ultrasonic sensor for efficient waste management. *Evolution in Electrical and Electronic Engineering, 5*(2), 92–97. https://periodical.uthm.edu.my/index.php/eeee/article/view/17147

Shahibollah, R. A., Fadzal, N., Fadzal, A. N., Omar, S., Jaafar, A. N., & Ahmad, N. (2025). IoT-enabled smart dustbin for real-time waste monitoring and scalable hygiene management. In *2025 IEEE 16th Control and System Graduate Research Colloquium (ICSGRC)* (pp. 75–80). IEEE. https://doi.org/10.1109/ICSGRC65918.2025.11159745

Shin, H. (2016). *Inputting device for quantity and weight loss of food garbage container* (Korean Patent No. 10-1581944). Korean Intellectual Property Office. https://patents.google.com/patent/KR101581944B1

Sigmanortec. (n.d.). *ESP32-DevKitC ESP32-WROOM-32D WiFi BT 38P development board*. Retrieved October 15, 2026, from https://sigmanortec.ro/en/esp32-devkitc-development-board-esp32-wroom-32d-38p

SparkFun Electronics. (n.d.). *Ultrasonic distance sensor - 5V (HC-SR04)*. Retrieved October 15, 2026, from https://www.sparkfun.com/ultrasonic-distance-sensor-hc-sr04.html

Thangam, S., Gurupriya, M., Yadav, C. S. K., Harshith, M., & Kumar, M. S. (2024). A smart waste management system using ESP8266. In *2024 4th International Conference on Sustainable Expert Systems (ICSES)* (pp. 1–5). IEEE. https://doi.org/10.1109/ICSES63445.2024.10762967

The Pi Hut. (n.d.). *TowerPro servo motor - SG90 analog*. Retrieved October 15, 2026, from https://thepihut.com/products/towerpro-servo-motor-sg90-analog

United Nations Department of Economic and Social Affairs. (n.d.). *Goal 11: Make cities and human settlements inclusive, safe, resilient and sustainable*. https://sdgs.un.org/goals/goal11

Wang, X., Chen, J., & Lin, Z. (2020). *Trash can having sensing flip lid having settable sensing distance and control method therefor* (European Patent No. 3 620 405 A1). European Patent Office. https://patents.google.com/patent/EP3620405A1

Wang, X., Zhang, Y., & Wang, Z. (2020). *Garbage bin intelligent control ware and garbage bin supervisory systems* (Chinese Patent No. CN111559590A). China National Intellectual Property Administration. https://patents.google.com/patent/CN111559590A

World Bank. (2022). *An assessment of municipal solid waste plans, collection, recycling and disposal of Metro Manila* (East Asia and Pacific Region: Marine Plastics Series). https://documents1.worldbank.org/curated/en/099125007222250938/pdf/P170994059e8e308c0a3ec02e0c4f57778b.pdf

Yan, S., & Chen, J. (2021). *Smart city garbage management system* (Chinese Patent No. CN 109324552 B). China National Intellectual Property Administration. https://patents.google.com/patent/CN109324552B

Yang, F., & Sandor, J. (2010). *Trash can with power operated lid* (U.S. Patent No. 7,781,995). U.S. Patent and Trademark Office. https://patents.google.com/patent/US7781995B2

Yang, S., Liao, D., Li, R., & Yu, Y. (2020). *Capacity automated inspection garbage bin with infrared induction function* (Chinese Patent No. CN 210504152 U). China National Intellectual Property Administration. https://patents.google.com/patent/CN210504152U

Yogesh, H. M., Shivalingappa, Suman, M., Vinodh, B., & Malini, V. L. (2025). IoT-enhanced smart dustbin with real-time weight and level monitoring using ESP32. *International Journal of Advanced Research in Science, Communication and Technology, 5*(10), 227-231. https://doi.org/10.48175/ijarsct-28841

CURRICULUM VITAE

RODEL S. BANOGBANOG
1230 Interior 8, Maaliwalas st, Kahilum2, Pandacan, Manila
Mobile No.: 09916540971
Email Address: rodel.banogbanog@tup.edu.ph

SKILLS

- Computer Systems Servicing NCII
- Electrical Installation and Maintenance NCII
- Basic knowledge C++, Java ang Python Programing
- Computer Installation and Server Configuration
- Basic knowledge in Information and Communication Technology
- Basic knowledge in Computer Configuration

EDUCATIONAL BACKGROUND

- Bachelor of Engineering Technology — August 2023
  Major in Computer Engineering Technology
  Technological University of the Philippines – Manila
- Ignacio Villamor Senior High School — June 2021
  2109 San Andres Ext, Sta. Ana Manila
- Alternative Learning System (ALS) in — June 2019
  Sta. Ana Elementary School, M. Roxas St, Sta. Ana
- Molave Vocational Technical School — June 2009
- Milligan Elementary School, Molave — June 2004

CONFERENCES/SEMINARS/TRAININGS ATTENDED

- ROTC Training Service — August 2024
  Technological University of the Philippines – Manila
- Sea gem Training Center — July 2021
- Kabaka Manpower Training Service — August 2013

CURRICULUM VITAE

BRIAN JAY CELESTE
829 Teresa St. Brgy. 661 Zone 70 Ermita, Manila 1000 Metro Manila
Mobile No.: 09272794551
Email Address: brianjay.celeste@tup.edu.ph

SKILLS

- Adaptable
- Communicator
- Critical Thinker
- Fast Learner
- Leadership Skills
- Diligent
- Computer Literate
- Programming and Web Development Skills
- Basic Electrical Circuits and Electronics Knowledge

EDUCATIONAL BACKGROUND

- Bachelor of Engineering Technology — August 2022 – Present
  Major in Computer Engineering Technology
  Technological University of the Philippines – Manila
- Technical-Vocational-Livelihood — June 2021
  ICT Strand – Mobile App and Web Development
  STI College Bacoor
- Technical-Vocational-Livelihood — August 2020
  ICT Strand – Mobile App and Web Development
  STI College Dasmariñas
- Technical-Vocational-Livelihood — August 2019
  ICT Strand – Mobile App and Web Development
  STI College Dasmariñas

CONFERENCES/SEMINARS/TRAININGS ATTENDED

- Beyond Pixels: Android and Visual Studio Mobile Development — August 2024
  Technological University of the Philippines – Manila
- Technobytes Tutorials — August 2021
  Introduction to Windows Forms App Development
  STI College Bacoor

MEMBERSHIP IN PROFESSIONAL ORGANIZATIONS

- CIT Student Council – Technical Associate — June 2023
  Technological University of the Philippines – Manila

- Technobytes — June 2022
  STI College Bacoor
- Gamer's Lounge — June 2021
  STI College Dasmariñas

AWARDS AND CERTIFICATIONS

- C-Language Competition Champion — May 2023
- 1st Place Research Paper — June 2022
- 2nd Place Research Poster — June 2022
- 3rd Place Sci-mulation – Air Vortex Cannon — June 2022
- JLPT N3 – Japanese Language Proficiency Test Certification — June 2023

WORK EXPERIENCE

- Technical Support Intern — June 2022
  STI College Bacoor
  Software and Hardware Troubleshooting
  Network Maintenance
- Aluminum Worker — June 2021
  Arami Tousou
  Aluminum Processing and Finishing
  Japanese–English Interpretation
  Technical Support Assistance

CURRICULUM VITAE

JADE C. GUNAYAN
110 Daisy st. Nbbs proper, Navotas City Metro Manila
Mobile No.: 09533713171
Email Address: jade.gunayan@tup.edu.ph

SKILLS

- Adaptable
- Communicator
- Critical Thinker
- Fast Learner
- Leadership Skills
- Diligent

EDUCATIONAL BACKGROUND

- Bachelor Engineering Technology — Present
  Major in Computer Engineering Technology
- Senior High School — 2021–2023
  San Rafael Technological Vocational High School
- Junior High School — 2017–2021
  San Pascual Agricultural High School
- Elementary — 2012–2017
  Bongbong Elementary School

CONFERENCES/SEMINARS/TRAININGS ATTENDED

- CWTS Training Service — August 2024
  Technological University of the Philippines – Manila

CURRICULUM VITAE

KING PERTH G. PEPITO
Blk 1. No. 19 C3-Rd, Pamasawata III, Kaunlaran Village, Caloocan City
Mobile No.: 09052165338
Email Address: kingperth.pepito@tup.edu.ph

SKILLS

- Computer Literate
- Project Management
- Attention to detail
- Accountability and Responsibility

EDUCATIONAL BACKGROUND

- Bachelor of Engineering Technology — August 2023
  Major in Computer Engineering Technology
  Technological University of the Philippines – Manila
- Technical-Vocational-Livelihood – ICT — June 2021
  Arellano University – Jose Rizal Campus
- Secondary School — June 2017
  Longos National High School
- Primary School — June 2012
  Ninoy Aquino Elementary School

CONFERENCES/SEMINARS/TRAININGS ATTENDED

- CWTS "Civic Welfare Training Service" — August 2024
  Technological University of the Philippines – Manila

MEMBERSHIP IN PROFESSIONAL ORGANIZATIONS

- ICPET "Institute of Computer Engineering Technologist" — August 2024
  Technological University of the Philippines – Manila
- MOAS "Ministry of Altar Servers" — February 2017
  Vice Coordinator
  San Roque Cathedral Parish

CURRICULUM VITAE

GENESIS WALTER D. REYES II
808 San Jose Street, Manuyo Uno, Las Piñas City
Mobile No.: 09766208735
Email Address: genesiswalter.reyesii@tup.edu.ph

SKILLS

- Computer Literate
- Project Management
- Attention to detail
- Accountability and Responsibility
- Proficient in these programming languages:
  - C
  - Python
  - Java
- Proficient in Electrical Circuits
- Proficient in Electronics

EDUCATIONAL BACKGROUND

- Bachelor of Engineering Technology 2023–2025
  Major in Computer Engineering Technology
  Technological University of the Philippines – Manila
- Senior High School (STEM Strand) 2021–2023
  Science, Technology, Engineering, Mathematics.
  St. Joseph's Academy, Las Piñas City
- Junior High School 2017–2020
  St. Joseph's Academy, Las Piñas City

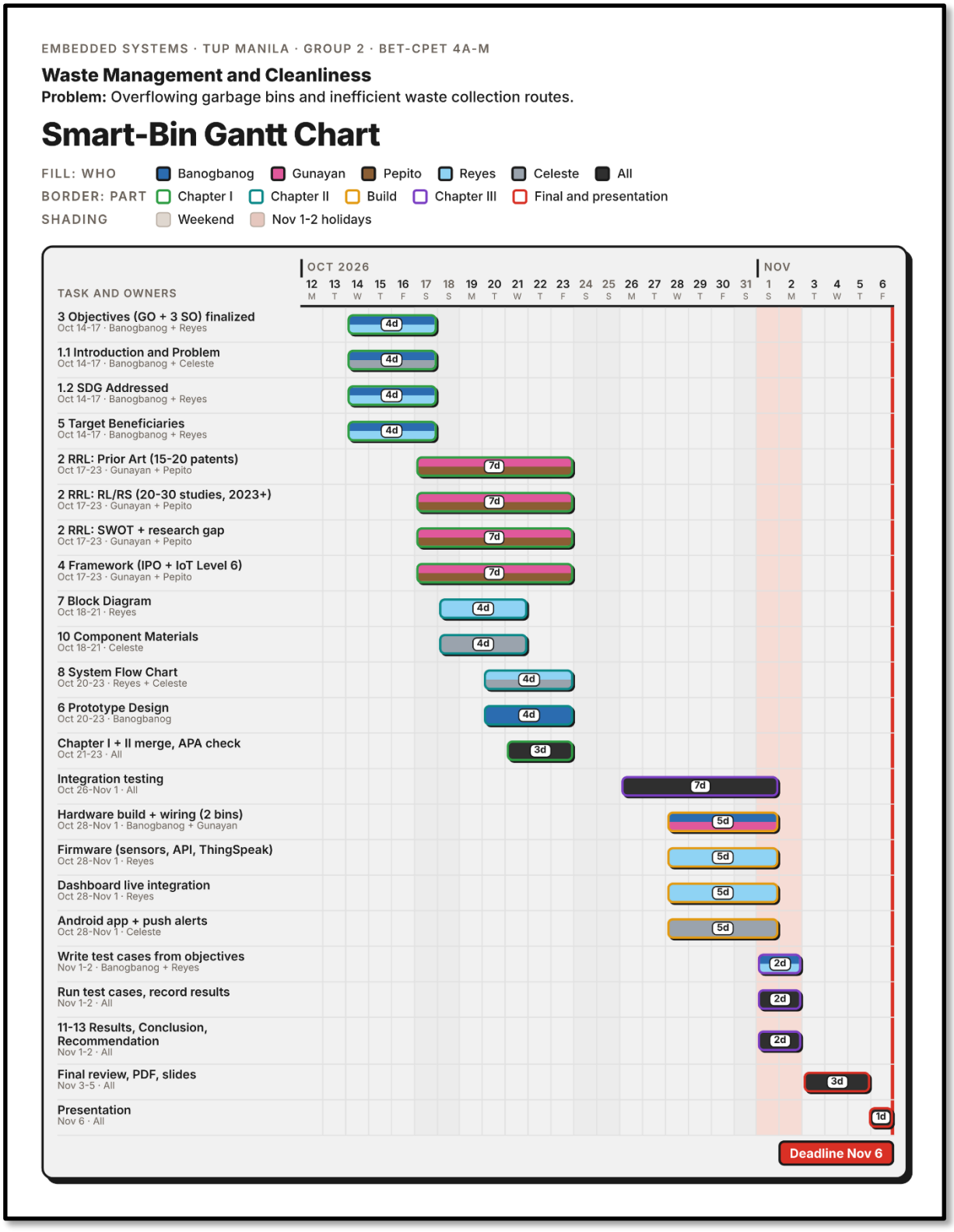
CONFERENCES/SEMINARS/TRAININGS ATTENDED

- Beyond Pixels: Android and Visual Studio 2024
  Android and Visual Studio Mobile Development

APPENDICES

Appendix A

Gantt Chart



Appendix B

Bill of Materials of the Smart-Bin Prototype

Component	Quantity	Unit price	Amount
ESP32 DevKit V1 (38-pin, WROOM-32D)	2	₱250.00	₱500.00
HC-SR04 ultrasonic sensor	2	₱45.00	₱90.00
FC-51 IR obstacle avoidance sensor	2	₱35.00	₱70.00
SG90 micro servo motor (180°)	2	₱65.00	₱130.00
2.4-in. TFT touch display (ILI9341 + XPT2046)	2	₱280.00	₱560.00
BMI160 six-axis inertial sensor module	2	₱80.00	₱160.00
BME680 environmental sensor	2	₱400.00	₱800.00
HX711 load cell amplifier board	2	₱60.00	₱120.00
5kg straight bar load cell	2	₱120.00	₱240.00
KY-016 RGB LED module (common cathode)	2	₱8.00	₱16.00
MH-FMD passive buzzer module	2	₱12.00	₱24.00
18650 battery shield, 4-cell (Bin A)	1	₱310.00	₱310.00
18650 battery shield, 2-cell (Bin B)	1	₱206.00	₱206.00
18650 Li-ion cell, 3.7V 2500mAh	6	₱203.00	₱1,218.00
Ceramic capacitor 100nF (pack of 10)	2	₱75.00	₱150.00
Electrolytic capacitor 10µF (pack of 10)	1	₱39.00	₱39.00
Electrolytic capacitor 1000µF (pack of 10)	1	₱165.00	₱165.00
Resistor 1kΩ (pack of 10)	1	₱19.00	₱19.00
Resistor 2.2kΩ (pack of 10)	1	₱19.00	₱19.00
Resistor 10kΩ (pack of 10)	1	₱19.00	₱19.00
Smart-Bin building materials	2	₱250	₱500
Universal PCB and materials	2	₱300	₱600
Total			**₱5,955.00**

Note. Quantities apply to both bins. The market prices as of the check in October 2026.

₱1,191.00 per person.

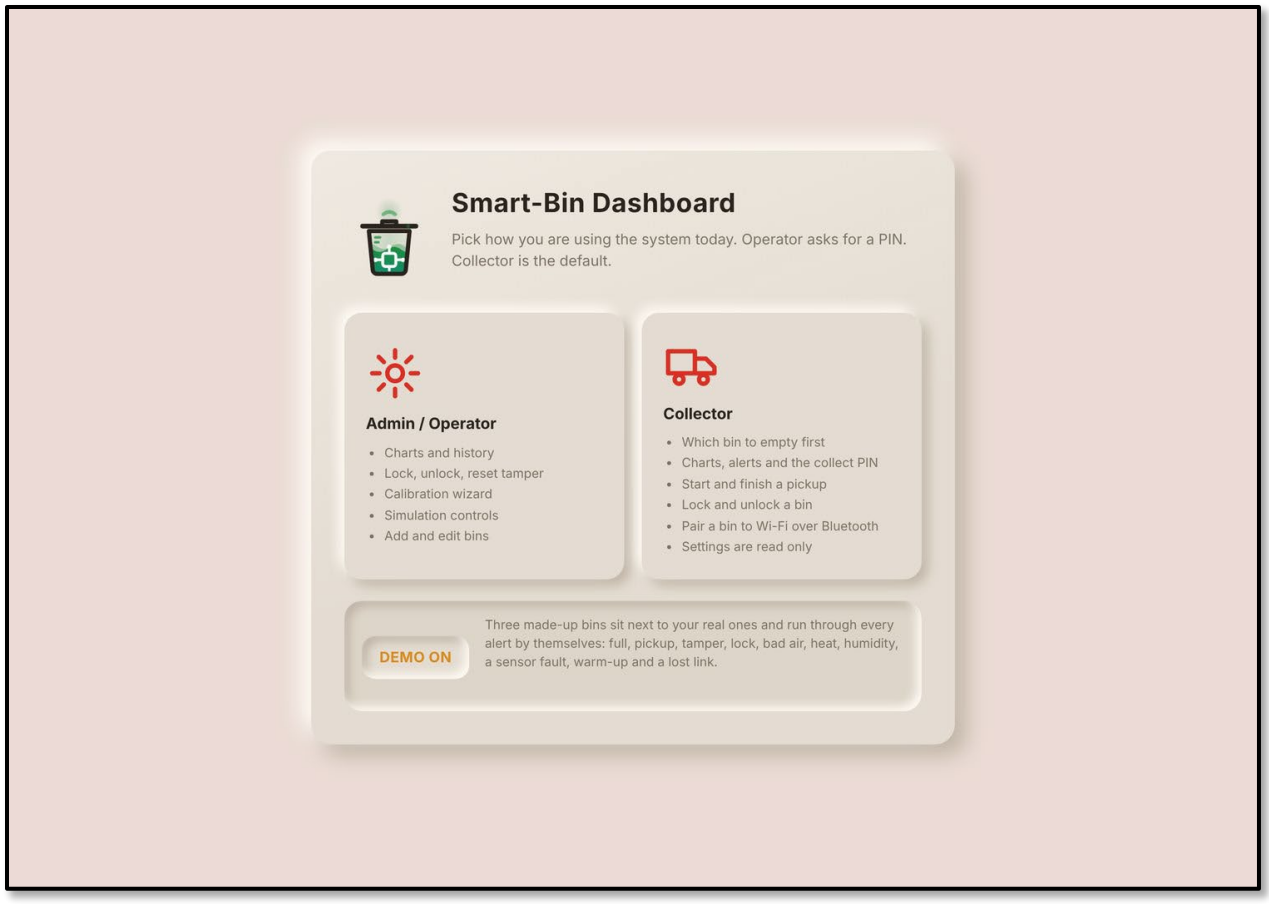
Appendix C

User Interface Screens

This appendix shows the two user interfaces of the Smart-Bin: the web dashboard served by the ESP32 and the Android app. The screens were captured from the actual software, the dashboard in a desktop browser and app version 2.2 on an Android emulator. Both ran on their built-in demo data, so the bin names and readings in the figures are sample values and not measurements. Figures C1 to C16 cover the web dashboard, from the start page and operator PIN through the Overview, Charts, Settings, Simulation and Alerts tabs. Figures C17 to C29 cover the same functions in the Android app, along with its app settings, connection and Bluetooth setup pages.

Figure C1

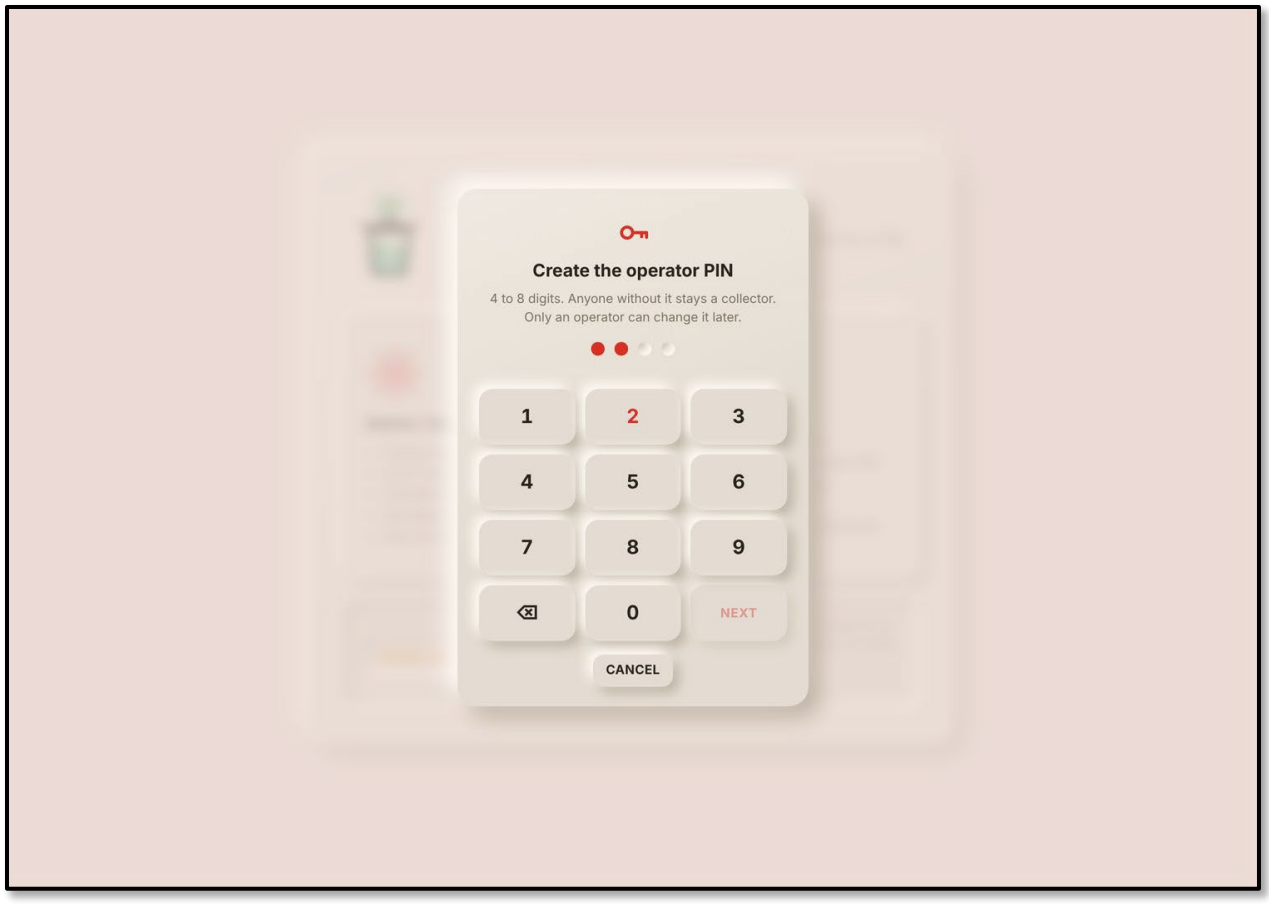
Start Page of the Web Dashboard



The dashboard opens on this page every time. The user picks a role first. Admin / Operator can change settings, calibrate, simulate and add bins, and it asks for a PIN. Collector is the default and can see everything but only acts on pickups, locks and Bluetooth setup. The DEMO key at the bottom adds three made-up bins that act out every alert by themselves, and it can only be switched on or off here.

Figure C2

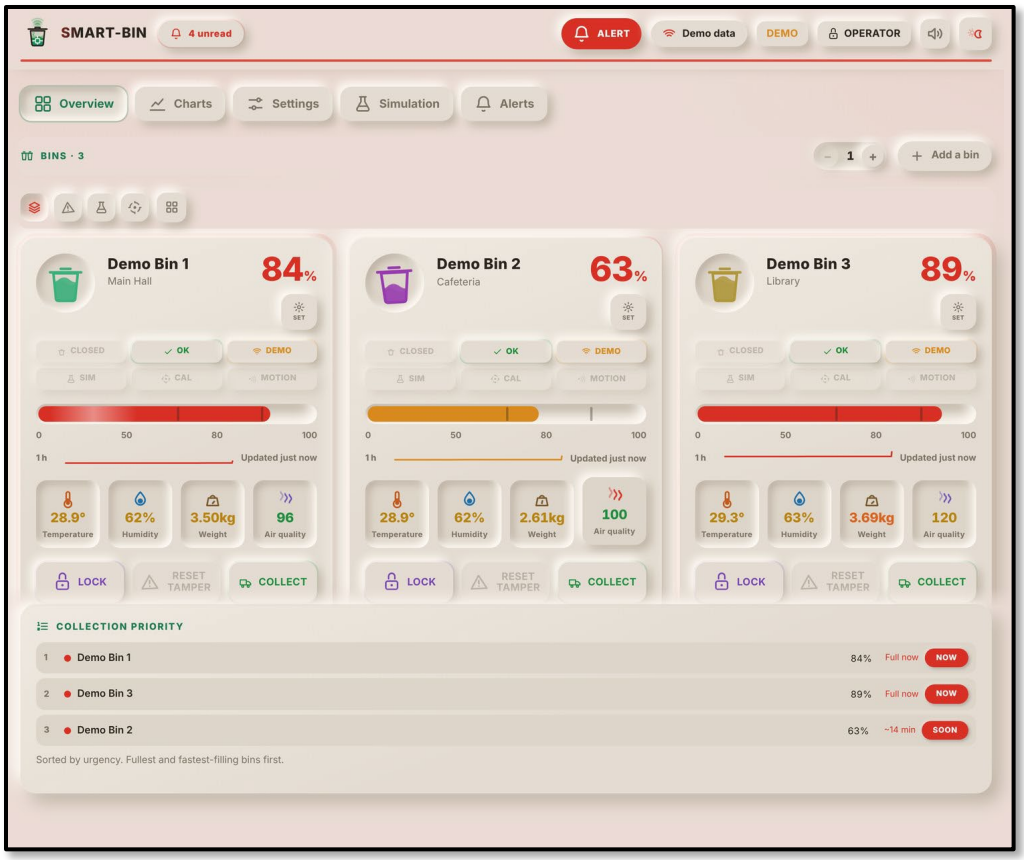
Creating the Operator PIN



The first time Admin / Operator is picked, the dashboard asks for a new PIN of 4 to 8 digits, typed twice. Only a salted hash of the PIN is stored in the browser. After 5 wrong tries the keypad waits 30 s, and the wait doubles each time. The dashboard also drops back to collector after 2 min with the tab hidden.

Figure C3

Overview Tab with Bin Cards and Collection Priority

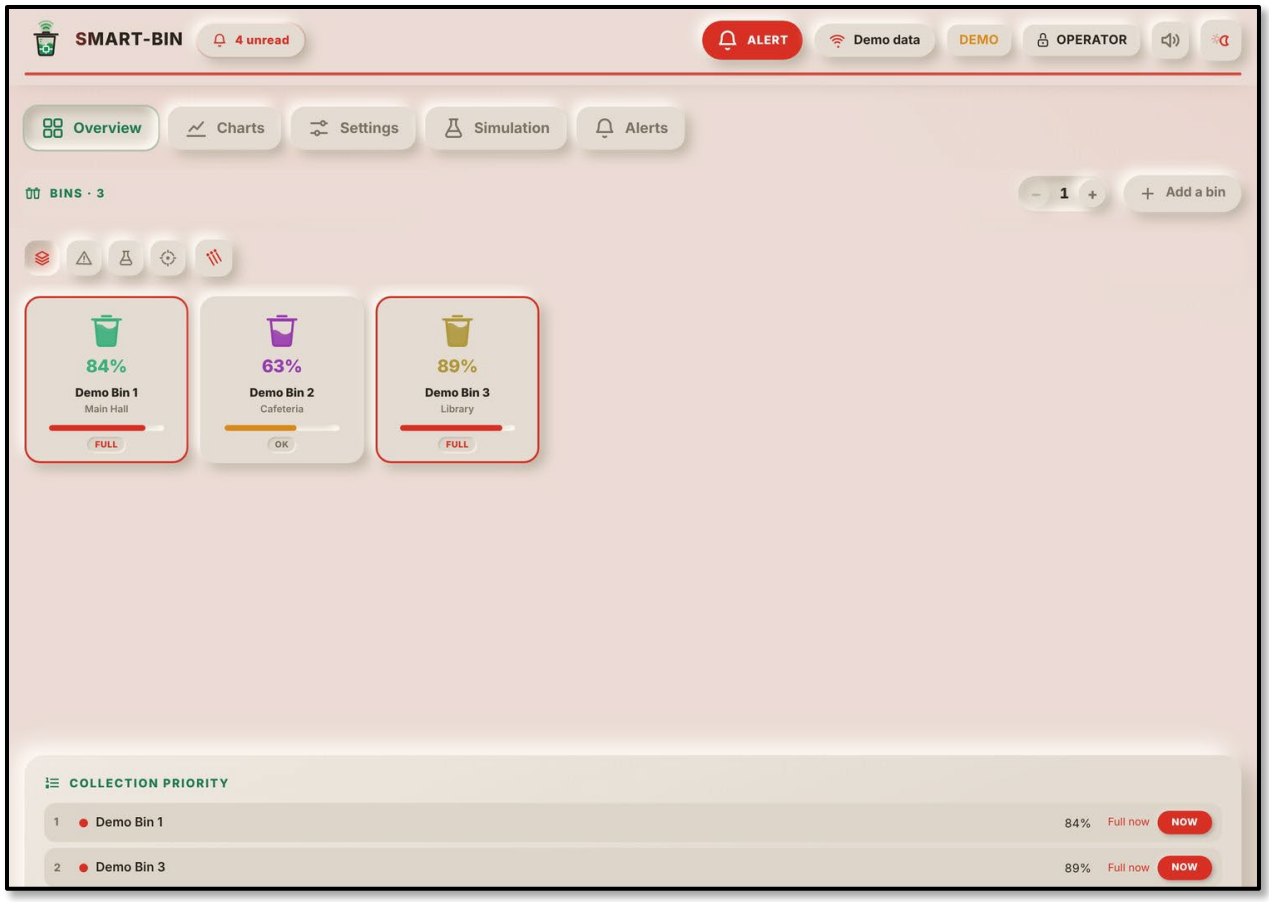


Each bin has one card. The large number is the fill level and the bar under it marks 50% and the alert limit (80% here). The chips show the lid, the bin status, the link in use, and whether simulation, calibration or motion is active. The four tiles show temperature, humidity, weight and the air quality index from the BME680, and a tile turns red when its reading passes its limit. The keys at the bottom send LOCK or UNLOCK, RESET TAMPER and COLLECT to the bin.

Collection Priority lists the bins in the order they should be emptied: full bins first, then the bins that will fill soonest, from the fill rate of the last 15 min. The tags NOW, SOON, TODAY, LATER and IDLE are the same in the phone app. The header shows the unread alerts, the system mode (ALERT here, because bins are full), the link (Demo data), the role, and the sound and theme keys.

Figure C4

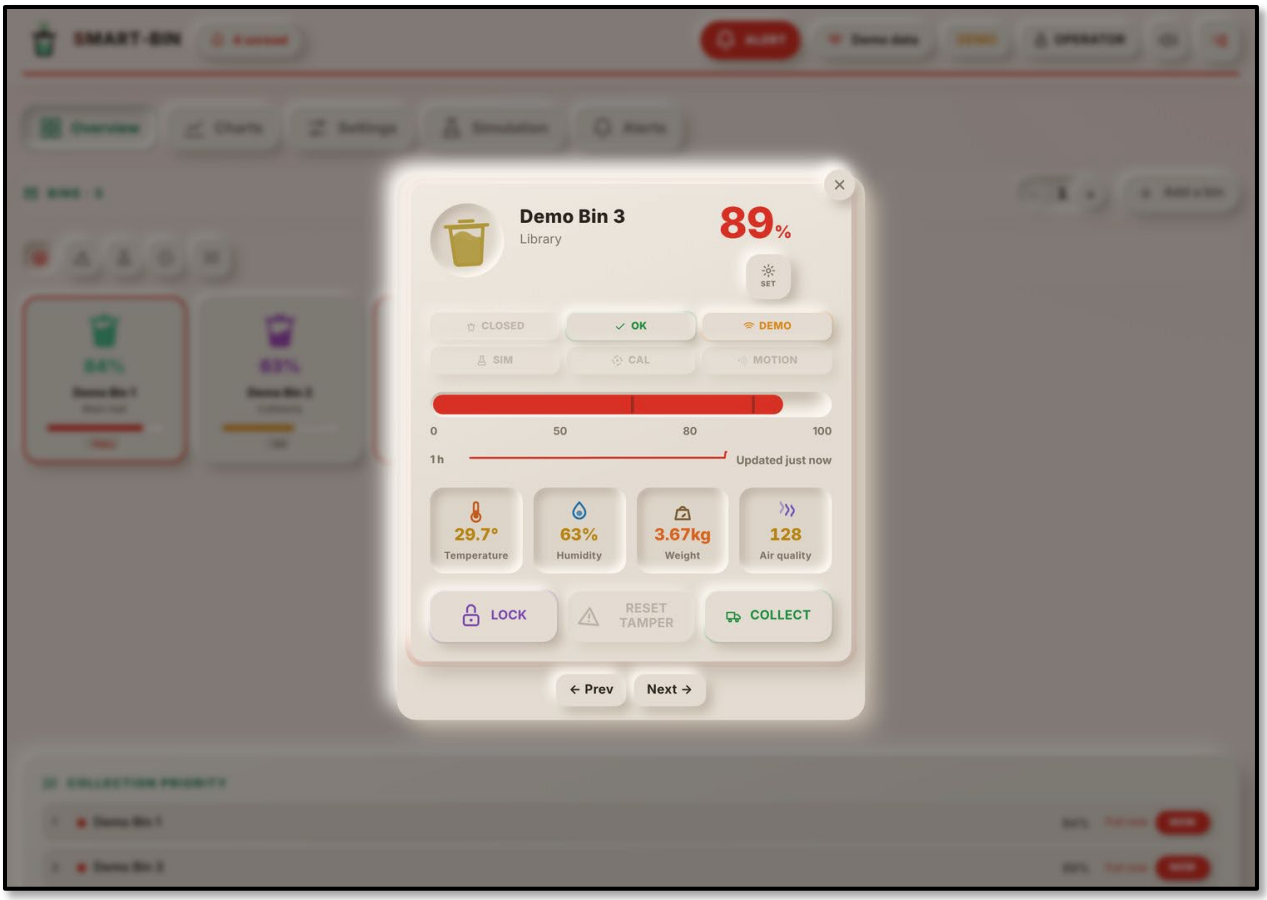
Overview Tab in Grid View



With 3 or more bins the cards can be switched to small tiles with the grid key, so many bins fit on one screen. Each tile shows the fill level, the name, the location and the status. The four filter keys above the bins show all bins, only the bins that need attention, bins in simulation, or bins being calibrated.

Figure C5

Bin Detail Opened from the Grid



Clicking a tile opens the full card of that bin over the page, with the same readings and keys as the Overview card. Prev and Next step through the bins without closing it.

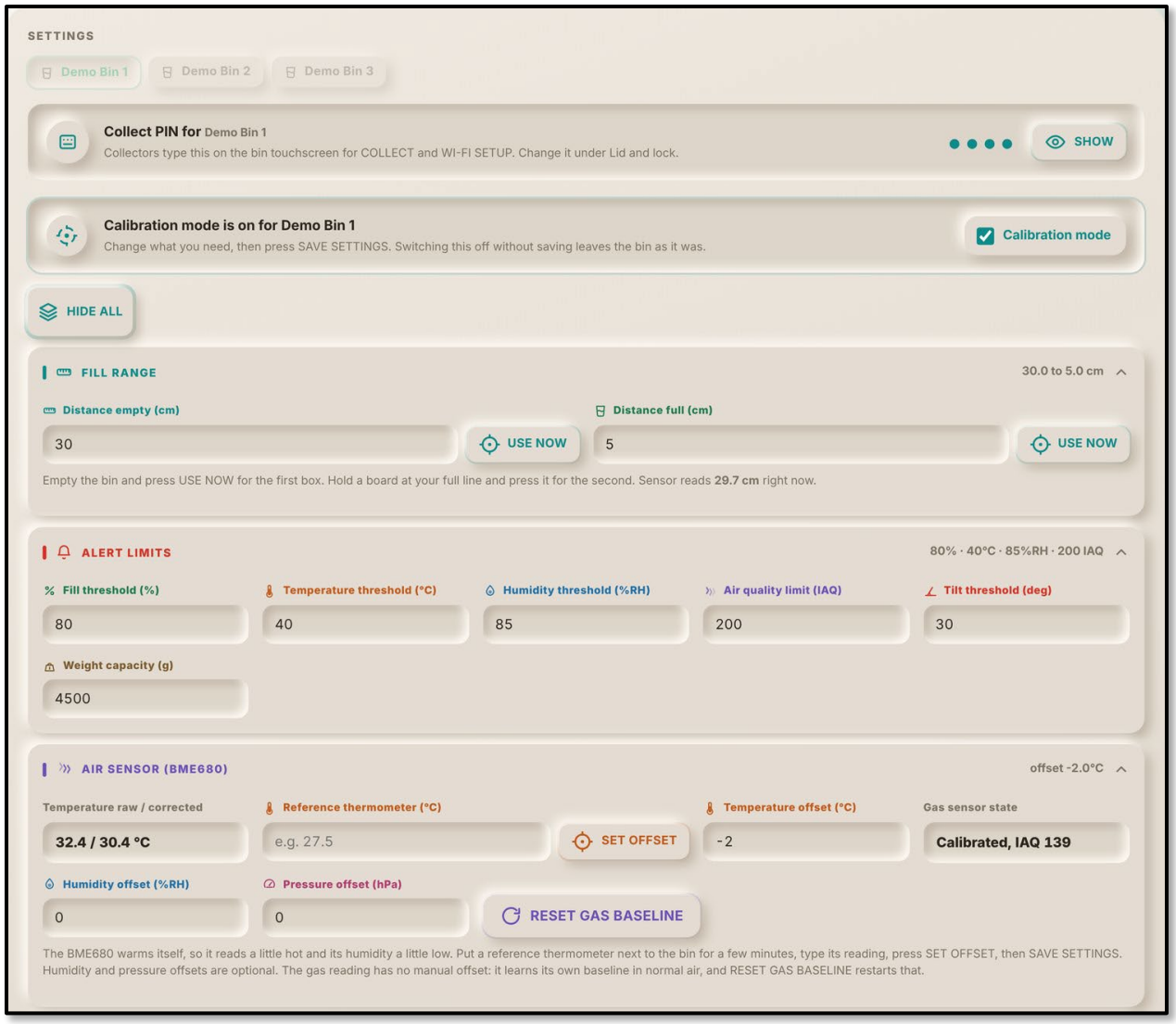
Figure C6

Charts Tab

The Fill Level chart shows the fill of every bin over the last 15 min, 1 h, 6 h or 24 h, with the alert limit as a dashed red line. One bin or all bins can be picked. The Environment chart shows temperature, humidity and air quality of one bin. In the ALL view each reading is drawn as a share of its own alert limit, so the dashed line at 100% is the limit for all three. Picking one reading, or PRESSURE, draws it in its real unit instead.

Figure C7

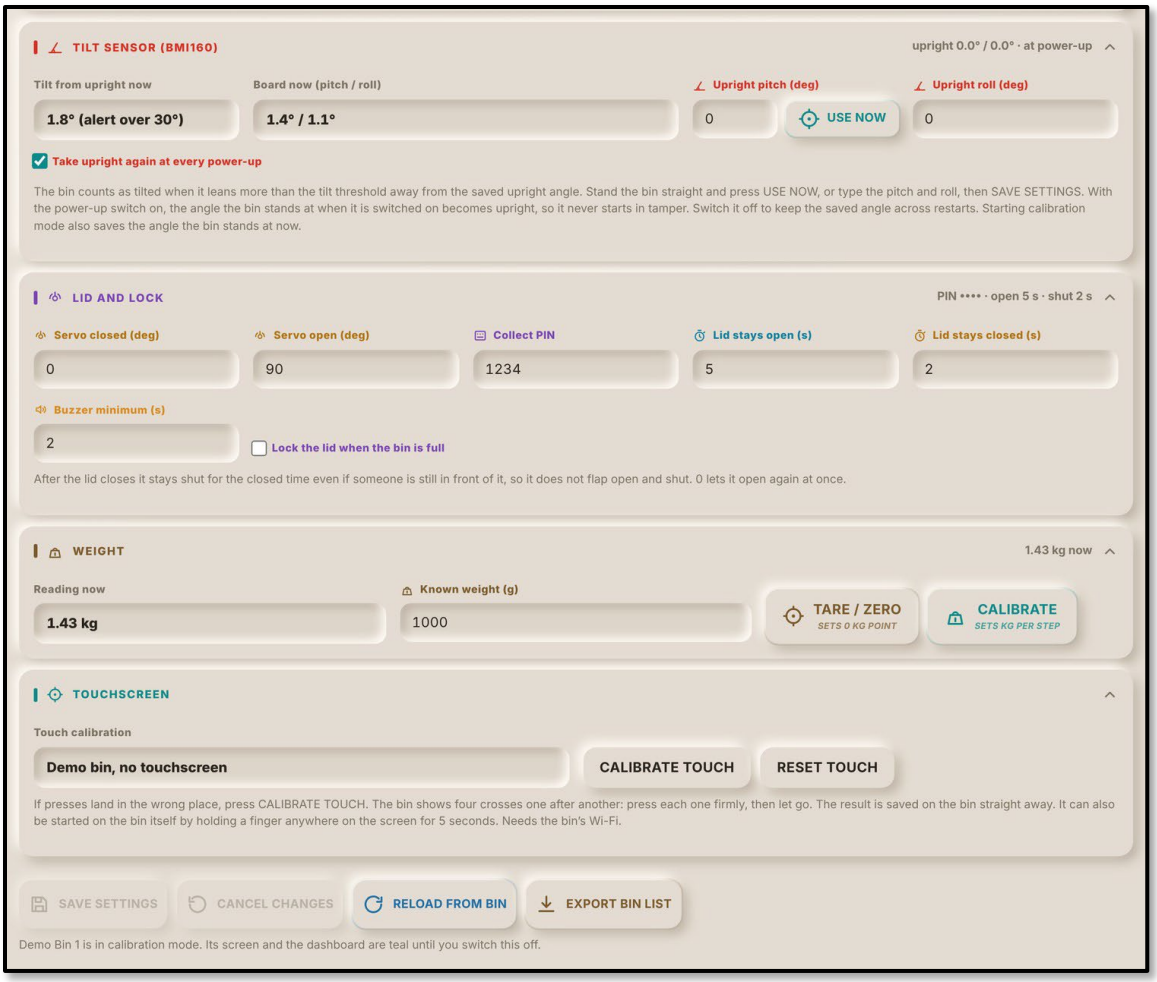
Settings Tab in Calibration Mode: Fill Range, Alert Limits and Air Sensor



An operator switches on Calibration mode for one bin. The page and the bin's own screen turn teal, and the fields can then be edited. The first groups are Fill range (the distances for empty and full, with USE NOW to take the live sensor reading), Alert limits, and Air sensor (temperature offset against a reference thermometer and the gas baseline reset). A collector sees the same values but cannot change them.

Figure C8

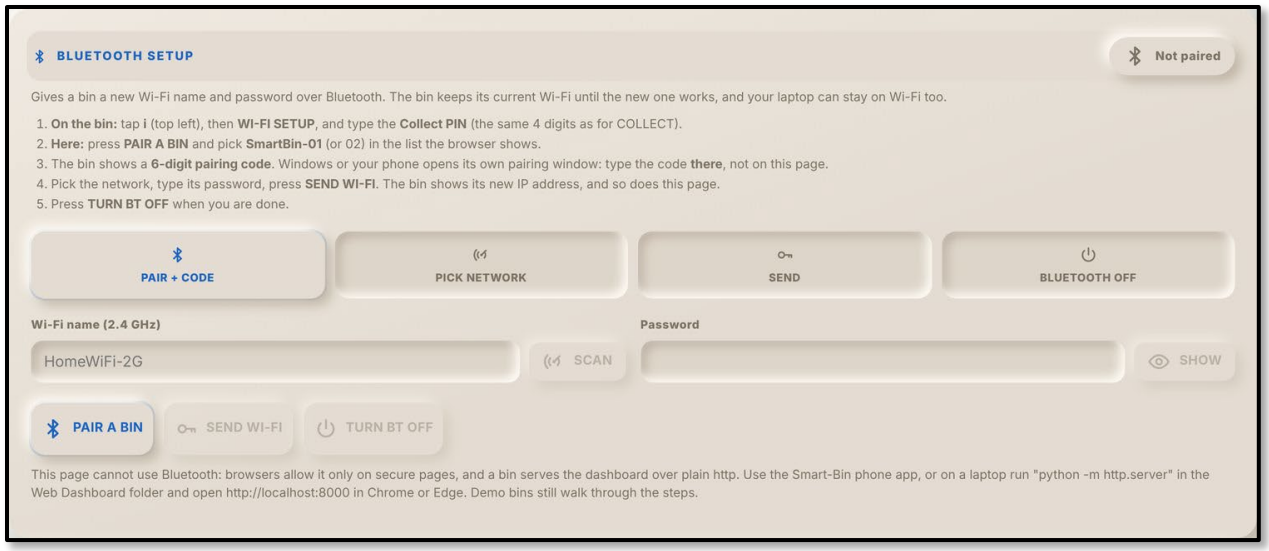
Settings Tab in Calibration Mode: Tilt Sensor, Lid, Weight and Touchscreen



The rest of the calibration page holds Tilt sensor (the live tilt, the board's pitch and roll, the saved upright angle with USE NOW, and the switch that takes the upright again at every power-up), Lid and lock (servo angles, Collect PIN, how long the lid stays open and how long it stays closed, buzzer time, auto-lock when full), Weight (tare and calibration with a known weight) and Touchscreen (touch calibration). SAVE SETTINGS sends the values to the bin, which keeps them after a restart.

Figure C9

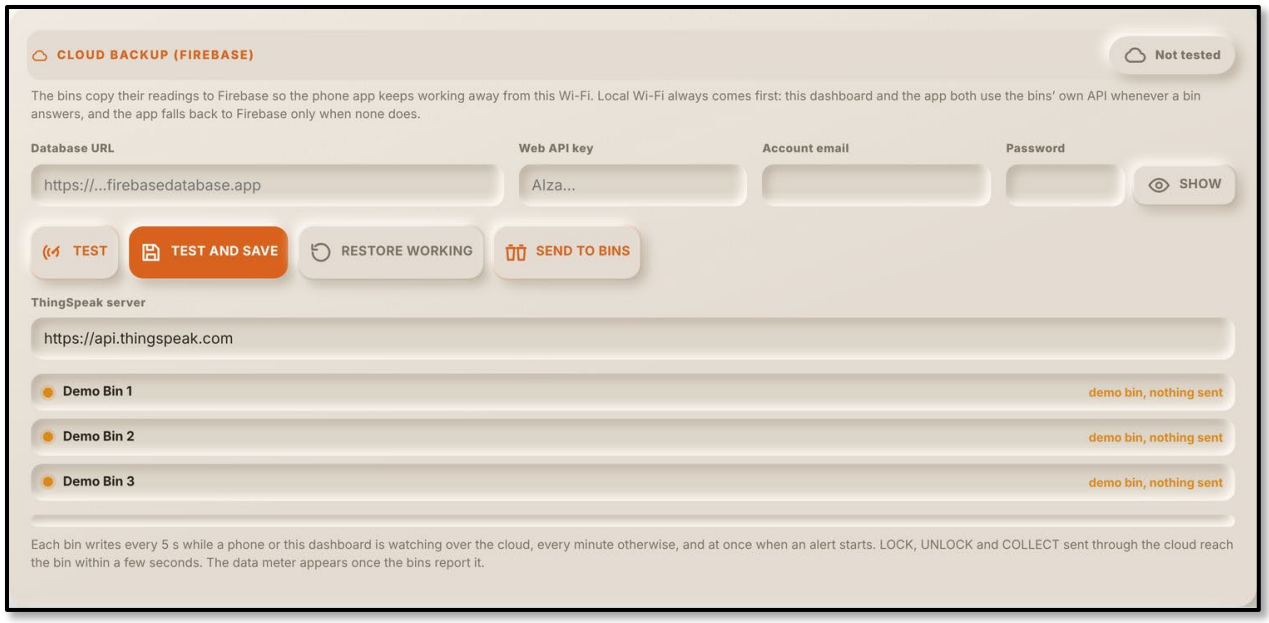
Bluetooth Setup Panel



This panel gives a bin a new Wi-Fi network over Bluetooth LE. The steps are shown across the top: pair with the 6-digit code that the bin shows on its screen, pick a network from the bin's scan, send the name and password, then switch Bluetooth off. Browsers allow Bluetooth only on secure pages, so this works when the dashboard is opened from a laptop and not from the bin's own http page; the panel says so.

Figure C10

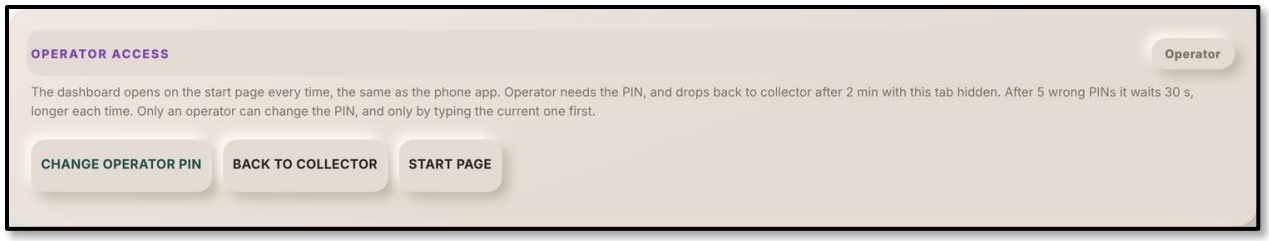
Cloud Backup Panel for Firebase and ThingSpeak



The Firebase account (database URL, Web API key, email and password) is entered here. TEST signs in and checks access, TEST AND SAVE keeps the account, and SEND TO BINS gives it to every bin on Wi-Fi so they all write to the same database. The rows below show what each bin last sent. Local Wi-Fi is always tried first; Firebase and then ThingSpeak are used only when a bin does not answer on Wi-Fi.

Figure C11

Operator Access Panel



The operator can change the PIN, go back to collector, or return to the start page from here.

Figure C12

Demo Options Panel

Shown only while DEMO is on. The demo bins can be paused or sped up, made to act out a full bin, a tamper, bad air or an emptied bin, and made to answer through Wi-Fi, Firebase, ThingSpeak or not at all. This shows every alert and link state in class without the hardware.

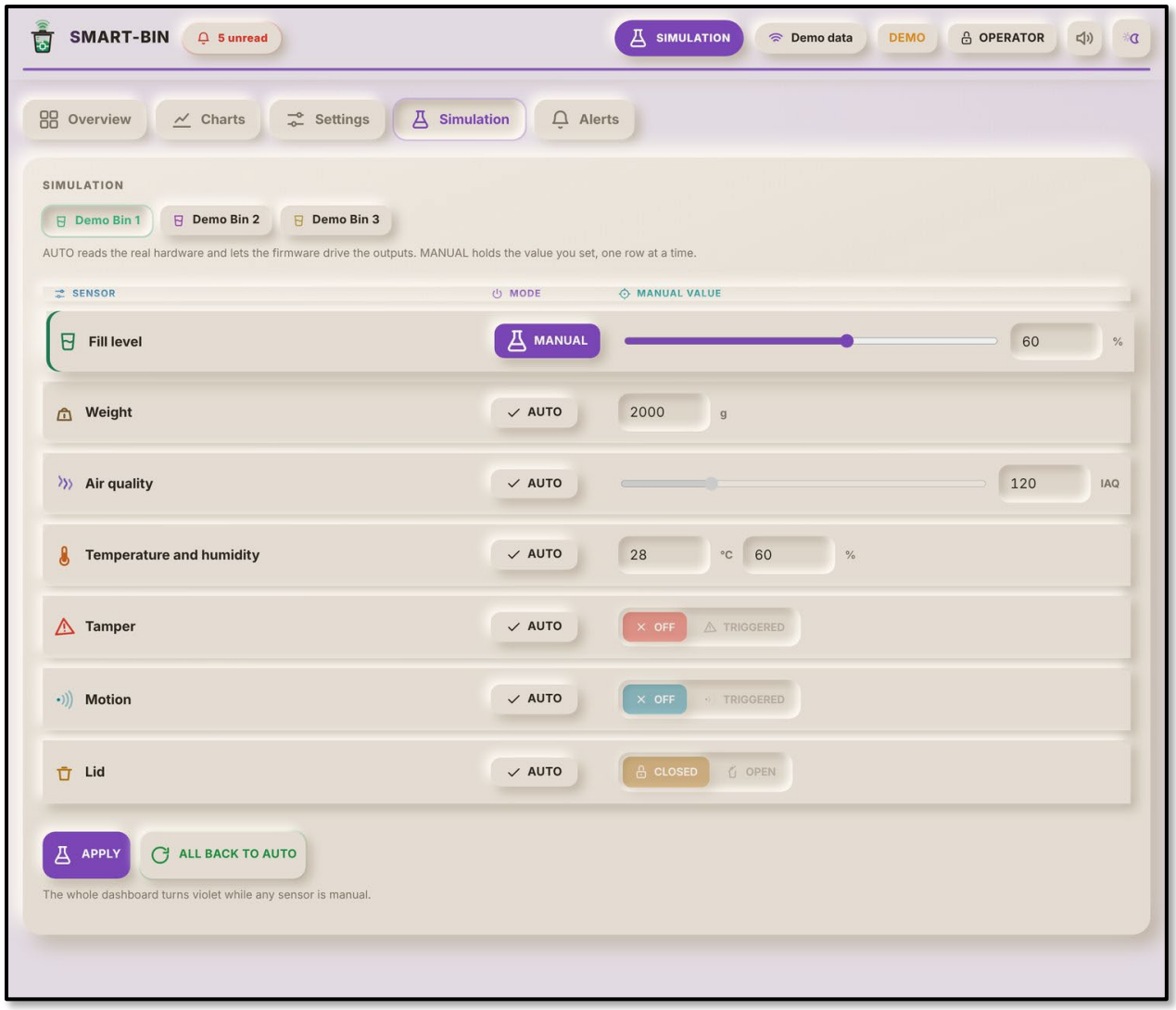
Figure C13

Sound Test Panel

Every sound the dashboard plays is listed by group, from alerts to key clicks. Pressing a key plays it even with sounds switched off. The phone app has the same sounds under the same names.

Figure C14

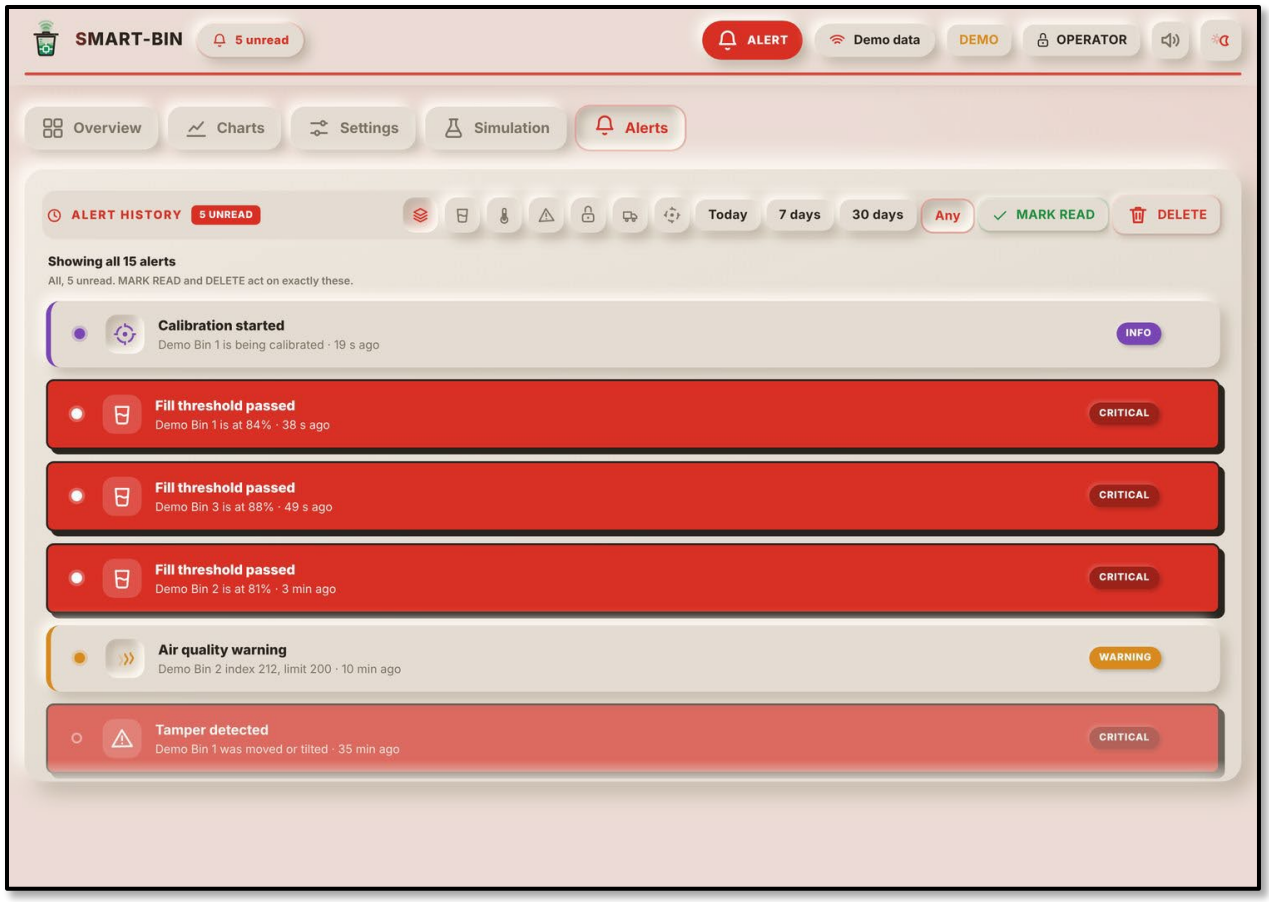
Simulation Tab with One Sensor in Manual



Each sensor row is AUTO, where the bin reads its real hardware, or MANUAL, where it holds the value typed here. In this figure the fill level is held at 60%. APPLY sends the rows to the bin over Wi-Fi, and ALL BACK TO AUTO ends the simulation. While any row is manual the whole dashboard turns violet and the mode chip says SIMULATION, so simulated readings are never mistaken for real ones.

Figure C15

Alerts Tab



The alert history lists every alert with its bin, time and severity (CRITICAL, WARNING, INFO or RESOLVED). The symbols filter by kind (full bins, sensor limits, tamper, lock, pickups and calibration) and the keys beside them by date. MARK READ and DELETE act only on the alerts shown. A dot marks an unread alert.

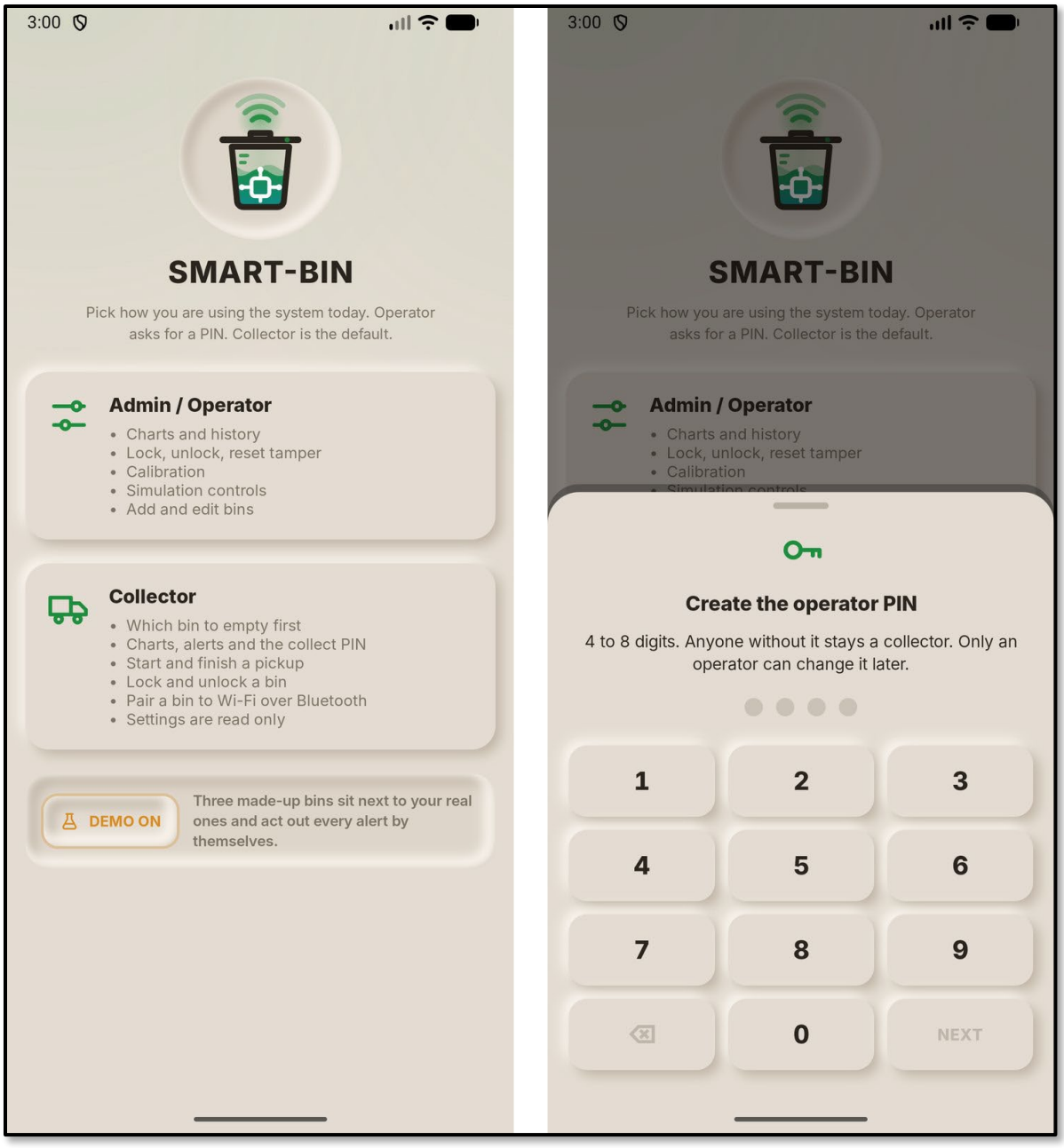
Figure C16

Overview Tab in Dark Theme

The theme key in the header switches between the light and dark themes. Both use the same layout and colors for each status.

Figure C17

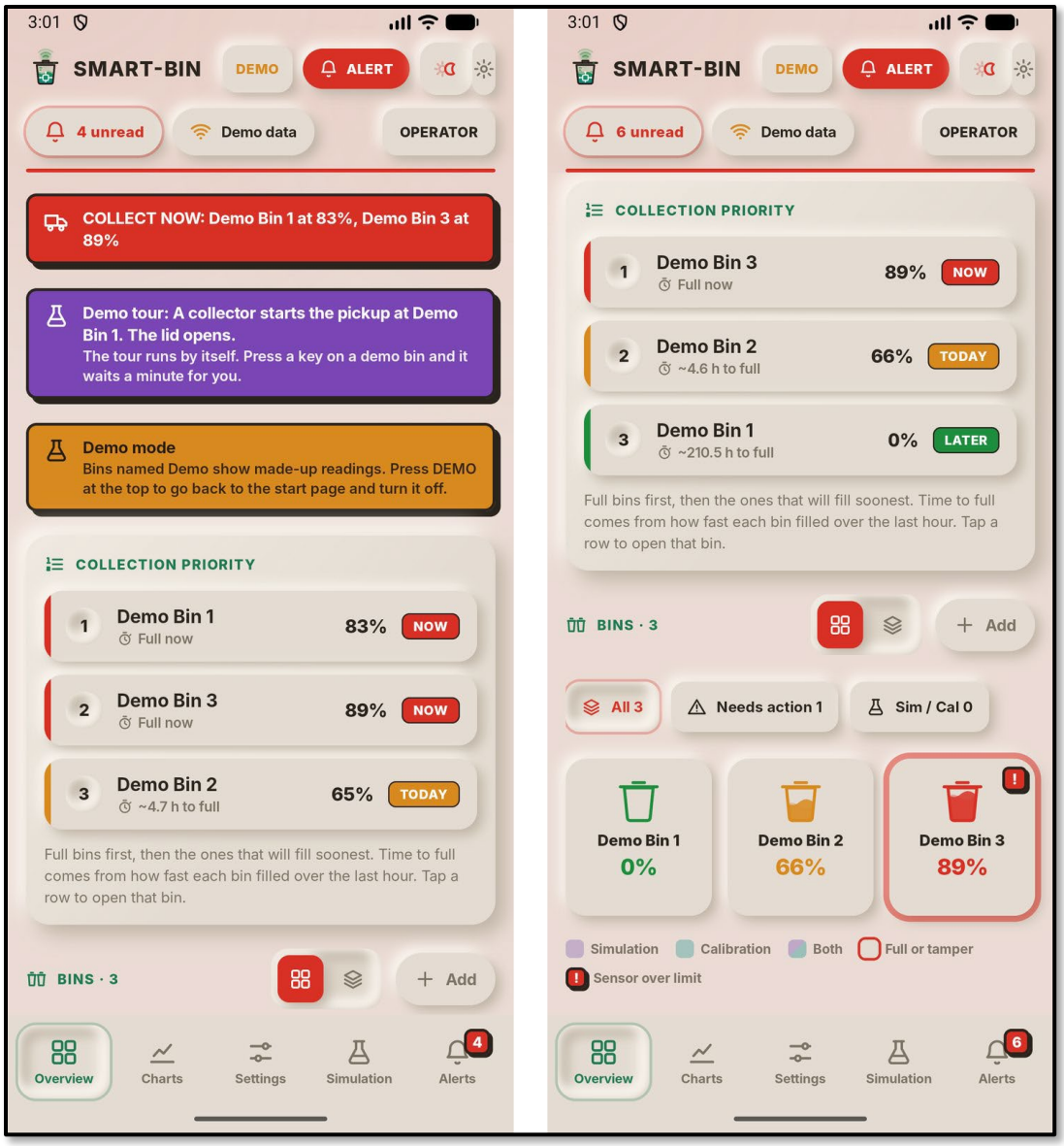
Start Page and Operator PIN in the Android App



The app opens on the same start page as the dashboard, with the Admin / Operator and Collector roles and the DEMO key. The left screen has DEMO on. Picking Admin / Operator the first time opens the sheet on the right, where the operator PIN is typed once and then again to confirm it. The app keeps its own operator PIN, separate from the Collect PIN of the bins.

Figure C18

Overview Tab with Banners, Collection Priority and Bin Grid



Banners at the top list what needs action now: here two bins are full (COLLECT NOW), the demo tour is running, and demo mode is on. Collection Priority orders the bins the same way as the dashboard, with the time to full and the NOW, TODAY and LATER tags. Below it the bins are shown as a grid; the filter keys show all bins, only the bins that need action, or bins in simulation or calibration, and the legend explains the tile colors. The badge on the Alerts tab counts unread alerts.

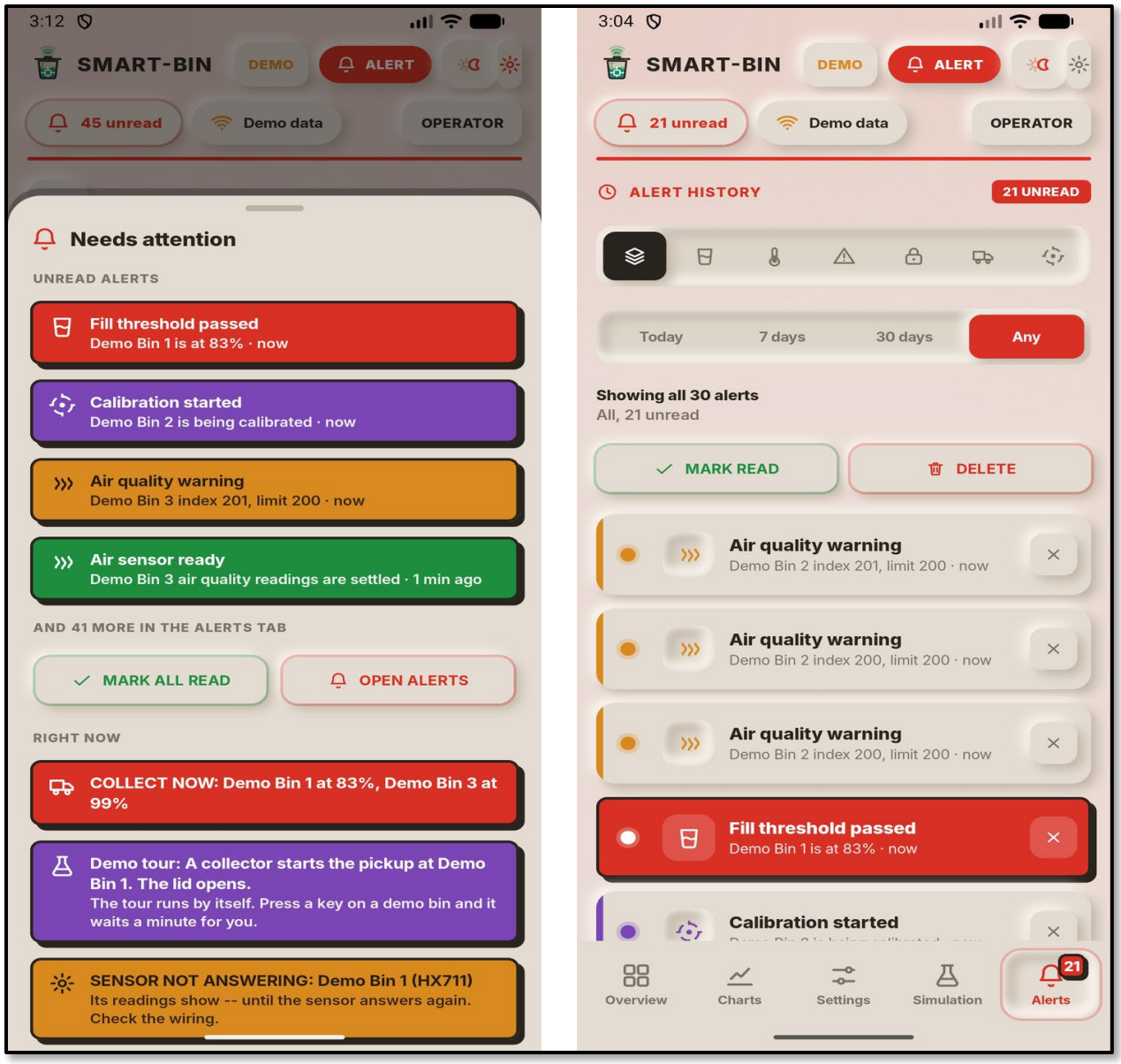
Figure C19

Starting and Finishing a Pickup from a Bin Card

In list view each bin has a full card like the dashboard's. On the left, COLLECT was pressed on Demo Bin 2: the lid chip says OPEN, the status chip says PICKUP, and the COLLECT key turned into DONE. On the right, DONE was pressed: the lid is closed, the status is back to OK and the fill reads 0%. The air quality tile on the left is red because the index (206) is over its limit of 200.

Figure C20

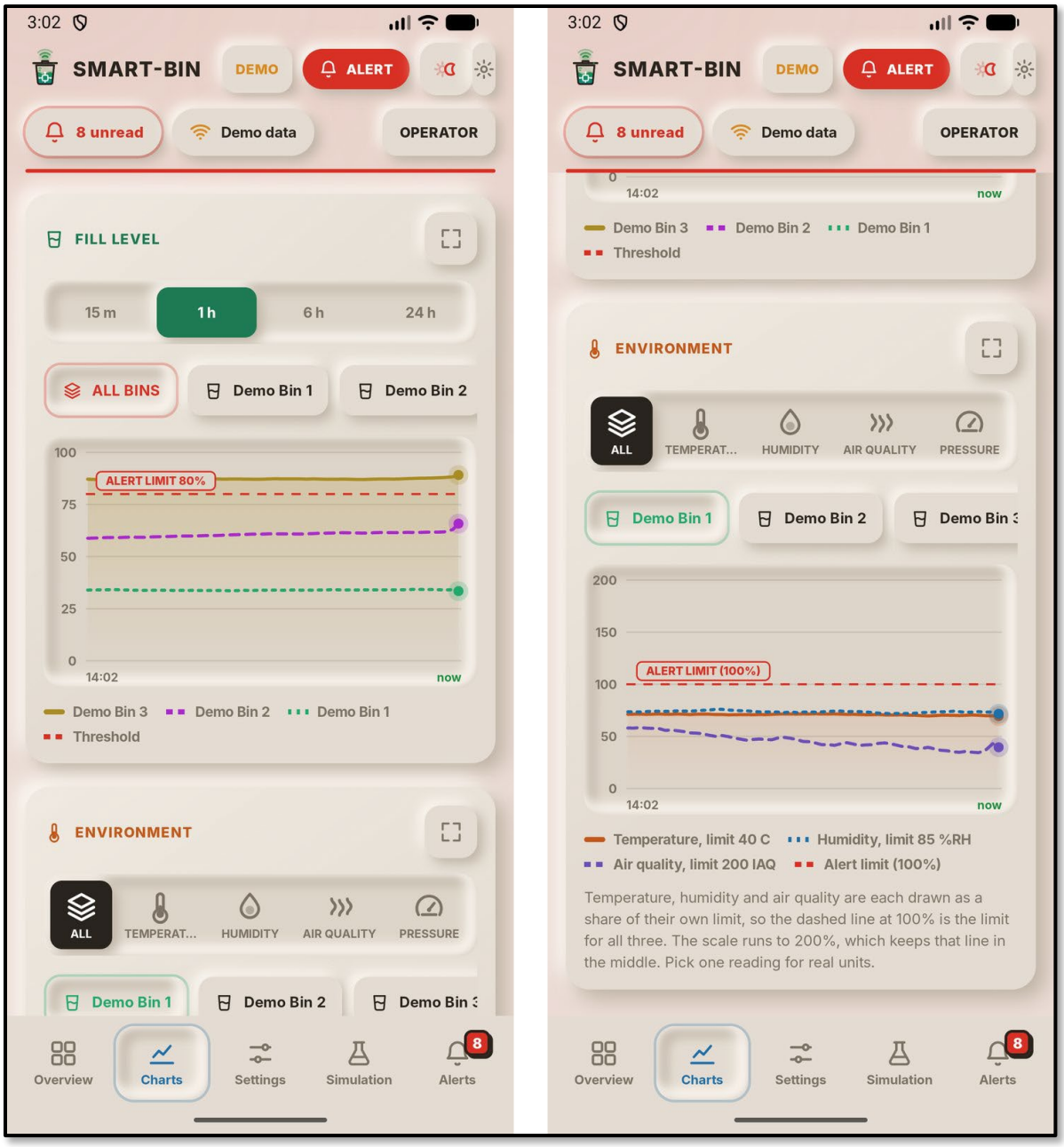
Needs Attention Sheet and Alert History



Tapping the unread pill in the header opens the Needs attention sheet on the left. It lists the unread alerts with their colors, then the conditions that are true right now, with MARK ALL READ and OPEN ALERTS. The Alerts tab on the right keeps the full history, filtered by kind and by date in the same way as the dashboard. MARK READ and DELETE act on the alerts shown, and the cross on a row removes that one alert.

Figure C21

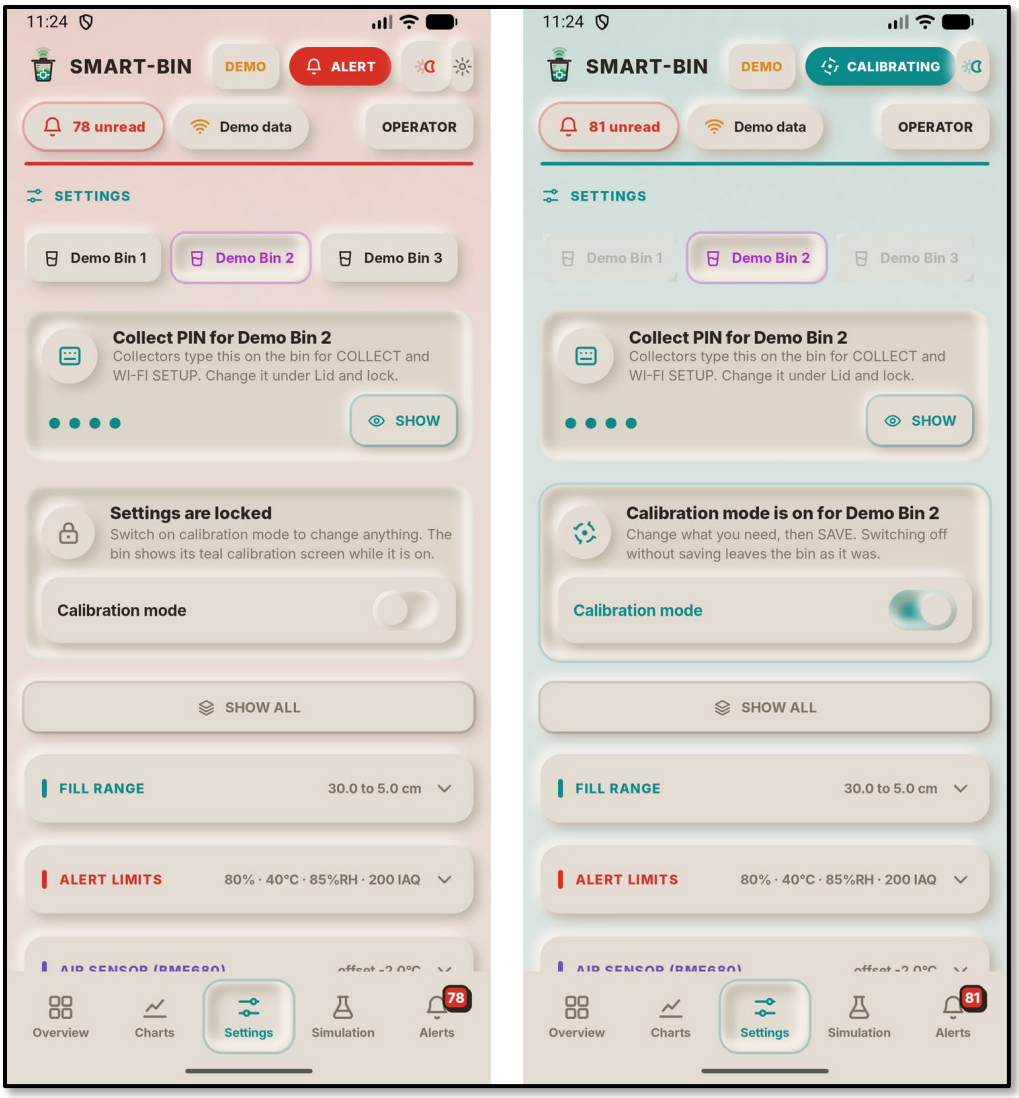
Charts Tab in the Android App



The Fill Level and Environment charts match the dashboard: a time range of 15 min to 24 h, one bin or all bins, and dashed lines for the alert limits. The key at the top right of each chart turns the phone sideways and shows the chart full screen.

Figure C22

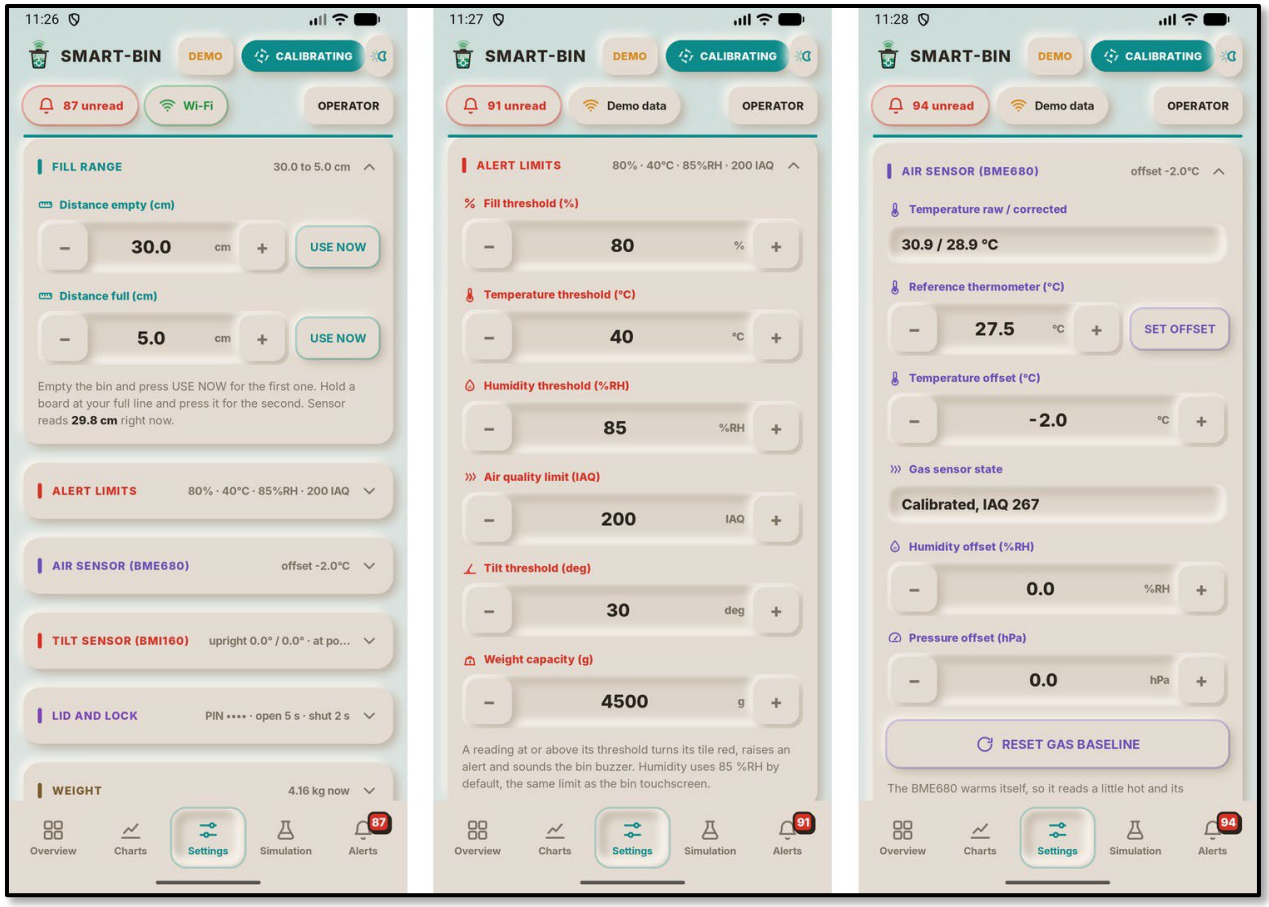
Settings Tab with Calibration Mode Off and On



On the left the settings of Demo Bin 2 are locked; values can be read but not changed. The Collect PIN card at the top shows the PIN the collector types on the bin. On the right an operator has switched on Calibration mode, and the whole app turns teal with CALIBRATING in the header. The other bins are greyed out because only one bin is calibrated at a time. The app reads calibration mode back from the bin, so the dashboard and the bin's screen turn teal at the same moment, whichever of the three started it.

Figure C23

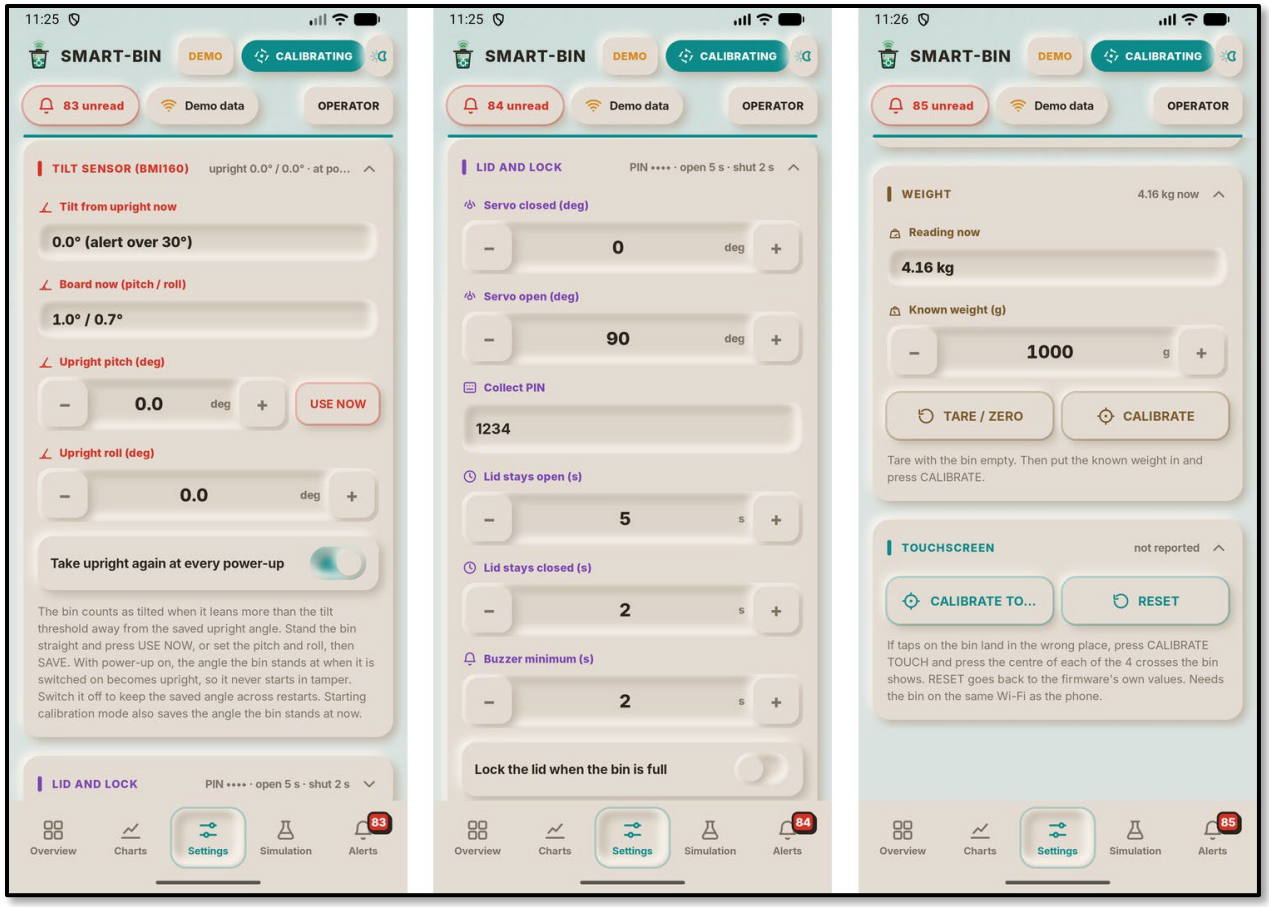
Calibration Settings: Fill Range, Alert Limits and Air Sensor



Each value has minus and plus keys. Fill range sets the distances from the ultrasonic sensor to the bottom of an empty bin and to the full line; USE NOW copies the live reading, which the hint shows (29.8cm here). Alert limits sets the fill, temperature, humidity, air quality and tilt limits and the weight capacity. The air sensor group corrects the BME680: a reference thermometer reading sets the temperature offset, humidity and pressure have their own offsets, and RESET GAS BASELINE restarts the gas sensor's clean-air learning.

Figure C24

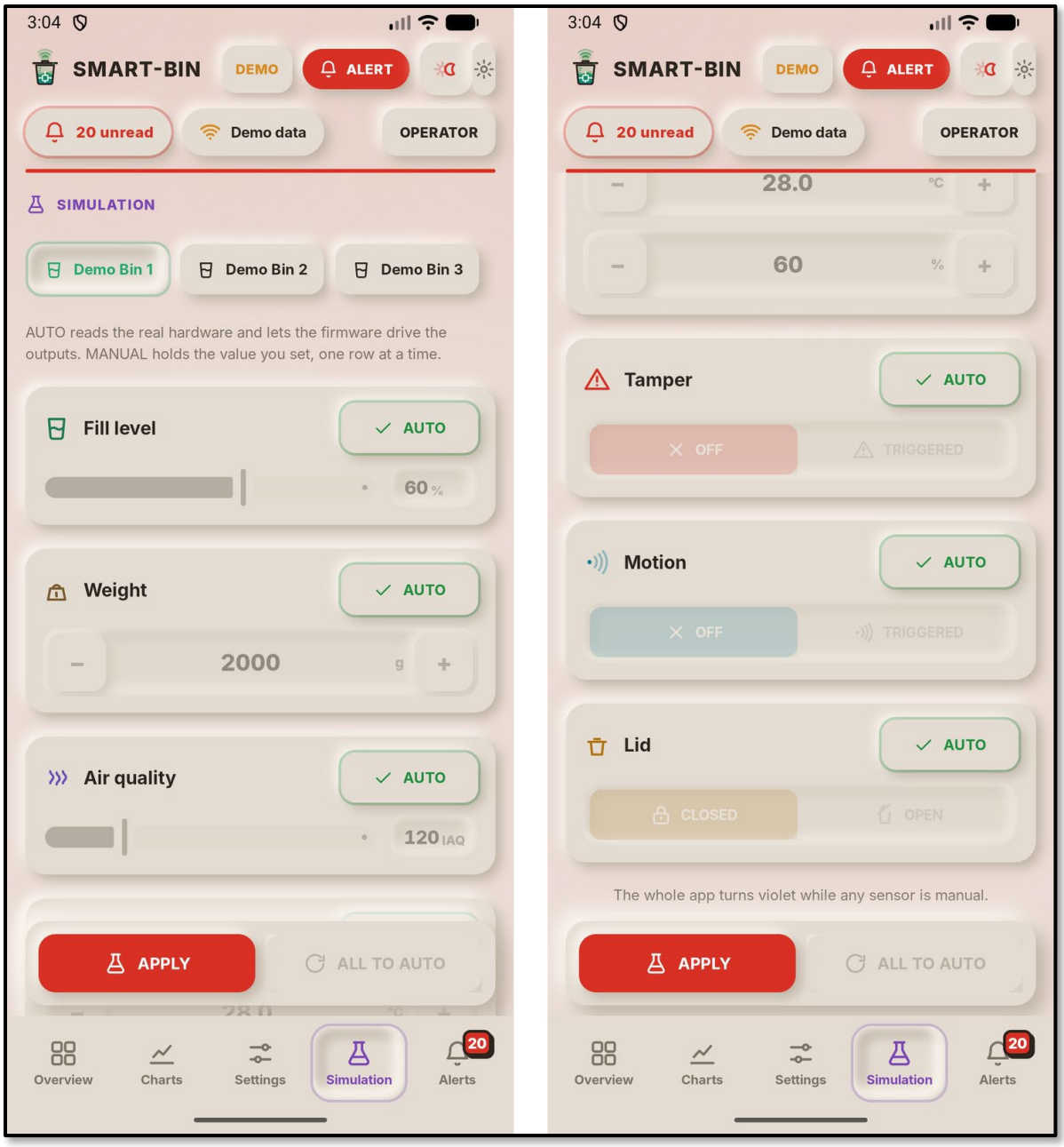
Calibration Settings: Tilt Sensor, Lid, Weight and Touchscreen



Tilt sensor (BMI160) shows how far the bin leans from upright and the board's pitch and roll right now. The saved upright pitch and roll can be typed in or taken from the live reading with USE NOW, and the switch below takes the upright again every time the bin is powered up, which is on by default. Lid and lock holds the servo angles, the Collect PIN, how long the lid stays open after motion and how long it then stays closed (5s and 2s), the minimum buzzer time and the option to lock the lid when the bin is full. Weight calibration is done with TARE / ZERO on the empty bin, then CALIBRATE with a known weight inside (1000g here). Touchscreen starts the four-cross touch calibration on the bin or resets it.

Figure C25

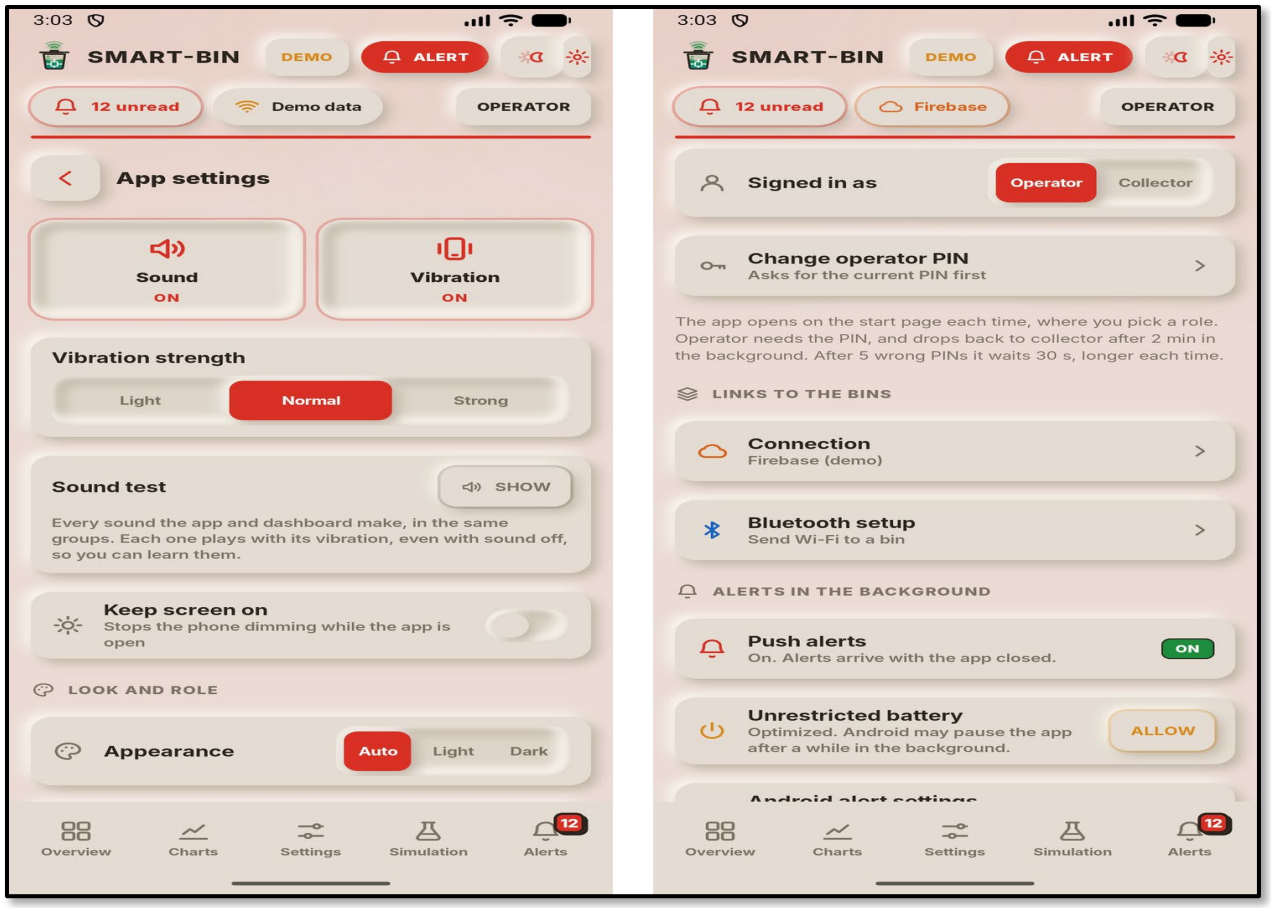
Simulation Tab in the Android App



Each sensor of the selected bin can be set to MANUAL with its own value, as on the dashboard. APPLY sends the values to the bin over Wi-Fi and ALL TO AUTO ends the simulation. While any sensor is manual the whole app turns violet.

Figure C26

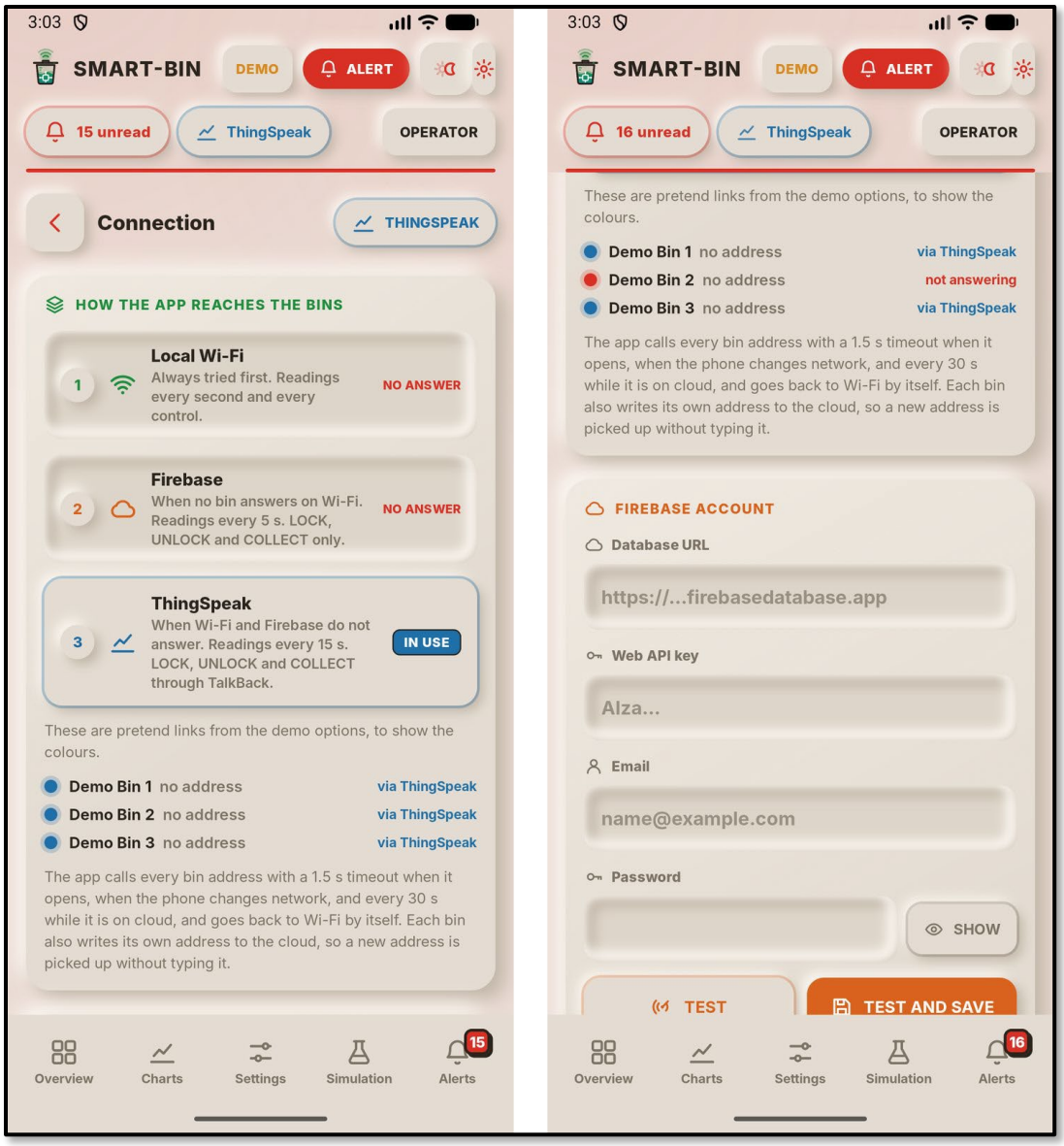
App Settings



The gear key opens the settings of the phone app itself. Sound and Vibration turn the key sounds and haptics on or off, with three vibration strengths and a sound test. Appearance follows the phone (Auto) or forces light or dark. The second screen shows the signed-in role, the operator PIN, the Connection and Bluetooth setup pages, and the background options: push alerts while the app is closed, and unrestricted battery so Android does not pause the app.

Figure C27

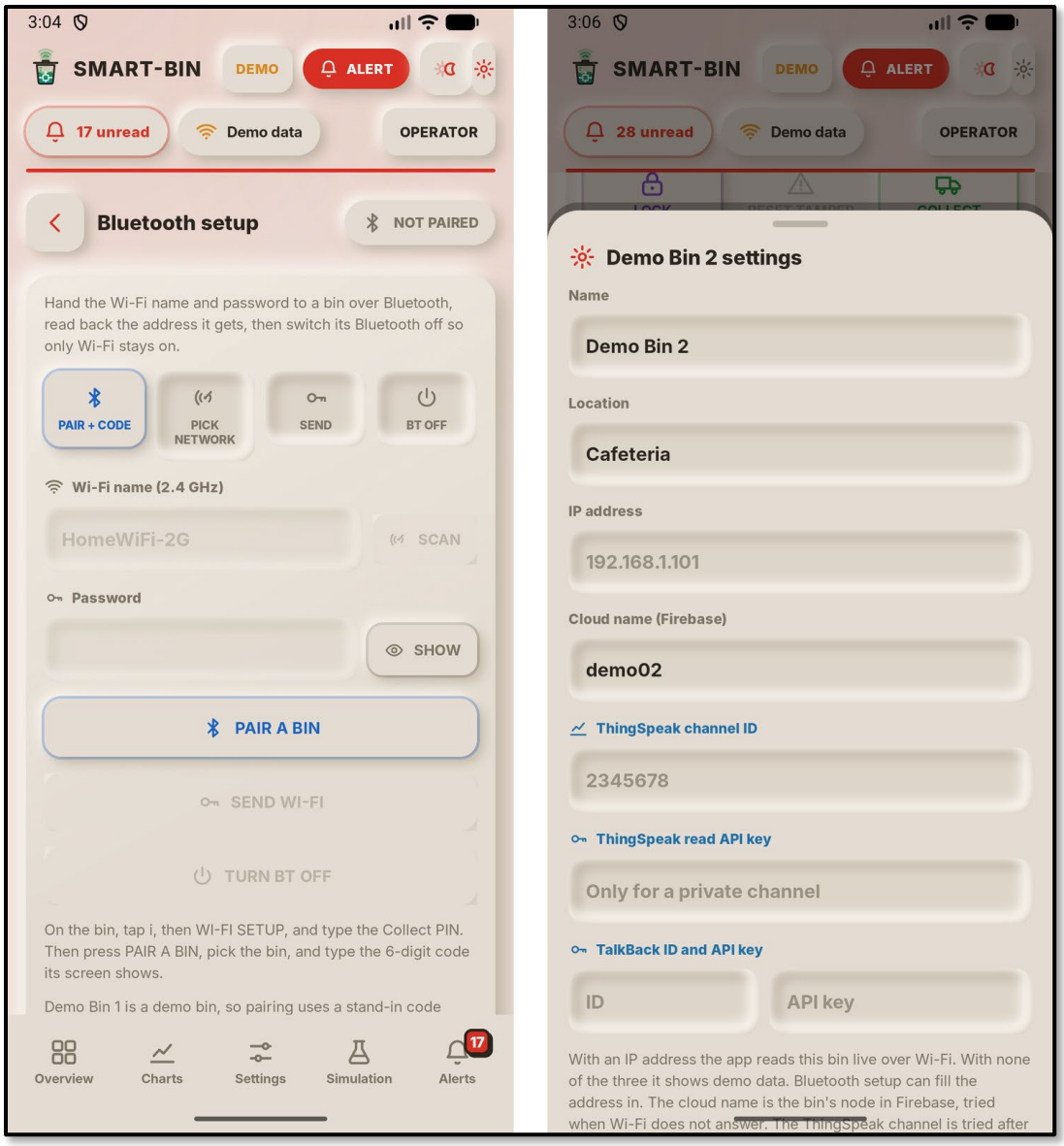
Connection Page



This page shows how the app reaches the bins, in order: local Wi-Fi is always tried first and read every second; Firebase is used when no bin answers on Wi-Fi and is read every 5 s; ThingSpeak is the last route, read every 15 s, with LOCK, UNLOCK and COLLECT sent through TalkBack. Over the clouds only those three commands are allowed. In this figure the demo options pretend that Wi-Fi and Firebase are down, so ThingSpeak is IN USE and the header chip turns blue. The second screen lists the route each bin is using and the Firebase account fields.

Figure C28

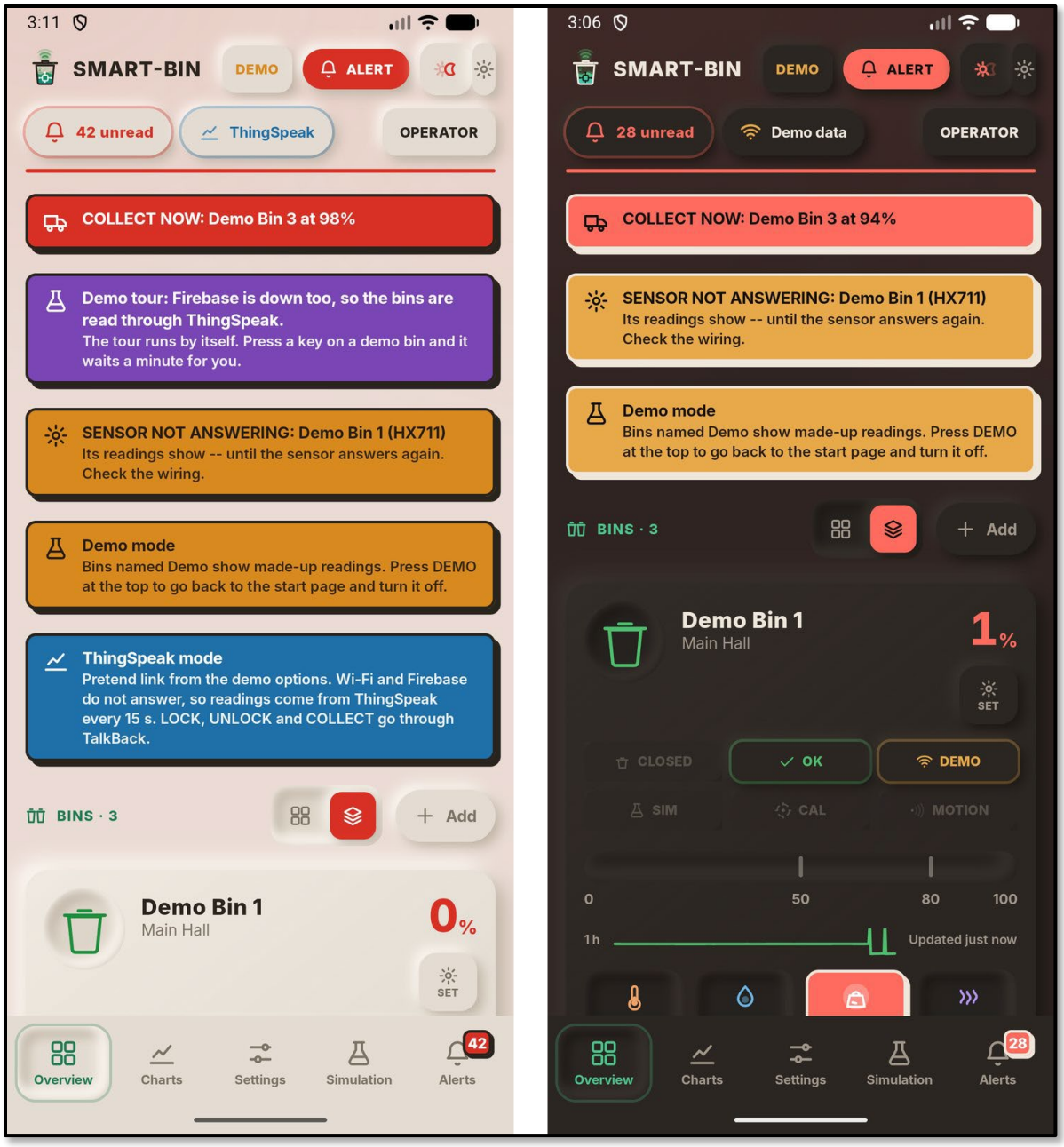
Bluetooth Setup Page and Bin Settings Sheet



Bluetooth setup gives a bin a new Wi-Fi network: PAIR A BIN, type the 6-digit code shown on the bin's screen, pick the network, enter its password and SEND WI-FI, then turn the bin's Bluetooth off. With a demo bin the app shows a stand-in for the bin's screen. The sheet on the right opens from a bin's SET key and holds its name, location, IP address, Firebase cloud name and ThingSpeak keys. The values shown are the demo bin's placeholders, not real addresses or keys.

Figure C29

Overview in List View, Light and Dark Themes



The same Overview in list view in the light theme (left) and the dark theme (right). Appearance in App settings picks the theme, or follows the phone. The banners, chips and tiles keep the same colors in both themes.

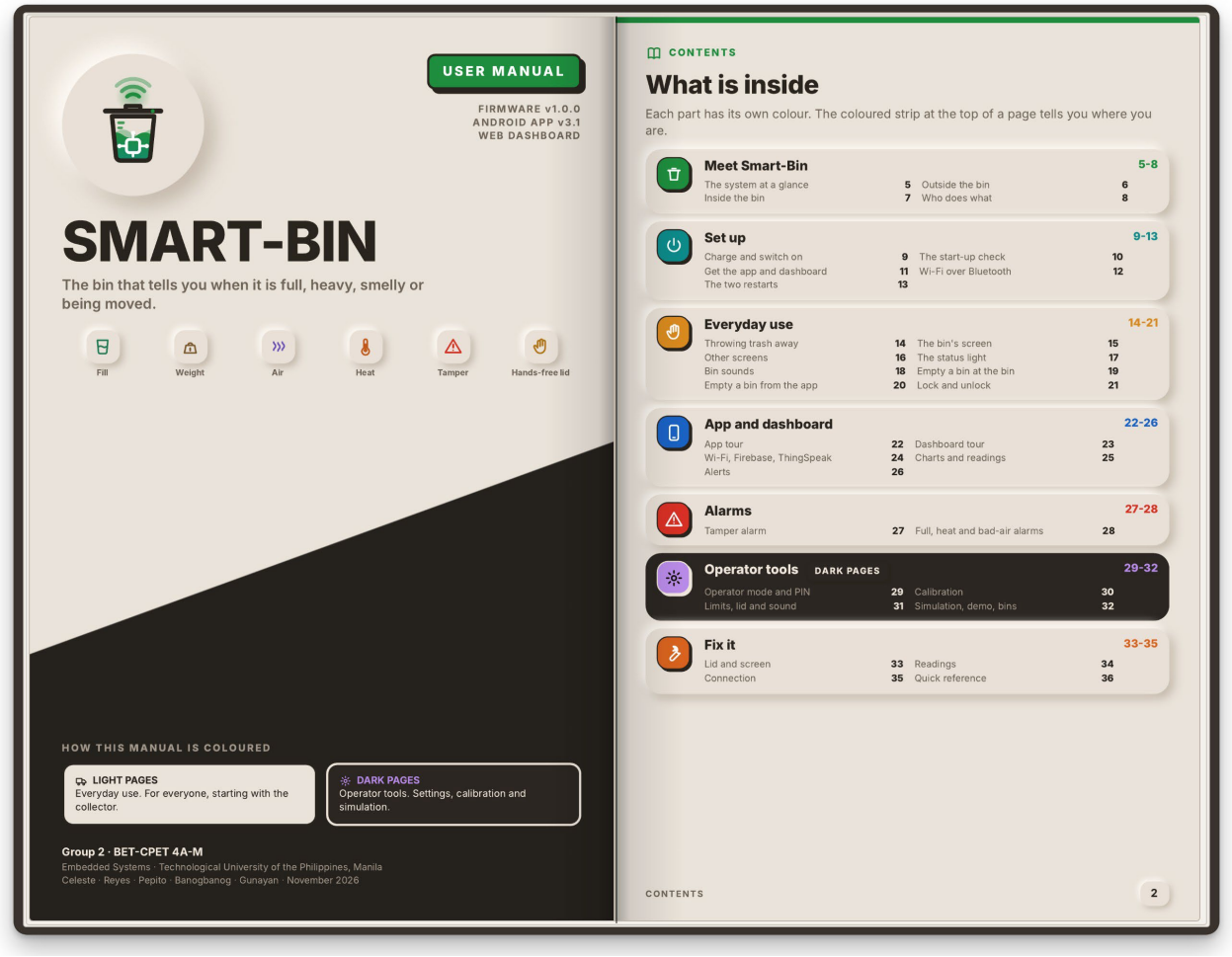
Appendix D

User Manual

This appendix reproduces the Smart-Bin User Manual, the printed guide for the people who use the bins. The manual has 36 pages of 5.5 by 8.5in, half of a US Letter sheet, so each figure shows one opened spread of two facing pages. Light pages cover everyday use for collectors and everyone else, and dark pages hold the operator tools. Every screen, value and step in it follows the final system: firmware v1.0.0, Android app v3.1 and the web dashboard.

Figure D1

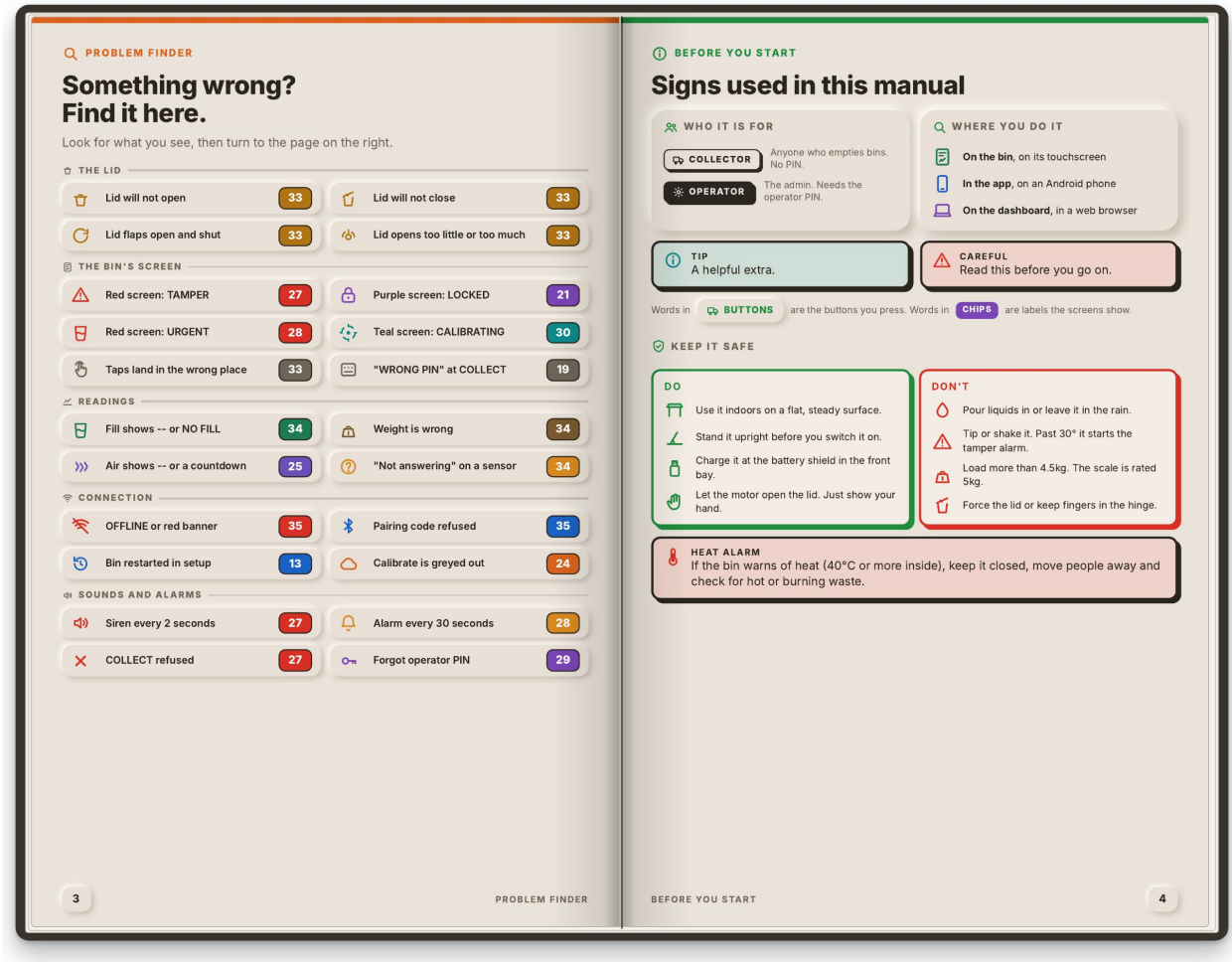
User Manual Pages 1 and 2: Cover and Contents



The cover splits light and dark to show the two kinds of pages, and the contents list every section by color with its page range.

Figure D2

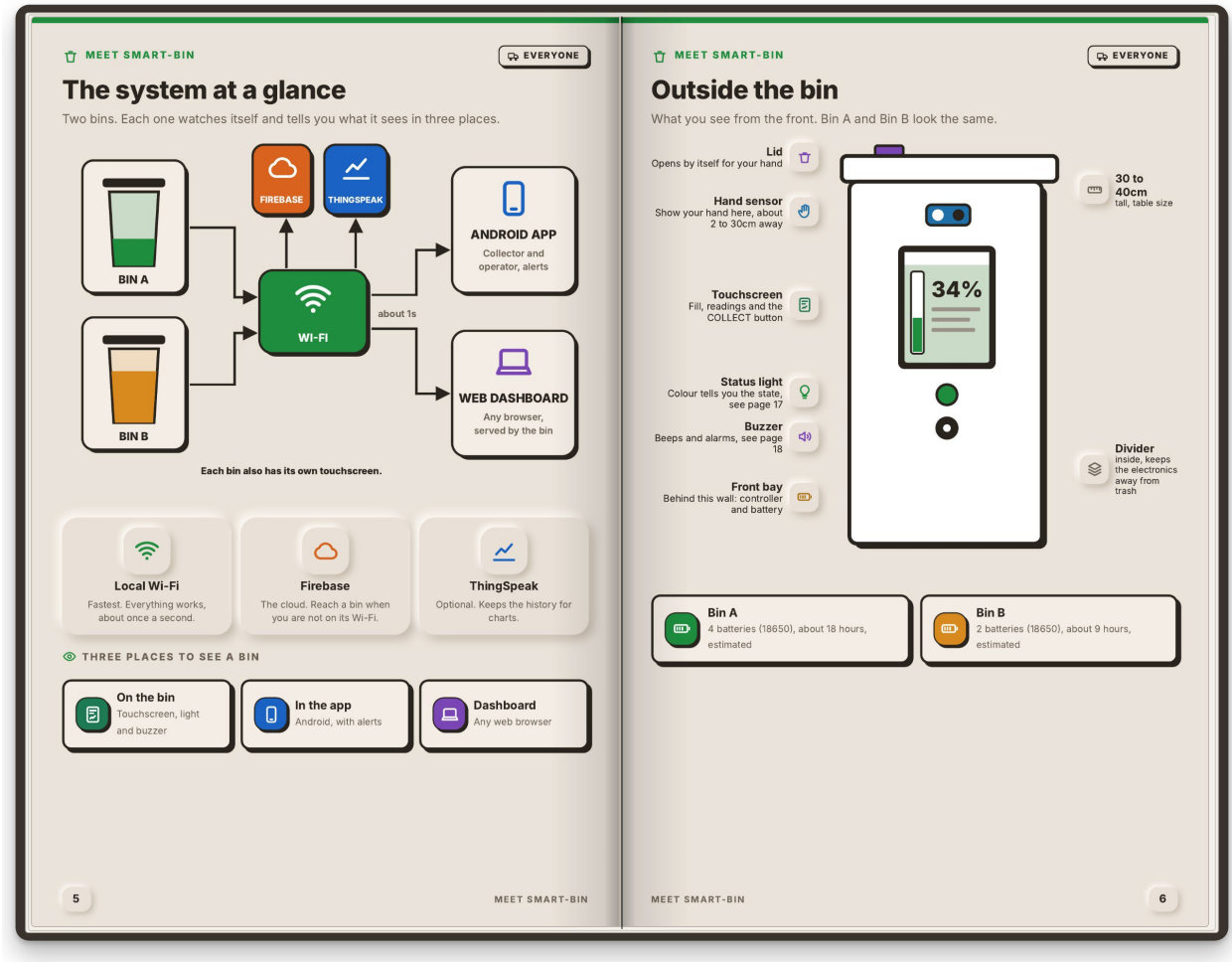
User Manual Pages 3 and 4: Problem Finder and Signs Used



The problem finder sends a user straight to the right page from a symptom, and the signs page explains the icons, role badges and safety notes used throughout.

Figure D3

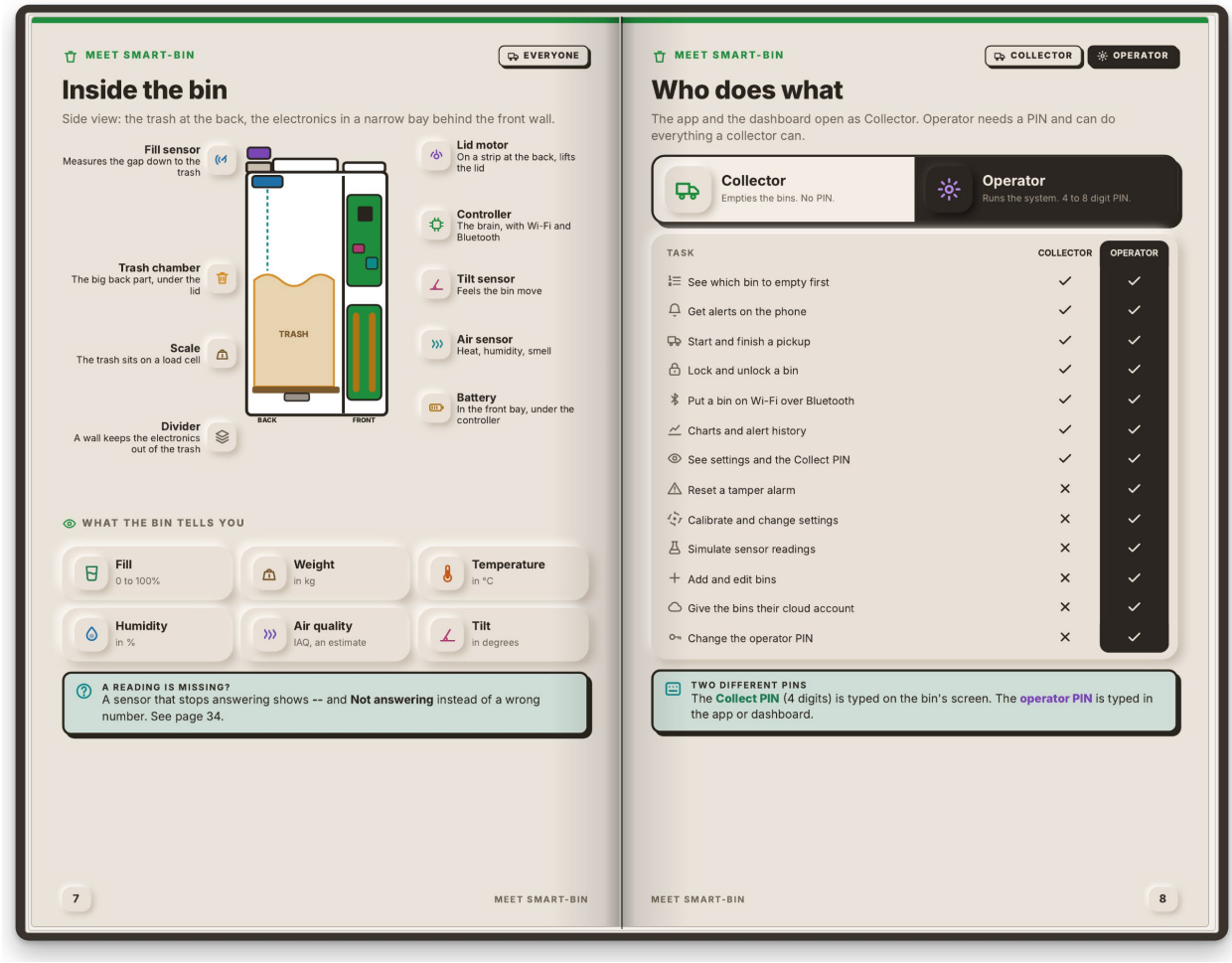
User Manual Pages 5 and 6: The System at a Glance and Outside the Bin



The first section shows how the bins, Wi-Fi, Firebase, ThingSpeak, the app and the dashboard connect, then labels each part on the outside of a bin.

Figure D4

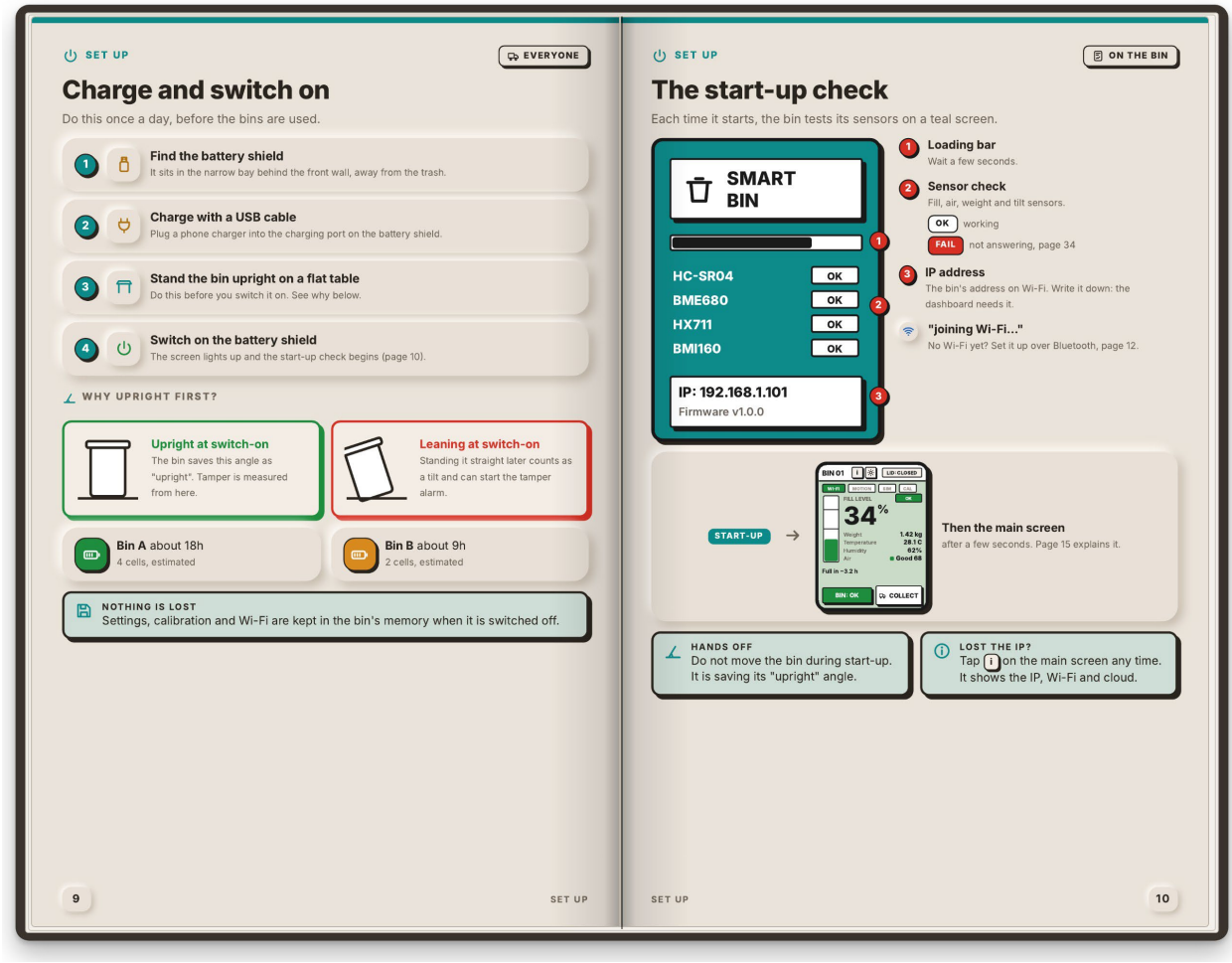
User Manual Pages 7 and 8: Inside the Bin and Who Does What



The inside view names the sensors and the electronics bay, and the role table compares what a collector and an operator can do.

Figure D5

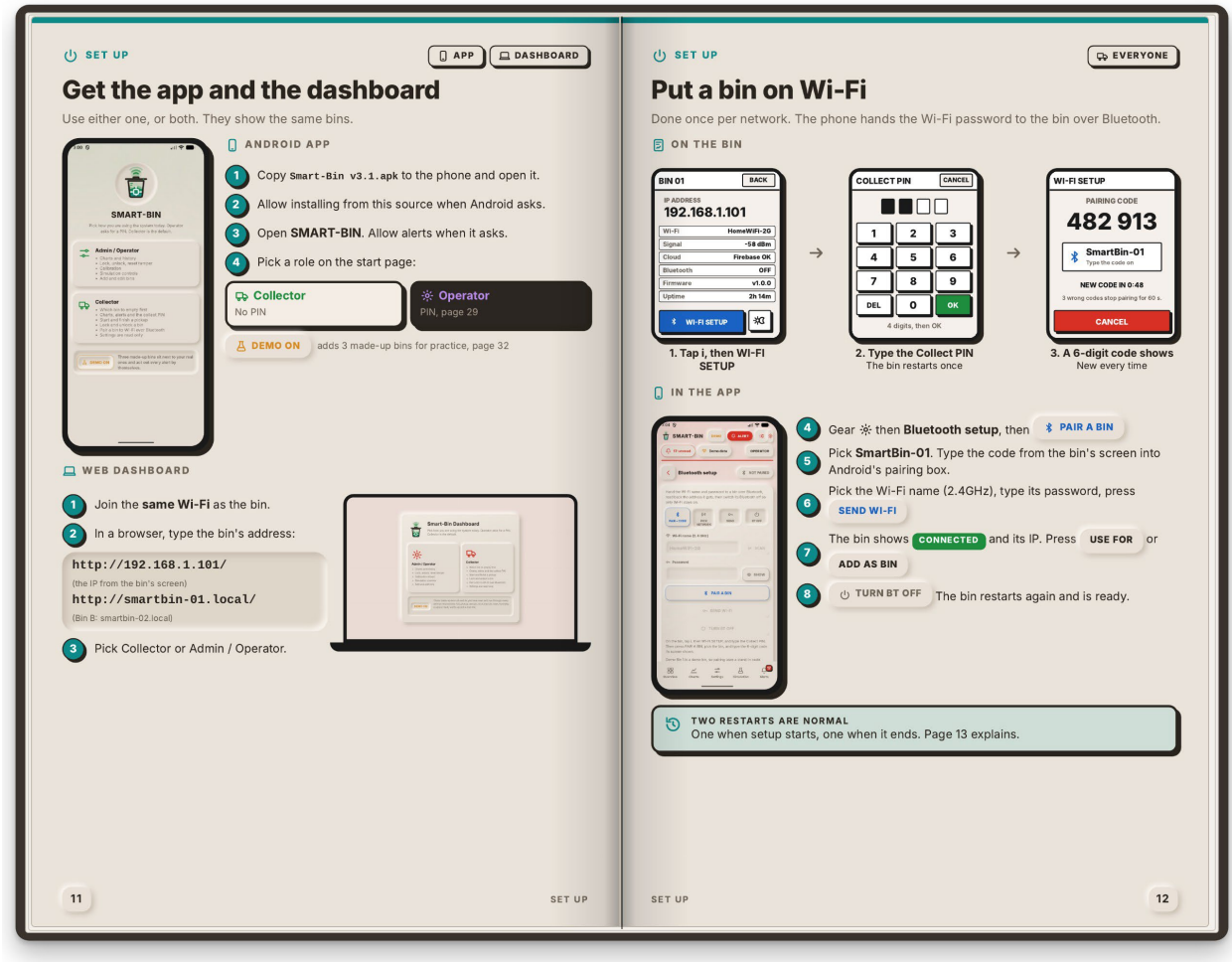
User Manual Pages 9 and 10: Charge and Switch On, and the Start-Up Check



The set-up section starts with charging and switching on, then shows the start-up screen and what each sensor check means.

Figure D6

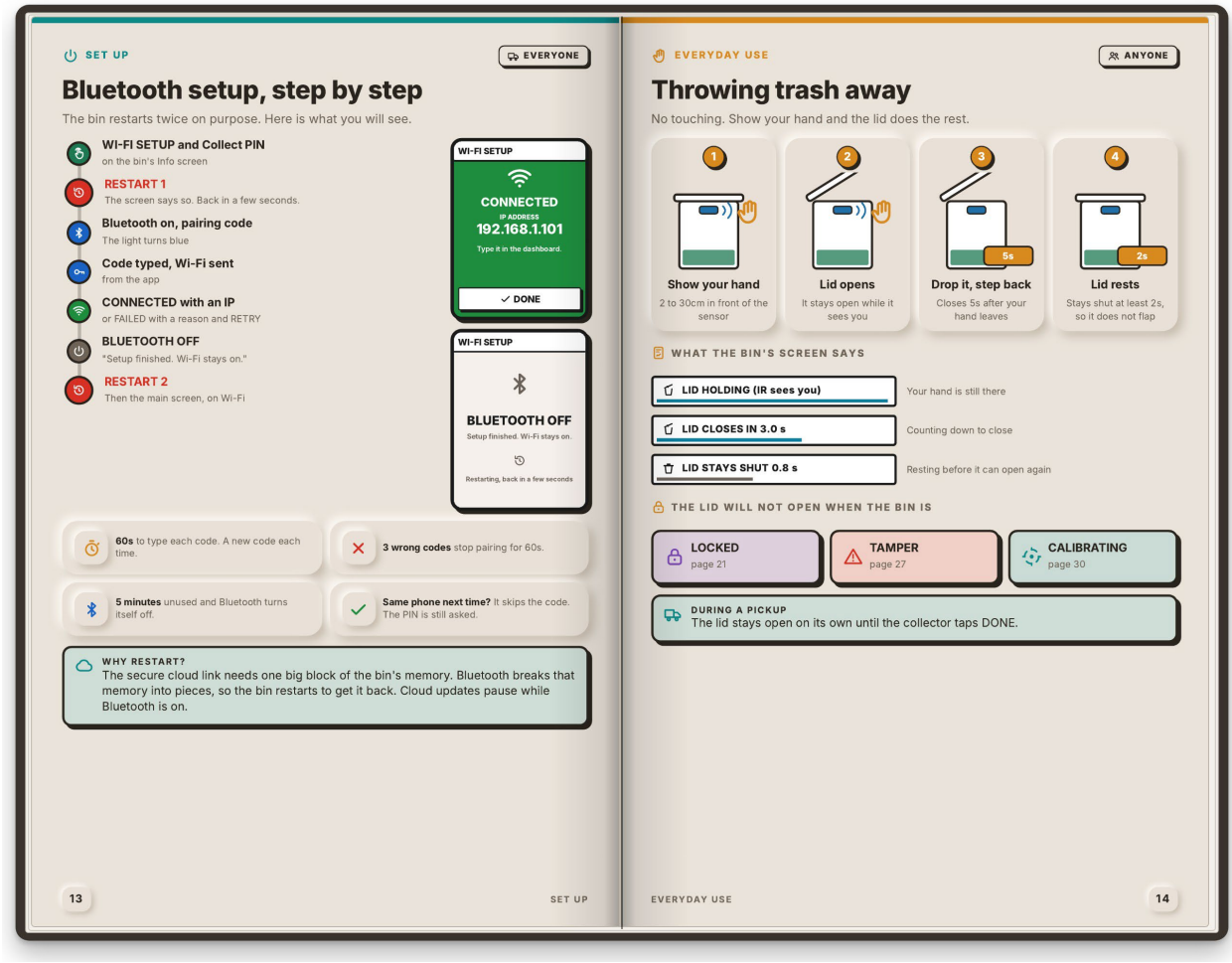
User Manual Pages 11 and 12: Getting the App and Dashboard, and Putting a Bin on Wi-Fi



These pages show where to get the Android app and open the dashboard, and how a bin joins Wi-Fi over Bluetooth.

Figure D7

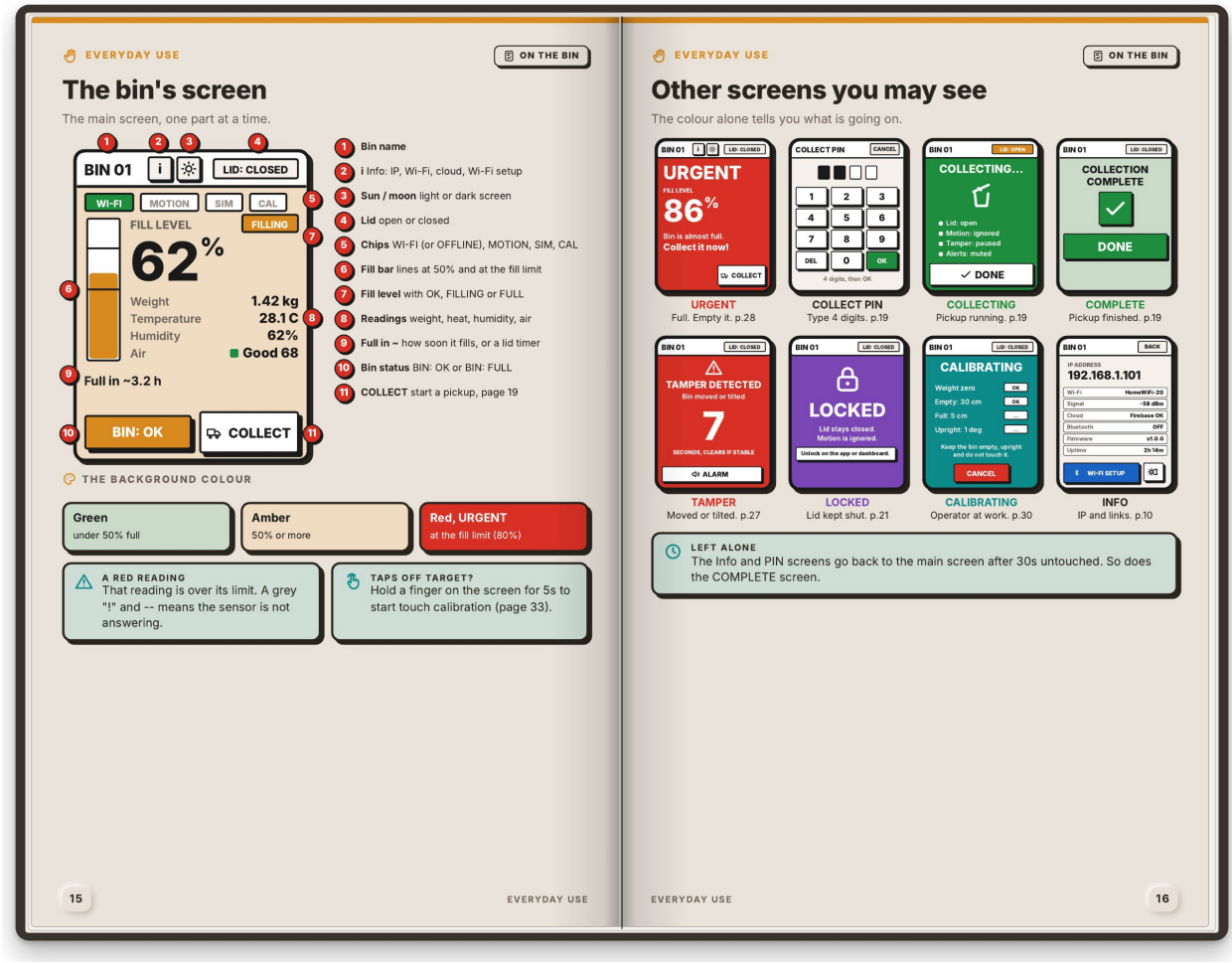
User Manual Pages 13 and 14: Bluetooth Setup and Throwing Trash Away



The Bluetooth steps include the two restarts a bin makes during setup, and the everyday section opens with the hands-free lid.

Figure D8

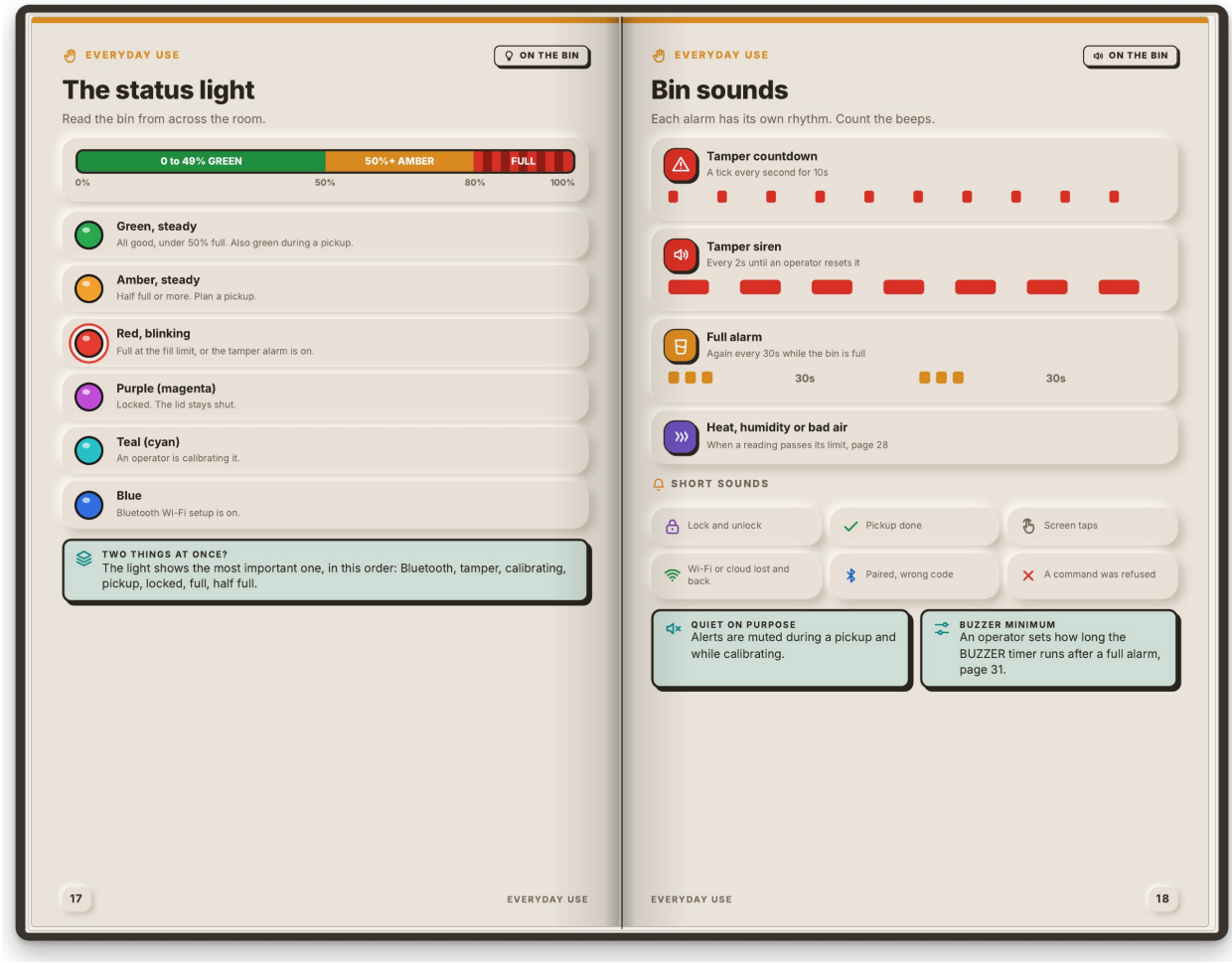
User Manual Pages 15 and 16: The Bin's Screen and Other Screens



The touchscreen pages are drawn from the firmware: the main screen with its labeled parts, then the urgent, PIN, pickup, tamper, locked and calibration screens.

Figure D9

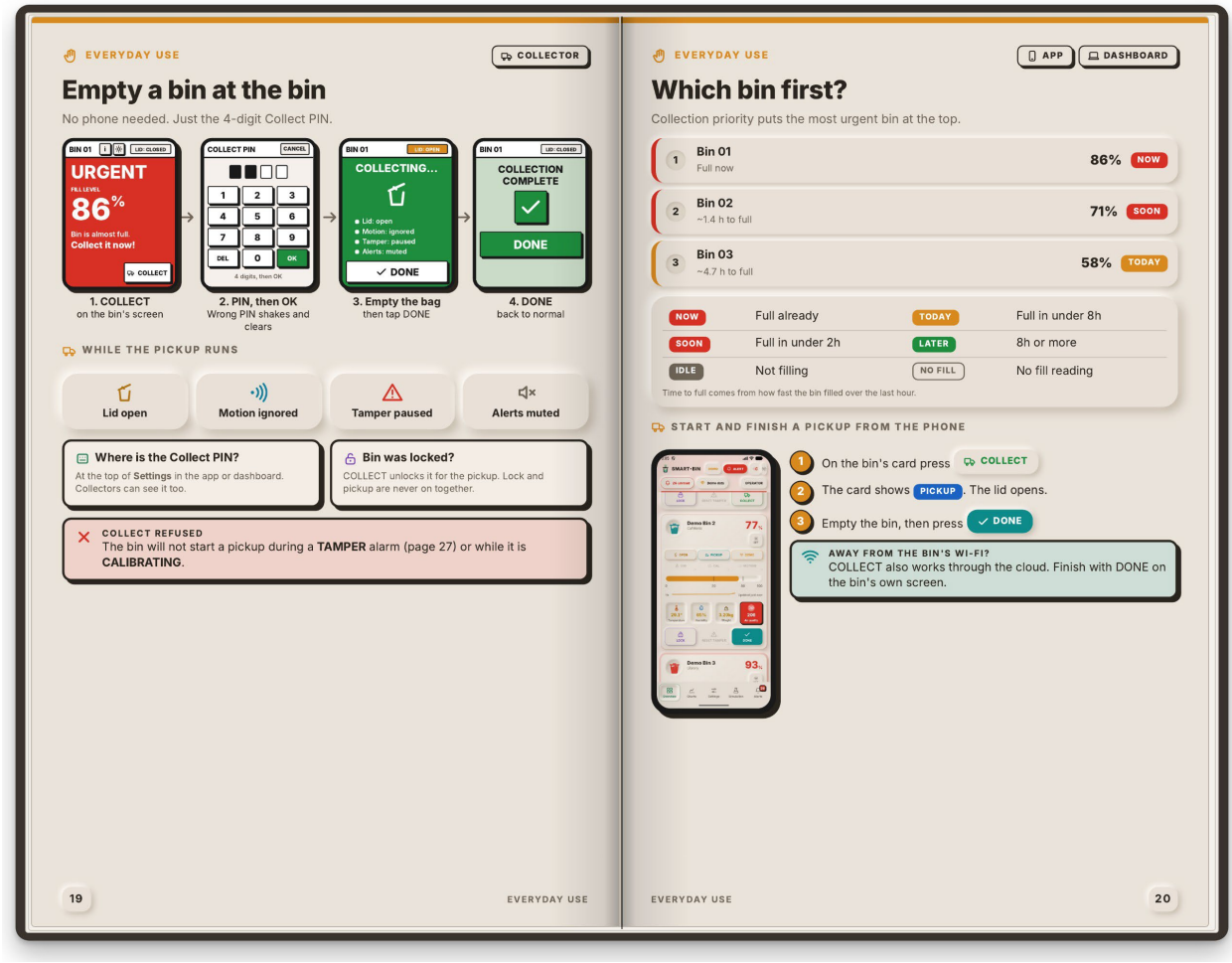
User Manual Pages 17 and 18: The Status Light and Bin Sounds



The status light page gives each LED color and its priority order, and the sounds page shows the rhythm of every alarm.

Figure D10

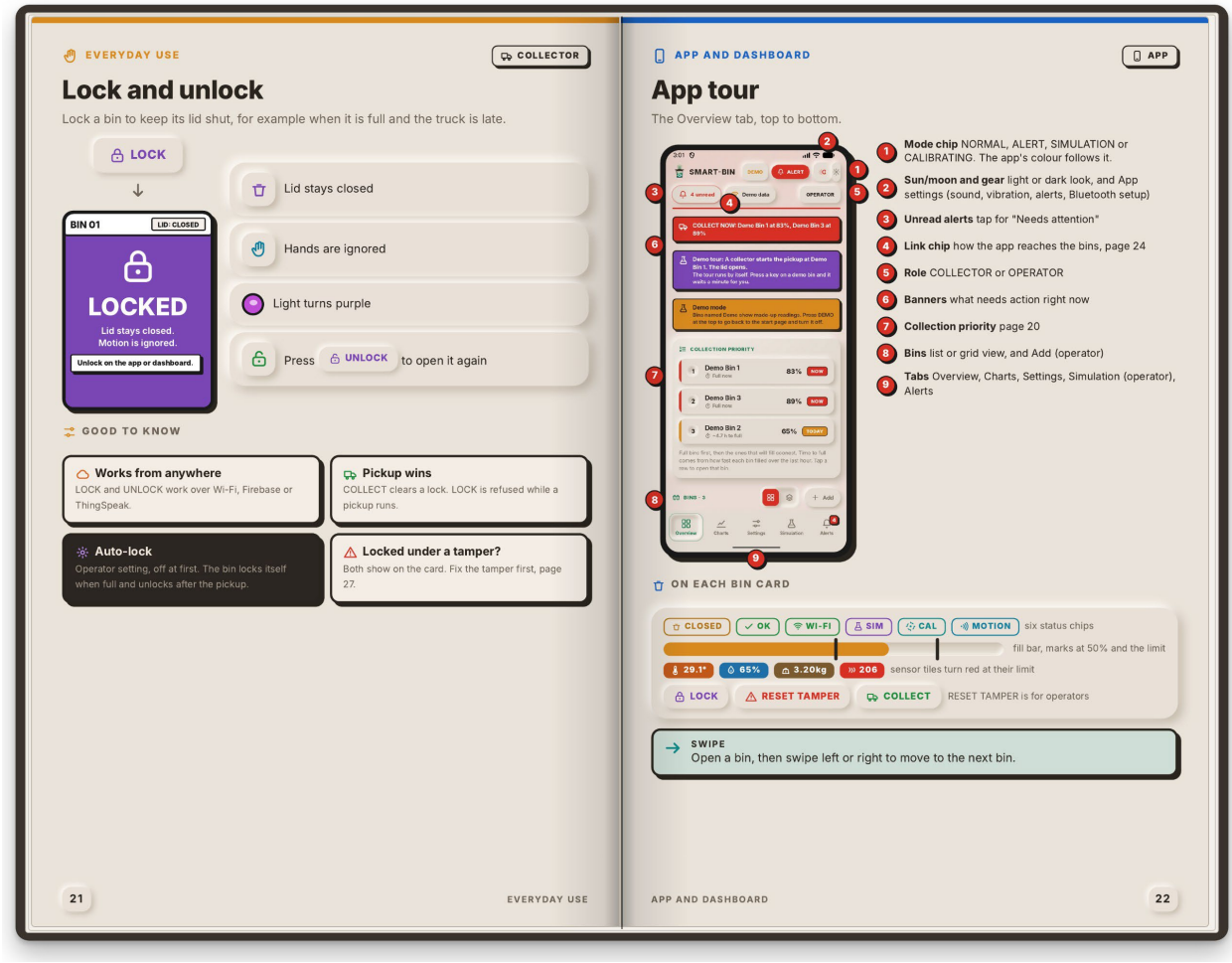
User Manual Pages 19 and 20: Emptying a Bin at the Bin and Collection Priority



Pickup at the bin uses the Collect PIN on the touchscreen, and the next page explains the NOW, SOON, TODAY, LATER and IDLE tags.

Figure D11

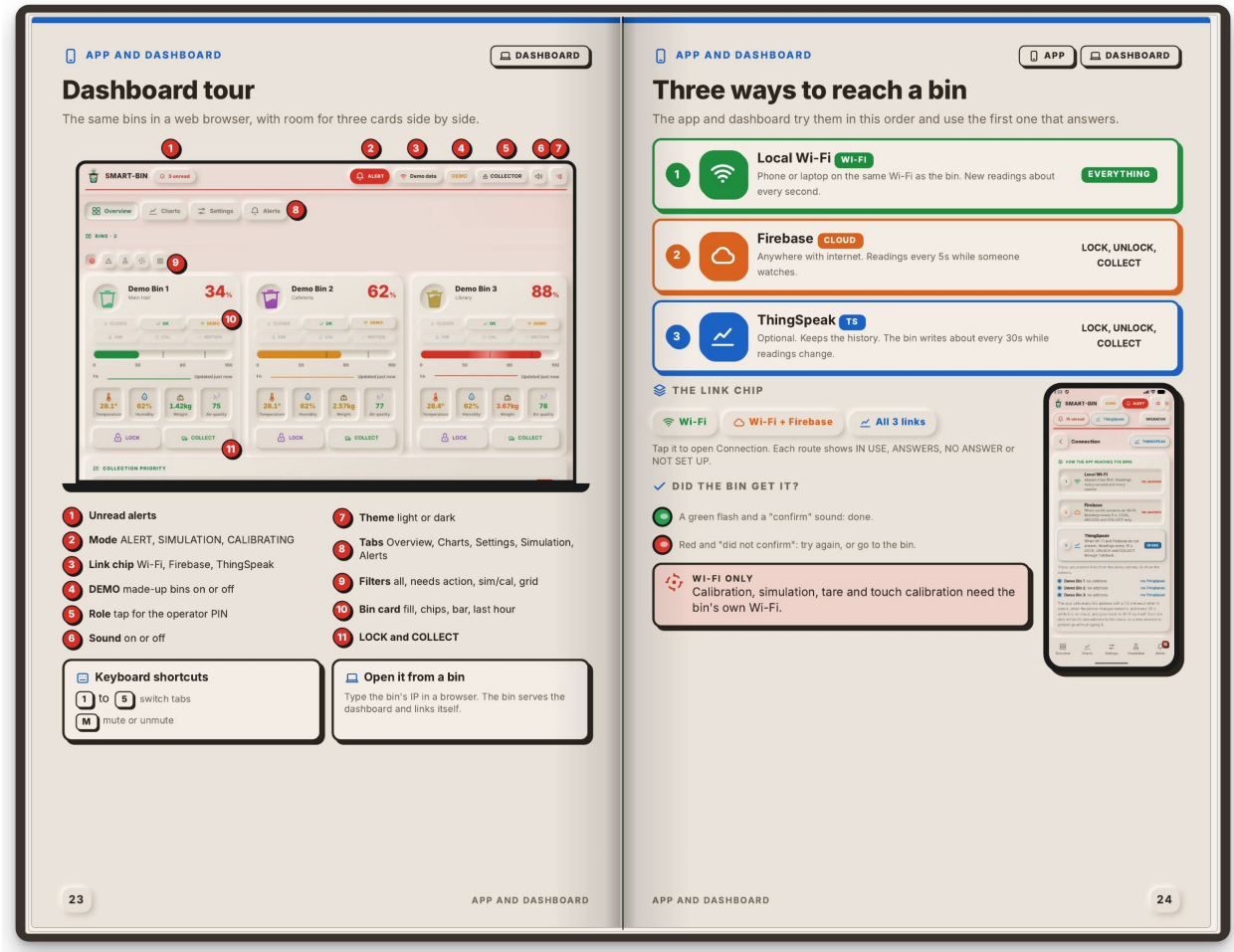
User Manual Pages 21 and 22: Lock and Unlock, and the App Tour



The lock page covers locking from the screen, the app and the dashboard, and the app tour numbers each part of the Overview tab.

Figure D12

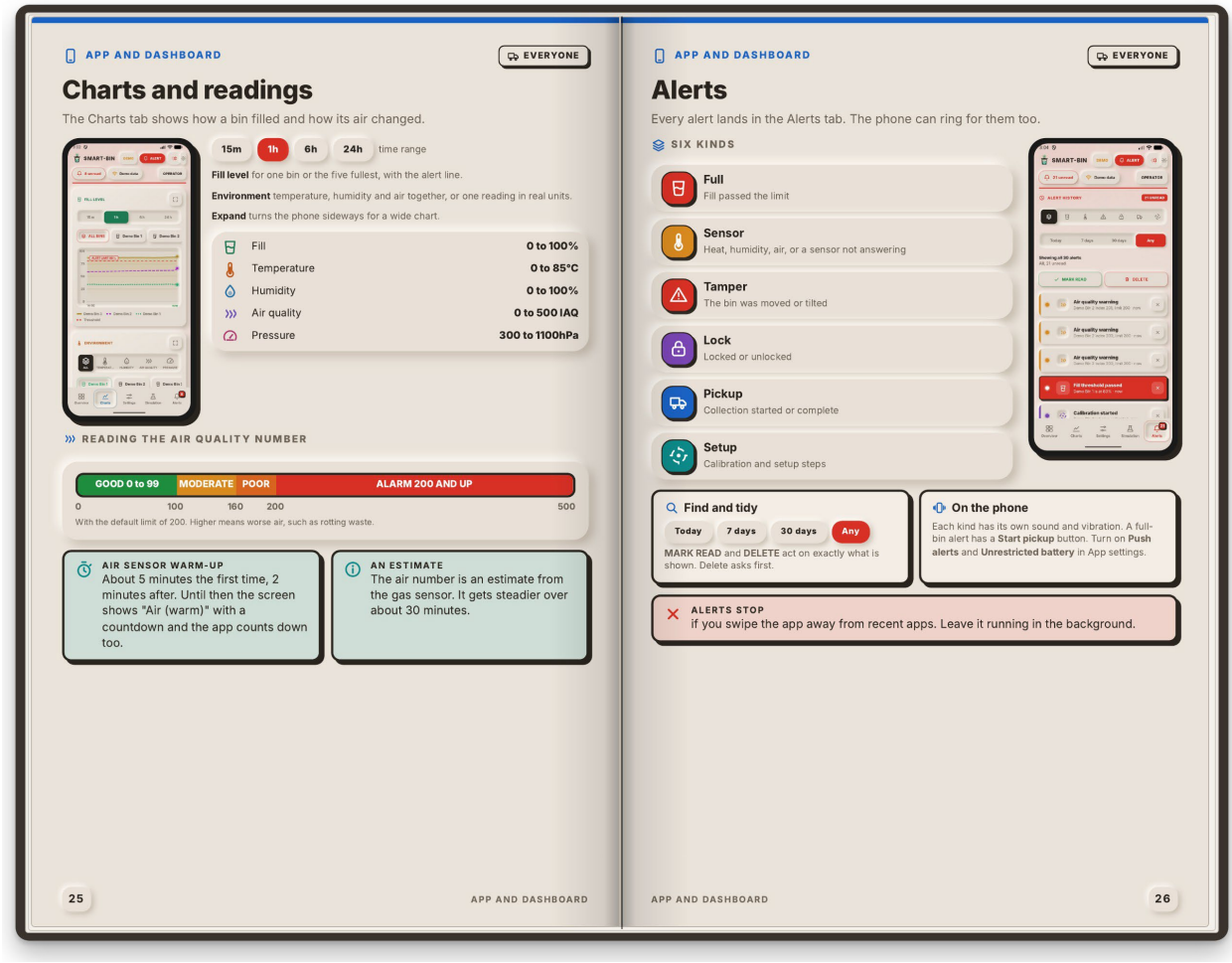
User Manual Pages 23 and 24: Dashboard Tour and Three Ways to Reach a Bin



The dashboard tour labels the header, tabs and bin cards, and the next page compares local Wi-Fi, Firebase and ThingSpeak.

Figure D13

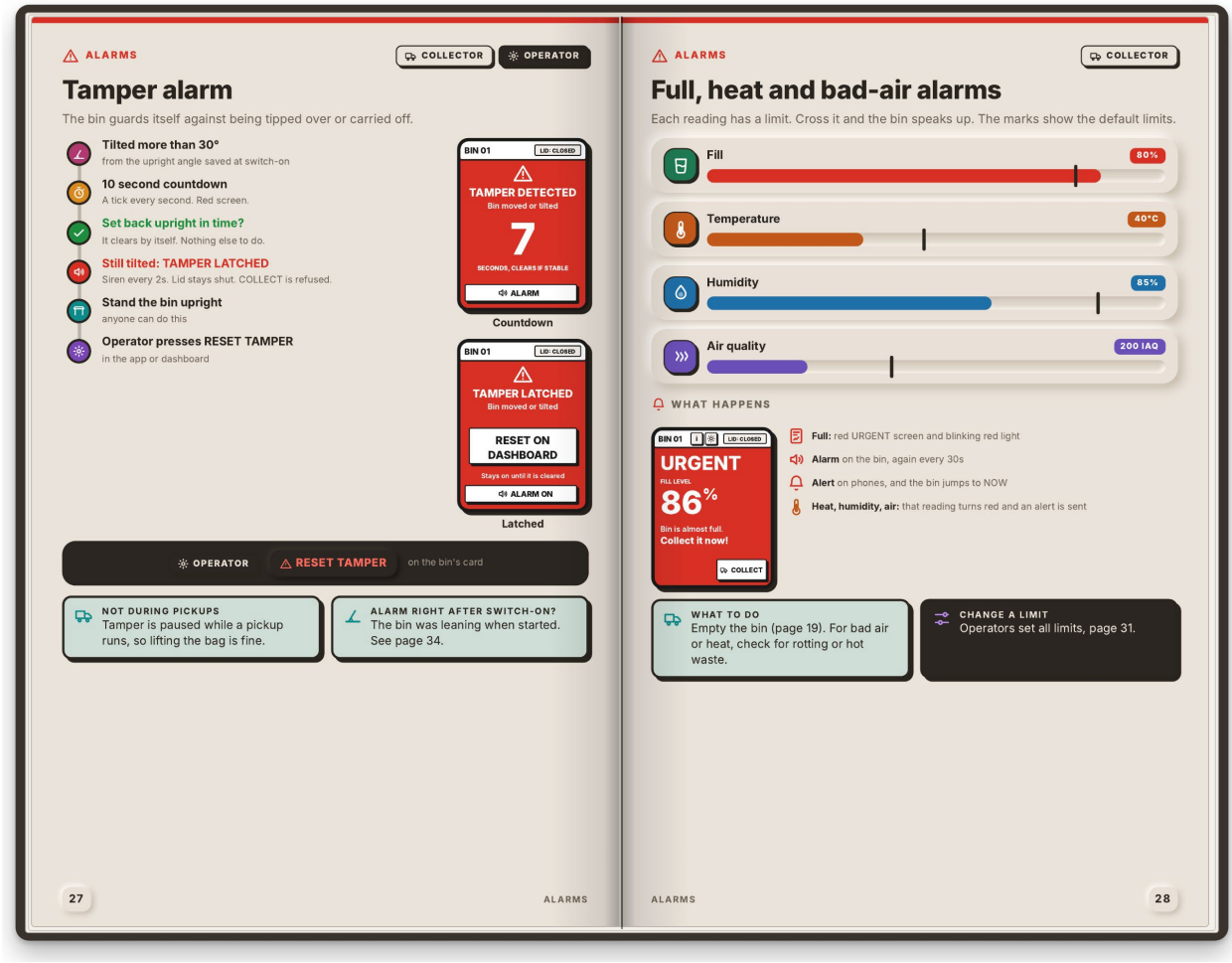
User Manual Pages 25 and 26: Charts and Readings, and Alerts



The charts page explains the time ranges and the air quality bands, and the alerts page shows the history filters.

Figure D14

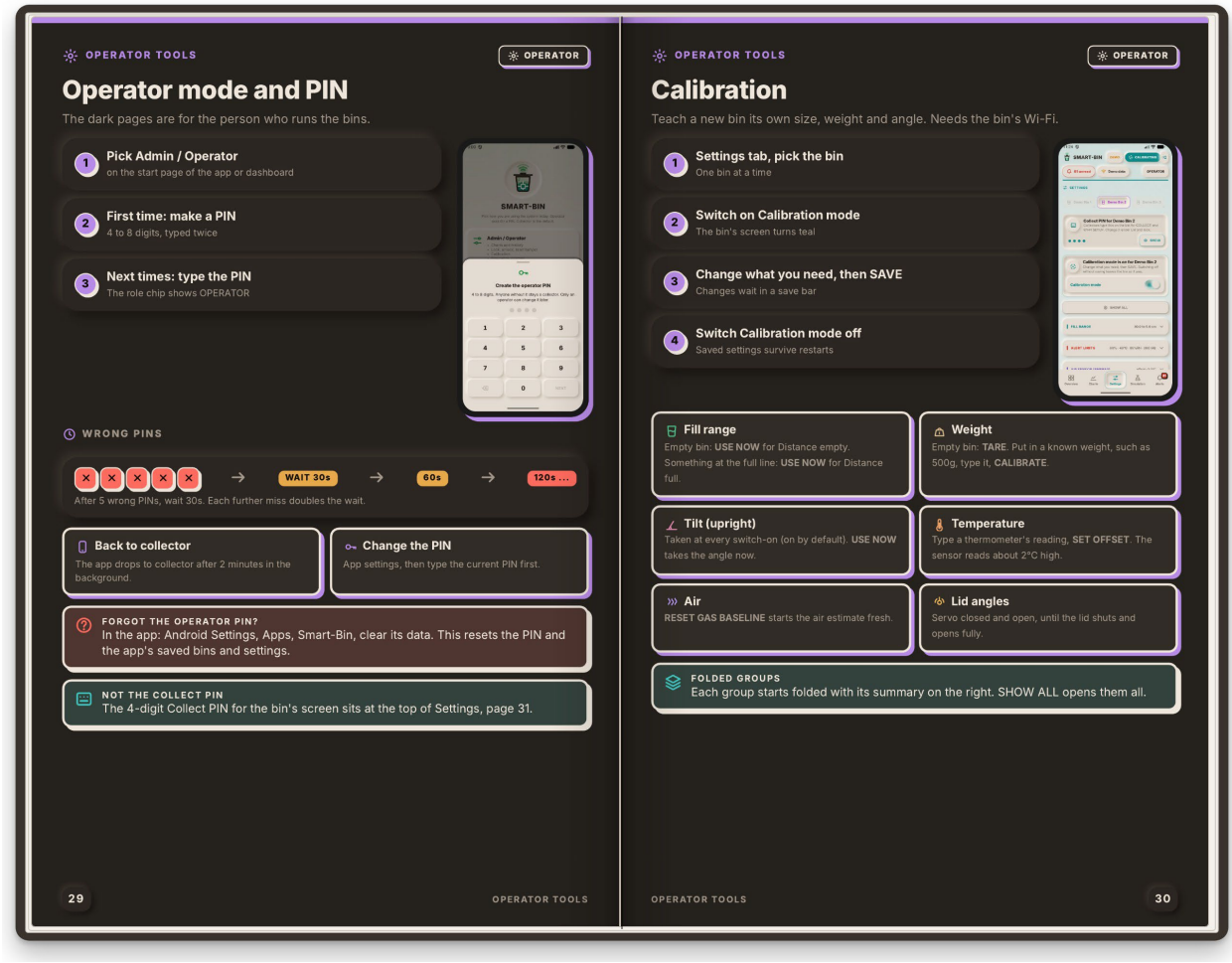
User Manual Pages 27 and 28: Tamper Alarm and Full, Heat and Bad-Air Alarms



The alarm section shows the 10s tamper countdown and the siren that follows, then the full, heat, humidity and bad-air alarms.

Figure D15

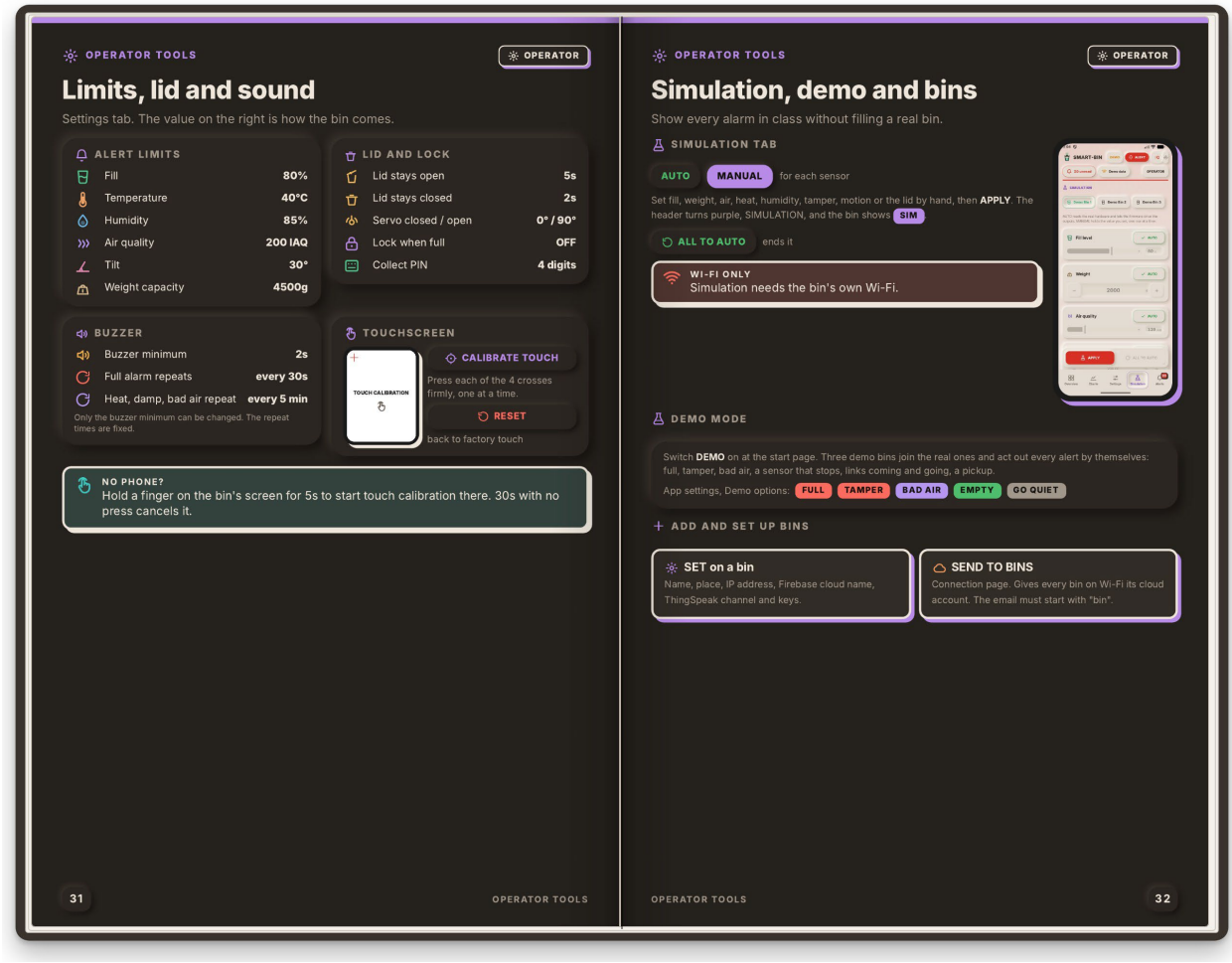
User Manual Pages 29 and 30: Operator Mode and PIN, and Calibration



The dark pages start here, for operators only: creating the operator PIN, then calibrating fill, weight, tilt, temperature, air and lid.

Figure D16

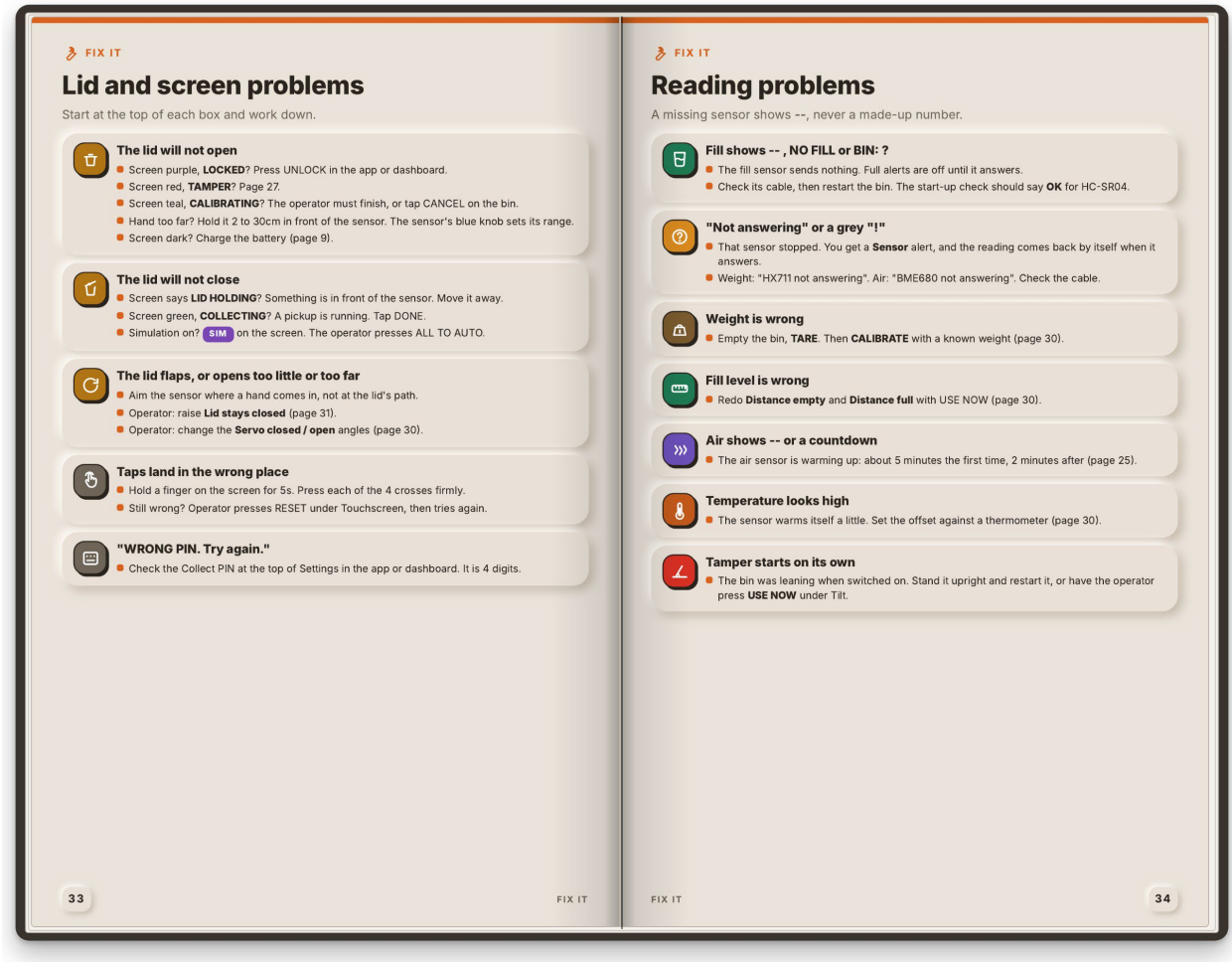
User Manual Pages 31 and 32: Limits, Lid and Sound, and Simulation, Demo and Bins



These pages list every setting with its default value, then show simulation, the demo bins and adding a bin.

Figure D17

User Manual Pages 33 and 34: Lid and Screen Problems, and Reading Problems



FIX IT

Lid and screen problems

Start at the top of each box and work down.

The lid will not open
- Screen purple, **LOCKED**? Press UNLOCK in the app or dashboard.
- Screen red, **TAMPER**? Page 27.
- Screen teal, **CALIBRATING**? The operator must finish, or tap CANCEL on the bin.
- Hand too far? Hold it 2 to 30cm in front of the sensor. The sensor's blue knob sets its range.
- Screen dark? Charge the battery (page 9).

The lid will not close
- Screen says **LID HOLDING**? Something is in front of the sensor. Move it away.
- Screen green, **COLLECTING**? A pickup is running. Tap DONE.
- Simulation on? SIM on the screen. The operator presses ALL TO AUTO.

The lid flaps, or opens too little or too far
- Aim the sensor where a hand comes in, not at the lid's path.
- Operator: raise **Lid stays closed** (page 31).
- Operator: change the **Servo closed / open** angles (page 30).

Taps land in the wrong place
- Hold a finger on the screen for 5s. Press each of the 4 crosses firmly.
- Still wrong? Operator presses RESET under Touchscreen, then tries again.

"WRONG PIN. Try again."
- Check the Collect PIN at the top of Settings in the app or dashboard. It is 4 digits.

33 FIX IT

FIX IT

Reading problems

A missing sensor shows --, never a made-up number.

Fill shows -- , NO FILL or BIN: ?
- The fill sensor sends nothing. Full alerts are off until it answers.
- Check its cable, then restart the bin. The start-up check should say **OK** for HC-SR04.

"Not answering" or a grey "!"
- That sensor stopped. You get a **Sensor** alert, and the reading comes back by itself when it answers.
- Weight: "HX711 not answering". Air: "BME680 not answering". Check the cable.

Weight is wrong
- Empty the bin, **TARE**. Then **CALIBRATE** with a known weight (page 30).

Fill level is wrong
- Redo **Distance empty** and **Distance full** with USE NOW (page 30).

Air shows -- or a countdown
- The air sensor is warming up: about 5 minutes the first time, 2 minutes after (page 25).

Temperature looks high
- The sensor warms itself a little. Set the offset against a thermometer (page 30).

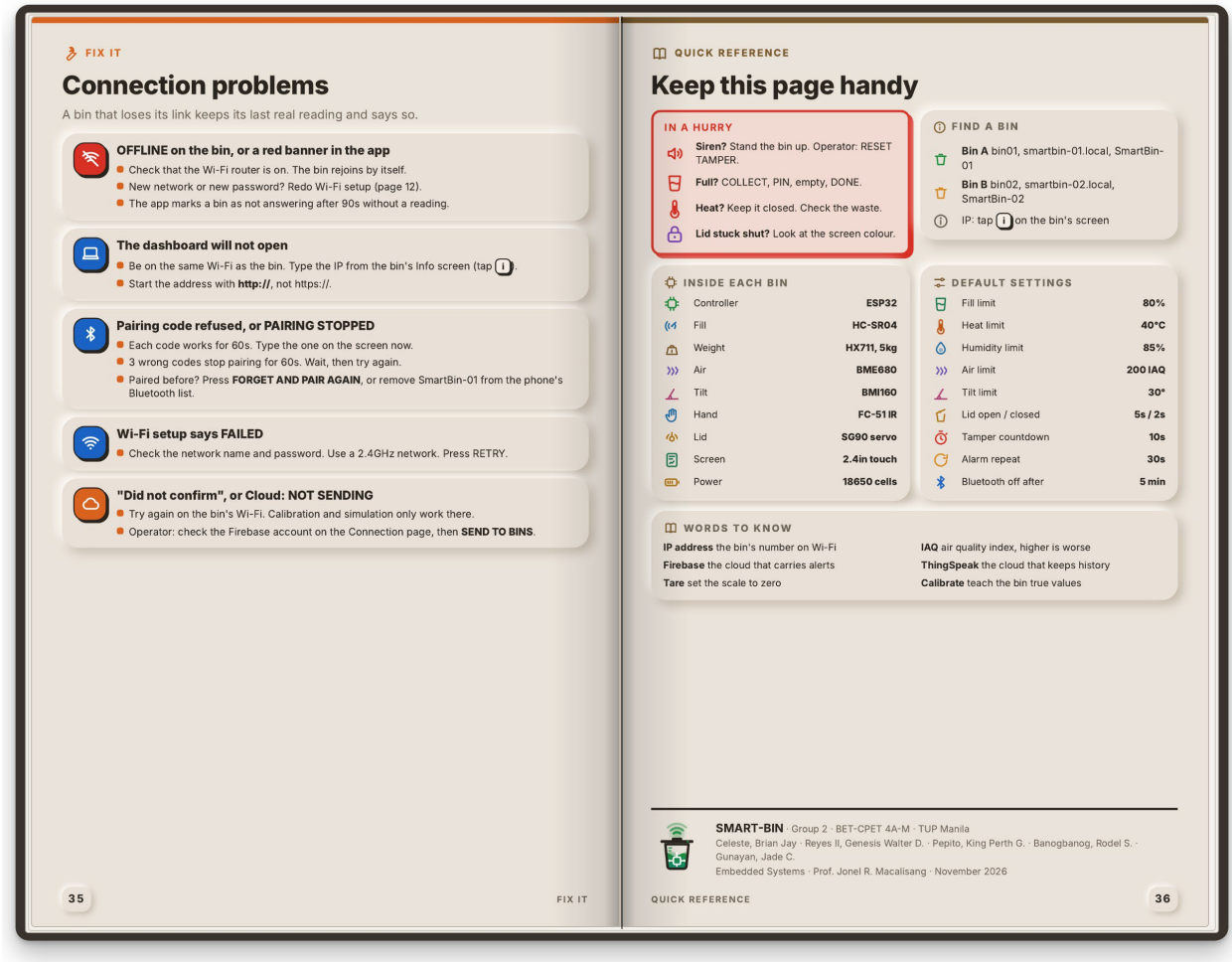
Tamper starts on its own
- The bin was leaning when switched on. Stand it upright and restart it, or have the operator press **USE NOW** under Tilt.

FIX IT 34

The fix-it section pairs each symptom with its likely cause and what to try, starting with the lid, the screen and the readings.

Figure D18

User Manual Pages 35 and 36: Connection Problems and Quick Reference



The last pair covers Wi-Fi and cloud problems, and the back cover gathers the defaults, colors and PINs on one page.

Appendix E

Source Code

This appendix lists every source file of the Smart-Bin system, in full, from the main branch of the group's repository, https://github.com/GenReyes/Smart-Bin-Embedded-Systems. Part 1 is the firmware that runs on both ESP32 boards, with the Firebase security rules and the script that packs the web dashboard into the firmware. Part 2 is the web dashboard, one HTML file holding its markup, CSS and JavaScript, which the bin serves over Wi-Fi. Part 3 is the Kotlin code of the Android app, grouped into data (talking to the bins and the clouds), notify (sounds, vibration and notifications) and ui (the screens). Part 4 is the app's manifest and XML resources. Part 5 is the Gradle build files, the GitHub Actions workflow that builds the APK, and the two development tools: the sound generator and the mock server used to test the app without hardware.

No passwords or keys appear in the code. They are kept in secrets.h on the board and typed into the app or dashboard at run time; secrets.example.h in Part 1 is the layout of secrets.h with placeholders in place of the real values. dashboard_gz.h is not printed: embed_dashboard.py generates it from the Part 2 file, and it holds the same dashboard as 3,965 lines of compressed bytes. Sound, font, image and other binary files are not code and are not listed.

Part 1. Firmware (ESP32)

firmware/SmartBin/SmartBin.ino

```
// ============================================================================
// SmartBin.ino - entry point
// Starts every module in a fixed order, then hands control to FreeRTOS tasks.
// The Arduino loop() is not used: it deletes itself, so all work happens in
// the tasks created below. If the board resets in a loop, open the Serial
// Monitor at 115200 baud; the first line printed here shows the bin ID and
// firmware version, and a crash before it points to a module's init().
// ============================================================================
// Smart-Bin ESP32 firmware. Group 2, BET-CpET 4A-M, TUP Manila.
// Before uploading: copy secrets.example.h to secrets.h and fill it in,
// and set BIN_NUMBER in config.h (1 for Bin A, 2 for Bin B).

#include "config.h"
#include "state.h"
#include "sensors.h"
#include "control.h"
#include "buzzer.h"
#include "net.h"
#include "cloud.h"
#include "ui.h"

void setup() {
  Serial.begin(115200);
  delay(200);
  Serial.printf("\nSmart-Bin %s firmware v%s\n", BIN_ID, FW_VERSION);

```

```
  // Order matters: the state module creates the mutex and queues every other
  // module uses, so it must run first. The UI comes next so the screen shows
  // the boot page while the sensors are still being probed.
  stateInit();
  uiInit();
  buzzerInit();
  controlInit();
  sensorsInit();
  netInit();

  // Task layout: name, stack bytes, priority, core. Core 1 runs the time
  // critical work (control has the highest priority so the lid and alarms
  // react first); core 0 runs the slow network jobs next to the Wi-Fi stack.
  // A "stack canary" or "stack overflow" reset names the task whose stack
  // size here is too small.
  xTaskCreatePinnedToCore(uiTask, "ui", 8192, nullptr, 2, &hUi, 1);
  xTaskCreatePinnedToCore(cloudTask, "cloud", 12288, nullptr, 1, &hCloud, 0);
  xTaskCreatePinnedToCore(pushTask, "push", 4096, nullptr, 2, &hPush, 0);
  xTaskCreatePinnedToCore(controlTask, "control", 4096, nullptr, 4, &hControl, 1);
  xTaskCreatePinnedToCore(sensorTask, "sensor", 4096, nullptr, 3, &hSensor, 1);

  // Interrupts only wake a task; no work is done inside the ISR itself.
  attachInterrupt(digitalPinToInterrupt(PIN_PIR), isrPir, CHANGE);
  attachInterrupt(digitalPinToInterrupt(PIN_TOUCH_IRQ), isrTouch, FALLING);
#if USE_IMU_INT
  pinMode(PIN_IMU_INT, INPUT);
#endif

  // The local web server starts before Wi-Fi joins so it is ready the moment
  // an IP address is given (see onWifiEvent in net.cpp).
  startWebServer();
  wifiConnect();
}

// Not used. Deleting the loop task frees its stack for the FreeRTOS tasks.
void loop() { vTaskDelete(nullptr); }
```

firmware/SmartBin/config.h

```
// ============================================================================
// config.h - build-time settings for one bin
// Pins, timing constants and the Bluetooth UUIDs. Values that users may
// change at run time (thresholds, PIN, sounds) live in struct Cal in state.h
// instead and are saved in flash.
// ============================================================================
#pragma once
#include <Arduino.h>

// Which bin this board is: 1 = Bin A (bin01), 2 = Bin B (bin02).
// Change BIN_NUMBER before flashing the second board. Both bins running the
// same number will overwrite each other's data in Firebase and ThingSpeak.
#ifndef BIN_NUMBER
#define BIN_NUMBER 1
#endif

#define FW_VERSION "1.0.0"

// Pins, from the Implementation Plan v2 pin table.
// GPIO 34 to 39 are input only on the ESP32, which is why the input lines
// SPI MISO 35, touch IRQ 36 and IMU INT 34 sit there. The buzzer is on GPIO 4,
// not GPIO 12: GPIO 12 is a boot strapping pin, and with the buzzer holding it
// high the ESP32 would not boot.
constexpr int PIN_TRIG = 5;
constexpr int PIN_ECHO = 18;
constexpr int PIN_BUZZER = 4;
constexpr int PIN_SERVO = 13;
constexpr int PIN_HX_SCK = 14;
constexpr int PIN_HX_DT = 32;
constexpr int PIN_TOUCH_CS = 15;
constexpr int PIN_TFT_DC = 16;
constexpr int PIN_TFT_CS = 17;
constexpr int PIN_SPI_MOSI = 19;
constexpr int PIN_SPI_SCK = 23;
constexpr int PIN_SPI_MISO = 35;
constexpr int PIN_TOUCH_IRQ = 36;
constexpr int PIN_SDA = 21;
constexpr int PIN_SCL = 22;
constexpr int PIN_LED_R = 25;
constexpr int PIN_LED_G = 26;
constexpr int PIN_LED_B = 33;
constexpr int PIN_PIR = 27;
constexpr int PIN_IMU_INT = 34;     // optional BMI160 INT1 wire; set USE_IMU_INT to 1 once it is wired

// Hardware polarity switches. If the LED shows the wrong colors or the buzzer
// is silent while the Serial log says it should sound, flip these.
```

```
#define USE_IMU_INT 0
#define BUZZER_ACTIVE_LOW 0          // 1 for "low level trigger" modules
#define LED_ACTIVE_LOW 0             // 1 for a common-anode RGB LED

// Touch calibration (raw XPT2046 values at the screen edges). Print raw values
// with TOUCH_DEBUG 1, press the four corners, and copy the numbers here.
#define TOUCH_DEBUG 0
constexpr int TS_RAW_X0 = 3800, TS_RAW_X1 = 300;   // raw x at screen x = 0 and x = 239
constexpr int TS_RAW_Y0 = 3800, TS_RAW_Y1 = 300;   // raw y at screen y = 0 and y = 319
#define TS_SWAP_XY 0
constexpr uint8_t TFT_ROTATION = 2;                // 0 = portrait 240 x 320

// Timing (ESP32 Integration Guide, sections 3 and 6A).
// All times are in milliseconds unless the name ends in _S. FB_ = Firebase,
// TS_ = ThingSpeak channel writes, TB_ = ThingSpeak TalkBack polling,
// BT_ = Bluetooth Wi-Fi setup. TS_MIN_GAP_MS keeps writes above the 15 s
// limit of the free ThingSpeak plan; a write sent faster than that returns 0.
constexpr uint32_t FB_WATCHED_MS = 5000;
constexpr uint32_t FB_IDLE_MS = 60000;
constexpr uint32_t FB_CMD_WATCHED_MS = 2000;
constexpr uint32_t FB_CMD_IDLE_MS = 10000;
constexpr uint32_t FB_WATCHERS_MS = 30000;
constexpr uint32_t FB_USAGE_MS = 600000;
constexpr uint32_t TS_WRITE_MS = 30000;
constexpr uint32_t TS_MIN_GAP_MS = 15500;
constexpr uint32_t TS_HEARTBEAT_MS = 300000;
constexpr uint32_t TB_WATCHED_MS = 5000;
constexpr uint32_t TB_IDLE_MS = 15000;
constexpr uint32_t CMD_MAX_AGE_MS = 60000;
constexpr uint32_t TAMPER_COUNTDOWN_S = 10;
constexpr uint32_t TAMPER_ALARM_EVERY_MS = 2000;   // latched tamper siren repeats this often
constexpr uint32_t SCREEN_TIMEOUT_MS = 30000;
constexpr uint32_t BT_IDLE_OFF_MS = 300000;
constexpr uint32_t BT_CODE_MS = 60000;
constexpr uint32_t BT_LOCKOUT_MS = 60000;
// 10 GiB, the monthly download quota of the Firebase free plan. Used only to
// estimate usage; Firebase does not report it to the device.
constexpr uint64_t FB_MONTH_LIMIT = 10737418240ULL;

// 0 = always use the fast "someone is watching" pace. Set to 1 once the app writes
// /sb/watchers (Integration Guide 6A); the dashboard already does.
#define FB_REQUIRE_WATCHERS 0

// Bluetooth service used by the app and dashboard for Wi-Fi setup. These must
// match the UUIDs in the Android app (data/Ble.kt) and the web dashboard.
#define BT_SVC_UUID  "b1a70001-5c2e-4f8a-9d33-0a9c2b6f17c1"
#define BT_SSID_UUID "b1a70002-5c2e-4f8a-9d33-0a9c2b6f17c1"
#define BT_PASS_UUID "b1a70003-5c2e-4f8a-9d33-0a9c2b6f17c1"
#define BT_CTRL_UUID "b1a70004-5c2e-4f8a-9d33-0a9c2b6f17c1"
#define BT_STAT_UUID "b1a70005-5c2e-4f8a-9d33-0a9c2b6f17c1"
#define BT_SCAN_UUID "b1a70006-5c2e-4f8a-9d33-0a9c2b6f17c1"

// Names derived from BIN_NUMBER. BIN_HOST gives http://smartbin-01.local on
// networks that support mDNS.
#if BIN_NUMBER == 1
#define BIN_ID "bin01"
#define BIN_LABEL "BIN 01"
#define BIN_HOST "smartbin-01"
#define BIN_BLE_NAME "SmartBin-01"
#else
#define BIN_ID "bin02"
#define BIN_LABEL "BIN 02"
#define BIN_HOST "smartbin-02"
#define BIN_BLE_NAME "SmartBin-02"
#endif
```

firmware/SmartBin/secrets.example.h

```
// ============================================================================
// secrets.example.h - template for secrets.h
// The firmware includes "secrets.h", not this file. A build error saying
// secrets.h is missing means this copy step was skipped.
// ============================================================================
#pragma once

// Copy this file to secrets.h in the same folder and fill it in.
// secrets.h is git-ignored. Never commit real keys, passwords or the Collect PIN.

// Wi-Fi (2.4 GHz only). Can also be changed later from WI-FI SETUP on the touchscreen.
#define WIFI_SSID "your-wifi-name"
#define WIFI_PASSWORD "your-wifi-password"

// Firebase Realtime Database. Empty = Firebase off (local Wi-Fi only).
// See firmware/README.md section 4. The bin signs in with its own account,
// for example bin01@your-project.test.
```

```
#define FIREBASE_URL ""          // https://<project>-default-rtdb.asia-southeast1.firebasedatabase.app
#define FIREBASE_API_KEY ""      // AIza... (Project settings > Web API key)
#define FIREBASE_EMAIL ""
#define FIREBASE_PASSWORD ""

// ThingSpeak, one channel per bin. Empty = ThingSpeak off. See firmware/README.md section 5.
#define THINGSPEAK_SERVER "https://api.thingspeak.com"
#define THINGSPEAK_WRITE_KEY_BIN_A ""
#define THINGSPEAK_WRITE_KEY_BIN_B ""
#define THINGSPEAK_TALKBACK_ID_BIN_A ""
#define THINGSPEAK_TALKBACK_KEY_BIN_A ""
#define THINGSPEAK_TALKBACK_ID_BIN_B ""
#define THINGSPEAK_TALKBACK_KEY_BIN_B ""

// Default Collect PIN (4 digits). It can be changed later from the dashboard or app.
#define COLLECT_PIN "0000"
```

firmware/SmartBin/state.h

```
// ============================================================================
// state.h - shared data model
// Every task reads and writes the same BinState (st), calibration (cal) and
// simulation (sim) objects. Always wrap access in lockState()/unlockState(),
// or take a copy with snapshot()/calSnapshot()/simSnapshot(). Reading st
// without the lock can show half-updated values on the screen or dashboard.
// ============================================================================
#pragma once
#include <Arduino.h>
#include <ArduinoJson.h>
#include "config.h"

// The one status shown everywhere. Priority, highest first, is decided in
// controlTask: CALIBRATING > COLLECTING > TAMPER > LOCKED > OK.
enum BinStatus : uint8_t { ST_OK = 0, ST_LOCKED, ST_TAMPER, ST_COLLECTING, ST_CALIBRATING };
extern const char* const STATUS_NAMES[5];

// User settings, saved in flash as one binary blob ("cal"). Distances are in
// cm, weights in grams, temperatures in deg C, gas_th is the IAQ index limit,
// quiet_from/quiet_to are minutes after midnight (local time, UTC+8).
// Adding or reordering a field changes sizeof(Cal), and loadCal() then
// ignores the old saved blob and falls back to these defaults.
struct Cal {
  float d_empty = 30, d_full = 5, fill_th = 80, temp_th = 40, hum_th = 85, gas_th = 200, tilt_th = 30, weight_cap = 4500;
  uint8_t auto_lock = 0;
  int16_t servo_closed = 0, servo_open = 90;
  uint16_t collect_pin = 0;
  uint16_t pir_hold_s = 5, buzz_s = 2;
  float temp_off = -2, hum_off = 0, press_off = 0;
  uint8_t buzz_on = 1, buzz_vol = 2, buzz_ui = 1, buzz_lid = 0;
  uint16_t buzz_repeat_s = 30;
  uint8_t quiet_on = 0;
  uint16_t quiet_from = 1320, quiet_to = 360;
};

// Raw XPT2046 readings at the screen edges, set by the touch calibration.
struct TouchCal {
  uint8_t swap = TS_SWAP_XY;
  int16_t x0 = TS_RAW_X0, x1 = TS_RAW_X1, y0 = TS_RAW_Y0, y1 = TS_RAW_Y1;
};
extern TouchCal touchCal;
extern bool touchCalCustom;
void saveTouchCal();
void loadTouchCal();
void resetTouchCal();

// Simulation overrides for testing without real waste. mask bits:
// 1 fill, 2 weight, 4 gas, 8 temperature and humidity, 16 tilt (tamper),
// 32 motion, 64 lid. A set bit replaces that sensor's value in the view.
struct Sim {
  uint8_t mask = 0;
  float fill = 60, weight = 2000, gas = 120, temp = 28, humidity = 60;
  bool tamper = false, motion = false, lid = false;
};

// One set of sensor values. iaq_acc is the air-quality accuracy:
// 0 warming up, 1 low, 2 medium, 3 high.
struct Reading {
  float fill = 0, distance_cm = 0, weight_g = 0, temp = 0, temp_raw = 0, humidity = 0, pressure = 0, iaq = 0, tilt = 0, gas_k
= 0;
  uint8_t iaq_acc = 0;
  bool motion = false;
};

struct LastCmd {
  char cmd[20] = "";
```

```
  uint64_t ts = 0;
  bool ok = false;
};

// The live state of the bin. raw holds sensor values, v holds what users see
// (raw with simulation applied by recomputeView()). The ok* flags are false
// while a sensor is not answering; they drive the "faults" list in the JSON.
struct BinState {
  Reading raw;            // what the sensors read
  Reading v;              // what everyone sees (raw with simulation applied)
  bool lidOpen = false;
  BinStatus status = ST_OK;
  bool locked = false, collecting = false, calibrating = false, autoLocked = false;
  uint8_t tamperPhase = 0;            // 0 none, 1 countdown, 2 latched
  uint32_t tamperStartMs = 0;
  bool okHC = false, okHX = false, okBME = false, okIMU = false;
  bool booting = true;
  bool wifi = false, cloudOk = false, tsOk = false, btOn = false;
  int rssi = 0;
  uint32_t lidCloseAtMs = 0;          // 0 = no countdown; lid holds while motion is high
  bool alarmFull = false, alarmEnv = false, buzzerOn = false;
  uint32_t buzzerUntilMs = 0;
  float fillRatePerHour = 0;
  uint8_t calSteps = 0;               // bit 0 tare, bit 1 empty distance, bit 2 full distance
  uint32_t doneAtMs = 0;              // when COLLECT_DONE ran, for the A5 screen
  LastCmd last;
  uint32_t version = 0;
  uint32_t calVersion = 0;
};

// Where a command came from. Results are written back only for Firebase
// commands (they carry a ref key); screen commands skip some beeps because
// the screen already gave feedback.
enum CmdSource : uint8_t { SRC_LOCAL = 0, SRC_SCREEN, SRC_FIREBASE, SRC_TALKBACK };

struct Command {
  char cmd[20];
  float grams;
  char sound[16];
  uint8_t src;
  char ref[28];
};

struct CmdResult {
  char cmd[20];
  bool ok;
  char why[40];
  uint8_t src;
  char ref[28];
  uint64_t ts;
};

struct AlertEv {
  char kind[10];
  char sev[10];
  char title[40];
  char meta[72];
};

// Task notification bits. A task waits on these with xTaskNotifyWait, so one
// notify wakes it immediately instead of waiting for its next timeout.
enum : uint32_t {
  EV_PIR = 1 << 0, EV_TOUCH = 1 << 1, EV_IMU = 1 << 2, EV_STATE = 1 << 3, EV_CMD = 1 << 4,
  EV_CAL = 1 << 5, EV_WIFI = 1 << 6, EV_ALERT = 1 << 7, EV_RESULT = 1 << 8, EV_BT = 1 << 9,
};

extern BinState st;
extern Cal cal;
extern Sim sim;
extern SemaphoreHandle_t stLock;
extern QueueHandle_t cmdQueue, resultQueue, alertQueue;
extern TaskHandle_t hControl, hSensor, hUi, hCloud, hPush;

// Network and cloud credentials. Loaded from flash, falling back to the
// values in secrets.h when nothing has been saved yet.
struct NetCfg {
  String ssid, pass;
  String fbUrl, fbKey, fbEmail, fbPass;
  String tsServer, tsWriteKey, tbId, tbKey;
};
extern NetCfg net;

// --- Lock, copy and publish ---
// publish() bumps st.version and wakes the UI, cloud and push tasks.
// Call it after any change that users should see.
void stateInit();
```

```
void lockState();
void unlockState();
BinState snapshot();
Cal calSnapshot();
Sim simSnapshot();
void publish(uint32_t extraBits = 0);
void notifyAll(uint32_t bits);

// --- Flash storage (Preferences namespace "smartbin") ---
void saveCal();
void loadCal();
void saveNet();
void loadNet();
uint8_t loadTheme();
void saveTheme(uint8_t dark);
void saveHx(int32_t offset, float scale);
bool loadHx(int32_t& offset, float& scale);
void saveGasBaseline(float g);
float loadGasBaseline();
void saveImuRef(float x, float y, float z);
bool loadImuRef(float& x, float& y, float& z);

// --- JSON for the local API, live events and Firebase ---
void calToJson(const Cal& c, JsonObject o);
bool calFromJson(JsonObjectConst o, Cal& c);
void statusToJson(const BinState& s, const Cal& c, const Sim& sm, JsonObject o, bool withCal, bool forCloud);
String statusJson(bool withCal = true, bool forCloud = false);
String localIp();

// --- Commands and alerts ---
// sendCommand() only queues; controlTask runs it. validateCommand() is also
// called by the web server first so it can answer 400/409 right away.
bool sendCommand(const char* cmd, uint8_t src, float grams = 0, const char* sound = nullptr, const char* ref = nullptr);
bool validateCommand(const char* cmd, const BinState& s, String& why);
void raiseAlert(const char* kind, const char* sev, const char* title, const char* meta);

// --- Time. nowMs() returns 0 until SNTP has set the clock. ---
uint64_t nowMs();
bool clockSet();
int localMinuteOfDay();
void monthKeyUtc(char* out, size_t n);
```

firmware/SmartBin/state.cpp

```
// ============================================================================
// state.cpp - shared state, flash storage, JSON and command plumbing
// ============================================================================
#include "state.h"
#include "sensors.h"
#include <Preferences.h>
#include <WiFi.h>
#include <sys/time.h>
#include "secrets.h"

const char* const STATUS_NAMES[5] = {"OK", "LOCKED", "TAMPER", "COLLECTING", "CALIBRATING"};

BinState st;
Cal cal;
Sim sim;
NetCfg net;
SemaphoreHandle_t stLock;
QueueHandle_t cmdQueue, resultQueue, alertQueue;
TaskHandle_t hControl = nullptr, hSensor = nullptr, hUi = nullptr, hCloud = nullptr, hPush = nullptr;

// Preferences (NVS flash) is not safe to use from two tasks at once, so every
// access goes through prefsBegin()/prefsEnd(), which hold prefsLock.
static Preferences prefs;
static SemaphoreHandle_t prefsLock;

// Queue depths are small on purpose: if a queue is full, sendCommand()
// fails and the caller reports "Busy" instead of blocking a task.
void stateInit() {
  stLock = xSemaphoreCreateMutex();
  prefsLock = xSemaphoreCreateMutex();
  cmdQueue = xQueueCreate(8, sizeof(Command));
  resultQueue = xQueueCreate(8, sizeof(CmdResult));
  alertQueue = xQueueCreate(8, sizeof(AlertEv));
  cal.collect_pin = (uint16_t)atoi(COLLECT_PIN);
  loadCal();
  loadTouchCal();
  loadNet();
}

void lockState() { xSemaphoreTake(stLock, portMAX_DELAY); }
void unlockState() { xSemaphoreGive(stLock); }
```

```

BinState snapshot() {
  lockState();
  BinState s = st;
  unlockState();
  return s;
}

Cal calSnapshot() {
  lockState();
  Cal c = cal;
  unlockState();
  return c;
}

Sim simSnapshot() {
  lockState();
  Sim s = sim;
  unlockState();
  return s;
}

// Wakes the three tasks that display or send state. A handle is null until
// its task has been created in setup().
void notifyAll(uint32_t bits) {
  TaskHandle_t hs[] = {hUi, hCloud, hPush};
  for (TaskHandle_t h : hs)
    if (h) xTaskNotify(h, bits, eSetBits);
}

void publish(uint32_t extraBits) {
  lockState();
  st.version++;
  unlockState();
  notifyAll(EV_STATE | extraBits);
}

static void prefsBegin(bool ro) {
  xSemaphoreTake(prefsLock, portMAX_DELAY);
  prefs.begin("smartbin", ro);
}
static void prefsEnd() {
  prefs.end();
  xSemaphoreGive(prefsLock);
}

// Calibration is stored as a raw struct. loadCal() only accepts a blob of the
// exact current size, so a firmware with a changed Cal struct starts from
// defaults instead of reading garbage.
void saveCal() {
  Cal c = calSnapshot();
  prefsBegin(false);
  prefs.putBytes("cal", &c, sizeof(c));
  prefsEnd();
}

void loadCal() {
  prefsBegin(true);
  if (prefs.getBytesLength("cal") == sizeof(Cal)) prefs.getBytes("cal", &cal, sizeof(Cal));
  prefsEnd();
}

TouchCal touchCal;
bool touchCalCustom = false;

void saveTouchCal() {
  prefsBegin(false);
  prefs.putBytes("touch", &touchCal, sizeof(touchCal));
  prefsEnd();
  touchCalCustom = true;
}

void loadTouchCal() {
  prefsBegin(true);
  if (prefs.getBytesLength("touch") == sizeof(TouchCal)) {
    prefs.getBytes("touch", &touchCal, sizeof(TouchCal));
    touchCalCustom = true;
  }
  prefsEnd();
}

void resetTouchCal() {
  touchCal = TouchCal();
  prefsBegin(false);
  prefs.remove("touch");
  prefsEnd();
```

```
  touchCalCustom = false;
}

// Wi-Fi and Firebase settings changed from the screen, Bluetooth setup or
// /api/cloud are kept here and survive a reflash of the same partition.
void saveNet() {
  prefsBegin(false);
  prefs.putString("ssid", net.ssid);
  prefs.putString("pass", net.pass);
  prefs.putString("fbUrl", net.fbUrl);
  prefs.putString("fbKey", net.fbKey);
  prefs.putString("fbEmail", net.fbEmail);
  prefs.putString("fbPass", net.fbPass);
  prefsEnd();
}

// ThingSpeak keys come only from secrets.h; each bin picks its own channel.
void loadNet() {
  prefsBegin(true);
  net.ssid = prefs.getString("ssid", WIFI_SSID);
  net.pass = prefs.getString("pass", WIFI_PASSWORD);
  net.fbUrl = prefs.getString("fbUrl", FIREBASE_URL);
  net.fbKey = prefs.getString("fbKey", FIREBASE_API_KEY);
  net.fbEmail = prefs.getString("fbEmail", FIREBASE_EMAIL);
  net.fbPass = prefs.getString("fbPass", FIREBASE_PASSWORD);
  prefsEnd();
  net.tsServer = THINGSPEAK_SERVER;
#if BIN_NUMBER == 1
  net.tsWriteKey = THINGSPEAK_WRITE_KEY_BIN_A;
  net.tbId = THINGSPEAK_TALKBACK_ID_BIN_A;
  net.tbKey = THINGSPEAK_TALKBACK_KEY_BIN_A;
#else
  net.tsWriteKey = THINGSPEAK_WRITE_KEY_BIN_B;
  net.tbId = THINGSPEAK_TALKBACK_ID_BIN_B;
  net.tbKey = THINGSPEAK_TALKBACK_KEY_BIN_B;
#endif
  if (net.tsWriteKey.startsWith("WRITE_KEY")) net.tsWriteKey = "";
}

uint8_t loadTheme() {
  prefsBegin(true);
  uint8_t t = prefs.getUChar("theme", 0);
  prefsEnd();
  return t;
}

void saveTheme(uint8_t dark) {
  prefsBegin(false);
  prefs.putUChar("theme", dark);
  prefsEnd();
}

void saveHx(int32_t offset, float scale) {
  prefsBegin(false);
  prefs.putInt("hxOff", offset);
  prefs.putFloat("hxScale", scale);
  prefsEnd();
}

// Load-cell offset and scale from the last TARE and CAL_WEIGHT. Returns false
// when the scale was never calibrated (weights will then be rough).
bool loadHx(int32_t& offset, float& scale) {
  prefsBegin(true);
  bool has = prefs.isKey("hxScale");
  offset = prefs.getInt("hxOff", 0);
  scale = prefs.getFloat("hxScale", 420.0f);
  prefsEnd();
  return has;
}

void saveGasBaseline(float g) {
  prefsBegin(false);
  prefs.putFloat("gasBase", g);
  prefsEnd();
}

float loadGasBaseline() {
  prefsBegin(true);
  float g = prefs.getFloat("gasBase", 0);
  prefsEnd();
  return g;
}

void saveImuRef(float x, float y, float z) {
  float v[3] = {x, y, z};
  prefsBegin(false);
```

```
  prefs.putBytes("imuRef", v, sizeof(v));
  prefsEnd();
}

bool loadImuRef(float& x, float& y, float& z) {
  float v[3];
  prefsBegin(true);
  bool ok = prefs.getBytesLength("imuRef") == sizeof(v) && prefs.getBytes("imuRef", v, sizeof(v)) == sizeof(v);
  prefsEnd();
  if (ok) { x = v[0]; y = v[1]; z = v[2]; }
  return ok;
}

// Round to 1 or 2 decimals so the JSON stays short and readable.
static double r1(float v) { return round(v * 10.0) / 10.0; }
static double r2(float v) { return round(v * 100.0) / 100.0; }

void calToJson(const Cal& c, JsonObject o) {
  o["d_empty"] = r1(c.d_empty);
  o["d_full"] = r1(c.d_full);
  o["fill_th"] = r1(c.fill_th);
  o["temp_th"] = r1(c.temp_th);
  o["hum_th"] = r1(c.hum_th);
  o["gas_th"] = r1(c.gas_th);
  o["tilt_th"] = r1(c.tilt_th);
  o["weight_cap"] = r1(c.weight_cap);
  o["auto_lock"] = c.auto_lock;
  o["servo_closed"] = c.servo_closed;
  o["servo_open"] = c.servo_open;
  o["collect_pin"] = c.collect_pin;
  o["pir_hold_s"] = c.pir_hold_s;
  o["buzz_s"] = c.buzz_s;
  o["temp_off"] = r1(c.temp_off);
  o["hum_off"] = r1(c.hum_off);
  o["press_off"] = r1(c.press_off);
  o["buzz_on"] = c.buzz_on;
  o["buzz_vol"] = c.buzz_vol;
  o["buzz_ui"] = c.buzz_ui;
  o["buzz_lid"] = c.buzz_lid;
  o["buzz_repeat_s"] = c.buzz_repeat_s;
  o["quiet_on"] = c.quiet_on;
  o["quiet_from"] = c.quiet_from;
  o["quiet_to"] = c.quiet_to;
}

// Reads one numeric key, accepts numbers or numeric strings, and clamps it to
// [lo, hi]. Out-of-range values are clamped, not rejected, so a typo cannot
// push the bin into an unusable setting.
template <typename T>
static bool take(JsonObjectConst o, const char* k, T& dst, float lo, float hi) {
  JsonVariantConst v = o[k];
  if (v.isNull() || !(v.is<float>() || v.is<int>() || v.is<const char*>())) return false;
  float f = v.is<const char*>() ? atof(v.as<const char*>()) : v.as<float>();
  if (isnan(f)) return false;
  f = constrain(f, lo, hi);
  dst = (T)f;
  return true;
}

// Applies any known calibration keys. Returns false when the request had none
// (the API then answers 400). The empty distance must stay larger than the
// full distance or the fill formula divides by zero or goes negative.
bool calFromJson(JsonObjectConst o, Cal& c) {
  bool any = false;
  any |= take(o, "d_empty", c.d_empty, 5, 200);
  any |= take(o, "d_full", c.d_full, 1, 100);
  any |= take(o, "fill_th", c.fill_th, 10, 100);
  any |= take(o, "temp_th", c.temp_th, 10, 85);
  any |= take(o, "hum_th", c.hum_th, 10, 100);
  any |= take(o, "gas_th", c.gas_th, 50, 500);
  any |= take(o, "tilt_th", c.tilt_th, 5, 90);
  any |= take(o, "weight_cap", c.weight_cap, 100, 50000);
  any |= take(o, "auto_lock", c.auto_lock, 0, 1);
  any |= take(o, "servo_closed", c.servo_closed, 0, 180);
  any |= take(o, "servo_open", c.servo_open, 0, 180);
  any |= take(o, "collect_pin", c.collect_pin, 0, 9999);
  any |= take(o, "pir_hold_s", c.pir_hold_s, 1, 120);
  any |= take(o, "buzz_s", c.buzz_s, 0, 60);
  any |= take(o, "temp_off", c.temp_off, -20, 20);
  any |= take(o, "hum_off", c.hum_off, -30, 30);
  any |= take(o, "press_off", c.press_off, -100, 100);
  any |= take(o, "buzz_on", c.buzz_on, 0, 1);
  any |= take(o, "buzz_vol", c.buzz_vol, 1, 3);
  any |= take(o, "buzz_ui", c.buzz_ui, 0, 1);
  any |= take(o, "buzz_lid", c.buzz_lid, 0, 1);
  any |= take(o, "buzz_repeat_s", c.buzz_repeat_s, 10, 300);
```

```
  any |= take(o, "quiet_on", c.quiet_on, 0, 1);
  any |= take(o, "quiet_from", c.quiet_from, 0, 1439);
  any |= take(o, "quiet_to", c.quiet_to, 0, 1439);
  if (c.d_full >= c.d_empty) c.d_full = c.d_empty - 1;
  return any;
}

String localIp() { return WiFi.isConnected() ? WiFi.localIP().toString() : String(""); }

// Builds the status object used by /api/status, the live event stream and
// Firebase. A sensor's values are left out while it is faulty (and not
// simulated), so the apps show "--" instead of stale numbers.
void statusToJson(const BinState& s, const Cal& c, const Sim& sm, JsonObject o, bool withCal, bool forCloud) {
  const Reading& v = s.v;
  bool hc = s.okHC || (sm.mask & 1), hx = s.okHX || (sm.mask & 2), bme = s.okBME || (sm.mask & 12) == 12;
  o["id"] = BIN_ID;
  o["name"] = BIN_LABEL;
  if (hc) {
    o["fill"] = r1(v.fill);
    o["distance_cm"] = r1(v.distance_cm);
  }
  if (hx) o["weight_g"] = (long)lround(v.weight_g);
  if (bme || (sm.mask & 8)) {
    o["temp"] = r2(v.temp);
    o["temp_raw"] = r2(v.temp_raw);
    o["humidity"] = r1(v.humidity);
  }
  if (s.okBME) o["pressure"] = r1(v.pressure);
  if (s.okBME) o["gas_k"] = r1(v.gas_k);
  if (s.okBME && v.iaq_acc == 0 && !(sm.mask & 4)) o["iaq_warm_s"] = airWarmLeft();
  if (s.okBME || (sm.mask & 4)) {
    o["iaq"] = (long)lround(v.iaq);
    o["iaq_acc"] = v.iaq_acc;
  }
  o["tilt"] = r1(v.tilt);
  o["motion"] = v.motion;
  o["lid"] = s.lidOpen ? "OPEN" : "CLOSED";
  o["status"] = STATUS_NAMES[s.status];
  o["sim_mask"] = sm.mask;
  if (s.tamperPhase) o["tamper_phase"] = s.tamperPhase == 1 ? "COUNTDOWN" : "LATCHED";
  JsonArray f = o["faults"].to<JsonArray>();
  if (!s.okHC) f.add("HC-SR04");
  if (!s.okHX) f.add("HX711");
  if (!s.okBME) f.add("BME680");
  if (!s.okIMU) f.add("BMI160");
  if (withCal) calToJson(c, o["cal"].to<JsonObject>());
  if (withCal) {
    JsonObject t = o["touch"].to<JsonObject>();
    t["swap"] = touchCal.swap;
    t["x0"] = touchCal.x0;
    t["x1"] = touchCal.x1;
    t["y0"] = touchCal.y0;
    t["y1"] = touchCal.y1;
    t["custom"] = touchCalCustom;
  }
  if (s.last.cmd[0]) {
    JsonObject lc = o["last_cmd"].to<JsonObject>();
    lc["cmd"] = s.last.cmd;
    lc["ts"] = s.last.ts;
    lc["ok"] = s.last.ok;
  }
  o["uptime_s"] = (uint32_t)(millis() / 1000);
  o["rssi"] = s.rssi;
  o["fw"] = FW_VERSION;
  // ".sv": "timestamp" asks Firebase to fill in its own server time.
  if (forCloud) {
    o["local_ip"] = localIp();
    o["ts"][".sv"] = "timestamp";
  }
}

String statusJson(bool withCal, bool forCloud) {
  lockState();
  BinState s = st;
  Cal c = cal;
  Sim sm = sim;
  unlockState();
  JsonDocument doc;
  statusToJson(s, c, sm, doc.to<JsonObject>(), withCal, forCloud);
  String out;
  serializeJson(doc, out);
  return out;
}

// Every command the bin accepts. The apps send these exact strings.
static const char* const KNOWN[] = {"LOCK", "UNLOCK", "CLEAR_TAMPER", "COLLECT", "COLLECT_DONE", "CALIBRATE_START",
```

```
                                    "CALIBRATE_END", "TARE", "CAL_WEIGHT", "RESET_GAS", "BUZZ_TEST",
                                    "TOUCH_CAL", "TOUCH_CAL_RESET"};

// Rejects commands that would conflict with the current state. The "why"
// text is shown to the user, so keep it short.
bool validateCommand(const char* cmd, const BinState& s, String& why) {
  bool known = false;
  for (const char* k : KNOWN)
    if (!strcmp(k, cmd)) known = true;
  if (!known) { why = "Unknown command"; return false; }
  if (!strcmp(cmd, "COLLECT")) {
    if (s.calibrating) { why = "Calibrating"; return false; }
    if (s.tamperPhase) { why = "TAMPER active"; return false; }
  }
  if (!strcmp(cmd, "LOCK") && s.collecting) { why = "Collection in progress"; return false; }
  if (!strcmp(cmd, "CALIBRATE_START") && s.collecting) { why = "Collection in progress"; return false; }
  if ((!strcmp(cmd, "TARE") || !strcmp(cmd, "CAL_WEIGHT")) && !s.okHX) { why = "Load cell not answering"; return false; }
  if (!strcmp(cmd, "RESET_GAS") && !s.okBME) { why = "BME680 not answering"; return false; }
  return true;
}

// Queues a command for controlTask and wakes it. Waits at most 50 ms for a
// free slot; false means the queue was full.
bool sendCommand(const char* cmd, uint8_t src, float grams, const char* sound, const char* ref) {
  Command c{};
  strlcpy(c.cmd, cmd, sizeof(c.cmd));
  c.grams = grams;
  c.src = src;
  if (sound) strlcpy(c.sound, sound, sizeof(c.sound));
  if (ref) strlcpy(c.ref, ref, sizeof(c.ref));
  if (xQueueSend(cmdQueue, &c, pdMS_TO_TICKS(50)) != pdTRUE) return false;
  if (hControl) xTaskNotify(hControl, EV_CMD, eSetBits);
  return true;
}

// Queues an alert for cloudTask, which logs it to Serial as "[alert] ..." and
// posts it to Firebase. If alerts never reach the app, check the Serial log
// for these lines first, then the Firebase rules.
void raiseAlert(const char* kind, const char* sev, const char* title, const char* meta) {
  AlertEv a{};
  strlcpy(a.kind, kind, sizeof(a.kind));
  strlcpy(a.sev, sev, sizeof(a.sev));
  strlcpy(a.title, title, sizeof(a.title));
  strlcpy(a.meta, meta, sizeof(a.meta));
  xQueueSend(alertQueue, &a, 0);
  if (hCloud) xTaskNotify(hCloud, EV_ALERT, eSetBits);
}

// The clock counts as set once it is past November 2023 (Unix 1700000000).
bool clockSet() { return time(nullptr) > 1700000000; }

// Philippine time is UTC+8 with no daylight saving, so a fixed offset is used.
int localMinuteOfDay() {
  if (!clockSet()) return -1;
  return (int)(((uint64_t)time(nullptr) + 8 * 3600ULL) / 60ULL % 1440ULL);
}

// Converts today's date to "YYYY-MM" without the C time library (civil-from-
// days algorithm). Used as the key for the monthly Firebase usage counter.
void monthKeyUtc(char* out, size_t n) {
  out[0] = 0;
  if (!clockSet()) return;
  int64_t z = (int64_t)(time(nullptr) / 86400) + 719468;
  int64_t era = z / 146097;
  int64_t doe = z - era * 146097;
  int64_t yoe = (doe - doe / 1460 + doe / 36524 - doe / 146096) / 365;
  int64_t doy = doe - (365 * yoe + yoe / 4 - yoe / 100);
  int64_t mp = (5 * doy + 2) / 153;
  int m = (int)(mp < 10 ? mp + 3 : mp - 9);
  int y = (int)(yoe + era * 400 + (m <= 2));
  snprintf(out, n, "%04d-%02d", y, m);
}

uint64_t nowMs() {
  if (!clockSet()) return 0;
  struct timeval tv;
  gettimeofday(&tv, nullptr);
  return (uint64_t)tv.tv_sec * 1000ULL + tv.tv_usec / 1000;
}
```

firmware/SmartBin/sensors.h

```
// ============================================================================
// sensors.h - sensor task and calibration requests
// The request* functions only set a flag; sensorTask does the actual work on
```

```
// its next pass, so they are safe to call from any task.
// ============================================================================
#pragma once
#include <Arduino.h>

void sensorsInit();
void sensorTask(void*);
void recomputeView();

void requestTare();
void requestCalWeight(float grams);
void requestGasReset();
void requestImuReference();
int airWarmLeft();
```

firmware/SmartBin/sensors.cpp

```
// ============================================================================
// sensors.cpp - reads every sensor and keeps st.raw up to date
//   HC-SR04 ultrasonic   -> distance and fill %   every 200 ms
//   BMI160 accelerometer -> tilt in degrees       every 200 ms (raw I2C)
//   HX711 load cell      -> weight in grams       whenever a sample is ready
//   BME680               -> temp, humidity, pressure, gas, IAQ  every 3 s
// A sensor that stops answering sets its ok* flag to false; controlTask turns
// that into a "Sensor not answering" alert and the screen shows "--".
// ============================================================================
#include "sensors.h"
#include "state.h"
#include "buzzer.h"
#include <Wire.h>
#include <Adafruit_BME680.h>
#include <HX711.h>

static Adafruit_BME680 bme(&Wire);
static uint8_t imuAddr = 0;

// --- BMI160 over raw I2C (no library) ---
// Register map from the BMI160 datasheet: 0x00 CHIP_ID (0xD1), 0x03 PMU
// status, 0x12 accel data, 0x40 ACC_CONF, 0x41 ACC_RANGE, 0x7E CMD.
static bool imuWrite(uint8_t reg, uint8_t val) {
  Wire.beginTransmission(imuAddr);
  Wire.write(reg);
  Wire.write(val);
  return Wire.endTransmission() == 0;
}

static bool imuRead(uint8_t reg, uint8_t* buf, uint8_t len) {
  Wire.beginTransmission(imuAddr);
  Wire.write(reg);
  if (Wire.endTransmission(false) != 0) return false;
  if (Wire.requestFrom(imuAddr, len) != len) return false;
  for (uint8_t i = 0; i < len; i++) buf[i] = Wire.read();
  return true;
}

// Finds the BMI160 at 0x68 or 0x69 (depends on the SDO pin), soft-resets it,
// sets 100 Hz / +-2 g and switches the accelerometer to normal mode. Returns
// false if the chip ID is wrong or the accelerometer never powers up; the
// boot screen then shows BMI160 FAIL. Check SDA 21 / SCL 22 wiring first.
static bool imuStart() {
  const uint8_t addrs[2] = {0x68, 0x69};
  uint8_t id = 0;
  for (uint8_t a : addrs) {
    imuAddr = a;
    if (imuRead(0x00, &id, 1) && id == 0xD1) break;
    id = 0;
  }
  if (id != 0xD1) return false;
  imuWrite(0x7E, 0xB6);
  delay(100);
  imuWrite(0x40, 0x28);
  imuWrite(0x41, 0x03);
  for (int i = 0; i < 3; i++) {
    imuWrite(0x7E, 0x11);
    delay(10);
    uint8_t pmu = 0;
    if (imuRead(0x03, &pmu, 1) && ((pmu >> 4) & 3) == 1) return true;
  }
  return false;
}

// Reads X, Y, Z acceleration as signed 16-bit values (little endian).
static bool imuAccel(int16_t a[3]) {
  uint8_t d[6];
  if (!imuRead(0x12, d, 6)) return false;
```

```
  a[0] = (int16_t)(d[1] << 8 | d[0]);
  a[1] = (int16_t)(d[3] << 8 | d[2]);
  a[2] = (int16_t)(d[5] << 8 | d[4]);
  return true;
}
static HX711 scale;

// Flags set by the request* functions and cleared here after the work is done.
static volatile bool reqTare = false, reqCalW = false, reqGas = false, reqImuRef = false;
static volatile float reqGrams = 0;

// Load cell: grams = (raw - hxOffset) / hxScale. 420 is a rough default for a
// 5 kg bar until CAL_WEIGHT is run with a known weight.
static int32_t hxOffset = 0;
static float hxScale = 420.0f;
static float imuRef[3] = {0, 0, 1};

// Highest gas resistance seen in clean air. IAQ is computed against it, so a
// bad baseline (e.g. warmed up next to garbage) makes the air look clean.
// RESET_GAS clears it and restarts the warm-up.
static float gasBaseline = 0;
static uint32_t gasStartMs = 0;
static bool baselineFromNvs = false;

void requestTare() { reqTare = true; }
void requestCalWeight(float grams) { reqGrams = grams; reqCalW = true; }
void requestGasReset() { reqGas = true; }
void requestImuReference() { reqImuRef = true; }

// Seconds left in the BME680 warm-up: 5 min on first start, 2 min when a
// baseline was saved earlier. The screen shows this countdown as "Air (warm)".
int airWarmLeft() {
  uint32_t burn = baselineFromNvs ? 120 : 300;
  uint32_t age = (millis() - gasStartMs) / 1000;
  return age < burn ? (int)(burn - age) : 0;
}

// Builds st.v from st.raw plus the simulation overrides. The caller must hold
// the state lock. With a fill override, the distance is worked backwards so
// the dashboard's distance and fill stay consistent.
void recomputeView() {
  Reading v = st.raw;
  const Sim& s = sim;
  const Cal& c = cal;
  if (s.mask & 1) {
    v.fill = constrain(s.fill, 0, 100);
    v.distance_cm = c.d_empty - v.fill / 100.0f * (c.d_empty - c.d_full);
  }
  if (s.mask & 2) v.weight_g = s.weight;
  if (s.mask & 4) {
    v.iaq = s.gas;
    v.iaq_acc = 3;
  }
  if (s.mask & 8) {
    v.temp = s.temp;
    v.temp_raw = s.temp - c.temp_off;
    v.humidity = s.humidity;
  }
  if (s.mask & 16) v.tilt = s.tamper ? max(c.tilt_th + 8.0f, 38.0f) : 0.0f;
  if (s.mask & 32) v.motion = s.motion;
  st.v = v;
}

// Probes each sensor once at boot. A sensor that fails here is retried by
// sensorTask, so a loose wire that reconnects comes back without a reset.
void sensorsInit() {
  Wire.begin(PIN_SDA, PIN_SCL);
  Wire.setClock(100000);

  pinMode(PIN_TRIG, OUTPUT);
  digitalWrite(PIN_TRIG, LOW);
  pinMode(PIN_ECHO, INPUT);

  // The BME680 address is 0x76 or 0x77 depending on its SDO pin; both are tried.
  // Heater 320 deg C for 150 ms is the Adafruit default for gas readings.
  bool bmeOk = bme.begin(0x76) || bme.begin(0x77);
  if (bmeOk) {
    bme.setTemperatureOversampling(BME680_OS_8X);
    bme.setHumidityOversampling(BME680_OS_2X);
    bme.setPressureOversampling(BME680_OS_4X);
    bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
    bme.setGasHeater(320, 150);
  }

  bool imuOk = imuStart();
  float x, y, z;
```

```
  if (loadImuRef(x, y, z)) { imuRef[0] = x; imuRef[1] = y; imuRef[2] = z; }

  scale.begin(PIN_HX_DT, PIN_HX_SCK);
  loadHx(hxOffset, hxScale);
  if (fabsf(hxScale) < 0.001f) hxScale = 420.0f;

  gasBaseline = loadGasBaseline();
  baselineFromNvs = gasBaseline > 0;
  gasStartMs = millis();

  lockState();
  st.okBME = bmeOk;
  st.okIMU = imuOk;
  unlockState();
}

// Median of up to 5 values. Throws away single bad ultrasonic echoes.
static float median5(float* a, int n) {
  float b[5];
  for (int i = 0; i < n; i++) b[i] = a[i];
  for (int i = 1; i < n; i++)
    for (int j = i; j > 0 && b[j - 1] > b[j]; j--) { float t = b[j]; b[j] = b[j - 1]; b[j - 1] = t; }
  return b[n / 2];
}

// Estimated IAQ index (0 good .. 500 bad) from gas resistance and humidity.
// This is not Bosch's BSEC algorithm: 25% of the score comes from humidity
// distance to 40% RH, 75% from gas resistance compared with the clean-air
// baseline. Accuracy rises from 1 to 3 over the 30 minutes after warm-up. Returns NaN
// during warm-up, and the old IAQ value is kept.
static float iaqFrom(float gasOhm, float hum, uint8_t& acc) {
  uint32_t age = (millis() - gasStartMs) / 1000;
  uint32_t burn = baselineFromNvs ? 120 : 300;
  if (age < burn) {
    if (gasOhm > gasBaseline) gasBaseline = gasOhm;
    acc = 0;
    return NAN;
  }
  if (gasOhm > gasBaseline) gasBaseline = gasOhm;
  else gasBaseline = gasBaseline * 0.99995f + gasOhm * 0.00005f;
  acc = age < burn + 600 ? 1 : age < burn + 1800 ? 2 : 3;
  const float humBase = 40.0f, humWeight = 0.25f;
  float hOff = hum - humBase;
  float humScore = hOff > 0 ? (100.0f - humBase - hOff) / (100.0f - humBase) * humWeight * 100.0f
                            : (humBase + hOff) / humBase * humWeight * 100.0f;
  float gOff = gasBaseline - gasOhm;
  float gasScore = gOff > 0 ? (gasOhm / gasBaseline) * (100.0f - humWeight * 100.0f) : 100.0f - humWeight * 100.0f;
  float quality = constrain(humScore + gasScore, 0.0f, 100.0f);
  return (100.0f - quality) * 5.0f;
}

// Runs on core 1, priority 3. Wakes every 50 ms or on EV_IMU. Each block below
// has its own timer so one slow sensor does not hold up the others.
void sensorTask(void*) {
  float dist[5];
  int distN = 0, distI = 0, echoMiss = 0;
  float hxBuf[5];
  int hxN = 0, hxI = 0;
  int32_t tareAcc = 0;
  int64_t calAcc = 0;
  int tareN = 0, calN = 0;
  uint32_t lastTrig = 0, lastImu = 0, lastHxOk = millis(), lastEnv = 0, envReadyAt = 0, lastRate = 0, lastBaseSave =
millis();
  bool envPending = false;
  int bmeFail = 0, imuFail = 0, sameCount = 0;
  int16_t lastA[3] = {0, 0, 0};
  float tiltF = 0;
  float rateBuf[31];
  int rateN = 0;
  uint8_t lastAcc = 0;
  bool firstEnv = false;
  Reading pub;
  bool pubFaults[4] = {false, false, false, false};
  uint32_t bootStart = millis();

  for (;;) {
    uint32_t ev = 0;
    xTaskNotifyWait(0, UINT32_MAX, &ev, pdMS_TO_TICKS(50));
    uint32_t now = millis();
    Cal c = calSnapshot();
    bool changed = false;

    // --- Ultrasonic fill level ---
    // 10 us trigger pulse, echo timeout 25 ms (about 4 m). us / 58 = cm.
    // Readings outside 2..400 cm are ignored. After 10 missed echoes in a row
    // the sensor is marked faulty. If fill stays at 0 or 100, check the
```

```
  // empty/full distances in the calibration, not the sensor.
  if (now - lastTrig >= 200) {
    lastTrig = now;
    digitalWrite(PIN_TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(PIN_TRIG, LOW);
    unsigned long us = pulseIn(PIN_ECHO, HIGH, 25000);
    if (us > 0) {
      float cm = us / 58.0f;
      if (cm >= 2 && cm <= 400) {
        dist[distI] = cm;
        distI = (distI + 1) % 5;
        if (distN < 5) distN++;
        echoMiss = 0;
      }
    } else if (++echoMiss > 10) {
      distN = 0;
    }
    bool ok = distN >= 3;
    float d = ok ? median5(dist, distN) : 0;
    float fill = ok ? constrain((c.d_empty - d) / (c.d_empty - c.d_full) * 100.0f, 0.0f, 100.0f) : 0;
    lockState();
    if (st.okHC != ok) { st.okHC = ok; changed = true; }
    st.raw.distance_cm = d;
    st.raw.fill = fill;
    unlockState();
  }

  // --- Tilt from the accelerometer ---
  // Tilt is the angle between the current gravity vector and the reference
  // saved at calibration. 25 identical readings in a row (5 s) mean the
  // chip has frozen, so it is restarted. A 0.6/0.4 low-pass filter stops a
  // single bump from starting the tamper countdown.
  if ((ev & EV_IMU) || now - lastImu >= 200) {
    lastImu = now;
    int16_t a[3] = {0, 0, 0};
    bool ok = imuAccel(a) && (a[0] || a[1] || a[2]);
    if (ok && a[0] == lastA[0] && a[1] == lastA[1] && a[2] == lastA[2]) {
      if (++sameCount >= 25) {
        sameCount = 0;
        ok = false;
        imuFail = 10;
        imuStart();
      }
    } else {
      sameCount = 0;
    }
    if (ok) {
      memcpy(lastA, a, sizeof(lastA));
      imuFail = 0;
      float x = a[0], y = a[1], z = a[2];
      float m = sqrtf(x * x + y * y + z * z);
      if (m > 1) {
        x /= m; y /= m; z /= m;
        if (reqImuRef) {
          reqImuRef = false;
          imuRef[0] = x; imuRef[1] = y; imuRef[2] = z;
          saveImuRef(x, y, z);
        }
        float dot = constrain(x * imuRef[0] + y * imuRef[1] + z * imuRef[2], -1.0f, 1.0f);
        float deg = acosf(dot) * 57.29578f;
        tiltF = tiltF * 0.6f + deg * 0.4f;
      }
    } else if (++imuFail > 10) {
      tiltF = 0;
    }
    lockState();
    bool okNow = imuFail <= 10;
    if (st.okIMU != okNow) { st.okIMU = okNow; changed = true; }
    st.raw.tilt = tiltF;
    unlockState();
  }

  // --- Load cell ---
  // TARE and CAL_WEIGHT average 10 raw samples before saving. The weight is
  // a moving average of 5 samples; anything under 5 g counts as 0. No sample
  // for 1.5 s marks the HX711 faulty (check DT 32 / SCK 14 and the 5V supply).
  if (scale.is_ready()) {
    long raw = scale.read();
    lastHxOk = now;
    if (reqTare) {
      tareAcc += raw;
      if (++tareN >= 10) {
        hxOffset = tareAcc / tareN;
        tareAcc = 0; tareN = 0; reqTare = false; hxN = 0;
        saveHx(hxOffset, hxScale);
```

```
        buzz(T_CAL_STEP);
      }
    } else if (reqCalW) {
      calAcc += raw;
      if (++calN >= 10) {
        float avg = (float)calAcc / calN;
        if (reqGrams > 0 && fabsf(avg - hxOffset) > 10) hxScale = (avg - hxOffset) / reqGrams;
        calAcc = 0; calN = 0; reqCalW = false; hxN = 0;
        saveHx(hxOffset, hxScale);
        buzz(T_CAL_STEP);
      }
    }
    hxBuf[hxI] = (raw - hxOffset) / hxScale;
    hxI = (hxI + 1) % 5;
    if (hxN < 5) hxN++;
    float sum = 0;
    for (int i = 0; i < hxN; i++) sum += hxBuf[i];
    float g = sum / hxN;
    if (fabsf(g) < 5) g = 0;
    lockState();
    if (!st.okHX) { st.okHX = true; changed = true; }
    st.raw.weight_g = g;
    unlockState();
  } else if (now - lastHxOk > 1500) {
    lockState();
    if (st.okHX) { st.okHX = false; changed = true; }
    unlockState();
  }

  // --- Gas baseline reset (RESET_GAS command) ---
  if (reqGas) {
    reqGas = false;
    gasBaseline = 0;
    baselineFromNvs = false;
    gasStartMs = now;
    saveGasBaseline(0);
  }

  // --- BME680, non-blocking ---
  // beginReading() starts a measurement and returns when it will be ready,
  // so the task never sleeps through the 150 ms heater cycle. Three failed
  // starts in a row mark the sensor faulty and re-run bme.begin().
  if (!envPending && now - lastEnv >= 3000) {
    lastEnv = now;
    envReadyAt = bme.beginReading();
    envPending = envReadyAt != 0;
    if (!envPending && ++bmeFail >= 3) {
      lockState();
      if (st.okBME) { st.okBME = false; changed = true; }
      unlockState();
      if (bme.begin(0x76) || bme.begin(0x77)) bme.setGasHeater(320, 150);
    }
  }
  if (envPending && now >= envReadyAt) {
    envPending = false;
    if (bme.endReading()) {
      bmeFail = 0;
      firstEnv = true;
      uint8_t acc = 0;
      float hum = constrain(bme.humidity + c.hum_off, 0.0f, 100.0f);
      float iaq = bme.gas_resistance > 0 ? iaqFrom(bme.gas_resistance, hum, acc) : NAN;
      lockState();
      if (!st.okBME) { st.okBME = true; changed = true; }
      st.raw.temp_raw = bme.temperature;
      st.raw.temp = bme.temperature + c.temp_off;
      st.raw.humidity = hum;
      st.raw.pressure = bme.pressure / 100.0f + c.press_off;
      st.raw.gas_k = bme.gas_resistance / 1000.0f;
      if (!isnan(iaq)) st.raw.iaq = constrain(iaq, 0.0f, 500.0f);
      st.raw.iaq_acc = acc;
      unlockState();
      // First usable IAQ after warm-up: beep once and tell the apps.
      if (acc >= 1 && lastAcc == 0) {
        buzz(T_AIR_READY);
        char meta[72];
        snprintf(meta, sizeof(meta), "%s air quality readings have started", BIN_LABEL);
        raiseAlert("sensor", "success", "Air sensor ready", meta);
      }
      lastAcc = acc;
      // Save the baseline at most once an hour to spare the flash.
      if (acc >= 3 && now - lastBaseSave > 3600000UL) {
        lastBaseSave = now;
        saveGasBaseline(gasBaseline);
      }
    } else if (++bmeFail >= 3) {
      lockState();
```

```
412         if (st.okBME) { st.okBME = false; changed = true; }
413         unlockState();
414       }
415     }
416
417     // --- Fill rate for the "Full in ~x h" estimate ---
418     // One sample a minute, up to 30 minutes. A drop of more than 5% means the
419     // bin was emptied, so the history restarts. Needs 5 samples (4 min).
420     if (now - lastRate >= 60000 || lastRate == 0) {
421       lastRate = now;
422       lockState();
423       float f = st.v.fill;
424       bool ok = st.okHC || (sim.mask & 1);
425       unlockState();
426       if (ok) {
427         if (rateN && f < rateBuf[rateN - 1] - 5) rateN = 0;
428         if (rateN == 31) { memmove(rateBuf, rateBuf + 1, 30 * sizeof(float)); rateN = 30; }
429         rateBuf[rateN++] = f;
430         float rate = rateN >= 5 ? (rateBuf[rateN - 1] - rateBuf[0]) / ((rateN - 1) / 60.0f) : 0;
431         lockState();
432         st.fillRatePerHour = rate > 0.05f ? rate : 0;
433         unlockState();
434       }
435     }
436
437     lockState();
438     // Leave the boot screen once the BME680 answered (or is known faulty) and
439     // at least 3.5 s have passed.
440     if (st.booting && (firstEnv || !st.okBME) && now - bootStart > 3500) {
441       st.booting = false;
442       changed = true;
443     }
444     recomputeView();
445     const Reading& v = st.v;
446     bool faults[4] = {st.okHC, st.okHX, st.okBME, st.okIMU};
447     // Publish only on a real change (1% fill, 20 g, 0.2 C, ...). This keeps the
448     // live dashboard and Firebase from being flooded by sensor noise.
449     if (fabsf(v.fill - pub.fill) >= 1 || fabsf(v.weight_g - pub.weight_g) >= 20 || fabsf(v.temp - pub.temp) >= 0.2f ||
450         fabsf(v.humidity - pub.humidity) >= 1 || fabsf(v.iaq - pub.iaq) >= 5 || fabsf(v.tilt - pub.tilt) >= 1 ||
451         fabsf(v.pressure - pub.pressure) >= 0.5f || v.iaq_acc != pub.iaq_acc || memcmp(faults, pubFaults, sizeof(faults)))
452       changed = true;
453     if (changed) {
454       pub = v;
455       memcpy(pubFaults, faults, sizeof(faults));
456     }
457     unlockState();
458     if (changed) {
459       publish();
460       if (hControl) xTaskNotify(hControl, EV_STATE, eSetBits);
461     }
462   }
463 }
```

firmware/SmartBin/control.h

```
1 // control.h - lid, LED, alarms and the command runner (controlTask).
2 #pragma once
3 #include <Arduino.h>
4
5 void controlInit();
6 void controlTask(void*);
7 void isrPir();
```

firmware/SmartBin/control.cpp

```
1  // ============================================================================
2  // control.cpp - decides what the bin does
3  // Runs every command from cmdQueue, works out the status, moves the lid,
4  // drives the RGB LED and raises the full / tamper / environment / sensor
5  // alarms. Priority 4 on core 1, the highest of all tasks, so the lid and
6  // alarms react even while the screen or cloud is busy.
7  // ============================================================================
8  #include "control.h"
9  #include "state.h"
10 #include "sensors.h"
11 #include "buzzer.h"
12 #include "ui.h"
13 #include <ESP32Servo.h>
14
15 static Servo lid;
16 static int lidAngle = -1;
17
18 // PIR interrupt. Only wakes controlTask; the pin itself is read in the task.
19 void IRAM_ATTR isrPir() {
```

```
  BaseType_t woke = pdFALSE;
  if (hControl) xTaskNotifyFromISR(hControl, EV_PIR, eSetBits, &woke);
  portYIELD_FROM_ISR(woke);
}

// LED colors (see updateLed): blue = Bluetooth setup, blinking red = tamper
// or full alarm, cyan = calibrating, green = collecting or OK, magenta =
// locked, yellow = 50% full or more.
static void led(bool r, bool g, bool b) {
  digitalWrite(PIN_LED_R, r ^ LED_ACTIVE_LOW);
  digitalWrite(PIN_LED_G, g ^ LED_ACTIVE_LOW);
  digitalWrite(PIN_LED_B, b ^ LED_ACTIVE_LOW);
}

// The servo uses LEDC timer 0 at 50 Hz with 500..2400 us pulses (SG90).
// The lid is driven closed at boot so a reset never leaves it open.
void controlInit() {
  pinMode(PIN_LED_R, OUTPUT);
  pinMode(PIN_LED_G, OUTPUT);
  pinMode(PIN_LED_B, OUTPUT);
  led(false, false, true);
  pinMode(PIN_PIR, INPUT);
  ESP32PWM::allocateTimer(0);
  lid.setPeriodHertz(50);
  lid.attach(PIN_SERVO, 500, 2400);
  Cal c = calSnapshot();
  lid.write(c.servo_closed);
  lidAngle = c.servo_closed;
}

static void setLid(int angle) {
  if (angle == lidAngle) return;
  lid.write(angle);
  lidAngle = angle;
}

// Records the outcome of a command. For commands from Firebase the result is
// queued so cloudTask can write it to /sb/results/<ref>, which is how the app
// knows its button press worked. A failed command from a remote source beeps.
static void finish(const Command& c, bool ok, const char* why) {
  lockState();
  strlcpy(st.last.cmd, c.cmd, sizeof(st.last.cmd));
  st.last.ts = nowMs();
  st.last.ok = ok;
  unlockState();
  if (c.src == SRC_FIREBASE && c.ref[0]) {
    CmdResult r{};
    strlcpy(r.cmd, c.cmd, sizeof(r.cmd));
    r.ok = ok;
    strlcpy(r.why, why ? why : "", sizeof(r.why));
    r.src = c.src;
    strlcpy(r.ref, c.ref, sizeof(r.ref));
    r.ts = nowMs();
    xQueueSend(resultQueue, &r, 0);
  }
  if (!ok && c.src != SRC_SCREEN) buzz(T_CMD_FAIL);
  publish(EV_RESULT);
}

// Runs one command. State changes happen under the lock; sounds and alerts
// come after it is released, because buzz() takes the same lock (the mutex
// is not recursive, so calling it while holding the lock would hang the task).
static void runCommand(const Command& c) {
  BinState s = snapshot();
  String why;
  if (!validateCommand(c.cmd, s, why)) {
    finish(c, false, why.c_str());
    return;
  }
  const char* k = c.cmd;
  char meta[72];
  lockState();
  if (!strcmp(k, "LOCK")) {
    st.locked = true;
    st.autoLocked = false;
  } else if (!strcmp(k, "UNLOCK")) {
    st.locked = false;
    st.autoLocked = false;
  } else if (!strcmp(k, "CLEAR_TAMPER")) {
    st.tamperPhase = 0;
  } else if (!strcmp(k, "COLLECT")) {
    st.collecting = true;
  // Collection finishing also releases an automatic (full-bin) lock, but not
  // a lock that a person set.
  } else if (!strcmp(k, "COLLECT_DONE")) {
    if (st.collecting) st.doneAtMs = millis();
```

```
    st.collecting = false;
    if (st.autoLocked) { st.locked = false; st.autoLocked = false; }
  } else if (!strcmp(k, "CALIBRATE_START")) {
    st.calibrating = true;
    st.calSteps = 0;
  } else if (!strcmp(k, "CALIBRATE_END")) {
    st.calibrating = false;
  } else if (!strcmp(k, "TARE")) {
    st.calSteps |= 1;
  }
  unlockState();

  if (!strcmp(k, "LOCK")) {
    buzz(T_LOCK);
    snprintf(meta, sizeof(meta), "%s locked", BIN_LABEL);
    raiseAlert("lock", "info", "Bin locked", meta);
  } else if (!strcmp(k, "UNLOCK")) {
    buzz(T_UNLOCK);
    snprintf(meta, sizeof(meta), "%s is open for use again", BIN_LABEL);
    raiseAlert("lock", "success", "Bin unlocked", meta);
  } else if (!strcmp(k, "CLEAR_TAMPER")) {
    buzz(T_TAMPER_SETTLED);
  } else if (!strcmp(k, "COLLECT")) {
    if (c.src != SRC_SCREEN) buzz(T_PIN_OK);
    snprintf(meta, sizeof(meta), "%s pickup started", BIN_LABEL);
    raiseAlert("pickup", "info", "Collection started", meta);
  } else if (!strcmp(k, "COLLECT_DONE")) {
    buzz(T_DONE);
    snprintf(meta, sizeof(meta), "%s is back to normal", BIN_LABEL);
    raiseAlert("pickup", "success", "Collection complete", meta);
  } else if (!strcmp(k, "CALIBRATE_START")) {
    requestImuReference();
    buzz(T_CAL_START);
    snprintf(meta, sizeof(meta), "%s is being calibrated", BIN_LABEL);
    raiseAlert("setup", "info", "Calibration started", meta);
  } else if (!strcmp(k, "CALIBRATE_END")) {
    buzz(c.src == SRC_SCREEN ? T_CAL_CANCEL : T_SAVED);
  } else if (!strcmp(k, "TARE")) {
    requestTare();
  } else if (!strcmp(k, "CAL_WEIGHT")) {
    requestCalWeight(c.grams > 0 ? c.grams : 500);
  } else if (!strcmp(k, "RESET_GAS")) {
    requestGasReset();
    buzz(T_CAL_STEP);
  } else if (!strcmp(k, "TOUCH_CAL")) {
    uiStartTouchCal();
  } else if (!strcmp(k, "TOUCH_CAL_RESET")) {
    uiResetTouchCal();
  } else if (!strcmp(k, "BUZZ_TEST")) {
    if (!buzzByName(c.sound[0] ? c.sound : "full")) {
      finish(c, false, "Unknown sound");
      return;
    }
  }
  finish(c, true, nullptr);
}

// Called every 100 ms. The first matching state wins.
static void updateLed(const BinState& s, const Cal& c, uint32_t now) {
  bool blink = (now / 500) % 2;
  if (s.btOn) led(false, false, true);
  else if (s.tamperPhase) led(blink, false, false);
  else if (s.calibrating) led(false, true, true);
  else if (s.collecting) led(false, true, false);
  else if (s.status == ST_LOCKED) led(true, false, true);
  else if (s.alarmFull) led(blink, false, false);
  else if (s.v.fill >= 50) led(true, true, false);
  else led(false, true, false);
}

// Wakes every 100 ms, on PIR change, on a new command or on a sensor update.
void controlTask(void*) {
  uint32_t lastTick = 0, lastWave = 0, lastFullBuzz = 0, lastEnvBuzz = 0, lastTamperTick = 0;
  bool prevFull = false, prevEnv = false, prevHC = true, prevHX = true, prevBME = true;
  bool prevMotion = false, firstPass = true;
  BinStatus prevStatus = ST_OK;
  uint8_t prevMask = 0;

  for (;;) {
    uint32_t ev = 0;
    xTaskNotifyWait(0, UINT32_MAX, &ev, pdMS_TO_TICKS(100));

    Command c;
    // Drain all waiting commands before evaluating the state.
    while (xQueueReceive(cmdQueue, &c, 0) == pdTRUE) runCommand(c);

```

```
    uint32_t now = millis();
    Cal cl = calSnapshot();
    bool pir = digitalRead(PIN_PIR);
    bool changed = false;
    char meta[72];

    lockState();
    st.raw.motion = pir;
    recomputeView();
    bool motion = st.v.motion;
    bool hcOk = st.okHC || (sim.mask & 1);
    // Alarms are ignored while collecting or calibrating, otherwise opening
    // the bin to empty it would set them off.
    bool fullNow = hcOk && st.v.fill >= cl.fill_th && !st.collecting && !st.calibrating;
    bool tiltBad = (st.okIMU || (sim.mask & 16)) && st.v.tilt > cl.tilt_th && !st.collecting && !st.calibrating;

    // --- Tamper state machine ---
    // 0 none -> 1 countdown (TAMPER_COUNTDOWN_S, beeps each second) -> 2 latched.
    // Going back under the tilt limit during the countdown cancels it. Once
    // latched it stays on until CLEAR_TAMPER, even if the bin is set upright.
    // The lock is released around buzz() because buzz() takes it itself.
    if (tiltBad && st.tamperPhase == 0) {
      st.tamperPhase = 1;
      st.tamperStartMs = now;
      changed = true;
    } else if (!tiltBad && st.tamperPhase == 1) {
      st.tamperPhase = 0;
      changed = true;
      unlockState();
      buzz(T_TAMPER_SETTLED);
      lockState();
    } else if (st.tamperPhase == 1 && now - st.tamperStartMs >= TAMPER_COUNTDOWN_S * 1000) {
      st.tamperPhase = 2;
      changed = true;
      snprintf(meta, sizeof(meta), "%s was moved or tilted (%.0f deg)", BIN_LABEL, st.v.tilt);
      unlockState();
      raiseAlert("tamper", "critical", "Tamper detected", meta);
      lockState();
    }

    // Optional auto-lock when full (setting "auto_lock"). Off by default.
    if (cl.auto_lock && fullNow && !st.locked && !st.collecting && !st.tamperPhase) {
      st.locked = true;
      st.autoLocked = true;
      changed = true;
      snprintf(meta, sizeof(meta), "%s locked itself at %.0f%%", BIN_LABEL, st.v.fill);
      unlockState();
      buzz(T_LOCK);
      raiseAlert("lock", "info", "Bin locked", meta);
      lockState();
    }

    // One status, highest priority first.
    BinStatus status = st.calibrating ? ST_CALIBRATING : st.collecting ? ST_COLLECTING
                     : st.tamperPhase ? ST_TAMPER : st.locked ? ST_LOCKED : ST_OK;
    if (status != st.status) { st.status = status; changed = true; }

    // --- Lid decision ---
    // Collecting: open. Locked, tamper or calibrating: closed. Lid simulated:
    // follow the simulation. Otherwise motion opens the lid, and it closes
    // pir_hold_s seconds after the last motion. If the lid opens and closes by
    // itself, check the PIR's own delay knob (it should be at minimum).
    bool wantOpen;
    if (status == ST_COLLECTING) {
      wantOpen = true;
      st.lidCloseAtMs = 0;
    } else if (status != ST_OK) {
      wantOpen = false;
      st.lidCloseAtMs = 0;
    } else if (sim.mask & 64) {
      wantOpen = sim.lid;
      st.lidCloseAtMs = 0;
    } else if (motion) {
      wantOpen = true;
      st.lidCloseAtMs = 0;
    } else if (st.lidOpen && st.lidCloseAtMs == 0) {
      st.lidCloseAtMs = now + cl.pir_hold_s * 1000UL;
      wantOpen = true;
    } else if (st.lidOpen && (int32_t)(now - st.lidCloseAtMs) < 0) {
      wantOpen = true;
    } else {
      wantOpen = false;
      st.lidCloseAtMs = 0;
    }
    bool lidChanged = wantOpen != st.lidOpen;
    if (lidChanged) { st.lidOpen = wantOpen; changed = true; }
```

```
278     if (motion != prevMotion) changed = true;
279
280     // Environment alarm: temperature or humidity over its limit, or IAQ over
281     // gas_th once the air sensor has warmed up.
282     st.alarmFull = fullNow && status == ST_OK;
283     bool gasOk = st.okBME || (sim.mask & 4);
284     bool envBad = !st.collecting && !st.calibrating &&
285                   (((st.okBME || (sim.mask & 8)) && (st.v.temp >= cl.temp_th || st.v.humidity >= cl.hum_th)) ||
286                    (gasOk && st.v.iaq_acc >= 1 && st.v.iaq >= cl.gas_th));
287     st.alarmEnv = envBad;
288     st.buzzerOn = buzzerBusy() && (st.alarmFull || st.tamperPhase || envBad);
289     BinState s = st;
290     unlockState();
291
292     // Hardware is touched only after the lock is released.
293     setLid(s.lidOpen ? cl.servo_open : cl.servo_closed);
294     if (lidChanged) buzz(s.lidOpen ? T_LID_OPEN : T_LID_CLOSE);
295     if (motion && !prevMotion && (s.status == ST_LOCKED || s.status == ST_TAMPER) && now - lastWave > 5000) {
296       lastWave = now;
297       buzz(T_LOCKED_WAVE);
298     }
299     prevMotion = motion;
300
301     // Countdown beeps change pitch for the last 3 seconds; the latched siren
302     // repeats every TAMPER_ALARM_EVERY_MS.
303     if (s.tamperPhase == 1 && now - lastTamperTick >= 1000) {
304       lastTamperTick = now;
305       uint32_t left = TAMPER_COUNTDOWN_S - min<uint32_t>(TAMPER_COUNTDOWN_S, (now - s.tamperStartMs) / 1000);
306       buzz(left <= 3 ? T_TAMPER_TICK_END : T_TAMPER_TICK);
307       changed = true;
308     }
309     if (s.tamperPhase == 2 && now - lastTamperTick >= TAMPER_ALARM_EVERY_MS) {
310       lastTamperTick = now;
311       buzz(T_TAMPER_LATCHED);
312     }
313
314     // Full alarm: alert once on the rising edge, then repeat the beep every
315     // buzz_repeat_s seconds. buzzerUntilMs drives the BUZZER row on screen.
316     if (s.alarmFull && (!prevFull || now - lastFullBuzz >= cl.buzz_repeat_s * 1000UL)) {
317       if (!prevFull) {
318         snprintf(meta, sizeof(meta), "%s is at %.0f%%", BIN_LABEL, s.v.fill);
319         raiseAlert("full", "critical", "Fill threshold passed", meta);
320       }
321       lastFullBuzz = now;
322       buzz(T_FULL);
323       lockState();
324       st.buzzerUntilMs = now + max<uint32_t>(cl.buzz_s, 1) * 1000UL;
325       unlockState();
326     }
327     prevFull = s.alarmFull;
328
329     // Environment alarm repeats every 5 minutes while it lasts.
330     if (envBad && (!prevEnv || now - lastEnvBuzz >= 300000)) {
331       if (!prevEnv) {
332         if (s.v.temp >= cl.temp_th) snprintf(meta, sizeof(meta), "%s at %.1f C, limit %.0f", BIN_LABEL, s.v.temp,
    cl.temp_th);
333         else if (s.v.humidity >= cl.hum_th) snprintf(meta, sizeof(meta), "%s at %.0f%%, limit %.0f%%", BIN_LABEL,
    s.v.humidity, cl.hum_th);
334         else snprintf(meta, sizeof(meta), "%s index %.0f, limit %.0f", BIN_LABEL, s.v.iaq, cl.gas_th);
335         raiseAlert("sensor", "warning", "Reading over its limit", meta);
336       }
337       lastEnvBuzz = now;
338       buzz(T_ENV_OVER);
339     } else if (!envBad && prevEnv) {
340       buzz(T_ENV_OK);
341     }
342     prevEnv = envBad;
343
344     // Sensor lost / back alerts. The first pass after boot only records the
345     // starting state and plays the boot sound (OK or FAULT).
346     if (!firstPass && !s.booting) {
347       const bool oks[3] = {s.okHC, s.okHX, s.okBME};
348       bool* prevs[3] = {&prevHC, &prevHX, &prevBME};
349       const char* names[3] = {"HC-SR04", "HX711", "BME680"};
350       for (int i = 0; i < 3; i++) {
351         if (oks[i] == *prevs[i]) continue;
352         *prevs[i] = oks[i];
353         buzz(oks[i] ? T_SENSOR_BACK : T_SENSOR_LOST);
354         snprintf(meta, sizeof(meta), "%s: %s %s", BIN_LABEL, names[i], oks[i] ? "answers again" : "stopped answering");
355         raiseAlert("sensor", oks[i] ? "success" : "warning", oks[i] ? "Sensor back" : "Sensor not answering", meta);
356       }
357     } else if (!s.booting) {
358       prevHC = s.okHC; prevHX = s.okHX; prevBME = s.okBME;
359       firstPass = false;
360       buzz(s.okHC && s.okHX && s.okBME && s.okIMU ? T_BOOT_OK : T_BOOT_FAULT);
361     }
```

```
362
363     // Sound when simulation is switched on or off from the dashboard.
364     Sim sm = simSnapshot();
365     if ((sm.mask != 0) != (prevMask != 0)) buzz(sm.mask ? T_SIM_ON : T_SIM_OFF);
366     prevMask = sm.mask;
367
368     if (s.status != prevStatus) {
369       prevStatus = s.status;
370       changed = true;
371     }
372     if (now - lastTick >= 100) {
373       lastTick = now;
374       updateLed(s, cl, now);
375     }
376     if (changed) publish();
377   }
378 }
```

firmware/SmartBin/buzzer.h

```
1  // buzzer.h - every sound the bin makes. The Tune order must match TUNES[]
2  // in buzzer.cpp (a static_assert checks the count).
3  #pragma once
4  #include <Arduino.h>
5
6  enum Tune : uint8_t {
7    T_KEY, T_PIN_KEY, T_PIN_DEL, T_CANCEL, T_THEME_LIGHT, T_THEME_DARK, T_OPEN, T_PIN_OK, T_PIN_WRONG, T_DONE,
8    T_BOOT_OK, T_BOOT_FAULT, T_LID_OPEN, T_LID_CLOSE, T_LOCKED_WAVE, T_LOCK, T_UNLOCK, T_CAL_START, T_CAL_STEP,
9    T_SAVED, T_SIM_ON, T_SIM_OFF, T_CAL_CANCEL, T_AIR_READY, T_BACK_MAIN,
10   T_FULL, T_TAMPER_TICK, T_TAMPER_TICK_END, T_TAMPER_LATCHED, T_ENV_OVER, T_SENSOR_LOST, T_TAMPER_SETTLED,
11   T_ENV_OK, T_SENSOR_BACK,
12   T_WIFI_UP, T_WIFI_LOST, T_CLOUD_DOWN, T_CLOUD_UP, T_CMD_FAIL,
13   T_BT_WAIT, T_BT_CODE, T_BT_PAIRED, T_BT_WRONG, T_BT_STOPPED, T_BT_JOINED, T_BT_JOIN_FAIL, T_BT_OFF, T_BT_AUTO_OFF,
14   T_COUNT
15 };
16
17 void buzzerInit();
18 void buzz(Tune t);
19 bool buzzByName(const char* name);
20 bool buzzerBusy();
```

firmware/SmartBin/buzzer.cpp

```
1  // ============================================================================
2  // buzzer.cpp - non-blocking sound player
3  // buzz() drops a tune ID into a one-slot queue; buzzerTask plays it with LEDC
4  // PWM on GPIO 4. Callers never wait for a sound to finish.
5  // ============================================================================
6  #include "buzzer.h"
7  #include "state.h"
8
9  // A note is a frequency in Hz and a length in ms. hz 0 is a rest. With hzEnd
10 // set the pitch slides from hz to hzEnd (used for the tamper siren).
11 // pri: higher can interrupt lower. kind decides which mute setting applies.
12 struct Note { uint16_t hz, ms, hzEnd; };
13 struct TuneDef { const char* name; const Note* n; uint8_t len; uint8_t pri; uint8_t kind; };
14 enum : uint8_t { K_UI = 0, K_LID, K_EVENT, K_CAL, K_ALARM };
15
16 // --- Tune data. Names match the sound names used by the apps and dashboard. ---
17 static const Note n_key[] = {{2400, 15}};
18 static const Note n_pin_key[] = {{2600, 12}};
19 static const Note n_pin_del[] = {{2000, 15}};
20 static const Note n_cancel[] = {{2300, 30}, {1800, 45}};
21 static const Note n_theme_light[] = {{2300, 35}, {2700, 50}};
22 static const Note n_theme_dark[] = {{2000, 35}, {1700, 50}};
23 static const Note n_open[] = {{1800, 30}, {2300, 40}};
24 static const Note n_pin_ok[] = {{1600, 60}, {1900, 60}, {2200, 60}, {1900, 60}, {2200, 120}};
25 static const Note n_pin_wrong[] = {{1700, 60}, {0, 30}, {1700, 60}, {0, 30}, {1500, 150}};
26 static const Note n_done[] = {{1800, 70}, {2100, 70}, {2400, 70}, {2800, 200}};
27 static const Note n_boot_ok[] = {{1600, 70}, {1900, 70}, {2200, 70}, {2800, 180}};
28 static const Note n_boot_fault[] = {{1600, 70}, {1900, 70}, {0, 40}, {1700, 100}, {0, 60}, {1500, 210}};
29 static const Note n_lid_open[] = {{1800, 25}, {2200, 30}};
30 static const Note n_lid_close[] = {{2200, 25}, {1800, 30}};
31 static const Note n_locked_wave[] = {{1600, 60}, {0, 40}, {1500, 120}};
32 static const Note n_lock[] = {{2000, 40}, {0, 20}, {1600, 120}};
33 static const Note n_unlock[] = {{1600, 40}, {0, 20}, {2000, 120}};
34 static const Note n_cal_start[] = {{2300, 40}, {0, 50}, {2300, 40}, {0, 50}, {2300, 40}, {0, 50}, {2700, 120}};
35 static const Note n_cal_step[] = {{2400, 40}};
36 static const Note n_saved[] = {{2200, 60}, {2600, 100}};
37 static const Note n_sim_on[] = {{1600, 60}, {1800, 60}, {2000, 60}, {2300, 140}};
38 static const Note n_sim_off[] = {{2300, 60}, {2000, 60}, {1800, 60}, {1600, 140}};
39 static const Note n_cal_cancel[] = {{2400, 40}, {0, 40}, {1900, 120}};
40 static const Note n_air_ready[] = {{2000, 40}, {2400, 70}};
41 static const Note n_back_main[] = {{2400, 30}, {2000, 50}};
```

```
static const Note n_full[] = {{2000, 180}, {0, 80}, {2000, 180}, {0, 80}, {2000, 180}, {0, 200}, {2600, 420}};
static const Note n_tamper_tick[] = {{2000, 60}};
static const Note n_tamper_tick_end[] = {{2400, 60}};
static const Note n_tamper[] = {{1600, 340, 2800}, {2800, 340, 1600}, {1600, 340, 2800}, {2800, 340, 1600}};
static const Note n_env_over[] = {{2400, 120}, {1900, 120}, {2400, 120}, {1900, 260}};
static const Note n_sensor_lost[] = {{1900, 100}, {0, 60}, {1500, 210}};
static const Note n_tamper_settled[] = {{1900, 60}, {2300, 60}, {2600, 120}};
static const Note n_env_ok[] = {{2600, 50}, {2200, 100}};
static const Note n_sensor_back[] = {{1500, 80}, {2000, 150}};
static const Note n_wifi_up[] = {{1800, 70}, {2200, 70}, {2700, 150}};
static const Note n_wifi_lost[] = {{2700, 90}, {2200, 90}, {1800, 190}};
static const Note n_cloud_down[] = {{2200, 60}, {0, 30}, {2400, 60}, {0, 30}, {1900, 130}};
static const Note n_cloud_up[] = {{1900, 60}, {0, 30}, {2400, 60}, {0, 30}, {2700, 120}};
static const Note n_cmd_fail[] = {{1900, 80}, {0, 30}, {1700, 80}, {0, 30}, {1500, 200}};
static const Note n_bt_wait[] = {{1800, 30}, {0, 30}, {1800, 30}, {2300, 60}};
static const Note n_bt_code[] = {{2400, 30}, {0, 60}, {2400, 30}};
static const Note n_bt_paired[] = {{2300, 40}, {2800, 40}, {2300, 40}, {2800, 140}};
static const Note n_bt_wrong[] = {{1800, 50}, {0, 25}, {1800, 50}, {1500, 140}};
static const Note n_bt_stopped[] = {{2200, 180}, {0, 50}, {1500, 320}};
static const Note n_bt_joined[] = {{1700, 60}, {2000, 60}, {2400, 60}, {2800, 180}};
static const Note n_bt_join_fail[] = {{1600, 70}, {0, 40}, {1500, 120}};
static const Note n_bt_off[] = {{2300, 50}, {1800, 50}, {0, 25}, {1500, 140}};
static const Note n_bt_auto_off[] = {{2300, 50}, {1800, 50}, {0, 25}, {1500, 60}, {0, 60}, {1500, 60}};

// Name, notes, note count, priority (0 low .. 5 alarm), kind.
static const TuneDef TUNES[] = {
  {"key", n_key, sizeof(n_key) / sizeof(Note), 0, K_UI},
  {"pin_key", n_pin_key, sizeof(n_pin_key) / sizeof(Note), 0, K_UI},
  {"pin_del", n_pin_del, sizeof(n_pin_del) / sizeof(Note), 0, K_UI},
  {"cancel", n_cancel, sizeof(n_cancel) / sizeof(Note), 0, K_UI},
  {"theme_light", n_theme_light, sizeof(n_theme_light) / sizeof(Note), 0, K_UI},
  {"theme_dark", n_theme_dark, sizeof(n_theme_dark) / sizeof(Note), 0, K_UI},
  {"open", n_open, sizeof(n_open) / sizeof(Note), 0, K_UI},
  {"pin_ok", n_pin_ok, sizeof(n_pin_ok) / sizeof(Note), 3, K_EVENT},
  {"pin_wrong", n_pin_wrong, sizeof(n_pin_wrong) / sizeof(Note), 3, K_EVENT},
  {"done", n_done, sizeof(n_done) / sizeof(Note), 3, K_EVENT},
  {"boot_ok", n_boot_ok, sizeof(n_boot_ok) / sizeof(Note), 1, K_EVENT},
  {"boot_fault", n_boot_fault, sizeof(n_boot_fault) / sizeof(Note), 4, K_EVENT},
  {"lid_open", n_lid_open, sizeof(n_lid_open) / sizeof(Note), 0, K_LID},
  {"lid_close", n_lid_close, sizeof(n_lid_close) / sizeof(Note), 0, K_LID},
  {"locked_wave", n_locked_wave, sizeof(n_locked_wave) / sizeof(Note), 2, K_EVENT},
  {"lock", n_lock, sizeof(n_lock) / sizeof(Note), 3, K_EVENT},
  {"unlock", n_unlock, sizeof(n_unlock) / sizeof(Note), 3, K_EVENT},
  {"cal_start", n_cal_start, sizeof(n_cal_start) / sizeof(Note), 2, K_CAL},
  {"cal_step", n_cal_step, sizeof(n_cal_step) / sizeof(Note), 1, K_CAL},
  {"saved", n_saved, sizeof(n_saved) / sizeof(Note), 2, K_CAL},
  {"sim_on", n_sim_on, sizeof(n_sim_on) / sizeof(Note), 2, K_EVENT},
  {"sim_off", n_sim_off, sizeof(n_sim_off) / sizeof(Note), 2, K_EVENT},
  {"cal_cancel", n_cal_cancel, sizeof(n_cal_cancel) / sizeof(Note), 2, K_CAL},
  {"air_ready", n_air_ready, sizeof(n_air_ready) / sizeof(Note), 1, K_EVENT},
  {"back_main", n_back_main, sizeof(n_back_main) / sizeof(Note), 1, K_EVENT},
  {"full", n_full, sizeof(n_full) / sizeof(Note), 5, K_ALARM},
  {"tamper_tick", n_tamper_tick, sizeof(n_tamper_tick) / sizeof(Note), 5, K_ALARM},
  {"tamper_tick_end", n_tamper_tick_end, sizeof(n_tamper_tick_end) / sizeof(Note), 5, K_ALARM},
  {"tamper", n_tamper, sizeof(n_tamper) / sizeof(Note), 5, K_ALARM},
  {"env_over", n_env_over, sizeof(n_env_over) / sizeof(Note), 4, K_ALARM},
  {"sensor_lost", n_sensor_lost, sizeof(n_sensor_lost) / sizeof(Note), 4, K_EVENT},
  {"tamper_settled", n_tamper_settled, sizeof(n_tamper_settled) / sizeof(Note), 4, K_EVENT},
  {"env_ok", n_env_ok, sizeof(n_env_ok) / sizeof(Note), 2, K_EVENT},
  {"sensor_back", n_sensor_back, sizeof(n_sensor_back) / sizeof(Note), 3, K_EVENT},
  {"wifi_up", n_wifi_up, sizeof(n_wifi_up) / sizeof(Note), 2, K_EVENT},
  {"wifi_lost", n_wifi_lost, sizeof(n_wifi_lost) / sizeof(Note), 3, K_EVENT},
  {"cloud_down", n_cloud_down, sizeof(n_cloud_down) / sizeof(Note), 2, K_EVENT},
  {"cloud_up", n_cloud_up, sizeof(n_cloud_up) / sizeof(Note), 2, K_EVENT},
  {"cmd_fail", n_cmd_fail, sizeof(n_cmd_fail) / sizeof(Note), 3, K_EVENT},
  {"bt_wait", n_bt_wait, sizeof(n_bt_wait) / sizeof(Note), 1, K_EVENT},
  {"bt_code", n_bt_code, sizeof(n_bt_code) / sizeof(Note), 2, K_EVENT},
  {"bt_paired", n_bt_paired, sizeof(n_bt_paired) / sizeof(Note), 3, K_EVENT},
  {"bt_wrong", n_bt_wrong, sizeof(n_bt_wrong) / sizeof(Note), 3, K_EVENT},
  {"bt_stopped", n_bt_stopped, sizeof(n_bt_stopped) / sizeof(Note), 4, K_EVENT},
  {"bt_joined", n_bt_joined, sizeof(n_bt_joined) / sizeof(Note), 3, K_EVENT},
  {"bt_join_fail", n_bt_join_fail, sizeof(n_bt_join_fail) / sizeof(Note), 3, K_EVENT},
  {"bt_off", n_bt_off, sizeof(n_bt_off) / sizeof(Note), 2, K_EVENT},
  {"bt_auto_off", n_bt_auto_off, sizeof(n_bt_auto_off) / sizeof(Note), 1, K_EVENT},
};

static_assert(sizeof(TUNES) / sizeof(TUNES[0]) == T_COUNT, "tune table and enum differ");

// playing holds the priority of the tune now sounding, 0xFF when silent.
static QueueHandle_t q;
static volatile uint8_t playing = 0xFF;
static constexpr uint8_t CH = 6, RES = 10;

// Volume 1/2/3 maps to PWM duty 100/256/512 of 1023 (50% is the loudest a
// passive buzzer gets).
static uint16_t duty() {
```

```
  uint8_t v = calSnapshot().buzz_vol;
  uint16_t d = v >= 3 ? 512 : v == 2 ? 256 : 100;
  return BUZZER_ACTIVE_LOW ? (1023 - d) : d;
}

static void silence() {
  ledcWriteTone(PIN_BUZZER, 0);
  ledcWrite(PIN_BUZZER, BUZZER_ACTIVE_LOW ? 1023 : 0);
}

static void tone(uint16_t hz) {
  ledcWriteTone(PIN_BUZZER, hz);
  ledcWrite(PIN_BUZZER, duty());
}

// Quiet hours, which may wrap past midnight (e.g. 22:00 to 06:00).
static bool quietNow(const Cal& c) {
  int m = localMinuteOfDay();
  if (!c.quiet_on || m < 0) return false;
  return c.quiet_from <= c.quiet_to ? (m >= c.quiet_from && m < c.quiet_to) : (m >= c.quiet_from || m < c.quiet_to);
}

// Plays one tune at a time. A new tune waiting in the queue cuts the current
// one short at the next note.
static void buzzerTask(void*) {
  uint8_t id;
  for (;;) {
    if (xQueueReceive(q, &id, portMAX_DELAY) != pdTRUE) continue;
    const TuneDef& t = TUNES[id];
    playing = t.pri;
    for (uint8_t i = 0; i < t.len; i++) {
      if (uxQueueMessagesWaiting(q)) break;
      const Note& n = t.n[i];
      if (!n.hz) {
        silence();
        vTaskDelay(pdMS_TO_TICKS(n.ms));
      } else if (n.hzEnd) {
        for (uint16_t s = 0; s < n.ms; s += 10) {
          tone(n.hz + (int32_t)(n.hzEnd - n.hz) * s / n.ms);
          vTaskDelay(pdMS_TO_TICKS(10));
        }
      } else {
        tone(n.hz);
        vTaskDelay(pdMS_TO_TICKS(n.ms));
      }
    }
    silence();
    playing = 0xFF;
  }
}

// 2 kHz carrier, 10-bit resolution, LEDC channel 6.
void buzzerInit() {
  ledcAttachChannel(PIN_BUZZER, 2000, RES, CH);
  silence();
  q = xQueueCreate(1, sizeof(uint8_t));
  xTaskCreatePinnedToCore(buzzerTask, "buzzer", 2048, nullptr, 2, nullptr, 1);
}

// Filters, in order: master switch, UI and lid sound switches, quiet hours
// (only priority-5 alarms play), alarms muted while collecting, only
// calibration and UI sounds while calibrating, and no interrupting a
// higher-priority tune. If a sound never plays, check these settings first.
void buzz(Tune id) {
  if (id >= T_COUNT || !q) return;
  const TuneDef& t = TUNES[id];
  Cal c = calSnapshot();
  BinState s = snapshot();
  if (!c.buzz_on) return;
  if (t.kind == K_UI && !c.buzz_ui) return;
  if (t.kind == K_LID && !c.buzz_lid) return;
  if (quietNow(c) && t.pri < 5) return;
  if (s.collecting && t.kind == K_ALARM) return;
  if (s.calibrating && t.kind != K_CAL && t.kind != K_UI) return;
  if (playing != 0xFF && t.pri < playing) return;
  uint8_t v = id;
  xQueueOverwrite(q, &v);
}

// Used by BUZZ_TEST from the apps. Skips all filters so the test always plays.
bool buzzByName(const char* name) {
  for (uint8_t i = 0; i < T_COUNT; i++)
    if (!strcmp(TUNES[i].name, name)) {
      uint8_t v = i;
      if (q) xQueueOverwrite(q, &v);
      return true;
```

```
    }
  return false;
}

bool buzzerBusy() { return playing != 0xFF; }
```

firmware/SmartBin/net.h

```
// net.h - Wi-Fi, the local web server (dashboard + REST API) and live events.
#pragma once
#include <Arduino.h>

void netInit();
void wifiConnect();
void startWebServer();
void pushTask(void*);
```

firmware/SmartBin/net.cpp

```
// ============================================================================
// net.cpp - Wi-Fi station, mDNS, SNTP and the local HTTP API
// Local API (port 80, CORS open so the dashboard file can be opened anywhere):
//   GET  /             the web dashboard (gzipped, from dashboard_gz.h)
//   GET  /api/status   full status JSON including calibration
//   GET  /api/events   Server-Sent Events: "status" on change, "ping" every 10 s
//   POST /api/command  {"cmd": "LOCK", "grams": 500, "sound": "full"}
//   POST /api/calibrate  any Cal keys, e.g. {"fill_th": 80}
//   POST /api/simulate {"mask": 3, "values": {"fill": 90}}
//   POST /api/cloud    {"url", "key", "email", "pass"} for Firebase
// Quick test from a laptop on the same network:
//   curl http://smartbin-01.local/api/status
// ============================================================================
#include "net.h"
#include "state.h"
#include "sensors.h"
#include "buzzer.h"
#include <WiFi.h>
#include <ESPmDNS.h>
#include <ESPAsyncWebServer.h>
#include <AsyncJson.h>
#include <esp_sntp.h>

// dashboard_gz.h is generated by tools/embed_dashboard.py. Without it the bin
// still builds and serves a small fallback page.
#if __has_include("dashboard_gz.h")
#include "dashboard_gz.h"
#define HAVE_DASHBOARD 1
#else
#define HAVE_DASHBOARD 0
#endif

static AsyncWebServer server(80);
static AsyncEventSource events("/api/events");
static bool everConnected = false;

// Wi-Fi callback, runs in the Wi-Fi event task. On GOT_IP it starts mDNS
// (smartbin-0x.local), starts SNTP time once, beeps and prints the address:
//   [wifi] 192.168.1.23  http://smartbin-01.local
// If that line never appears, the SSID or password is wrong or the network
// is 5 GHz only (the ESP32 supports 2.4 GHz only).
static void onWifiEvent(arduino_event_id_t event, arduino_event_info_t) {
  if (event == ARDUINO_EVENT_WIFI_STA_GOT_IP) {
    lockState();
    st.wifi = true;
    st.rssi = WiFi.RSSI();
    unlockState();
    MDNS.end();
    if (MDNS.begin(BIN_HOST)) MDNS.addService("http", "tcp", 80);
    if (!esp_sntp_enabled()) {
      esp_sntp_setoperatingmode(ESP_SNTP_OPMODE_POLL);
      esp_sntp_setservername(0, "pool.ntp.org");
      esp_sntp_setservername(1, "time.google.com");
      esp_sntp_init();
    }
    buzz(T_WIFI_UP);
    everConnected = true;
    Serial.printf("[wifi] %s  http://%s.local\n", WiFi.localIP().toString().c_str(), BIN_HOST);
    publish(EV_WIFI);
  } else if (event == ARDUINO_EVENT_WIFI_STA_DISCONNECTED) {
    bool was;
    lockState();
    was = st.wifi;
    st.wifi = false;
    unlockState();
```

```
    if (was) {
      buzz(T_WIFI_LOST);
      Serial.println("[wifi] lost");
      publish(EV_WIFI);
    }
  }
}

// persistent(false): the ESP32 SDK does not store credentials; this firmware
// keeps them in its own Preferences (see saveNet).
void netInit() {
  WiFi.persistent(false);
  WiFi.mode(WIFI_STA);
  WiFi.setHostname(BIN_HOST);
  WiFi.setAutoReconnect(true);
  WiFi.onEvent(onWifiEvent);
}

// Uses the saved network, or the one in secrets.h. Auto-reconnect handles drops.
void wifiConnect() {
  if (net.ssid.length() == 0 || net.ssid == "your-wifi-name") {
    Serial.println("[wifi] no network set; use WI-FI SETUP on the touchscreen");
    return;
  }
  WiFi.begin(net.ssid.c_str(), net.pass.c_str());
}

static void sendJson(AsyncWebServerRequest* r, int code, const String& body) {
  AsyncWebServerResponse* res = r->beginResponse(code, "application/json", body);
  res->addHeader("Cache-Control", "no-store");
  r->send(res);
}

// The status the bin will have after the command runs, returned at once so
// the dashboard can update its button without waiting for the next event.
static const char* expectStatus(const char* cmd, const BinState& s) {
  if (!strcmp(cmd, "LOCK")) return "LOCKED";
  if (!strcmp(cmd, "COLLECT")) return "COLLECTING";
  if (!strcmp(cmd, "CALIBRATE_START")) return "CALIBRATING";
  if (!strcmp(cmd, "UNLOCK") || !strcmp(cmd, "CLEAR_TAMPER") || !strcmp(cmd, "COLLECT_DONE") || !strcmp(cmd,
"CALIBRATE_END"))
    return s.tamperPhase && strcmp(cmd, "CLEAR_TAMPER") ? "TAMPER" : (s.locked && strcmp(cmd, "UNLOCK") && !s.autoLocked) ?
"LOCKED" : "OK";
  return STATUS_NAMES[s.status];
}

// POST /api/command. 400 = unknown command, 409 = not allowed right now (the
// body says why), 503 = command queue full.
static void handleCommand(AsyncWebServerRequest* r, JsonVariant& j) {
  const char* cmd = j["cmd"] | "";
  BinState s = snapshot();
  String why;
  JsonDocument out;
  out["cmd"] = cmd;
  if (!validateCommand(cmd, s, why)) {
    out["ok"] = false;
    out["why"] = why;
    String b;
    serializeJson(out, b);
    sendJson(r, why == "Unknown command" ? 400 : 409, b);
    return;
  }
  if (!sendCommand(cmd, SRC_LOCAL, j["grams"] | 0.0f, j["sound"] | (const char*)nullptr)) {
    sendJson(r, 503, "{\"ok\":false,\"why\":\"Busy, try again\"}");
    return;
  }
  out["ok"] = true;
  out["status"] = expectStatus(cmd, s);
  String b;
  serializeJson(out, b);
  sendJson(r, 200, b);
}

// POST /api/calibrate. Sending d_empty or d_full during calibration ticks
// the matching step on the bin's calibration screen.
static void handleCalibrate(AsyncWebServerRequest* r, JsonVariant& j) {
  if (!j.is<JsonObject>()) { sendJson(r, 400, "{\"ok\":false,\"why\":\"Send a JSON object\"}"); return; }
  JsonObjectConst o = j.as<JsonObjectConst>();
  lockState();
  Cal c = cal;
  bool any = calFromJson(o, c);
  if (any) {
    cal = c;
    if (!o["d_empty"].isNull()) st.calSteps |= 2;
    if (!o["d_full"].isNull()) st.calSteps |= 4;
    st.calVersion++;
```

```
    recomputeView();
  }
  bool calibrating = st.calibrating;
  unlockState();
  if (!any) { sendJson(r, 400, "{\"ok\":false,\"why\":\"No known calibration keys\"}"); return; }
  saveCal();
  buzz(calibrating ? T_CAL_STEP : T_SAVED);
  publish(EV_CAL);
  sendJson(r, 200, "{\"ok\":true}");
}

// POST /api/simulate. mask 0 turns simulation off. Values not sent keep their
// previous simulated value.
static void handleSimulate(AsyncWebServerRequest* r, JsonVariant& j) {
  lockState();
  sim.mask = (uint8_t)(j["mask"] | 0) & 0x7F;
  JsonObjectConst v = j["values"].as<JsonObjectConst>();
  if (!v.isNull()) {
    sim.fill = v["fill"] | sim.fill;
    sim.weight = v["weight"] | sim.weight;
    sim.gas = v["gas"] | sim.gas;
    sim.temp = v["temp"] | sim.temp;
    sim.humidity = v["humidity"] | sim.humidity;
    if (!v["tamper"].isNull()) sim.tamper = v["tamper"].as<bool>();
    if (!v["motion"].isNull()) sim.motion = v["motion"].as<bool>();
    if (!v["lid"].isNull()) sim.lid = v["lid"].as<bool>() || (v["lid"].is<const char*>() && !strcmp(v["lid"], "OPEN"));
  }
  recomputeView();
  unlockState();
  if (hControl) xTaskNotify(hControl, EV_STATE, eSetBits);
  publish();
  sendJson(r, 200, "{\"ok\":true}");
}

// POST /api/cloud. Saves Firebase settings and tells cloudTask to sign in
// again. An empty URL turns Firebase off.
static void handleCloud(AsyncWebServerRequest* r, JsonVariant& j) {
  String url = j["url"] | "", key = j["key"] | "", email = j["email"] | "", pass = j["pass"] | "";
  url.trim();
  while (url.endsWith("/")) url.remove(url.length() - 1);
  if (url.length() && !url.startsWith("https://") && !url.startsWith("http://")) {
    sendJson(r, 400, "{\"ok\":false,\"why\":\"The database URL starts with https://\"}");
    return;
  }
  net.fbUrl = url;
  net.fbKey = key;
  net.fbEmail = email;
  net.fbPass = pass;
  saveNet();
  if (hCloud) xTaskNotify(hCloud, EV_WIFI, eSetBits);
  sendJson(r, 200, "{\"ok\":true}");
}

// The extra CORS header allows the dashboard, opened from a public page,
// to call a bin on the private network (Chrome's Private Network Access).
void startWebServer() {
  DefaultHeaders::Instance().addHeader("Access-Control-Allow-Origin", "*");
  DefaultHeaders::Instance().addHeader("Access-Control-Allow-Methods", "GET, POST, OPTIONS");
  DefaultHeaders::Instance().addHeader("Access-Control-Allow-Headers", "Content-Type, X-SB-Op");
  DefaultHeaders::Instance().addHeader("Access-Control-Allow-Private-Network", "true");

  server.on("/api/status", HTTP_GET, [](AsyncWebServerRequest* r) { sendJson(r, 200, statusJson(true, false)); });

  auto add = [](const char* uri, ArJsonRequestHandlerFunction fn) {
    AsyncCallbackJsonWebHandler* h = new AsyncCallbackJsonWebHandler(uri, fn);
    h->setMethod(HTTP_POST);
    h->setMaxContentLength(1024);
    server.addHandler(h);
  };
  add("/api/command", handleCommand);
  add("/api/calibrate", handleCalibrate);
  add("/api/simulate", handleSimulate);
  add("/api/cloud", handleCloud);

  // A new live-event client gets the full status at once; 3000 is the
  // reconnect delay in ms suggested to the browser.
  events.onConnect([](AsyncEventSourceClient* c) { c->send(statusJson(true, false).c_str(), "status", millis(), 3000); });
  server.addHandler(&events);

  server.on("/", HTTP_GET, [](AsyncWebServerRequest* r) {
#if HAVE_DASHBOARD
    AsyncWebServerResponse* res = r->beginResponse(200, "text/html", DASHBOARD_GZ, DASHBOARD_GZ_LEN);
    res->addHeader("Content-Encoding", "gzip");
    r->send(res);
#else
    r->send(200, "text/html",
```

```
            "<!doctype html><meta name=viewport content='width=device-width'><title>" BIN_LABEL "</title>"
            "<body style='font-family:sans-serif;padding:24px'><h1>" BIN_LABEL "</h1>"
            "<p>The dashboard is not built into this firmware. Open <a href='/api/status'>/api/status</a>, or run "
            "tools/embed_dashboard.py and upload again.</p>");
#endif
  });

  server.onNotFound([](AsyncWebServerRequest* r) {
    if (r->method() == HTTP_OPTIONS) r->send(204);
    else sendJson(r, 404, "{\"ok\":false,\"why\":\"Not found\"}");
  });
  server.begin();
}

// Sends a "status" event to all live dashboards when st.version changes, at
// most every 250 ms, and a "ping" every 10 s so browsers notice a dead link.
// Also refreshes the Wi-Fi signal strength every 5 s.
void pushTask(void*) {
  uint32_t lastSent = 0, lastPush = 0, lastPing = 0, lastRssi = 0;
  for (;;) {
    uint32_t ev = 0;
    xTaskNotifyWait(0, UINT32_MAX, &ev, pdMS_TO_TICKS(250));
    uint32_t now = millis();
    if (now - lastRssi > 5000 && WiFi.isConnected()) {
      lastRssi = now;
      lockState();
      st.rssi = WiFi.RSSI();
      unlockState();
    }
    if (!events.count()) continue;
    lockState();
    uint32_t v = st.version;
    unlockState();
    if (v != lastSent && now - lastPush >= 250) {
      events.send(statusJson(true, false).c_str(), "status", now);
      lastSent = v;
      lastPush = now;
    }
    if (now - lastPing >= 10000) {
      lastPing = now;
      events.send("1", "ping", now);
    }
  }
}
```

firmware/SmartBin/cloud.h

```
// cloud.h - Firebase and ThingSpeak (runs as cloudTask on core 0).
#pragma once
#include <Arduino.h>

void cloudTask(void*);
bool cloudFirebaseOn();
```

firmware/SmartBin/cloud.cpp

```
// ============================================================================
// cloud.cpp - Firebase Realtime Database and ThingSpeak over HTTPS (REST)
// Firebase paths used (rules in firmware/firebase.rules.json):
//   sb/bins/<id>/status   status JSON (PUT on change, at least every 60 s)
//   sb/bins/<id>/cal      calibration (PUT when it changes)
//   sb/commands/<id>/<k>  commands from the app: read, then deleted
//   sb/results/<k>        result of each command, written back by the bin
//   sb/alerts/<id>        alert log, trimmed to the newest 50
//   sb/watchers           app/dashboard heartbeats (who is looking)
//   sb/usage              estimated monthly download, shared by both bins
// ThingSpeak fields: 1 fill %, 2 weight g, 3 temp C, 4 humidity %, 5 IAQ,
// 6 pressure hPa, 7 tilt deg, 8 flags (1 motion + 2 lid open + 4 x status).
// Serial log tags: [firebase], [thingspeak], [talkback], [alert].
// ============================================================================
#include "cloud.h"
#include "state.h"
#include "buzzer.h"
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <HTTPClient.h>
#include <vector>
#include <algorithm>

// One HTTP client per service, each with its own retry timer, so a slow
// ThingSpeak does not delay Firebase.
struct Svc {
  WiFiClientSecure tls;
  WiFiClient plain;
```

```
  HTTPClient http;
  uint32_t retryAt = 0;
  uint8_t fails = 0;
};

static Svc fb, ts, auth;
static String idToken;
static uint32_t tokenUntil = 0;
static bool fbOk = false;
static int watchers = 0;
static uint64_t monthBytes = 0;
static String monthKey;
static float usageFraction = 0;

bool cloudFirebaseOn() { return net.fbUrl.length() && net.fbKey.length() && net.fbEmail.length() && net.fbPass.length(); }

// One HTTP request with keep-alive. Returns the HTTP code, or a negative
// value for a connection error (no Wi-Fi, DNS, TLS handshake).
static int request(Svc& s, const String& url, const char* method, const String& body, String* out,
                   const char* ctype = "application/json") {
  Client& c = url.startsWith("https://") ? (Client&)s.tls : (Client&)s.plain;
  s.http.setReuse(true);
  s.http.setTimeout(8000);
  s.http.setConnectTimeout(6000);
  if (!s.http.begin((NetworkClient&)c, url)) return -1;
  if (body.length()) s.http.addHeader("Content-Type", ctype);
  int code = s.http.sendRequest(method, (uint8_t*)body.c_str(), body.length());
  if (out) *out = code > 0 ? s.http.getString() : String();
  else if (code > 0) s.http.getString();
  s.http.end();
  return code;
}

// After a failure, wait 5, 15, 30 then 60 s before the next try and close the
// connection so the next attempt starts fresh.
static void backoff(Svc& s) {
  static const uint16_t steps[] = {5, 15, 30, 60};
  s.retryAt = millis() + steps[min<uint8_t>(s.fails, 3)] * 1000UL;
  if (s.fails < 10) s.fails++;
  s.tls.stop();
  s.plain.stop();
}

static bool due(const Svc& s) { return (int32_t)(millis() - s.retryAt) >= 0; }

// Tracks whether Firebase is reachable; the change beeps, logs and shows on
// the Info screen as "Firebase OK" or "NOT SENDING".
static void setFbOk(bool ok) {
  if (ok == fbOk) return;
  fbOk = ok;
  lockState();
  st.cloudOk = ok;
  unlockState();
  buzz(ok ? T_CLOUD_UP : T_CLOUD_DOWN);
  Serial.printf("[firebase] %s\n", ok ? "reachable" : "cannot be reached");
  publish();
}

// URL-encodes keys and tokens for query strings.
static String enc(const String& s) {
  String o;
  const char* hex = "0123456789ABCDEF";
  for (size_t i = 0; i < s.length(); i++) {
    char ch = s[i];
    if (isalnum((unsigned char)ch) || ch == '-' || ch == '_' || ch == '.' || ch == '~') o += ch;
    else { o += '%'; o += hex[(ch >> 4) & 15]; o += hex[ch & 15]; }
  }
  return o;
}

// Email/password sign-in through Firebase Auth's REST API. The ID token lasts
// about an hour; it is renewed 5 minutes early. A "sign-in failed (400)" line
// usually means a wrong API key, email or password, or that Email/Password
// sign-in is not enabled in the Firebase console. The http:// branch is for
// the local mock server used in testing.
static bool signIn() {
  String url = net.fbUrl.startsWith("http://")
                   ? net.fbUrl + "/__auth/signInWithPassword?key=" + enc(net.fbKey)
                   : "https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=" + enc(net.fbKey);
  JsonDocument d;
  d["email"] = net.fbEmail;
  d["password"] = net.fbPass;
  d["returnSecureToken"] = true;
  String body, resp;
  serializeJson(d, body);
  auth.http.setReuse(false);
```

```
  Client& c = url.startsWith("https://") ? (Client&)auth.tls : (Client&)auth.plain;
  auth.http.setTimeout(10000);
  if (!auth.http.begin((NetworkClient&)c, url)) return false;
  auth.http.addHeader("Content-Type", "application/json");
  int code = auth.http.POST(body);
  if (code > 0) resp = auth.http.getString();
  auth.http.end();
  auth.tls.stop();
  if (code != 200) {
    Serial.printf("[firebase] sign-in failed (%d) %s\n", code, resp.substring(0, 120).c_str());
    return false;
  }
  JsonDocument r;
  if (deserializeJson(r, resp)) return false;
  idToken = r["idToken"] | "";
  uint32_t secs = atoi(r["expiresIn"] | "3600");
  tokenUntil = millis() + (secs > 400 ? secs - 300 : secs) * 1000UL;
  return idToken.length() > 0;
}

static String fbUrl(const String& path, const String& query = "") {
  return net.fbUrl + "/" + path + ".json?auth=" + enc(idToken) + query;
}

// Every Firebase call goes through here: signs in when needed, retries once
// on 401 (expired token), backs off on network or 5xx errors. Writes are
// counted to estimate monthly download (each write is downloaded once per
// app or dashboard watching).
static int fbCall(const char* method, const String& path, const String& body, String* out, const String& query = "",
                  bool count = true) {
  if (!idToken.length() || (int32_t)(millis() - tokenUntil) >= 0) {
    if (!signIn()) {
      backoff(fb);
      setFbOk(false);
      return -1;
    }
  }
  int code = request(fb, fbUrl(path, query), method, body, out);
  if (code == 401) {
    idToken = "";
    code = request(fb, fbUrl(path, query), method, body, out);
  }
  if (code <= 0 || code >= 500) {
    backoff(fb);
    setFbOk(false);
    return code;
  }
  fb.fails = 0;
  setFbOk(code < 400);
  if (count && (!strcmp(method, "PUT") || !strcmp(method, "PATCH") || !strcmp(method, "POST")))
    monthBytes += (uint64_t)body.length() * max(1, watchers);
  return code;
}

static String currentMonth() {
  char b[12];
  monthKeyUtc(b, sizeof(b));
  return b;
}

// Restores this month's byte count after a reboot.
static void loadUsage() {
  String out;
  if (fbCall("GET", String("sb/usage/by_bin/") + BIN_ID, "", &out, "", false) != 200) return;
  JsonDocument d;
  if (deserializeJson(d, out) || !d.is<JsonObject>()) return;
  String m = d["month"] | "";
  monthKey = currentMonth();
  if (m == monthKey) monthBytes = d["bytes"] | (uint64_t)0;
}

// Saves this bin's count, then sums both bins and publishes the shared
// usage level (ok / warn 70% / near 90% / bad 100%) for the apps to show.
static void writeUsage() {
  String m = currentMonth();
  if (!m.length()) return;
  if (m != monthKey) { monthKey = m; monthBytes = 0; }
  JsonDocument me;
  me["month"] = m;
  me["bytes"] = monthBytes;
  String b;
  serializeJson(me, b);
  fbCall("PUT", String("sb/usage/by_bin/") + BIN_ID, b, nullptr, "", false);
  String out;
  if (fbCall("GET", "sb/usage/by_bin", "", &out, "", false) != 200) return;
  JsonDocument all;
```

```
  if (deserializeJson(all, out)) return;
  uint64_t total = 0;
  for (JsonPair p : all.as<JsonObject>())
    if (m == (p.value()["month"] | "")) total += p.value()["bytes"] | (uint64_t)0;
  usageFraction = (float)((double)total / (double)FB_MONTH_LIMIT);
  JsonDocument u;
  u["month"] = m;
  u["bytes_month"] = total;
  u["down_bytes"] = total;
  u["limit_bytes"] = FB_MONTH_LIMIT;
  u["fraction"] = round(usageFraction * 10000.0) / 10000.0;
  u["level"] = usageFraction >= 1 ? "bad" : usageFraction >= 0.9 ? "near" : usageFraction >= 0.7 ? "warn" : "ok";
  u["estimate"] = true;
  b = "";
  serializeJson(u, b);
  fbCall("PATCH", "sb/usage", b, nullptr, "", false);
}

// Counts apps/dashboards that sent a heartbeat in the last 30 s. Uses the
// "ts" index when the clock is set; falls back to reading the whole list.
static void readWatchers() {
  String out;
  int code = -1;
  if (clockSet()) {
    uint64_t since = nowMs() - 30000ULL;
    code = fbCall("GET", "sb/watchers", "", &out, "&orderBy=%22ts%22&startAt=" + String((unsigned long long)since), false);
  }
  if (code != 200) code = fbCall("GET", "sb/watchers", "", &out, "", false);
  if (code != 200) return;
  JsonDocument d;
  if (deserializeJson(d, out) || !d.is<JsonObject>()) { watchers = 0; return; }
  int n = 0;
  uint64_t now = nowMs();
  for (JsonPair p : d.as<JsonObject>()) {
    uint64_t t = p.value()["ts"] | (uint64_t)0;
    if (!now || (t && now - t <= 30000ULL)) n++;
  }
  watchers = n;
}

// Reads queued commands and deletes each one first, so a command never runs
// twice after a reboot. Only LOCK, UNLOCK and COLLECT are accepted from the
// cloud; calibration and simulation need the bin's own Wi-Fi. Commands older
// than CMD_MAX_AGE_MS are refused, so a lost phone cannot replay old taps.
static void pollCommands() {
  String out;
  if (fbCall("GET", String("sb/commands/") + BIN_ID, "", &out, "", false) != 200) return;
  if (out == "null" || out.length() < 3) return;
  JsonDocument d;
  if (deserializeJson(d, out) || !d.is<JsonObject>()) return;
  uint64_t now = nowMs();
  for (JsonPair p : d.as<JsonObject>()) {
    String key = p.key().c_str();
    const char* cmd = p.value()["cmd"] | "";
    uint64_t t = p.value()["ts"] | (uint64_t)0;
    fbCall("DELETE", String("sb/commands/") + BIN_ID + "/" + key, "", nullptr, "", false);
    bool stale = now && t && now > t && now - t > CMD_MAX_AGE_MS;
    bool allowed = !strcmp(cmd, "LOCK") || !strcmp(cmd, "UNLOCK") || !strcmp(cmd, "COLLECT");
    if (stale || !allowed) {
      JsonDocument r;
      r["ok"] = false;
      r["cmd"] = cmd;
      r["why"] = stale ? "Too old, ignored" : "Needs the bin's Wi-Fi";
      r["ts"][".sv"] = "timestamp";
      String b;
      serializeJson(r, b);
      fbCall("PUT", "sb/results/" + key, b, nullptr);
      continue;
    }
    Serial.printf("[firebase] command %s\n", cmd);
    if (!sendCommand(cmd, SRC_FIREBASE, 0, nullptr, key.c_str())) {
      JsonDocument r;
      r["ok"] = false;
      r["cmd"] = cmd;
      r["why"] = "Busy";
      String b;
      serializeJson(r, b);
      fbCall("PUT", "sb/results/" + key, b, nullptr);
    }
  }
}

// Writes queued command results to sb/results/<ref>.
static void flushResults() {
  CmdResult r;
  while (xQueueReceive(resultQueue, &r, 0) == pdTRUE) {
```

```
    if (!r.ref[0]) continue;
    JsonDocument d;
    d["ok"] = r.ok;
    d["cmd"] = r.cmd;
    if (r.why[0]) d["why"] = r.why;
    d["ts"][".sv"] = "timestamp";
    String b;
    serializeJson(d, b);
    fbCall("PUT", String("sb/results/") + r.ref, b, nullptr);
  }
}

// Posts queued alerts (POST makes a time-ordered key). Always logged to Serial.
static void flushAlerts(bool fbOn) {
  AlertEv a;
  while (xQueueReceive(alertQueue, &a, 0) == pdTRUE) {
    Serial.printf("[alert] %s: %s (%s)\n", a.title, a.meta, a.sev);
    if (!fbOn) continue;
    JsonDocument d;
    d["kind"] = a.kind;
    d["sev"] = a.sev;
    d["title"] = a.title;
    d["meta"] = a.meta;
    d["ts"][".sv"] = "timestamp";
    String b;
    serializeJson(d, b);
    fbCall("POST", String("sb/alerts/") + BIN_ID, b, nullptr);
  }
}

// Keeps the newest 50 alerts. Keys made by POST sort by time, so the lowest
// keys are the oldest.
static void trimAlerts() {
  String out;
  if (fbCall("GET", String("sb/alerts/") + BIN_ID, "", &out, "&shallow=true", false) != 200) return;
  JsonDocument d;
  if (deserializeJson(d, out) || !d.is<JsonObject>()) return;
  JsonObject o = d.as<JsonObject>();
  int n = o.size();
  if (n <= 50) return;
  std::vector<String> keys;
  for (JsonPair p : o) keys.push_back(p.key().c_str());
  std::sort(keys.begin(), keys.end());
  for (int i = 0; i < n - 50; i++) fbCall("DELETE", String("sb/alerts/") + BIN_ID + "/" + keys[i], "", nullptr, "", false);
}

// What was last sent to a service, used to skip uploads when nothing changed.
struct Sent {
  float fill = -100, weight = -1000, temp = -100, hum = -100, iaq = -100, tilt = -100;
  uint8_t status = 255, tamper = 255, acc = 255;
  bool lid = false, motion = false, full = false, env = false;
  uint8_t faults = 255;
};

static uint8_t faultBits(const BinState& s) { return (!s.okHC) | (!s.okHX << 1) | (!s.okBME << 2) | (!s.okIMU << 3); }

// A change in status, lid, tamper, alarms or faults: sent right away.
static bool importantChange(const Sent& a, const BinState& s) {
  return a.status != s.status || a.lid != s.lidOpen || a.tamper != s.tamperPhase || a.full != s.alarmFull ||
         a.env != s.alarmEnv || a.faults != faultBits(s);
}

// Sensor drift above the same steps used in sensorTask: sent on the slower pace.
static bool minorChange(const Sent& a, const BinState& s) {
  const Reading& v = s.v;
  return fabsf(a.fill - v.fill) >= 1 || fabsf(a.weight - v.weight_g) >= 20 || fabsf(a.temp - v.temp) >= 0.2f ||
         fabsf(a.hum - v.humidity) >= 1 || fabsf(a.iaq - v.iaq) >= 5 || fabsf(a.tilt - v.tilt) >= 1 ||
         a.motion != v.motion || a.acc != v.iaq_acc;
}

static void remember(Sent& a, const BinState& s) {
  a.fill = s.v.fill; a.weight = s.v.weight_g; a.temp = s.v.temp; a.hum = s.v.humidity; a.iaq = s.v.iaq;
  a.tilt = s.v.tilt; a.status = s.status; a.tamper = s.tamperPhase; a.acc = s.v.iaq_acc; a.lid = s.lidOpen;
  a.motion = s.v.motion; a.full = s.alarmFull; a.env = s.alarmEnv; a.faults = faultBits(s);
}

// One ThingSpeak channel update. The reply is the entry number; "0" means the
// write was refused (wrong key, or less than 15 s since the last write).
static bool tsWrite(const BinState& s) {
  const Reading& v = s.v;
  int flags = (v.motion ? 1 : 0) + (s.lidOpen ? 2 : 0) + 4 * s.status;
  String url = net.tsServer + "/update?api_key=" + enc(net.tsWriteKey) + "&field1=" + String(v.fill, 1) +
               "&field2=" + String(v.weight_g, 0) + "&field3=" + String(v.temp, 2) + "&field4=" + String(v.humidity, 1) +
               "&field5=" + String(v.iaq, 0) + "&field6=" + String(v.pressure, 1) + "&field7=" + String(v.tilt, 1) +
               "&field8=" + String(flags) + "&status=" + enc(localIp());
  String out;
```

```
  int code = request(ts, url, "GET", "", &out);
  out.trim();
  bool ok = code == 200 && out != "0";
  if (code <= 0 || code >= 500) backoff(ts);
  else ts.fails = 0;
  lockState();
  st.tsOk = ok;
  unlockState();
  if (!ok) Serial.printf("[thingspeak] write failed (%d) %s\n", code, out.c_str());
  return ok;
}

static Svc tb;

// ThingSpeak TalkBack: takes the next queued command (LOCK, UNLOCK or COLLECT).
static void talkBack() {
  String url = net.tsServer + "/talkbacks/" + enc(net.tbId) + "/commands/execute.json";
  String out;
  int code = request(tb, url, "POST", "api_key=" + enc(net.tbKey), &out, "application/x-www-form-urlencoded");
  if (code <= 0 || code >= 500) { backoff(tb); return; }
  tb.fails = 0;
  if (code != 200 || out.length() < 3) return;
  JsonDocument d;
  if (deserializeJson(d, out)) return;
  String cmd = d["command_string"] | "";
  cmd.trim();
  cmd.toUpperCase();
  if (cmd == "LOCK" || cmd == "UNLOCK" || cmd == "COLLECT") {
    Serial.printf("[talkback] command %s\n", cmd.c_str());
    sendCommand(cmd.c_str(), SRC_TALKBACK);
  }
}

// Main cloud loop, every 500 ms or when woken. Paused while Wi-Fi is down or
// Bluetooth setup is running (the radio is shared), in which case alerts are
// only printed to Serial.
void cloudTask(void*) {
  for (Svc* s : {&fb, &ts, &auth, &tb}) {
    // The server certificate is not checked (no CA bundle on the board). The
    // traffic is still encrypted.
    s->tls.setInsecure();
    s->tls.setHandshakeTimeout(12);
  }
  Sent fbSent, tsSent;
  uint32_t lastStatus = 0, lastCmd = 0, lastWatch = 0, lastUsage = 0, lastTrim = 0, lastTs = 0, lastTb = 0, lastCalVer = 0;
  bool usageLoaded = false, wasConnected = false;
  bool calSent = false;

  for (;;) {
    uint32_t ev = 0;
    xTaskNotifyWait(0, UINT32_MAX, &ev, pdMS_TO_TICKS(500));
    // New network or new Firebase settings: sign in again from scratch.
    if (ev & EV_WIFI) {
      idToken = "";
      fb.retryAt = 0;
      fb.fails = 0;
      calSent = false;
    }
    BinState s = snapshot();
    bool online = WiFi.isConnected() && !s.btOn;
    bool fbOn = cloudFirebaseOn();
    if (!online) {
      if (wasConnected) {
        for (Svc* x : {&fb, &ts, &tb}) { x->tls.stop(); x->plain.stop(); }
      }
      wasConnected = false;
      AlertEv a;
      while (xQueueReceive(alertQueue, &a, 0) == pdTRUE) Serial.printf("[alert] %s: %s\n", a.title, a.meta);
      continue;
    }
    wasConnected = true;
    uint32_t now = millis();

    // --- Firebase ---
    // Status pace: right away on an important change, every 5 s on minor
    // changes while someone watches (15 s above 90% of the free quota,
    // 60 s at 100%), and a 60 s heartbeat otherwise.
    if (fbOn && due(fb)) {
      if (!usageLoaded && clockSet()) {
        usageLoaded = true;
        loadUsage();
      }
      flushResults();
      flushAlerts(true);

      if (now - lastWatch >= FB_WATCHERS_MS || lastWatch == 0) {
```

```
        lastWatch = now;
        readWatchers();
      }
      bool watched = watchers > 0 || !FB_REQUIRE_WATCHERS;
      uint32_t every = !watched ? FB_IDLE_MS : usageFraction >= 0.9f ? 15000 : FB_WATCHED_MS;
      if (usageFraction >= 1.0f) every = FB_IDLE_MS;
      bool imp = importantChange(fbSent, s);
      bool minor = minorChange(fbSent, s);
      if (imp || (minor && now - lastStatus >= every) || now - lastStatus >= FB_IDLE_MS || lastStatus == 0) {
        String body = statusJson(false, true);
        if (fbCall("PUT", String("sb/bins/") + BIN_ID + "/status", body, nullptr) == 200) {
          remember(fbSent, s);
          lastStatus = now;
        }
      }
      if (!calSent || s.calVersion != lastCalVer) {
        JsonDocument d;
        calToJson(calSnapshot(), d.to<JsonObject>());
        String b;
        serializeJson(d, b);
        if (fbCall("PUT", String("sb/bins/") + BIN_ID + "/cal", b, nullptr) == 200) {
          calSent = true;
          lastCalVer = s.calVersion;
        }
      }
      if (now - lastCmd >= (watched ? FB_CMD_WATCHED_MS : FB_CMD_IDLE_MS)) {
        lastCmd = now;
        pollCommands();
      }
      if (now - lastUsage >= FB_USAGE_MS || lastUsage == 0) {
        lastUsage = now;
        writeUsage();
      }
      if (now - lastTrim >= 600000 || lastTrim == 0) {
        lastTrim = now;
        trimAlerts();
      }
    } else if (!fbOn) {
      flushAlerts(false);
      CmdResult r;
      while (xQueueReceive(resultQueue, &r, 0) == pdTRUE) {}
    }

    // --- ThingSpeak: every 30 s on change, 15.5 s on an alarm edge, and a
    // 5 minute heartbeat so the charts never show a gap. ---
    if (net.tsWriteKey.length() && due(ts)) {
      bool alertEdge = s.alarmFull != tsSent.full || s.tamperPhase != tsSent.tamper || s.status != tsSent.status;
      uint32_t since = now - lastTs;
      bool write = lastTs == 0 || (alertEdge && since >= TS_MIN_GAP_MS) ||
                   (since >= TS_WRITE_MS && (minorChange(tsSent, s) || importantChange(tsSent, s))) || since >=
TS_HEARTBEAT_MS;
      if (write) {
        lastTs = now;
        if (tsWrite(s)) remember(tsSent, s);
      }
    }

    // --- TalkBack polling every 5 s (15 s when nobody watches) ---
    if (net.tbId.length() && net.tbKey.length() && due(tb)) {
      uint32_t every = (fbOn && watchers == 0 && FB_REQUIRE_WATCHERS) ? TB_IDLE_MS : TB_WATCHED_MS;
      if (now - lastTb >= every) {
        lastTb = now;
        talkBack();
      }
    }
  }
}
```

firmware/SmartBin/ble_setup.h

```
// ble_setup.h - Bluetooth LE Wi-Fi setup (started from WI-FI SETUP on the screen).
#pragma once
#include <Arduino.h>

enum BtPhase : uint8_t { BT_IDLE = 0, BT_WAIT, BT_CODE, BT_LOCKOUT, BT_PAIRED, BT_CONN, BT_OK, BT_FAIL, BT_OFF };

struct BtView {
  BtPhase phase;
  uint32_t code;
  uint32_t phaseMs;      // millis() when the phase began
  uint32_t offAtMs;      // millis() when Bluetooth switches itself off
  char ssid[33];
  char why[40];
  char ip[16];
  uint8_t netCount;
```

```
  char nets[6][33];
  int8_t rssi[6];
};

void bleStart();
void bleStop(bool fromRemote);
BtView bleView();
void bleRetry();
```

firmware/SmartBin/ble_setup.cpp

```
// ========================================================================
// ble_setup.cpp - give the bin a new Wi-Fi network over Bluetooth LE
// Flow: WI-FI SETUP + Collect PIN on the screen -> advertise "SmartBin-0x" ->
// the phone or laptop connects -> the bin shows a 6-digit passkey -> user
// types it -> paired -> app writes SSID and password, then 1 to CTRL -> the
// bin tries to join for up to 15 s and reports on STAT. Three wrong passkeys
// stop pairing for 60 s. Bluetooth turns itself off after 5 minutes idle.
// Characteristics (UUIDs in config.h): SSID w, PASS w, CTRL w (1 join, 2 stop),
// STAT read/notify JSON, SCAN read JSON of the 6 strongest networks.
// ========================================================================
#include "ble_setup.h"
#include "state.h"
#include "buzzer.h"
#include <WiFi.h>
#include <NimBLEDevice.h>

// view is read by the UI task and written by BLE callbacks, so vLock guards it.
static SemaphoreHandle_t vLock;
static BtView view;
static volatile bool running = false, stopReq = false, stopRemote = false, joinReq = false, retryReq = false;
static String newSsid, newPass;
static uint8_t wrongCodes = 0;
static volatile bool codeShown = false;
static NimBLEServer* server = nullptr;
static NimBLECharacteristic *chStat = nullptr, *chScan = nullptr;
static uint16_t connHandle = 0xFFFF;
static TaskHandle_t hBle = nullptr;

// Changes the setup phase and wakes the UI task to redraw the BT screen.
static void setPhase(BtPhase p) {
  xSemaphoreTake(vLock, portMAX_DELAY);
  view.phase = p;
  view.phaseMs = millis();
  xSemaphoreGive(vLock);
  if (hUi) xTaskNotify(hUi, EV_BT, eSetBits);
}

BtView bleView() {
  if (!vLock) { BtView v{}; return v; }
  xSemaphoreTake(vLock, portMAX_DELAY);
  BtView v = view;
  xSemaphoreGive(vLock);
  return v;
}

static void notifyStat(const String& json) {
  if (!chStat) return;
  chStat->setValue(json);
  chStat->notify();
}

// Connection and pairing callbacks from the NimBLE stack.
class ServerCb : public NimBLEServerCallbacks {
  void onConnect(NimBLEServer*, NimBLEConnInfo& info) override {
    connHandle = info.getConnHandle();
    codeShown = false;
    xSemaphoreTake(vLock, portMAX_DELAY);
    view.offAtMs = millis() + BT_IDLE_OFF_MS;
    xSemaphoreGive(vLock);
  }
  void onDisconnect(NimBLEServer*, NimBLEConnInfo&, int) override {
    connHandle = 0xFFFF;
    BtPhase p = bleView().phase;
    if (p == BT_CODE || p == BT_PAIRED) setPhase(BT_WAIT);
    if (p != BT_LOCKOUT && p != BT_OFF && running) NimBLEDevice::startAdvertising();
  }
  // Called when the phone asks to pair: pick a random 6-digit code, show it
  // on the screen and return it to the stack.
  uint32_t onPassKeyDisplay() override {
    uint32_t code = esp_random() % 1000000;
    xSemaphoreTake(vLock, portMAX_DELAY);
    view.code = code;
    xSemaphoreGive(vLock);
    codeShown = true;
```

```
    setPhase(BT_CODE);
    buzz(T_BT_CODE);
    return code;
  }
  void onAuthenticationComplete(NimBLEConnInfo& info) override {
    if (info.isEncrypted() && info.isAuthenticated()) {
      wrongCodes = 0;
      setPhase(BT_PAIRED);
      buzz(T_BT_PAIRED);
      return;
    }
    server->disconnect(info.getConnHandle());
    // Only a code that was shown and then typed wrong counts. A phone or laptop that
    // still remembers an old pairing fails here without any code, and that must not
    // use up the 3 tries.
    bool counted = codeShown;
    codeShown = false;
    if (!counted) {
      setPhase(BT_WAIT);
      return;
    }
    if (++wrongCodes >= 3) {
      NimBLEDevice::stopAdvertising();
      setPhase(BT_LOCKOUT);
      buzz(T_BT_STOPPED);
    } else {
      setPhase(BT_WAIT);
      buzz(T_BT_WRONG);
    }
  }
};

// Writes from the app. Every write pushes the auto-off time back 5 minutes.
class CharCb : public NimBLECharacteristicCallbacks {
  void onWrite(NimBLECharacteristic* c, NimBLEConnInfo&) override {
    std::string v = c->getValue();
    NimBLEUUID u = c->getUUID();
    if (u == NimBLEUUID(BT_SSID_UUID)) newSsid = String(v.c_str());
    else if (u == NimBLEUUID(BT_PASS_UUID)) newPass = String(v.c_str());
    else if (u == NimBLEUUID(BT_CTRL_UUID) && v.size()) {
      uint8_t b = (uint8_t)v[0];
      if (b == 1 || b == '1') joinReq = true;
      else if (b == 2 || b == '2') { stopRemote = true; stopReq = true; }
    }
    xSemaphoreTake(vLock, portMAX_DELAY);
    view.offAtMs = millis() + BT_IDLE_OFF_MS;
    xSemaphoreGive(vLock);
  }
  // The SCAN value is built fresh on each read from the last Wi-Fi scan.
  void onRead(NimBLECharacteristic* c, NimBLEConnInfo&) override {
    if (c != chScan) return;
    BtView v = bleView();
    JsonDocument d;
    JsonArray a = d.to<JsonArray>();
    for (int i = 0; i < v.netCount; i++) {
      JsonObject o = a.add<JsonObject>();
      o["ssid"] = v.nets[i];
      o["rssi"] = v.rssi[i];
    }
    String s;
    serializeJson(d, s);
    c->setValue(s);
  }
};

static ServerCb serverCb;
static CharCb charCb;

// Keeps the 6 strongest unique network names from the last async scan,
// strongest first.
static void collectScan() {
  int n = WiFi.scanComplete();
  if (n < 0) return;
  struct Net { String s; int r; };
  Net best[6];
  int k = 0;
  for (int i = 0; i < n; i++) {
    String s = WiFi.SSID(i);
    if (!s.length()) continue;
    bool dup = false;
    for (int j = 0; j < k; j++) if (best[j].s == s) dup = true;
    if (dup) continue;
    int r = WiFi.RSSI(i);
    if (k < 6) best[k++] = {s, r};
    else {
      int w = 0;
```

```
      for (int j = 1; j < 6; j++) if (best[j].r < best[w].r) w = j;
      if (r > best[w].r) best[w] = {s, r};
    }
  }
  for (int i = 0; i < k; i++)
    for (int j = i + 1; j < k; j++)
      if (best[j].r > best[i].r) { Net t = best[i]; best[i] = best[j]; best[j] = t; }
  xSemaphoreTake(vLock, portMAX_DELAY);
  view.netCount = k;
  for (int i = 0; i < k; i++) {
    strlcpy(view.nets[i], best[i].s.c_str(), sizeof(view.nets[i]));
    view.rssi[i] = best[i].r;
  }
  xSemaphoreGive(vLock);
  WiFi.scanDelete();
}

// Tries the new network for 15 s. On success it is saved and the IP is sent
// to the app; on failure the bin goes back to its old network. "Wrong
// password" and "Network not found" come from the ESP32's own status codes.
static void doJoin() {
  String ssid = newSsid, pass = newPass;
  xSemaphoreTake(vLock, portMAX_DELAY);
  strlcpy(view.ssid, ssid.c_str(), sizeof(view.ssid));
  xSemaphoreGive(vLock);
  setPhase(BT_CONN);
  notifyStat("{\"state\":\"CONNECTING\"}");
  String oldSsid = net.ssid, oldPass = net.pass;
  WiFi.disconnect(false, false);
  vTaskDelay(pdMS_TO_TICKS(200));
  WiFi.begin(ssid.c_str(), pass.c_str());
  uint32_t t0 = millis();
  wl_status_t s = WL_IDLE_STATUS;
  while (millis() - t0 < 15000) {
    s = WiFi.status();
    if (s == WL_CONNECTED || s == WL_CONNECT_FAILED || s == WL_NO_SSID_AVAIL) break;
    vTaskDelay(pdMS_TO_TICKS(250));
  }
  if (s == WL_CONNECTED) {
    net.ssid = ssid;
    net.pass = pass;
    saveNet();
    String ip = WiFi.localIP().toString();
    xSemaphoreTake(vLock, portMAX_DELAY);
    strlcpy(view.ip, ip.c_str(), sizeof(view.ip));
    view.offAtMs = millis() + BT_IDLE_OFF_MS;
    xSemaphoreGive(vLock);
    setPhase(BT_OK);
    buzz(T_BT_JOINED);
    notifyStat("{\"state\":\"CONNECTED\",\"ip\":\"" + ip + "\",\"ssid\":\"" + ssid + "\"}");
    return;
  }
  const char* why = s == WL_NO_SSID_AVAIL ? "Network not found" : s == WL_CONNECT_FAILED ? "Wrong password" : "Could not
join";
  xSemaphoreTake(vLock, portMAX_DELAY);
  strlcpy(view.why, why, sizeof(view.why));
  xSemaphoreGive(vLock);
  setPhase(BT_FAIL);
  buzz(T_BT_JOIN_FAIL);
  notifyStat(String("{\"state\":\"FAILED\",\"why\":\"") + why + "\"}");
  WiFi.disconnect(false, false);
  if (oldSsid.length()) WiFi.begin(oldSsid.c_str(), oldPass.c_str());
}

// Stops advertising and releases the whole BLE stack (deinit(true) frees its
// memory). cloudTask resumes once st.btOn is false again.
static void shutdown(bool remote, bool automatic) {
  NimBLEDevice::stopAdvertising();
  NimBLEDevice::deinit(true);
  server = nullptr;
  chStat = chScan = nullptr;
  running = false;
  lockState();
  st.btOn = false;
  unlockState();
  publish();
  if (remote || automatic) {
    setPhase(BT_OFF);
    buzz(automatic ? T_BT_AUTO_OFF : T_BT_OFF);
  } else {
    setPhase(BT_IDLE);
  }
}

// Watches for stop, timeout, join and retry requests every 200 ms, refreshes
// the scan every 30 s, and deletes itself when Bluetooth turns off.
```

```
static void bleTask(void*) {
  uint32_t lastScan = 0;
  for (;;) {
    vTaskDelay(pdMS_TO_TICKS(200));
    uint32_t now = millis();
    if (stopReq) {
      bool remote = stopRemote;
      stopReq = false;
      stopRemote = false;
      shutdown(remote, false);
      break;
    }
    BtView v = bleView();
    if ((int32_t)(now - v.offAtMs) >= 0 && v.phase != BT_CONN) {
      shutdown(false, true);
      break;
    }
    if (joinReq) {
      joinReq = false;
      doJoin();
    }
    if (retryReq) {
      retryReq = false;
      setPhase(connHandle != 0xFFFF ? BT_PAIRED : BT_WAIT);
    }
    if (v.phase == BT_CODE && now - v.phaseMs >= BT_CODE_MS && connHandle != 0xFFFF) {
      server->disconnect(connHandle);
      setPhase(BT_WAIT);
    }
    if (v.phase == BT_LOCKOUT && now - v.phaseMs >= BT_LOCKOUT_MS) {
      wrongCodes = 0;
      setPhase(BT_WAIT);
      NimBLEDevice::startAdvertising();
      buzz(T_BT_WAIT);
    }
    collectScan();
    if (now - lastScan > 30000 && v.phase != BT_CONN && WiFi.scanComplete() != WIFI_SCAN_RUNNING) {
      lastScan = now;
      WiFi.scanNetworks(true);
    }
  }
  hBle = nullptr;
  vTaskDelete(nullptr);
}

// Security: bonding, MITM protection and secure connections, with the bin as
// a display-only device. Every characteristic needs an encrypted,
// authenticated link, so nothing can be written before the passkey step.
void bleStart() {
  if (!vLock) vLock = xSemaphoreCreateMutex();
  if (running) return;
  running = true;
  wrongCodes = 0;
  stopReq = false;
  stopRemote = false;
  joinReq = false;
  retryReq = false;
  xSemaphoreTake(vLock, portMAX_DELAY);
  memset(&view, 0, sizeof(view));
  view.offAtMs = millis() + BT_IDLE_OFF_MS;
  xSemaphoreGive(vLock);
  setPhase(BT_WAIT);
  lockState();
  st.btOn = true;
  unlockState();
  publish();

  NimBLEDevice::init(BIN_BLE_NAME);
  NimBLEDevice::setSecurityAuth(true, true, true);
  NimBLEDevice::setSecurityIOCap(BLE_HS_IO_DISPLAY_ONLY);
  server = NimBLEDevice::createServer();
  server->setCallbacks(&serverCb, false);
  NimBLEService* svc = server->createService(BT_SVC_UUID);
  const uint32_t W = NIMBLE_PROPERTY::WRITE | NIMBLE_PROPERTY::WRITE_ENC | NIMBLE_PROPERTY::WRITE_AUTHEN;
  const uint32_t R = NIMBLE_PROPERTY::READ | NIMBLE_PROPERTY::READ_ENC | NIMBLE_PROPERTY::READ_AUTHEN;
  svc->createCharacteristic(BT_SSID_UUID, W)->setCallbacks(&charCb);
  svc->createCharacteristic(BT_PASS_UUID, W)->setCallbacks(&charCb);
  svc->createCharacteristic(BT_CTRL_UUID, W)->setCallbacks(&charCb);
  chStat = svc->createCharacteristic(BT_STAT_UUID, R | NIMBLE_PROPERTY::NOTIFY);
  chStat->setValue("{\"state\":\"WAITING\"}");
  chScan = svc->createCharacteristic(BT_SCAN_UUID, R);
  chScan->setCallbacks(&charCb);
  server->start();
  NimBLEAdvertising* adv = NimBLEDevice::getAdvertising();
  adv->setName(BIN_BLE_NAME);
  adv->addServiceUUID(BT_SVC_UUID);
```

```
  adv->enableScanResponse(true);
  adv->start();
  WiFi.scanNetworks(true);
  buzz(T_BT_WAIT);
  xTaskCreatePinnedToCore(bleTask, "ble", 6144, nullptr, 1, &hBle, 0);
}

// Asks bleTask to shut Bluetooth down on its next pass.
void bleStop(bool fromRemote) {
  if (!running) return;
  stopRemote = fromRemote;
  stopReq = true;
}

void bleRetry() { retryReq = true; }
```

firmware/SmartBin/ui.h

```
// ui.h - 2.4-inch ILI9341 touchscreen (240 x 320, portrait) and its task.
#pragma once
#include <Arduino.h>

void uiInit();
void uiTask(void*);
void isrTouch();
void uiStartTouchCal();
void uiResetTouchCal();
```

firmware/SmartBin/ui.cpp

```
// ============================================================================
// ui.cpp - touchscreen user interface
// uiTask (core 1, priority 2) redraws about every 100 ms. Each screen is drawn
// by one draw*() function; pickScreen() decides which one is shown.
// To avoid flicker, nothing is redrawn unless it changed: every widget passes
// a short "signature" string to dirty(), and only a different signature
// triggers a redraw. If part of a screen does not update, its signature is
// probably missing the value that changed.
// Touches are matched against the hit boxes that buttons register while they
// are drawn (addHit), so a button that is not drawn cannot be pressed.
// ============================================================================
#include "ui.h"
#include "state.h"
#include "buzzer.h"
#include "ble_setup.h"
#include "sensors.h"
#include <SPI.h>
#include <WiFi.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ILI9341.h>
#include <XPT2046_Touchscreen.h>
#include <Fonts/FreeSans9pt7b.h>
#include <Fonts/FreeSansBold9pt7b.h>
#include <Fonts/FreeSansBold12pt7b.h>
#include <Fonts/FreeSansBold18pt7b.h>
#include <Fonts/FreeSansBold24pt7b.h>

// The display and the touch controller share one SPI bus (separate CS pins).
static Adafruit_ILI9341 tft(&SPI, PIN_TFT_DC, PIN_TFT_CS, -1);
static XPT2046_Touchscreen touch(PIN_TOUCH_CS);

// Touch IRQ (GPIO 36, active low). Wakes uiTask so a press is read within
// 30 ms instead of waiting for the next 100 ms frame.
void IRAM_ATTR isrTouch() {
  BaseType_t woke = pdFALSE;
  if (hUi) xTaskNotifyFromISR(hUi, EV_TOUCH, eSetBits, &woke);
  portYIELD_FROM_ISR(woke);
}

// Converts a web color 0xRRGGBB to the display's 16-bit RGB565.
static constexpr uint16_t C(uint32_t rgb) {
  return (uint16_t)(((rgb >> 8) & 0xF800) | ((rgb >> 5) & 0x07E0) | ((rgb >> 3) & 0x001F));
}

// Color palette: background, surface, border, text, the five level colors
// (green, amber, red, purple, teal), blue, cyan, the text colors used on top
// of each, light tints, and dim/rule grays. Colors match the dashboard and app.
struct Pal {
  uint16_t bg, surf, bd, tx, tx2, g, a, r, p, t, bl, cy, on_g, on_a, on_r, on_p, on_t, on_bl, on_cy, tg, ta, tr, tp, tt, dim,
rule;
};
static const Pal LIGHT = {C(0xF5F0EB), C(0xFFFFFF), C(0x1A1A1A), C(0x1A1A1A), C(0x6B635B), C(0x1E8E3E), C(0xD98A1F),
                          C(0xD93025), C(0x7A45B5), C(0x0D8A8A), C(0x1A62C4), C(0x0E7F9E), C(0xFFFFFF), C(0x1A1A1A),
                          C(0xFFFFFF), C(0xFFFFFF), C(0xFFFFFF), C(0xFFFFFF), C(0xFFFFFF), C(0xCADCC8), C(0xEFDCC2),
```

```
                          C(0xEFCAC3), C(0xDCCEE0), C(0xC7DCD8), C(0xB5AFA9), C(0xC6C2BE)};
static const Pal DARK = {C(0x2A2520), C(0x3A3530), C(0xE8E0D8), C(0xEFE7DC), C(0xA89E94), C(0x4FBF6E), C(0xE8A94A),
                         C(0xFF6B5E), C(0xB98BE8), C(0x3FC7C2), C(0x79AEF5), C(0x5FD0E8), C(0x13100D), C(0x13100D),
                         C(0x13100D), C(0x13100D), C(0x13100D), C(0x13100D), C(0x13100D), C(0x324731), C(0x544229),
                         C(0x59342E), C(0x493B4C), C(0x2F4944), C(0x6E6862), C(0x5E5852)};
static const Pal* P = &LIGHT;
static uint8_t darkTheme = 0;

// Level colors: LG green (OK), LA amber (filling), LR red (full/alarm),
// LP purple (locked), LT teal (system).
enum Lv : uint8_t { LG, LA, LR, LP, LT };
static uint16_t fillOf(Lv k) { const uint16_t v[] = {P->g, P->a, P->r, P->p, P->t}; return v[k]; }
static uint16_t onOf(Lv k) { const uint16_t v[] = {P->on_g, P->on_a, P->on_r, P->on_p, P->on_t}; return v[k]; }
static uint16_t tintOf(Lv k) { const uint16_t v[] = {P->tg, P->ta, P->tr, P->tp, P->tt}; return v[k]; }

// Fonts from smallest to largest. CAP[] is each font's cap height in pixels,
// used to center text vertically.
enum Ft : uint8_t { F_TINY = 0, F_SR, F_S, F_M, F_L, F_XL };
static const GFXfont* const FONTS[] = {nullptr, &FreeSans9pt7b, &FreeSansBold9pt7b, &FreeSansBold12pt7b,
                                       &FreeSansBold18pt7b, &FreeSansBold24pt7b};
static const uint8_t CAP[] = {7, 13, 13, 17, 25, 34};

static Ft fontFor(int px, bool regular = false) {
  if (px <= 12) return F_TINY;
  if (px <= 17) return regular ? F_SR : F_S;
  if (px <= 24) return F_M;
  if (px <= 36) return F_L;
  return F_XL;
}

static void useFont(Ft f, uint8_t mag) {
  tft.setFont(FONTS[f]);
  tft.setTextSize(mag);
  tft.setTextWrap(false);
}

// Width of a string in pixels for a given font.
static int textW(const char* s, Ft f, uint8_t mag = 1) {
  useFont(f, mag);
  int16_t x1, y1;
  uint16_t w, h;
  tft.getTextBounds(s, 0, 50, &x1, &y1, &w, &h);
  return f == F_TINY ? (int)strlen(s) * 6 * mag : (int)w + x1;
}

// Draws text in a box w pixels wide; px is the wanted text height. If the
// text does not fit it steps down to a smaller font. align: 'l', 'c', 'r'.
static void txt(int x, int y, int w, const char* s, int px, uint16_t col, char align = 'l', bool regular = false, uint8_t mag
= 1) {
  Ft f = fontFor(px, regular);
  int tw = textW(s, f, mag);
  while (tw > w && f > F_TINY) {
    f = (Ft)(f - 1);
    if (f == F_SR && !regular) f = F_TINY;
    tw = textW(s, f, mag);
  }
  int cx = align == 'c' ? x + (w - tw) / 2 : align == 'r' ? x + w - tw : x;
  useFont(f, mag);
  tft.setTextColor(col);
  if (f == F_TINY) tft.setCursor(cx, y + (px - 7 * mag) / 2);
  else tft.setCursor(cx, y + (px + CAP[f] * mag) / 2);
  tft.print(s);
}

// Like txt() but cuts long text and adds "..", for Wi-Fi names.
static void txtFit(int x, int y, int w, const char* s, int px, uint16_t col, char align = 'l', bool regular = false) {
  char b[40];
  strlcpy(b, s, sizeof(b));
  Ft f = fontFor(px, regular);
  size_t n = strlen(b);
  if (textW(b, f) > w) {
    while (n > 2) {
      b[--n] = 0;
      char t[44];
      snprintf(t, sizeof(t), "%s..", b);
      if (textW(t, f) <= w) { strlcpy(b, t, sizeof(b)); break; }
    }
  }
  txt(x, y, w, b, px, col, align, regular);
}

// Word-wraps centered text over several lines lh pixels apart.
static void wrapTxt(int x, int y, int w, const char* s, int px, uint16_t col, int lh) {
  char line[64] = "", word[40];
  const char* p = s;
  int ly = y;
```

```
  while (*p) {
    int n = 0;
    while (*p == ' ') p++;
    while (*p && *p != ' ' && n < 39) word[n++] = *p++;
    word[n] = 0;
    if (!n) break;
    char test[104];
    snprintf(test, sizeof(test), "%s%s%s", line, line[0] ? " " : "", word);
    if (textW(test, fontFor(px, true)) > w && line[0]) {
      txt(x, ly, w, line, px, col, 'c', true);
      ly += lh;
      strlcpy(line, word, sizeof(line));
    } else {
      strlcpy(line, test, sizeof(line));
    }
  }
  if (line[0]) txt(x, ly, w, line, px, col, 'c', true);
}

// --- Drawing helpers in the app's neo-brutalist style: thick dark border
// and a hard offset shadow. ---
static void frame(int x, int y, int w, int h, uint16_t fill, int border, uint16_t bd) {
  tft.fillRect(x, y, w, h, bd);
  tft.fillRect(x + border, y + border, w - 2 * border, h - 2 * border, fill);
}

static void box(int x, int y, int w, int h, uint16_t fill, int border = 3, int shadow = 0) {
  if (shadow) tft.fillRect(x + shadow, y + shadow, w, h, P->bd);
  frame(x, y, w, h, fill, border, P->bd);
}

static void thick(int x0, int y0, int x1, int y1, int t, uint16_t c) {
  for (int dx = -(t / 2); dx <= t / 2; dx++)
    for (int dy = -(t / 2); dy <= t / 2; dy++) tft.drawLine(x0 + dx, y0 + dy, x1 + dx, y1 + dy, c);
}

static void arc(int cx, int cy, int r, int a0, int a1, int t, uint16_t c) {
  for (int a = a0; a <= a1; a += 4) {
    float rad = a * DEG_TO_RAD;
    tft.fillCircle(cx + cosf(rad) * r, cy + sinf(rad) * r, t / 2, c);
  }
}

// Line icons drawn with thick lines on a 24 x 24 grid scaled to size s.
enum Icon : uint8_t { I_BIN, I_LIDOPEN, I_SPEAKER, I_LOCK, I_TRI, I_CHECK, I_X, I_BT, I_TRUCK, I_SUN, I_MOON, I_RING,
I_WIFI };

static void icon(Icon k, int x, int y, int s, uint16_t c, uint16_t bgc, int phase = 0) {
  float u = s / 24.0f;
  int t = max(2, (int)(s / 9));
  auto X = [&](float v) { return x + (int)(v * u); };
  auto Y = [&](float v) { return y + (int)(v * u); };
  switch (k) {
    case I_BIN:
      thick(X(5), Y(8), X(19), Y(8), t, c);
      thick(X(3), Y(8), X(21), Y(8), t, c);
      thick(X(5), Y(8), X(6.4), Y(20), t, c);
      thick(X(19), Y(8), X(17.6), Y(20), t, c);
      thick(X(6.4), Y(20), X(17.6), Y(20), t, c);
      thick(X(9), Y(4.5), X(15), Y(4.5), t, c);
      break;
    case I_LIDOPEN:
      thick(X(6), Y(11), X(18), Y(11), t, c);
      thick(X(6), Y(11), X(7.2), Y(20), t, c);
      thick(X(18), Y(11), X(16.8), Y(20), t, c);
      thick(X(7.2), Y(20), X(16.8), Y(20), t, c);
      thick(X(3.5), Y(8.5), X(16.5), Y(4.3), t, c);
      break;
    case I_SPEAKER:
      tft.fillTriangle(X(3), Y(12), X(12), Y(5.5), X(12), Y(18.5), c);
      tft.fillRect(X(3), Y(9.5), (int)(5 * u), (int)(5 * u) + 1, c);
      if (phase != 1) arc(X(13), Y(12), (int)(4 * u), -45, 45, t, c);
      if (phase != 2) arc(X(13), Y(12), (int)(8 * u), -50, 50, t, c);
      break;
    case I_LOCK:
      frame(X(4.5), Y(10.5), (int)(15 * u), (int)(10.5 * u), bgc, t, c);
      arc(X(12), Y(8.5), (int)(4 * u), 180, 360, t, c);
      thick(X(8), Y(8.5), X(8), Y(10.5), t, c);
      thick(X(16), Y(8.5), X(16), Y(10.5), t, c);
      thick(X(12), Y(14.5), X(12), Y(17.5), t, c);
      break;
    case I_TRI:
      thick(X(12), Y(3.2), X(22), Y(20.5), t, c);
      thick(X(22), Y(20.5), X(2), Y(20.5), t, c);
      thick(X(2), Y(20.5), X(12), Y(3.2), t, c);
      thick(X(12), Y(9.5), X(12), Y(14.5), t + 1, c);
```

```
      tft.fillCircle(X(12), Y(17.3), t, c);
      break;
    case I_CHECK:
      thick(X(4), Y(12.5), X(9), Y(17.5), t + 1, c);
      thick(X(9), Y(17.5), X(20), Y(6.5), t + 1, c);
      break;
    case I_X:
      thick(X(6), Y(6), X(18), Y(18), t + 1, c);
      thick(X(18), Y(6), X(6), Y(18), t + 1, c);
      break;
    case I_BT:
      thick(X(7), Y(7.5), X(17), Y(16.5), t, c);
      thick(X(17), Y(16.5), X(12), Y(21), t, c);
      thick(X(12), Y(21), X(12), Y(3), t, c);
      thick(X(12), Y(3), X(17), Y(7.5), t, c);
      thick(X(17), Y(7.5), X(7), Y(16.5), t, c);
      break;
    case I_TRUCK:
      frame(X(2), Y(7), (int)(11 * u), (int)(9 * u), bgc, max(2, t - 1), c);
      thick(X(13), Y(10), X(18), Y(10), t - 1, c);
      thick(X(18), Y(10), X(21.5), Y(13), t - 1, c);
      thick(X(21.5), Y(13), X(21.5), Y(16), t - 1, c);
      thick(X(13), Y(16), X(21.5), Y(16), t - 1, c);
      tft.fillCircle(X(6.5), Y(18.5), (int)(2.2 * u), c);
      tft.fillCircle(X(17), Y(18.5), (int)(2.2 * u), c);
      break;
    case I_SUN:
      tft.fillCircle(X(12), Y(12), (int)(4.5 * u), c);
      for (int a = 0; a < 360; a += 45) {
        float r = a * DEG_TO_RAD;
        thick(X(12 + cosf(r) * 7), Y(12 + sinf(r) * 7), X(12 + cosf(r) * 9.5f), Y(12 + sinf(r) * 9.5f), max(1, t - 1), c);
      }
      break;
    case I_MOON:
      tft.fillCircle(X(11), Y(12.5), (int)(8.5 * u), c);
      tft.fillCircle(X(15.5), Y(8.5), (int)(7 * u), bgc);
      break;
    case I_RING:
      for (int a = 0; a < 360; a += 40) arc(X(12), Y(12), (int)(9 * u), a + phase, a + phase + 22, t + 1, c);
      break;
    case I_WIFI:
      arc(X(12), Y(20), (int)(5 * u), 220, 320, t, c);
      arc(X(12), Y(20), (int)(10 * u), 220, 320, t, c);
      tft.fillCircle(X(12), Y(20), t, c);
      break;
  }
}

// Button IDs used by the touch handler (onPress). B_K0..B_K9 are PIN keys.
enum BtnId : uint8_t {
  B_NONE = 0, B_COLLECT, B_INFO, B_THEME, B_DONE, B_DONE_BACK, B_CAL_CANCEL, B_BACK, B_WIFI_SETUP, B_BT_CANCEL,
  B_BT_DONE, B_BT_RETRY, B_PIN_CANCEL, B_K0, B_K1, B_K2, B_K3, B_K4, B_K5, B_K6, B_K7, B_K8, B_K9, B_KDEL, B_KOK
};
struct Hit { int16_t x, y, w, h; uint8_t id; };
static Hit hits[24];
static uint8_t hitN = 0;
static void addHit(int x, int y, int w, int h, uint8_t id) {
  for (uint8_t i = 0; i < hitN; i++)
    if (hits[i].id == id && hits[i].x == x && hits[i].y == y) return;
  if (hitN < 24) hits[hitN++] = {(int16_t)x, (int16_t)y, (int16_t)w, (int16_t)h, id};
}

// A button: shadowed box, optional icon, label. If the label and icon do not
// fit, the icon shrinks and is dropped last.
static void btn(int x, int y, int w, int h, const char* label, uint16_t fill, uint16_t on, int px, int ic, int isz, uint8_t
id,
                bool pressed = false) {
  int o = pressed ? 4 : 0;
  if (pressed) tft.fillRect(x, y, w + 4, h + 4, P->bg == fill ? P->surf : P->bg);
  box(x + o, y + o, w, h, fill, 3, pressed ? 0 : 4);
  int lw = textW(label, fontFor(px));
  int gap = 5;
  while (ic >= 0 && lw + isz + gap > w - 12 && isz > 14) {
    isz -= 2;
    gap = 3;
  }
  if (ic >= 0 && lw + isz + gap > w - 12) ic = -1;
  int tw = lw + (ic >= 0 ? isz + gap : 0);
  int sx = x + o + (w - tw) / 2;
  if (ic >= 0) {
    icon((Icon)ic, sx, y + o + (h - isz) / 2, isz, on, fill);
    sx += isz + gap;
  }
  txt(sx, y + o + (h - px) / 2, x + o + w - 6 - sx, label, px, on);
  if (id) addHit(x, y, w + 4, h + 4, id);
}
```

```
309
310 static void chip(int x, int y, int w, int h, const char* label, uint16_t fill, uint16_t on, int px = 12, bool sh = false, int
    border = 2) {
311   if (sh) tft.fillRect(x + 3, y + 3, w, h, P->bd);
312   frame(x, y, w, h, fill, border, P->bd);
313   txt(x + 2, y + (h - px) / 2, w - 4, label, px, on, 'c');
314 }
315
316 // FNV-1a hash of a widget signature.
317 static uint32_t hashOf(const char* s) {
318   uint32_t h = 2166136261u;
319   while (*s) { h ^= (uint8_t)*s++; h *= 16777619u; }
320   return h;
321 }
322
323 // wcache holds the last signature hash of each widget slot (id 0..39).
324 // fullDraw forces every widget to redraw, used after a screen is cleared.
325 static bool fullDraw = true;
326 static uint32_t wcache[40];
327 static bool dirty(uint8_t id, const char* sig) {
328   uint32_t h = hashOf(sig);
329   if (fullDraw || wcache[id] != h) { wcache[id] = h; return true; }
330   return false;
331 }
332
333 // Screens and overlays. Overlays (Info, PIN, Bluetooth) sit above the normal
334 // screens and close by themselves after SCREEN_TIMEOUT_MS without a touch.
335 enum Screen : uint8_t { S_BOOT, S_CAL, S_MAIN, S_URGENT, S_PIN, S_COLLECT, S_DONE, S_TAMPER, S_LOCKED, S_INFO, S_BT,
    S_TOUCHCAL };
336 enum Overlay : uint8_t { O_NONE, O_INFO, O_PIN, O_BT };
337 static Screen cur = S_BOOT;
338 static Overlay overlay = O_NONE;
339 static uint32_t overlayMs = 0;
340 static bool pinForBt = false;
341 static char pinBuf[5] = "";
342 static uint32_t pinWrongUntil = 0;
343 static uint32_t doneShownAt = 0;
344 static uint32_t lastTouchMs = 0;
345 static uint8_t blink = 0, spin = 0;
346 static uint32_t lastAnim = 0;
347
348 static const char* lidText(bool open) { return open ? "LID: OPEN" : "LID: CLOSED"; }
349
350 // Top bar: bin name, Info and theme buttons, optional BACK button and the lid chip.
351 static void topBar(const BinState& s, bool icons, int lidMode, const char* extraBack = nullptr) {
352   char sig[48];
353   snprintf(sig, sizeof(sig), "%d%d%d", icons, lidMode, darkTheme);
354   if (dirty(0, sig)) {
355     tft.fillRect(0, 0, 240, 34, P->surf);
356     tft.fillRect(0, 34, 240, 3, P->bd);
357     int nw = icons ? 60 : lidMode >= 0 ? 130 : 150;
358     txt(8, 9, nw, BIN_LABEL, 16, P->tx);
359     if (icons) {
360       tft.fillRect(76, 6, 26, 26, P->bd);
361       frame(74, 4, 26, 26, P->bg, 2, P->bd);
362       txt(74, 10, 26, "i", 14, P->tx, 'c');
363       addHit(0, 0, 72, 34, B_INFO);
364       addHit(72, 0, 30, 34, B_INFO);
365       tft.fillRect(106, 6, 26, 26, P->bd);
366       frame(104, 4, 26, 26, P->bg, 2, P->bd);
367       icon(darkTheme ? I_MOON : I_SUN, 107, 7, 20, P->tx, P->bg);
368       addHit(102, 0, 34, 34, B_THEME);
369     }
370     if (extraBack) {
371       frame(168, 4, 64, 26, P->bg, 2, P->bd);
372       txt(168, 11, 64, extraBack, 12, P->tx, 'c');
373       addHit(160, 0, 80, 34, B_BACK);
374     }
375   }
376   if (lidMode >= 0) {
377     snprintf(sig, sizeof(sig), "lid%d%d", lidMode, darkTheme);
378     if (dirty(1, sig)) {
379       bool open = lidMode == 1;
380       chip(142, 6, 90, 22, lidText(open), open ? P->a : P->bg, open ? P->on_a : P->tx);
381     }
382   }
383 }
384
385 // Status chips under the top bar: WI-FI/OFFLINE, MOTION, SIM, CAL.
386 static void drawChips(const BinState& s, const Sim& sm, uint16_t tint) {
387   char sig[24];
388   snprintf(sig, sizeof(sig), "%d%d%d%d%d", s.wifi, s.v.motion, sm.mask != 0, s.calibrating, darkTheme);
389   if (!dirty(2, sig)) return;
390   tft.fillRect(0, 40, 240, 24, tint);
391   int wx;
392   if (s.wifi) { chip(8, 42, 52, 20, "WI-FI", P->g, P->on_g); wx = 66; }
```

```
  else { chip(8, 42, 70, 20, "OFFLINE", P->r, P->on_r); wx = 84; }
  addHit(4, 38, wx - 4, 28, B_INFO);
  if (s.v.motion) chip(wx, 42, 62, 20, "MOTION", P->cy, P->on_cy);
  else chip(wx, 42, 62, 20, "MOTION", P->surf, P->dim);
  int sx = wx + 68;
  if (sm.mask) chip(sx, 42, 40, 20, "SIM", P->p, P->on_p);
  else chip(sx, 42, 40, 20, "SIM", P->surf, P->dim);
  if (s.calibrating) chip(sx + 46, 42, 40, 20, "CAL", P->t, P->on_t);
  else chip(sx + 46, 42, 40, 20, "CAL", P->surf, P->dim);
}

// Vertical fill bar with marks at 50% and at the fill threshold.
static void fillBar(int x, int y, int w, int h, float pct, uint16_t col, float th) {
  box(x, y, w, h, P->surf, 3, 0);
  int inner = h - 6;
  int fh = (int)roundf(inner * constrain(pct, 0.0f, 100.0f) / 100.0f);
  if (fh > 0) {
    tft.fillRect(x + 3, y + 3 + inner - fh, w - 6, fh, col);
    tft.fillRect(x + 3, y + 3 + inner - fh, w - 6, min(2, fh), P->bd);
  }
  for (float t : {50.0f, th}) {
    int ty = y + 3 + (int)roundf(inner * (1 - t / 100.0f));
    tft.fillRect(x, ty - 1, w, 2, P->bd);
  }
}

// One sensor row: label and value. Turns red when over its limit and shows
// "--" with a "!" when the sensor is faulty.
static void reading(uint8_t id, int y, const char* label, const char* value, bool alarm, bool fault, int dot, uint16_t rowBg)
{
  char sig[64];
  snprintf(sig, sizeof(sig), "%s|%d%d%d%d", value, alarm, fault, dot, darkTheme);
  if (!dirty(id, sig)) return;
  tft.fillRect(52, y - 2, 188, 22, rowBg);
  if (fault) {
    txt(56, y + 2, 90, label, 13, P->tx2, 'l', true);
    frame(178, y + 1, 12, 15, rowBg, 2, P->tx2);
    txt(178, y + 2, 12, "!", 12, P->tx2, 'c');
    txt(196, y + 1, 36, "--", 15, P->tx2, 'r');
    return;
  }
  if (alarm) {
    tft.fillRect(57, y + 2, 180, 18, P->bd);
    frame(54, y - 1, 180, 18, P->r, 2, P->bd);
    txt(60, y + 2, 90, label, 13, P->on_r, 'l', true);
    txt(120, y + 1, 106, value, 15, P->on_r, 'r');
    return;
  }
  txt(56, y + 2, 90, label, 13, P->tx2, 'l', true);
  int vw = textW(value, fontFor(15));
  if (dot >= 0) tft.fillRect(232 - vw - 15, y + 5, 9, 9, fillOf((Lv)dot));
  txt(110, y + 1, 122, value, 15, P->tx, 'r');
}

// A row with a shrinking progress line (lid closing, buzzer time left).
static void timerRow(uint8_t id, int y, int ic, const char* label, float frac, uint16_t col, uint16_t rowBg) {
  char sig[48];
  snprintf(sig, sizeof(sig), "%s|%d|%d", label, (int)(frac * 50), darkTheme);
  if (!dirty(id, sig)) return;
  frame(8, y, 224, 20, P->surf, 2, P->bd);
  icon((Icon)ic, 11, y + 2, 14, P->tx, P->surf);
  txt(29, y + 3, 200, label, 12, P->tx);
  tft.fillRect(10, y + 15, 220, 3, P->bg);
  tft.fillRect(10, y + 15, (int)(220 * constrain(frac, 0.0f, 1.0f)), 3, col);
}

static void clearRow(uint8_t id, int y, int h, uint16_t bgc) {
  if (dirty(id, "none")) tft.fillRect(0, y, 240, h, bgc);
}

// Bottom row: bin status box and the COLLECT button.
static void bottomRow(const char* status, uint16_t fill, uint16_t on, int px) {
  char sig[40];
  snprintf(sig, sizeof(sig), "%s%d", status, darkTheme);
  if (!dirty(3, sig)) return;
  tft.fillRect(8 + 4, 274 + 4, 112, 36, P->bd);
  frame(8, 274, 112, 36, fill, 3, P->bd);
  txt(10, 274 + 11, 108, status, px, on, 'c');
  btn(128, 270, 104, 44, "COLLECT", P->surf, P->tx, 15, I_TRUCK, 20, B_COLLECT);
}

// "Full in ~x h" text from the fill rate. Hidden above 72 h or when unknown.
static void fmtTtf(const BinState& s, const Cal& c, char* out, size_t n) {
  out[0] = 0;
  if (s.fillRatePerHour <= 0 || s.v.fill >= c.fill_th) return;
  float h = (c.fill_th - s.v.fill) / s.fillRatePerHour;
```

```
  if (h > 72) return;
  if (h < 1) snprintf(out, n, "Full in ~%d min", max(1, (int)roundf(h * 60)));
  else snprintf(out, n, "Full in ~%.1f h", h);
}

static void lidRows(const BinState& s, const Cal& c, uint32_t now, int y, uint16_t rowBg, uint8_t id) {
  char label[40];
  if (s.lidOpen && s.v.motion && s.status == ST_OK) {
    timerRow(id, y, I_LIDOPEN, "LID HOLDING (motion)", 1, P->cy, rowBg);
  } else if (s.lidOpen && s.lidCloseAtMs) {
    float left = max<int32_t>(0, (int32_t)(s.lidCloseAtMs - now)) / 1000.0f;
    snprintf(label, sizeof(label), "LID CLOSES IN %.1f s", left);
    timerRow(id, y, I_LIDOPEN, label, left / max(1, (int)c.pir_hold_s), P->cy, rowBg);
  }
}

static bool lidRowActive(const BinState& s) { return s.lidOpen && (s.v.motion || s.lidCloseAtMs) && s.status == ST_OK; }
static bool buzzRowActive(const BinState& s, uint32_t now) { return s.buzzerOn || (int32_t)(s.buzzerUntilMs - now) > 0; }

static void buzzRow(const BinState& s, const Cal& c, uint32_t now, int y, uint16_t rowBg, uint8_t id) {
  char label[32];
  int32_t left = (int32_t)(s.buzzerUntilMs - now);
  if (left > 0) {
    snprintf(label, sizeof(label), "BUZZER %.1f s", left / 1000.0f);
    timerRow(id, y, I_SPEAKER, label, left / (max(1, (int)c.buzz_s) * 1000.0f), P->r, rowBg);
  } else {
    timerRow(id, y, I_SPEAKER, "BUZZER ON", 1, P->r, rowBg);
  }
}

// --- Main screen: fill level, weight, temperature, humidity, air ---
// The background tint follows the level: green under 50%, amber from 50%,
// red at the fill threshold.
static void drawMain(const BinState& s, const Cal& c, const Sim& sm, uint32_t now) {
  bool hcFault = !(s.okHC || (sm.mask & 1));
  int fill = (int)roundf(s.v.fill);
  Lv lvl = fill >= c.fill_th ? LR : fill >= 50 ? LA : LG;
  uint16_t tint = tintOf(lvl);
  char sig[64];
  snprintf(sig, sizeof(sig), "main%d%d", lvl, darkTheme);
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(tint);
  }
  topBar(s, true, s.lidOpen ? 1 : 0);
  drawChips(s, sm, tint);

  snprintf(sig, sizeof(sig), "%d|%d|%d|%.0f", fill, hcFault, darkTheme, c.fill_th);
  if (dirty(5, sig)) {
    tft.fillRect(8, 66, 224, 78, tint);
    fillBar(10, 68, 36, 150, hcFault ? 0 : fill, fillOf(lvl), c.fill_th);
    txt(56, 68, 100, "FILL LEVEL", 12, P->tx2);
    if (hcFault) {
      chip(150, 64, 82, 20, "NO READING", P->surf, P->tx2, 11);
      txt(56, 82, 176, "--", 56, P->tx);
    } else {
      chip(172, 64, 60, 20, fill >= c.fill_th ? "FULL" : fill >= 50 ? "FILLING" : "OK", fillOf(lvl), onOf(lvl));
      char n[8];
      snprintf(n, sizeof(n), "%d", fill);
      txt(56, 82, 176, n, 56, P->tx);
      int nw = textW(n, F_XL);
      txt(56 + nw + 2, 98, 40, "%", 26, P->tx);
    }
  }

  char val[24];
  bool hx = s.okHX || (sm.mask & 2), bme = s.okBME || (sm.mask & 8), bmeGas = s.okBME || (sm.mask & 4), gasOk = bmeGas &&
s.v.iaq_acc > 0;
  snprintf(val, sizeof(val), "%.2f kg", s.v.weight_g / 1000.0f);
  reading(6, 146, "Weight", val, hx && s.v.weight_g >= c.weight_cap, !hx, -1, tint);
  snprintf(val, sizeof(val), "%.1f C", s.v.temp);
  reading(7, 164, "Temperature", val, bme && s.v.temp >= c.temp_th, !bme, -1, tint);
  snprintf(val, sizeof(val), "%d%%", (int)roundf(s.v.humidity));
  reading(8, 182, "Humidity", val, bme && s.v.humidity >= c.hum_th, !bme, -1, tint);
  float gp = s.v.iaq / max(1.0f, c.gas_th) * 100;
  Lv al = gp >= 80 ? LR : gp >= 50 ? LA : LG;
  snprintf(val, sizeof(val), "%s %d", gp >= 80 ? "Poor" : gp >= 50 ? "Moderate" : "Good", (int)roundf(s.v.iaq));
  // During BME680 warm-up the Air row shows the countdown and the raw gas
  // resistance in kOhm instead of an index.
  if (bmeGas && s.v.iaq_acc == 0) {
    int left = airWarmLeft();
    if (left > 0) snprintf(val, sizeof(val), "%d:%02d  %.0fk", left / 60, left % 60, s.v.gas_k);
    else if (s.v.gas_k > 0) snprintf(val, sizeof(val), "Starting %.0fk", s.v.gas_k);
    else strcpy(val, "No gas reading");
    reading(9, 200, "Air (warm)", val, false, false, -1, tint);
  }
```

```
  else reading(9, 200, "Air", val, gasOk && gp >= 100, !gasOk, al, tint);

  // Up to two timer rows (lid, buzzer); otherwise the "Full in" line.
  bool lr = lidRowActive(s), br = buzzRowActive(s, now);
  if (lr && br) {
    lidRows(s, c, now, 222, tint, 10);
    buzzRow(s, c, now, 244, tint, 11);
  } else if (lr || br) {
    if (lr) lidRows(s, c, now, 224, tint, 10);
    else buzzRow(s, c, now, 224, tint, 10);
    clearRow(11, 244, 22, tint);
  } else {
    char t[32];
    fmtTtf(s, c, t, sizeof(t));
    snprintf(sig, sizeof(sig), "ttf%s%d", hcFault ? "" : t, darkTheme);
    if (dirty(10, sig)) {
      tft.fillRect(0, 220, 240, 26, tint);
      if (!hcFault && t[0]) txt(8, 230, 224, t, 13, P->tx);
    }
    clearRow(11, 246, 22, tint);
  }
  if (hcFault) bottomRow("BIN: ?", P->surf, P->tx2, 14);
  else bottomRow(fill >= c.fill_th ? "BIN: FULL" : "BIN: OK", fillOf(lvl), onOf(lvl), 14);
}

// --- Urgent screen: shown when fill passes the threshold ---
// Blinking border and speaker icon every 500 ms.
static void drawUrgent(const BinState& s, const Cal& c, const Sim& sm, uint32_t now) {
  char sig[48];
  snprintf(sig, sizeof(sig), "urg%d", darkTheme);
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(P->r);
  }
  topBar(s, true, s.lidOpen ? 1 : 0);
  if (fullDraw) {
    txt(12, 46, 170, "URGENT", 40, P->on_r);
    txt(12, 98, 200, "FILL LEVEL", 12, P->on_r);
    txt(12, 196, 216, "Bin is almost full.", 16, P->on_r, 'l', true);
    txt(12, 216, 216, "Collect it now!", 20, P->on_r);
  }
  snprintf(sig, sizeof(sig), "%d%d", (int)roundf(s.v.fill), sm.mask != 0);
  if (dirty(5, sig)) {
    tft.fillRect(10, 112, 176, 82, P->r);
    char n[8];
    snprintf(n, sizeof(n), "%d", (int)roundf(s.v.fill));
    txt(12, 112, 160, n, 80, P->on_r, 'l', false, 2);
    int nw = textW(n, F_XL, 2);
    txt(14 + nw, 146, 50, "%", 36, P->on_r);
    tft.fillRect(186, 94, 50, 26, P->r);
    if (sm.mask) chip(188, 96, 44, 20, "SIM", P->p, P->on_p);
  }
  snprintf(sig, sizeof(sig), "b%d", blink);
  if (dirty(12, sig)) {
    uint16_t bc = blink ? P->bd : P->r;
    for (int i = 0; i < 5; i++) tft.drawRect(i, 37 + i, 240 - 2 * i, 283 - 2 * i, bc);
    tft.fillRect(190, 46, 38, 38, P->r);
    if (blink) icon(I_SPEAKER, 190, 46, 38, P->on_r, P->r);
  }
  bool lr = lidRowActive(s), br = buzzRowActive(s, now);
  if (br) buzzRow(s, c, now, 244, P->r, 10);
  else if (lr) lidRows(s, c, now, 244, P->r, 10);
  else clearRow(10, 242, 24, P->r);
  bottomRow("BIN: FULL", P->surf, P->tx, 14);
}

// --- Collect PIN keypad. Also used to unlock WI-FI SETUP. ---
// A wrong PIN shows the red banner for 1.6 s.
static void drawPin(uint32_t now) {
  bool wrong = (int32_t)(pinWrongUntil - now) > 0;
  char sig[24];
  snprintf(sig, sizeof(sig), "pin%d%d", wrong, darkTheme);
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(P->bg);
    tft.fillRect(0, 0, 240, 34, P->surf);
    tft.fillRect(0, 34, 240, 3, P->bd);
    frame(6, 4, 84, 26, P->r, 2, P->bd);
    icon(I_X, 13, 10, 13, P->on_r, P->r);
    txt(30, 11, 56, "CANCEL", 12, P->on_r);
    addHit(0, 0, 96, 36, B_PIN_CANCEL);
    txt(98, 9, 134, "COLLECT PIN", 16, P->tx, 'r');
    const char* keys[] = {"1", "2", "3", "4", "5", "6", "7", "8", "9", "<", "0", "OK"};
    const uint8_t ids[] = {B_K1, B_K2, B_K3, B_K4, B_K5, B_K6, B_K7, B_K8, B_K9, B_KDEL, B_K0, B_KOK};
    for (int i = 0; i < 12; i++) {
      int r = i / 3, cc = i % 3;
```

```
      bool ok = i == 11;
      btn(8 + cc * 76, 98 + r * 46, 68, 42, keys[i], ok ? P->g : P->surf, ok ? P->on_g : P->tx, 22, -1, 0, ids[i]);
    }
    if (wrong) {
      frame(8, 288, 224, 26, P->r, 3, P->bd);
      txt(8, 294, 224, "WRONG PIN. Try again.", 14, P->on_r, 'c');
      for (int i = 0; i < 5; i++) tft.drawRect(i, 37 + i, 240 - 2 * i, 283 - 2 * i, P->r);
    } else {
      txt(8, 296, 224, "4 digits, then OK", 13, P->tx2, 'c', true);
    }
  }
  int n = wrong ? 4 : strlen(pinBuf);
  snprintf(sig, sizeof(sig), "%d", n);
  if (dirty(5, sig)) {
    for (int i = 0; i < 4; i++) {
      box(20 + i * 52, 46, 44, 40, P->surf, 3, 3);
      if (i < n) tft.fillCircle(20 + i * 52 + 22, 66, 6, P->tx);
    }
  }
}

// --- Collecting screen: lid held open until DONE ---
static void drawCollect(uint32_t now) {
  char sig[24];
  snprintf(sig, sizeof(sig), "col%d", darkTheme);
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(P->g);
    BinState s = snapshot();
    topBar(s, false, 1);
    icon(I_LIDOPEN, 88, 84, 64, P->on_g, P->g);
    const char* rows[] = {"Lid: open", "Motion: ignored", "Tamper: paused", "Alerts: muted"};
    for (int i = 0; i < 4; i++) {
      tft.fillRect(28, 172 + i * 22, 9, 9, P->on_g);
      txt(44, 164 + i * 22, 180, rows[i], 16, P->on_g, 'l', true);
    }
    btn(8, 262, 224, 48, "DONE", P->surf, P->tx, 22, I_CHECK, 24, B_DONE);
  }
  char dots[24];
  int d = (now / 300) % 4;
  snprintf(dots, sizeof(dots), "COLLECTING%.*s", d, "...");
  if (dirty(5, dots)) {
    tft.fillRect(0, 44, 240, 34, P->g);
    txt(8, 48, 224, dots, 26, P->on_g, 'c');
  }
}

// --- Collection complete: goes back to the main screen after 30 s ---
static void drawDone(uint32_t now) {
  char sig[24];
  snprintf(sig, sizeof(sig), "done%d", darkTheme);
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(P->tg);
    BinState s = snapshot();
    topBar(s, false, 0);
    txt(8, 50, 224, "COLLECTION", 22, P->tx, 'c');
    txt(8, 76, 224, "COMPLETE", 22, P->tx, 'c');
    box(84, 108, 72, 72, P->g, 3, 4);
    icon(I_CHECK, 94, 118, 52, P->on_g, P->g);
    btn(8, 198, 224, 56, "DONE", P->g, P->on_g, 26, -1, 0, B_DONE_BACK);
  }
  int left = 30 - (int)((now - doneShownAt) / 1000);
  if (left < 0) left = 0;
  snprintf(sig, sizeof(sig), "%d", left);
  if (dirty(5, sig)) {
    char t[48];
    snprintf(t, sizeof(t), "Back to the main screen in %d s", left);
    tft.fillRect(0, 264, 240, 20, P->tg);
    txt(8, 266, 224, t, 13, P->tx, 'c', true);
    frame(8, 286, 224, 16, P->surf, 2, P->bd);
    tft.fillRect(10, 288, 220 * left / 30, 12, P->g);
  }
}

// --- Tamper screen: countdown, then latched until reset from the dashboard ---
static void drawTamper(const BinState& s, const Cal& c, uint32_t now) {
  bool latched = s.tamperPhase == 2;
  char sig[24];
  snprintf(sig, sizeof(sig), "tmp%d%d", latched, darkTheme);
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(P->r);
    txt(8, 82, 224, latched ? "TAMPER LATCHED" : "TAMPER DETECTED", 22, P->on_r, 'c');
    txt(8, 112, 224, "Bin moved or tilted", 15, P->on_r, 'c', true);
    if (latched) {
```

```
      box(16, 160, 208, 84, P->surf, 3, 4);
      txt(16, 176, 208, "RESET ON", 22, P->tx, 'c');
      txt(16, 204, 208, "DASHBOARD", 22, P->tx, 'c');
      txt(8, 252, 224, "Stays on until it is cleared", 13, P->on_r, 'c', true);
    } else {
      txt(8, 236, 224, "SECONDS, CLEARS IF STABLE", 12, P->on_r, 'c');
    }
    tft.fillRect(53, 279, 140, 32, P->bd);
    frame(50, 276, 140, 32, P->surf, 3, P->bd);
    icon(I_SPEAKER, 70, 281, 22, P->tx, P->surf);
    txt(98, 284, 90, latched ? "ALARM ON" : "ALARM", 15, P->tx);
  }
  char t[24];
  snprintf(t, sizeof(t), "Tilt: %.0f deg", s.v.tilt);
  if (dirty(5, t)) {
    tft.fillRect(8, 130, 224, 20, P->r);
    txt(8, 132, 224, t, 15, P->on_r, 'c', true);
  }
  if (!latched) {
    int left = (int)TAMPER_COUNTDOWN_S - (int)((now - s.tamperStartMs) / 1000);
    if (left < 0) left = 0;
    snprintf(t, sizeof(t), "%d", left);
    if (dirty(6, t)) {
      tft.fillRect(8, 150, 224, 84, P->r);
      txt(8, 152, 224, t, 84, P->on_r, 'c', false, 2);
    }
    float frac = 1.0f - (now - s.tamperStartMs) / (TAMPER_COUNTDOWN_S * 1000.0f);
    snprintf(t, sizeof(t), "%d", (int)(frac * 52));
    if (dirty(7, t)) {
      frame(16, 254, 208, 14, P->surf, 2, P->bd);
      tft.fillRect(18, 256, (int)(204 * constrain(frac, 0.0f, 1.0f)), 10, P->bd);
    }
  }
  snprintf(t, sizeof(t), "%d", blink);
  if (dirty(12, t)) {
    tft.fillRect(88, 16, 64, 58, P->r);
    icon(I_TRI, 88 + (blink ? 2 : -2), 14, 60, P->on_r, P->r);
    uint16_t bc = blink ? P->bd : P->r;
    for (int i = 0; i < 8; i++) tft.drawRect(i, i, 240 - 2 * i, 320 - 2 * i, bc);
  }
}

// --- Locked screen: lid stays closed, unlock from the app or dashboard ---
static void drawLocked(const BinState& s) {
  char sig[24];
  snprintf(sig, sizeof(sig), "lock%d", darkTheme);
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(P->p);
    icon(I_LOCK, 80, 46, 80, P->on_p, P->p);
    txt(8, 134, 224, "LOCKED", 40, P->on_p, 'c');
    txt(8, 184, 224, "Lid stays closed.", 15, P->on_p, 'c', true);
    txt(8, 204, 224, "Motion is ignored.", 15, P->on_p, 'c', true);
    frame(8, 228, 224, 34, P->surf, 3, P->bd);
    txt(8, 238, 224, "Unlock on the app or dashboard.", 13, P->tx, 'c');
  }
  topBar(s, true, 0);
  bottomRow("BIN: LOCKED", P->surf, P->tx, 13);
}

// --- Boot screen: progress bar and the result of each sensor check ---
// A FAIL row here matches a "faults" entry in /api/status.
static void drawBoot(const BinState& s, uint32_t now) {
  if (dirty(4, "boot")) {
    fullDraw = true;
    tft.fillScreen(P->t);
    box(16, 18, 208, 66, P->surf, 3, 4);
    icon(I_BIN, 28, 30, 38, P->tx, P->surf);
    txt(76, 30, 140, "SMART", 20, P->tx);
    txt(76, 54, 140, "BIN", 20, P->tx);
    frame(16, 100, 208, 18, P->surf, 3, P->bd);
  }
  tft.fillRect(19, 103, min(202, (int)(now / 20)), 12, P->bd);
  const char* names[] = {"HC-SR04", "BME680", "HX711", "BMI160"};
  bool ok[] = {s.okHC, s.okBME, s.okHX, s.okIMU};
  for (int i = 0; i < 4; i++) {
    char sig[24];
    bool done = now > 1500u + i * 500u;
    snprintf(sig, sizeof(sig), "%d%d%d", i, done, ok[i]);
    if (!dirty(5 + i, sig)) continue;
    int y = 136 + i * 26;
    tft.fillRect(16, y, 210, 22, P->t);
    txt(20, y + 3, 120, names[i], 14, P->on_t);
    if (!done) chip(168, y, 52, 18, "...", P->surf, P->tx);
    else if (ok[i]) chip(168, y, 52, 18, "OK", P->surf, P->tx);
    else chip(168, y, 52, 18, "FAIL", P->r, P->on_r);
```

```
  }
  char ip[40];
  snprintf(ip, sizeof(ip), "IP: %s", s.wifi ? localIp().c_str() : "joining Wi-Fi...");
  if (dirty(9, ip)) {
    box(16, 252, 208, 56, P->surf, 3, 4);
    txt(26, 261, 190, ip, 16, P->tx);
    txt(26, 284, 190, "Firmware v" FW_VERSION, 13, P->tx2, 'l', true);
  }
}

// --- Calibration screen: three steps ticked by TARE, d_empty and d_full ---
static void drawCal(const BinState& s, const Cal& c, uint32_t now) {
  if (dirty(4, darkTheme ? "cal1" : "cal0")) {
    fullDraw = true;
    tft.fillScreen(P->t);
    frame(8, 222, 224, 44, P->surf, 3, P->bd);
    txt(8, 228, 224, "Keep the bin empty and", 13, P->tx, 'c');
    txt(8, 245, 224, "do not touch it.", 13, P->tx, 'c');
    btn(60, 268, 120, 42, "CANCEL", P->r, P->on_r, 15, -1, 0, B_CAL_CANCEL);
  }
  topBar(s, false, 0);
  char sig[40];
  snprintf(sig, sizeof(sig), "CALIBRATING%.*s", (int)((now / 300) % 4), "...");
  if (dirty(5, sig)) {
    tft.fillRect(0, 44, 240, 34, P->t);
    txt(8, 48, 224, "CALIBRATING", 26, P->on_t, 'c');
  }
  snprintf(sig, sizeof(sig), "%d", spin);
  if (dirty(6, sig)) {
    tft.fillRect(90, 80, 60, 60, P->t);
    icon(I_RING, 92, 82, 56, P->on_t, P->t, spin * 15);
  }
  char rows[3][24];
  strcpy(rows[0], "Weight zero");
  snprintf(rows[1], 24, "Empty: %.0f cm", c.d_empty);
  snprintf(rows[2], 24, "Full: %.0f cm", c.d_full);
  for (int i = 0; i < 3; i++) {
    bool ok = s.calSteps & (1 << i);
    char rs[40];
    snprintf(rs, sizeof(rs), "%s%d", rows[i], ok);
    if (!dirty(7 + i, rs)) continue;
    int y = 150 + i * 24;
    tft.fillRect(16, y, 210, 22, P->t);
    txt(20, y + 4, 140, rows[i], 15, P->on_t, 'l', true);
    chip(168, y, 52, 20, ok ? "OK" : "...", P->surf, P->tx);
  }
}

static String uptimeText() {
  uint32_t m = millis() / 60000;
  char b[24];
  if (m < 60) snprintf(b, sizeof(b), "%lu m", (unsigned long)m);
  else snprintf(b, sizeof(b), "%lu h %lu m", (unsigned long)(m / 60), (unsigned long)(m % 60));
  return b;
}

extern bool cloudFirebaseOn();

// --- Info screen: IP address, Wi-Fi, signal, cloud, Bluetooth, firmware,
// uptime. First place to look when the bin does not show up online. ---
static void drawInfo(const BinState& s) {
  if (dirty(4, darkTheme ? "info1" : "info0")) {
    fullDraw = true;
    tft.fillScreen(P->bg);
    btn(8, 266, 164, 44, "WI-FI SETUP", P->bl, P->on_bl, 15, I_BT, 20, B_WIFI_SETUP);
    box(180, 266, 52, 44, P->surf, 3, 4);
    icon(darkTheme ? I_MOON : I_SUN, 194, 275, 24, P->tx, P->surf);
    addHit(176, 262, 64, 58, B_THEME);
  }
  topBar(s, false, -1, "BACK");
  String ip = s.wifi ? localIp() : String("Not connected");
  if (dirty(5, ip.c_str())) {
    box(8, 46, 224, 66, P->surf, 3, 4);
    txt(18, 54, 200, "IP ADDRESS", 12, P->tx2);
    txt(18, 72, 204, ip.c_str(), 27, P->tx);
  }
  char rows[6][2][28];
  strcpy(rows[0][0], "Wi-Fi");
  strlcpy(rows[0][1], s.wifi ? WiFi.SSID().c_str() : net.ssid.c_str(), 28);
  strcpy(rows[1][0], "Signal");
  if (s.wifi) snprintf(rows[1][1], 28, "%d dBm", s.rssi);
  else strcpy(rows[1][1], "--");
  strcpy(rows[2][0], "Cloud");
  strcpy(rows[2][1], !cloudFirebaseOn() ? "OFF" : s.cloudOk ? "Firebase OK" : "NOT SENDING");
  strcpy(rows[3][0], "Bluetooth");
  strcpy(rows[3][1], s.btOn ? "ON" : "OFF");
```

```
  strcpy(rows[4][0], "Firmware");
  strcpy(rows[4][1], "v" FW_VERSION);
  strcpy(rows[5][0], "Uptime");
  strlcpy(rows[5][1], uptimeText().c_str(), 28);
  for (int i = 0; i < 6; i++) {
    char sig[40];
    snprintf(sig, sizeof(sig), "%s", rows[i][1]);
    if (!dirty(6 + i, sig)) continue;
    int y = 124 + i * 23;
    tft.fillRect(8, y, 224, 20, P->bg);
    txt(10, y + 3, 90, rows[i][0], 13, P->tx2, 'l', true);
    txt(100, y + 2, 130, rows[i][1], 15, P->tx, 'r');
    tft.drawFastHLine(8, y + 20, 224, P->rule);
  }
}

static void btTop() {
  tft.fillRect(0, 0, 240, 34, P->bl);
  tft.fillRect(0, 34, 240, 3, P->bd);
  txt(8, 9, 170, "WI-FI SETUP", 16, P->on_bl);
  chip(190, 8, 42, 18, "BT", P->surf, P->tx);
}

static void mmss(char* b, size_t n, int32_t ms) {
  if (ms < 0) ms = 0;
  int s = ms / 1000;
  snprintf(b, n, "%d:%02d", s / 60, s % 60);
}

// --- Bluetooth Wi-Fi setup screens, one layout per phase (see ble_setup.cpp) ---
static void drawBt(uint32_t now) {
  BtView v = bleView();
  char sig[64];
  snprintf(sig, sizeof(sig), "bt%d%d%s%lu%d", v.phase, darkTheme, v.ip, (unsigned long)v.code, v.netCount);
  bool cancel = v.phase != BT_OK && v.phase != BT_FAIL && v.phase != BT_OFF;
  if (dirty(4, sig)) {
    fullDraw = true;
    tft.fillScreen(P->bg);
    btTop();
    if (cancel) btn(8, 266, 224, 44, "CANCEL", P->r, P->on_r, 15, -1, 0, B_BT_CANCEL);
    switch (v.phase) {
      case BT_IDLE:
      case BT_WAIT:
        txt(8, 120, 224, BIN_BLE_NAME, 22, P->tx, 'c');
        wrapTxt(16, 152, 208, "On the dashboard or app press PAIR A BIN and pick this name.", 13, P->tx2, 17);
        break;
      case BT_CODE: {
        txt(8, 46, 224, "PAIRING CODE", 14, P->tx2, 'c');
        box(8, 66, 224, 72, P->surf, 3, 4);
        char code[12];
        snprintf(code, sizeof(code), "%03lu %03lu", (unsigned long)(v.code / 1000), (unsigned long)(v.code % 1000));
        txt(8, 76, 224, code, 46, P->tx, 'c');
        wrapTxt(16, 146, 208, "Type it in the pairing window that pops up on the laptop or phone.", 13, P->tx2, 17);
        txt(8, 242, 224, "3 wrong codes stop pairing for 60 s.", 12, P->tx2, 'c');
        break;
      }
      case BT_LOCKOUT:
        box(8, 48, 224, 130, P->r, 3, 4);
        txt(16, 62, 208, "PAIRING STOPPED", 22, P->on_r, 'c');
        txt(16, 98, 208, "3 wrong codes", 20, P->on_r, 'c');
        wrapTxt(16, 192, 208, "Only someone who can see this screen can pair. The phone that tried was turned away.", 13, P->tx2, 17);
        break;
      case BT_PAIRED:
        box(96, 44, 48, 48, P->g, 3, 4);
        icon(I_CHECK, 102, 50, 36, P->on_g, P->g);
        txt(8, 98, 224, "PAIRED", 22, P->tx, 'c');
        wrapTxt(16, 126, 208, "Pick the network on the dashboard.", 13, P->tx2, 17);
        for (int i = 0; i < min<int>(v.netCount, 3); i++) {
          int y = 168 + i * 27;
          frame(8, y, 224, 23, P->surf, 2, P->bd);
          txtFit(14, y + 4, 156, v.nets[i], 13, P->tx);
          char r[12];
          snprintf(r, sizeof(r), "%d dBm", v.rssi[i]);
          txt(172, y + 4, 54, r, 12, P->tx2, 'r');
        }
        break;
      case BT_CONN:
        txt(8, 128, 224, "CONNECTING...", 22, P->tx, 'c');
        box(24, 166, 192, 34, P->surf, 3, 3);
        txtFit(32, 174, 176, v.ssid, 16, P->tx, 'c');
        txt(20, 214, 200, "Password received.", 13, P->tx2, 'c', true);
        break;
      case BT_OK:
        box(8, 48, 224, 140, P->g, 3, 4);
        txt(16, 58, 208, "CONNECTED", 22, P->on_g, 'c');
```

```
        txt(16, 88, 208, "IP ADDRESS", 12, P->on_g, 'c');
        txt(16, 102, 208, v.ip, 28, P->on_g, 'c');
        txtFit(16, 140, 208, v.ssid, 14, P->on_g, 'c', true);
        txt(16, 162, 208, "Type it in the dashboard.", 13, P->on_g, 'c', true);
        wrapTxt(16, 200, 208, "Bluetooth turns off by itself in 5 minutes.", 13, P->tx2, 17);
        btn(8, 266, 224, 44, "DONE", P->surf, P->tx, 18, I_CHECK, 20, B_BT_DONE);
        break;
      case BT_FAIL:
        box(8, 48, 224, 126, P->r, 3, 4);
        txt(16, 62, 208, "FAILED", 28, P->on_r, 'c');
        txt(16, 102, 208, v.why, 20, P->on_r, 'c');
        txt(16, 132, 208, "Check it and try again.", 14, P->on_r, 'c', true);
        wrapTxt(16, 190, 208, "Other reasons: network not found, or a 5 GHz network.", 13, P->tx2, 17);
        btn(8, 266, 108, 44, "RETRY", P->bl, P->on_bl, 16, -1, 0, B_BT_RETRY);
        btn(124, 266, 108, 44, "CANCEL", P->surf, P->tx, 16, -1, 0, B_BT_CANCEL);
        break;
      case BT_OFF:
        box(84, 60, 72, 72, P->surf, 3, 4);
        icon(I_CHECK, 94, 70, 52, P->tx, P->surf);
        txt(8, 150, 224, "BLUETOOTH OFF", 24, P->tx, 'c');
        txt(20, 184, 200, "Setup finished. Wi-Fi stays on.", 14, P->tx2, 'c', true);
        txt(8, 262, 224, "Back to the main screen", 12, P->tx2, 'c');
        break;
    }
  }
  // Parts of the BT screens that change every second (countdowns, spinner).
  char t[32], mm[12];
  switch (v.phase) {
    case BT_IDLE:
    case BT_WAIT:
      snprintf(sig, sizeof(sig), "%d%d", blink, (int)((v.offAtMs - now) / 1000));
      if (dirty(5, sig)) {
        tft.fillRect(84, 44, 76, 76, P->bg);
        int o = blink ? 0 : 2;
        box(88 + o, 48 + o, 64 - 2 * o, 64 - 2 * o, P->surf, 3, 4);
        icon(I_BT, 96 + o, 56 + o, 48 - 2 * o, P->bl, P->surf);
        mmss(mm, sizeof(mm), (int32_t)(v.offAtMs - now));
        snprintf(t, sizeof(t), "BT OFF IN %s", mm);
        tft.fillRect(20, 220, 204, 36, P->bg);
        box(24, 222, 192, 30, P->surf, 3, 3);
        txt(32, 229, 176, t, 14, P->tx, 'c');
      }
      break;
    case BT_CODE:
      mmss(mm, sizeof(mm), (int32_t)(v.phaseMs + BT_CODE_MS - now));
      if (dirty(5, mm)) {
        snprintf(t, sizeof(t), "NEW CODE IN %s", mm);
        tft.fillRect(20, 202, 204, 34, P->bg);
        box(24, 204, 192, 28, P->surf, 3, 3);
        txt(32, 210, 176, t, 14, P->tx, 'c');
      }
      break;
    case BT_LOCKOUT:
      mmss(mm, sizeof(mm), (int32_t)(v.phaseMs + BT_LOCKOUT_MS - now));
      if (dirty(5, mm)) {
        snprintf(t, sizeof(t), "Try again in %s", mm);
        tft.fillRect(12, 128, 216, 40, P->r);
        txt(8, 134, 224, t, 16, P->on_r, 'c', true);
      }
      break;
    case BT_CONN:
      snprintf(sig, sizeof(sig), "%d", spin);
      if (dirty(5, sig)) {
        tft.fillRect(86, 48, 68, 68, P->bg);
        icon(I_RING, 88, 50, 64, P->bl, P->bg, spin * 15);
      }
      break;
    case BT_OFF: {
      int32_t left = (int32_t)(v.phaseMs + 1500 - now);
      snprintf(sig, sizeof(sig), "%d", max<int32_t>(0, left / 100));
      if (dirty(5, sig)) {
        frame(60, 282, 120, 12, P->surf, 2, P->bd);
        tft.fillRect(62, 284, 116 * max<int32_t>(0, left) / 1500, 8, P->bd);
      }
      if (left <= 0) overlay = O_NONE;
      break;
    }
    default:
      break;
  }
}

// --- Touch calibration ---
// Four crosses at the corners. Each press is averaged over at least 3 samples.
// Started by the TOUCH_CAL command or by holding the screen for 5 s.
static volatile bool tcReq = false, tcResetReq = false;
```

```
1078 static bool tcActive = false, tcWaitUp = false;
1079 static uint8_t tcStep = 0;
1080 static uint32_t tcLastMs = 0, tcDoneMs = 0;
1081 static long tcSx = 0, tcSy = 0;
1082 static int tcN = 0;
1083 static int tcRaw[4][2];
1084 static const int TC_X[4] = {20, 219, 20, 219};
1085 static const int TC_Y[4] = {20, 20, 299, 299};
1086
1087 void uiStartTouchCal() {
1088   tcReq = true;
1089   if (hUi) xTaskNotify(hUi, EV_STATE, eSetBits);
1090 }
1091
1092 void uiResetTouchCal() {
1093   tcResetReq = true;
1094   if (hUi) xTaskNotify(hUi, EV_STATE, eSetBits);
1095 }
1096
1097 // Calibration screen: cross for the current step, or "TOUCH SAVED".
1098 static void drawTouchCal() {
1099   char sig[16];
1100   snprintf(sig, sizeof(sig), "tc%d%d", tcStep, darkTheme);
1101   if (!dirty(4, sig)) return;
1102   fullDraw = true;
1103   tft.fillScreen(P->bg);
1104   if (tcStep < 4) {
1105     txt(8, 112, 224, "TOUCH CALIBRATION", 16, P->tx, 'c');
1106     char b[32];
1107     snprintf(b, sizeof(b), "Press the cross  %d of 4", tcStep + 1);
1108     txt(8, 142, 224, b, 14, P->tx, 'c', true);
1109     wrapTxt(20, 170, 200, "Press firmly on the center, then let go. Wait 30 s to cancel.", 13, P->tx2, 17);
1110     int x = TC_X[tcStep], y = TC_Y[tcStep];
1111     tft.drawFastHLine(x - 14, y, 29, P->r);
1112     tft.drawFastVLine(x, y - 14, 29, P->r);
1113     tft.drawCircle(x, y, 7, P->r);
1114   } else {
1115     txt(8, 120, 224, "TOUCH SAVED", 22, P->tx, 'c');
1116     txt(8, 160, 224, "Buttons should line up now", 14, P->tx2, 'c', true);
1117   }
1118 }
1119
1120 // Which screen to show, highest priority first: touch calibration, boot,
1121 // calibration, Bluetooth, Info/PIN overlays, collection complete, tamper,
1122 // collecting, locked, urgent, main.
1123 static Screen pickScreen(const BinState& s, const Cal& c, const Sim& sm, uint32_t now) {
1124   if (tcActive) return S_TOUCHCAL;
1125   if (s.booting) return S_BOOT;
1126   if (s.calibrating) return S_CAL;
1127   if (overlay == O_BT) return S_BT;
1128   if ((overlay == O_INFO || overlay == O_PIN) && now - overlayMs > SCREEN_TIMEOUT_MS) {
1129     overlay = O_NONE;
1130     buzz(T_BACK_MAIN);
1131   }
1132   if (overlay == O_INFO) return S_INFO;
1133   if (overlay == O_PIN) return S_PIN;
1134   if (doneShownAt && !s.collecting) {
1135     if (now - doneShownAt < 30000) return S_DONE;
1136     doneShownAt = 0;
1137     buzz(T_BACK_MAIN);
1138   }
1139   if (s.tamperPhase) return S_TAMPER;
1140   if (s.collecting) return S_COLLECT;
1141   if (s.status == ST_LOCKED) return S_LOCKED;
1142   if ((s.okHC || (sm.mask & 1)) && s.v.fill >= c.fill_th) return S_URGENT;
1143   return S_MAIN;
1144 }
1145
1146 // Clears the widget cache and hit boxes whenever the screen changes, then
1147 // draws the current screen.
1148 static void render(uint32_t now) {
1149   BinState s = snapshot();
1150   Cal c = calSnapshot();
1151   Sim sm = simSnapshot();
1152   if (s.doneAtMs && !doneShownAt && now - s.doneAtMs < 2000) doneShownAt = s.doneAtMs;
1153   Screen want = pickScreen(s, c, sm, now);
1154   if (want != cur) {
1155     cur = want;
1156     memset(wcache, 0, sizeof(wcache));
1157     hitN = 0;
1158   }
1159   fullDraw = false;
1160   switch (cur) {
1161     case S_BOOT: drawBoot(s, now); break;
1162     case S_CAL: drawCal(s, c, now); break;
1163     case S_MAIN: drawMain(s, c, sm, now); break;
```

```
    case S_URGENT: drawUrgent(s, c, sm, now); break;
    case S_PIN: drawPin(now); break;
    case S_COLLECT: drawCollect(now); break;
    case S_DONE: drawDone(now); break;
    case S_TAMPER: drawTamper(s, c, now); break;
    case S_LOCKED: drawLocked(s); break;
    case S_INFO: drawInfo(s); break;
    case S_BT: drawBt(now); break;
    case S_TOUCHCAL: drawTouchCal(); break;
  }
}

static void resetScreen() {
  memset(wcache, 0, sizeof(wcache));
  hitN = 0;
}

static void openOverlay(Overlay o) {
  overlay = o;
  overlayMs = millis();
  resetScreen();
}

// What each button does. PIN digits are collected here; OK compares them with
// the Collect PIN from the calibration settings.
static void onPress(uint8_t id) {
  BinState s = snapshot();
  if (id >= B_K0 && id <= B_K9) {
    if (strlen(pinBuf) < 4) {
      size_t n = strlen(pinBuf);
      pinBuf[n] = '0' + (id - B_K0);
      pinBuf[n + 1] = 0;
    }
    pinWrongUntil = 0;
    buzz(T_PIN_KEY);
    overlayMs = millis();
    return;
  }
  switch (id) {
    case B_COLLECT:
      pinForBt = false;
      pinBuf[0] = 0;
      buzz(T_OPEN);
      openOverlay(O_PIN);
      break;
    case B_INFO:
      buzz(T_OPEN);
      openOverlay(O_INFO);
      break;
    case B_THEME:
      darkTheme = !darkTheme;
      P = darkTheme ? &DARK : &LIGHT;
      saveTheme(darkTheme);
      buzz(darkTheme ? T_THEME_DARK : T_THEME_LIGHT);
      resetScreen();
      break;
    case B_BACK:
    case B_PIN_CANCEL:
      buzz(T_CANCEL);
      pinBuf[0] = 0;
      openOverlay(O_NONE);
      break;
    case B_KDEL: {
      size_t n = strlen(pinBuf);
      if (n) pinBuf[n - 1] = 0;
      buzz(T_PIN_DEL);
      overlayMs = millis();
      break;
    }
    case B_KOK: {
      char want[8];
      snprintf(want, sizeof(want), "%04u", calSnapshot().collect_pin);
      if (strlen(pinBuf) == 4 && !strcmp(pinBuf, want)) {
        pinBuf[0] = 0;
        if (pinForBt) {
          buzz(T_OPEN);
          openOverlay(O_BT);
          bleStart();
        } else {
          buzz(T_PIN_OK);
          openOverlay(O_NONE);
          sendCommand("COLLECT", SRC_SCREEN);
        }
      } else {
        pinBuf[0] = 0;
        pinWrongUntil = millis() + 1600;
```

```
        buzz(T_PIN_WRONG);
        overlayMs = millis();
      }
      break;
    }
    case B_DONE:
      sendCommand("COLLECT_DONE", SRC_SCREEN);
      doneShownAt = millis();
      break;
    case B_DONE_BACK:
      doneShownAt = 0;
      buzz(T_BACK_MAIN);
      resetScreen();
      break;
    case B_CAL_CANCEL:
      sendCommand("CALIBRATE_END", SRC_SCREEN);
      break;
    case B_WIFI_SETUP:
      pinForBt = true;
      pinBuf[0] = 0;
      buzz(T_OPEN);
      openOverlay(O_PIN);
      break;
    case B_BT_CANCEL:
      buzz(T_CANCEL);
      bleStop(false);
      openOverlay(O_NONE);
      break;
    case B_BT_DONE:
      buzz(T_KEY);
      openOverlay(O_NONE);
      break;
    case B_BT_RETRY:
      buzz(T_KEY);
      bleRetry();
      resetScreen();
      break;
  }
  (void)s;
}

// Reads the touch controller and maps raw values to screen pixels using the
// saved calibration. Pressure z > 200 counts as a press. Set TOUCH_DEBUG to 1
// in config.h to print raw values when the touch is offset.
static bool readTouch(int& x, int& y, int& rawX, int& rawY) {
  if (!touch.touched()) return false;
  TS_Point p = touch.getPoint();
  rawX = p.x;
  rawY = p.y;
  int rx = p.x, ry = p.y;
  if (touchCal.swap) { rx = p.y; ry = p.x; }
#if TOUCH_DEBUG
  Serial.printf("[touch] raw %d %d z %d\n", rx, ry, p.z);
#endif
  x = constrain(map(rx, touchCal.x0, touchCal.x1, 0, 239), 0, 239);
  y = constrain(map(ry, touchCal.y0, touchCal.y1, 0, 319), 0, 319);
  return p.z > 200;
}

static void tcBegin(uint32_t now) {
  tcActive = true;
  tcWaitUp = true;
  tcStep = 0;
  tcN = 0;
  tcSx = tcSy = 0;
  tcLastMs = now;
  overlay = O_NONE;
  resetScreen();
  buzz(T_CAL_START);
}

static void tcEnd() {
  tcActive = false;
  resetScreen();
}

// Works out the raw value at each screen edge from the four presses. If the
// presses changed more along the other axis, the X and Y wires are swapped.
// Presses less than 800 raw units apart are rejected (probably a missed press).
static void tcFinish(uint32_t now) {
  float dxx = (tcRaw[1][0] + tcRaw[3][0]) / 2.0f - (tcRaw[0][0] + tcRaw[2][0]) / 2.0f;
  float dxy = (tcRaw[1][1] + tcRaw[3][1]) / 2.0f - (tcRaw[0][1] + tcRaw[2][1]) / 2.0f;
  bool sw = fabsf(dxy) > fabsf(dxx);
  float px[4], py[4];
  for (int i = 0; i < 4; i++) {
    px[i] = sw ? tcRaw[i][1] : tcRaw[i][0];
```

```
    py[i] = sw ? tcRaw[i][0] : tcRaw[i][1];
  }
  float left = (px[0] + px[2]) / 2, right = (px[1] + px[3]) / 2;
  float top = (py[0] + py[1]) / 2, bottom = (py[2] + py[3]) / 2;
  if (fabsf(right - left) < 800 || fabsf(bottom - top) < 800) {
    Serial.println("[touch] calibration rejected, presses too close together");
    buzz(T_CAL_CANCEL);
    raiseAlert("setup", "warning", "Touch calibration failed", "Presses were too close together. Try again.");
    tcEnd();
    return;
  }
  float kx = (right - left) / (TC_X[1] - TC_X[0]);
  float ky = (bottom - top) / (TC_Y[2] - TC_Y[0]);
  TouchCal t;
  t.swap = sw;
  t.x0 = lroundf(left - kx * TC_X[0]);
  t.x1 = lroundf(right + kx * (239 - TC_X[1]));
  t.y0 = lroundf(top - ky * TC_Y[0]);
  t.y1 = lroundf(bottom + ky * (319 - TC_Y[2]));
  touchCal = t;
  saveTouchCal();
  Serial.printf("[touch] saved swap %d x %d..%d y %d..%d\n", t.swap, t.x0, t.x1, t.y0, t.y1);
  buzz(T_SAVED);
  char meta[72];
  snprintf(meta, sizeof(meta), "%s touchscreen calibrated", BIN_LABEL);
  raiseAlert("setup", "success", "Touch calibrated", meta);
  publish(EV_CAL);
  tcStep = 4;
  tcDoneMs = now;
}

// Touch calibration input: wait for release, collect samples while pressed,
// store the average on release. Gives up after 30 s with no press.
static void tcHandle(bool pressed, int rx, int ry, uint32_t now) {
  if (tcStep >= 4) {
    if (now - tcDoneMs > 2500) tcEnd();
    return;
  }
  if (tcWaitUp) {
    if (!pressed) tcWaitUp = false;
    tcLastMs = now;
    return;
  }
  if (pressed) {
    tcSx += rx;
    tcSy += ry;
    tcN++;
    tcLastMs = now;
    return;
  }
  if (tcN >= 3) {
    tcRaw[tcStep][0] = tcSx / tcN;
    tcRaw[tcStep][1] = tcSy / tcN;
    tcStep++;
    if (tcStep == 4) tcFinish(now);
    else buzz(T_CAL_STEP);
  }
  tcSx = tcSy = 0;
  tcN = 0;
  if (tcActive && tcStep < 4 && now - tcLastMs > 30000) {
    buzz(T_CAL_CANCEL);
    tcEnd();
  }
}

// The touch CS is driven high before the display starts so the touch chip
// stays off the bus. SPI at 40 MHz for the display.
void uiInit() {
  SPI.begin(PIN_SPI_SCK, PIN_SPI_MISO, PIN_SPI_MOSI);
  pinMode(PIN_TOUCH_CS, OUTPUT);
  digitalWrite(PIN_TOUCH_CS, HIGH);
  tft.begin(40000000);
  tft.setRotation(TFT_ROTATION);
  touch.begin(SPI);
  touch.setRotation(TFT_ROTATION);
  darkTheme = loadTheme();
  P = darkTheme ? &DARK : &LIGHT;
  pinMode(PIN_TOUCH_IRQ, INPUT);
  tft.fillScreen(P->t);
}

// Polls touch every 30 ms while pressed, 100 ms otherwise. A press is accepted
// at most every 220 ms (debounce). Holding anywhere for 5 s opens touch
// calibration, a way out when the touch is too far off to hit any button.
void uiTask(void*) {
  bool down = false;
```

```
  uint32_t lastDraw = 0, holdSince = 0;
  for (;;) {
    uint32_t ev = 0;
    xTaskNotifyWait(0, UINT32_MAX, &ev, pdMS_TO_TICKS(down ? 30 : 100));
    uint32_t now = millis();
    if (now - lastAnim >= 500) {
      lastAnim = now;
      blink = !blink;
    }
    spin = (now / 120) % 24;

    if (tcReq) {
      tcReq = false;
      tcBegin(now);
    }
    if (tcResetReq) {
      tcResetReq = false;
      resetTouchCal();
      buzz(T_SAVED);
      publish(EV_CAL);
    }
    {
      int x, y, rx = 0, ry = 0;
      bool pressed = readTouch(x, y, rx, ry);
      if (!pressed) holdSince = 0;
      else if (!holdSince) holdSince = now;
      if (tcActive) {
        tcHandle(pressed, rx, ry, now);
        down = pressed;
        holdSince = 0;
      } else if (pressed && holdSince && now - holdSince > 5000) {
        holdSince = 0;
        tcBegin(now);
      } else if (pressed && !down && now - lastTouchMs > 220) {
        down = true;
        lastTouchMs = now;
        // Last hit box first, so a box drawn on top wins.
        for (int i = hitN - 1; i >= 0; i--) {
          const Hit& h = hits[i];
          if (x >= h.x && x < h.x + h.w && y >= h.y && y < h.y + h.h) {
            if (h.id < B_K0 && h.id != B_KOK && h.id != B_KDEL) buzz(T_KEY);
            onPress(h.id);
            break;
          }
        }
      } else if (!pressed) {
        down = false;
      }
    }
    if ((ev & ~EV_TOUCH) || now - lastDraw >= 100) {
      lastDraw = now;
      render(now);
    }
  }
}
```

firmware/firebase.rules.json

```
{
  "rules": {
    "sb_users": {
      "$uid": {
        ".read": "auth != null && auth.uid === $uid"
      }
    },
    "sb": {
      ".read": "auth != null && (root.child('sb_users').child(auth.uid).exists() || auth.token.email.matches(/^bin.*@/))",
      "bins": {
        "$b": {
          ".write": "auth != null && auth.token.email.matches(/^bin.*@/)"
        }
      },
      "commands": {
        "$b": {
          "$c": {
            ".write": "auth != null && ((!data.exists() && root.child('sb_users').child(auth.uid).exists()) || (!newData.exists() && auth.token.email.matches(/^bin.*@/)))",
            ".validate": "!newData.exists() || (newData.hasChildren(['cmd', 'ts']) && newData.child('cmd').isString())"
          }
        }
      },
      "results": {
        "$r": {
          ".write": "auth != null"
        }
```

```
      },
      "alerts": {
        "$b": {
          ".write": "auth != null && auth.token.email.matches(/^bin.*@/)"
        }
      },
      "watchers": {
        ".indexOn": "ts",
        "$w": {
          ".write": "auth != null && root.child('sb_users').child(auth.uid).exists()"
        }
      },
      "usage": {
        ".write": "auth != null && auth.token.email.matches(/^bin.*@/)"
      }
    }
  }
}
```

firmware/tools/embed_dashboard.py

```
#!/usr/bin/env python3
"""Packs Web Dashboard/smartbin-dashboard.html into firmware/SmartBin/dashboard_gz.h
so each bin serves the dashboard at http://<bin-ip>/ . Run it again after changing the
dashboard, then upload the firmware.

    python firmware/tools/embed_dashboard.py
"""
import gzip
import pathlib

root = pathlib.Path(__file__).resolve().parents[2]
src = root / "Web Dashboard" / "smartbin-dashboard.html"
dst = root / "firmware" / "SmartBin" / "dashboard_gz.h"

data = gzip.compress(src.read_bytes(), compresslevel=9, mtime=0)
rows = [", ".join("0x%02x" % b for b in data[i:i + 20]) for i in range(0, len(data), 20)]
dst.write_text(
    "#pragma once\n// Generated by firmware/tools/embed_dashboard.py from Web Dashboard/smartbin-dashboard.html. Do not edit.\n"
    "#include <Arduino.h>\n"
    "const size_t DASHBOARD_GZ_LEN = %d;\n"
    "const uint8_t DASHBOARD_GZ[] PROGMEM = {\n  %s\n};\n" % (len(data), ",\n  ".join(rows)),
    encoding="ascii",
)
print("%s: %d bytes of HTML -> %d bytes gzipped" % (dst.name, src.stat().st_size, len(data)))
```

Part 2. Web Dashboard

Web Dashboard/smartbin-dashboard.html

```
<!DOCTYPE html>
<!--
================================================================================
smartbin-dashboard.html - Smart-Bin web dashboard (one self-contained file)
Served gzipped by the bin itself at http://<bin-ip>/ (see net.cpp and
tools/embed_dashboard.py) and can also be opened straight from disk.
Layout of this file:
  <style>   neumorphic + neo-brutalist theme, light and dark, mode tints
  <body>    start page (gate), PIN dialog, header, five tab sections
  <script>  state, transport (Wi-Fi, Firebase, ThingSpeak), alerts,
            rendering, charts, settings, simulation, demo tour, sounds,
            operator PIN and Bluetooth Wi-Fi setup
Debug tip: open the browser's developer tools (F12). Network shows each
/api/status, /api/events, Firebase and ThingSpeak request; Console shows
script errors. The dashboard keeps its bin list and settings in
localStorage (keys starting with "smartbin."); clearing site data resets it.
================================================================================
-->
<html lang="en" data-theme="light" data-mode="normal">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Smart Bin Dashboard</title>
<style>
/* ---------- Theme tokens ----------
   Light values here; [data-theme="dark"] overrides them further down.
   --shL/--shD are the light and dark halves of every soft shadow, so
   changing them restyles every raised and sunk surface at once.
   [data-mode] (normal, sim, calib, ...) on <html> tints the page per mode. */
:root{
```

```
  --base:#E4DCD2; --tint:#E4DCD2;
  --shL:#FFFBF5; --shD:#BEB2A4;
  --ink:#2A241E; --ink2:#7C7265;
  --well:#DED5CA;
  --ok:#1E8E3E; --warn:#D98A1F; --bad:#D93025; --sim:#7A45B5; --cal:#0D8A8A;
  --accent:#1E8E3E; --accentSoft:rgba(30,142,62,.22); --onAccent:#FFFFFF;
  --o1:-6px -6px 12px var(--shL), 6px 6px 12px var(--shD);
  --o2:-11px -11px 22px var(--shL), 11px 11px 22px var(--shD);
  --i1:inset -5px -5px 10px var(--shL), inset 5px 5px 10px var(--shD);
  --i2:inset -8px -8px 16px var(--shL), inset 8px 8px 16px var(--shD);
  --r:20px;
}
html[data-theme="dark"]{
  --base:#2B2622; --tint:#2B2622;
  --shL:#3B342E; --shD:#1A1613;
  --ink:#EFE7DC; --ink2:#A2978B;
  --well:#262220;
  --ok:#4FBF6E; --warn:#E8A94A; --bad:#FF6B5E; --sim:#B98BE8; --cal:#3FC7C2;
  --accent:#4FBF6E; --accentSoft:rgba(79,191,110,.22); --onAccent:#13100D;
}
:root{--accentBar:var(--accent);--simSoft:rgba(122,69,181,.3);--calSoft:rgba(13,138,138,.3);
  --c-temp:#C2591B; --c-humid:#1F6FA8; --c-weight:#7A5A2E; --c-gas:#6B4FB8; --c-press:#B03A6F;
  --c-fill:#1E7A52; --c-bt:#1A62C4; --c-lid:#B07415; --c-motion:#0E7F9E; --c-link:#1E8E3E; --c-cloud:#D9631E; --c-ts:#1A62C4;
--c-tamper:#D93025;
  --lv-cool:#1F6FA8; --lv-low:#1E8E3E; --lv-mid:#B88410; --lv-high:#D9641F; --lv-crit:#D93025}
html[data-theme="dark"]{--simSoft:rgba(185,139,232,.34);--calSoft:rgba(63,199,194,.32);
  --c-temp:#F0A368; --c-humid:#6CB8E8; --c-weight:#D6B98C; --c-gas:#B596F0; --c-press:#E58AB4;
  --c-fill:#4FC48D; --c-bt:#79AEF5; --c-lid:#E8B45C; --c-motion:#5FD0E8; --c-link:#4FBF6E; --c-cloud:#F29A5C; --c-ts:#79AEF5;
--c-tamper:#FF6B5E;
  --lv-cool:#6CB8E8; --lv-low:#4FBF6E; --lv-mid:#E8C45A; --lv-high:#F0A368; --lv-crit:#FF6B5E}
html[data-mode="sim"]{--accent:var(--sim);--accentSoft:rgba(122,69,181,.22);--tint:#E2DAE1}
html[data-mode="calib"]{--accent:var(--cal);--accentSoft:rgba(13,138,138,.2);--tint:#DBE3E0}
html[data-mode="dual"]{--accent:#43679F;--accentSoft:rgba(67,103,159,.24);--tint:#DEDDE6;
  --accentBar:linear-gradient(90deg,var(--sim) 0%,#43679F 50%,var(--cal) 100%)}
html[data-mode="alert"]{--accent:var(--bad);--accentSoft:rgba(217,48,37,.2);--tint:#EADCD5}
html[data-theme="dark"][data-mode="sim"]{--accentSoft:rgba(185,139,232,.22);--tint:#2C2530}
html[data-theme="dark"][data-mode="calib"]{--accentSoft:rgba(63,199,194,.2);--tint:#222B2A}
html[data-theme="dark"][data-mode="dual"]{--accent:#7CA9D5;--accentSoft:rgba(124,169,213,.24);--tint:#262A33;
  --accentBar:linear-gradient(90deg,var(--sim) 0%,#7CA9D5 50%,var(--cal) 100%)}
html[data-theme="dark"][data-mode="alert"]{--accentSoft:rgba(255,107,94,.2);--tint:#302422}
html[data-mode="sim"] .ico-accent,html[data-mode="calib"] .ico-accent{color:var(--accent)}

*{box-sizing:border-box}
html,body{margin:0;padding:0}
body{background:var(--tint);color:var(--ink);transition:background .6s ease;
  font-family:Inter,"Segoe UI",system-ui,-apple-system,Arial,sans-serif;font-size:15px;line-height:1.45}
button,input,select{font:inherit;color:inherit}
button{cursor:pointer;background:none;border:0}
:focus-visible{outline:3px solid var(--accent);outline-offset:3px;border-radius:6px}
h2{font-size:12px;letter-spacing:1.4px;font-weight:800;text-transform:uppercase;color:var(--ink2);margin:0 0 14px}

.nm{background:var(--base);border-radius:var(--r);box-shadow:var(--o2)}
.nm-sm{background:var(--base);border-radius:14px;box-shadow:var(--o1)}
.nm-in{background:var(--well);border-radius:14px;box-shadow:var(--i1)}
.nm-deep{background:var(--well);border-radius:16px;box-shadow:var(--i2)}
.glow{box-shadow:var(--o2), 0 0 0 2px var(--accentSoft)}

.btn{background:var(--base);border-radius:14px;padding:11px 18px;font-weight:700;letter-spacing:.3px;
  box-shadow:var(--o1);transition:box-shadow .18s ease,transform .18s ease,color .2s}
.btn:hover{color:var(--accent)}
.btn:active,.btn[aria-pressed="true"]{box-shadow:var(--i1);transform:translateY(1px);color:var(--accent)}
.btn[disabled]{opacity:.4;cursor:not-allowed;box-shadow:var(--o1)}
.btn.sm{padding:7px 13px;font-size:13px;border-radius:11px}
.btn.accent{color:var(--onAccent);background:var(--accent);box-shadow:var(--o1)}
.btn.accent:hover{color:var(--onAccent);filter:brightness(1.06)}
.btn.danger{color:#fff;background:var(--bad);border:2px solid var(--ink);border-radius:6px;box-shadow:4px 4px 0 var(--ink)}
.btn.danger:active{box-shadow:1px 1px 0 var(--ink);transform:translate(3px,3px)}
.btn.wide{width:100%;text-align:center}
.iconbtn{width:38px;height:38px;border-radius:12px;background:var(--base);box-shadow:var(--o1);
  display:grid;place-items:center;color:var(--ink2);transition:box-shadow .18s,color .18s}
.iconbtn:hover{color:var(--accent)}
.iconbtn:active,.iconbtn[aria-expanded="true"]{box-shadow:var(--i1);color:var(--accent)}

input[type=text],input[type=number],input[type=password],select{background:var(--well);border:0;border-
radius:12px;padding:10px 13px;width:100%;
  box-shadow:var(--i1);transition:box-shadow .2s}
input:focus{box-shadow:var(--i1), 0 0 0 2px var(--accentSoft);outline:none}
input[type=number]{font-variant-numeric:tabular-nums}
input[type=range]{accent-color:var(--accent);width:100%}
input[type=checkbox]{accent-color:var(--accent);transform:scale(1.3)}

.ico{width:24px;height:24px;display:block;overflow:visible}
.ico.lg{width:60px;height:60px;overflow:hidden}
.ico.xs{width:17px;height:17px}

@media(prefers-reduced-motion:no-preference){
```

```
  .wave{animation:drift 3.4s linear infinite}
  .vap{animation:rise 2.6s ease-in-out infinite}
  .vap.v2{animation-delay:.5s} .vap.v3{animation-delay:1s}
  .fast .vap{animation-duration:1.1s}
  .pulse{animation:breathe 2.8s ease-in-out infinite}
  .ring{animation:spin 2.4s linear infinite;transform-origin:12px 12px}
  .shake{animation:shake .45s ease-in-out infinite}
  .arcs>*{animation:sonar 1.6s ease-out infinite}
  .arcs>:nth-child(2){animation-delay:.25s} .arcs>:nth-child(3){animation-delay:.5s}
  .wifi.live>*{animation:sonar 2.2s ease-out infinite}
  .wifi.live>:nth-child(2){animation-delay:.3s} .wifi.live>:nth-child(3){animation-delay:.6s}
  .wheel{animation:spin 1.1s linear infinite}
  .glowdot{animation:breathe 2s ease-in-out infinite}
}
@keyframes drift{to{transform:translateX(24px)}}
@keyframes rise{0%{transform:translateY(2px);opacity:.25}50%{opacity:1}100%{transform:translateY(-4px);opacity:.2}}
@keyframes breathe{0%,100%{transform:scale(.82);opacity:.45}50%{transform:scale(1.12);opacity:1}}
@keyframes spin{to{transform:rotate(360deg)}}
@keyframes shake{0%,100%{transform:rotate(-6deg)}50%{transform:rotate(6deg)}}
@keyframes sonar{0%{opacity:0}30%{opacity:1}100%{opacity:0}}
.lidflap{transform-origin:6px 9px;transition:transform .45s cubic-bezier(.3,1.4,.5,1)}
.open .lidflap{transform:rotate(-38deg)}
.shackle{transition:transform .3s ease}
.locked .shackle{transform:translateY(0)} .unlocked .shackle{transform:translateY(-3px)}
.merc,.needle,.wavewrap{transition:all .6s ease}
.pulse{transform-origin:center}

header.top{position:sticky;top:0;z-index:40;background:var(--tint);padding:12px 16px;
  box-shadow:0 10px 18px -16px var(--shD);transition:background .6s ease}
header.top:after{content:"";display:block;height:3px;border-radius:3px;background:var(--accentBar);
  max-width:1280px;margin:12px auto -2px;opacity:.85;transition:background .4s}
.topin{max-width:1280px;margin:0 auto;display:flex;gap:12px;align-items:center;flex-wrap:wrap}
.brand{display:flex;align-items:center;gap:10px;font-weight:800;font-size:18px;letter-spacing:.6px}
.brand .ico{color:var(--accent)}
.spacer{flex:1}
.link{display:flex;align-items:center;gap:9px;font-size:13px;font-weight:700;padding:8px 15px;border-radius:999px;
  background:var(--base);box-shadow:var(--o1)}
.link .ico{color:var(--accent)}
#demoBtn{color:var(--warn);display:inline-flex;align-items:center;gap:6px}
#demoBtn .ico{width:15px;height:15px}
.btnets{display:flex;flex-wrap:wrap;gap:8px;margin-top:10px}
.btnets:empty{display:none}
.btnets .btn{font-size:12.5px;padding:8px 12px;display:flex;gap:8px;align-items:center}
.btnets .btn small{opacity:.65;font-weight:600}
.btnets .btn[aria-pressed="true"]{color:var(--accent)}
.modechip{display:flex;align-items:center;gap:8px;font-size:12px;font-weight:800;letter-spacing:.6px;
  padding:8px 14px;border-radius:999px;background:var(--accentBar);color:var(--onAccent);box-shadow:var(--o1)}

.wrap{max-width:1280px;margin:0 auto;padding:22px 16px 70px}
.tabs{display:flex;gap:10px;flex-wrap:wrap;margin-bottom:20px}
.tab{display:flex;align-items:center;gap:9px;padding:10px 17px;border-radius:14px;font-weight:700;
  background:var(--base);box-shadow:var(--o1);color:var(--ink2);transition:all .2s}
.tab:hover{color:var(--accent)}
.tab[aria-selected="true"]{box-shadow:var(--i1);color:var(--accent)}

.banner{display:flex;gap:12px;align-items:center;flex-wrap:wrap;padding:13px 17px;margin-bottom:14px;
  border:2px solid var(--ink);border-radius:8px;box-shadow:5px 5px 0 var(--ink);font-weight:700;color:#fff}
.banner .ico{color:#fff}
.banner.red{background:var(--bad)} .banner.purple{background:var(--sim)}
.banner.teal{background:var(--cal)}
.banner.green{background:var(--ok)}
.banner.amber{background:var(--warn);color:#241C10}
.banner.amber .ico{color:#241C10}
.banner small{font-weight:600;opacity:.92}

.grid{display:grid;gap:22px}
.bins{grid-template-columns:repeat(auto-fill,minmax(340px,1fr))}
.lower{grid-template-columns:1fr 1fr;margin-top:22px}
@media(max-width:860px){.lower{grid-template-columns:1fr}}

.card{padding:20px;position:relative;transition:box-shadow .4s}
.card.hot{box-shadow:var(--o2), 0 0 0 2px var(--accentSoft)}
.cardtop{display:flex;gap:16px;align-items:flex-start}
.hero{width:76px;height:76px;flex:none;border-radius:50%;display:grid;place-items:center;
  background:var(--well);box-shadow:var(--i1);overflow:hidden}
.who{flex:1;min-width:0}
.binname{font-size:19px;font-weight:800;letter-spacing:.3px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
.binplace{font-size:13px;color:var(--ink2)}
.fillnum{font-size:42px;font-weight:800;line-height:1;font-variant-numeric:tabular-nums;color:var(--accent)}
.fillnum span{font-size:17px}
.chips{display:flex;gap:8px;flex-wrap:wrap;margin-top:10px}
.chip{display:flex;align-items:center;gap:6px;font-size:11.5px;font-weight:800;letter-spacing:.4px;
  padding:6px 11px;border-radius:999px;background:var(--base);box-shadow:var(--o1);color:var(--ink2)}
.chip.loud{color:#fff;box-shadow:3px 3px 0 var(--ink);border:2px solid var(--ink);border-radius:6px}
.bar{height:26px;margin:18px 0 7px;position:relative;overflow:hidden;border-radius:13px}
.bar i{position:absolute;inset:3px auto 3px 3px;border-radius:11px;transition:width .6s ease,background .4s}
```

```
.bar b{position:absolute;top:4px;bottom:4px;width:3px;border-radius:2px;background:var(--ink);opacity:.35}
.ticks{display:flex;justify-content:space-between;font-size:11px;color:var(--ink2);font-weight:700}
.faultnote{margin:6px 0 0;font-size:12px;font-weight:800;color:var(--bad)}
.cardfoot{display:flex;align-items:center;gap:10px;margin:10px 0 2px;font-size:11.5px;font-weight:700;color:var(--ink2)}
.cardfoot .h1{font-size:11px;font-weight:800}
.cardfoot svg{flex:1;height:22px;overflow:visible}
.cardfoot svg path{fill:none;stroke-width:2;stroke-linecap:round;stroke-linejoin:round;vector-effect:non-scaling-stroke}
.cardfoot [data-r="ago"]{white-space:nowrap}
.cardfoot [data-r="ago"].old{color:var(--bad)}
.sensors{display:grid;grid-template-columns:repeat(4,1fr);gap:11px;margin-top:17px}
@media(max-width:520px){.sensors{grid-template-columns:repeat(2,1fr)}}
.sensor{padding:12px 6px 10px;text-align:center;display:flex;flex-direction:column;align-items:center;gap:5px}
.sensor .ico{color:var(--ink2)}
.sensor .v{font-size:17px;font-weight:800;font-variant-numeric:tabular-nums;line-height:1}
.sensor .k{font-size:10.5px;font-weight:700;color:var(--ink2);letter-spacing:.3px}
.sensor.alarm{background:var(--bad);box-shadow:4px 4px 0 var(--ink);border:2px solid var(--ink);border-
radius:10px;color:#fff}
.sensor.alarm .ico,.sensor.alarm .k{color:#fff}
.actions{display:flex;gap:11px;margin-top:18px;flex-wrap:wrap}
.actions .btn{flex:1;min-width:116px;display:flex;align-items:center;justify-content:center;gap:8px}
.editrow{margin-top:16px;padding:16px;display:grid;gap:11px;grid-template-columns:repeat(auto-fit,minmax(140px,1fr))}
.editrow .field{grid-column:span 1}

.addrow{display:flex;align-items:center;gap:14px;margin-top:16px;flex-wrap:wrap}
.addmini{display:inline-flex;align-items:center;gap:9px;padding:9px 16px;border-radius:999px;font-size:13.5px;
  font-weight:700;color:var(--ink2);background:var(--base);box-shadow:var(--o1)}
.addmini:hover{color:var(--accent)}
.addmini .ico{width:18px;height:18px}
.addnote{font-size:12.5px;color:var(--ink2)}

.panel{padding:22px}
.prio{display:flex;flex-direction:column;gap:11px}
.prio .row{display:flex;align-items:center;gap:13px;padding:13px 15px;border-radius:14px;background:var(--base);box-
shadow:var(--o1)}
.prio .n{width:28px;height:28px;flex:none;border-radius:50%;display:grid;place-items:center;font-size:13px;font-weight:800;
  background:var(--well);box-shadow:var(--i1)}
.prio .nm2{flex:1;font-weight:700;min-width:0;overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
.prio .pc{font-weight:800;font-variant-numeric:tabular-nums}
.tag{font-size:10.5px;font-weight:800;letter-spacing:.5px;padding:4px 9px;border-radius:6px;color:#fff;border:2px solid var(-
-ink)}
.ttf{display:flex;align-items:center;gap:12px;padding:12px 15px;border-radius:14px;background:var(--base);box-shadow:var(--
o1);margin-bottom:11px}
.ttf .nm2{flex:1;font-weight:700}
.ttf b{font-variant-numeric:tabular-nums}

.chartbox{padding:22px;margin-bottom:22px}
.chartbox canvas{width:100%;height:240px;display:block;border-radius:14px;background:var(--well);box-shadow:var(--i1)}
.legend{display:flex;gap:18px;font-size:12px;font-weight:700;margin-top:14px;flex-wrap:wrap;color:var(--ink2)}
.legend i{display:inline-block;width:16px;height:4px;border-radius:2px;margin-right:7px;vertical-align:middle}
.range{display:flex;gap:8px}

.seg{display:flex;gap:9px;margin-bottom:18px;flex-wrap:wrap}
.fields{display:grid;grid-template-columns:repeat(auto-fit,minmax(190px,1fr));gap:15px}
.field label{display:block;font-size:11.5px;font-weight:800;letter-spacing:.4px;color:var(--ink2);margin-bottom:7px}
.hint{font-size:12.5px;color:var(--ink2);margin-top:10px}
.switch{display:flex;align-items:center;gap:11px;font-weight:700;font-size:13.5px}
.switch input{width:auto;box-shadow:none;margin:0}

fieldset{border:0;margin:0;padding:0;min-width:0;transition:opacity .35s ease,filter .35s ease}
fieldset[disabled] .sbody{opacity:.5;filter:saturate(.4)}
.calbar{display:flex;gap:18px;align-items:center;flex-wrap:wrap;padding:18px 20px;border-radius:16px;margin-top:4px}
.calbar .calico{width:44px;height:44px;flex:none;border-radius:50%;display:grid;place-items:center;
  background:var(--base);box-shadow:var(--o1);color:var(--ink2);transition:color .4s}
.calbar.on{box-shadow:var(--i1), 0 0 0 2px var(--accentSoft)}
.calbar.on .calico{color:var(--accent)}
.calbar .caltxt{flex:1;min-width:200px}
.calbar .caltxt b{display:block;font-size:15px}
.calbar .caltxt span{font-size:12.5px;color:var(--ink2)}
.switch.big{gap:12px;font-size:14px;padding:11px 16px;border-radius:14px;background:var(--base);box-shadow:var(--
o1);cursor:pointer}
.switch.big input{transform:scale(1.45)}
.calbar.on .switch.big{color:var(--accent)}
.withbtn{display:flex;gap:9px;align-items:center}
.withbtn .btn{flex:none;white-space:nowrap}
.readout{padding:11px 14px;font-weight:800;border-radius:12px;font-variant-numeric:tabular-nums}
.steps{display:grid;gap:13px}
.step{display:flex;gap:15px;align-items:center;padding:15px;border-radius:16px;background:var(--base);box-shadow:var(--o1)}
.step.active{box-shadow:var(--o1), 0 0 0 2px var(--accentSoft)}
.step.done{opacity:.75}
.step .num{width:34px;height:34px;flex:none;border-radius:50%;display:grid;place-items:center;font-weight:800;
  background:var(--well);box-shadow:var(--i1);color:var(--ink2)}
.step.active .num,.step.done .num{background:var(--accent);color:var(--onAccent);box-shadow:var(--o1)}
.step .txt{flex:1;min-width:0}
.step .txt b{display:block;font-size:14.5px}
.step .txt span{font-size:12.5px;color:var(--ink2)}

```

```
table.sim{width:100%;border-collapse:separate;border-spacing:0 11px}
table.sim th{text-align:left;font-size:10.5px;letter-spacing:.8px;color:var(--ink2);padding:0 12px;text-transform:uppercase}
table.sim td{padding:13px 12px;background:var(--base);vertical-align:middle}
table.sim td:first-child{border-radius:14px 0 0 14px;font-weight:700}
table.sim td:last-child{border-radius:0 14px 14px 0}
table.sim tr{box-shadow:var(--o1)}
.simname{display:flex;align-items:center;gap:11px}
.modebtn{min-width:96px;text-align:center;display:flex;align-items:center;justify-content:center;gap:7px}
.modebtn[data-mode="MANUAL"]{background:var(--sim);color:#fff;box-shadow:var(--o1)}
.simval{display:flex;gap:11px;align-items:center}
.simval input[type=number]{width:102px}
.simval span{font-size:12px;color:var(--ink2);font-weight:700}

.alerts{display:flex;flex-direction:column;gap:12px}
.alert{display:flex;gap:14px;align-items:center;padding:14px 16px;border-radius:14px;background:var(--base);box-shadow:var(--o1)}
.alert .av{width:40px;height:40px;flex:none;border-radius:12px;display:grid;place-items:center;background:var(--well);box-shadow:var(--i1)}
.alert .ttl{font-weight:800;font-size:14px}
.alert .meta{font-size:12px;color:var(--ink2)}
.empty{padding:34px;text-align:center;color:var(--ink2);font-weight:700}

.gate{position:fixed;inset:0;background:var(--tint);display:flex;align-items:center;justify-content:center;z-index:100;padding:20px;overflow:auto}
.gatehead{display:flex;align-items:center;gap:18px;margin:0 0 18px}
.gatehead .sblogo{width:92px;height:92px;flex:none}
.gatehead h1{margin:0 0 4px}
.gatehead p{margin:0}
@media(max-width:480px){.gatehead{flex-direction:column;text-align:center}}
.gatedemo{display:flex;align-items:center;gap:16px;margin-top:22px;padding:16px 18px;border-radius:18px;background:var(--well);box-shadow:var(--i1);flex-wrap:wrap}
.gatedemo .btn{display:inline-flex;align-items:center;gap:8px;flex:none}
.gatedemo .btn[aria-pressed="true"]{color:var(--warn)}
.gatedemo p{flex:1;min-width:200px;margin:0;font-size:13px;color:var(--ink2)}
.sndgrp{margin:0 0 14px}
.sndgrp h3{margin:0 0 8px;font-size:12px;letter-spacing:1px;text-transform:uppercase;color:var(--ink2)}
.sndgrp div{display:flex;flex-wrap:wrap;gap:8px}
.sndgrp .btn{font-size:12.5px;padding:8px 12px;display:inline-flex;align-items:center;gap:7px}
.sndgrp .btn .ico{width:14px;height:14px}
.sndgrp .btn.playing{color:var(--accent)}
.gatecard{max-width:660px;width:100%;padding:34px}
.gatecard h1{margin:0 0 7px;font-size:27px;letter-spacing:.4px}
.gatecard p{margin:0 0 24px;color:var(--ink2)}
.roles{display:grid;grid-template-columns:1fr 1fr;gap:18px}
@media(max-width:580px){.roles{grid-template-columns:1fr}}
.role{padding:22px;text-align:left;border-radius:18px;background:var(--base);box-shadow:var(--o1);transition:box-shadow .2s}
.role:hover{box-shadow:var(--o1), 0 0 0 2px var(--accentSoft)}
.role .ico{color:var(--accent);margin-bottom:10px}
.role h3{margin:0 0 8px;font-size:16px}
.role ul{margin:0;padding-left:19px;font-size:13px;color:var(--ink2);line-height:1.65}
.card[data-bmode="sim"]:after,.card[data-bmode="dual"]:after,.card[data-bmode="cal"]:after{
  content:"";position:absolute;inset:-4px;border-radius:24px;pointer-events:none;border:3px solid var(--simSoft)}
.card[data-bmode="cal"]:after{border-color:var(--calSoft)}
.card[data-bmode="dual"]:before{content:"";position:absolute;inset:-9px;border-radius:29px;
  pointer-events:none;border:3px solid var(--calSoft)}

.chips{display:grid;grid-template-columns:repeat(auto-fit,minmax(62px,1fr));gap:9px;margin-top:13px}
.chip{--cc:var(--ink2);justify-content:center;padding:9px 7px;font-size:10.5px;border-radius:12px;
  min-width:0;letter-spacing:.3px}
.chip{white-space:normal;text-align:center;min-height:44px;line-height:1.2}
.chip>span{overflow:visible;white-space:normal;word-break:keep-all}
.chip .ico{width:15px;height:15px;flex:none}
.chip.on{color:var(--cc);box-shadow:var(--o1)}
.chip.on{box-shadow:var(--o1), 0 0 0 2px color-mix(in srgb,var(--cc) 30%,transparent)}
.chip.on .ico{color:var(--cc)}
.chip.off{opacity:.38;color:var(--ink2)}
.actions{display:grid;grid-template-columns:repeat(auto-fit,minmax(102px,1fr));gap:11px}
.actions .btn{--cc:var(--ink2);min-width:0;padding:11px 10px;font-size:13px}
.actions .btn span{overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
.btn{--cc:var(--ink2)}
.btn.tint,.btn.tint .ico{color:var(--cc)}
.btn.tint{box-shadow:var(--o1)}
.btn.tint{box-shadow:var(--o1), 0 0 0 2px color-mix(in srgb,var(--cc) 30%,transparent)}
.btn.tint:hover{color:var(--cc)}
.btn.tint:hover{box-shadow:var(--o1), 0 0 0 4px color-mix(in srgb,var(--cc) 34%,transparent)}
.btn.solid{background:var(--cc);color:var(--onAccent);box-shadow:var(--o1)}
.btn.solid .ico{color:var(--onAccent)}
.btn.solid:hover{color:var(--onAccent);filter:brightness(1.07)}
.btn[disabled],.btn.tint[disabled],.btn.solid[disabled]{color:var(--ink2);background:var(--base);
  box-shadow:var(--o1);opacity:.4}
.btn[disabled] .ico{color:var(--ink2)}

.field label{display:flex;align-items:center;gap:7px;color:var(--lc,var(--ink2))}
.field label .ico{width:15px;height:15px;color:var(--lc,var(--ink2))}
.twoway{display:inline-flex;gap:5px;padding:5px;border-radius:13px;background:var(--well);box-shadow:var(--i1)}
.tw{display:flex;align-items:center;gap:7px;padding:8px 13px;border-radius:9px;font-size:12px;
```

```
  font-weight:700;color:var(--ink2);transition:background .25s,color .25s,box-shadow .25s}
.tw .ico{width:15px;height:15px}
.tw[aria-pressed="true"]{background:var(--cc);color:var(--onAccent);box-shadow:var(--o1)}
.tw[aria-pressed="true"] .ico{color:var(--onAccent)}
.tw[disabled]{opacity:.42;cursor:not-allowed}
.modebtn[data-mode="AUTO"]{--cc:var(--ok)}
.modebtn[data-mode="MANUAL"]{background:var(--sim);color:#fff;box-shadow:var(--o1)}
.modebtn[data-mode="MANUAL"] .ico{color:#fff}

.sensor{--cc:var(--ink2)}
.sensor[data-s="temp"]{--cc:var(--c-temp)}
.sensor[data-s="humid"]{--cc:var(--c-humid)}
.sensor[data-s="weight"]{--cc:var(--c-weight)}
.sensor[data-s="gas"]{--cc:var(--c-gas)}
.sensor .ico{color:var(--cc)}
.sensor .v{color:var(--lvc,var(--ink));transition:color .5s ease}
.sensor[data-lv="cool"]{--lvc:var(--lv-cool)}
.sensor[data-lv="low"]{--lvc:var(--lv-low)}
.sensor[data-lv="mid"]{--lvc:var(--lv-mid)}
.sensor[data-lv="high"]{--lvc:var(--lv-high)}
.sensor.alarm{--cc:#fff;--lvc:#fff}
.sensor .k{font-size:10px}

.sect{--cc:var(--ink2)}
.sect[data-sec="range"]{--cc:var(--cal)}
.sect[data-sec="limits"]{--cc:var(--bad)}
.sect[data-sec="lid"]{--cc:var(--sim)}
.sect[data-sec="weight"]{--cc:var(--c-weight)}
.sect[data-sec="air"]{--cc:var(--c-gas)}
.sect[data-sec="touch"]{--cc:var(--cal)}
.sect{margin-top:12px;border-radius:16px;background:var(--base);box-shadow:var(--o1)}
.dhd{margin:6px 0 8px;font-size:12px;letter-spacing:1px;text-transform:uppercase;color:var(--ink2)}
#demoBody .seg{margin-bottom:12px}
.shead{all:unset;box-sizing:border-box;width:100%;min-height:54px;padding:0 16px;display:flex;align-
items:center;gap:10px;cursor:pointer;border-radius:16px}
.shead:focus-visible{outline:2px solid var(--cc);outline-offset:2px}
.shead:before{content:"";width:4px;height:15px;border-radius:2px;background:var(--cc);flex:none}
.shead .ico{width:18px;height:18px;color:var(--cc);flex:none}
.shead .stitle{font-size:12px;font-weight:800;letter-spacing:1.2px;text-transform:uppercase;color:var(--cc);white-
space:nowrap}
.shead .ssum{flex:1;text-align:right;font-size:12.5px;font-weight:700;color:var(--ink2);white-
space:nowrap;overflow:hidden;text-overflow:ellipsis}
.shead .schev{color:var(--ink2);transition:transform .25s}
.sect.open .schev{transform:rotate(180deg)}
.sect .sbody{display:none;padding:2px 16px 16px}
.sect.open .sbody{display:block}
.sect .sbody>.fields:first-child{margin-top:4px}
.sect input:focus{box-shadow:var(--i1), 0 0 0 2px var(--accentSoft)}
.sect input:focus{box-shadow:var(--i1), 0 0 0 2px color-mix(in srgb,var(--cc) 38%,transparent)}
.simname{--cc:var(--ink2)}
.simname .ico{color:var(--cc)}
table.sim tr[data-manual="1"] td:first-child{box-shadow:inset 4px 0 0 var(--cc)}

.ttf{border-left:5px solid var(--ink2)}
.ttf[data-lv="near"]{border-left-color:var(--bad)}
.ttf[data-lv="mid"]{border-left-color:var(--warn)}
.ttf[data-lv="far"]{border-left-color:var(--ok)}
.ttf .when{font-variant-numeric:tabular-nums;font-weight:800;margin-left:auto}
.ttf .lvtag{font-size:10px;font-weight:800;letter-spacing:.5px;padding:4px 10px;border-radius:20px;color:#fff;min-
width:56px;text-align:center;
  box-shadow:-2px -2px 4px rgba(255,255,255,.18), 2px 2px 4px rgba(0,0,0,.22)}

.alert{border-left:5px solid var(--ink2)}
.alert[data-lv="warning"]{border-left-color:var(--warn)}
.alert[data-lv="info"]{border-left-color:var(--sim)}
.alert[data-lv="success"]{border-left-color:var(--ok)}
.alert[data-lv="critical"]{background:var(--bad);color:#fff;border:2px solid var(--ink);
  border-radius:8px;box-shadow:5px 5px 0 var(--ink)}
.alert[data-lv="critical"] .av{background:rgba(255,255,255,.17);box-shadow:none;color:#fff}
.alert[data-lv="critical"] .meta{color:rgba(255,255,255,.88)}
.alert .lvtag{margin-left:auto;font-size:10px;font-weight:800;letter-spacing:.6px;padding:4px 10px;
  border-radius:20px;color:#fff;white-space:nowrap;
  box-shadow:-2px -2px 4px rgba(255,255,255,.18), 2px 2px 4px rgba(0,0,0,.22)}
.alert[data-lv="critical"] .lvtag{background:rgba(0,0,0,.28)}

.addrow{justify-content:flex-end;margin:0 0 16px}
.addpick{display:flex;align-items:center;gap:4px;padding:4px;border-radius:999px;background:var(--well);box-shadow:var(--i1)}
.addpick button{width:28px;height:28px;border-radius:50%;font-weight:800;font-size:16px;line-height:1;
  color:var(--ink2);background:var(--base);box-shadow:var(--o1)}
.addpick button:hover{color:var(--accent)}
.addpick button[disabled]{opacity:.35;cursor:not-allowed}
.addpick span{min-width:22px;text-align:center;font-weight:800;font-variant-numeric:tabular-nums}

@media(max-width:600px){
  .wrap{padding:16px 12px 56px}
  header.top{padding:10px 12px}
```

```
  .brand{font-size:16px}
  .link,.modechip{font-size:11px;padding:7px 11px}
  .tabs{gap:7px}
  .tab{padding:9px 13px;font-size:13px;gap:7px}
  .card{padding:16px}
  .cardtop{gap:13px}
  .hero{width:62px;height:62px}
  .hero .ico.lg{width:46px;height:46px}
  .binname{font-size:17px}
  .fillnum{font-size:34px}
  .fillnum span{font-size:14px}
  .chips{grid-template-columns:repeat(auto-fit,minmax(58px,1fr));gap:8px}
  .sensors{grid-template-columns:repeat(2,1fr)}
  .actions{grid-template-columns:repeat(2,1fr)}
  .panel,.chartbox,.gatecard{padding:17px}
  .fields{grid-template-columns:1fr}
  .calbar{padding:15px;gap:13px}
  .switch.big{width:100%;justify-content:center}
  table.sim td{padding:11px 9px}
  .simval input[type=number]{width:84px}
  .addrow{justify-content:space-between}
  .prio .row,.ttf{padding:11px 13px;gap:10px}
  .alert{padding:12px 13px;gap:11px}
  .alert .lvtag{display:none}
}
@media(max-width:380px){
  .chips{grid-template-columns:repeat(auto-fit,minmax(58px,1fr))}
  .actions{grid-template-columns:1fr}
  .tab span{display:none}
  .tab{padding:10px 14px}
}

/* --- v3 additions --- */
body{background:var(--tint)}
body{background:
  radial-gradient(900px 520px at 12% -8%, color-mix(in srgb,var(--accent) 7%,transparent), transparent 70%),
  radial-gradient(760px 460px at 105% 4%, color-mix(in srgb,var(--accent) 5%,transparent), transparent 72%),
  var(--tint);background-attachment:fixed}
.nm,.panel,.card,.chartbox{background-image:linear-gradient(160deg,color-mix(in srgb,var(--shL) 42%,transparent),transparent 58%)}

.brand{cursor:default;position:relative;padding:4px 2px}
.brand .btxt{background:linear-gradient(95deg,var(--ink) 0%,var(--ink) 40%,var(--accent) 55%,var(--ink) 72%,var(--ink) 100%);
  background-size:280% 100%;background-position:100% 0;-webkit-background-clip:text;background-clip:text;color:transparent;
  transition:background-position 1.1s ease,letter-spacing .35s ease}
.brand:hover .btxt{background-position:0 0;letter-spacing:1.3px}
.brand .ico{transition:transform .5s cubic-bezier(.25,1.5,.45,1)}
.brand:hover .ico{transform:translateY(-2px) rotate(-8deg)}
.brand:hover .ico .lidflap{transform:rotate(-34deg)}

.sunmoon .sunside,.sunmoon .moonside{transition:opacity .45s ease,transform .5s cubic-bezier(.2,.9,.3,1.3)}
.sunmoon.is-light .sunside{opacity:.3;transform:scale(.82)}
.sunmoon.is-light .moonside{opacity:1;color:var(--accent)}
.sunmoon.is-dark .moonside{opacity:.3;transform:scale(.82)}
.sunmoon.is-dark .sunside{opacity:1;color:var(--accent)}
#theme{width:42px;height:42px}
#theme .ico{width:21px;height:21px;color:var(--accent)}

.statuspill{--cc:var(--warn);display:flex;align-items:center;gap:8px;padding:8px 15px;border-radius:999px;
  font-size:12px;font-weight:800;letter-spacing:.4px;color:var(--cc);background:var(--base);box-shadow:var(--o1)}
.statuspill{box-shadow:var(--o1), 0 0 0 2px color-mix(in srgb,var(--cc) 34%,transparent)}
.statuspill .ico{width:16px;height:16px;color:var(--cc)}
.statuspill[aria-expanded="true"]{box-shadow:var(--i1)}
.bannerpop{position:absolute;left:16px;right:16px;top:100%;margin:10px auto 0;max-width:1280px;z-index:60;
  display:flex;flex-direction:column;gap:10px;padding:14px;border-radius:18px;background:var(--base);
  box-shadow:var(--o2), 0 18px 40px -22px rgba(0,0,0,.5);max-height:62vh;overflow-y:auto}
.bannerpop .banner{margin:0}

.sechead{display:flex;align-items:center;gap:14px;flex-wrap:wrap;margin-bottom:14px;min-height:46px;position:relative;z-index:2;
  padding:6px 4px;margin-left:-4px;margin-right:-4px;border-radius:14px}
.panel .sechead{background:var(--base)}
section > .sechead{background:var(--tint)}
.sectitle,.ph{display:flex;align-items:center;gap:9px;margin:0;position:relative;z-index:1;
  font-size:12px;letter-spacing:1.4px;font-weight:800;text-transform:uppercase;color:var(--cc,var(--ink2))}
.sectitle .ico,.ph .ico{width:17px;height:17px;color:var(--cc,var(--ink2))}
.panel>.ph,.chartbox>.ph{margin:0 0 18px}
.sechead .sectitle,.sechead .ph{margin:0}
.sechead .addrow{margin:0 0 0 auto}
.alertctl{margin-left:auto;display:flex;gap:9px;align-items:center;flex-wrap:wrap}
.alertctl select{width:auto;padding:7px 11px;font-size:12.5px;font-weight:700}

.scrollpane{overflow:visible}
.alertpane{max-height:min(58vh,560px);overflow-y:auto;padding:26px;margin:-26px}
@media(min-width:900px) and (min-height:620px){
  #binsPane{max-height:calc(100vh - 430px);min-height:500px;overflow-y:auto;padding:26px;margin:-26px}
```

```
}
.scrollpane::-webkit-scrollbar,.bannerpop::-webkit-scrollbar{width:11px}
.scrollpane::-webkit-scrollbar-track,.bannerpop::-webkit-scrollbar-track{background:transparent}
.scrollpane::-webkit-scrollbar-thumb,.bannerpop::-webkit-scrollbar-thumb{
  background:var(--base);border:3px solid transparent;background-clip:content-box;border-radius:999px;
  box-shadow:inset 0 0 0 1px color-mix(in srgb,var(--ink2) 30%,transparent)}
.scrollpane{scrollbar-width:thin;scrollbar-color:var(--shD) transparent}

.tab{--cc:var(--ink2)}
.tab[aria-selected="true"]{color:var(--cc)}
.tab[aria-selected="true"]{box-shadow:var(--i1), 0 0 0 2px color-mix(in srgb,var(--cc) 26%,transparent)}
.tab:hover{color:var(--cc)}
.tab .ico{color:inherit}

.chip{font-size:clamp(8.5px,2.1vw,10.5px);padding:8px 5px;gap:5px}
.chip .ico{width:14px;height:14px}
.actions .btn{white-space:normal;min-height:52px;line-height:1.15;text-align:center}
.actions .btn span{white-space:normal;overflow:visible;font-size:clamp(10.5px,2.3vw,13px)}
.btn .sub{display:block;font-size:9.5px;font-weight:700;opacity:.72;letter-spacing:.3px;margin-top:2px}

.gearbtn{width:46px;height:46px;border-radius:14px;display:flex;flex-direction:column;align-items:center;
  justify-content:center;gap:1px;background:var(--base);box-shadow:var(--o1);color:var(--ink2)}
.gearbtn .ico{width:16px;height:16px}
.gearbtn em{font-style:normal;font-size:7.5px;font-weight:800;letter-spacing:.4px}
.gearbtn:hover,.gearbtn[aria-expanded="true"]{color:var(--accent)}
.gearbtn[aria-expanded="true"]{box-shadow:var(--i1)}

.binbtn{display:inline-flex;align-items:center;gap:8px}
.binbtn .ico{width:15px;height:15px;color:var(--cc,var(--ink2))}
.binbtn[data-sim="1"]{--cc:var(--sim)}
.binbtn[data-sim="1"]{box-shadow:var(--o1), 0 0 0 2px color-mix(in srgb,var(--sim) 32%,transparent)}
.binbtn[aria-pressed="true"]{color:var(--cc,var(--accent))}
.binbtn[aria-pressed="true"]{box-shadow:var(--i1), 0 0 0 2px color-mix(in srgb,var(--cc,var(--accent)) 30%,transparent)}

.twoway.multi{flex-wrap:wrap}
.tw .dot{width:10px;height:10px;border-radius:50%;display:block;box-shadow:inset 0 0 0 1px rgba(0,0,0,.18)}
table.sim th{display:table-cell}
table.sim th .thh{display:flex;align-items:center;gap:7px;color:var(--cc,var(--ink2))}
table.sim th .ico{width:14px;height:14px}
.alert .del{width:30px;height:30px;flex:none;border-radius:9px;display:grid;place-items:center;color:var(--ink2);
  background:var(--base);box-shadow:var(--o1);opacity:0;transition:opacity .25s,color .25s}
.alert:hover .del,.alert:focus-within .del{opacity:1}
.alert .del:hover{color:var(--bad)}
.alert[data-lv="critical"] .del{background:rgba(255,255,255,.18);color:#fff;box-shadow:none}
@media(hover:none){.alert .del{opacity:1}}

.chip{flex-direction:column;gap:4px;padding:9px 5px;min-height:62px;overflow:hidden;justify-content:center}
.chip>span{display:block;width:100%;overflow-wrap:break-word;word-break:normal;line-height:1.15}
.chip .ico{width:16px;height:16px}
@media(max-width:600px){.chip{min-height:58px}}

.btstate{margin-left:auto;display:flex;align-items:center;gap:8px;font-size:12px;font-weight:800;
  letter-spacing:.4px;padding:7px 14px;border-radius:999px;color:var(--ink2);background:var(--base);box-shadow:var(--o1)}
.btstate[data-on="1"]{color:var(--c-bt)}
.btstate[data-on="1"]{box-shadow:var(--o1), 0 0 0 2px color-mix(in srgb,var(--c-bt) 32%,transparent)}
.btsteps{display:grid;grid-template-columns:repeat(4,1fr);gap:10px}
@media(max-width:600px){.btsteps{grid-template-columns:repeat(2,1fr)}}
.btstep{display:flex;flex-direction:column;align-items:center;gap:7px;text-align:center;padding:14px 8px;
  border-radius:14px;background:var(--well);box-shadow:var(--i1);color:var(--ink2);transition:all .35s ease}
.btstep .ico{width:19px;height:19px}
.btstep b{font-size:11px;font-weight:800;letter-spacing:.4px;line-height:1.2}
.btstep.on{color:var(--c-bt);background:var(--base);box-shadow:var(--o1)}
.btstep.on{box-shadow:var(--o1), 0 0 0 2px color-mix(in srgb,var(--c-bt) 30%,transparent)}
.btstep.done{color:var(--ok)}
.btresult{margin-top:18px;padding:16px;border-radius:14px;background:var(--well);box-shadow:var(--i1);
  display:flex;align-items:center;gap:14px;flex-wrap:wrap}
.btresult .ip{font-weight:800;font-size:19px;font-variant-numeric:tabular-nums;color:var(--ok)}

.hide{display:none !important}

/* --- v6: collect PIN card for the collector --- */
/* Panel button rows are a flex row, not the card grid, so hiding the write
   buttons for a collector leaves RELOAD at its own width instead of stretched. */
.panel>.actions{display:flex;flex-wrap:wrap}
.panel>.actions .btn{flex:0 0 auto}

.pincard{display:flex;align-items:center;gap:16px;flex-wrap:wrap;padding:18px 20px;
  border-radius:16px;margin:0 0 20px}
.pincard .pinico{width:46px;height:46px;flex:none;border-radius:50%;display:grid;place-items:center;
  background:var(--base);box-shadow:var(--o1);color:var(--cal)}
.pincard .pinico .ico{width:22px;height:22px;display:block}
.pincard .pintxt{flex:1;min-width:200px}
.pincard .pintxt b{display:block;font-size:15px}
.pincard .pintxt span{font-size:12.5px;color:var(--ink2)}
.pinval{font-size:34px;font-weight:800;letter-spacing:7px;line-height:1.1;color:var(--cal);
  padding:2px 0 2px 10px;font-variant-numeric:tabular-nums}
```

```
.pinval.masked{letter-spacing:4px}
@media(max-width:600px){.pincard{padding:15px}.pinval{font-size:28px;letter-spacing:5px}}

/* --- v5: uniform status chips, always 3 across and 2 down --- */
.chips{display:grid;grid-template-columns:repeat(3,minmax(0,1fr));gap:7px;margin-top:14px}
.chip{flex-direction:row;align-items:center;justify-content:center;gap:5px;
  min-height:0;height:28px;padding:0 8px;border-radius:9px;
  font-size:10.5px;letter-spacing:.3px;line-height:1;white-space:nowrap;overflow:hidden}
.chip>span{display:inline;width:auto;white-space:nowrap;overflow-wrap:normal;line-height:1}
.chip .ico{width:12px;height:12px;flex:none}
@media(max-width:420px){.chip{padding:0 6px;gap:4px;font-size:10px}.chip .ico{width:11px;height:11px}}

/* --- v3: read / unread alerts --- */
.alert .rd{width:26px;height:26px;flex:none;border-radius:50%;display:grid;place-items:center;
  padding:0;background:none;box-shadow:none;color:var(--cc,var(--ink2))}
.alert .rd i{width:11px;height:11px;border-radius:50%;background:currentColor;
  box-shadow:0 0 0 3px color-mix(in srgb,currentColor 24%,transparent);transition:transform .25s ease,box-shadow .25s ease,background .25s ease}
.alert[data-read="1"] .rd i{background:transparent;box-shadow:inset 0 0 0 2px currentColor;opacity:.5}
.alert[data-read="1"]{opacity:.66}
.alert[data-read="1"] .ttl{font-weight:700}
.alert[data-lv="critical"] .rd{color:#fff}
.alert .rd:hover i{transform:scale(1.3)}
.unreadpip{font-style:normal;font-size:10px;font-weight:800;letter-spacing:.5px;padding:3px 8px;
  border-radius:5px;background:var(--c-tamper);color:#fff}
.popsec{font-size:10.5px;font-weight:800;letter-spacing:1.1px;text-transform:uppercase;
  color:var(--ink2);padding:4px 4px 0}
.popact{display:flex;gap:9px;flex-wrap:wrap;padding:0 0 4px}
.popact .btn{display:inline-flex;align-items:center;gap:8px}

/* --- motion layer --- */
.btn,.tab,.iconbtn,.role,.addmini,.chip,.prio .row,.ttf,.alert,.step,.card{position:relative}
.btn,.tab,.iconbtn,.role,.addmini{overflow:hidden}
.ripple{position:absolute;border-radius:50%;background:currentColor;opacity:.16;pointer-events:none;
  transform:scale(0);animation:ripple .62s cubic-bezier(.2,.8,.3,1) forwards}
.halo{opacity:0;transform-origin:12px 17.2px;transition:opacity .45s ease}
.sensor.alarm .halo{opacity:.38;animation:breathe 2.6s ease-in-out infinite}
.sensor.alarm .vap{animation-duration:1.15s}

@keyframes ripple{to{transform:scale(1);opacity:0}}
@keyframes riseIn{from{opacity:0;transform:translateY(22px) scale(.97)}to{opacity:1;transform:none}}
@keyframes slideDown{from{opacity:0;transform:translateY(-14px)}to{opacity:1;transform:none}}
@keyframes slideUp{from{opacity:0;transform:translateY(20px)}to{opacity:1;transform:none}}
@keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:none}}
@keyframes expand{from{opacity:0;transform:scaleY(.8);transform-origin:top}to{opacity:1;transform:none}}
@keyframes pop{0%{transform:scale(1)}35%{transform:scale(1.14)}100%{transform:scale(1)}}
@keyframes haze{0%,100%{box-shadow:var(--o2), 0 0 0 2px var(--accentSoft)}
  50%{box-shadow:var(--o2), 0 0 0 7px var(--accentSoft)}}
@keyframes sheen{from{transform:translateX(-120%)}to{transform:translateX(320%)}}
@keyframes stepGlow{0%,100%{box-shadow:var(--o1), 0 0 0 2px var(--accentSoft)}
  50%{box-shadow:var(--o1), 0 0 0 6px var(--accentSoft)}}

@media(prefers-reduced-motion:no-preference){
  .card{transition:transform .38s cubic-bezier(.2,.8,.3,1),box-shadow .38s ease,background .6s ease}
  .card:hover{transform:translateY(-5px);
    box-shadow:-15px -15px 30px var(--shL), 15px 15px 30px var(--shD), 0 0 0 2px var(--accentSoft)}
  .card.urgent{animation:haze 2.6s ease-in-out infinite}
  .hero .ico{transition:transform .45s cubic-bezier(.2,.9,.3,1.3)}
  .card:hover .hero .ico{transform:scale(1.08) rotate(-3deg)}
  .card:hover .fillnum{transform:scale(1.04)}
  .fillnum{transition:transform .35s cubic-bezier(.2,.9,.3,1.3)}

  .sensor{transition:transform .26s cubic-bezier(.2,.8,.3,1),box-shadow .26s ease}
  .sensor.nm-in:hover{transform:translateY(-4px);box-shadow:var(--o1)}
  .sensor.nm-in:hover .ico{color:var(--accent)}
  .sensor .ico{transition:color .3s ease,transform .3s ease}
  .sensor.nm-in:hover .ico{transform:scale(1.12)}

  .btn .ico,.tab .ico,.iconbtn .ico,.chip .ico{transition:transform .32s cubic-bezier(.2,.9,.3,1.3)}
  .btn:hover .ico{transform:scale(1.18)}
  .btn.danger:hover .ico{transform:scale(1.18) rotate(-8deg)}
  .tab:hover .ico{transform:translateY(-3px)}
  .iconbtn:hover .ico{transform:rotate(60deg)}
  .chip:hover{transform:translateY(-2px)}
  .chip{transition:transform .22s ease}

  .addmini{transition:transform .3s ease,box-shadow .3s ease,color .25s}
  .addmini:hover{transform:translateY(-2px);box-shadow:var(--o2)}
  .addmini .ico{transition:transform .5s cubic-bezier(.25,1.5,.45,1)}
  .addmini:hover .ico{transform:rotate(90deg)}

  .prio .row,.ttf,.alert{transition:transform .28s cubic-bezier(.2,.8,.3,1),box-shadow .28s}
  .prio .row:hover,.ttf:hover,.alert:hover{transform:translateX(6px);box-shadow:var(--o2)}
  .prio .row:hover .n{background:var(--accent);color:var(--onAccent);box-shadow:var(--o1)}
  .prio .n{transition:background .3s,color .3s,box-shadow .3s}
  .role{transition:transform .32s cubic-bezier(.2,.8,.3,1),box-shadow .32s}
```

```
  .role:hover{transform:translateY(-6px)}
  .role:hover .ico{transform:scale(1.1) rotate(6deg)}
  .role .ico{transition:transform .45s cubic-bezier(.2,.9,.3,1.3)}

  .step{transition:transform .3s ease,box-shadow .3s ease}
  .step:hover{transform:translateX(5px)}
  .step.active{animation:stepGlow 2.8s ease-in-out infinite}
  .step .num{transition:background .35s,color .35s,transform .35s cubic-bezier(.2,.9,.3,1.4)}
  .step.done .num{transform:scale(1.06)}

  .enter{animation:riseIn .55s cubic-bezier(.2,.8,.3,1) both}
  .banner{animation:slideDown .45s cubic-bezier(.2,.8,.3,1) both}
  .toast{animation:slideUp .4s cubic-bezier(.2,.8,.3,1) both}
  section[data-view]:not(.hide){animation:fadeIn .38s ease both}
  .editrow:not(.hide){animation:expand .36s cubic-bezier(.2,.8,.3,1) both}
  .modechip.flash,.statuspill.flash{animation:pop .65s cubic-bezier(.2,.9,.3,1.3)}
  table.sim tr{transition:transform .26s ease,box-shadow .26s}
  table.sim tr:hover{transform:translateX(4px);box-shadow:var(--o2)}
  .link:hover,.modechip:hover{transform:translateY(-2px)}
  .link,.modechip{transition:transform .25s ease,background .4s ease}

  .bar i:after{content:"";position:absolute;inset:0;border-radius:11px;overflow:hidden}
  .urgent .bar i:before{content:"";position:absolute;top:0;bottom:0;width:38%;border-radius:11px;
    background:linear-gradient(90deg,transparent,rgba(255,255,255,.45),transparent);
    animation:sheen 2.1s ease-in-out infinite}
  .bar i{overflow:hidden}
}
.toast{position:fixed;left:16px;right:16px;bottom:18px;max-width:560px;margin:0 auto;z-index:90}

/* --- v9: soft button glows no longer collide with the hard banner shadows --- */
.bannerpop{padding:16px 24px 18px 16px;gap:12px}
.bannerpop .banner{margin:0 0 4px}
.popact{padding:12px 10px 12px 8px;gap:16px;margin-top:2px}
.popact .btn.tint{box-shadow:3px 3px 8px var(--shD),-3px -3px 8px var(--shL),
  0 0 0 2px color-mix(in srgb,var(--cc) 30%,transparent)}
.popact .btn.tint:hover{box-shadow:3px 3px 8px var(--shD),-3px -3px 8px var(--shL),
  0 0 0 4px color-mix(in srgb,var(--cc) 34%,transparent)}
.popact .btn.tint:active{box-shadow:var(--i1),0 0 0 2px color-mix(in srgb,var(--cc) 30%,transparent)}
/* The alert list used to clip its own glow into a white line along the top and left. */
.alertpane{padding:26px;margin:-26px}
.alertpane .alert:hover{box-shadow:var(--o1)}
.alert[data-lv="critical"]:hover{box-shadow:7px 7px 0 var(--ink)}
.alertctl .btn.tint,.sechead .btn.tint{box-shadow:3px 3px 8px var(--shD),-3px -3px 8px var(--shL),
  0 0 0 2px color-mix(in srgb,var(--cc) 30%,transparent)}
.alertctl .btn.tint:hover{box-shadow:3px 3px 8px var(--shD),-3px -3px 8px var(--shL),
  0 0 0 4px color-mix(in srgb,var(--cc) 34%,transparent)}
.alertctl .btn.tint:active{box-shadow:var(--i1),0 0 0 2px color-mix(in srgb,var(--cc) 30%,transparent)}
.alertctl .btn.tint[disabled]{box-shadow:3px 3px 8px var(--shD),-3px -3px 8px var(--shL)}
.legend .now{font-weight:800;color:var(--ink)}
@media(min-width:900px){.field.span2{grid-column:span 2}}
.withbtn input{min-width:0;flex:1}

/* scroll fade masks — soften the hard cutoff at scroll edges */
.alertpane,.scrollpane{-webkit-mask-image:linear-gradient(to bottom,transparent 0,#000 26px,#000 calc(100% -
26px),transparent 100%);
  mask-image:linear-gradient(to bottom,transparent 0,#000 26px,#000 calc(100% - 26px),transparent 100%)}
.alertpane.at-top,.scrollpane.at-top{-webkit-mask-image:linear-gradient(to bottom,#000 0,#000 calc(100% - 26px),transparent
100%);
  mask-image:linear-gradient(to bottom,#000 0,#000 calc(100% - 26px),transparent 100%)}
.alertpane.at-bot,.scrollpane.at-bot{-webkit-mask-image:linear-gradient(to bottom,transparent 0,#000 26px,#000 100%);
  mask-image:linear-gradient(to bottom,transparent 0,#000 26px,#000 100%)}
.alertpane.at-top.at-bot,.scrollpane.at-top.at-bot{-webkit-mask-image:none;mask-image:none}

/* ---- v10 additions ---- */
.ovtools{display:flex;align-items:center;gap:10px;margin-bottom:12px;flex-wrap:wrap;position:relative;z-index:2;
  background:var(--tint);padding:8px 4px;margin-left:-4px;margin-right:-4px;border-radius:14px}
.filterchips{display:flex;gap:6px;flex-wrap:wrap}
.fchip{position:relative;width:38px;height:38px;border-radius:12px;background:var(--base);box-shadow:var(--o1);
  cursor:pointer;border:0;color:var(--ink2);display:flex;align-items:center;justify-content:center;
  transition:box-shadow .18s,color .18s,transform .18s}
.fchip svg{width:18px;height:18px;pointer-events:none;transition:transform .25s cubic-bezier(.2,.9,.3,1.3),color .18s}
.fchip:hover{box-shadow:var(--o2);transform:translateY(-2px)}
.fchip:hover svg{transform:scale(1.15);color:var(--fc,var(--accent))}
.fchip:active{box-shadow:var(--i1);transform:translateY(0)}
.fchip.on{box-shadow:var(--i1);color:var(--fc,var(--accent));background:color-mix(in srgb,var(--fc,var(--accent)) 14%,var(--
base))}
.fchip.on svg{color:var(--fc,var(--accent))}
.fbadge{position:absolute;top:-3px;right:-3px;min-width:16px;height:16px;font-size:10px;font-weight:800;
  line-height:16px;text-align:center;padding:0 4px;border-radius:8px;color:#fff;
  background:var(--fc,var(--accent));box-shadow:0 1px 3px rgba(0,0,0,.25);pointer-events:none;display:none}
#viewToggle{position:relative;overflow:hidden}
#viewToggle svg{pointer-events:none}
.vico{transition:opacity .2s ease,transform .25s cubic-bezier(.2,.9,.3,1.3);position:absolute}
.vico.hide{opacity:0;transform:rotate(90deg) scale(.5)}
.vico:not(.hide){opacity:1;transform:rotate(0) scale(1)}
```

```
/* Compact tile grid */
#bins.gv{display:grid;grid-template-columns:repeat(auto-fill,minmax(150px,1fr));gap:12px}
#bins.gv .card{display:none}
#bins.gv .tile{display:flex}
#bins:not(.gv) .tile{display:none}
.tile{flex-direction:column;align-items:center;gap:4px;padding:12px 10px 10px;border-radius:16px;
  background:var(--base);box-shadow:var(--o1);cursor:pointer;position:relative;
  transition:box-shadow .18s,transform .15s;min-width:0;text-align:center}
.tile:hover{box-shadow:var(--o2);transform:translateY(-2px)}
.tile[data-bm="sim"]{background:var(--simSoft)}
.tile[data-bm="cal"]{background:var(--calSoft)}
.tile[data-bm="dual"]{background:linear-gradient(135deg,var(--simSoft) 50%,var(--calSoft) 50%)}
.tile .tname{font-size:12px;font-weight:800;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;max-width:120px}
.tile .tplace{font-size:10px;color:var(--ink2);font-weight:600;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;max-
width:120px;margin:-2px 0 2px}
.tile .tfill{font-size:18px;font-weight:800;letter-spacing:-.3px}
.tile .ico.lg{width:44px;height:44px;overflow:hidden}
.tile .tbar{width:80%;height:6px;border-radius:3px;background:var(--well);box-shadow:var(--i1);overflow:hidden;margin:2px 0}
.tile .tbar i{display:block;height:100%;border-radius:3px;transition:width .4s ease}
.tile .tstatus{font-size:9px;font-weight:800;letter-spacing:.4px;color:var(--ink2);padding:2px 8px;border-
radius:10px;background:var(--well);box-shadow:inset -1px -1px 2px var(--shL),inset 1px 1px 2px var(--shD)}
.tile .tstatus.alert-status{color:var(--bad)}
.tile .twarn{position:absolute;top:6px;right:8px;width:22px;height:22px;display:flex;align-items:center;justify-
content:center;
  font-size:12px;font-weight:900;color:#fff;background:var(--bad);border-radius:50%;
  box-shadow:0 0 6px var(--bad), 0 0 12px rgba(217,48,37,.4);border:2px solid var(--base)}
@media(prefers-reduced-motion:no-preference){
  .tile .twarn{animation:warnPulse 1.8s ease-in-out infinite}
}
@keyframes warnPulse{0%,100%{box-shadow:0 0 6px var(--bad),0 0 12px rgba(217,48,37,.4)}50%{box-shadow:0 0 10px var(--bad),0 0
20px rgba(217,48,37,.55),0 0 30px rgba(217,48,37,.2)}}

/* Tile detail overlay */
.tile-overlay{position:fixed;inset:0;z-index:300;display:flex;align-items:center;justify-content:center;
  background:rgba(0,0,0,.45);backdrop-filter:blur(3px)}
.tile-overlay.hide{display:none}
.tile-detail{position:relative;max-width:480px;width:calc(100vw - 32px);
  border-radius:var(--r);box-shadow:var(--o2);background:var(--base);padding:14px 14px 10px}
.tile-detail .card{box-shadow:none;border-radius:14px;overflow:hidden}
.tile-detail .card::before,.tile-detail .card::after{display:none!important}
.tile-detail .sensors{grid-template-columns:repeat(4,1fr)}
@media(max-width:480px){.tile-detail .sensors{grid-template-columns:repeat(2,1fr)}}
.tile-nav{display:flex;gap:8px;justify-content:center;padding:8px 0 4px}
.tile-close{position:absolute;top:-6px;right:10px;z-index:10;width:32px;height:32px;
  background:var(--base);border:0;border-radius:50%;box-shadow:var(--o1);cursor:pointer;
  color:var(--ink2);display:flex;align-items:center;justify-content:center;
  transition:box-shadow .18s,transform .2s cubic-bezier(.2,.9,.3,1.3),color .15s}
.tile-close svg{width:16px;height:16px;pointer-events:none;transition:transform .25s cubic-bezier(.2,.9,.3,1.3)}
.tile-close:hover{box-shadow:var(--i1);color:var(--bad);transform:rotate(90deg)}
.tile-close:active{transform:rotate(90deg) scale(.9)}
.tile-detail .cardtop{padding-right:42px}

/* Settings overlay (SET button) */
.set-overlay{position:fixed;inset:0;z-index:350;display:flex;align-items:center;justify-content:center;
  background:rgba(0,0,0,.45);backdrop-filter:blur(3px)}
.set-overlay.hide{display:none}
.set-panel{position:relative;max-width:400px;width:calc(100vw - 32px);
  border-radius:var(--r);box-shadow:var(--o2);background:var(--base);padding:20px 18px 16px}
.set-panel h3{margin:0 0 14px;font-size:15px;font-weight:800;display:flex;align-items:center;gap:8px}
.set-panel h3 .ico{width:18px;height:18px;color:var(--accent)}
.set-panel .set-fields{display:grid;gap:11px;grid-template-columns:1fr 1fr}
.set-panel .field{grid-column:span 1}
@media(max-width:400px){.set-panel .set-fields{grid-template-columns:1fr}}
.set-panel .set-actions{display:flex;gap:10px;margin-top:16px;justify-content:flex-end}
.set-close{position:absolute;top:10px;right:10px;width:32px;height:32px;
  background:var(--base);border:0;border-radius:50%;box-shadow:var(--o1);cursor:pointer;
  color:var(--ink2);display:flex;align-items:center;justify-content:center;
  transition:box-shadow .18s,transform .2s cubic-bezier(.2,.9,.3,1.3),color .15s}
.set-close svg{width:14px;height:14px;pointer-events:none}
.set-close:hover{box-shadow:var(--i1);color:var(--bad);transform:rotate(90deg)}

/* Collection priority panel */
.collectnext-panel{padding:20px;margin-top:22px}
.collecthead{display:flex;align-items:center;justify-content:space-between;margin-bottom:12px}
.collecthead h2{margin:0}
.crow{display:flex;align-items:center;gap:8px;padding:8px 10px;border-radius:12px;
  background:var(--well);margin-bottom:6px;cursor:pointer;transition:box-shadow .15s}
.crow:hover{box-shadow:var(--o1)}
.crow .cn{font-size:11px;font-weight:800;color:var(--ink2);min-width:18px}
.crow .cnm{flex:1;font-weight:600;font-size:13px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap}
.crow .cpc{font-size:12px;font-variant-numeric:tabular-nums;min-width:34px;text-align:right}
.crow .cwhen{font-size:11px;color:var(--ink2);min-width:52px;text-align:right}
.crow .lvtag{font-size:10px;font-weight:800;letter-spacing:.5px;padding:5px 14px;border-radius:20px;color:#fff;
  min-width:56px;text-align:center;display:inline-block;
  box-shadow:-2px -2px 4px rgba(255,255,255,.18), 2px 2px 4px rgba(0,0,0,.22)}
.coll-more{display:none}
```

```
.coll-show .coll-more{display:flex}
#collectToggle{flex-shrink:0}

/* Alert filter */
.alertctl{flex-wrap:wrap;gap:6px}
#alertFilters{display:flex;gap:5px;flex-wrap:wrap}

/* Adaptive picker */
.pickwrap{position:relative;display:inline-block}
.picklist{position:absolute;top:calc(100% + 6px);left:0;z-index:200;min-width:220px;max-height:320px;
  overflow-y:auto;border-radius:14px;background:var(--base);box-shadow:var(--o2);padding:8px}
.picklist.hide{display:none}
.picklist input{margin-bottom:8px;font-size:13px}
.picklist button{display:flex;width:100%;align-items:center;gap:8px;padding:7px 10px;border-radius:10px;
  background:none;border:0;cursor:pointer;font-size:13px;font-weight:600;text-align:left;color:var(--ink)}
.picklist button:hover,.picklist button[aria-pressed="true"]{background:var(--accentSoft);color:var(--accent)}
.picklist button .bcc{width:9px;height:9px;border-radius:50%;flex-shrink:0}

/* ---- v11 additions ---- */
/* Alert history tools (from Garbage-Guard): kind symbols, date range, scoped actions */
#alertKinds{display:flex;gap:7px;flex-wrap:wrap}
#alertKinds .fchip,#filterChips .fchip{width:36px;height:36px}
#filterChips{gap:7px}
#alertRange{display:flex;gap:6px;flex-wrap:wrap}
.alertscope{margin:14px 2px 12px;font-size:13px;font-weight:800;color:var(--ink)}
.alertscope small{display:block;font-weight:600;color:var(--ink2);margin-top:2px}
.aconfirm{display:flex;align-items:center;gap:10px;flex-wrap:wrap;margin:0 0 14px;padding:12px 14px;border:2px solid var(--
ink);border-radius:8px;box-shadow:4px 4px 0 var(--ink);background:var(--bad);color:#fff;font-weight:700;font-size:13px}
.aconfirm span{flex:1;min-width:200px}
.aconfirm .btn{background:var(--base);color:var(--ink)}
.aconfirm.hide{display:none}
/* Cloud backup panel */
.cloudrows{display:flex;flex-direction:column;gap:8px;margin-top:14px}
.cloudrow{display:flex;align-items:center;gap:10px;font-size:13px;font-weight:700;padding:9px 12px;border-
radius:12px;background:var(--well,var(--base));box-shadow:var(--i1)}
.cloudrow .dot{width:10px;height:10px;border-radius:50%;background:var(--cc);box-shadow:0 0 0 3px color-mix(in srgb,var(--cc)
22%,transparent);flex:none}
.cloudrow .cn{flex:1;min-width:0}
.cloudrow .cs{color:var(--cc);white-space:nowrap;font-size:12px}
.cloudmsg{margin-top:14px;padding:10px 13px;border:2px solid var(--ink);border-radius:8px;box-shadow:3px 3px 0 var(--
ink);color:#fff;font-weight:700;font-size:13px}
.cloudmsg.ok{background:var(--ok)}.cloudmsg.bad{background:var(--bad)}.cloudmsg.hide{display:none}
#cloudState[data-on="1"]{color:var(--c-cloud);box-shadow:var(--o1),0 0 0 2px color-mix(in srgb,var(--c-cloud)
32%,transparent)}
.usebar{height:12px;border-radius:6px;background:var(--well,var(--base));box-shadow:var(--i1);overflow:hidden;margin-
top:14px}
.usebar i{display:block;height:100%;width:3.1%;border-radius:6px;background:var(--c-cloud)}

/* v12: operator PIN */
.pinwrap{position:fixed;inset:0;z-index:120;display:flex;align-items:center;justify-content:center;padding:16px;
  background:color-mix(in srgb,var(--tint) 72%,transparent);backdrop-filter:blur(6px)}
.pinbox{width:100%;max-width:360px;max-height:100%;overflow:auto;padding:26px 22px 20px;text-align:center}
.pinbox h3{margin:8px 0 6px;font-size:18px}
.pinbox p{margin:0;color:var(--ink2);font-size:13px}
.pinico{color:var(--accent);display:grid;place-items:center}
.pinico .ico{width:38px;height:38px}
.pindots{display:flex;justify-content:center;gap:10px;margin:16px 0 6px;min-height:14px}
.pindots i{width:14px;height:14px;border-radius:50%;background:var(--well);box-shadow:var(--i1)}
.pindots i.on{background:var(--accent);box-shadow:none}
.pinbox .pinmsg{min-height:18px;font-weight:700;font-size:12.5px;margin:4px 0 10px}
.pinbox .pinmsg.bad{color:var(--bad)}
.pinbox .pinmsg.ok{color:var(--ok)}
.pinpad{display:grid;grid-template-columns:repeat(3,1fr);gap:10px;margin-bottom:14px}
.pinpad .btn{padding:14px 0;font-size:20px;font-variant-numeric:tabular-nums}
.pinpad .btn.go{font-size:13px;letter-spacing:.6px;color:var(--accent)}
.pinbox.shake{animation:pinshake .32s}
@keyframes pinshake{20%,60%{transform:translateX(-7px)}40%,80%{transform:translateX(7px)}}
@media (prefers-reduced-motion: reduce){.pinbox.shake{animation:none}}
.binscreen{background:#14181C;border-radius:14px;padding:12px 10px;margin:0 0 14px;color:#fff;text-align:center}
.binscreen small{display:block;font-size:10px;letter-spacing:1px;color:#8A96A3;font-weight:700}
.binscreen .pc{display:block;color:#5FD0E8;font-size:11px;letter-spacing:1.8px;font-weight:800;margin-top:4px}
.binscreen b{display:block;font-size:34px;letter-spacing:5px;font-variant-numeric:tabular-nums;line-height:1.25}
.binscreen .exp{font-size:11.5px;letter-spacing:.2px;font-variant-numeric:tabular-nums}
.binscreen .exp.low{color:#FF8A7A}
/* Notices never block a key: taps go through, and any touch clears them. */
.toast{pointer-events:none}
#rolechip{display:inline-flex;align-items:center;gap:6px}
#rolechip .ico{width:15px;height:15px}

/* Fill chart note */
.chart-note{font-size:11px;color:var(--ink2);margin:4px 0 0;font-style:italic}

/* ---------- v2 logo: a bin with a fill wave, a controller chip and a Wi-Fi signal.
   It plays a short build-up (lid opens, the bin fills, the chip lights, the signal
   goes out) and then holds its still frame. */
.sblogo{overflow:visible;display:block}
```

```
.sblogo .sb-ln{stroke:var(--ink)}
.sblogo .sb-ac{stroke:var(--ok)}
.sblogo .sb-chp{stroke:var(--onBrand)}
.sblogo .sb-nd{fill:var(--onBrand)}
.sblogo .sb-led{fill:var(--ok)}
.sblogo .g0{stop-color:var(--ok)}
.sblogo .g1{stop-color:var(--brand2)}
:root{--onBrand:#FFFFFF;--brand2:#0D8A8A}
html[data-theme="dark"]{--onBrand:#13100D;--brand2:#3FC7C2}
.sblogo *{transform-box:view-box}
.sblogo .sb-lid{transform-origin:13px 22.5px}
.sblogo .sb-wave{animation:sbWave 3.2s linear infinite}
.sblogo .sb-wave.b{animation-duration:4.6s;animation-direction:reverse}
.sblogo .sb-sig path{animation:sbPing 3.6s ease-in-out infinite}
.sblogo .sb-sig .s2{animation-delay:.25s}
.sblogo .sb-sig .s3{animation-delay:.5s}
.sblogo .sb-led{animation:sbLed 2.4s ease-in-out infinite}
.sblogo.play .sb-lid{animation:sbLid 1.1s cubic-bezier(.3,.7,.3,1.2) both}
.sblogo.play .sb-fill{animation:sbFill 1.5s cubic-bezier(.2,.8,.2,1) .25s both}
.sblogo.play .sb-chip{transform-origin:32px 46.5px;animation:sbChip .6s cubic-bezier(.2,.9,.3,1.4) 1s both}
.sblogo.play .sb-ticks{animation:sbFade .4s ease 1.2s both}
.sblogo.play .sb-glow{transform-origin:32px 12px;animation:sbGlow 1.2s ease 1.3s both}
.sblogo.play .sb-sig path{animation:sbIn .45s ease both,sbPing 3.6s ease-in-out 2.4s infinite}
.sblogo.play .sb-sig .s1{animation-delay:1.35s,2.4s}
.sblogo.play .sb-sig .s2{animation-delay:1.5s,2.65s}
.sblogo.play .sb-sig .s3{animation-delay:1.65s,2.9s}
@keyframes sbWave{from{transform:translateX(0)}to{transform:translateX(-10px)}}
@keyframes sbPing{0%,70%,100%{opacity:1}35%{opacity:.35}}
@keyframes sbLed{0%,100%{opacity:1}50%{opacity:.35}}
@keyframes sbLid{0%{transform:none}35%{transform:translateY(-3px) rotate(-16deg)}70%{transform:translateY(-1px) rotate(-6deg)}100%{transform:none}}
@keyframes sbFill{from{transform:translateY(17px)}to{transform:none}}
@keyframes sbChip{from{opacity:0;transform:scale(.4)}to{opacity:1;transform:none}}
@keyframes sbFade{from{opacity:0}to{opacity:1}}
@keyframes sbGlow{0%{opacity:0;transform:scale(.3)}60%{opacity:1;transform:scale(1.25)}100%{opacity:1;transform:none}}
@keyframes sbIn{from{opacity:0;transform:translateY(3px)}to{opacity:1;transform:none}}
@media(prefers-reduced-motion:reduce){.sblogo *,.sblogo.play *{animation:none!important}}
#brand{cursor:pointer}
#brand .sblogo{width:40px;height:40px;margin:-8px 0}
</style>
</head>
<body>

<!-- Start page. Shown on every visit; picking a role hides it. Demo mode is
     switched on or off only here (setDemo). -->
<div class="gate" id="gate">
  <div class="gatecard nm">
    <div class="gatehead">
      <div id="gateLogo"></div>
      <div><h1>Smart-Bin Dashboard</h1>
      <p>Pick how you are using the system today. Operator asks for a PIN. Collector is the default.</p></div>
    </div>
    <div class="roles">
      <button class="role" data-role="operator">
        <svg class="ico lg" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" stroke-linecap="round"><circle cx="12" cy="12" r="3.2"/><path d="M12 2.6v3M12 18.4v3M2.6 12h3M18.4 12h3M5.3 5.3l2.1 2.1M16.6 16.6l2.1 2.1M18.7 5.3l-2.1 2.1M7.4 16.6l-2.1 2.1"/></svg>
        <h3>Admin / Operator</h3>
        <ul><li>Charts and history</li><li>Lock, unlock, reset tamper</li><li>Calibration wizard</li><li>Simulation controls</li><li>Add and edit bins</li></ul>
      </button>
      <button class="role" data-role="collector">
        <svg class="ico lg" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="1.8" stroke-linecap="round" stroke-linejoin="round"><path d="M2 7h11v9H2zM13 10h4.5l3.5 3.5V16h-8z"/><circle cx="6.5" cy="18.5" r="1.8"/><circle cx="16.5" cy="18.5" r="1.8"/></svg>
        <h3>Collector</h3>
        <ul><li>Which bin to empty first</li><li>Charts, alerts and the collect PIN</li><li>Start and finish a pickup</li><li>Lock and unlock a bin</li><li>Pair a bin to Wi-Fi over Bluetooth</li><li>Settings are read only</li></ul>
      </button>
    </div>
    <div class="gatedemo">
      <button class="btn" id="gateDemo" aria-pressed="false"></button>
      <p id="gateDemoNote"></p>
    </div>
  </div>
</div>

<!-- Operator PIN dialog. Also reused to show the Bluetooth pairing code. -->
<div class="pinwrap hide" id="pinWrap" role="dialog" aria-modal="true" aria-labelledby="pinTitle">
  <div class="pinbox nm" id="pinBox">
    <div class="binscreen hide" id="pinScreen" aria-live="polite"></div>
    <div class="pinico" id="pinIco"></div>
    <h3 id="pinTitle"></h3>
    <p id="pinText"></p>
    <div class="pindots" id="pinDots"></div>
    <p class="pinmsg" id="pinMsg"></p>
```

```
    <div class="pinpad" id="pinPad"></div>
    <button class="btn sm" id="pinCancel">CANCEL</button>
  </div>
</div>

<!-- Header: logo, mode chip, link chip, role, theme key and the banner pop-up. -->
<header class="top">
  <div class="topin">
    <div class="brand" id="brand" title="Smart-Bin">
      <span id="brandLogo"></span>
      <span class="btxt">SMART-BIN</span>
    </div>
    <button class="statuspill hide" id="statusPill" aria-expanded="false">
      <span id="pillIco"></span><span id="pillText">0</span>
    </button>
    <div class="spacer"></div>
    <div class="modechip" id="modechip"><span id="modeico"></span><span id="modetext">NORMAL</span></div>
    <div class="link" id="link"><span id="wifiico"></span><span id="linktext">Demo data</span></div>
    <button class="btn sm hide" id="demoBtn" title="Demo mode is on. Press to go back to the start page, where demo is turned off.">DEMO</button>
    <button class="btn sm" id="rolechip">COLLECTOR</button>
    <button class="iconbtn" id="sound" title="Sounds on or off" aria-pressed="true"><span id="soundIco"></span></button>
    <button class="iconbtn" id="theme" title="Switch light or dark"><span id="themeIco"></span></button>
  </div>
  <div class="bannerpop hide" id="bannerPop"></div>
</header>

<div class="wrap">
  <div class="tabs" role="tablist" id="tabs"></div>

  <!-- Overview tab: bin cards (or the tile grid in grid view with 3 or more
       bins) and Collection priority. Cards are built by buildCards/renderCard. -->
  <section data-view="overview">
    <div class="sechead">
      <h2 class="sectitle" style="--cc:var(--c-fill)" id="binsTitle"></h2>
      <div class="addrow hide" id="addRow">
        <span class="addnote" id="addNote"></span>
        <div class="addpick">
          <button id="addLess" aria-label="Fewer bins">&minus;</button>
          <span id="addCount">1</span>
          <button id="addMore" aria-label="More bins">+</button>
        </div>
        <button class="addmini" id="addTile"><svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.4" stroke-linecap="round"><path d="M12 5v14M5 12h14"/></svg><span id="addLabel">Add a bin</span></button>
      </div>
    </div>
    <div class="ovtools hide" id="ovTools">
      <div class="filterchips" id="filterChips">
        <button class="fchip on" data-filter="all" title="All bins" aria-label="All bins" style="--fc:var(--accent)" data-ico="layers"><span class="fbadge"></span></button>
        <button class="fchip" data-filter="attn" title="Needs attention" aria-label="Needs attention" style="--fc:var(--c-tamper)" data-ico="tamper"><span class="fbadge"></span></button>
        <button class="fchip" data-filter="sim" title="Simulation active" aria-label="Simulation active" style="--fc:var(--sim)" data-ico="flask"><span class="fbadge"></span></button>
        <button class="fchip" data-filter="cal" title="Calibrating" aria-label="Calibrating" style="--fc:var(--cal)" data-ico="calib"><span class="fbadge"></span></button>
      </div>
      <button class="iconbtn" id="viewToggle" title="Switch to grid view" aria-label="Toggle view">
        <svg class="ico xs vico" id="vicoGrid" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2"><rect x="3" y="3" width="7" height="7" rx="1.4"/><rect x="14" y="3" width="7" height="7" rx="1.4"/><rect x="3" y="14" width="7" height="7" rx="1.4"/><rect x="14" y="14" width="7" height="7" rx="1.4"/></svg>
        <svg class="ico xs vico hide" id="vicoList" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2" stroke-linecap="round"><path d="M8 6h13M8 12h13M8 18h13"/><circle cx="4" cy="6" r="1" fill="currentColor"/><circle cx="4" cy="12" r="1" fill="currentColor"/><circle cx="4" cy="18" r="1" fill="currentColor"/></svg>
      </button>
    </div>
    <div class="scrollpane" id="binsPane"><div class="grid bins" id="bins"></div></div>
    <div class="panel nm collectnext-panel">
      <div class="collecthead">
        <h2 class="ph" style="--cc:var(--c-fill)" id="collectHead"></h2>
        <button class="btn sm" id="collectToggle">SHOW ALL</button>
      </div>
      <div id="collectNext"></div>
      <p class="hint" id="collectHint">Sorted by urgency. Fullest and fastest-filling bins first.</p>
      <div id="prio" style="display:none"></div>
      <div id="ttf" style="display:none"></div>
    </div>
  </section>

  <!-- Analytics tab: fill level and environment charts drawn on <canvas>. -->
  <section data-view="charts" class="hide">
    <div class="chartbox nm">
      <div style="display:flex;justify-content:space-between;align-items:center;gap:14px;flex-wrap:wrap;margin-bottom:16px">
        <h2 class="ph" style="--cc:var(--c-fill);margin:0" id="fillHead"></h2>
        <div class="range" id="range">
          <button class="btn sm" data-range="15">15 m</button>
```

```
          <button class="btn sm" data-range="60" aria-pressed="true">1 h</button>
          <button class="btn sm" data-range="360">6 h</button>
          <button class="btn sm" data-range="1440">24 h</button>
        </div>
      </div>
      <div class="seg" id="fillSeg"></div>
      <canvas id="cFill"></canvas>
      <div class="legend" id="legFill"></div>
    </div>
    <div class="chartbox nm">
      <h2 class="ph" style="--cc:var(--c-temp)" id="envHead"></h2>
      <div class="seg" id="envMode"></div>
      <div class="seg" id="envSeg"></div>
      <canvas id="cEnv"></canvas>
      <div class="legend" id="legEnv"></div>
      <p class="hint" id="envNote"></p>
    </div>
  </section>

  <!-- Settings tab: calibration of one bin, Bluetooth Wi-Fi setup, Firebase,
         operator access, demo options and the sound test. Fields stay disabled
         until an operator turns calibration on (refreshCalGate). -->
  <section data-view="settings" class="hide">
    <div class="panel nm">
      <h2>Settings</h2>
      <div class="seg" id="setSeg"></div>

      <div class="pincard nm-in hide" id="pinCard">
        <span class="pinico" id="pinCardIco"></span>
        <span class="pintxt">
          <b>Collect PIN for <span id="pinName">this bin</span></b>
          <span id="pinSub">Type this on the bin touchscreen when you tap COLLECT.</span>
        </span>
        <span class="pinval" id="pinVal">----</span>
        <button class="btn sm" id="pinEye"></button>
      </div>

      <div class="calbar nm-in hide" id="roNote">
        <span class="calico" id="roIco"></span>
        <span class="caltxt">
          <b>These settings are read only</b>
          <span>You can see every value and every bin, but only the operator can change them or run a calibration.</span>
        </span>
      </div>

      <div class="calbar nm-in" id="calBar">
        <span class="calico" id="calIco"></span>
        <span class="caltxt">
          <b id="calTitle">Settings are locked</b>
          <span id="calSub">Switch on calibration mode to change anything here. The bin shows its teal calibration screen
while it is on.</span>
        </span>
        <label class="switch big" for="calToggle"><input type="checkbox" id="calToggle"><span>Calibration mode</span></label>
      </div>

      <div class="actions" style="margin:16px 0 0"><button class="btn sm tint" id="sectAll" style="flex:0 0 auto;--cc:var(--
accent)"></button></div>
      <fieldset id="setFields" disabled>
        <div class="sect" data-sec="range">
        <h2 style="margin-top:24px">Fill range</h2>
        <div class="fields">
          <div class="field"><label for="s_dempty">Distance empty (cm)</label>
            <div class="withbtn"><input id="s_dempty" type="number" min="1" max="400" step="0.5"><button class="btn sm" data-
capture="s_dempty">USE NOW</button></div></div>
          <div class="field"><label for="s_dfull">Distance full (cm)</label>
            <div class="withbtn"><input id="s_dfull" type="number" min="1" max="400" step="0.5"><button class="btn sm" data-
capture="s_dfull">USE NOW</button></div></div>
        </div>
        <p class="hint">Empty the bin and press USE NOW for the first box. Hold a board at your full line and press it for
the second. Sensor reads <b id="s_dist">-- cm</b> right now.</p>
        </div>

        <div class="sect" data-sec="limits">
        <h2 style="margin-top:24px">Alert limits</h2>
        <div class="fields">
          <div class="field"><label for="s_fill">Fill threshold (%)</label><input id="s_fill" type="number" min="10"
max="100"></div>
          <div class="field"><label for="s_temp">Temperature threshold (&amp;deg;C)</label><input id="s_temp" type="number"
min="10" max="85"></div>
          <div class="field"><label for="s_hum">Humidity threshold (%RH)</label><input id="s_hum" type="number" min="30"
max="100"></div>
          <div class="field"><label for="s_gas">Air quality limit (IAQ)</label><input id="s_gas" type="number" min="50"
max="500"></div>
          <div class="field"><label for="s_tilt">Tilt threshold (deg)</label><input id="s_tilt" type="number" min="5"
max="90"></div>
          <div class="field"><label for="s_wcap">Weight capacity (g)</label><input id="s_wcap" type="number" min="100"
```

```
max="5000" step="50"></div>
        </div>

        </div>
        <div class="sect" data-sec="air">
        <h2 style="margin-top:24px">Air sensor (BME680)</h2>
        <div class="fields">
          <div class="field"><label>Temperature raw / corrected</label><div class="readout nm-in" id="s_tlive">-- / --
&deg;C</div></div>
          <div class="field span2"><label for="s_tref">Reference thermometer (&deg;C)</label>
            <div class="withbtn"><input id="s_tref" type="number" min="-40" max="85" step="0.1" placeholder="e.g.
27.5"><button class="btn sm" id="btnRefT">SET OFFSET</button></div></div>
          <div class="field"><label for="s_toff">Temperature offset (&deg;C)</label><input id="s_toff" type="number" min="-
10" max="10" step="0.1"></div>
          <div class="field"><label>Gas sensor state</label><div class="readout nm-in" id="s_iaq">--</div></div>
          <div class="field"><label for="s_hoff">Humidity offset (%RH)</label><input id="s_hoff" type="number" min="-20"
max="20" step="0.5"></div>
          <div class="field"><label for="s_poff">Pressure offset (hPa)</label><input id="s_poff" type="number" min="-50"
max="50" step="0.1"></div>
          <div class="field span2" style="display:flex;align-items:flex-end"><button class="btn" id="btnGasReset">RESET GAS
BASELINE</button></div>
        </div>
        <p class="hint">The BME680 warms itself, so it reads a little hot and its humidity a little low. Put a reference
thermometer next to the bin for a few minutes, type its reading, press SET OFFSET, then SAVE SETTINGS. Humidity and pressure
offsets are optional. The gas reading has no manual offset: it learns its own baseline in normal air, and RESET GAS BASELINE
restarts that.</p>
        </div>
        <div class="sect" data-sec="lid">
        <h2 style="margin-top:24px">Lid and lock</h2>
        <div class="fields">
          <div class="field"><label for="s_sclosed">Servo closed (deg)</label><input id="s_sclosed" type="number" min="0"
max="180"></div>
          <div class="field"><label for="s_sopen">Servo open (deg)</label><input id="s_sopen" type="number" min="0"
max="180"></div>
          <div class="field"><label for="s_pin">Collect PIN</label><input id="s_pin" type="number" min="0" max="9999"></div>
          <div class="field"><label for="s_pirhold">Lid stays open (s)</label><input id="s_pirhold" type="number" min="1"
max="60" step="1"></div>
          <div class="field"><label for="s_buzz">Buzzer minimum (s)</label><input id="s_buzz" type="number" min="1" max="30"
step="1"></div>
          <div class="field" style="display:flex;align-items:flex-end"><label class="switch"><input id="s_autolock"
type="checkbox">Lock the lid when the bin is full</label></div>
        </div>

        </div>
        <div class="sect" data-sec="weight">
        <h2 style="margin-top:24px">Weight</h2>
        <div class="fields">
          <div class="field"><label>Reading now</label><div class="readout nm-in" id="s_live">0.00 kg</div></div>
          <div class="field"><label for="s_known">Known weight (g)</label><input id="s_known" type="number" value="1000"
min="10" max="5000"></div>
          <div class="field" style="display:flex;align-items:flex-end;gap:10px">
            <button class="btn" id="btnTare">TARE / ZERO</button>
            <button class="btn" id="btnCalW">CALIBRATE</button>
          </div>
        </div>
        </div>
      </fieldset>

      <div class="sect" data-sec="touch">
      <h2 style="margin-top:24px">Touchscreen</h2>
      <div class="fields">
        <div class="field span2"><label>Touch calibration</label><div class="readout nm-in" id="s_touch">--</div></div>
        <div class="field span2" style="display:flex;align-items:flex-end;gap:10px">
          <button class="btn" id="btnTouchCal">CALIBRATE TOUCH</button>
          <button class="btn" id="btnTouchReset">RESET TOUCH</button>
        </div>
      </div>
      <p class="hint">If presses land in the wrong place, press CALIBRATE TOUCH. The bin shows four crosses one after
another: press each one firmly, then let go. The result is saved on the bin straight away. It can also be started on the bin
itself by holding a finger anywhere on the screen for 5 seconds. Needs the bin&rsquo;s Wi-Fi.</p>
      </div>

      <div class="actions" style="margin-top:26px">
        <button class="btn" id="btnSave" style="flex:0 0 auto" disabled>SAVE SETTINGS</button>
        <button class="btn" id="btnCancel" style="flex:0 0 auto" disabled>CANCEL CHANGES</button>
        <button class="btn" id="btnReload" style="flex:0 0 auto">Reload from bin</button>
        <button class="btn" id="btnExport" style="flex:0 0 auto">Export bin list</button>
      </div>
      <p class="hint" id="setMsg">Values are read-only until calibration mode is on. Saving sends them to the bin and they
survive a restart.</p>
    </div>

    <div class="panel nm" id="btPanel" style="margin-top:22px">
      <div class="sechead">
        <h2 class="ph" style="--cc:var(--c-bt)" id="btHead"></h2>
        <span class="btstate" id="btState">Not paired</span>
```

```
      </div>
      <p class="hint" style="margin:-6px 0 8px">Gives a bin a new Wi-Fi name and password over Bluetooth. The bin keeps its current Wi-Fi until the new one works, and your laptop can stay on Wi-Fi too.</p>
      <ol class="hint" style="margin:0 0 16px;padding-left:20px;line-height:1.6">
        <li><b>On the bin:</b> tap <b>i</b> (top left), then <b>WI-FI SETUP</b>, and type the <b>Collect PIN</b> (the same 4 digits as for COLLECT).</li>
        <li><b>Here:</b> press <b>PAIR A BIN</b> and pick <b>SmartBin-01</b> (or 02) in the list the browser shows.</li>
        <li>The bin shows a <b>6-digit pairing code</b>. Windows or your phone opens its own pairing window: type the code <b>there</b>, not on this page.</li>
        <li>Pick the network, type its password, press <b>SEND WI-FI</b>. The bin shows its new IP address, and so does this page.</li>
        <li>Press <b>TURN BT OFF</b> when you are done.</li>
      </ol>
      <div class="btsteps" id="btSteps"></div>
      <div class="fields" style="margin-top:18px">
        <div class="field"><label for="bt_ssid">Wi-Fi name (2.4 GHz)</label>
          <div class="withbtn"><input id="bt_ssid" type="text" list="bt_nets" spellcheck="false" placeholder="HomeWiFi-2G" disabled>
          <button class="btn sm" id="btScan" disabled></button></div>
          <datalist id="bt_nets"></datalist>
          <div class="btnets" id="btNetList"></div></div>
        <div class="field"><label for="bt_pass">Password</label>
          <div class="withbtn"><input id="bt_pass" type="password" spellcheck="false" disabled>
          <button class="btn sm" id="btEye" disabled></button></div></div>
      </div>
      <div class="actions" style="margin-top:20px">
        <button class="btn" id="btPair"></button>
        <button class="btn" id="btSend" disabled></button>
        <button class="btn" id="btOff" disabled></button>
      </div>
      <div class="btresult hide" id="btResult"></div>
      <p class="hint" id="btMsg"></p>
      <p class="hint" id="btWhy"></p>
      <p class="hint" id="btRoleNote"></p>
    </div>

    <div class="panel nm" id="cloudPanel" style="margin-top:22px">
      <div class="sechead">
        <h2 class="ph" style="--cc:var(--c-cloud)" id="cloudHead"></h2>
        <span class="btstate" id="cloudState">Not tested</span>
      </div>
      <p class="hint" style="margin:-6px 0 16px">The bins copy their readings to Firebase so the phone app keeps working away from this Wi-Fi. Local Wi-Fi always comes first: this dashboard and the app both use the bins' own API whenever a bin answers, and the app falls back to Firebase only when none does.</p>
      <div class="fields">
        <div class="field span2"><label for="fb_url">Database URL</label><input id="fb_url" type="text" inputmode="url" spellcheck="false" placeholder="https://...firebasedatabase.app"></div>
        <div class="field"><label for="fb_key">Web API key</label><input id="fb_key" type="text" spellcheck="false" placeholder="AIza..."></div>
        <div class="field"><label for="fb_email">Account email</label><input id="fb_email" type="text" inputmode="email" spellcheck="false"></div>
        <div class="field"><label for="fb_pass">Password</label>
          <div class="withbtn"><input id="fb_pass" type="password" spellcheck="false"><button class="btn sm" id="fbEye"></button></div></div>
      </div>
      <div class="actions" style="margin-top:20px">
        <button class="btn" id="fbTest" style="flex:0 0 auto"></button>
        <button class="btn" id="fbSave" style="flex:0 0 auto"></button>
        <button class="btn" id="fbRestore" style="flex:0 0 auto"></button>
        <button class="btn" id="fbSend" style="flex:0 0 auto"></button>
      </div>
      <div class="fields" style="margin-top:16px">
        <div class="field span2"><label for="ts_server">ThingSpeak server</label><input id="ts_server" type="text" inputmode="url" spellcheck="false" placeholder="https://api.thingspeak.com"></div>
      </div>
      <div class="cloudmsg hide" id="cloudMsg"></div>
      <div class="cloudrows" id="cloudRows"></div>
      <div class="usebar"><i id="useFill" style="width:0%"></i></div>
      <p class="hint" id="cloudUse">Each bin writes every 5 s while a phone or this dashboard is watching over the cloud, every minute otherwise, and at once when an alert starts. LOCK, UNLOCK and COLLECT sent through the cloud reach the bin within a few seconds. The data meter appears once the bins report it.</p>
      <p class="hint" id="cloudRoleNote"></p>
    </div>

    <div class="panel nm" id="opPanel" style="margin-top:22px">
      <div class="sechead">
        <h2 class="ph" style="--cc:var(--sim)">Operator access</h2>
        <span class="btstate" id="opState"></span>
      </div>
      <p class="hint" style="margin:-6px 0 16px">The dashboard opens on the start page every time, the same as the phone app. Operator needs the PIN, and drops back to collector after 2 min with this tab hidden. After 5 wrong PINs it waits 30 s, longer each time. Only an operator can change the PIN, and only by typing the current one first.</p>
      <div class="actions">
        <button class="btn" id="opChange" style="flex:0 0 auto">CHANGE OPERATOR PIN</button>
        <button class="btn" id="opLockNow" style="flex:0 0 auto">BACK TO COLLECTOR</button>
        <button class="btn" id="opStart" style="flex:0 0 auto">START PAGE</button>
```

```
      </div>
      <p class="hint" id="opNote"></p>
    </div>

    <div class="panel nm hide" id="demoPanel" style="margin-top:22px">
      <div class="sechead"><h2 class="ph" style="--cc:var(--sim)" id="demoHead"></h2><span class="btstate" id="demoState"></span></div>
      <p class="hint" style="margin:-6px 0 14px">Act out what simulation cannot, on the demo bins only, the same as the app's demo options. Any choice here pauses the automatic tour for a minute. RESET puts it all back.</p>
      <div id="demoBody"></div>
    </div>

    <div class="panel nm" id="sndPanel" style="margin-top:22px">
      <div class="sechead"><h2 class="ph" style="--cc:var(--accent)" id="sndHead"></h2></div>
      <p class="hint" style="margin:-6px 0 16px">Press a key to hear that sound. The phone app has the same sounds under the same names, and vibrates with each one. This test plays even with sounds turned off.</p>
      <div id="sndGroups"></div>
    </div>
  </section>
  <!-- Simulation tab: built by buildSim from the SENSORS list. -->
  <section data-view="sim" class="hide">
    <div class="panel nm">
      <h2>Simulation</h2>
      <div class="seg" id="simSeg"></div>
      <p class="hint" style="margin:-6px 0 16px">AUTO reads the real hardware and lets the firmware drive the outputs. MANUAL holds the value you set, one row at a time.</p>
      <table class="sim"><thead><tr><th id="thSensor"></th><th style="width:132px" id="thMode"></th><th id="thValue"></th></tr></thead><tbody id="simBody"></tbody></table>
      <div class="actions" style="margin-top:20px">
        <button class="btn accent" id="btnSim" style="flex:0 0 auto">APPLY</button>
        <button class="btn" id="btnSimOff" style="flex:0 0 auto">All back to AUTO</button>
      </div>
      <p class="hint" id="simMsg">The whole dashboard turns violet while any sensor is manual.</p>
    </div>
  </section>

  <!-- Alerts tab: alert history with kind and date filters. -->
  <section data-view="alerts" class="hide">
    <div class="panel nm">
      <div class="sechead">
        <h2 class="ph" style="--cc:var(--c-tamper)" id="alertHead"></h2>
        <div class="alertctl">
          <div class="filterchips" id="alertKinds"></div>
          <div id="alertRange"></div>
          <button class="btn sm" id="btnMarkAll" disabled></button>
          <button class="btn sm" id="btnClear"></button>
        </div>
      </div>
      <p class="alertscope" id="alertScope"></p>
      <div class="aconfirm hide" id="alertConfirm"><span id="alertConfirmText"></span><button class="btn sm" id="alertNo">CANCEL</button><button class="btn sm" id="alertYes">DELETE</button></div>
      <div class="scrollpane alertpane"><div class="alerts" id="alertList"></div></div>
    </div>
  </section>
</div>

<script>
// ============================================================================
// Script. Everything runs in the page; there is no build step.
// Main loop: setInterval(poll, 1000) near the end. poll() reads every bin by
// the first route that answers and then calls render(). Most renderers keep a
// "signature" string (cardSig, alertSig, banSig ...) and skip the DOM update
// when it has not changed, which is what stops the page from flickering. If
// something on screen does not update, its signature is the first suspect.
// ============================================================================
// Bin colors: evenly spread hues; after 12 bins the hue shifts by 37 degrees
// and the lightness changes, so every bin stays distinct.
const HUES=[153,287,47,207,100,260,20,180,313,73,233,127];
function binColor(i){
  const dark=document.documentElement.getAttribute('data-theme')==='dark';
  const round=Math.floor(i/HUES.length);
  const h=(HUES[i%HUES.length]+round*37)%360;
  const s=50,l=dark?(round===0?64:round===1?55:72):(round===0?46:round===1?38:54);
  return 'hsl('+h+','+s+'%,'+l+'%)';
}
// Default calibration, the same keys and values as the firmware's Cal struct.
const DEF_CAL={d_empty:30,d_full:5,fill_th:80,temp_th:40,hum_th:85,gas_th:200,tilt_th:30,weight_cap:4500,auto_lock:0,servo_closed:0,servo_open:90,collect_pin:1234,pir_hold_s:5,buzz_s:2,temp_off:-2,hum_off:0,press_off:0};
// Simulation rows. bit is the bit in the firmware's Sim mask (0 fill .. 6 lid).
const SENSORS=[
  {key:'fill',bit:0,label:'Fill level',icon:'fill',kind:'range',min:0,max:100,step:1,unit:'%'},
  {key:'weight',bit:1,label:'Weight',icon:'weight',kind:'number',min:0,max:5000,step:10,unit:'g'},
  {key:'gas',bit:2,label:'Air quality',icon:'gas',kind:'range',min:0,max:500,step:5,unit:'IAQ'},
  {key:'temp',bit:3,label:'Temperature and humidity',icon:'temp',kind:'pair'},
  {key:'tamper',bit:4,label:'Tamper',icon:'tamper',kind:'toggle'},
```

```
  {key:'motion',bit:5,label:'Motion',icon:'motion',kind:'toggle'},
  {key:'lid',bit:6,label:'Lid',icon:'lid',kind:'lid'}
];
const ENV_MODES=[
  ['both','ALL','layers','var(--ink2)'],
  ['temp','TEMPERATURE','temp','var(--c-temp)'],
  ['humid','HUMIDITY','humid','var(--c-humid)'],
  ['gas','AIR QUALITY','gas','var(--c-gas)'],
  ['press','PRESSURE','press','var(--c-press)']
];
/* BME680 datasheet ranges. A single-reading chart runs to the top of its range.
   Pressure is charted only: it has no tile on the bin card and no alert. */
const SPEC={
  temp:{min:0,max:85,ticks:[0,20,40,60,85],unit:'C',name:'Temperature',dec:1,col:'--c-temp'},
  humid:{min:0,max:100,ticks:[0,25,50,75,100],unit:'%',name:'Humidity',dec:0,col:'--c-humid'},
  press:{min:300,max:1100,ticks:[300,500,700,900,1100],unit:'hPa',name:'Pressure',dec:1,col:'--c-press'},
  gas:{min:0,max:500,ticks:[0,100,200,300,400,500],unit:'IAQ',name:'Air quality',dec:0,col:'--c-gas'}
};
const TABS=[
  {v:'overview',t:'Overview',i:'grid',c:'var(--c-fill)'},
  {v:'charts',t:'Charts',i:'chart',c:'var(--c-humid)'},
  {v:'settings',t:'Settings',i:'sliders',c:'var(--cal)'},
  {v:'sim',t:'Simulation',i:'flask',c:'var(--sim)'},
  {v:'alerts',t:'Alerts',i:'bell',c:'var(--c-tamper)'}
];

/* ---------- icons ---------- */
const S=(b,c)=>'<svg class="ico '+(c||'')+'" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">'+b+'</svg>';
const ICONS={
  grid:()=>S('<rect x="3" y="3" width="7.5" height="7.5" rx="1.6"/><rect x="13.5" y="3" width="7.5" height="7.5" rx="1.6"/><rect x="3" y="13.5" width="7.5" height="7.5" rx="1.6"/><rect x="13.5" y="13.5" width="7.5" height="7.5" rx="1.6"/>'),
  chart:()=>S('<path d="M3 20h18"/><path d="M5 15l4.5-5 3.5 3L19 6"/>'),
  sliders:()=>S('<path d="M4 7h10M18 7h2M4 17h4M12 17h8"/><circle cx="16" cy="7" r="2.2"/><circle cx="10" cy="17" r="2.2"/>'),
  flask:()=>S('<path d="M9.5 3v6.2L4.6 18a2 2 0 0 0 1.7 3h11.4a2 2 0 0 0 1.7-3L14.5 9.2V3"/><path d="M8 3h8M7.4 15h9.2"/>'),
  bell:()=>S('<path d="M18 9a6 6 0 1 0-12 0c0 6-2 7-2 7h16s-2-1-2-7"/><path d="M10.4 20.5a2 2 0 0 0 3.2 0"/>'),
  plus:()=>S('<path d="M12 5v14M5 12h14"/>'),
  gear:()=>S('<circle cx="12" cy="12" r="3"/><path d="M12 2.8v2.6M12 18.6v2.6M2.8 12h2.6M18.6 12h2.6M5.5 5.5l1.8 1.8M16.7 16.7l1.8 1.8M18.5 5.5l-1.8 1.8M7.3 16.7l-1.8 1.8"/>','xs'),
  truck:()=>S('<path d="M2 7h11v9H2zM13 10h4.5l3.5 3.5V16h-8z"/><circle cx="6.5" cy="18.5" r="1.8"/><circle cx="16.5" cy="18.5" r="1.8"/>','xs'),
  check:()=>S('<path d="M4 12.5l5 5L20 6.5"/>','xs'),
  temp:()=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round">'+
    '<circle class="halo" cx="12" cy="17.2" r="5.4" fill="currentColor" stroke="none"/>'+
    '<path d="M14.2 14.4V5.2a2.2 2.2 0 1 0-4.4 0v9.2a4.6 4.6 0 1 0 4.4 0z"/>'+
    '<rect class="merc" x="10.6" y="9" width="2.8" height="6" rx="1.4" fill="currentColor" stroke="none"/>'+
    '<circle cx="12" cy="17.2" r="2.6" fill="currentColor" stroke="none"/></svg>',
  humid:()=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linejoin="round">'+
    '<path d="M12 3.2s6.2 6.6 6.2 10.3a6.2 6.2 0 1 1-12.4 0C5.8 9.8 12 3.2 12 3.2z"/>'+
    '<circle class="pulse" cx="12" cy="14.4" r="3" fill="currentColor" stroke="none" opacity=".45"/></svg>',
  gas:()=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round">'+
    '<path class="vap" d="M6.5 17c0-2.6 2.6-2.6 2.6-5.2S6.5 9.2 6.5 6.6"/>'+
    '<path class="vap v2" d="M12 18c0-2.6 2.6-2.6 2.6-5.2S12 10.2 12 7.6"/>'+
    '<path class="vap v3" d="M17.5 17c0-2.6 2.6-2.6 2.6-5.2s-2.6-2.6-2.6-5.2"/></svg>',
  cloud:()=>S('<path d="M7 18.6h10.4a4.2 4.2 0 0 0 .5-8.4 6 6 0 0 0-11.5-1A4.8 4.8 0 0 0 7 18.6z"/>'),
  press:()=>S('<path d="M4.6 17.6a8.6 8.6 0 1 1 14.8 0"/><path d="M12 14l3.6-4.4"/><circle cx="12" cy="14.2" r="1.5" fill="currentColor" stroke="none"/><path d="M7.2 17.6h9.6"/>'),
  weight:()=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">'+
    '<path d="M4.6 20.5h14.8l-1.9-9.6H6.5z"/><path d="M9 10.9a3 3 0 1 1 6 0"/>'+
    '<path class="needle" d="M12 17.6v-3.1" stroke-width="2.2" style="transform-origin:12px 17.6px"/></svg>',
  lid:(open)=>'<svg class="ico '+(open?'open':'')+'" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">'+
    '<path d="M6.6 9.5l1.1 11h8.6l1.1-11"/><g class="lidflap"><path d="M4.5 9.5h15"/><path d="M9.6 7.1h4.8"/></g></svg>',
  motion:(on)=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round">'+
    '<circle cx="5.5" cy="12" r="1.8" fill="currentColor" stroke="none"/>'+
    '<g class="'+(on?'arcs':'')+'" opacity="'+(on?'1':'.4')+'"><path d="M10 7.5a7 7 0 0 1 0 9"/><path d="M13.6 5a11 11 0 0 1 0 14"/><path d="M17.2 2.6a15 15 0 0 1 0 18.8"/></g></svg>',
  lock:(locked)=>'<svg class="ico '+(locked?'locked':'unlocked')+'" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round">'+
    '<rect x="4.5" y="10.5" width="15" height="10.5" rx="2.4"/><path class="shackle" d="M8 10.5V7.4a4 4 0 0 1 8 0v3.1"/>'+
    '<circle cx="12" cy="15.7" r="1.5" fill="currentColor" stroke="none"/></svg>',
  tamper:()=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round">'+
    '<g class="shake" style="transform-origin:12px 13px"><path d="M12 3.4L22 20H2z"/><path d="M12 9.6v4.2M12 17h.01"/></g></svg>',
  calib:()=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round">'+
    '<circle class="ring" cx="12" cy="12" r="8.4" stroke-dasharray="7 6"/><circle cx="12" cy="12" r="2.4" fill="currentColor" stroke="none"/>'+
    '<path d="M12 1.6v2.4M12 20v2.4M1.6 12h2.4M20 12h2.4"/></svg>',
  sim:()=>ICONS.flask(),
```

```
  fill:()=>S('<path d="M5.4 4.4h13.2l-1.3 15.2a2 2 0 0 1-2 1.8H8.7a2 2 0 0 1-2-1.8z"/><path d="M6.9 12.6c1.6-1.1 3.3-1.1 4.9 0s3.3 1.1 4.9 0"/>'),
  bins:()=>S('<path d="M3.2 7.6h7l-.8 11.8H4zM13.8 7.6h7l-.8 11.8h-5.4z"/><path d="M2.4 7.6h8.6M13 7.6h8.6M5.4 5h2.6M16 5h2.6"/>'),
  clock:()=>S('<circle cx="12" cy="12" r="8.4"/><path d="M12 7.2V12l3.2 2"/>'),
  trash:()=>S('<path d="M4.4 6.8h15.2M9.6 6.8V4.6h4.8v2.2M6.8 6.8l1 12.6h8.4l1-12.6M10.2 10.4v5.4M13.8 10.4v5.4"/>'),
  stopwatch:()=>S('<circle cx="12" cy="13.6" r="7.4"/><path d="M12 9.8v3.8l2.6 1.6M9.6 2.8h4.8M19.4 7.2l1.4-1.4"/>'),
  palette:()=>S('<path d="M12 3.4a8.6 8.6 0 1 0 0 17.2c1.2 0 1.9-.8 1.9-1.8 0-1-.9-1.5-.9-2.4s.7-1.6 1.7-1.6h1.3a4.6 4.6 0 0 0 4.6-4.6c0-3.8-3.9-6.8-8.6-6.8z"/><circle cx="7.8" cy="11.6" r="1.1" fill="currentColor" stroke="none"/><circle cx="10.6" cy="7.8" r="1.1" fill="currentColor" stroke="none"/><circle cx="15" cy="8.4" r="1.1" fill="currentColor" stroke="none"/>'),
  speaker:()=>S('<path d="M4.6 9.4h3.4l4.2-3.6v12.4L8 14.6H4.6z"/><path d="M15.6 9.4a3.6 3.6 0 0 1 0 5.2M18 6.8a7 7 0 0 1 0 10.4"/>'),
  layers:()=>S('<path d="M12 3.4l8.6 4.6L12 12.6 3.4 8z"/><path d="M3.4 12.4L12 17l8.6-4.6M3.4 16.6L12 21.2l8.6-4.6"/>'),
  rank:()=>S('<path d="M9.6 6.6h10.8M9.6 12h10.8M9.6 17.4h10.8"/><path d="M4.8 5v3.6M3.6 5h1.2M3.4 10.4h2.8M3.4 15.4h2.6v1.6h-2.6v1.8h2.6"/>'),
  eye:()=>S('<path d="M2.4 12s3.6-6.4 9.6-6.4S21.6 12 21.6 12s-3.6 6.4-9.6 6.4S2.4 12 2.4 12z"/><circle cx="12" cy="12" r="2.8"/>'),
  sunmoon:(dark)=>'<svg class="ico sunmoon '+(dark?"is-dark":"is-light")+'" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round">'+
    '<g class="sunside"><circle cx="8.2" cy="12" r="3"/><path d="M8.2 5.4v1.6M8.2 17v1.6M3.4 7.2l1.2 1.2M11.8 15.6l1.2 1.2M3.4 16.8l1.2-1.2M1.2 12h1.6"/></g>'+
    '<g class="moonside"><path d="M19.4 7.6a5.4 5.4 0 1 0 0 8.8 6.2 6.2 0 0 1 0-8.8z"/></g></svg>',
  bluetooth:()=>S('<path d="M7.4 7.4l9.2 9.2L12 20.6V3.4l4.6 4L7.4 16.6"/>'),
  radar:()=>S('<path d="M12 12l5.6-5.6"/><circle cx="12" cy="12" r="1.6" fill="currentColor" stroke="none"/><path d="M8.4 15.6a5.1 5.1 0 0 1 0-7.2M5.6 18.4a9 9 0 0 1 0-12.8"/><path d="M15.6 8.4a5.1 5.1 0 0 1 0 7.2"/>'),
  key:()=>S('<circle cx="8.2" cy="12" r="3.6"/><path d="M11.8 12h9v3M17.6 12v2.6"/>'),
  eyeoff:()=>S('<path d="M4 4l16 16"/><path d="M9.6 9.7a2.8 2.8 0 0 0 3.9 3.9"/><path d="M6.4 6.8C3.9 8.5 2.4 12 2.4 12s3.6 6.4 9.6 6.4c1.6 0 3-.4 4.2-1M18.3 15.1c1.9-1.6 3.3-3.1 3.3-3.1S18 5.6 12 5.6c-.8 0-1.5.1-2.2.3"/>'),
  ruler:()=>S('<rect x="2.6" y="8" width="18.8" height="8" rx="1.8"/><path d="M7 8v3M11 8v4M15 8v3M19 8v4"/>'),
  percent:()=>S('<path d="M18.5 5.5l-13 13"/><circle cx="7.6" cy="7.6" r="2.3"/><circle cx="16.4" cy="16.4" r="2.3"/>'),
  save:()=>S('<path d="M4.5 4.5h11l4 4v11h-15z"/><path d="M8 4.5v5h6v-5M8 19.5v-5.5h8v5.5"/>'),
  reload:()=>S('<path d="M20.5 12a8.5 8.5 0 0 1-2.6-6.1"/><path d="M20.8 4.4v4.8h-4.8"/>'),
  download:()=>S('<path d="M12 3.6v11.2M7.6 10.6l4.4 4.4 4.4-4.4"/><path d="M4.4 19.4h15.2"/>'),
  undo:()=>S('<path d="M3.6 12a8.4 8.4 0 1 0 2.6-6.1"/><path d="M3.3 4.4v4.8h4.8"/>'),
  capture:()=>S('<circle cx="12" cy="12" r="7.6"/><circle cx="12" cy="12" r="1.8" fill="currentColor" stroke="none"/><path d="M12 1.8v2.8M12 19.4v2.8M1.8 12h2.8M19.4 12h2.8"/>'),
  pin:()=>S('<rect x="3.4" y="4.6" width="17.2" height="14.8" rx="2.2"/><path d="M7.4 9.4h.01M12 9.4h.01M16.6 9.4h.01M7.4 14.6h9.2"/>'),
  tilt:()=>S('<path d="M3.4 20.2h17.2"/><path d="M6.6 20.2L15 6.4"/><path d="M11.4 20.2a7.5 7.5 0 0 0-1.6-4.4"/>'),
  servo:()=>S('<circle cx="12" cy="13.6" r="3.2"/><path d="M12 10.4V4.6"/><path d="M4.6 13.6a7.4 7.4 0 0 1 3.2-6.1M19.4 13.6a7.4 7.4 0 0 0-3.2-6.1"/>'),
  cross:()=>S('<path d="M6.4 6.4l11.2 11.2M17.6 6.4L6.4 17.6"/>'),
  power:()=>S('<path d="M12 3.4v8.2"/><path d="M6.8 6.6a7.6 7.6 0 1 0 10.4 0"/>'),
  wifi:(live)=>'<svg class="ico xs wifi '+(live?'live':'')+'" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2" stroke-linecap="round">'+
    '<path d="M8.5 15.2a5 5 0 0 1 7 0"/><path d="M5.5 11.8a9.4 9.4 0 0 1 13 0"/><path d="M2.4 8.4a14 14 0 0 1 19.2 0"/>'+
    '<circle cx="12" cy="18.6" r="1.4" fill="currentColor" stroke="none" class="glowdot"/></svg>',
  bin:(id,pct,col,prefix)=>{
    const cid=(prefix||'clip')+'-'+id;
    const y=(42-(pct/100)*30).toFixed(1);
    return '<svg class="ico lg" viewBox="0 0 48 48" style="color:'+col+'">'+
    '<defs><clipPath id="'+cid+'"><path d="M11.5 14h25l-2.6 26.2a2.4 2.4 0 0 1-2.4 2.1H16.5a2.4 2.4 0 0 1-2.4-2.1z"/></clipPath></defs>'+
    '<g clip-path="url(#'+cid+')"><g class="wavewrap" transform="translate(0,'+y+')">'+
    '<path class="wave" d="M-24 4 q6 -5 12 0 t12 0 t12 0 t12 0 t12 0 t12 0 v36 h-72z" fill="currentColor" opacity=".85"/>'+
    '<path class="wave" d="M-24 7 q6 -5 12 0 t12 0 t12 0 t12 0 t12 0 t12 0 v36 h-72z" fill="currentColor" opacity=".35" style="animation-duration:5s"/>'+
    '</g></g>'+
    '<path d="M11.5 14h25l-2.6 26.2a2.4 2.4 0 0 1-2.4 2.1H16.5a2.4 2.4 0 0 1-2.4-2.1z" fill="none" stroke="currentColor" stroke-width="2.6"/>'+
    '<path d="M8 13h32M19 9.4h10" fill="none" stroke="currentColor" stroke-width="2.6" stroke-linecap="round"/></svg>';
  }
};
const icon=(n,a)=>(ICONS[n]?ICONS[n](a):'');

/* ---------- state ---------- */
// The bin list is saved in localStorage, so it survives a reload. Only bins
// with an address (IP, ThingSpeak channel or Firebase name) count as real.
const store={
  read(){try{return JSON.parse(localStorage.getItem('smartbin.bins')||'null')}catch(e){return null}},
  write(v){try{localStorage.setItem('smartbin.bins',JSON.stringify(v))}catch(e){}}
};
function newBin(seed){
  return {id:seed.id,name:seed.name,place:seed.place,ip:seed.ip||'',online:false,live:false,
    cloud:seed.cloud||'',tsCh:seed.tsCh||'',tsKey:seed.tsKey||'',tbId:seed.tbId||'',tbKey:seed.tbKey||'',
    route:null,seen:{local:0,fb:0,ts:0},lastAt:0,ipHint:'',

data:{fill:seed.fill||20,distance_cm:20,weight_g:800,temp:28,temp_raw:30,humidity:62,pressure:1008,gas:70,iaq_acc:3,tilt:1,lid:'CLOSED',status:'OK',motion:false},
    cal:Object.assign({},DEF_CAL),
    sim:{mask:0,values:{fill:60,weight:2000,gas:120,temp:28,humidity:60,tamper:0,motion:0,lid:0,rgb:1,buzzer:0}},
    hist:[],prev:{}};
}
const saved=store.read();
const DEMO_SEEDS=[
  {id:'demo1',name:'Demo Bin 1',place:'Main Hall',fill:34},
```

```
  {id:'demo2',name:'Demo Bin 2',place:'Cafeteria',fill:62},
  {id:'demo3',name:'Demo Bin 3',place:'Library',fill:88}
];
const mkDemo=s=>Object.assign(newBin(s),{demo:true});
const fbSaved=(()=>{try{const v=JSON.parse(localStorage.getItem('smartbin.fb')||'null');return !!(v&&v.url)}catch(e){return
false}})();
const isRealSeed=s=>!!(s&&(s.ip||(s.tsCh&&s.tsKey)||(s.cloud&&fbSaved)));
const realStart=(saved||[]).filter(isRealSeed).map(newBin);
let demoPref=null;try{demoPref=localStorage.getItem('smartbin.demo')}catch(e){}
const demoStart=demoPref===null?realStart.length===0:demoPref==='1';
// Everything the page draws comes from this object; render() reads it.
const state={
  role:'collector',view:'overview',rangeMin:60,alerts:[],calib:null,envBin:null,addCount:1,chartPick:[],envMode:'both',
  ovView:'list',ovFilter:'all',collectShowAll:false,akind:'all',arange:'any',ovTile:null,
  bins:demoStart?realStart.concat(DEMO_SEEDS.map(mkDemo)):realStart,
  demo:demoStart,selSet:null,selSim:null
};
state.selSet=state.selSim=state.envBin=state.bins[0]?state.bins[0].id:null;
var TOUR={i:0,next:0,pauseUntil:0,msg:'',busy:false};

const $=s=>document.querySelector(s);
const $$=s=>Array.from(document.querySelectorAll(s));
const byId=id=>state.bins.find(b=>b.id===id);
const clamp=(v,a,b)=>Math.max(a,Math.min(b,v));
const fmt=(v,d=1)=>Number.isFinite(v)?v.toFixed(d):'--';
const colorOf=i=>binColor(i);
function persist(){
  state.bins.forEach(b=>{if(b.demo&&(b.ip||(b.tsCh&&b.tsKey)))b.demo=false;});
  store.write(state.bins.filter(b=>!b.demo).map(b=>({id:b.id,name:b.name,place:b.place,ip:b.ip,fill:b.data.fill,
  cloud:b.cloud,tsCh:b.tsCh,tsKey:b.tsKey,tbId:b.tbId,tbKey:b.tbKey})))}

function seedHistory(b,i){
  const now=Date.now(),pts=[];
  let f=Math.max(2,b.data.fill-24-i*8);
  for(let m=1440;m>=0;m-=4){
    f=Math.min(99,f+Math.random()*0.075);
    pts.push({t:now-m*60000,fill:+f.toFixed(1),
      temp:+(b.data.temp-2+Math.sin(m/90)*1.5+Math.random()*.4).toFixed(1),
      humidity:+clamp(b.data.humidity+Math.sin(m/140)*7+Math.random()*1.4,30,95).toFixed(1),
      pressure:+(1008+Math.sin(m/260)*4+Math.cos(m/90)*0.8+Math.random()*0.25).toFixed(1),
      gas:Math.round(clamp(40+f*1.6+Math.sin(m/120)*9+Math.random()*6,10,500))});
  }
  b.hist=pts;
}

const LABEL_ICONS={
  s_dempty:['ruler','var(--cal)'],      s_dfull:['fill','var(--c-fill)'],
  s_fill:['percent','var(--c-fill)'],   s_temp:['temp','var(--c-temp)'],   s_hum:['humid','var(--c-humid)'],
  s_gas:['gas','var(--c-gas)'],         s_tilt:['tilt','var(--c-tamper)'],
  s_wcap:['weight','var(--c-weight)'],  s_sclosed:['servo','var(--c-lid)'],
  s_sopen:['servo','var(--c-lid)'],     s_pin:['pin','var(--sim)'],
  s_known:['weight','var(--c-weight)'], s_pirhold:['stopwatch','var(--c-motion)'],
  s_buzz:['speaker','var(--warn)'],
  s_toff:['temp','var(--c-temp)'],      s_tref:['temp','var(--c-temp)'],
  s_hoff:['humid','var(--c-humid)'],    s_poff:['press','var(--c-press)']
};
const BTN_STYLES={
  btnSave:['save','tint','var(--ok)','SAVE SETTINGS'],
  btnCancel:['undo','tint','var(--warn)','CANCEL CHANGES'],
  btnReload:['reload','tint','var(--c-humid)','RELOAD FROM BIN'],
  btnExport:['download','tint','var(--c-weight)','EXPORT BIN LIST'],
  btnTare:['capture','tint','var(--c-weight)','TARE / ZERO'],
  btnCalW:['weight','tint','var(--cal)','CALIBRATE'],
  btnRefT:['capture','tint','var(--c-temp)','SET OFFSET'],
  btnGasReset:['reload','tint','var(--c-gas)','RESET GAS BASELINE'],
  btnSim:['flask','solid','var(--sim)','APPLY'],
  btnSimOff:['reload','tint','var(--ok)','ALL BACK TO AUTO']
};
function decorateStatics(){
  for(const id in LABEL_ICONS){
    const inp=document.getElementById(id);if(!inp)continue;
    const lab=inp.closest('.field')&&inp.closest('.field').querySelector('label');
    if(!lab||lab.dataset.deco)continue;
    lab.dataset.deco='1';
    lab.style.setProperty('--lc',LABEL_ICONS[id][1]);
    lab.insertAdjacentHTML('afterbegin',icon(LABEL_ICONS[id][0]));
  }
  for(const id in BTN_STYLES){
    const el=document.getElementById(id);if(!el||el.dataset.deco)continue;
    const [ic,cls,cc,txt]=BTN_STYLES[id];
    el.dataset.deco='1';
    el.classList.remove('accent');el.classList.add(cls);
    el.style.setProperty('--cc',cc);
    el.innerHTML=icon(ic)+'<span>'+txt+'</span>';
    el.style.display='inline-flex';el.style.alignItems='center';el.style.gap='9px';
  }
```

```
  $$('[data-capture]').forEach(b=>{
    if(b.dataset.deco)return;b.dataset.deco='1';
    b.classList.add('tint');b.style.setProperty('--cc','var(--cal)');
    b.innerHTML=icon('capture')+'<span>USE NOW</span>';
    b.style.display='inline-flex';b.style.alignItems='center';b.style.gap='7px';
  });
  const al=document.getElementById('s_autolock');
  if(al&&!al.parentElement.dataset.deco){
    al.parentElement.dataset.deco='1';
    al.parentElement.style.setProperty('--lc','var(--sim)');
  }
}

function paintHeadings(){
  const H={binsTitle:['bins','BINS'],collectHead:['rank','COLLECTION PRIORITY'],alertHead:['clock','ALERT HISTORY'],
    fillHead:['fill','FILL LEVEL'],envHead:['temp','ENVIRONMENT']};
  for(const id in H){const el=document.getElementById(id);if(el)el.innerHTML=icon(H[id][0])+'<span>'+H[id][1]+'</span>';}
  const bc=$('#btnClear');
  if(bc&&!bc.dataset.deco){bc.dataset.deco='1';bc.classList.add('tint');
    bc.style.setProperty('--cc','var(--c-tamper)');
    bc.innerHTML=icon('trash')+'<span>DELETE</span>';
    bc.style.display='inline-flex';bc.style.alignItems='center';bc.style.gap='8px';}
  const bm=$('#btnMarkAll');
  if(bm&&!bm.dataset.deco){bm.dataset.deco='1';bm.classList.add('tint');
    bm.style.setProperty('--cc','var(--ok)');
    bm.innerHTML=icon('check')+'<span>MARK READ</span>';
    bm.style.display='inline-flex';bm.style.alignItems='center';bm.style.gap='8px';}
  const TH={thSensor:['sliders','SENSOR','var(--c-humid)'],thMode:['power','MODE','var(--sim)'],
    thValue:['capture','MANUAL VALUE','var(--cal)']};
  for(const id in TH){const el=document.getElementById(id);
    if(el)el.innerHTML='<span class="thh" style="--cc:'+TH[id][2]+'">'+icon(TH[id][0])+TH[id][1]+'</span>';}
  $$('#filterChips .fchip[data-
ico]').forEach(c=>{if(!c.querySelector('svg'))c.insertAdjacentHTML('afterbegin',icon(c.dataset.ico));});
  const SEC={range:'ruler',limits:'bell',lid:'servo',weight:'weight',air:'gas',touch:'capture'};
  /* Each group folds, closed at first like the app, so the one you need is a short scroll away.
     The summary on the right says what is inside without opening it. */
  $$('.sect').forEach(d=>{
    const h=d.querySelector('h2');if(!h||h.dataset.deco)return;h.dataset.deco='1';
    h.removeAttribute('style');
    const body=document.createElement('div');body.className='sbody';
    while(h.nextSibling)body.appendChild(h.nextSibling);
    d.appendChild(body);
    /* A div, not a button: the groups sit in a fieldset that is disabled until calibration is on. */
    const
btn=document.createElement('div');btn.setAttribute('role','button');btn.tabIndex=0;btn.className='shead';btn.setAttribute('ar
ia-expanded','false');
    btn.innerHTML=icon(SEC[d.dataset.sec]||'sliders')+'<span class="stitle">'+h.textContent+'</span><span
class="ssum"></span>'+
      '<svg class="ico schev" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2" stroke-linecap="round"
stroke-linejoin="round"><path d="M6 9l6 6 6-6"/></svg>';
    h.replaceWith(btn);
  });
  const subs={btnTare:'SETS 0 KG POINT',btnCalW:'SETS KG PER STEP'};
  for(const id in subs){const el=document.getElementById(id);
    if(el&&!el.querySelector('.sub'))el.querySelector('span').insertAdjacentHTML('beforeend','<em
class="sub">'+subs[id]+'</em>');}
  updateThemeIcon();
}
function updateThemeIcon(){
  const dark=document.documentElement.getAttribute('data-theme')==='dark';
  $('#themeIco').innerHTML=ICONS.sunmoon(dark);
}

// Web Bluetooth UUIDs of the bin's setup service (BT_*_UUID in config.h).
// Web Bluetooth works only in Chrome or Edge, and only on https:// or
// localhost pages, so it is unavailable from a bin's http:// page (btWhy
// below tells the user to use the phone app or a local http.server).
const BT={
  SVC:'b1a70001-5c2e-4f8a-9d33-0a9c2b6f17c1',
  SSID:'b1a70002-5c2e-4f8a-9d33-0a9c2b6f17c1',
  PASS:'b1a70003-5c2e-4f8a-9d33-0a9c2b6f17c1',
  CTRL:'b1a70004-5c2e-4f8a-9d33-0a9c2b6f17c1',
  STAT:'b1a70005-5c2e-4f8a-9d33-0a9c2b6f17c1',
  SCAN:'b1a70006-5c2e-4f8a-9d33-0a9c2b6f17c1'
};
const BT_STEPS=[['bluetooth','PAIR + CODE'],['radar','PICK NETWORK'],['key','SEND'],['power','BLUETOOTH OFF']];
const bt={dev:null,ch:{},step:0,demo:false,ip:'',name:'',busy:false,off:false,pollT:null,waitT:null};
function btRender(){
  $('#btSteps').innerHTML=BT_STEPS.map((s,i)=>
    '<div class="btstep
'+(bt.step>i?'done':bt.step===i?'on':'')+'">'+icon(bt.step>i?'check':s[0])+'<b>'+s[1]+'</b></div>').join('');
  const paired=bt.step>0,op=state.role==='operator';
  ['bt_ssid','bt_pass'].forEach(id=>{const e=$('#'+id);if(e)e.disabled=!paired});
  ['btScan','btEye'].forEach(id=>{const e=$('#'+id);if(e)e.disabled=!paired});
  $('#btSend').disabled=!paired;
  $('#btOff').disabled=bt.step<3;
```

```
  const note=$('#btRoleNote');
  if(note)note.textContent='';
  if($('#cloudHead')&&$('#cloudHead').innerHTML)cloudRender();
  const st=$('#btState');
  st.dataset.on=paired?'1':'0';
  st.innerHTML=icon('bluetooth')+'<span>'+(bt.demo&&paired?'Paired (demo)':paired?'Paired':'Not paired')+'</span>';
}
/* ---------- cloud backup (Firebase), from Garbage-Guard: local Wi-Fi first, Firebase second ---------- */
const FB_DEF={url:'',key:'',email:'',pass:''};
const cloud={working:Object.assign({},FB_DEF),tested:false,msg:null,show:false};
try{const v=JSON.parse(localStorage.getItem('smartbin.fb')||'null');if(v&&v.url){cloud.working=Object.assign({},FB_DEF,v);cloud.tested=true;}}catch(e){}
function fbVals(){return {url:$('#fb_url').value.trim(),key:$('#fb_key').value.trim(),email:$('#fb_email').value.trim(),pass:$('#fb_pass').value}}
function fbFill(v){$('#fb_url').value=v.url;$('#fb_key').value=v.key;$('#fb_email').value=v.email;$('#fb_pass').value=v.pass}
function fbCheck(f){
  const mock=/^http:\/\/[^/]+$/.test(f.url);
  if(!mock&&!/^https:\/\/.+\.(firebasedatabase\.app|firebaseio\.com)$/.test(f.url))return 'The database URL is wrong. It starts with https:// and ends in .firebasedatabase.app, with no / at the end.';
  if(!mock&&!/^AIza.{35}$/.test(f.key))return 'The Web API key is wrong. It starts with AIza and is 39 characters long.';
  if(!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(f.email)||f.pass.length<6)return 'Wrong email or password.';
  return '';
}
function cloudRender(){
  const op=state.role==='operator';
  ['fb_url','fb_key','fb_email','fb_pass','fbSave','fbRestore','fbSend','ts_server','btnTouchCal','btnTouchReset'].forEach(id=>{const e=$('#'+id);if(e)e.disabled=!op});
  const st=$('#cloudState');st.dataset.on=cloud.tested?'1':'0';
  st.innerHTML=icon('cloud')+'<span>'+(cloud.tested?'Working':'Not tested')+'</span>';
  const m=$('#cloudMsg');m.classList.toggle('hide',!cloud.msg);if(cloud.msg){m.className='cloudmsg '+cloud.msg[0];m.textContent=cloud.msg[1];}
  $('#cloudRows').innerHTML=state.bins.slice(0,6).map(b=>{const s=cloudRowText(b);
    return '<div class="cloudrow" style="--cc:'+s[0]+'"><span class="dot"></span><span class="cn">'+esc(b.name)+'</span><span class="cs">'+s[1]+'</span></div>'}).join('')+
    (state.bins.length>6?'<div class="cloudrow" style="--cc:var(--ink2)"><span class="cn">and '+(state.bins.length-6)+' more</span></div>':'')+
    (fbReady()?'<div class="cloudrow" style="--cc:var(--c-cloud)"><span class="dot"></span><span class="cn">This dashboard</span><span class="cs">'+(FB.watching?'tells the bins it is watching, so they write every 5 s':'reads the bins over Wi-Fi, so it is not asking the cloud')+'</span></div>':'');
  $('#cloudRoleNote').textContent=op?'':'You are signed in as a collector. You can test the details, but only an operator can change or save them.';
}
function cloudDecorate(){
  const defs={fbTest:['radar','tint','var(--c-cloud)','TEST'],fbSave:['save','solid','var(--c-cloud)','TEST AND SAVE'],fbRestore:['undo','tint','var(--ink2)','RESTORE WORKING'],fbEye:['eye','tint','var(--ink2)','SHOW']};
  for(const id in defs){const el=$('#'+id);if(!el)continue;const [ic,cls,cc,tx]=defs[id];
    el.classList.add(cls);el.style.setProperty('--cc',cc);el.innerHTML=icon(ic)+'<span>'+tx+'</span>';
    el.style.display='inline-flex';el.style.alignItems='center';el.style.gap='8px';}
  $('#cloudHead').innerHTML=icon('cloud')+'<span>CLOUD BACKUP (FIREBASE)</span>';
  fbFill(cloud.working);cloudRender();

$('#fbEye').addEventListener('click',()=>{cloud.show=!cloud.show;$('#fb_pass').type=cloud.show?'text':'password';$('#fbEye').innerHTML=icon(cloud.show?'eyeoff':'eye')+'<span>'+(cloud.show?'HIDE':'SHOW')+'</span>';});
  const test=async save=>{const v=fbVals(),e=fbCheck(v);
    if(e){cloud.msg=['bad',e+(save?' Nothing was saved.':'')];cloudRender();return;}
    cloud.msg=['ok','Signing in\u2026'];cloudRender();
    try{
      const n=await fbTest(v);
      cloud.tested=true;
      if(save){cloud.working=Object.assign({},v);fbUse(v);
        try{localStorage.setItem('smartbin.fb',JSON.stringify(v))}catch(err){}
        cloud.msg=['ok','Signed in and found '+n+' bin'+(n===1?'':'s')+' in the database. Saved on this computer. Press SEND TO BINS to give the bins this account too.'];
        state.bins.forEach(b=>{if(!b.hist.length&&isLive(b))b.hist=[]});sfx('saved');}
      else cloud.msg=['ok','Signed in as '+v.email+' and found '+n+' bin'+(n===1?'':'s')+'. Nothing was saved yet.'];
    }catch(err){cloud.msg=['bad',(err.message||String(err))+(save?' Nothing was saved.':'')];sfx('denied');}
    cloudRender();};
  $('#fbTest').addEventListener('click',()=>test(false));
  $('#fbSave').addEventListener('click',()=>test(true));
  $('#fbRestore').addEventListener('click',()=>{fbFill(cloud.working);cloud.msg=['ok','Put back the last details that passed a test.'];cloudRender();});
  const send=$('#fbSend');
  if(send){send.classList.add('tint');send.style.setProperty('--cc','var(--c-cloud)');
    send.innerHTML=icon('bins')+'<span>SEND TO BINS</span>';send.style.display='inline-flex';send.style.alignItems='center';send.style.gap='8px';
    send.addEventListener('click',sendCloudToBins);}
  const tsIn=$('#ts_server');

if(tsIn){tsIn.value=TS.server;tsIn.addEventListener('change',()=>{TS.server=(tsIn.value.trim()||'https://api.thingspeak.com').replace(/\/+$/,'');
    tsIn.value=TS.server;try{localStorage.setItem('smartbin.ts',TS.server)}catch(e){}toast('ThingSpeak server set to '+TS.server+'.');});}
}
function btDecorate(){
```

```
  const defs={btPair:['bluetooth','tint','var(--c-bt)','PAIR A BIN'],btSend:['key','solid','var(--c-bt)','SEND WI-FI'],
    btOff:['power','tint','var(--warn)','TURN BT OFF'],btScan:['radar','tint','var(--c-bt)','SCAN'],
    btEye:['eye','tint','var(--ink2)','SHOW']};
  for(const id in defs){const el=$('#'+id);if(!el)continue;
    const [ic,cls,cc,tx]=defs[id];
    el.classList.add(cls);el.style.setProperty('--cc',cc);
    el.innerHTML=icon(ic)+'<span>'+tx+'</span>';
    el.style.display='inline-flex';el.style.alignItems='center';el.style.gap='8px';}
  $('#btHead').innerHTML=icon('bluetooth')+'<span>BLUETOOTH SETUP</span>';
  $('#btWhy').textContent=btSupported()
    ?'Web Bluetooth is ready in this browser.'
    :'This page cannot use Bluetooth: browsers allow it only on secure pages, and a bin serves the dashboard over plain http. Use the Smart-Bin phone app, or on a laptop run "python -m http.server" in the Web Dashboard folder and open http://localhost:8000 in Chrome or Edge. Demo bins still walk through the steps.';
  btRender();
}
function btSupported(){return typeof navigator!=='undefined'&&!!navigator.bluetooth}
const btWait=ms=>new Promise(r=>setTimeout(r,ms));
const btDec=v=>new TextDecoder().decode(v);
function btMsg(t){$('#btMsg').textContent=t}
const btDevs=new WeakSet();
/* Connect and find the five characteristics. A laptop or phone that remembers an old
   pairing often drops the first try, so it tries 3 times. */
// Bluetooth Wi-Fi setup: connect, read SCAN, write SSID and PASS, write 1 to
// CTRL, then follow STAT until CONNECTED or FAILED (see ble_setup.cpp).
async function btConnect(){
  for(let i=0;i<3;i++){
    try{
      if(i)btMsg('The bin dropped the link, trying again ('+(i+1)+' of 3)...');
      const srv=bt.dev.gatt.connected?bt.dev.gatt:await bt.dev.gatt.connect();
      await btWait(400);
      const svc=await srv.getPrimaryService(BT.SVC);
      bt.ch={};
      for(const k of ['SSID','PASS','CTRL','STAT','SCAN']){try{bt.ch[k]=await svc.getCharacteristic(BT[k]);}catch(e){}}
      if(bt.ch.STAT&&bt.ch.SSID)return true;
    }catch(e){try{bt.dev.gatt.disconnect()}catch(_){}}
    await btWait(1200);
  }
  return false;
}
/* Every characteristic is encrypted, so the first read makes Windows or the phone ask for
   the code the bin shows. That can take as long as the person needs to type it, so the
   read is tried again while the code (60 s) is still good. */
async function btSecure(){
  const end=Date.now()+70000;let last=null;
  while(Date.now()<end){
    try{return btDec(await bt.ch.STAT.readValue());}
    catch(e){last=e;
      if(!bt.dev.gatt.connected&&!(await btConnect()))throw e;
      await btWait(1500);}
  }
  throw last||new Error('No code was typed in time');
}
function btWatch(dev){
  if(btDevs.has(dev))return;
  btDevs.add(dev);
  dev.addEventListener('gattserverdisconnected',btLost);
}
function btLost(){
  if(bt.demo||bt.busy)return;
  if(bt.step===2){btMsg('The Bluetooth link dropped while the bin joins the Wi-Fi. Reconnecting to read the result...');btRejoin();return;}
  if(bt.step===1){bt.step=0;btRender();sfx('bin_lost');
    btMsg('The bin closed the Bluetooth link. On the bin, open WI-FI SETUP again, then press PAIR A BIN.');}
}
async function btRejoin(){
  if(await btConnect()){try{await bt.ch.STAT.startNotifications();bt.ch.STAT.addEventListener('characteristicvaluechanged',btOnNotify);}catch(e){}btReadStat();
}
}
/* A notification is cut to 20 bytes on a small link, so a reply that does not parse is read in full. */
function btOnNotify(e){
  let j=null;try{j=JSON.parse(btDec(e.target.value))}catch(err){}
  if(j)btStatus(j);else btReadStat();
}
async function btReadStat(){
  if(!bt.ch.STAT)return;
  try{btStatus(JSON.parse(btDec(await bt.ch.STAT.readValue())));}catch(e){}
}
function btStop(){clearInterval(bt.pollT);clearTimeout(bt.waitT);bt.pollT=bt.waitT=null;}
async function btPair(){
  const b=byId(state.selSet);
  if(bt.busy)return;
  btStop();
  if(!btSupported()){
    if(b&&b.demo){bt.demo=true;btCodeStart();return;}
```

```
    sfx('denied');
    btMsg(location.protocol==='http:'&&location.hostname!=='localhost'
      ?'This page came from the bin over plain http, and browsers only allow Bluetooth on secure pages. Use the Smart-Bin phone app, or open the dashboard file on a laptop as the note above says. Demo bins can still walk through the steps.'
      :'This browser has no Web Bluetooth. Use Chrome or Edge as the note above says, or the Smart-Bin phone app. Demo bins can still walk through the steps.');
    return;
  }
  bt.demo=false;bt.busy=true;bt.step=0;bt.ip='';btRender();
  $('#btResult').classList.add('hide');
  try{
    btMsg('Pick SmartBin-01 or SmartBin-02 in the browser list.');
    bt.dev=await navigator.bluetooth.requestDevice({filters:[{services:[BT.SVC]},{namePrefix:'SmartBin'}],optionalServices:[BT.SVC]});
    bt.name=bt.dev.name||'the bin';
    btWatch(bt.dev);
    if(!(await btConnect())){
      sfx('bt_wrong');
      btMsg(bt.name+' keeps dropping the Bluetooth link. Windows (or the phone) most likely remembers it from an earlier pairing. '+
        'Open Settings > Bluetooth & devices, remove '+bt.name+', then press PAIR A BIN again. On the bin, cancel and open WI-FI SETUP again first.');
      return;
    }
    btMsg('Connected to '+bt.name+'. The bin now shows a 6-digit code. Type it into the pairing window Windows or your phone opens, not on this page. The code lasts 60 s.');
    await btSecure();
    try{await bt.ch.STAT.startNotifications();bt.ch.STAT.addEventListener('characteristicvaluechanged',btOnNotify);}catch(e){}
    sfx('bt_paired');
    bt.step=1;btRender();
    btMsg('Paired with '+bt.name+'. The link is encrypted now. Pick the Wi-Fi network next.');
    bt.busy=false;
    btScan();
  }catch(e){
    if(e&&e.name==='NotFoundError')btMsg('No bin was picked. Press PAIR A BIN when the bin shows WI-FI SETUP.');
    else{
      sfx('bt_wrong');
      btMsg('Pairing did not finish ('+(e&&e.message||e)+'). If the code was wrong, the bin shows a new one; after 3 wrong codes it waits 60 s. '+
        'If it keeps failing, remove '+(bt.name||'the bin')+' in Settings > Bluetooth & devices and press PAIR A BIN again.');
    }
    try{bt.dev&&bt.dev.gatt.disconnect()}catch(_){}
    bt.step=0;btRender();
  }finally{bt.busy=false;}
}
function btShowNets(nets){
  const list=nets.map(n=>typeof n==='string'?{ssid:n}:n).filter(n=>n&&n.ssid);
  $('#bt_nets').innerHTML=list.map(n=>'<option value="'+esc(n.ssid)+'">').join('');
  const cur=$('#bt_ssid').value.trim();
  $('#btNetList').innerHTML=list.map(n=>'<button type="button" class="btn sm" data-ssid="'+esc(n.ssid)+'" aria-pressed="'+(n.ssid===cur)+'">'+
    icon('wifi',true)+esc(n.ssid)+(Number.isFinite(+n.rssi)&&n.rssi!==undefined?' <small>'+n.rssi+' dBm</small>':'')+'</button>').join('');
}
function btFakeNets(){
  btShowNets([{ssid:'HomeWiFi-2G',rssi:-48},{ssid:'TUP-Manila',rssi:-61},{ssid:'Smart_Bin_Lab',rssi:-70},{ssid:'PLDTHOMEFIBR24',rssi:-79}]);
}
/* The bin scans when WI-FI SETUP opens. An empty list means it is still scanning, so try again after 5 s. */
async function btScan(){
  const tail=' Tap one to fill in the name, or type it yourself.';
  sfx('tap');
  if(bt.demo){btFakeNets();btMsg('Demo networks listed.'+tail);return;}
  if(!bt.ch.SCAN){btMsg('This bin does not share a network list, so type the name yourself.');return;}
  for(let i=0;i<3;i++){
    try{
      const nets=JSON.parse(btDec(await bt.ch.SCAN.readValue()));
      if(nets.length){btShowNets(nets);btMsg('Found '+nets.length+' network'+(nets.length===1?'':'s')+' (strongest first). Only 2.4 GHz ones work.'+tail);return;}
      btMsg('The bin is still scanning for networks. Trying again in 5 s...');
    }catch(e){btMsg('Could not read the network list ('+(e.message||e)+'). Trying again in 5 s...');}
    await btWait(5000);
    if(bt.step!==1)return;
  }
  btMsg('The bin found no networks. Check that the router is on 2.4 GHz and close by, press SCAN again, or type the name yourself.');
}
function btStatus(j){
  if(bt.step!==2)return;
  const st=String(j.state||'').toUpperCase();
  if(st==='CONNECTED'&&j.ip){btStop();bt.ip=j.ip;bt.step=3;btRender();btShowResult(j.ssid);}
  else if(st==='FAILED'){btStop();bt.step=1;btRender();sfx('wifi_failed');
    btMsg('The bin could not join: '+(j.why||'it gave no reason')+'. It went back to its old Wi-Fi. Check the name and password, then press SEND WI-FI again.');}
```

```
  else if(st==='CONNECTING')btMsg('The bin is joining '+($('#bt_ssid').value.trim()||'the network')+'. This takes up to 15 s.');
}
async function btSend(){
  const ssid=$('#bt_ssid').value.trim(),pass=$('#bt_pass').value;
  if(!ssid){sfx('denied');btMsg('Type the Wi-Fi name first.');return;}
  if(ssid.length>32||pass.length>63){sfx('denied');btMsg('The Wi-Fi name can be 32 characters at most and the password 63.');return;}
  if(pass&&pass.length<8){sfx('denied');btMsg('A Wi-Fi password has at least 8 characters. Leave it empty only for an open network.');return;}
  sfx('confirm');
  bt.step=2;btRender();$('#btResult').classList.add('hide');
  btMsg('Sending the Wi-Fi name and password and waiting for the bin to join...');
  if(bt.demo){
    const b=byId(state.selSet);
    bt.waitT=setTimeout(()=>{
      if(/fail|wrong/i.test(ssid)){btStatus({state:'FAILED',why:'Network not found'});return;}
      btStatus({state:'CONNECTED',ip:'192.168.1.'+(101+Math.max(0,state.bins.indexOf(b))),ssid});},1800);
    return;
  }
  try{
    if(!bt.dev.gatt.connected&&!(await btConnect()))throw new Error('the bin is out of reach');
    const enc=new TextEncoder();
    await bt.ch.SSID.writeValue(enc.encode(ssid));
    await bt.ch.PASS.writeValue(enc.encode(pass));
    await bt.ch.CTRL.writeValue(Uint8Array.of(1));
  }catch(e){sfx('cmd_fail');btMsg('Could not send: '+(e.message||e)+'. Press SEND WI-FI again.');bt.step=1;btRender();return;}
  /* Notifications can be missed, so read the status every 2 s too. The bin gives up after 15 s. */
  bt.pollT=setInterval(()=>{if(bt.step===2&&bt.dev.gatt.connected)btReadStat();},2000);
  bt.waitT=setTimeout(async()=>{
    if(bt.step!==2)return;
    await btReadStat();
    if(bt.step!==2)return;
    btStop();bt.step=1;btRender();sfx('wifi_failed');
    btMsg('No answer from the bin after 25 s. Check the touchscreen: it shows whether it joined. Then press SEND WI-FI again or PAIR A BIN.');
  },25000);
}
function btShowResult(ssid){
  const b=byId(state.selSet),real=b&&!b.demo&&!bt.demo;
  sfx('wifi_joined');
  const num=(/(\d+)\s*$/.exec(bt.name||'')||[])[1];
  const label=bt.demo?(b?'USE FOR '+esc(b.name):'USE'):real?'USE FOR '+esc(b.name):'ADD AS BIN '+(num||'');
  const r=$('#btResult');r.classList.remove('hide');
  r.innerHTML=icon('wifi',true)+'<span>Joined '+esc(ssid||'the network')+'. Address: <span class="ip">'+esc(bt.ip)+'</span></span>'+
    '<button class="btn sm solid" style="--cc:var(--ok)" id="btUse">'+icon('check')+'<span>'+label.trim()+'</span></button>';
  btMsg('Press '+label.trim()+' to store this address, then TURN BT OFF.');
}
function btUse(){
  if(!bt.ip)return;
  let b=byId(state.selSet);
  if(!bt.demo&&(!b||b.demo)){
    if(state.role!=='operator'&&state.bins.some(x=>!x.demo)){toast('Pick the bin in Settings first, or ask an operator to add it.');sfx('denied');return;}
    const num=(/(\d+)\s*$/.exec(bt.name||'')||[])[1]||String(state.bins.filter(x=>!x.demo).length+1).padStart(2,'0');
    b=state.bins.find(x=>!x.demo&&x.ip===bt.ip);
    if(!b){b=newBin({id:'bin'+num+'-'+Date.now().toString(36),name:'Bin '+num,place:'',ip:bt.ip,cloud:'bin'+num});state.bins.unshift(b);}
    state.selSet=b.id;
  }
  if(b.demo){toast(b.name+' is a demo bin, so nothing was stored.');return;}
  b.ip=bt.ip;relink(b);persist();cardSig='';render();loadSettings();sfx('saved');
  toast(b.name+' will now be read at '+bt.ip+'.');
}
async function btOff(){
  btStop();
  bt.off=true;
  if(!bt.demo&&bt.ch.CTRL){try{await bt.ch.CTRL.writeValue(Uint8Array.of(2));}catch(e){}}
  try{if(bt.dev&&bt.dev.gatt&&bt.dev.gatt.connected)bt.dev.gatt.disconnect();}catch(e){}
  bt.step=4;btRender();bt.off=false;
  sfx('bt_off');
  btMsg('Bluetooth is off on the bin. It stays on its Wi-Fi until someone opens WI-FI SETUP again.');
}
const reduce=()=>window.matchMedia('(prefers-reduced-motion: reduce)').matches;
function setAddCount(n){
  state.addCount=clamp(n,1,6);
  const c=$('#addCount');if(!c)return;
  c.textContent=state.addCount;
  $('#addLabel').textContent=state.addCount===1?'Add a bin':'Add '+state.addCount+' bins';
  $('#addLess').disabled=state.addCount<=1;
  $('#addMore').disabled=state.addCount>=6;
}
function tweenPct(el,b,to){
  const paint=v=>{el.innerHTML=Math.round(v)+'<span>%</span>'};
```

```
  const from=Number.isFinite(b._shown)?b._shown:0;
  if(reduce()||(el.dataset.init&&Math.abs(to-from)<0.6)){b._shown=to;paint(to);return;}
  el.dataset.init='1';
  cancelAnimationFrame(b._raf||0);
  const t0=performance.now(),dur=520;
  const run=now=>{
    const k=Math.min(1,(now-t0)/dur),e=1-Math.pow(1-k,3),v=from+(to-from)*e;
    b._shown=v;paint(v);
    if(k<1)b._raf=requestAnimationFrame(run);
  };
  b._raf=requestAnimationFrame(run);
}
function levelColor(p,c){return p>=(c.fill_th||80)?'var(--bad)':p>=50?'var(--warn)':'var(--ok)'}
function statusColor(st,p,c){
  if(st==='CALIBRATING')return 'var(--cal)';
  if(st==='TAMPER')return 'var(--bad)';
  if(st==='LOCKED')return 'var(--sim)';
  if(st==='COLLECTING')return 'var(--cal)';
  return levelColor(p,c);
}

/* ---------- transport ---------- */
// The bin's base URL. An IP without http:// gets it added.
function binBase(b){const ip=String(b.ip||'').trim().replace(/\/+$/,'');return /^https?:\/\//.test(ip)?ip:'http://'+ip}
// Every local call has a 1.5 s timeout so a dead bin cannot stall the loop.
// callRaw returns the HTTP status (for 400/409 reasons); call() throws on
// any non-2xx reply.
async function callRaw(b,path,body){
  if(!b.ip)throw new Error('no ip');
  const ctrl=new AbortController(),timer=setTimeout(()=>ctrl.abort(),1500);
  try{
    const res=await fetch(binBase(b)+path,{method:body?'POST':'GET',
      headers:body?{'Content-Type':'application/json'}:undefined,
      body:body?JSON.stringify(body):undefined,signal:ctrl.signal,cache:'no-store'});
    let j=null;try{j=await res.json()}catch(e){}
    return {ok:res.ok&&(!j||j.ok!==false),status:res.status,j};
  }finally{clearTimeout(timer)}
}
async function call(b,path,body){
  if(!b.ip)throw new Error('no ip');
  const ctrl=new AbortController(),timer=setTimeout(()=>ctrl.abort(),1500);
  try{
    const res=await fetch(binBase(b)+path,{method:body?'POST':'GET',
      headers:body?{'Content-Type':'application/json'}:undefined,
      body:body?JSON.stringify(body):undefined,signal:ctrl.signal,cache:'no-store'});
    if(!res.ok)throw new Error('HTTP '+res.status);
    return await res.json();
  }finally{clearTimeout(timer)}
}
// Copies a status JSON from any route into b.data with safe defaults, so the
// renderers never see undefined values.
function normalise(b,j){
  const d=b.data,n=(a,f)=>Number.isFinite(+a)?+a:f;
  d.fill=clamp(n(j.fill??j.fill_percentage,d.fill),0,100);
  d.distance_cm=n(j.distance_cm??j.distance,d.distance_cm);
  d.weight_g=n(j.weight_g??j.weight_grams??j.weight,d.weight_g);
  d.temp=n(j.temp??j.temperature,d.temp);
  d.temp_raw=n(j.temp_raw,d.temp-(b.cal.temp_off||0));
  d.humidity=n(j.humidity,d.humidity);
  d.pressure=n(j.pressure??j.pressure_hpa,d.pressure);
  d.gas=n(j.iaq??j.gas??j.gas_level,d.gas);
  d.iaq_acc=n(j.iaq_acc,d.iaq_acc);
  d.gas_k=n(j.gas_k,d.gas_k);
  d.warmEnd=Number.isFinite(+j.iaq_warm_s)&&j.iaq_warm_s!==null?Date.now()+j.iaq_warm_s*1000:null;
  d.tilt=n(j.tilt??j.tilt_deg,d.tilt);
  if(j.motion!==undefined||j.pir!==undefined)d.motion=!!(j.motion??j.pir);

if(j.lid!==undefined||j.lid_state!==undefined)d.lid=(j.lid??j.lid_state)===1||String(j.lid??j.lid_state).toUpperCase()==='OPE
N'?'OPEN':'CLOSED';
  const sm=['OK','LOCKED','TAMPER','COLLECTING','CALIBRATING'];
  d.status=typeof j.status==='number'?(sm[j.status]||'OK'):String(j.status||j.bin_status||'OK').toUpperCase();
  if(j.cal)Object.assign(b.cal,j.cal);
  if(j.touch)b.touch=j.touch;
  if(Number.isFinite(+j.sim_mask))b.sim.mask=+j.sim_mask;
  d.faults=Array.isArray(j.faults)?j.faults:[];
  d.tamper_phase=j.tamper_phase||'';
  if(j.last_cmd)d.last_cmd=j.last_cmd;
  if(j.fw)b.fw=j.fw;
  if(Number.isFinite(+j.rssi))b.rssi=+j.rssi;
}
// --- Demo data: readings that drift like a real bin when no address is set ---
function stepEnv(b){
  const d=b.data,s=b.sim,on=bit=>(s.mask>>bit)&1;
  /* BME680 demo: the air quality index drifts toward a level set by how full the bin is,
     the raw temperature carries about 2 C of self-heating that the offset removes. */
  d.gas=on(2)?+s.values.gas:Math.round(clamp(d.gas+((35+d.fill*2.2)-d.gas)*0.02+(Math.random()-0.5)*5,10,500));
```

```
  if(on(3)){d.temp=+s.values.temp;d.temp_raw=d.temp-(b.cal.temp_off||0);d.humidity=+s.values.humidity;}
  else{
    d.temp_raw=clamp((d.temp_raw||30)+(Math.random()-0.5)*0.14+d.fill*0.0009,20,70);
    d.temp=+(d.temp_raw+(b.cal.temp_off||0)).toFixed(2);
    d.humidity=clamp(d.humidity+(Math.random()-0.5)*0.6,35,95);
  }
  d.pressure=+(clamp((d.pressure||1008)+(Math.random()-0.5)*0.06,985,1030)).toFixed(2);
  d.iaq_acc=b._accT?Math.min(3,Math.floor((Date.now()-b._accT)/20000)):3;
  d.warmEnd=b._accT&&d.iaq_acc===0?b._accT+20000:null;
  if(d.iaq_acc>=3)b._accT=0;
}
const DEMO={speed:1,pick:null};
const DEMO_SPEEDS=[[0,'Pause'],[1/60,'Real'],[1,'Normal'],[10,'Fast']];
function demoInUse(){return DEMO.speed!==1||state.bins.some(b=>b.demo&&(b._demoLink||(b._hold&&Object.keys(b._hold).length)))}
function demoPanelRender(){
  const p=$('#demoPanel');if(!p)return;
  const demos=state.bins.filter(b=>b.demo);
  p.classList.toggle('hide',!state.demo||!demos.length);
  if(!state.demo||!demos.length)return;
  if(!demos.some(b=>b.id===DEMO.pick))DEMO.pick=demos[0].id;
  const b=byId(DEMO.pick),link=b._demoLink||'';
  const btn=(attr,val,label,ic,cc,on)=>'<button class="btn sm binbtn" style="--cc:'+cc+'" data-'+attr+'="'+val+'" aria-pressed="'+!!on+'">'+icon(ic)+'<span>'+label+'</span></button>';
  const html='<h3 class="dhd">Fill speed</h3><div class="seg">'+DEMO_SPEEDS.map(x=>btn('dspeed',x[0],x[1].toUpperCase(),'stopwatch','var(--sim)',DEMO.speed===x[0])).join('')+'</div>'+
    '<h3 class="dhd">Act out on a bin</h3><div class="seg">'+demos.map(x=>btn('dpick',x.id,esc(x.name),'fill','var(--sim)',x.id===DEMO.pick)).join('')+'</div>'+
    '<div class="seg">'+btn('devent','full','FULL','fill','var(--bad)')+btn('devent','tamper','TAMPER','tamper','var(--c-tamper)')+
      btn('devent','air','BAD AIR','gas','var(--c-gas)')+btn('devent','empty','EMPTY','truck','var(--cal)')+'</div>'+
    '<h3 class="dhd">How '+esc(b.name)+' answers</h3><div class="seg">'+btn('dlink','','WI-FI','wifi','var(--c-link)',link==='')+
      btn('dlink','fb','FIREBASE','cloud','var(--c-cloud)',link==='fb')+btn('dlink','ts','THINGSPEAK','chart','var(--c-ts)',link==='ts')+
      btn('dlink','down','GO QUIET','power','var(--warn)',link==='down')+'</div>'+
    '<div class="actions">'+btn('dreset','1','RESET','undo','var(--sim)')+'</div>';
  const body=$('#demoBody');if(body.dataset.sig!==html){body.dataset.sig=html;body.innerHTML=html;}
  $('#demoHead').innerHTML=icon('flask')+'<span>DEMO OPTIONS</span>';
  $('#demoState').textContent=demoInUse()?'In use':'Not in use';
}
function demoAct(el){
  const b=byId(DEMO.pick);tourPause();
  if(el.dataset.dspeed!=null){DEMO.speed=+el.dataset.dspeed;sfx('choice');}
  else if(el.dataset.dpick!=null){DEMO.pick=el.dataset.dpick;sfx('choice');}
  else if(el.dataset.devent&&b){const d=b.data,c=b.cal,e=el.dataset.devent;
    if(e==='full')d.fill=Math.min(99,c.fill_th+4);
    if(e==='tamper')tourHold(b,{tilt:40});
    if(e==='air'){tourHold(b,{gas:c.gas_th+45});setTimeout(()=>{tourDrop(b,['gas']);b.data.gas=60;},20000);}
    if(e==='empty'){d.fill=0;d.weight_g=0;}
    sfx('confirm');toast({full:b.name+' is now over its fill limit.',tamper:b.name+' was tipped. Reset it from its card.',
      air:'Air quality spiked in '+b.name+'. It settles by itself.',empty:b.name+' is empty.'}[e]);}
  else if(el.dataset.dlink!=null&&b){const v=el.dataset.dlink||null;
    if(v==='down'&&b._demoLink!=='down')b._quietAt=Date.now();b._demoLink=v;sfx(v==='down'?'link_lost':v?'cloud':'link_up');}
  else if(el.dataset.dreset){DEMO.speed=1;state.bins.forEach(x=>{if(x.demo){x._demoLink=null;x._hold={};}});sfx('confirm');toast('Demo options reset.');}
  demoPanelRender();render();
}
function stepDemo(b){
  const d=b.data,s=b.sim,on=bit=>(s.mask>>bit)&1,h=b._hold||{};
  stepEnv(b);
  if(h.gas!=null)d.gas=h.gas;
  if(h.temp!=null){d.temp=h.temp;d.temp_raw=h.temp-(b.cal.temp_off||0);}
  if(h.humidity!=null)d.humidity=h.humidity;
  d.faults=h.faults||[];
  if(d.status==='CALIBRATING'){d.lid='CLOSED';d.distance_cm=+(b.cal.d_empty-Math.random()*0.4).toFixed(1);return;}
  if(on(0))d.fill=clamp(+s.values.fill,0,100);
  else if(d.status!=='COLLECTING')d.fill=clamp(d.fill+(0.022+Math.random()*0.015)*DEMO.speed,0,100);
  d.weight_g=on(1)?+s.values.weight:Math.round(clamp(d.fill/100*b.cal.weight_cap*0.92+Math.random()*25,0,5000));
  d.tilt=h.tilt!=null?h.tilt:on(4)&&+s.values.tamper?38:clamp(Math.random()*2.2,0,90);
  d.motion=on(5)?!!+s.values.motion:Math.random()<0.025;
  d.distance_cm=+(b.cal.d_empty-(d.fill/100)*(b.cal.d_empty-b.cal.d_full)).toFixed(1);
  if(d.status==='COLLECTING'){d.lid='OPEN';return;}
  if(d.tilt>b.cal.tilt_th){d.status='TAMPER';d.lid='CLOSED';return;}
  if(d.status==='TAMPER')d.status='OK';
  if(d.status==='LOCKED'){d.lid='CLOSED';return;}
  if(b.cal.auto_lock&&d.fill>=b.cal.fill_th){d.status='LOCKED';d.lid='CLOSED';return;}
  d.lid=on(6)?(+s.values.lid?'OPEN':'CLOSED'):(d.motion?'OPEN':'CLOSED');
}
/* ---------- live links: the bin's own Wi-Fi first, then Firebase, then ThingSpeak ----------
   ESP32 Integration Guide sections 4 to 7 and the Interrupt-Driven Design Guide section 5.
   A bin with an IP address, a Firebase cloud name (once Firebase is saved) or a ThingSpeak
   channel is live: it never shows made-up numbers, it shows the last real reading and
   "not answering" instead. A bin with none of these keeps the demo data for class. */
```

```
const ROUTE={local:['Wi-Fi','var(--c-link)','wifi'],fb:['Firebase','var(--c-cloud)','cloud'],ts:['ThingSpeak','var(--c-ts)','chart']};
// A dashboard opened from an https:// page cannot call http:// bins (mixed
// content), so local Wi-Fi is skipped there and only the clouds are used.
const SECURE=location.protocol==='https:';
const CLOUD_CMDS=['LOCK','UNLOCK','COLLECT'];
const enc=encodeURIComponent;
const CLIENT_ID=(function(){let v='';try{v=localStorage.getItem('smartbin.client')||''}catch(e){}
  if(!v){v='dash-'+Math.random().toString(36).slice(2,10);try{localStorage.setItem('smartbin.client',v)}catch(e){}}return v})();
const FB={cfg:null,token:'',until:0,uid:'',signing:null,watching:false,lastWatch:0,usageAt:0};
const TS={server:'https://api.thingspeak.com'};
try{const t=localStorage.getItem('smartbin.ts');if(t)TS.server=t}catch(e){}
const links={};
function L(b){return links[b.id]||(links[b.id]={es:null,esIp:'',esFails:0,esRetryAt:0,nextLocal:0,localFails:0,busyLocal:false,nextFb:0,busyFb:false,nextTs:0,busyTs:false})}
function fbUse(v){FB.cfg=v&&v.url?Object.assign({},v):null;FB.token='';FB.until=0;}
if(cloud.tested)fbUse(cloud.working);
function fbReady(){return !!(FB.cfg&&FB.cfg.url&&FB.cfg.key&&FB.cfg.email&&FB.cfg.pass)}
// A live bin never shows demo numbers. With no answer it shows its last real
// reading and "not answering".
function isLive(b){return !!((b.ip&&!SECURE)||(b.tsCh&&b.tsKey)||(fbReady()&&b.cloud))}
function timeout(ms){return typeof AbortSignal!=='undefined'&&AbortSignal.timeout?AbortSignal.timeout(ms):undefined}

/* Firebase over plain REST, the same calls as the app's Cloud.kt. */
function fbFriendly(m){
  m=String(m||'');
  if(/INVALID_LOGIN_CREDENTIALS|INVALID_PASSWORD|EMAIL_NOT_FOUND|INVALID_EMAIL/.test(m))return 'Wrong email or password.';
  if(/API key not valid|API_KEY_INVALID/.test(m))return 'The Web API key is wrong. Copy it again from the Firebase console.';
  if(/PASSWORD_LOGIN_DISABLED|OPERATION_NOT_ALLOWED/.test(m))return 'Email/Password sign-in is off in Firebase Authentication.';
  if(/TOO_MANY_ATTEMPTS/.test(m))return 'Too many tries, wait a few minutes.';
  if(/USER_DISABLED/.test(m))return 'This account is disabled.';
  if(/Permission denied|HTTP 401/.test(m))return 'The account is not on the allowed list (sb_users), or the rules were not pasted.';
  if(/HTTP 404/.test(m))return 'The database URL is wrong.';
  if(/Failed to fetch|NetworkError|aborted|timed out|TimeoutError/i.test(m))return 'No internet, or the database URL is wrong.';
  return m.length>90?m.slice(0,90):m;
}
async function fbSignIn(cfg){
  const url=cfg.url.startsWith('http://')?cfg.url.replace(/\/+$/,'')+'/__auth/signInWithPassword?key='+enc(cfg.key)
    :'https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key='+enc(cfg.key.trim());
  let r,j={};
  try{r=await fetch(url,{method:'POST',headers:{'Content-Type':'application/json'},
    body:JSON.stringify({email:cfg.email.trim(),password:cfg.pass,returnSecureToken:true}),signal:timeout(10000)});
    j=await r.json().catch(()=>({}));}
  catch(e){throw new Error(fbFriendly(e.message||e))}
  if(!r.ok)throw new Error(fbFriendly((j.error&&(j.error.message||j.error))||('HTTP '+r.status)));
  return {token:j.idToken,uid:j.localId,until:Date.now()+Math.max(60,(+j.expiresIn||3600)-300)*1000};
}
async function fbToken(){
  if(FB.token&&Date.now()<FB.until)return FB.token;
  if(!FB.signing)FB.signing=fbSignIn(FB.cfg).then(t=>{FB.token=t.token;FB.uid=t.uid;FB.until=t.until;return t.token}).finally(()=>{FB.signing=null});
  return FB.signing;
}
async function fbRest(method,path,body,query,cfg){
  const c=cfg||FB.cfg;
  if(!c)throw new Error('Firebase is not set up');
  const tok=cfg?cfg._token:await fbToken();
  let r,t='';
  try{r=await fetch(c.url.replace(/\/+$/,'')+'/'+path+'.json?auth='+enc(tok)+(query||''),{method,
    headers:body?{'Content-Type':'application/json'}:undefined,body:body?JSON.stringify(body):undefined,signal:timeout(8000),cache:'no-store'});
    t=await r.text();}
  catch(e){throw new Error(fbFriendly(e.message||e))}
  if(r.status===401&&!cfg)FB.token='';
  if(!r.ok){let m='HTTP '+r.status;try{const j=JSON.parse(t);m=(j.error&&(j.error.message||j.error))||m}catch(e){}throw new Error(fbFriendly(m))}
  return t&&t.trim()!=='null'?JSON.parse(t):null;
}
async function fbTest(v){
  const t=await fbSignIn(v),c=Object.assign({},v,{_token:t.token});
  const me=await fbRest('GET','sb_users/'+enc(t.uid),null,'',c);
  if(me===null)throw new Error('The account is not on the allowed list (sb_users).');
  const bins=await fbRest('GET','sb/bins',null,'&shallow=true',c);
  return bins&&typeof bins==='object'?Object.keys(bins).length:0;
}
async function sendCloudToBins(){
  if(state.role!=='operator'){toast('Only an operator can send the cloud account to the bins.');return;}
  const v=fbVals(),e=fbCheck(v);
  if(e){toast(e);return;}
  const targets=state.bins.filter(b=>b.ip&&!SECURE);
  if(!targets.length){toast('No bin has an IP address yet, so there is nothing to send to.');return;}
```

```
  let ok=0;const miss=[];
  await Promise.all(targets.map(async b=>{try{const r=await callRaw(b,'/api/cloud',{url:v.url,key:v.key,email:v.email,pass:v.pass});
    if(r.ok)ok++;else miss.push(b.name)}catch(err){miss.push(b.name)}}));
  sfx(ok?'cmd_ok':'cmd_fail');
  cloud.msg=[ok?'ok':'bad','Sent to '+ok+' of '+targets.length+' bins'+(miss.length?'. Not answering: '+miss.join(', '):'')+'. They sign in again within a few seconds.'];
  cloudRender();
}
async function fbWatchTick(now){
  if(!fbReady())return;
  const want=!document.hidden&&state.bins.some(b=>isLive(b)&&b.cloud&&b.route!=='local');
  if(want&&now-FB.lastWatch>=10000){
    FB.lastWatch=now;
    try{await fbRest('PUT','sb/watchers/'+CLIENT_ID,{ts:{'.sv':'timestamp'},device:(navigator.userAgentData&&navigator.userAgentData.platform)||navigator.platform||'Computer',
      kind:'dashboard',mode:'foreground',bin:'all'});FB.watching=true;}catch(e){}
  }else if(!want&&FB.watching)fbUnwatch();
}
function fbUnwatch(){
  if(!FB.watching||!fbReady()||!FB.token)return;
  FB.watching=false;FB.lastWatch=0;

  try{fetch(FB.cfg.url.replace(/\/+$/,'')+'/sb/watchers/'+CLIENT_ID+'.json?auth='+enc(FB.token),{method:'DELETE',keepalive:true}).catch(()=>{})}catch(e){}
}
async function fbUsageTick(now){
  if(!fbReady()||now-FB.usageAt<60000)return;
  FB.usageAt=now;
  try{
    const u=await fbRest('GET','sb/usage');
    if(!u)return;
    const bytes=+(u.bytes_month??u.down_bytes)||0,lim=+u.limit_bytes||10737418240,frac=bytes/lim;
    const gb=bytes/1073741824;
    const fill=$('#useFill');if(fill)fill.style.width=Math.min(100,Math.max(0.5,frac*100)).toFixed(1)+'%';
    const p=$('#cloudUse');
    if(p)p.textContent='About '+(gb<0.1?(bytes/1048576).toFixed(0)+' MB':gb.toFixed(2)+' GB')+' of the 10 GB the free Firebase plan allows this month ('+
      (frac*100).toFixed(1)+'%, an estimate from the bins; the exact figure is in the Firebase console). Bins write every 5 s while someone watches over the cloud, every minute otherwise, and at once when an alert starts.';
  }catch(e){}
}

/* ThingSpeak: field1 fill, 2 weight, 3 temperature, 4 humidity, 5 IAQ, 6 pressure, 7 tilt, 8 motion + 2 lid + 4 status. */
function tsMap(o){
  const f=n=>{const v=parseFloat(o['field'+n]);return Number.isFinite(v)?v:undefined};
  const flags=(f(8)||0)|0;
  return {fill:f(1),weight_g:f(2),temp:f(3),humidity:f(4),iaq:f(5),pressure:f(6),tilt:f(7),motion:!!(flags&1),lid:(flags&2)?'OPEN':'CLOSED',status:flags>>2};
}
async function tsHistory(b){
  if(!b.tsCh||!b.tsKey||b._tsHist)return;
  b._tsHist=true;
  try{
    const r=await fetch(TS.server+'/channels/'+enc(b.tsCh)+'/feeds.json?api_key='+enc(b.tsKey)+'&minutes=1440',{signal:timeout(15000),cache:'no-store'});
    if(!r.ok)throw new Error('HTTP '+r.status);
    const j=await r.json();
    const pts=(j.feeds||[]).map(o=>{const m=tsMap(o);return {t:Date.parse(o.created_at),fill:m.fill,temp:m.temp,humidity:m.humidity,pressure:m.pressure,gas:m.iaq}})
      .filter(p=>Number.isFinite(p.t)&&Number.isFinite(p.fill));
    if(pts.length){const last=pts[pts.length-1].t;b.hist=pts.concat(b.hist.filter(p=>p.t>last));if(state.view==='charts')drawCharts();}
  }catch(e){b._tsHist=false;}
}

function onStatus(b,j,route){
  normalise(b,j);
  const now=Date.now();
  b.seen[route]=now;b.lastAt=now;
  if(j.local_ip&&!b.ip)b.ipHint=j.local_ip;
}
// Live updates over Server-Sent Events (/api/events). Two failed opens in a
// row pause the stream for 60 s and fall back to polling /api/status.
function openStream(b){
  const l=L(b);
  if(l.es||!b.ip||SECURE||typeof EventSource==='undefined'||document.hidden||Date.now()<l.esRetryAt)return;
  let opened=false;
  const es=new EventSource(binBase(b)+'/api/events');
  l.es=es;l.esIp=b.ip;
  es.onopen=()=>{opened=true;l.esFails=0;};
  es.addEventListener('status',e=>{try{onStatus(b,JSON.parse(e.data),'local')}catch(x){}});
  es.addEventListener('ping',()=>{b.seen.local=Date.now();});
```

```
  es.onerror=()=>{
    if(!opened||es.readyState===2){
      l.esFails++;
      if(l.esFails>=2){es.close();if(l.es===es)l.es=null;l.esRetryAt=Date.now()+60000;}
    }
  };
}
function closeStream(b){const l=L(b);if(l.es){l.es.close();l.es=null;}}
function clearReadings(b){
Object.assign(b.data,{fill:0,distance_cm:0,weight_g:0,temp:0,temp_raw:0,humidity:0,pressure:0,gas:0,iaq_acc:0,tilt:0,lid:'CLOSED',status:'OK',motion:false});
  b.hist=[];b.prev={};
}
function relink(b){
  closeStream(b);
  const l=L(b);
  Object.assign(l,{esRetryAt:0,esFails:0,localFails:0,nextLocal:0,nextFb:0,nextTs:0});
  b.route=null;b.seen={local:0,fb:0,ts:0};b._tsHist=false;
  if(isLive(b)&&!b.lastAt)clearReadings(b);
  if(b.tsCh&&b.tsKey)tsHistory(b);
}
// Poll intervals: local 0.9 s (2 s after a miss, 30 s after 3 misses),
// Firebase 5 s (15 s after an error), ThingSpeak 15 s (30 s after an error).
async function pollLocal(b,l){
  if(l.busyLocal||Date.now()<l.nextLocal)return;
  l.busyLocal=true;
  try{onStatus(b,await call(b,'/api/status'),'local');l.localFails=0;l.nextLocal=Date.now()+900;}
  catch(e){l.localFails++;l.nextLocal=Date.now()+(l.localFails<3?2000:30000);}
  finally{l.busyLocal=false;}
}
async function pollFb(b,l){
  if(l.busyFb||Date.now()<l.nextFb)return;
  l.busyFb=true;l.nextFb=Date.now()+5000;
  try{
    const j=await fbRest('GET','sb/bins/'+enc(b.cloud)+'/status');
    if(j&&typeof j==='object'){
      // A Firebase status older than 90 s is ignored, as in the app.
      if(!j.ts||Date.now()-j.ts<90000)onStatus(b,j,'fb');
      else if(j.local_ip&&!b.ip)b.ipHint=j.local_ip;
    }
  }catch(e){l.nextFb=Date.now()+15000;}
  finally{l.busyFb=false;}
}
async function pollTs(b,l){
  if(l.busyTs||Date.now()<l.nextTs)return;
  l.busyTs=true;l.nextTs=Date.now()+15000;
  try{
    const r=await fetch(TS.server+'/channels/'+enc(b.tsCh)+'/feeds/last.json?api_key='+enc(b.tsKey)+'&status=true',{signal:timeout(8000),cache:'no-store'});
    const t=(await r.text()).trim();
    if(r.ok&&t&&t!=='-1'&&t!=='null'){
      const o=JSON.parse(t),at=Date.parse(o.created_at);
      if(!Number.isFinite(at)||Date.now()-at<600000)onStatus(b,tsMap(o),'ts');
      if(o.status&&o.status!=='null'&&!b.ip)b.ipHint=o.status;
    }
  }catch(e){l.nextTs=Date.now()+30000;}
  finally{l.busyTs=false;}
}
// One round per second. Route choice: Wi-Fi if heard in the last 4 s (15 s
// with an open stream), else Firebase if heard in 20 s, else ThingSpeak if
// heard in 45 s, else the bin counts as lost and plays "bin_lost".
async function poll(){
  const now=Date.now();
  let binSnd='';
  for(const b of state.bins){
    if(!isLive(b)){
      closeStream(b);b.online=b.live=false;b.route=null;
      if(b._demoLink!=='down'){stepDemo(b);pushHistory(b);}
      watchAlerts(b);continue;
    }
    const l=L(b);
    if(b.ip&&!SECURE){
      if(l.es&&l.esIp!==b.ip)closeStream(b);
      openStream(b);
      const streaming=l.es&&l.es.readyState===1;
      if(!document.hidden&&(!streaming||now-b.seen.local>12000))pollLocal(b,l);
    }else closeStream(b);
    const streamOpen=l.es&&l.es.readyState===1;
    const localOk=now-b.seen.local<4000||(streamOpen&&now-b.seen.local<15000);
    if(!localOk&&!document.hidden){
      if(fbReady()&&b.cloud)pollFb(b,l);
      if(now-b.seen.fb>=20000&&b.tsCh&&b.tsKey)pollTs(b,l);
    }
    const route=localOk?'local':now-b.seen.fb<20000?'fb':now-b.seen.ts<45000?'ts':null;
```

```
    const was=b.live;
    b.route=route;b.live=b.online=!!route;
    if(b._linked&&was!==b.live)binSnd=b.live?'bin_back':'bin_lost';
    if(b.live)b._linked=true;
    if(b.live){pushHistory(b);watchAlerts(b);}
  }
  linkSound(binSnd);tourTick(now);
  fbWatchTick(now);fbUsageTick(now);
  if($('#cloudRows')&&state.view==='settings'&&now-(poll._cloudAt||0)>2000){poll._cloudAt=now;cloudRender();}
  render();
}

/* Commands. On the bin's Wi-Fi the 200 reply is the confirmation. Over Firebase or
   ThingSpeak the dashboard waits for the bin's next status, as the app does (section 7). */
const EXPECT={LOCK:s=>s==='LOCKED',UNLOCK:s=>s!=='LOCKED',COLLECT:s=>s==='COLLECTING'};
function confirmCloud(b,cmd,pushId,ms,via){
  const t0=Date.now();
  const tick=async()=>{
    const age=Date.now()-t0;
    let res=null;
    if(pushId&&fbReady()){try{res=await fbRest('GET','sb/results/'+enc(pushId))}catch(e){}}
    if(res&&res.cmd===cmd){
      if(res.ok){sfx('cmd_ok');toast(b.name+' confirmed '+cmd+' through '+via+'.');}
      else{sfx('cmd_fail');toast(b.name+' refused '+cmd+(res.why?': '+res.why:'')+'.');}
      fbRest('DELETE','sb/results/'+enc(pushId)).catch(()=>{});render();return;
    }
    if(age>1500&&b.live&&EXPECT[cmd](b.data.status)){sfx('cmd_ok');toast(b.name+' confirmed '+cmd+' through
'+via+'.');render();return;}
    if(age>=ms){sfx('cmd_fail');toast(b.name+' did not confirm '+cmd+' within '+Math.round(ms/1000)+' s.');return;}
    setTimeout(tick,1000);
  };
  setTimeout(tick,1000);
}
// Sends a command by the best route. Only LOCK, UNLOCK and COLLECT may go
// through the clouds; the rest need the bin's Wi-Fi.
async function sendLive(b,cmd,extra){
  const body=Object.assign({cmd},extra||{});
  if(b.ip&&!SECURE&&(b.route==='local'||!b.route||!CLOUD_CMDS.includes(cmd))){
    try{
      const r=await callRaw(b,'/api/command',body);
      if(r.ok){sfx('cmd_ok');if(r.j&&r.j.status)b.data.status=String(r.j.status).toUpperCase();L(b).nextLocal=0;return true;}
      sfx('cmd_fail');toast(b.name+' refused '+cmd+(r.j&&r.j.why?': '+r.j.why:' (HTTP '+r.status+')')+'.');return false;
    }catch(e){
      if(!CLOUD_CMDS.includes(cmd)){sfx('cmd_fail');toast(b.name+' did not answer. '+cmd.replace(/_/g,' ')+' needs the bin's
Wi-Fi.');return false;}
    }
  }
  if(!CLOUD_CMDS.includes(cmd)){sfx('cmd_fail');toast(cmd.replace(/_/g,' ')+' needs '+b.name+'’s Wi-Fi. Over the cloud only
LOCK, UNLOCK and COLLECT work.');return false;}
  if(fbReady()&&b.cloud){
    try{
      const r=await fbRest('POST','sb/commands/'+enc(b.cloud),{cmd,ts:{'.sv':'timestamp'},from:'dashboard'});
      toast(cmd+' sent to '+b.name+' through Firebase. Waiting for the bin…');
      confirmCloud(b,cmd,r&&r.name,8000,'Firebase');return true;
    }catch(e){}
  }
  if(b.tbId&&b.tbKey){
    try{
      const r=await fetch(TS.server+'/talkbacks/'+enc(b.tbId)+'/commands.json',{method:'POST',
        headers:{'Content-Type':'application/x-www-form-
urlencoded'},body:'api_key='+enc(b.tbKey)+'&command_string='+enc(cmd),signal:timeout(10000)});
      if(!r.ok)throw new Error('HTTP '+r.status);
      toast(cmd+' queued for '+b.name+' on ThingSpeak TalkBack. The bin picks it up within 15 s.');
      confirmCloud(b,cmd,null,45000,'ThingSpeak');return true;
    }catch(e){}
  }
  sfx('cmd_fail');toast(b.name+' cannot be reached on any link, so '+cmd+' was not sent.');return false;
}

function cloudRowText(b){
  if(!isLive(b))return ['var(--warn)','demo bin, nothing sent'];
  if(b.route==='local')return ['var(--c-link)','on Wi-Fi, last reading '+ago(b.seen.local)];
  if(b.route==='fb')return ['var(--c-cloud)','through Firebase, last reading '+ago(b.seen.fb)];
  if(b.route==='ts')return ['var(--c-ts)','through ThingSpeak, last reading '+ago(b.seen.ts)];
  return ['var(--c-tamper)','not answering'+(b.lastAt?', last reading '+ago(b.lastAt):'')];
}
/* The link sound follows the whole dashboard, not each bin: Wi-Fi, a cloud, or nothing.
   A single bin dropping or coming back plays its own sound only when the overall link stays. */
function overallLink(){
  const live=state.bins.filter(isLive);
  if(!live.length)return '';
  const r=live.map(b=>b.route);
  return r.includes('local')?'local':r.includes('fb')?'fb':r.includes('ts')?'ts':'down';
}
function linkSound(binSnd){
  const ov=overallLink(),was=linkSound.last;
```

```
  linkSound.last=ov;
  if(!inMain())return;
  if(was!==undefined&&ov!==was&&ov)sfx(ov==='local'?'link_up':ov==='down'?'link_lost':'cloud');
  else if(binSnd)sfx(binSnd);
}
function inMain(){const g=$('#gate');return !!g&&g.classList.contains('hide')}
function linkSlot(b){
  if(!isLive(b)&&b._demoLink==='down')return [icon('wifi',false),'OFFLINE',false,'var(--c-tamper)','Demo: this bin stopped answering'];
  if(!isLive(b)&&b._demoLink){const r=ROUTE[b._demoLink];return [icon(r[2]),b._demoLink==='fb'?'CLOUD':'TS',true,r[1],'Demo: read through '+r[0]+' because its Wi-Fi is down'];}
  if(!isLive(b))return [icon('wifi',false),'DEMO',true,'var(--warn)','Demo data, no IP address or cloud link set'];
  if(!b.live)return [icon('wifi',false),'OFFLINE',false,'var(--c-tamper)','Bin is not answering'+(b.lastAt?'. Last reading '+ago(b.lastAt):'')];
  const r=ROUTE[b.route];
  return [b.route==='local'?icon('wifi',true):icon(r[2]),b.route==='local'?'WI-FI':b.route==='fb'?'CLOUD':'TS',true,r[1],'Reading this bin through '+r[0]];
}
// At start-up: if this page was served by a bin, that bin is added (or
// matched by its cloud name) and read live without typing its address.
async function liveStart(){
  state.bins.forEach((b,i)=>{if(isLive(b))clearReadings(b);else if(!b.hist.length)seedHistory(b,i);if(b.tsCh&&b.tsKey)tsHistory(b);});
  document.addEventListener('visibilitychange',()=>{if(document.hidden){state.bins.forEach(closeStream);fbUnwatch();}});
  window.addEventListener('pagehide',fbUnwatch);
  if(!SECURE&&location.protocol==='http:'&&location.host&&!/^(localhost|127\.|\[::1\])/.test(location.hostname)){
    try{
      const r=await fetch('/api/status',{signal:timeout(1500),cache:'no-store'});
      if(r.ok){
        const j=await r.json(),here=location.host;
        if(j&&j.id&&!state.bins.some(b=>b.ip===here)){
          let b=state.bins.find(x=>!x.demo&&x.cloud===j.id&&!x.ip)||state.bins.find(x=>!x.demo&&!x.ip&&!x.cloud);
          if(!b){
b=newBin({id:'b'+Date.now().toString(36),name:String(j.name||j.id).replace(/^BIN\b/i,'Bin'),place:'',ip:here,cloud:j.id});
            state.bins.unshift(b);
          }
          b.ip=here;if(!b.cloud)b.cloud=j.id;relink(b);persist();
          if(!byId(state.selSet))state.selSet=state.selSim=state.envBin=b.id;
          cardSig='';render();loadSettings();buildSim();
          toast(b.name+' is the bin serving this page, so it is read live at '+here+'.');
        }
      }
    }catch(e){}
  }
}
function pushHistory(b){
  const now=Date.now(),last=b.hist[b.hist.length-1];
  if(last&&now-last.t<10000)return;
  b.hist.push({t:now,fill:b.data.fill,temp:b.data.temp,humidity:b.data.humidity,pressure:b.data.pressure,gas:b.data.gas});
  const cut=now-864e5;
  while(b.hist.length&&b.hist[0].t<cut)b.hist.shift();
}

/* ---------- alerts ---------- */
let alertSeq=0,lastUnread=0;
const SEV_RANK=['critical','warning','info','success'];
const SEV_TONE={critical:['bell','var(--bad)','red'],warning:['bell','var(--warn)','amber'],
  info:['bell','var(--sim)','purple'],success:['bell','var(--ok)','green']};
const unreadCount=()=>state.alerts.reduce((n,a)=>n+(a.read?0:1),0);
const KIND_BY_IC={fill:'full',temp:'sensor',humid:'sensor',gas:'sensor',tamper:'tamper',lock:'lock',truck:'pickup',calib:'setup'};
// Adds one row to the alert history and plays its sound (not on the start page).
function addAlert(sev,ic,title,meta,k){
  const kind=k||KIND_BY_IC[ic]||'setup';
  /* No alert sounds on the start page. A pickup alert right after the DONE key would
     play over the key's own tune, so it only sounds when nothing else just played. */
  if(inMain()){
    if(kind==='pickup'){if(Date.now()-SND.at>500)sfx('alert_pickup');}
    else sfx(kind==='full'?'alert_full':kind==='tamper'?'alert_tamper':kind==='sensor'?(sev==='success'?'alert_info':'alert_sensor'):(kind==='lock'&&ic==='lock')?'alert_lock':'alert_info');
  }
  state.alerts.unshift({id:++alertSeq,sev,ic,k:k||KIND_BY_IC[ic]||'setup',title,meta,t:Date.now(),read:false});
  state.alerts=state.alerts.slice(0,120);
}
/* Demo history so the alert tools have something to act on. Only when no bin has an address. */
function seedAlerts(){
  if(state.bins.some(isLive))return;
  const n=state.bins.map(b=>b.name),a=n[0]||'Bin 01',b2=n[1]||a,M=60000;
  [[2,'critical','fill','Fill threshold passed',b2+' is at 81%'],[9,'warning','gas','Air quality warning',b2+' index 212, limit 200'],
   [34,'critical','tamper','Tamper detected',a+' was moved or tilted'],[41,'warning','humid','Humidity high',b2+' at 88%, limit 85%'],
   [60,'info','lock','Bin locked',a+' locked from the dashboard'],[62,'success','check','Bin unlocked',a+' is open for use again','lock'],
```

```
    [180,'success','truck','Collection complete',b2+' is back to normal'],[300,'info','calib','Calibration started',a+' is being calibrated'],
    [302,'success','check','Settings saved',a+': empty 30 cm, full 5 cm, alert at 80%','setup'],[1260,'critical','temp','Temperature high',a+' at 41.2 \u00B0C'],
    [4320,'success','truck','Collection complete',a+' is back to normal'],[17280,'critical','fill','Fill threshold passed',a+' is at 83%']]

        .forEach((s,i)=>state.alerts.push({id:++alertSeq,sev:s[1],ic:s[2],k:s[5]||KIND_BY_IC[s[2]]||'setup',title:s[3],meta:s[4],t:Date.now()-s[0]*M,read:i>=2}));
  lastUnread=unreadCount();
}
function markAllRead(){
  const n=unreadCount();
  if(!n)return;
  state.alerts.forEach(a=>{a.read=true});
  alertSig='';banSig='';renderAlerts();renderBanners();
  toast('Marked '+n+' alert'+(n>1?'s':'')+' as read.');
}
function toggleRead(id){
  const a=state.alerts.find(x=>x.id===id);
  if(!a)return;
  a.read=!a.read;
  alertSig='';banSig='';renderAlerts();renderBanners();
}
// Compares this reading with the previous one (b.prev) and raises an alert
// only on a change: full, temperature, humidity, air, tamper, lock, unlock,
// calibration, sensor lost or back, and air sensor ready.
function watchAlerts(b){
  const d=b.data,p=b.prev,c=b.cal;
  const full=d.fill>=c.fill_th,hot=d.temp>=c.temp_th,humid=d.humidity>=c.hum_th,gassy=d.gas>=c.gas_th;
  if(full&&!p.full)addAlert('critical','fill','Fill threshold passed',b.name+' is at '+Math.round(d.fill)+'%');
  if(hot&&!p.hot)addAlert('critical','temp','Temperature high',b.name+' at '+fmt(d.temp)+' °C');
  if(humid&&!p.humid)addAlert('warning','humid','Humidity high',b.name+' at '+Math.round(d.humidity)+'%, limit '+c.hum_th+'%');
  if(gassy&&!p.gassy)addAlert('warning','gas','Air quality warning',b.name+' index '+Math.round(d.gas)+', limit '+c.gas_th);
  if(d.status==='TAMPER'&&p.status!=='TAMPER')addAlert('critical','tamper','Tamper detected',b.name+' was moved or tilted');
  if(d.status==='LOCKED'&&p.status!=='LOCKED')addAlert('info','lock','Bin locked',b.name+' locked from the dashboard');
  if(d.status!=='LOCKED'&&p.status==='LOCKED')addAlert('success','check','Bin unlocked',b.name+' is open for use again','lock');
  if(d.status==='CALIBRATING'&&p.status!=='CALIBRATING')addAlert('info','calib','Calibration started',b.name+' is being calibrated');
  const fl=(d.faults||[]).join(', ');
  if(fl&&fl!==p.faults)addAlert('warning','gear','Sensor not answering',b.name+': '+fl,'sensor');
  if(!fl&&p.faults)addAlert('success','check','Sensor back',b.name+' reads every sensor again','sensor');
  if(p.acc===0&&d.iaq_acc>0)addAlert('success','gas','Air sensor ready',b.name+' air quality readings are settled','sensor');
  b.prev={full,hot,humid,gassy,status:d.status,faults:fl,acc:d.iaq_acc};
}

/* ---------- commands ---------- */
// Every command key ends here. Live bins go to sendLive; demo bins change
// their own data so the demo behaves like the real thing.
async function command(id,cmd,extra){
  const b=byId(id);if(!b)return;
  if(b.demo&&!TOUR.busy)tourPause();
  sfx(CMD_SFX[cmd]||'confirm');
  if(isLive(b)){await sendLive(b,cmd,extra);render();return;}
  const d=b.data;
  if(cmd==='LOCK'){d.status='LOCKED';d.lid='CLOSED';}
  if(cmd==='UNLOCK')d.status='OK';
  if(cmd==='CLEAR_TAMPER'){d.status='OK';d.tilt=0;tourDrop(b,['tilt']);}
  if(cmd==='COLLECT'){d.status='COLLECTING';d.lid='OPEN';}
  if(cmd==='COLLECT_DONE'){d.status='OK';d.lid='CLOSED';d.fill=0;d.weight_g=0;
    addAlert('success','truck','Collection complete',b.name+' is back to normal');}
  if(cmd==='TARE')d.weight_g=0;
  if(cmd==='RESET_GAS'){d.iaq_acc=0;b._accT=Date.now();d.warmEnd=b._accT+20000;}
  if(cmd==='CALIBRATE_START')d.status='CALIBRATING';
  if(cmd==='CALIBRATE_END')d.status='OK';
  render();
}
let toastEl=null,toastTimer=null;
function toast(msg){
  if(toastEl)toastEl.remove();
  toastEl=document.createElement('div');
  toastEl.className='banner amber toast';
  toastEl.innerHTML=icon('bell')+'<span>'+msg+'</span>';
  document.body.appendChild(toastEl);
  clearTimeout(toastTimer);toastTimer=setTimeout(()=>{toastEl&&toastEl.remove();toastEl=null},4500);
}

/* ---------- mode ---------- */
function currentMode(){
  const cal=!!state.calib,sim=state.bins.some(b=>b.sim.mask>0);
  if(cal&&sim)return 'dual';
  if(cal)return 'calib';
  if(sim)return 'sim';
  if(state.bins.some(b=>b.data.status==='TAMPER'||b.data.fill>=b.cal.fill_th))return 'alert';
```

```
  return 'normal';
}
function binMode(b){
  const cal=!!(state.calib&&state.calib.bin===b.id),sim=b.sim.mask>0;
  return cal&&sim?'dual':cal?'cal':sim?'sim':'none';
}
let lastMode='';
/* --- overview filter & view --- */
function applyOvFilter(){
  const f=state.ovFilter;
  const counts={all:state.bins.length,attn:0,sim:0,cal:0};
  state.bins.forEach(b=>{
    const card=$('#card-'+b.id);
    const tile=$('#tile-'+b.id);
    let show=true;
    if(f==='attn')show=b.data.fill>=b.cal.fill_th||b.data.status!=='OK';
    else if(f==='sim')show=b.sim.mask>0;
    else if(f==='cal')show=b.data.status==='CALIBRATING';
    if(card)card.classList.toggle('hide',!show);
    if(tile)tile.classList.toggle('hide',!show);
    if(b.data.fill>=b.cal.fill_th||b.data.status!=='OK')counts.attn++;
    if(b.sim.mask>0)counts.sim++;
    if(b.data.status==='CALIBRATING')counts.cal++;
  });
  $$('#filterChips .fchip').forEach(c=>{
    const k=c.dataset.filter;
    c.classList.toggle('on',k===f);
    const n=counts[k]||0;
    const badge=c.querySelector('.fbadge');
    if(badge){
      if(k!=='all'&&n>0){badge.textContent=n;badge.style.display='';}
      else{badge.style.display='none';}
    }
  });
}
function applyOvView(){
  const gv=state.ovView==='grid'&&state.bins.length>=3;
  const host=$('#bins');
  if(host)host.classList.toggle('gv',gv);
  const btn=$('#viewToggle');
  if(btn){
    btn.title=gv?'Switch to list view':'Switch to grid view';
    const gi=$('#vicoGrid'),li=$('#vicoList');
    if(gi&&li){gi.classList.toggle('hide',gv);li.classList.toggle('hide',!gv);}
  }
  const tp=$('#ovTools');
  if(tp)tp.classList.toggle('hide',state.bins.length<3);
}
function updateTiles(){
  state.bins.forEach((b,i)=>{
    const t=$('#tile-'+b.id);if(!t)return;
    const pct=Math.round(b.data.fill),c=b.cal;
    const full=pct>=c.fill_th,tamper=b.data.status==='TAMPER';
    const bm=binMode(b);
    t.dataset.bm=bm;
    t.style.outline=(full||tamper)?'2.5px solid var(--bad)':'';
    t.style.outlineOffset='-2px';
    const fn=t.querySelector('.tfill');if(fn)fn.textContent=pct+'%';
    let warn=t.querySelector('.twarn');
    const hasWarn=b.data.temp>=c.temp_th||b.data.humidity>=c.hum_th||b.data.gas>=c.gas_th;
    if(hasWarn&&!warn){warn=document.createElement('span');warn.className='twarn';warn.textContent='!';t.appendChild(warn);}
    if(warn)warn.style.display=hasWarn?'':'none';
    const bar=t.querySelector('.tbar i');
    if(bar){bar.style.width=clamp(pct,0,100)+'%';bar.style.background=full?'var(--bad)':pct>c.fill_th*0.7?'var(--
warn)':colorOf(i);}
    const st=t.querySelector('.tstatus');
    if(st){const txt=tamper?'TAMPER':full?'FULL':b.data.status==='CALIBRATING'?'CALIB':b.data.lid==='OPEN'?'LID OPEN':'OK';
      st.textContent=txt;st.classList.toggle('alert-status',tamper||full);}
    const svg=t.querySelector('svg.ico.lg');
    if(svg){svg.style.color=colorOf(i);
      const wrap=svg.querySelector('.wavewrap');
      if(wrap)wrap.setAttribute('transform','translate(0,'+(42-(clamp(pct,0,100)/100)*30).toFixed(1)+')');}
  });
}
/* tile overlay */
function showTileOverlay(bid){
  state.ovTile=bid;
  let ov=$('#tileOverlay');
  if(!ov){
    ov=document.createElement('div');ov.id='tileOverlay';ov.className='tile-overlay hide';
    ov.innerHTML='<div class="tile-detail" id="tileDetail"><button class="tile-close" id="tileClose"><svg viewBox="0 0 24 24"
fill="none" stroke="currentColor" stroke-width="2.5" stroke-linecap="round"><path d="M6 6l12 12M18 6L6
18"/></svg></button><div id="tileCard"></div>'+
      '<div class="tile-nav"><button class="btn sm" id="tilePrev">&#8592; Prev</button><button class="btn sm"
id="tileNext">Next &#8594;</button></div></div>';
    document.body.appendChild(ov);
```

```
    ov.addEventListener('click',e=>{if(e.target===ov)hideTileOverlay();});
    $('#tileClose').addEventListener('click',hideTileOverlay);
    $('#tilePrev').addEventListener('click',()=>stepTile(-1));
    $('#tileNext').addEventListener('click',()=>stepTile(1));
    document.addEventListener('keydown',e=>{if(state.ovTile){if(e.key==='Escape')hideTileOverlay();else
if(e.key==='ArrowLeft')stepTile(-1);else if(e.key==='ArrowRight')stepTile(1);}});
  }
  /* fill content first, then show - prevents late appearance */
  fillTileDetail(bid);
  ov.classList.remove('hide');
}
function hideTileOverlay(){state.ovTile=null;const ov=$('#tileOverlay');if(ov)ov.classList.add('hide');}
function stepTile(dir){
  const visible=state.bins.filter(b=>!$('#card-'+b.id).classList.contains('hide'));
  const idx=visible.findIndex(b=>b.id===state.ovTile);
  if(idx<0)return;
  const next=visible[(idx+dir+visible.length)%visible.length];
  if(next)fillTileDetail(next.id);
}
let ovDetailSig='';
function fillTileDetail(bid){
  state.ovTile=bid;
  const tc=$('#tileCard');if(!tc)return;
  const card=$('#card-'+bid);
  if(!card)return;
  /* sig-check: build a stable fingerprint from data, not innerHTML (which shifts every renderCard) */
  const b=byId(bid);
  const bsig=b?[b.data.fill,b.data.status,b.data.locked,b.data.temp,b.data.gas,b.data.humidity,b.data.pressure,
    b.data.iaq_acc,b.sim.mask,b.name,b.place,state.role,
    state.calib?state.calib.bin:''].join('|'):'';
  const sig=bid+'|'+bsig;
  if(sig===ovDetailSig&&tc.firstElementChild)return;
  ovDetailSig=sig;
  tc.innerHTML=card.outerHTML;
  const clone=tc.firstElementChild;
  if(clone){
    clone.removeAttribute('id');/* avoid duplicate IDs */
    clone.setAttribute('data-bin',bid);/* keep bin reference for event delegation */
    clone.classList.remove('hot');
    clone.style.cssText='';/* clear any inline sim/cal border styles */
    /* rename clipPath IDs so they don't collide with the original card */
    clone.querySelectorAll('clipPath[id]').forEach(cp=>{
      const oldId=cp.id,newId='ov-'+oldId;
      cp.id=newId;
      const g=cp.closest('svg')?.querySelector('[clip-path="url(#'+oldId+')"]');
      if(g)g.setAttribute('clip-path','url(#'+newId+')');
    });
    /* gear/SET button stays enabled in overlay */
    clone.querySelectorAll('[data-r="edit"]').forEach(r=>r.classList.add('hide'));
  }
}

/* --- live-update overlay card in place (no re-clone) --- */
function renderOverlayCard(){
  const tc=$('#tileCard');if(!tc)return;
  const clone=tc.firstElementChild;if(!clone)return;
  const bid=state.ovTile;
  const b=byId(bid);if(!b)return;
  const d=b.data,c=b.cal,bi=state.bins.indexOf(b);
  const bcol=bi>=0?colorOf(bi):statusColor(d.status,d.fill,c);
  const q=r=>clone.querySelector('[data-r="'+r+'"]');

  /* wave position + color */
  const hero=q('hero');
  if(hero){
    const kind=d.status==='CALIBRATING'?'cal':'bin';
    if(kind==='bin'){
      const svg=hero.querySelector('svg'),wrap=hero.querySelector('.wavewrap');
      if(svg)svg.style.color=bcol;
      if(wrap)wrap.setAttribute('transform','translate(0,'+(42-(clamp(d.fill,0,100)/100)*30).toFixed(1)+')');
    }
  }

  /* fill percentage */
  const pctEl=q('pct');
  if(pctEl){const v=Math.round(d.fill);pctEl.innerHTML=v+'<span>%</span>';}

  /* bar */
  const bar=q('bar');
  if(bar){bar.style.width='calc('+clamp(d.fill,0,100)+'% - 6px)';bar.style.background=levelColor(d.fill,c);}

  /* sensors */
  const vals={

temp:[fmt(d.temp,1)+'°',d.temp>=c.temp_th,d.temp<22?'cool':(d.temp/c.temp_th>=1?'crit':d.temp/c.temp_th>=.8?'high':d.temp/c.t
emp_th>=.5?'mid':'low')],
```

```
    humid:[Math.round(d.humidity)+'%',d.humidity>=c.hum_th,d.humidity>=c.hum_th?'crit':d.humidity>=c.hum_th-10?'high':d.humidity>=60?'mid':d.humidity<=35?'cool':'low'],
    weight:[(d.weight_g/1000).toFixed(2)+'kg',d.weight_g>=c.weight_cap,(()=>{const f=d.weight_g/(c.weight_cap||4500);return f>=1?'crit':f>=.8?'high':f>=.5?'mid':'low';})()],
    gas:[String(Math.round(d.gas)),d.gas>=c.gas_th,(()=>{const f=d.gas/(c.gas_th||200);return f>=1?'crit':f>=.8?'high':f>=.5?'mid':'low';})()]
  };
  clone.querySelectorAll('[data-s]').forEach(el=>{
    const v=vals[el.dataset.s];if(!v)return;
    const t=el.querySelector('.v');if(t&&t.textContent!==v[0])t.textContent=v[0];
    el.dataset.lv=v[2];
    el.classList.toggle('alarm',v[1]);el.classList.toggle('nm-in',!v[1]);
  });

  /* status chips - use same sig-check as renderCard */
  const simOn=b.sim.mask>0,calOn=binMode(b)==='cal'||binMode(b)==='dual';
  const tilting=d.tamper_phase==='COUNTDOWN'&&d.status!=='TAMPER'&&d.status!=='CALIBRATING';
  const csig=[d.lid,d.status,d.motion,simOn,calOn,b.live,b.route,isLive(b),b._demoLink,tilting].join('|');
  const ch=q('chips');
  if(ch&&ch.dataset.sig!==csig){
    ch.dataset.sig=csig;
    const stKey=d.status==='CALIBRATING'?'OK':d.status;
    const stIco=stKey==='TAMPER'?icon('tamper'):stKey==='LOCKED'?icon('lock',true)
      :stKey==='COLLECTING'?icon('truck'):icon('check');
    const stCol={OK:'var(--ok)',LOCKED:'var(--sim)',TAMPER:'var(--c-tamper)',
      COLLECTING:'var(--cal)'}[stKey]||'var(--ink2)';
    const st=STATUS_SHORT[stKey]||[stKey,stKey];
    ch.innerHTML=[
      slot(icon('lid',d.lid==='OPEN'),d.lid==='OPEN'?'OPEN':'CLOSED',d.lid==='OPEN','var(--c-lid)',
        d.lid==='OPEN'?'Lid is open':'Lid is closed'),
      tilting?slot(icon('tamper'),'TILTED',true,'var(--warn)','Tipped over. Tamper alert if it stays tipped'):slot(stIco,st[0],true,stCol,st[1]),
      slot.apply(null,linkSlot(b)),
      slot(icon('flask'),'SIM',simOn,'var(--sim)',simOn?'Simulation is on':'Simulation is off'),
      slot(icon('calib'),'CAL',calOn,'var(--cal)',calOn?'Calibration mode is on':'Calibration mode is off'),
      slot(icon('motion',d.motion),'MOTION',d.motion,'var(--c-motion)',
        d.motion?'Motion detected':'No motion')
    ].join('');
  }

  /* action buttons */
  const op=state.role==='operator',busy=d.status==='CALIBRATING';
  const locked=d.status==='LOCKED',tamper=d.status==='TAMPER',collecting=d.status==='COLLECTING';
  const want=[];
  want.push(locked
    ?['UNLOCK','UNLOCK','solid','lock',!busy,'var(--sim)']
    :['LOCK','LOCK','tint','lock',!(busy||tamper),'var(--sim)']);
  if(op)want.push(['CLEAR_TAMPER','RESET TAMPER',tamper?'danger':'tint','tamper',tamper,'var(--c-tamper)']);
  want.push(collecting
    ?['COLLECT_DONE','DONE','solid','check',true,'var(--cal)']
    :['COLLECT','COLLECT','tint','truck',!busy,'var(--c-link)']);
  const asig=want.map(w=>w.join('|')).join(',');
  const act=q('actions');
  if(act&&act.dataset.sig!==asig){
    act.dataset.sig=asig;
    act.innerHTML=want.map(w=>{
      const tip=w[0]==='CLEAR_TAMPER'&&!tamper?' title="Nothing to reset on this bin"':'';
      return '<button class="btn '+w[2]+'" style="--cc:'+w[5]+'" data-cmd="'+w[0]+
        '" data-bin="'+bid+'"'+(w[4]?'':' disabled')+tip+'>'+icon(w[3],w[0]==='UNLOCK')+
        '<span>'+w[1]+'</span></button>';
    }).join('');
  }
}

/* --- settings overlay (SET button) --- */
function openSetOverlay(bid){
  const b=byId(bid);if(!b)return;
  let ov=$('#setOverlay');
  if(!ov){
    ov=document.createElement('div');ov.id='setOverlay';ov.className='set-overlay hide';
    ov.innerHTML='<div class="set-panel nm" id="setPanel">'+
      '<button class="set-close" id="setClose"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.5" stroke-linecap="round"><path d="M6 6l12 12M18 6L6 18"/></svg></button>'+
      '<h3>'+icon('gear')+' <span id="setTitle"></span></h3>'+
      '<div class="set-fields">'+
        '<div class="field"><label>Name</label><input type="text" id="setName"></div>'+
        '<div class="field"><label>Location</label><input type="text" id="setPlace"></div>'+
        '<div class="field"><label>IP address</label><input type="text" id="setIP" placeholder="192.168.1.101"></div>'+
        '<div class="field"><label>Firebase cloud name</label><input type="text" id="setCloud" placeholder="bin01" spellcheck="false"></div>'+
        '<div class="field"><label>ThingSpeak channel ID</label><input type="text" id="setTsCh" inputmode="numeric" placeholder="1234567"></div>'+
        '<div class="field"><label>ThingSpeak read key</label><input type="text" id="setTsKey" spellcheck="false"></div>'+
        '<div class="field"><label>TalkBack ID</label><input type="text" id="setTbId" inputmode="numeric"></div>'+
        '<div class="field"><label>TalkBack API key</label><input type="text" id="setTbKey" spellcheck="false"></div>'+
      '</div>'+
```

```
      '<div class="set-actions">'+
        '<button class="btn sm danger" id="setRemove">Remove</button>'+
        '<button class="btn sm accent" id="setSave">Save</button>'+
      '</div></div>';
    document.body.appendChild(ov);
    ov.addEventListener('click',e=>{if(e.target===ov)closeSetOverlay();});
    $('#setClose').addEventListener('click',closeSetOverlay);
    $('#setSave').addEventListener('click',()=>{const bid=ov.dataset.bin;if(bid)saveFromSetOverlay(bid);});
    $('#setRemove').addEventListener('click',()=>{const bid=ov.dataset.bin;if(bid)removeFromSetOverlay(bid);});
    document.addEventListener('keydown',e=>{if(e.key==='Escape'&&ov&&!ov.classList.contains('hide'))closeSetOverlay();});
  }
  ov.dataset.bin=bid;
  $('#setTitle').textContent=b.name+' Settings';
  $('#setName').value=b.name;
  $('#setPlace').value=b.place;
  $('#setIP').value=b.ip;
  $('#setIP').placeholder=b.ipHint||'192.168.1.101';
  $('#setCloud').value=b.cloud||'';$('#setTsCh').value=b.tsCh||'';$('#setTsKey').value=b.tsKey||'';
  $('#setTbId').value=b.tbId||'';$('#setTbKey').value=b.tbKey||'';
  ov.classList.remove('hide');
}
function closeSetOverlay(){
  const ov=$('#setOverlay');if(ov)ov.classList.add('hide');
}
function saveFromSetOverlay(bid){
  const b=byId(bid);if(!b)return;
  const ov=$('#setOverlay');
  if(ov&&!ov.classList.contains('hide')&&ov.dataset.bin===bid){
    b.name=$('#setName').value.trim()||b.name;
    b.place=$('#setPlace').value.trim();
    b.ip=$('#setIP').value.trim();
    b.cloud=$('#setCloud').value.trim();b.tsCh=$('#setTsCh').value.trim();b.tsKey=$('#setTsKey').value.trim();
    b.tbId=$('#setTbId').value.trim();b.tbKey=$('#setTbKey').value.trim();
    relink(b);
  }else{
    const row=$('#card-'+bid+' [data-r="edit"]');
    if(row){b.name=row.querySelector('[data-k="name"]')?.value.trim()||b.name;
      b.place=row.querySelector('[data-k="place"]')?.value.trim()||'';
      b.ip=row.querySelector('[data-k="ip"]')?.value.trim()||'';relink(b);}
  }
  persist();cardSig='';render();loadSettings();buildSim();toast(b.name+' saved.');
  closeSetOverlay();
}
function removeFromSetOverlay(bid){
  const i=state.bins.findIndex(b=>b.id===bid);
  if(i>-1){const gone=state.bins.splice(i,1)[0];
    if(state.selSet===gone.id)state.selSet=state.bins[0]&&state.bins[0].id;
    if(state.selSim===gone.id)state.selSim=state.selSet;
    if(state.calib&&state.calib.bin===gone.id)state.calib=null;
    if(state.envBin===gone.id)state.envBin=state.selSet;
    persist();cardSig='';render();loadSettings();buildSim();toast(gone.name+' removed.');
    closeSetOverlay();hideTileOverlay();
  }
}

function applyMode(){
  const m=currentMode();
  if(m===lastMode)return;
  lastMode=m;
  document.documentElement.setAttribute('data-mode',m);
  const map={normal:['check','NORMAL'],sim:['flask','SIMULATION'],calib:['calib','CALIBRATING'],
    dual:['calib','SIM + CALIBRATION'],alert:['bell','ALERT']};
  $('#modeico').innerHTML=icon(map[m][0]);
  $('#modetext').textContent=map[m][1];
  const chip=$('#modechip');
  chip.classList.remove('flash');void chip.offsetWidth;chip.classList.add('flash');
}

/* ---------- build ---------- */
function buildTabs(){
  $('#tabs').innerHTML=TABS.map(t=>'<button class="tab" role="tab" style="--cc:'+t.c+'" data-view="'+t.v+
    '" aria-selected="'+(t.v===state.view)+'">'+icon(t.i)+'<span>'+t.t+'</span></button>').join('');
}
function makeTile(b,i){
  const d=b.data,c=b.cal;
  const pct=Math.round(d.fill);
  const full=pct>=c.fill_th,tamper=d.status==='TAMPER';
  const bm=binMode(b);
  const warn=(d.temp>=c.temp_th||d.humidity>=c.hum_th||d.gas>=c.gas_th)?'<span class="twarn">!</span>':'';
  const barCol=full?'var(--bad)':pct>c.fill_th*0.7?'var(--warn)':colorOf(i);
  const statusTxt=tamper?'TAMPER':full?'FULL':d.status==='CALIBRATING'?'CALIB':d.lid==='OPEN'?'LID OPEN':'OK';
  const statusCls=tamper||full?'tstatus alert-status':'tstatus';
  return '<div class="tile" id="tile-'+b.id+'" data-bm="'+bm+'" data-tid="'+b.id+'"'+((full||tamper)?' style="outline:2.5px
solid var(--bad);outline-offset:-2px"':'')+'>'+
    warn+
    ICONS.bin(b.id,pct,colorOf(i),'tclip')+
```

```
    '<span class="tfill" style="color:'+colorOf(i)+'">'+pct+'%</span>'+
    '<span class="tname">'+esc(b.name)+'</span>'+
    '<span class="tplace">'+esc(b.place||'')+'</span>'+
    '<div class="tbar"><i style="width:'+clamp(pct,0,100)+'%;background:'+barCol+'"></i></div>'+
    '<span class="'+statusCls+'">'+statusTxt+'</span>'+
    '</div>';
}
function buildCards(){
  const op=state.role==='operator';
  $('#bins').innerHTML=state.bins.length?state.bins.map((b,i)=>cardHTML(b)+makeTile(b,i)).join(''):'<div class="nm" style="grid-column:1/-1;padding:28px 24px;border-radius:24px"><h2 style="margin:0 0 8px">No bins linked yet</h2>'+
     '<p class="hint" style="margin:0">Open this page from the bin itself (type its IP address in the browser, for example http://192.168.0.112) and it links automatically. '+
     'Or switch to operator, add a bin and type its IP address. To look around with made-up bins, open the START PAGE (Settings) and turn DEMO on.</p></div>';
  $('#addRow').classList.toggle('hide',!op);
  $('#addNote').textContent=state.bins.length?'':'No bins yet.';
  const bt=$('#binsTitle');
  if(bt)bt.innerHTML=icon('bins')+'<span>BINS &middot; '+state.bins.length+'</span>';
  if(reduce())return;
  Array.from($('#bins').children).forEach((el,i)=>{
    el.style.animationDelay=(i*75)+'ms';el.classList.add('enter');
  });
  applyOvFilter();
  applyOvView();
}
function cardHTML(b){
  return '<article class="card nm" id="card-'+b.id+'" data-bin="'+b.id+'">'+
    '<div class="cardtop">'+
      '<div class="hero" data-r="hero"></div>'+
      '<div class="who"><div class="binname">'+esc(b.name)+'</div><div class="binplace">'+esc(b.place)+'</div></div>'+
      '<div style="display:flex;flex-direction:column;align-items:flex-end;gap:10px">'+
        '<div class="fillnum" data-r="pct">0<span>%</span></div>'+
        (state.role==='operator'?'<button class="gearbtn" data-edit="'+b.id+'" aria-expanded="false" title="Bin settings">'+icon('gear')+'<em>SET</em></button>':'')+
      '</div>'+
    '</div>'+
    '<div class="chips" data-r="chips"></div>'+
    '<div class="bar nm-deep"><i data-r="bar"></i><b style="left:50%"></b><b data-r="th"></b></div>'+
    '<div class="ticks"><span>0</span><span>50</span><span data-r="tl">80</span><span>100</span></div>'+
    '<p class="faultnote hide" data-r="fault">Fill sensor not answering. The bar shows the last reading.</p>'+
    '<div class="cardfoot"><span class="h1">1 h</span><svg data-r="spark" viewBox="0 0 100 22" preserveAspectRatio="none"><path/></svg><span data-r="ago"></span></div>'+
    '<div class="sensors" data-r="sensors"></div>'+
    '<div class="actions" data-r="actions"></div>'+
    '<div class="editrow nm-in hide" data-r="edit">'+
      '<div class="field"><label>Name</label><input type="text" data-k="name" value="'+esc(b.name)+'"></div>'+
      '<div class="field"><label>Location</label><input type="text" data-k="place" value="'+esc(b.place)+'"></div>'+
      '<div class="field"><label>IP address</label><input type="text" data-k="ip" value="'+esc(b.ip)+'" placeholder="192.168.1.101"></div>'+
      '<div class="field" style="display:flex;align-items:flex-end;gap:10px">'+
        '<button class="btn sm accent" data-save="'+b.id+'">Save</button>'+
        '<button class="btn sm danger" data-remove="'+b.id+'">Remove</button></div>'+
    '</div></article>';
}
/* The last hour of fill as a small line, and how long ago the last reading came, as in the app. */
function cardFoot(b,svg,agoEl,col){
  const now=Date.now(),H=3600000,pts=(b.hist||[]).filter(h=>now-h.t<=H);
  const d=pts.length>1?pts.map((h,i)=>(i?'L':'M')+((h.t-(now-H))/H*100).toFixed(2)+' '+(22-clamp(h.fill,0,100)/100*22).toFixed(2)).join(''):'';
  const path=svg.firstChild;if(path.getAttribute('d')!==d)path.setAttribute('d',d);path.style.stroke=col;
  const at=isLive(b)?b.lastAt:(b._demoLink==='down'?b._quietAt:now);
  const s=at?Math.round((now-at)/1000):null;
  agoEl.textContent=s==null?'No reading yet':s<5?'Updated just now':'Updated '+ago(at);
  agoEl.classList.toggle('old',isLive(b)?!b.live:b._demoLink==='down');
}
function esc(s){return String(s).replace(/[&<>"]/g,c=>({'&':'&amp;','<':'&lt;','>':'&gt;','"':'&quot;'}[c]))}

/* ---------- render ---------- */
let cardSig='';
// Redraws whatever changed: cards, tiles, banners, priority, alerts, link chip.
function render(){
  const sig=state.bins.map(b=>b.id).join(',')+'|'+state.role;
  if(sig!==cardSig){cardSig=sig;buildCards();}
  state.bins.forEach(renderCard);
  applyMode();renderCollectNext();renderBanners();renderAlerts();renderLink();
  if(state.view==='charts')drawCharts();
  if(state.view==='settings'){updateSettingsLive();demoPanelRender();}
  /* live-update overlay clone in place (no re-clone, no flicker) */
  if(state.ovTile){const ov=$('#tileOverlay');if(ov&&!ov.classList.contains('hide'))renderOverlayCard();}

}
// One bin card. Each part (wave, bar, sensor tiles, chips, keys) is updated in
// place instead of rebuilding the card, so animations are not restarted.
function renderCard(b){
```

```
  const root=$('#card-'+b.id);if(!root)return;
  const d=b.data,c=b.cal,bi=state.bins.indexOf(b);
  const bcol=bi>=0?colorOf(bi):statusColor(d.status,d.fill,c);
  const q=r=>root.querySelector('[data-r="'+r+'"]');
  root.classList.toggle('hot',d.status!=='OK'||d.fill>=c.fill_th);

  root.classList.toggle('urgent',d.status==='TAMPER'||d.fill>=c.fill_th);

  const hero=q('hero'),kind=d.status==='CALIBRATING'?'cal':'bin';
  if(hero.dataset.kind!==kind){
    hero.dataset.kind=kind;
    hero.innerHTML=kind==='cal'?'<span style="color:var(--cal)">'+ICONS.calib()+'</span>':ICONS.bin(b.id,d.fill,bcol);
  }
  if(kind==='bin'){
    const svg=hero.querySelector('svg'),wrap=hero.querySelector('.wavewrap');
    if(svg)svg.style.color=bcol;
    if(wrap)wrap.setAttribute('transform','translate(0,'+(42-(clamp(d.fill,0,100)/100)*30).toFixed(1)+')');
  }

  tweenPct(q('pct'),b,d.fill);
  q('fault').classList.toggle('hide',!(d.faults||[]).includes('HC-SR04'));
  cardFoot(b,q('spark'),q('ago'),levelColor(d.fill,c));
  const bar=q('bar');bar.style.width='calc('+clamp(d.fill,0,100)+'% - 6px)';bar.style.background=levelColor(d.fill,c);
  q('th').style.left=c.fill_th+'%';q('tl').textContent=c.fill_th;

  const bm=binMode(b);
  root.dataset.bmode=bm;
  const simOn=b.sim.mask>0,calOn=bm==='cal'||bm==='dual';
  const tilting=d.tamper_phase==='COUNTDOWN'&&d.status!=='TAMPER'&&d.status!=='CALIBRATING';
  const csig=[d.lid,d.status,d.motion,simOn,calOn,b.live,b.route,isLive(b),b._demoLink,tilting].join('|');
  const ch=q('chips');
  if(ch.dataset.sig!==csig){
    ch.dataset.sig=csig;
    /* CALIBRATING is shown by the CAL chip only, the same way SIM shows simulation.
       The status chip keeps reporting the underlying bin state. */
    const stKey=d.status==='CALIBRATING'?'OK':d.status;
    const stIco=stKey==='TAMPER'?icon('tamper'):stKey==='LOCKED'?icon('lock',true)
      :stKey==='COLLECTING'?icon('truck'):icon('check');
    const stCol={OK:'var(--ok)',LOCKED:'var(--sim)',TAMPER:'var(--c-tamper)',
      COLLECTING:'var(--cal)'}[stKey]||'var(--ink2)';
    const linkOk=b.live||!b.ip;
    const st=STATUS_SHORT[stKey]||[stKey,stKey];
    ch.innerHTML=[
      slot(icon('lid',d.lid==='OPEN'),d.lid==='OPEN'?'OPEN':'CLOSED',d.lid==='OPEN','var(--c-lid)',
        d.lid==='OPEN'?'Lid is open':'Lid is closed'),
      tilting?slot(icon('tamper'),'TILTED',true,'var(--warn)','Tipped over. Tamper alert if it stays tipped'):slot(stIco,st[0],true,stCol,st[1]),
      slot.apply(null,linkSlot(b)),
      slot(icon('flask'),'SIM',simOn,'var(--sim)',simOn?'Simulation is on':'Simulation is off'),
      slot(icon('calib'),'CAL',calOn,'var(--cal)',calOn?'Calibration mode is on':'Calibration mode is off'),
      slot(icon('motion',d.motion),'MOTION',d.motion,'var(--c-motion)',
        d.motion?'Motion detected':'No motion')
    ].join('');
  }

  const host=q('sensors');
  if(!host.dataset.built){
    host.dataset.built='1';
    host.innerHTML=[['temp','Temperature'],['humid','Humidity'],['weight','Weight'],['gas','Air quality']]
      .map(s=>'<div class="sensor nm-in" data-s="'+s[0]+'">'+icon(s[0])+'<span class="v">--</span><span class="k">'+s[1]+'</span></div>').join('');
  }
  const band=(f)=>f>=1?'crit':f>=.8?'high':f>=.5?'mid':'low';
  const vals={
    temp:[fmt(d.temp,1)+'°',d.temp>=c.temp_th,d.temp<22?'cool':band(d.temp/c.temp_th)],
    humid:[Math.round(d.humidity)+'%',d.humidity>=c.hum_th,d.humidity>=c.hum_th?'crit':d.humidity>=c.hum_th-10?'high':d.humidity>=60?'mid':d.humidity<=35?'cool':'low'],
    weight:[(d.weight_g/1000).toFixed(2)+'kg',d.weight_g>=c.weight_cap,band(d.weight_g/(c.weight_cap||4500))],
    gas:[String(Math.round(d.gas)),d.gas>=c.gas_th,band(d.gas/(c.gas_th||200))]
  };
  const warming=(isLive(b)||b.demo)&&d.iaq_acc===0&&!(b.sim.mask&4);
  if(warming){
    const l=d.warmEnd?Math.max(0,Math.round((d.warmEnd-Date.now())/1000)):null;
    vals.gas=[l?Math.floor(l/60)+':'+String(l%60).padStart(2,'0'):'...',false,'low'];
  }
  /* A sensor that stops answering shows -- and says so, instead of an old number. */
  const F=d.faults||[],dead={temp:F.includes('BME680'),humid:F.includes('BME680'),gas:F.includes('BME680'),weight:F.includes('HX711')};
  const lbl={temp:'Temperature',humid:'Humidity',weight:'Weight',gas:warming?'Air warming up':'Air quality'};
  for(const k in dead)if(dead[k]){vals[k]=['--',true,'crit'];lbl[k]='Not answering';}
  host.querySelectorAll('[data-s]').forEach(el=>{
    const kk=el.querySelector('.k');if(kk&&lbl[el.dataset.s]&&kk.textContent!==lbl[el.dataset.s])kk.textContent=lbl[el.dataset.s];
  });
  host.querySelectorAll('[data-s]').forEach(el=>{
```

```
    const v=vals[el.dataset.s];if(!v)return;
    const t=el.querySelector('.v');if(t.textContent!==v[0])t.textContent=v[0];
    el.dataset.lv=v[2];
    el.classList.toggle('alarm',v[1]);el.classList.toggle('nm-in',!v[1]);
  });
  tuneSensorIcons(root,d,c);
  updateTiles();

  const op=state.role==='operator',busy=d.status==='CALIBRATING';
  const locked=d.status==='LOCKED',tamper=d.status==='TAMPER',collecting=d.status==='COLLECTING';
  const want=[];
  /* LOCK and UNLOCK travel together: an action you can do but cannot undo
     strands a collector who locked a bin with no operator around. RESET TAMPER
     is operator only and is left out of the collector's card entirely, since a
     control they can never use is just noise. */
  want.push(locked
    ?['UNLOCK','UNLOCK','solid','lock',!busy,'var(--sim)']
    :['LOCK','LOCK','tint','lock',!(busy||tamper),'var(--sim)']);
  if(op)want.push(['CLEAR_TAMPER','RESET TAMPER',tamper?'danger':'tint','tamper',tamper,'var(--c-tamper)']);
  want.push(collecting
    ?['COLLECT_DONE','DONE','solid','check',true,'var(--cal)']
    :['COLLECT','COLLECT','tint','truck',!busy,'var(--c-link)']);
  const asig=want.map(w=>w.join('|')).join(',');
  const act=q('actions');
  if(act.dataset.sig!==asig){
    act.dataset.sig=asig;
    act.innerHTML=want.map(w=>{
      const tip=w[0]==='CLEAR_TAMPER'&&!tamper?' title="Nothing to reset on this bin"':'';
      return '<button class="btn '+w[2]+'" style="--cc:'+w[5]+'" data-cmd="'+w[0]+
        '" data-bin="'+b.id+'"'+(w[4]?'':' disabled')+tip+'>'+icon(w[3],w[0]==='UNLOCK')+
        '<span>'+w[1]+'</span></button>';
    }).join('');
  }
}
/* Every chip is one grid cell of the same size, so the label has to fit on one
   line even in the narrowest card (about 94px per cell with 3 bins at 1100px).
   The long words are shortened and the full wording moves to the tooltip. */
const STATUS_SHORT={OK:['OK','Bin is OK'],FULL:['FULL','Bin is full'],
  LOCKED:['LOCKED','Lid is locked'],TAMPER:['TAMPER','Bin was moved or tilted'],
  COLLECTING:['PICKUP','Collection in progress']};
function slot(ic,txt,active,cc,tip){
  return '<span class="chip '+(active?'on':'off')+'" style="--cc:'+(cc||'var(--ink2)')+'"'+
    (tip?' title="'+tip+'"':'')+'>'+ic+'<span>'+txt+'</span></span>';
}
function chip(ic,txt,bg){
  return '<span class="chip'+(bg?' loud':'')+'"'+(bg?' style="background:'+bg+'"':'')+'>'+ic+'<span>'+txt+'</span></span>';
}
function tuneSensorIcons(root,d,c){
  const merc=root.querySelector('.merc');
  if(merc){const f=clamp((d.temp-
10)/40,0.05,1),h=(6*f+1.4).toFixed(1);merc.setAttribute('height',h);merc.setAttribute('y',(15.4-h).toFixed(1));}
  const nd=root.querySelector('.needle');
  if(nd)nd.style.transform='rotate('+(clamp(d.weight_g/(c.weight_cap||4500),0,1.1)*150-75).toFixed(1)+'deg)';
}

// Fill rate in %/h over the last 15 minutes of history (needs 3 points);
// drives "time to full" and Collection priority.
function fillRate(b){
  const h=b.hist;if(h.length<3)return 0;
  const last=h[h.length-1];let i=h.length-1;
  while(i>0&&last.t-h[i].t<9e5)i--;
  const span=(last.t-h[i].t)/36e5;
  return span<=0.02?0:(last.fill-h[i].fill)/span;
}
/* renderPriority / renderTTF merged into renderCollectNext */
let collectSig='';
function ttfText(b){
  const r=fillRate(b),left=b.cal.fill_th-b.data.fill;
  if(left<=0)return {txt:'Full now',tc:'var(--bad)',tag:'NOW'};
  if(r<=0.05)return {txt:'Not filling',tc:'var(--ink2)',tag:'IDLE'};
  const h=left/r;
  const txt=h<1?'~'+Math.round(h*60)+' min':'~'+h.toFixed(1)+' h';
  if(h<2)return {txt,tc:'var(--bad)',tag:'SOON'};
  if(h<8)return {txt,tc:'var(--warn)',tag:'TODAY'};
  return {txt,tc:'var(--ok)',tag:'LATER'};
}
function collectUrgency(b){
  /* lower = more urgent */
  const p=b.data.fill,th=b.cal.fill_th;
  if(p>=th)return -1000+p;
  const r=fillRate(b);
  if(r>0.05)return (th-p)/r;   /* hours to full */
  return 9999-p;
}
function renderCollectNext(){
  if(!state.bins.length){
```

```
    const h=$('#collectNext');if(h&&collectSig!=='empty'){h.innerHTML='<p class="hint">No bins to collect from yet.</p>';collectSig='empty';}
    const t=$('#collectToggle');if(t)t.classList.add('hide');
    return;
  }
  const rows=state.bins.slice().sort((a,b2)=>collectUrgency(a)-collectUrgency(b2));
  const sig=rows.map(b=>b.id+Math.round(b.data.fill)+b.cal.fill_th).join(',')+'|'+state.collectShowAll;
  if(sig===collectSig)return;
  collectSig=sig;
  const N=3;
  const tog=$('#collectToggle');
  if(tog){
    if(rows.length<=N){tog.classList.add('hide');}
    else{tog.classList.remove('hide');tog.textContent=state.collectShowAll?'SHOW LESS':'SHOW ALL';}
  }
  const host=$('#collectNext');if(!host)return;
  host.className=state.collectShowAll?'coll-show':'';
  host.innerHTML=rows.map((b,i)=>{
    const p=Math.round(b.data.fill),th=b.cal.fill_th;
    const {txt,tc,tag}=ttfText(b);
    const extra=i>=N?'coll-more':'';
    return '<div class="crow '+extra+'" data-pb="'+b.id+'" title="Go to '+esc(b.name)+'">'+
      '<span class="cn">'+(i+1)+'</span>'+
      '<span style="width:9px;height:9px;border-radius:50%;background:'+tc+';flex-shrink:0"></span>'+
      '<span class="cnm">'+esc(b.name)+'</span>'+
      '<span class="cpc">'+p+'%</span>'+
      '<span class="cwhen" style="color:'+tc+'">'+txt+'</span>'+
      '<span class="lvtag" style="background:'+tc+'">'+tag+'</span></div>';
  }).join('')||'<div class="empty">No bins yet.</div>';
}
let banSig='';
function renderBanners(){
  const out=[];
  const cal=state.bins.filter(b=>b.data.status==='CALIBRATING');
  const sim=state.bins.filter(b=>b.sim.mask>0);
  const tam=state.bins.filter(b=>b.data.status==='TAMPER');
  const full=state.bins.filter(b=>b.data.fill>=b.cal.fill_th);
  if(cal.length)out.push(ban('teal','calib','CALIBRATING: '+cal.map(b=>b.name).join(', '),'Keep the bin empty and do not touch it.'));
  if(tam.length)out.push(ban('red','tamper','TAMPER: '+tam.map(b=>b.name).join(', '),'Check the bin before resetting the alert.'));
  if(full.length)out.push(ban('red','truck','COLLECT NOW: '+full.map(b=>b.name+' at '+Math.round(b.data.fill)+'%').join(', '),''));
  if(sim.length)out.push(ban('purple','flask','SIMULATION MODE','Some readings are set by hand, not by the sensors.'));
  if(!state.bins.length)out.push(ban('amber','wifi','No bins linked yet','Add a bin and type its IP address, or turn DEMO on at the start page to look around with made-up bins.'));
  else if(state.demo)out.push(ban('amber','demo','Demo mode','Bins marked DEMO show made-up readings. Press DEMO in the top bar to go back to the start page and turn it off.'));
  if(state.demo&&TOUR.msg)out.unshift(ban('purple','demo','Demo tour: '+TOUR.msg,'The tour runs by itself. Press a key on a demo bin and it waits a minute for you.'));
  const flt=state.bins.filter(b=>(b.data.faults||[]).length);
  if(flt.length)out.push(ban('amber','gear','SENSOR NOT ANSWERING: '+flt.map(b=>b.name+' ('+b.data.faults.join(', ')+')').join(', '),'Its readings show -- until the sensor answers again. Check the wiring.'));
  if(state.bins.some(isLive)){const down=state.bins.filter(b=>isLive(b)&&!b.live);
    if(down.length)out.push(ban('red','wifi',down.length===1?down[0].name+' is not answering':down.length+' bins are not answering','Readings shown are the last ones received. Check that the bin is on and on the same Wi-Fi, or set up Firebase or ThingSpeak.'));}

  const unread=unreadCount(),pending=state.alerts.filter(a=>!a.read);
  let html='';
  if(unread){
    html+='<div class="popsec">Unread alerts</div>'+
      pending.slice(0,6).map(a=>ban((SEV_TONE[a.sev]||SEV_TONE.info)[2],a.ic,a.title,a.meta+' · '+clock(a.t))).join('')+
      (unread>6?'<div class="popsec">and '+(unread-6)+' more in the Alerts tab</div>':'')+
      '<div class="popact"><button class="btn sm tint" style="--cc:var(--ok)" id="popRead">'+icon('check')+
      '<span>MARK ALL READ</span></button><button class="btn sm tint" style="--cc:var(--c-tamper)" id="popOpen">'+
      icon('bell')+'<span>OPEN ALERTS TAB</span></button></div>';
  }
  if(out.length)html+='<div class="popsec">Right now</div>'+out.join('');
  if(html!==banSig){
    banSig=html;
    $('#bannerPop').innerHTML=html||'<div class="empty">Nothing needs attention.</div>';
  }

  const pill=$('#statusPill'),n=out.length;
  pill.classList.toggle('hide',!(unread||n));
  if(!(unread||n)){
    $('#bannerPop').classList.add('hide');
    pill.setAttribute('aria-expanded','false');
    lastUnread=0;return;
  }
  let pIco,pCol,pTxt;
  if(unread){
    const tone=SEV_TONE[SEV_RANK.find(s=>pending.some(a=>a.sev===s))]||SEV_TONE.info;
    pIco=tone[0];pCol=tone[1];pTxt=unread+' unread';
  }else{
```

```
    const w=cal.length?['calib','var(--cal)']:tam.length?['tamper','var(--c-tamper)']
      :full.length?['truck','var(--c-tamper)']:sim.length?['flask','var(--sim)']:['wifi','var(--warn)'];
    pIco=w[0];pCol=w[1];pTxt=n+' active';
  }
  pill.style.setProperty('--cc',pCol);
  $('#pillIco').innerHTML=icon(pIco);
  $('#pillText').textContent=pTxt;
  pill.title=unread?'Unread alerts. Click to see them.':'Live conditions. Click to see them.';
  if(unread>lastUnread&&!reduce()){
    pill.classList.remove('flash');void pill.offsetWidth;pill.classList.add('flash');
  }
  lastUnread=unread;
}
function ban(cls,ic,t,s){return '<div class="banner
'+cls+'">'+icon(ic)+'<span>'+t+'</span>'+(s?'<small>'+s+'</small>':'')+'</div>'}
let alertSig='';
/* History tools from Garbage-Guard: one bar of symbols filters by kind, a second bar by date,
   and MARK READ and DELETE act on exactly the alerts shown. */
const AKINDS=[['all','layers','All','var(--accent)'],['full','fill','Full bins','var(--bad)'],['sensor','temp','Sensor
readings','var(--c-temp)'],['tamper','tamper','Tamper','var(--c-tamper)'],
  ['lock','lock','Lock and unlock','var(--sim)'],['pickup','truck','Pickups','var(--c-link)'],['setup','calib','Calibration
and settings','var(--cal)']];
const AKNAME={all:['alert','alerts'],full:['full-bin alert','full-bin alerts'],sensor:['sensor alert','sensor
alerts'],tamper:['tamper alert','tamper alerts'],
  lock:['lock event','lock events'],pickup:['pickup','pickups'],setup:['setup event','setup events']};
const ARANGE=[['today','Today'],['7','7 days'],['30','30 days'],['any','Any']];
function rangeStart(){const r=state.arange,d=new Date();if(r==='today'){d.setHours(0,0,0,0);return +d}if(r==='any')return
0;return Date.now()-(+r)*86400000}
function shownAlerts(){const s=rangeStart();return state.alerts.filter(a=>(state.akind==='all'||a.k===state.akind)&&a.t>=s)}
function scopeTxt(n){const nm=AKNAME[state.akind]||AKNAME.all,w=n===1?nm[0]:nm[1],r={today:' from today','7':' from the last
7 days','30':' from the last 30 days',any:''}[state.arange];
  return (state.akind==='all'&&state.arange==='any'?'all '+n+' '+w:n+' '+w)+r}
function renderAlertTools(){
  const kh=AKINDS.map(k=>'<button class="fchip'+(state.akind===k[0]?' on':'')+'" data-akind="'+k[0]+'" title="'+k[2]+'" aria-
label="'+k[2]+'" style="--fc:'+k[3]+'">'+icon(k[1])+'</button>').join('');
  if($('#alertKinds').dataset.sig!==kh){$('#alertKinds').dataset.sig=kh;$('#alertKinds').innerHTML=kh;}
  const rh=ARANGE.map(r=>'<button class="btn sm binbtn" data-arange="'+r[0]+'" aria-pressed="'+(state.arange===r[0])+'"
style="--cc:var(--c-tamper)">'+r[1]+'</button>').join('');
  if($('#alertRange').dataset.sig!==rh){$('#alertRange').dataset.sig=rh;$('#alertRange').innerHTML=rh;}
}
function markShown(){
  const s=shownAlerts().filter(a=>!a.read);if(!s.length)return;
  s.forEach(a=>{a.read=true});alertSig='';banSig='';renderAlerts();renderBanners();
  toast('Marked '+s.length+' alert'+(s.length>1?'s':'')+' as read.');
}
function askDelete(){
  const n=shownAlerts().length;if(!n)return;
  $('#alertConfirmText').textContent='Delete '+scopeTxt(n)+'? This cannot be undone.';
  $('#alertYes').innerHTML=icon('trash')+'<span>DELETE '+n+'</span>';
  $('#alertConfirm').classList.remove('hide');
}
function doDelete(){
  const s=shownAlerts(),ids=new Set(s.map(a=>a.id)),t=scopeTxt(s.length);
  state.alerts=state.alerts.filter(a=>!ids.has(a.id));
  $('#alertConfirm').classList.add('hide');alertSig='';banSig='';renderAlerts();renderBanners();
  toast('Deleted '+t+'.');
}
function renderAlerts(){
  const list=$('#alertList'),unread=unreadCount(),shown=shownAlerts(),unShown=shown.filter(a=>!a.read).length;
  renderAlertTools();
  const mk=$('#btnMarkAll');if(mk)mk.disabled=!unShown;
  const dl=$('#btnClear');if(dl)dl.disabled=!shown.length;
  const sc=$('#alertScope'),kn=(AKINDS.find(k=>k[0]===state.akind)||AKINDS[0])[2];
  const sh=state.alerts.length?(shown.length?'Showing '+scopeTxt(shown.length)+'<small>'+kn+(unShown?', '+unShown+'
unread':', all read')+'. MARK READ and DELETE act on exactly these.</small>':'No alerts match.<small>Change the kind or the
dates.</small>'):'';
  if(sc&&sc.dataset.sig!==sh){sc.dataset.sig=sh;sc.innerHTML=sh;}
  const hd=$('#alertHead');
  if(hd){
    const hh=icon('clock')+'<span>ALERT HISTORY</span>'+(unread?'<em class="unreadpip">'+unread+' UNREAD</em>':'');
    if(hd.dataset.sig!==hh){hd.dataset.sig=hh;hd.innerHTML=hh;}
  }
  if(!state.alerts.length){
    if(alertSig!=='@empty'){alertSig='@empty';
      list.innerHTML='<div class="empty">No alerts yet. This fills up as bins reach their limits.</div>';}
    return;
  }
  const sv={critical:['var(--bad)','CRITICAL'],warning:['var(--warn)','WARNING'],
    info:['var(--sim)','INFO'],success:['var(--ok)','RESOLVED']};
  const visible=shown;
  const html=visible.map(a=>{
    const s=sv[a.sev]||['var(--ink2)','NOTE'];
    const lbl=a.read?'Mark as unread':'Mark as read';
    return '<div class="alert" data-lv="'+a.sev+'" data-read="'+(a.read?1:0)+'" style="--cc:'+s[0]+'">'+
      '<button class="rd" data-read-toggle="'+a.id+'" title="'+lbl+'" aria-label="'+lbl+'"><i></i></button>'+
      '<span class="av" style="color:'+(a.sev==='critical'?'#fff':s[0])+'">'+icon(a.ic)+
```

```
      '</span><span style="flex:1;min-width:0"><span class="ttl">'+a.title+'</span><div class="meta">'+a.meta+' &middot;
'+ago(a.t)+'</div></span>'+
      '<span class="lvtag" style="'+(a.sev==='critical'?'':'background:'+s[0])+'">'+s[1]+'</span>'+
      '<button class="del" data-del="'+a.id+'" title="Remove this alert">'+icon('cross')+'</button></div>';
  }).join('')||(visible.length===0&&state.alerts.length?'<div class="empty">No alerts match. Change the kind or the
dates.</div>':'');
  if(html===alertSig)return;
  alertSig=html;list.innerHTML=html;
}
function ago(t){const s=Math.round((Date.now()-t)/1000);return s<60?s+' s ago':s<3600?Math.round(s/60)+' min
ago':s<172800?Math.round(s/3600)+' h ago':Math.round(s/86400)+' days ago'}
function renderLink(){
  const used={local:0,fb:0,ts:0};let live=0,cfg=0;
  state.bins.forEach(b=>{if(isLive(b))cfg++;if(b.live){live++;if(b.route)used[b.route]++;}});
  const names=Object.keys(used).filter(k=>used[k]);
  $('#wifiico').innerHTML=names.length===1&&names[0]!=='local'?icon(ROUTE[names[0]][2]):icon('wifi',live>0);
  const link=$('#link');
  link.style.setProperty('--lc',names.length?ROUTE[names[0]][1]:'var(--ink2)');
  link.style.color=names.length?ROUTE[names[0]][1]:'';
  link.title=names.map(k=>ROUTE[k][0]+': '+used[k]+' bin'+(used[k]>1?'s':'')).join(', ');
  $('#linktext').textContent=!state.bins.length?'No bins':!cfg?'Demo data':!live?'Not connected':
    names.length===3?'All 3 links':names.map(k=>ROUTE[k][0]).join(' + ')+(live<cfg?' ('+live+' of '+cfg+')':'');
}

/* ---------- charts ---------- */
// Charts are drawn by hand on <canvas> (no chart library), see line().
function drawCharts(){
  const since=Date.now()-state.rangeMin*60000;
  const picked=state.bins.filter(b=>state.chartPick.includes(b.id));
  let show=picked.length?picked:state.bins;
  /* adaptive fill selector — sig check prevents rebuild from closing an open dropdown */
  const fillSegHtml=adaptiveSeg('fillSeg','pick',state.chartPick,true);
  if($('#fillSeg').dataset.sig!==fillSegHtml){$('#fillSeg').dataset.sig=fillSegHtml;$('#fillSeg').innerHTML=fillSegHtml;}
  /* cap ALL BINS display at 5 fullest */
  let capped=false;
  if(!state.chartPick.length&&state.bins.length>5){
    show=state.bins.slice().sort((a,b2)=>b2.data.fill-a.data.fill).slice(0,5);
    capped=true;
  }
  const series=show.map(b=>({color:colorOf(state.bins.indexOf(b)),dash:dashOf(state.bins.indexOf(b)),
    mark:b.cal.fill_th,pts:b.hist.filter(p=>p.t>=since).map(p=>({t:p.t,v:p.fill}))}));
  const uniqTh=[...new Set(show.map(x=>x.cal.fill_th))];
  const fillMarks=[];
  show.forEach(x=>{
    const txt=(uniqTh.length>1?x.name.toUpperCase()+' LIMIT ':'ALERT LIMIT ')+x.cal.fill_th+'%';
    if(!fillMarks.some(m=>m.text===txt))
      fillMarks.push({v:x.cal.fill_th,color:getVar('--bad'),text:txt,dash:[7,6]});
  });
  line($('#cFill'),series,0,100,fillMarks);
  $('#legFill').innerHTML=show.map(b=>{const i=state.bins.indexOf(b);
    return '<span><i style="background:'+colorOf(i)+';'+(dashOf(i)?'opacity:.95;background:repeating-linear-
gradient(90deg,'+colorOf(i)+' 0 5px,transparent 5px 9px)':'')+'"></i>'+esc(b.name)+'</span>';
  }).join('')+'<span><i style="background:repeating-linear-gradient(90deg,var(--bad) 0 6px,transparent 6px
11px)"></i>Threshold</span>';
  let cn=$('#fillCappedNote');if(!cn){cn=document.createElement('p');cn.id='fillCappedNote';cn.className='chart-
note';$('#legFill').parentElement.appendChild(cn);}
  cn.textContent=capped?'Showing 5 fullest of '+state.bins.length+' bins. Pick a bin above to see others.':'';

  const b=byId(state.envBin)||state.bins[0];
  const SHORT=['ALL','TEMPERATURE','HUMIDITY','AIR QUALITY','PRESSURE'];
  const modeHtml=ENV_MODES.map((m,mi)=>'<button class="btn sm binbtn" style="--cc:'+m[3]+'" data-envmode="'+m[0]+
    '" aria-pressed="'+(state.envMode===m[0])+'">'+icon(m[2])+'<span>'+SHORT[mi]+'</span></button>').join('');
  if($('#envMode').dataset.sig!==modeHtml){$('#envMode').dataset.sig=modeHtml;$('#envMode').innerHTML=modeHtml;}
  const segHtml=adaptiveSeg('envSeg','env',[b&&b.id],false);
  if($('#envSeg').dataset.sig!==segHtml){$('#envSeg').dataset.sig=segHtml;$('#envSeg').innerHTML=segHtml;}
  if(!b)return;
  drawEnv(b,since);
}
const peak=pts=>pts.reduce((m,p)=>Math.max(m,p.v),0);
function niceMax(v){
  const steps=[5,10,15,20,25,50,100,150,200,250,500,1000,1250,2000,2500,5000];
  for(const s of steps)if(s*4>=v)return s*4;
  return Math.ceil(v/4000)*4000;
}
/* One reading at a time is drawn in its real unit, on the BME680 datasheet range (see SPEC).
   ALL would need several y-scales, which cannot be read honestly, so BOTH draws temperature,
   humidity and air quality as a share of their own limit (one dashed line at 100% is the limit
   for all three). Pressure has no limit, so it only has its own tab. */
const HKEY={temp:'temp',humid:'humidity',press:'pressure',gas:'gas'};
function drawEnv(b,since){
  const h=b.hist.filter(p=>p.t>=since),d=b.data;
  const tTh=b.cal.temp_th||40,hTh=b.cal.hum_th||85,gTh=b.cal.gas_th||200;
  const tc=getVar('--c-temp'),gc=getVar('--c-gas'),hc=getVar('--c-humid'),bad=getVar('--bad');
  const dashed=(c,a,b2)=>'repeating-linear-gradient(90deg,'+c+' 0 '+a+'px,transparent '+a+'px '+b2+'px)';
  let series,marks,min=0,max,ticks,note,leg='';
  if(SPEC[state.envMode]){
```

```
    const k=state.envMode,sp=SPEC[k],col=getVar(sp.col),key=HKEY[k];
    const pts=h.filter(p=>Number.isFinite(p[key])).map(p=>({t:p.t,v:p[key]}));
    const lim=k==='temp'?tTh:k==='humid'?hTh:k==='gas'?gTh:null;
    min=sp.min;max=sp.max;ticks=sp.ticks;
    series=[{color:col,dash:0,mark:lim,pts:pts}];
    marks=lim==null?[]:[{v:lim,color:bad,text:'ALERT LIMIT '+lim+' '+sp.unit,dash:[7,6]}];
    leg='<span><i style="background:'+col+'"></i>'+sp.name+', now <span class="now">'+fmt(d[key],sp.dec)+'
'+sp.unit+'</span></span>'+
      (lim==null?'':'<span><i style="background:'+dashed(bad,6,11)+'"></i>Alert limit '+lim+' '+sp.unit+'</span>')+
      '<span>Scale '+sp.min+' to '+sp.max+' '+sp.unit+'</span>';
    note={
      temp:'Temperature of '+b.name+' in degrees C, after the offset from Settings. The scale runs to 85 C, the most the
BME680 can read. The dashed line is its alert limit.',
      humid:'Relative humidity of '+b.name+', 0 to 100 percent, the full BME680 range. The dashed line is its alert limit.',
      press:'Air pressure around '+b.name+' in hPa, on the BME680 range of 300 to 1100 hPa, so normal weather changes look
small. Pressure is charted here only. It has no tile on the bin card and raises no alert.',
      gas:'Air quality index of '+b.name+' from the BME680 gas sensor, 0 to 500, lower is cleaner. The dashed line is its
alert limit.'
    }[k];
  }else{
    const tp=h.map(p=>({t:p.t,v:p.temp/tTh*100})),gp=h.map(p=>({t:p.t,v:p.gas/gTh*100}));
    const hp=h.filter(p=>Number.isFinite(p.humidity)).map(p=>({t:p.t,v:p.humidity/hTh*100}));
    max=200;ticks=[0,50,100,150,200]; /* fixed, so the 100% limit always sits in the middle */
    series=[{color:tc,dash:0,mark:100,pts:tp},{color:gc,dash:1,mark:100,pts:gp},{color:hc,dash:2,mark:100,pts:hp}];
    marks=[{v:100,color:bad,text:'ALERT LIMIT (100%)',dash:[7,6]}];
    note='Temperature, humidity and air quality are each drawn as a share of their own alert limit, so the single dashed line
at 100% is the limit for all three. The scale runs to 200%, which keeps that line in the middle. '+
      'Pick one reading to see real units. Pressure has its own tab, last.';
    leg='<span><i style="background:'+tc+'"></i>Temperature, limit '+tTh+' C</span>'+
      '<span><i style="background:'+dashed(hc,2.5,6.5)+'"></i>Humidity, limit '+hTh+' %RH</span>'+
      '<span><i style="background:'+dashed(gc,5,9)+'"></i>Air quality, limit '+gTh+' IAQ</span>'+
      '<span><i style="background:'+dashed(bad,6,11)+'"></i>Alert limit (100%)</span>';
  }
  const lg=$('#legEnv');
  if(lg.dataset.sig!==leg){lg.dataset.sig=leg;lg.innerHTML=leg;}
  lg.classList.toggle('hide',!leg);
  if($('#envNote').textContent!==note)$('#envNote').textContent=note;
  line($('#cEnv'),series,min,max,marks,ticks);
}
function dashOf(i){return [0,1,2,3][i%4]}
function dashPat(d){return d===1?[7,5]:d===2?[2.5,4]:d===3?[11,4,2.5,4]:[]}
let chartT=1,chartRaf=0;
function revealCharts(){
  if(reduce()){chartT=1;drawCharts();return;}
  cancelAnimationFrame(chartRaf);
  const t0=performance.now(),dur=700;
  const run=now=>{
    const k=Math.min(1,(now-t0)/dur);
    chartT=1-Math.pow(1-k,3);
    drawCharts();
    if(k<1)chartRaf=requestAnimationFrame(run);else chartT=1;
  };
  chartRaf=requestAnimationFrame(run);
}
function getVar(n){return getComputedStyle(document.documentElement).getPropertyValue(n).trim()}
function hex2rgb(h){
  h=(h||'').trim().replace('#','');
  if(h.length===3)h=h.split('').map(c=>c+c).join('');
  const n=parseInt(h||'888888',16);
  return [(n>>16)&255,(n>>8)&255,n&255];
}
function rgba(h,a){const c=hex2rgb(h);return 'rgba('+c[0]+','+c[1]+','+c[2]+','+a+')'}
function mixHex(a,b,t){
  const A=hex2rgb(a),B=hex2rgb(b);
  const m=i=>Math.round(A[i]+(B[i]-A[i])*t).toString(16).padStart(2,'0');
  return '#'+m(0)+m(1)+m(2);
}
function line(cv,series,min,max,marks,ticks){
  if(!cv)return;
  const dpr=window.devicePixelRatio||1,w=cv.clientWidth,h=cv.clientHeight;
  cv.width=w*dpr;cv.height=h*dpr;
  const g=cv.getContext('2d');g.setTransform(dpr,0,0,dpr,0,0);g.clearRect(0,0,w,h);
  const ink2=getVar('--ink2'),acc=getVar('--accent');
  const L=40,R=14,T=16,B=24,pw=w-L-R,ph=h-T-B,y=v=>T+ph-((v-min)/(max-min))*ph;
  g.font='11px Inter, system-ui, sans-serif';g.strokeStyle=ink2;g.lineWidth=1;
  const tk=ticks&&ticks.length?ticks:[0,1,2,3,4].map(i=>min+(max-min)*i/4);
  tk.forEach(v=>{g.globalAlpha=.25;g.beginPath();g.moveTo(L,y(v));g.lineTo(w-R,y(v));g.stroke();});
  g.globalAlpha=1;g.fillStyle=ink2;g.textAlign='right';g.textBaseline='middle';
  tk.forEach(v=>{g.fillText(Math.round(v),L-8,y(v));});
  const all=series.flatMap(s=>s.pts);
  if(all.length<2){g.textAlign='center';g.fillText('Collecting data',w/2,T+ph/2);return;}
  const t0=Math.min(...all.map(p=>p.t)),t1=Math.max(...all.map(p=>p.t));
  const x=t=>L+(t1===t0?pw:(t-t0)/(t1-t0)*pw);
  drawMarks(g,marks||[],L,R,w,T,ph,y);
  const hot=getVar('--bad')||'#D93025';
  const baseCol=getVar('--base')||'#E4DCD2';
```

```
  series.forEach(s=>{
    if(s.pts.length<2)return;
    const pts=chartT>=1?s.pts:s.pts.slice(0,Math.max(2,Math.ceil(s.pts.length*chartT)));
    const warm=mixHex(s.color,hot,.82);
    const sMark=s.mark;
    let stroke=s.color,fill;
    if(sMark!=null&&sMark>min&&sMark<max){
      const p=clamp((max-sMark)/(max-min),0,1);
      stroke=g.createLinearGradient(0,T,0,T+ph);
      stroke.addColorStop(0,warm);stroke.addColorStop(Math.max(0,p-0.001),warm);
      stroke.addColorStop(p,s.color);stroke.addColorStop(1,s.color);
      fill=g.createLinearGradient(0,T,0,T+ph);
      fill.addColorStop(0,rgba(warm,.42));fill.addColorStop(Math.max(0,p-0.001),rgba(warm,.26));
      fill.addColorStop(p,rgba(s.color,.30));fill.addColorStop(1,rgba(s.color,0));
    }else{
      fill=g.createLinearGradient(0,T,0,T+ph);
      fill.addColorStop(0,rgba(s.color,.34));fill.addColorStop(1,rgba(s.color,0));
    }
    g.beginPath();
    pts.forEach((p,i)=>{const X=x(p.t),Y=y(clamp(p.v,min,max));i?g.lineTo(X,Y):g.moveTo(X,Y);});
    g.lineTo(x(pts[pts.length-1].t),T+ph);g.lineTo(x(pts[0].t),T+ph);g.closePath();
    g.fillStyle=fill;g.fill();
    const trace=()=>{g.beginPath();
      pts.forEach((p,i)=>{const X=x(p.t),Y=y(clamp(p.v,min,max));i?g.lineTo(X,Y):g.moveTo(X,Y);});};
    g.lineJoin='round';g.lineCap='round';
    g.setLineDash([]);trace();
    g.strokeStyle=rgba(baseCol,.85);g.lineWidth=6.2;g.stroke();
    g.setLineDash(dashPat(s.dash||0));trace();
    g.strokeStyle=stroke;g.lineWidth=2.9;g.stroke();
    g.setLineDash([]);
    const last=pts[pts.length-1],ly=clamp(last.v,min,max);
    const dotCol=(sMark!=null&&last.v>=sMark)?warm:s.color;
    g.fillStyle=rgba(dotCol,.25);g.beginPath();g.arc(x(last.t),y(ly),8,0,7);g.fill();
    g.fillStyle=dotCol;g.beginPath();g.arc(x(last.t),y(ly),3.8+(chartT<1?2.5*(1-chartT):0),0,7);g.fill();
  });
  g.fillStyle=ink2;g.textBaseline='top';g.textAlign='left';g.fillText(clock(t0),L,T+ph+8);
  g.textAlign='right';g.fillStyle=acc;g.fillText('now',w-R,T+ph+8);
}
function roundRect(g,x,y,w,h,r){
  g.beginPath();g.moveTo(x+r,y);g.arcTo(x+w,y,x+w,y+h,r);g.arcTo(x+w,y+h,x,y+h,r);
  g.arcTo(x,y+h,x,y,r);g.arcTo(x,y,x+w,y,r);g.closePath();
}
/* A dashed line on its own does not say which reading it belongs to, so every
   threshold carries a small label chip pinned to the left end of its line. */
function drawMarks(g,marks,L,R,w,T,ph,y){
  const surface=getVar('--well')||'#DED5CA',used=[];
  marks.forEach(m=>{
    if(m.v==null)return;
    const yy=y(m.v);
    if(yy<T-1||yy>T+ph+1)return;
    g.save();
    g.setLineDash(m.dash&&m.dash.length?m.dash:[7,6]);
    g.strokeStyle=m.color;g.lineWidth=1.8;g.globalAlpha=.9;
    g.beginPath();g.moveTo(L,yy);g.lineTo(w-R,yy);g.stroke();
    g.restore();
    if(!m.text)return;
    g.save();
    g.font='700 10px Inter, system-ui, -apple-system, sans-serif';
    const bw=g.measureText(m.text).width+13,bh=16,bx=L+9;
    let by=yy-bh-4;
    if(by<T+1)by=yy+4;
    let guard=0;
    while(used.some(u=>Math.abs(u-by)<bh+3)&&guard++<8)by+=bh+4;
    if(by+bh>T+ph-1)by=Math.max(T+1,yy-bh-4);
    used.push(by);
    roundRect(g,bx,by,bw,bh,5);
    g.fillStyle=surface;g.globalAlpha=.94;g.fill();
    g.globalAlpha=1;g.strokeStyle=m.color;g.lineWidth=1.2;g.stroke();
    g.fillStyle=m.color;g.textAlign='left';g.textBaseline='middle';
    g.fillText(m.text,bx+6.5,by+bh/2+.5);
    g.restore();
  });
}
function clock(t){const d=new Date(t);return String(d.getHours()).padStart(2,'0')+':'+String(d.getMinutes()).padStart(2,'0')}

/* ---------- bin selectors ---------- */
const SEG_THRESH=8;
function segHTML(attr,sel){
  return state.bins.map(b=>'<button class="btn sm binbtn" data-'+attr+'="'+b.id+'" data-sim="'+(b.sim.mask?1:0)+
    '" aria-pressed="'+(b.id===sel)+'">'+icon('fill')+'<span>'+esc(b.name)+'</span></button>').join('');
}
/* adaptiveSeg: for fill and env pickers in charts */
function adaptiveSeg(hostId,attr,selIds,withAll){
  if(state.bins.length<=SEG_THRESH){
    /* button strip */
    const allBtn=withAll?'<button class="btn sm binbtn" data-'+attr+'="ALL" aria-
```

```
    pressed="'+(selIds.length===0||!selIds[0])+'">'+icon('layers')+'<span>ALL BINS</span></button>':'';
3581    return allBtn+state.bins.map((b,i)=>'<button class="btn sm binbtn" style="--cc:'+colorOf(i)+'" data-'+attr+'="'+b.id+'"
    data-sim="'+(b.sim.mask?1:0)+'" aria-
    pressed="'+selIds.includes(b.id)+'">'+icon('fill')+'<span>'+esc(b.name)+'</span></button>').join('');
3582  }
3583  /* picker button + floating list */
3584  const cur=state.bins.find(b=>selIds.includes(b.id));
3585  const label=withAll&&!cur?'ALL BINS':cur?esc(cur.name):'PICK BIN';
3586  const searchRow=state.bins.length>=9?'<input type="text" placeholder="Search bins…" id="'+hostId+'Search" aria-
    label="Search bins">':'';
3587  const items=(withAll?'<button data-'+attr+'="ALL" aria-
    pressed="'+(selIds.length===0||!selIds[0])+'">'+icon('layers')+'<span>ALL BINS</span></button>':'')+
3588    state.bins.map((b,i)=>'<button data-'+attr+'="'+b.id+'" data-sim="'+(b.sim.mask?1:0)+'" aria-
    pressed="'+selIds.includes(b.id)+'"><span class="bcc"
    style="background:'+colorOf(i)+'"></span>'+esc(b.name)+'</button>').join('');
3589  return '<div class="pickwrap" id="'+hostId+'Wrap"><button class="btn sm binbtn"
    id="'+hostId+'Btn">'+icon('fill')+'<span>'+label+'</span></button>'+
3590    '<div class="picklist hide" id="'+hostId+'List">'+searchRow+items+'</div></div>';
3591 }
3592 /* ---------- settings ---------- */
3593 const
    MAP={s_dempty:'d_empty',s_dfull:'d_full',s_fill:'fill_th',s_temp:'temp_th',s_hum:'hum_th',s_gas:'gas_th',s_tilt:'tilt_th',
3594  s_wcap:'weight_cap',s_sclosed:'servo_closed',s_sopen:'servo_open',s_pin:'collect_pin',
3595  s_pirhold:'pir_hold_s',s_buzz:'buzz_s',s_toff:'temp_off',s_hoff:'hum_off',s_poff:'press_off'};
3596 // MAP links each settings input id to its Cal key.
3597 function loadSettings(){
3598  const seg=adaptiveSeg('setSeg','set',[state.selSet],false);
3599  if($('#setSeg').dataset.sig!==seg){$('#setSeg').dataset.sig=seg;$('#setSeg').innerHTML=seg;}
3600  const b=byId(state.selSet);if(!b)return;
3601  for(const k in MAP)$('#'+k).value=b.cal[MAP[k]];
3602  $('#s_autolock').checked=!!b.cal.auto_lock;
3603  sectSummaries(b);
3604  sectAllRender();
3605  refreshCalGate();
3606 }
3607 function allSectsOpen(){return $$('.sect').every(d=>d.classList.contains('open'))}
3608 function sectAllRender(){const b=$('#sectAll');if(!b)return;const o=allSectsOpen();
3609  b.innerHTML=icon('layers')+'<span>'+(o?'HIDE ALL':'SHOW ALL')+'</span>';b.style.display='inline-
    flex';b.style.alignItems='center';b.style.gap='8px';}
3610 function sectSummaries(b){
3611  const c=b.cal,f=(v,d)=>Number(v).toFixed(d),d=b.data||{};
3612  const sum={range:f(c.d_empty,1)+' to '+f(c.d_full,1)+' cm',
3613    limits:f(c.fill_th,0)+'% \u00B7 '+f(c.temp_th,0)+'\u00B0C \u00B7 '+f(c.hum_th,0)+'%RH \u00B7 '+f(c.gas_th,0)+' IAQ',
3614    air:'offset '+f(c.temp_off,1)+'\u00B0C',
3615    lid:'PIN \u2022\u2022\u2022\u2022 \u00B7 '+f(c.pir_hold_s,0)+' s'+(c.auto_lock?' \u00B7 auto-lock':''),
3616    weight:Number.isFinite(d.weight_g)?(d.weight_g/1000).toFixed(2)+' kg now':'',
3617    touch:b.touch&&b.touch.custom!=null?(b.touch.custom?'calibrated':'default values'):''};
3618  $$('.sect').forEach(x=>{const e=x.querySelector('.ssum');if(e)e.textContent=sum[x.dataset.sec]||'';});
3619 }
3620 function calibrating(){const b=byId(state.selSet);return !!(b&&state.calib&&state.calib.bin===b.id)}
3621 function formDirty(){
3622  const b=byId(state.selSet);if(!b)return false;
3623  for(const k in MAP)if(+$('#'+k).value!==+b.cal[MAP[k]])return true;
3624  return ($('#s_autolock').checked?1:0)!==(b.cal.auto_lock?1:0);
3625 }
3626 function refreshDirty(){
3627  const on=state.role!=='collector'&&calibrating(),dirty=on&&formDirty();
3628  const save=$('#btnSave'),cancel=$('#btnCancel');
3629  save.disabled=!dirty;cancel.disabled=!dirty;
3630  save.classList.toggle('solid',dirty);save.classList.toggle('tint',!dirty);
3631 }
3632 /* The collector sees every setting and every bin but edits nothing, so the
3633    calibration switch is replaced by a read-only notice and the write buttons
3634    go away entirely rather than sitting there greyed out. */
3635 function renderPin(){
3636  const card=$('#pinCard'),b=byId(state.selSet),show=!!b;
3637  card.classList.toggle('hide',!show);
3638  if(!show)return;
3639  if(card.dataset.binid!==b.id){card.dataset.binid=b.id;card.dataset.masked='1';}
3640  const masked=card.dataset.masked!=='0';
3641  $('#pinCardIco').innerHTML=icon('pin');
3642  $('#pinName').textContent=b.name;
3643  $('#pinSub').textContent=state.role==='collector'?'Type this on the bin touchscreen when you tap COLLECT.':
3644    'Collectors type this on the bin touchscreen for COLLECT and WI-FI SETUP. Change it under Lid and lock.';
3645  $('#pinVal').textContent=masked?'\u2022\u2022\u2022\u2022':String(b.cal.collect_pin);
3646  $('#pinVal').classList.toggle('masked',masked);
3647  const eye=$('#pinEye');
3648  eye.classList.add('tint');eye.style.setProperty('--cc','var(--cal)');
3649  eye.style.display='inline-flex';eye.style.alignItems='center';eye.style.gap='8px';
3650  eye.innerHTML=icon(masked?'eye':'eyeoff')+'<span>'+(masked?'SHOW':'HIDE')+'</span>';
3651 }
3652 function refreshCalGate(){
3653  const col=state.role==='collector';
3654  const on=!col&&calibrating(),b=byId(state.selSet);
3655  $('#setFields').disabled=!on;
3656  $('#calBar').classList.toggle('hide',col);
```

```
  $('#roNote').classList.toggle('hide',!col);
  $('#roIco').innerHTML=icon('eye');
  ['btnSave','btnCancel','btnExport'].forEach(id=>$('#'+id).classList.toggle('hide',col));
  renderPin();
  refreshDirty();
  $('#calToggle').checked=on;
  $('#calBar').classList.toggle('on',on);
  $('#calIco').innerHTML=icon(on?'calib':'lock',true);
  $('#calTitle').textContent=on?'Calibration mode is on'+(b?' for '+b.name:''):'Settings are locked';
  $('#calSub').textContent=on
    ?'Change what you need, then press SAVE SETTINGS. Switching this off without saving leaves the bin as it was.'
    :'Switch on calibration mode to change anything here. The bin shows its teal calibration screen while it is on.';
  $$('#setSeg .btn').forEach(x=>{x.disabled=on});
  if(col)$('#setMsg').textContent='Read only. Pick any bin above to see its PIN and the limits it is running on.';
  updateSettingsLive();
}
function updateSettingsLive(){
  const b=byId(state.selSet);if(!b)return;
  const live=$('#s_live'),dist=$('#s_dist');
  if(live)live.textContent=(b.data.weight_g/1000).toFixed(2)+' kg';
  if(dist)dist.textContent=fmt(b.data.distance_cm,1)+' cm';
  const tc=$('#s_touch');
  if(tc){
    const t=b.touch;
    tc.textContent=!isLive(b)?'Demo bin, no touchscreen'
      :!t?'Waiting for the bin'
      :(t.custom?'Calibrated on the bin':'Default values')+' \u00B7 X '+t.x0+'\u2013'+t.x1+' \u00B7 Y
'+t.y0+'\u2013'+t.y1+(t.swap?' \u00B7 axes swapped':'');
  }
  const tl=$('#s_tlive'),iq=$('#s_iaq');
  if(tl)tl.textContent=fmt(b.data.temp_raw,1)+' / '+fmt(b.data.temp,1)+' \u00B0C';
  if(iq){
    const st=['Stabilizing','Low accuracy','Medium accuracy','Calibrated'][clamp(Math.round(b.data.iaq_acc),0,3)];
    const l=b.data.warmEnd?Math.max(0,Math.round((b.data.warmEnd-Date.now())/1000)):null;
    iq.textContent=isLive(b)&&b.data.iaq_acc===0
      ?'Warming up'+(l?', '+Math.floor(l/60)+':'+String(l%60).padStart(2,'0')+' left':', almost
ready')+(Number.isFinite(b.data.gas_k)?' \u00B7 gas '+Math.round(b.data.gas_k)+' k\u03A9':'')
      :st+', IAQ '+Math.round(b.data.gas);
  }
}
// POST /api/calibrate with the changed keys. Needs the bin's Wi-Fi.
async function saveSettings(){
  const b=byId(state.selSet);if(!b)return;
  const c={};for(const k in MAP)c[MAP[k]]=+$('#'+k).value;
  c.auto_lock=$('#s_autolock').checked?1:0;
  if(c.d_full>=c.d_empty){$('#setMsg').textContent='Distance full must be smaller than distance empty.';return;}
  if(Math.abs(c.temp_off)>10||Math.abs(c.hum_off)>20||Math.abs(c.press_off)>50){
    $('#setMsg').textContent='An offset that large means the sensor is faulty or wired wrong. Temperature up to 10 C,
humidity up to 20 %RH, pressure up to 50 hPa.';return;}
  if(c.temp_th>85||c.hum_th>100||c.gas_th>500){$('#setMsg').textContent='The BME680 reads up to 85 C, 100 %RH and an air
quality index of 500, so a limit above that can never trigger.';return;}
  if(c.hum_th<30){$('#setMsg').textContent='A humidity limit under 30 %RH would sound in normal indoor air. Use 30 or
more.';return;}
  Object.assign(b.cal,c);
  let msg='Saved to '+b.name+'.';
  if(b.ip){try{const r=await callRaw(b,'/api/calibrate',c);if(!r.ok)msg=b.name+' refused the values'+(r.j&&r.j.why?':
'+r.j.why:'')+'.';}catch(e){msg=b.name+' did not answer, so the change is only on this screen. Settings need the bin\u2019s
Wi-Fi.';}}
  else msg='Saved on this screen. Add '+b.name+'’s IP address to send it to the bin.';
  addAlert('success','check','Settings saved',b.name+': empty '+c.d_empty+' cm, full '+c.d_full+' cm, alert at
'+c.fill_th+'%','setup');
  $('#setMsg').textContent=msg;render();
}
/* ---------- simulation ---------- */
function buildSim(){
  const simSegHtml=adaptiveSeg('simSeg','sim',[state.selSim],false);
  if($('#simSeg').dataset.sig!==simSegHtml){$('#simSeg').dataset.sig=simSegHtml;$('#simSeg').innerHTML=simSegHtml;}
  const b=byId(state.selSim);if(!b){$('#simBody').innerHTML='';return;}
  $('#simBody').innerHTML=SENSORS.map(s=>{
    const manual=(b.sim.mask>>s.bit)&1;
    const arg=s.key==='lid'?!!b.sim.values.lid:s.key==='motion'?!!b.sim.values.motion:false;
    const cc={fill:'var(--c-fill)',weight:'var(--c-weight)',gas:'var(--c-gas)',temp:'var(--c-temp)',
      tamper:'var(--c-tamper)',motion:'var(--c-motion)',lid:'var(--c-lid)'}[s.key]||'var(--ink2)';
    return '<tr data-manual="'+(manual?1:0)+'" style="--cc:'+cc+'"><td><span class="simname" style="--
cc:'+cc+'">'+icon(s.icon,arg)+esc(s.label)+'</span></td>'+
      '<td><button class="btn sm modebtn" data-mode="'+(manual?'MANUAL':'AUTO')+'" data-bit="'+s.bit+'">'+
      icon(manual?'flask':'check')+'<span>'+(manual?'MANUAL':'AUTO')+'</span></button></td>'+
      '<td>'+simInput(s,b,manual)+'</td></tr>';
  }).join('');
}
function simInput(s,b,manual){
  const v=b.sim.values,dis=manual?'':'disabled';
  if(s.kind==='range')return '<div class="simval"><input type="range" min="'+s.min+'" max="'+s.max+'" step="'+s.step+'"
value="'+v[s.key]+'" data-sv="'+s.key+'" '+dis+'><input type="number" min="'+s.min+'" max="'+s.max+'" step="'+s.step+'"
value="'+v[s.key]+'" data-sv="'+s.key+'" '+dis+'><span>'+(s.unit||'')+'</span></div>';
  if(s.kind==='number')return '<div class="simval"><input type="number" min="'+s.min+'" max="'+s.max+'" step="'+s.step+'"
```

```
value="'+v[s.key]+'" data-sv="'+s.key+'" '+dis+'><span>g</span></div>';
  if(s.kind==='pair')return '<div class="simval"><input type="number" value="'+v.temp+'" step="0.1" data-sv="temp"
'+dis+'><span>&deg;C</span><input type="number" value="'+v.humidity+'" step="1" data-sv="humidity"
'+dis+'><span>%</span></div>';
  if(s.kind==='rgb')return multiway('rgb',v.rgb,[[0,'OFF','var(--ink2)'],[1,'GREEN','var(--ok)'],
    [2,'AMBER','var(--warn)'],[3,'RED','var(--bad)'],[4,'WHITE','var(--ink)']],manual);
  if(s.kind==='buzzer')return multiway('buzzer',v.buzzer,[[0,'SILENT','var(--ink2)'],[1,'LID BEEP','var(--c-link)'],
    [2,'WARNING','var(--warn)'],[3,'TAMPER','var(--bad)']],manual);
  if(s.kind==='lid')return twoway('lid',v.lid,[['0','CLOSED','lock'],['1','OPEN','lid']],manual,'var(--c-lid)');
  const cc=s.key==='tamper'?'var(--c-tamper)':'var(--c-motion)';
  return twoway(s.key,v[s.key],[['0','OFF','cross'],['1','TRIGGERED',s.key==='tamper'?'tamper':'motion']],manual,cc);
}
function multiway(key,val,opts,enabled){
  return '<div class="twoway multi" data-sv="'+key+'" data-val="'+(val||0)+'">'+
    opts.map(o=>'<button class="tw" style="--cc:'+o[2]+'" data-v="'+o[0]+'" aria-pressed="'+
      (Number(val||0)===o[0])+'"'+(enabled?'':' disabled')+'><i class="dot"
style="background:'+o[2]+'"></i><span>'+o[1]+'</span></button>').join('')+'</div>';
}
function twoway(key,val,opts,enabled,cc){
  return '<div class="twoway" data-sv="'+key+'" data-val="'+(val?1:0)+'" style="--cc:'+cc+'">'+
    opts.map(o=>'<button class="tw" data-v="'+o[0]+'" aria-pressed="'+(String(val?1:0)===o[0])+'"'+
      (enabled?'':' disabled')+'>'+icon(o[2],o[0]==='1')+'<span>'+o[1]+'</span></button>').join('')+'</div>';
}
// POST /api/simulate with the mask and values. Needs the bin's Wi-Fi.
async function applySim(){
  const b=byId(state.selSim);if(!b)return;
  $$('#simBody [data-sv]').forEach(el=>{
    b.sim.values[el.dataset.sv]=el.classList.contains('twoway')?+el.dataset.val:+el.value;
  });
  let msg=b.sim.mask?'Simulation applied to '+b.name+'.':b.name+' is reading real sensors again.';
  if(b.ip){try{const r=await callRaw(b,'/api/simulate',{mask:b.sim.mask,values:b.sim.values});if(!r.ok)throw new
Error('refused');}
    catch(e){msg=b.name+' did not answer, so this only changes the screen.';}}
  $('#simMsg').textContent=msg;render();
}

/* ---------- views ---------- */
function setRole(r){
  if(state.role==='operator'&&r==='collector'&&opBooted)sfx('op_out');
  if(r==='collector'&&state.calib){
    const b=byId(state.calib.bin);state.calib=null;
    if(b)command(b.id,'CALIBRATE_END');
  }
  state.role=r;cardSig='';
  $('#rolechip').innerHTML=icon('lock',r!=='operator')+'<span>'+(r==='operator'?'OPERATOR':'COLLECTOR')+'</span>';
  opRender();
  const hidden=r==='collector'?['sim']:[];
  $$('.tab').forEach(t=>t.classList.toggle('hide',hidden.includes(t.dataset.view)));
  if(hidden.includes(state.view))setView('overview');
  else{render();if(state.view==='settings')loadSettings();}
}
function setView(v){
  state.view=v;
  $$('.tab').forEach(t=>t.setAttribute('aria-selected',String(t.dataset.view===v)));
  $$('section[data-view]').forEach(s=>s.classList.toggle('hide',s.dataset.view!==v));
  if(v==='charts')revealCharts();
  if(v==='settings')loadSettings();
  if(v==='sim')buildSim();
}

/* ---------- events ---------- */
document.addEventListener('click',async e=>{
  const t=e.target;
  const role=t.closest('[data-role]');
  if(role){
    if(role.dataset.role==='operator'){askOperator(true);return;}
    setRole('collector');leaveGate();return;
  }
  const tab=t.closest('.tab'); if(tab){setView(tab.dataset.view);return;}
  const cmd=t.closest('[data-cmd]'); if(cmd){command(cmd.dataset.bin,cmd.dataset.cmd);return;}
  const rg=t.closest('[data-range]');
  if(rg){state.rangeMin=+rg.dataset.range;$$('#range .btn').forEach(x=>x.setAttribute('aria-
pressed',String(x===rg)));revealCharts();return;}
  const pick=t.closest('[data-pick]');
  if(pick){
    const id=pick.dataset.pick;
    if(id==='ALL')state.chartPick=[];
    else{const i=state.chartPick.indexOf(id);i>-1?state.chartPick.splice(i,1):state.chartPick.push(id);}
    revealCharts();return;
  }
  const emd=t.closest('[data-envmode]');
  if(emd){state.envMode=emd.dataset.envmode;revealCharts();return;}
  const env=t.closest('[data-env]'); if(env){state.envBin=env.dataset.env;revealCharts();return;}
  const sset=t.closest('[data-set]'); if(sset){state.selSet=sset.dataset.set;loadSettings();return;}
  const cap=t.closest('[data-capture]');
  if(cap&&!cap.disabled){
```

```
    const b=byId(state.selSet);if(!b)return;
    $('#'+cap.dataset.capture).value=fmt(b.data.distance_cm,1);
    $('#setMsg').textContent='Captured '+fmt(b.data.distance_cm,1)+' cm from the sensor. Press SAVE SETTINGS to keep it.';
    return;
  }
  const ssim=t.closest('[data-sim]'); if(ssim){state.selSim=ssim.dataset.sim;buildSim();return;}
  const tw=t.closest('.tw');
  if(tw&&!tw.disabled){
    const box=tw.closest('.twoway');
    box.dataset.val=tw.dataset.v;
    box.querySelectorAll('.tw').forEach(x=>x.setAttribute('aria-pressed',String(x===tw)));
    const b=byId(state.selSim);if(b)b.sim.values[box.dataset.sv]=+tw.dataset.v;
    return;
  }
  if(t.closest('#btUse')){btUse();return;}
  const snd=t.closest('[data-snd]');
  if(snd){SND.armed=true;sfx(snd.dataset.snd,true);
    snd.classList.add('playing');setTimeout(()=>snd.classList.remove('playing'),600);return;}
  const rdt=t.closest('[data-read-toggle]');
  if(rdt){toggleRead(+rdt.dataset.readToggle);return;}
  if(t.closest('#pinEye')){
    const c=$('#pinCard');c.dataset.masked=c.dataset.masked==='0'?'1':'0';renderPin();return;
  }
  if(t.closest('#popRead')){markAllRead();return;}
  if(t.closest('#popOpen')){
    $('#bannerPop').classList.add('hide');
    $('#statusPill').setAttribute('aria-expanded','false');
    setView('alerts');return;
  }
  const del=t.closest('[data-del]');

if(del){state.alerts=state.alerts.filter(a=>a.id!==+del.dataset.del);alertSig='';banSig='';renderAlerts();renderBanners();ret
urn;}
  const pill=t.closest('#statusPill');
  if(pill){const pop=$('#bannerPop'),show=pop.classList.contains('hide');
    pop.classList.toggle('hide',!show);pill.setAttribute('aria-expanded',String(show));return;}
  if(!t.closest('#bannerPop')&&!t.closest('#statusPill')){
    const pop=$('#bannerPop');
    if(pop&&!pop.classList.contains('hide')){pop.classList.add('hide');$('#statusPill').setAttribute('aria-
expanded','false');}
  }
  const mb=t.closest('.modebtn');
  if(mb){const b=byId(state.selSim);b.sim.mask^=(1<<+mb.dataset.bit);buildSim();applyMode();return;}
  const ed=t.closest('[data-edit]');
  if(ed){
    const inOverlay=!!t.closest('#tileOverlay');
    if(inOverlay){openSetOverlay(ed.dataset.edit);return;}
    /* normal view: inline toggle */
    const row=$('#card-'+ed.dataset.edit+' [data-r="edit"]');
    if(row){const show=row.classList.contains('hide');
      row.classList.toggle('hide',!show);ed.setAttribute('aria-expanded',String(show));}
    return;
  }
  const sv=t.closest('[data-save]');
  if(sv){
    const inOverlay=!!t.closest('#setOverlay');
    if(inOverlay){saveFromSetOverlay(sv.dataset.save);return;}
    /* normal view inline save */
    const b=byId(sv.dataset.save),row=$('#card-'+b.id+' [data-r="edit"]');
    if(b&&row){b.name=row.querySelector('[data-k="name"]').value.trim()||b.name;
      b.place=row.querySelector('[data-k="place"]').value.trim();
      b.ip=row.querySelector('[data-k="ip"]').value.trim();relink(b);
      persist();cardSig='';render();loadSettings();buildSim();toast(b.name+' saved.');}
    return;
  }
  const rm=t.closest('[data-remove]');
  if(rm){
    const inOverlay=!!t.closest('#setOverlay');
    if(inOverlay){removeFromSetOverlay(rm.dataset.remove);return;}
    /* normal view inline remove */
    const i=state.bins.findIndex(b=>b.id===rm.dataset.remove);
    if(i>-1){const gone=state.bins.splice(i,1)[0];
      if(state.selSet===gone.id)state.selSet=state.bins[0]&&state.bins[0].id;
      if(state.selSim===gone.id)state.selSim=state.selSet;
      if(state.calib&&state.calib.bin===gone.id)state.calib=null;
      if(state.envBin===gone.id)state.envBin=state.selSet;
      persist();cardSig='';render();loadSettings();buildSim();toast(gone.name+' removed.');}
    return;
  }
  const less=t.closest('#addLess'),more=t.closest('#addMore');
  if(less||more){setAddCount(state.addCount+(more?1:-1));return;}
  if(t.closest('#addTile')){
    let first=null;
    for(let k=0;k<state.addCount;k++){
      const n=state.bins.length+1;
      const b=newBin({id:'b'+(Date.now()+k).toString(36),name:'Bin '+String(n).padStart(2,'0'),place:'New
```

```
location',ip:'',fill:5});
      seedHistory(b,n);state.bins.push(b);
      if(!first)first=b;
    }
    persist();cardSig='';render();
    if(first){
      if(state.addCount===1){
        if(state.ovView==='grid'&&state.bins.length>=3){openSetOverlay(first.id);}
        else{const row=$('#card-'+first.id+' [data-r="edit"]');if(row)row.classList.remove('hide');}
      }
      else toast('Added '+state.addCount+' bins. Open each one with the gear to set its name and IP.');
      $('#card-'+first.id).scrollIntoView({behavior:'smooth',block:'center'});
    }
    setAddCount(1);return;}
});
let demoStash=[];
function demoBtnRender(){
  const d=$('#demoBtn');
  if(d){d.classList.toggle('hide',!state.demo);d.innerHTML=icon('demo')+'<span>DEMO</span>';}
  const g=$('#gateDemo');
  if(g){g.setAttribute('aria-pressed',state.demo?'true':'false');g.innerHTML=icon('demo')+'<span>'+(state.demo?'DEMO ON':'DEMO OFF')+'</span>';}
  const n=$('#gateDemoNote');
  if(n)n.textContent=state.demo
    ?'Three made-up bins sit next to your real ones and run through every alert by themselves: full, pickup, tamper, lock, bad air, heat, humidity, a sensor fault, warm-up and a lost link.'
    :'Turn DEMO on to add three made-up bins that show every alert by themselves. Demo can only be turned on or off here, on the start page.';
}
// Demo on: real bins are put aside (demoStash) and demo bins take their
// place. Demo off: the real bins come back.
function setDemo(on){
  state.demo=on;
  try{localStorage.setItem('smartbin.demo',on?'1':'0')}catch(e){}
  if(on){
    const extra=demoStash.length?demoStash:DEMO_SEEDS.map(mkDemo);
    extra.forEach((b,i)=>{if(!b.hist.length)seedHistory(b,i)});
    state.bins=state.bins.filter(b=>!b.demo).concat(extra);
    demoStash=[];
    TOUR.i=0;TOUR.next=0;TOUR.pauseUntil=0;
  }else{
    demoStash=state.bins.filter(b=>b.demo);
    demoStash.forEach(tourClear);
    state.bins=state.bins.filter(b=>!b.demo);
    TOUR.msg='';
  }
  const first=state.bins[0]?state.bins[0].id:null;
  if(!byId(state.selSet))state.selSet=first;
  if(!byId(state.selSim))state.selSim=state.selSet;
  if(!byId(state.envBin))state.envBin=state.selSet;
  state.chartPick=state.chartPick.filter(id=>byId(id));
  if(state.calib&&!byId(state.calib.bin))state.calib=null;
  demoBtnRender();cardSig='';banSig='';render();loadSettings();buildSim();
}
/* The start page opens on every visit. Demo is turned on or off only here. */
function showGate(){
  if(state.role==='operator')setRole('collector');
  if(!$('#pinWrap').classList.contains('hide'))pinClose();
  demoBtnRender();
  $('#gate').classList.remove('hide');
  sfx('back');
  playLogo('#gateLogo .sblogo');
}
$('#gateDemo').addEventListener('click',()=>{
  SND.armed=true;
  setDemo(!state.demo);
  sfx(state.demo?'demo':'switch_off');
});
$('#demoBtn').addEventListener('click',showGate);
$('#opStart').addEventListener('click',showGate);
demoBtnRender();

/* ---------- demo tour ----------
   With DEMO on and nobody pressing anything on a demo bin, the demo bins act out every
   outcome in turn, with the same alerts, banners and sounds as a real bin. A key pressed
   on a demo bin pauses the tour for a minute. */
function tourPause(){TOUR.pauseUntil=Date.now()+60000;TOUR.msg='';banSig='';}
function tourClear(b){delete b._hold;b._demoLink=null;}
function tourBins(){const d=state.bins.filter(b=>b.demo);return [d[0],d[1%d.length],d[2%d.length]];}
function tourHold(b,o){b._hold=Object.assign({},b._hold||{},o);}
function tourDrop(b,k){if(b._hold){k.forEach(x=>delete b._hold[x]);}}
function tourCmd(b,c){TOUR.busy=true;try{command(b.id,c)}finally{TOUR.busy=false}}
// Demo tour: a script of situations (full, tamper, lost link ...) that plays
// out on the demo bins while nobody is touching the page.
const TOUR_STEPS=[
  ([a])=>{a.data.fill=Math.min(99,a.cal.fill_th+4);return a.name+' fills past its '+a.cal.fill_th+'% limit.';},
```

```
  ([a])=>{if(a.data.status==='LOCKED')tourCmd(a,'UNLOCK');tourCmd(a,'COLLECT');return 'A collector starts the pickup at '+a.name+'. The lid opens.';},
  ([a])=>{tourCmd(a,'COLLECT_DONE');return a.name+' is emptied and back to normal.';},
  ([,b])=>{tourHold(b,{tilt:40});return b.name+' is tipped over. Tamper alert.';},
  ([,b])=>{tourDrop(b,['tilt']);b.data.tilt=1;tourCmd(b,'CLEAR_TAMPER');return 'The tamper alert on '+b.name+' is reset.';},
  ([,,c])=>{tourCmd(c,'LOCK');return c.name+' is locked from the dashboard.';},
  ([,,c])=>{tourCmd(c,'UNLOCK');return c.name+' is unlocked again.';},
  ([,,c])=>{tourHold(c,{gas:c.cal.gas_th+45});return 'The air in '+c.name+' goes bad.';},
  ([,b,c])=>{tourDrop(c,['gas']);c.data.gas=60;tourHold(b,{temp:+c.cal.temp_th+3.5});return b.name+' gets too hot.';},
  ([,b])=>{tourDrop(b,['temp']);b.data.temp=b.data.temp_raw=29;tourHold(b,{humidity:+b.cal.hum_th+5});return b.name+' gets too humid.';},
  ([a,b])=>{tourDrop(b,['humidity']);b.data.humidity=62;tourHold(a,{faults:['HX711']});return 'The weight sensor of '+a.name+' stops answering.';},
  ([a])=>{tourDrop(a,['faults']);return 'The weight sensor of '+a.name+' answers again.';},
  ([,,c])=>{tourCmd(c,'RESET_GAS');return 'The air sensor of '+c.name+' warms up. The reading is held until it is settled.';},
  ([,b])=>{b._demoLink='fb';sfx('cloud');return b.name+' loses Wi-Fi and is read through Firebase.';},
  ([,b])=>{b._demoLink='ts';sfx('cloud');return 'Firebase is down too, so '+b.name+' is read through ThingSpeak.';},
  ([,b])=>{b._demoLink='down';sfx('bin_lost');return b.name+' stops answering on every link.';},
  ([,b])=>{b._demoLink=null;sfx('link_up');return b.name+' is back on Wi-Fi.';},
  ([,b])=>{tourCmd(b,'CALIBRATE_START');return b.name+' is being calibrated.';},
  ([,b])=>{tourCmd(b,'CALIBRATE_END');return 'Calibration of '+b.name+' is done. The tour starts again.';}
];
function tourTick(now){
  if(!state.demo||!inMain()||document.hidden||!state.bins.some(b=>b.demo)){TOUR.next=Math.max(TOUR.next,now+6000);return;}
  if(now<TOUR.pauseUntil||now<TOUR.next)return;
  if(state.calib&&byId(state.calib.bin)&&byId(state.calib.bin).demo){TOUR.next=now+10000;return;}
  TOUR.next=now+10000;
  const step=TOUR_STEPS[TOUR.i%TOUR_STEPS.length];
  TOUR.i=(TOUR.i+1)%TOUR_STEPS.length;
  try{TOUR.msg=step(tourBins())||'';}catch(e){TOUR.msg='';}
  banSig='';
}
$('#rolechip').addEventListener('click',()=>{
  if(state.role==='operator'){setRole('collector');toast('Back to collector. Operator needs the PIN again.');}
  else askOperator(false);
  if(typeof btRender==='function')btRender();
});
$('#theme').addEventListener('click',()=>{
  const d=document.documentElement.getAttribute('data-theme')==='dark';
  document.documentElement.setAttribute('data-theme',d?'light':'dark');
  sfx(d?'theme_light':'theme_dark');
  updateThemeIcon();
  if(state.view==='charts')drawCharts();
});
$('#btnSave').addEventListener('click',saveSettings);
$('#btnReload').addEventListener('click',async()=>{
  const b=byId(state.selSet);if(!b)return;
  if(!b.ip){$('#setMsg').textContent='Add an IP address for '+b.name+' first.';return;}
  try{const j=await call(b,'/api/status');if(j.cal)Object.assign(b.cal,j.cal);loadSettings();
    $('#setMsg').textContent='Loaded the values stored in '+b.name+'.';}
  catch(e){$('#setMsg').textContent=b.name+' did not answer.';}
});
$('#btnExport').addEventListener('click',()=>{
  const cfg=JSON.stringify(state.bins.map(b=>({id:b.id,name:b.name,place:b.place,ip:b.ip,cloud:b.cloud,ts_channel:b.tsCh,talkback_id:b.tbId})),null,2);
  const url=URL.createObjectURL(new Blob([cfg],{type:'application/json'}));
  const a=document.createElement('a');a.href=url;a.download='smartbin-bins.json';a.click();
  setTimeout(()=>URL.revokeObjectURL(url),2000);
});
$('#btPair').addEventListener('click',btPair);
$('#btScan').addEventListener('click',btScan);
$('#btNetList').addEventListener('click',e=>{
  const b=e.target.closest('[data-ssid]');if(!b||b.disabled)return;
  const i=$('#bt_ssid');i.value=b.dataset.ssid;i.dispatchEvent(new Event('input',{bubbles:true}));
  $$('#btNetList [data-ssid]').forEach(x=>x.setAttribute('aria-pressed',x===b?'true':'false'));
  sfx('pin_key');$('#bt_pass').focus();
  $('#btMsg').textContent=b.dataset.ssid+' chosen. Type its password, then press SEND WI-FI.';
});
$('#btSend').addEventListener('click',btSend);
$('#btOff').addEventListener('click',btOff);
$('#btEye').addEventListener('click',()=>{
  const i=$('#bt_pass'),show=i.type==='password';
  i.type=show?'text':'password';
  $('#btEye').innerHTML=icon(show?'eyeoff':'eye')+'<span>'+(show?'HIDE':'SHOW')+'</span>';
});
$('#btnMarkAll').addEventListener('click',markShown);
$('#btnClear').addEventListener('click',askDelete);
$('#alertYes').addEventListener('click',doDelete);
$('#alertNo').addEventListener('click',()=>$('#alertConfirm').classList.add('hide'));
$('#setFields').addEventListener('input',refreshDirty);
$('#btnCancel').addEventListener('click',()=>{
  const b=byId(state.selSet);if(!b)return;
  loadSettings();
  $('#setMsg').textContent='Changes discarded. The fields show what is stored on '+b.name+' again.';
```

```
});
$('#btnTare').addEventListener('click',()=>{
  const b=byId(state.selSet);if(!b)return;
  command(b.id,'TARE');
  $('#setMsg').textContent='Scale zeroed on '+b.name+'. The reading should sit at 0.00 kg.';
});
$('#btnCalW').addEventListener('click',()=>{
  const b=byId(state.selSet);if(!b)return;
  const g=+$('#s_known').value;
  if(!(g>0)){$('#setMsg').textContent='Type the weight you put in the bin, in grams.';return;}
  command(b.id,'CAL_WEIGHT',{grams:g});
  $('#setMsg').textContent='Load cell calibrated with '+g+' g. Take the weight off and check it returns to 0.00 kg.';
});
$('#btnRefT').addEventListener('click',()=>{
  const b=byId(state.selSet);if(!b)return;
  const ref=parseFloat($('#s_tref').value);
  if(!Number.isFinite(ref)){$('#setMsg').textContent='Type the reading of your reference thermometer first.';return;}
  const raw=Number.isFinite(b.data.temp_raw)?b.data.temp_raw:b.data.temp-(b.cal.temp_off||0);
  const off=+(ref-raw).toFixed(1),o=$('#s_toff');
  o.value=off;o.dispatchEvent(new Event('input',{bubbles:true}));
  $('#setMsg').textContent='Offset set to '+off+' C (reference '+ref+' minus raw '+fmt(raw,1)+'). Press SAVE SETTINGS to keep it.';
});
$('#btnTouchCal').addEventListener('click',()=>{
  const b=byId(state.selSet);if(!b)return;
  if(!isLive(b)){$('#setMsg').textContent=b.name+' is a demo bin, so there is no touchscreen to calibrate.';return;}
  command(b.id,'TOUCH_CAL');
  $('#setMsg').textContent=b.name+' is showing the touch calibration screen. Press each of the 4 crosses firmly, then let go. It saves by itself.';
});
$('#btnTouchReset').addEventListener('click',()=>{
  const b=byId(state.selSet);if(!b)return;
  if(!isLive(b)){$('#setMsg').textContent=b.name+' is a demo bin, so there is no touchscreen to reset.';return;}
  command(b.id,'TOUCH_CAL_RESET');
  $('#setMsg').textContent='Touch calibration on '+b.name+' is back to the firmware defaults.';
});
$('#btnGasReset').addEventListener('click',()=>{
  const b=byId(state.selSet);if(!b)return;
  command(b.id,'RESET_GAS');
  $('#setMsg').textContent='Gas baseline cleared on '+b.name+'. The air quality index needs time in normal air before it is fully accurate again.';
});
$('#calToggle').addEventListener('change',e=>{
  const b=byId(state.selSet);
  if(!b){e.target.checked=false;return;}
  if(e.target.checked){
    state.calib={bin:b.id};
    command(b.id,'CALIBRATE_START');
    $('#setMsg').textContent=b.name+' is in calibration mode. Its screen and the dashboard are teal until you switch this off.';
  }else{
    state.calib=null;
    command(b.id,'CALIBRATE_END');
    $('#setMsg').textContent='Calibration mode off. '+b.name+' is back to normal and the fields show what is stored on the bin.';
  }
  loadSettings();render();
});
$('#btnSim').addEventListener('click',applySim);
$('#btnSimOff').addEventListener('click',()=>{const b=byId(state.selSim);if(b)b.sim.mask=0;buildSim();applySim();});
$('#simBody').addEventListener('input',e=>{
  const el=e.target.closest('[data-sv]');if(!el||el.classList.contains('twoway'))return;
  $$('#simBody [data-sv="'+el.dataset.sv+'"]').forEach(x=>{if(x!==el&&!x.classList.contains('twoway'))x.value=el.value});
  const b=byId(state.selSim);if(b)b.sim.values[el.dataset.sv]=+el.value;
});
document.addEventListener('pointerdown',e=>{
  const host=e.target.closest('.btn,.tab,.iconbtn,.role,.addmini');
  if(!host||host.disabled||reduce())return;
  const r=host.getBoundingClientRect(),size=Math.max(r.width,r.height)*1.7;
  const sp=document.createElement('span');
  sp.className='ripple';
  sp.style.width=sp.style.height=size+'px';
  sp.style.left=(e.clientX-r.left-size/2)+'px';
  sp.style.top=(e.clientY-r.top-size/2)+'px';
  host.appendChild(sp);
  setTimeout(()=>sp.remove(),640);
});
window.addEventListener('resize',()=>{if(state.view==='charts')drawCharts()});
/* Number keys open the tabs in their order, M turns sounds on or off. Not while typing. */
document.addEventListener('keydown',e=>{
  if(e.ctrlKey||e.metaKey||e.altKey||!inMain())return;
  const t=e.target;if(t&&(t.matches('input,textarea,select')||t.isContentEditable))return;
  if(document.querySelector('.modal:not(.hide),.pinwrap:not(.hide)'))return;
  if(/^[1-9]$/.test(e.key)){const tabs=$$('.tab').filter(x=>x.offsetParent!==null),tb=tabs[+e.key-1];
    if(tb){e.preventDefault();tb.click();}return;}
  if(e.key==='m'||e.key==='M'){e.preventDefault();$('#sound').click();}
```

```
});
document.addEventListener('keydown',e=>{const h=e.target.closest&&e.target.closest('.shead');if(h&&(e.key==='Enter'||e.key===' ')){e.preventDefault();h.click();}});

    buildTabs();setAddCount(1);decorateStatics();paintHeadings();btDecorate();cloudDecorate();seedAlerts();setRole('collector');setView('overview');render();liveStart();

/* overview filter chips */
document.addEventListener('click',e=>{
  const sh=e.target.closest('.shead');
  if(sh){const d=sh.closest('.sect'),o=d.classList.toggle('open');sh.setAttribute('aria-expanded',o);sfx(o?'open':'back');sectAllRender();return;}
  const da=e.target.closest('#demoBody [data-dspeed],#demoBody [data-dpick],#demoBody [data-devent],#demoBody [data-dlink],#demoBody [data-dreset]');
  if(da){demoAct(da);return;}
  if(e.target.closest('#sectAll')){const open=!allSectsOpen();
    $$('.sect').forEach(d=>{d.classList.toggle('open',open);const h=d.querySelector('.shead');if(h)h.setAttribute('aria-expanded',open);});
    sfx(open?'open':'back');sectAllRender();return;}
  const fc=e.target.closest('#filterChips .fchip');
  if(fc){state.ovFilter=fc.dataset.filter;applyOvFilter();return;}
  /* view toggle */
  if(e.target.closest('#viewToggle')){state.ovView=state.ovView==='grid'?'list':'grid';applyOvView();return;}
  /* tile click */
  const tl=e.target.closest('.tile[data-tid]');
  if(tl&&state.ovView==='grid'){showTileOverlay(tl.dataset.tid);return;}
  /* collection priority row click */
  const cr=e.target.closest('.crow[data-pb]');
  if(cr){setView('overview');const card=$('#card-'+cr.dataset.pb);if(card)card.scrollIntoView({behavior:'smooth',block:'center'});return;}
  /* collect toggle */
  if(e.target.id==='collectToggle'){state.collectShowAll=!state.collectShowAll;collectSig='';renderCollectNext();return;}
  /* alert filter chips */
  const ak=e.target.closest('#alertKinds [data-akind]');
  if(ak){state.akind=ak.dataset.akind;$('#alertConfirm').classList.add('hide');alertSig='';renderAlerts();return;}
  const ar=e.target.closest('#alertRange [data-arange]');
  if(ar){state.arange=ar.dataset.arange;$('#alertConfirm').classList.add('hide');alertSig='';renderAlerts();return;}
  /* adaptive picker toggle — only match buttons inside a .pickwrap */
  const pb=e.target.closest('.pickwrap > [id$="Btn"]');
  if(pb){const listId=pb.id.replace('Btn','List');const lst=document.getElementById(listId);if(lst){lst.classList.toggle('hide');return;}}
  /* close any open picker when clicking elsewhere */
  if(!e.target.closest('.pickwrap')){$$('.picklist').forEach(l=>l.classList.add('hide'));}
});
/* picker search */
document.addEventListener('input',e=>{
  if(e.target.id&&e.target.id.endsWith('Search')){
    const q=e.target.value.toLowerCase();
    const list=e.target.closest('.picklist');if(!list)return;
    list.querySelectorAll('button').forEach(b=>{b.style.display=q&&!b.textContent.toLowerCase().includes(q)?'none':'';});
  }
});
/* scroll-edge detection for fade masks */
function tagScrollEdge(el){
  if(!el)return;
  el.classList.toggle('at-top',el.scrollTop<=2);
  el.classList.toggle('at-bot',el.scrollTop+el.clientHeight>=el.scrollHeight-2);
}
document.addEventListener('scroll',e=>{
if(e.target.classList&&(e.target.classList.contains('alertpane')||e.target.classList.contains('scrollpane')||e.target.id==='binsPane'))tagScrollEdge(e.target);
},true);
/* initial tag for static panes */
new MutationObserver(()=>{tagScrollEdge($('.alertpane'));tagScrollEdge($('#binsPane'));}).observe(document.body,{childList:true,subtree:true});
setTimeout(()=>{tagScrollEdge($('.alertpane'));tagScrollEdge($('#binsPane'));},200);

// The main loop starts here.
setInterval(poll,1000);

/* ---------- v12: sounds ----------
   The same tunes as the phone app (Android App/tools/make_sounds.py), played
   live with Web Audio. Rising pitch means forward, on or OK; falling means
   back, off or no. A laptop has no vibration motor, so the dashboard has
   sound only. Browsers keep audio silent until the first click or key. */
const SOUNDS={
  tap:['ui',[[1320,22]]],choice:['ui',[[1180,16],[1480,18]]],
  switch_on:['ui',[[880,40],[1320,55]]],switch_off:['ui',[[1320,40],[880,55]]],
  step_up:['ui',[[1400,24]]],step_down:['ui',[[1100,24]]],tab:['ui',[[990,26],[0,10],[1245,30]]],
  open:['ui',[[740,35],[990,50]]],back:['ui',[[990,35],[740,50]]],
  confirm:['ui',[[880,50],[1175,50],[1760,90]]],danger:['ui',[[440,90],[0,30],[440,90]]],
  denied:['ui',[[392,70],[0,40],[330,120]]],
  theme_light:['ui',[[1175,45],[1568,75]]],theme_dark:['ui',[[784,45],[587,75]]],
```

```
  saved:['event',[[1047,60],[1319,100]]],
  lock:['event',[[660,40],[0,20],[440,120]]],unlock:['event',[[440,40],[0,20],[660,120]]],
  pickup:['event',[[523,70],[659,70],[784,70],[659,70],[784,150]]],
  done:['event',[[784,80],[1047,80],[1319,80],[1568,230]]],
  calib:['event',[[988,40],[0,50],[988,40],[0,50],[988,40],[0,50],[1319,130]]],
  sim:['event',[[523,60],[622,60],[740,60],[880,140]]],
  link_up:['event',[[659,70],[880,70],[1319,150]]],
  link_lost:['event',[[880,90],[659,90],[440,190]]],
  cloud:['event',[[880,60],[0,30],[1047,60],[0,30],[784,130]]],
  logo:['ui',[[784,70],[988,70],[1175,70],[1568,170]]],
  boot:['event',[[523,80],[659,80],[784,80],[1047,220]]],
  demo:['event',[[659,60],[784,60],[988,60],[784,60],[1319,160]]],
  cmd_ok:['event',[[1319,50],[0,25],[1760,110]]],
  cmd_fail:['event',[[659,80],[0,30],[523,80],[0,30],[392,200]]],
  bt_paired:['event',[[1175,40],[1568,40],[1175,40],[1976,140]]],
  bt_wrong:['event',[[466,50],[0,25],[466,50],[370,140]]],
  bt_blocked:['event',[[622,180],[0,50],[311,320]]],
  wifi_joined:['event',[[880,60],[1175,60],[1480,60],[1760,180]]],
  wifi_failed:['event',[[740,60],[0,25],[554,60],[0,25],[440,90],[370,180]]],
  bt_off:['event',[[1568,35],[1175,35],[784,35],[523,150]]],
  pin_set:['event',[[1047,50],[1397,50],[1047,50],[1397,120]]],
  bin_lost:['event',[[740,100],[0,60],[554,210]]],bin_back:['event',[[554,80],[740,150]]],
  pin_key:['ui',[[1660,14]]],
  pin_wrong:['event',[[349,60],[0,30],[349,60],[0,30],[262,160]]],
  pin_lockout:['event',[[523,260,260],[0,60],[196,300]]],
  op_in:['event',[[587,50],[0,25],[880,50],[1175,60],[1760,160]]],
  op_out:['event',[[1175,50],[880,50],[0,25],[587,140]]],
  alert_full:['alert',[[880,180],[0,80],[880,180],[0,80],[880,180],[0,200],[1175,420]]],
  alert_tamper:['alert',[[700,1400,340],[1400,700,340],[700,1400,340],[1400,700,340]]],
  alert_sensor:['alert',[[1047,120],[784,120],[1047,120],[784,260]]],
  alert_lock:['alert',[[660,70],[0,30],[440,220]]],
  alert_pickup:['alert',[[523,90],[659,90],[784,90],[1047,260]]],
  alert_info:['alert',[[988,90],[1319,170]]]
};
/* The sound test list, the same groups and names as the phone app's Sound test. */
const SND_GROUPS=[
  ['Alerts',[['alert_full','Bin full','fill',null,'var(--bad)'],['alert_tamper','Tamper','tamper',null,'var(--c-
tamper)'],['alert_sensor','Sensor limit','temp',null,'var(--c-temp)'],['alert_lock','Bin locked','lock',true,'var(--
sim)'],['alert_pickup','Pickup alert','truck',null,'var(--cal)'],['alert_info','Info','bell',null,'var(--c-bt)']]],
  ['Bins and links',[['link_up','Wi-Fi link','wifi',true,'var(--c-link)'],['cloud','Cloud link','cloud',null,'var(--c-
cloud)'],['link_lost','Link lost','wifi',false,'var(--bad)'],['bin_lost','Bin quiet','radar',null,'var(--
warn)'],['bin_back','Bin back','bins',null,'var(--ok)'],['cmd_ok','Command OK','check',null,'var(--ok)'],['cmd_fail','Command
failed','cross',null,'var(--bad)']]],
  ['Lid and pickup',[['lock','Lock','lock',true,'var(--sim)'],['unlock','Unlock','lock',false,'var(--c-
lid)'],['pickup','Pickup start','truck',null,'var(--cal)'],['done','Pickup finished','check',null,'var(--c-fill)']]],
  ['Setup',[['calib','Calibrate','calib',null,'var(--cal)'],['sim','Simulation','flask',null,'var(--
sim)'],['saved','Saved','save',null,'var(--ok)'],['denied','Not allowed','cross',null,'var(--warn)']]],
  ['Bluetooth Wi-Fi setup',[['bt_paired','Paired','bluetooth',null,'var(--c-bt)'],['bt_wrong','Wrong code','key',null,'var(--
warn)'],['bt_blocked','Pairing stopped','lock',true,'var(--bad)'],['wifi_joined','Wi-Fi joined','wifi',true,'var(--
ok)'],['wifi_failed','Wi-Fi failed','wifi',false,'var(--c-tamper)'],['bt_off','Bluetooth off','power',null,'var(--c-
motion)']]],
  ['Operator PIN',[['op_in','Operator in','sliders',null,'var(--accent)'],['op_out','Operator out','undo',null,'var(--c-
weight)'],['pin_key','PIN key','pin',null,'var(--c-humid)'],['pin_wrong','Wrong PIN','key',null,'var(--
bad)'],['pin_lockout','PIN lockout','stopwatch',null,'var(--c-tamper)'],['pin_set','PIN saved','save',null,'var(--c-gas)']]],
  ['App',[['boot','Start','power',null,'var(--ok)'],['logo','Logo','bins',null,'var(--c-gas)'],['demo','Demo
on','flask',null,'var(--warn)'],['theme_light','Light theme','sunmoon',false,'var(--c-lid)'],['theme_dark','Dark
theme','sunmoon',true,'var(--c-humid)']]],
  ['Keys',[['tap','Tap','capture',null,'var(--c-humid)'],['choice','Choice','sliders',null,'var(--c-
gas)'],['tab','Tab','layers',null,'var(--c-motion)'],['open','Open','eye',null,'var(--c-
fill)'],['back','Back','undo',null,'var(--c-weight)'],['confirm','Confirm','check',null,'var(--
ok)'],['danger','Danger','trash',null,'var(--bad)'],['switch_on','Switch on','power',null,'var(--c-
link)'],['switch_off','Switch off','power',null,'var(--ink2)'],['step_up','Step up','plus',null,'var(--c-
temp)'],['step_down','Step down','percent',null,'var(--c-press)']]]
];
const SND_LEVEL={ui:.5,event:.72,alert:.85},SND_TIMBRE={ui:[.18,0],event:[.22,.05],alert:[.35,.22]};
const
CMD_SFX={LOCK:'lock',UNLOCK:'unlock',COLLECT:'pickup',COLLECT_DONE:'done',CALIBRATE_START:'calib',CALIBRATE_END:'switch_off',
CLEAR_TAMPER:'confirm',TARE:'confirm',CAL_WEIGHT:'confirm',RESET_GAS:'confirm',TOUCH_CAL:'calib',TOUCH_CAL_RESET:'confirm'};
var SND={on:true,ctx:null,armed:false,at:0};
try{SND.on=localStorage.getItem('smartbin.sound')!=='0'}catch(e){}
function sfx(name,force){
  const s=SOUNDS[name];
  if(!(SND.on||force)||!SND.armed||!s)return;
  SND.at=Date.now();
  try{SND.ctx=SND.ctx||new (window.AudioContext||window.webkitAudioContext)();}catch(e){return}
  const ctx=SND.ctx;if(ctx.state==='suspended')ctx.resume();
  const [h2,h3]=SND_TIMBRE[s[0]],lvl=SND_LEVEL[s[0]]*0.32,norm=1+h2+h3;
  let t=ctx.currentTime+0.01;
  s[1].forEach(n=>{
    const sweep=n.length===3,dur=(sweep?n[2]:n[1])/1000;
    if(!sweep&&n[0]===0){t+=dur;return;}
    const f0=n[0],f1=sweep?n[1]:n[0],g=ctx.createGain();
    g.connect(ctx.destination);
    g.gain.setValueAtTime(0,t);
    g.gain.linearRampToValueAtTime(lvl,t+0.004);
    g.gain.exponentialRampToValueAtTime(0.0006,t+dur);
```

```
    [[1,1],[2,h2],[3,h3]].forEach(([m,a])=>{
      if(!a)return;
      const o=ctx.createOscillator(),og=ctx.createGain();
      og.gain.value=a/norm;
      o.frequency.setValueAtTime(f0*m,t);
      if(f1!==f0)o.frequency.linearRampToValueAtTime(f1*m,t+dur);
      o.connect(og);og.connect(g);o.start(t);o.stop(t+dur+0.02);
    });
    t+=dur;
  });
}
const SPEAKER=on=>'<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><path d="M4 9.5h3.5L12 5.5v13l-4.5-4H4z"/>'+
  (on?'<path d="M15.5 9.2a4 4 0 0 1 0 5.6M18.2 6.6a7.6 7.6 0 0 1 0 10.8"/>':'<path d="M16 9.5l5 5M21 9.5l-5 5"/>')+'</svg>';
function soundRender(){
  $('#soundIco').innerHTML=SPEAKER(SND.on);
  $('#sound').setAttribute('aria-pressed',String(SND.on));
}
$('#sound').addEventListener('click',()=>{
  SND.armed=true;SND.on=!SND.on;
  try{localStorage.setItem('smartbin.sound',SND.on?'1':'0')}catch(e){}
  soundRender();sfx('switch_on');
  toast(SND.on?'Sounds on.':'Sounds off.');
});
/* A short sound for every key, chosen by what the key does. Commands, theme,
   sound and the PIN pad play their own. */
const ID_SFX={btnSim:'sim',btnSimOff:'switch_off',btnSave:'saved',btnExport:'saved',fbSave:'saved',
  btnClear:'danger',alertYes:'danger',alertNo:'back',btnCancel:'back',btnMarkAll:'confirm',popRead:'confirm',
  addMore:'step_up',addLess:'step_down',statusPill:'open',viewToggle:'choice'};
document.addEventListener('pointerdown',()=>{SND.armed=true},{capture:true,once:true});
document.addEventListener('pointerdown',()=>{if(toastEl){toastEl.remove();toastEl=null;clearTimeout(toastTimer);}},true);
document.addEventListener('keydown',()=>{SND.armed=true},{capture:true,once:true});
document.addEventListener('click',e=>{
  SND.armed=true;
  const h=e.target.closest('button,.tile,.crow,.fchip');
  if(!h||h.disabled)return;
  if(h.closest('[data-cmd],#pinWrap,[data-snd]')||['theme','sound','rolechip','gateDemo','brand','btPair','btSend','btOff','btScan'].includes(h.id))return;
  sfx(ID_SFX[h.id]||(h.classList.contains('tab')?'tab':h.classList.contains('role')?'choice':
    h.closest('[data-del],[data-remove]')?'danger':h.classList.contains('accent')?'confirm':'tap'));
},true);
document.addEventListener('change',e=>{
  if(e.target.matches('input[type=checkbox]')&&e.target.id!=='calToggle')sfx(e.target.checked?'switch_on':'switch_off');
},true);

/* ---------- v12: operator PIN ----------
   The same rules as the phone app: collector is the default, operator needs a
   4 to 8 digit PIN, only a salted hash is kept, 5 wrong tries start a 30 s
   wait that doubles each time, and operator drops to collector after 2 min
   with the tab hidden. SHA-256 is written out here because browsers hide
   WebCrypto on plain-http pages, which is how a bin serves this dashboard. */
var opBooted=false;
// Plain JavaScript SHA-256, used for the stretched PIN hash (see above).
function sha256(bytes){
  const K=[0x428a2f98,0x71374491,0xb5c0fbcf,0xe9b5dba5,0x3956c25b,0x59f111f1,0x923f82a4,0xab1c5ed5,0xd807aa98,0x12835b01,0x243185be,0x550c7dc3,0x72be5d74,0x80deb1fe,0x9bdc06a7,0xc19bf174,
0xe49b69c1,0xefbe4786,0x0fc19dc6,0x240ca1cc,0x2de92c6f,0x4a7484aa,0x5cb0a9dc,0x76f988da,0x983e5152,0xa831c66d,0xb00327c8,0xbf597fc7,0xc6e00bf3,0xd5a79147,0x06ca6351,0x14292967,
0x27b70a85,0x2e1b2138,0x4d2c6dfc,0x53380d13,0x650a7354,0x766a0abb,0x81c2c92e,0x92722c85,0xa2bfe8a1,0xa81a664b,0xc24b8b70,0xc76c51a3,0xd192e819,0xd6990624,0xf40e3585,0x106aa070,
0x19a4c116,0x1e376c08,0x2748774c,0x34b0bcb5,0x391c0cb3,0x4ed8aa4a,0x5b9cca4f,0x682e6ff3,0x748f82ee,0x78a5636f,0x84c87814,0x8cc70208,0x90befffa,0xa4506ceb,0xbef9a3f7,0xc67178f2];
  const H=[0x6a09e667,0xbb67ae85,0x3c6ef372,0xa54ff53a,0x510e527f,0x9b05688c,0x1f83d9ab,0x5be0cd19];
  const len=bytes.length,padded=new Uint8Array(((len+9+63)>>6)<<6);
  padded.set(bytes);padded[len]=0x80;
  const dv=new DataView(padded.buffer);
  dv.setUint32(padded.length-4,len*8);dv.setUint32(padded.length-8,Math.floor(len/0x20000000));
  const W=new Uint32Array(64),rotr=(x,n)=>(x>>>n)|(x<<(32-n));
  for(let o=0;o<padded.length;o+=64){
    for(let i=0;i<16;i++)W[i]=dv.getUint32(o+i*4);
    for(let i=16;i<64;i++){
      const s0=rotr(W[i-15],7)^rotr(W[i-15],18)^(W[i-15]>>>3),s1=rotr(W[i-2],17)^rotr(W[i-2],19)^(W[i-2]>>>10);
      W[i]=(W[i-16]+s0+W[i-7]+s1)>>>0;
    }
    let [a,b,c,d,e,f,g,h]=H;
    for(let i=0;i<64;i++){
      const t1=(h+(rotr(e,6)^rotr(e,11)^rotr(e,25))+((e&f)^(~e&g))+K[i]+W[i])>>>0;
      const t2=((rotr(a,2)^rotr(a,13)^rotr(a,22))+((a&b)^(a&c)^(b&c)))>>>0;
      h=g;g=f;f=e;e=(d+t1)>>>0;d=c;c=b;b=a;a=(t1+t2)>>>0;
    }
    H[0]=(H[0]+a)>>>0;H[1]=(H[1]+b)>>>0;H[2]=(H[2]+c)>>>0;H[3]=(H[3]+d)>>>0;
    H[4]=(H[4]+e)>>>0;H[5]=(H[5]+f)>>>0;H[6]=(H[6]+g)>>>0;H[7]=(H[7]+h)>>>0;
```

```
  }
  const out=new Uint8Array(32),ov=new DataView(out.buffer);
  H.forEach((x,i)=>ov.setUint32(i*4,x));
  return out;
}
const hex=u=>Array.from(u,x=>x.toString(16).padStart(2,'0')).join('');
var OPLOCK={
  MIN:4,MAX:8,FREE:5,AWAY:120000,
  get(k){try{return localStorage.getItem('smartbin.op.'+k)}catch(e){return null}},
  put(k,v){try{localStorage.setItem('smartbin.op.'+k,v)}catch(e){}},
  has(){return !!this.get('hash')},
  waitLeft(){return Math.max(0,(+this.get('until')||0)-Date.now())},
  triesLeft(){return Math.max(0,this.FREE-(+this.get('misses')||0))},
  valid(p){return /^\d+$/.test(p)&&p.length>=this.MIN&&p.length<=this.MAX},
  hash(pin,salt){
    /* Stretched: 20 000 rounds, so guessing every PIN offline is slow. */
    const enc=new TextEncoder(),pb=enc.encode(salt+':'+pin);
    let h=sha256(pb);
    for(let i=0;i<20000;i++){const m=new Uint8Array(h.length+pb.length);m.set(h);m.set(pb,h.length);h=sha256(m);}
    return hex(h);
  },
  set(pin){
    const salt=hex(crypto.getRandomValues(new Uint8Array(16)));
    this.put('salt',salt);this.put('hash',this.hash(pin,salt));this.put('misses','0');this.put('until','0');
  },
  check(pin){
    if(!this.has()||this.waitLeft()>0)return false;
    if(this.hash(pin,this.get('salt')||'')===this.get('hash')){this.put('misses','0');return true;}
    const m=(+this.get('misses')||0)+1;this.put('misses',String(m));
    if(m>=this.FREE)this.put('until',String(Date.now()+30000*Math.pow(2,Math.min(m-this.FREE,6))));
    return false;
  }
};
const pinUI={mode:null,first:false,pin:'',pinNew:''};
const KEY_SVG='<svg class="ico" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><circle cx="8.2" cy="12" r="3.6"/><path d="M11.8 12h9v3M17.6 12v2.6"/></svg>';
function leaveGate(){
  $('#gate').classList.add('hide');
  TOUR.next=Date.now()+5000;
  setTimeout(()=>{sfx('boot');playLogo('#brandLogo .sblogo');},180);
}
function askOperator(first){
  pinUI.first=first;pinUI.pin='';pinUI.pinNew='';
  pinOpen(OPLOCK.has()?'unlock':'set',null);
}
function pinOpen(mode,msg){
  pinUI.mode=mode;pinUI.pin='';
  $('#pinWrap').classList.remove('hide');
  pinMsg(msg);pinRender();
}
function pinClose(){$('#pinWrap').classList.add('hide');$('#pinScreen').classList.add('hide');clearInterval(btc.timer);pinUI.mode=null;pinUI.pin='';pinUI.pinNew='';}
/* Bluetooth pairing code: new and random each time, 60 s to live, 3 wrong codes stop pairing for 60 s. */
var btc={code:'',at:0,fails:0,until:0,timer:null,LEN:6,LIFE:60000};
function btNewCode(){btc.code=String(crypto.getRandomValues(new Uint32Array(1))[0]%1000000).padStart(6,'0');btc.at=Date.now();}
function btCodeStart(){
  const w=btc.until-Date.now(),b=byId(state.selSet);
  if(w>0){sfx('denied');$('#btMsg').textContent=(b?b.name:'The bin')+' stopped pairing after 3 wrong codes. Try again in '+Math.ceil(w/1000)+' s.';return;}
  btc.fails=0;btNewCode();pinUI.pinNew='';
  pinOpen('bt',null);
  $('#pinScreen').classList.remove('hide');
  clearInterval(btc.timer);btc.timer=setInterval(btScreen,250);btScreen();
}
function btScreen(){
  if(pinUI.mode!=='bt'){clearInterval(btc.timer);return;}
  let left=Math.ceil((btc.LIFE-(Date.now()-btc.at))/1000);
  if(left<=0){btNewCode();left=60;}
  const b=byId(state.selSet);
  $('#pinScreen').innerHTML='<small>STAND-IN FOR '+esc((b?b.name:'the bin').toUpperCase())+'\u2019S SCREEN</small><span class="pc">PAIRING CODE</span><b>'+
    btc.code.slice(0,3)+' '+btc.code.slice(3)+'</b><small class="exp'+(left<=10?' low':'')+'">Expires in '+left+' s</small>';
}
function btCodeCheck(){
  const b=byId(state.selSet),n=b?b.name:'the bin';
  if(Date.now()-btc.at>btc.LIFE){btNewCode();pinUI.pin='';sfx('bt_wrong');pinMsg('That code expired. The bin shows a new one.');btScreen();return;}
  if(pinUI.pin===btc.code){
    pinClose();sfx('bt_paired');
    bt.step=1;btRender();btFakeNets();
    $('#btMsg').textContent='Paired with '+n+' (demo). The link is encrypted now. Pick the Wi-Fi network next. A name with "fail" in it shows what a failed join looks like.';
    return;
  }
```

```
  pinUI.pin='';btc.fails++;
  if(btc.fails>=3){
    pinClose();btc.until=Date.now()+60000;sfx('bt_blocked');
    $('#btMsg').textContent='3 wrong codes. '+n+' stops pairing for 60 s.';
    return;
  }
  sfx('bt_wrong');pinShake();
  const left=3-btc.fails;pinMsg('Wrong code. '+left+' '+(left===1?'try':'tries')+' left.');
}
function pinMsg(m,ok){const el=$('#pinMsg');el.textContent=m||'';el.className='pinmsg '+(m?(ok?'ok':'bad'):'');}
function pinShake(){const b=$('#pinBox');b.classList.remove('shake');void b.offsetWidth;b.classList.add('shake');}
function pinRender(){
  const m=pinUI.mode,again=!!pinUI.pinNew;
  if(!m)return;
  const T={unlock:['Operator PIN','Collector is the default. Enter the operator PIN to calibrate, simulate and edit bins.','UNLOCK'],
    set:[pinUI.change?'New operator PIN':'Create the operator PIN',again?'Type it once more to be sure.':
      OPLOCK.MIN+' to '+OPLOCK.MAX+' digits. Anyone without it stays a collector. Only an operator can change it later.',again?'SAVE':'NEXT'],
    change:['Change operator PIN','Enter the current PIN first.','NEXT'],
    bt:['Type the pairing code','Demo: this box stands in for the pairing window Windows or your phone opens. Type the 6 digits the bin shows.','PAIR']}[m];
  $('#pinIco').innerHTML=m==='set'?KEY_SVG:m==='bt'?icon('bluetooth'):icon('lock',true);
  $('#pinTitle').textContent=T[0];$('#pinText').textContent=T[1];
  const lo=m==='bt'?btc.LEN:OPLOCK.MIN,n=Math.max(lo,pinUI.pin.length);
  $('#pinDots').innerHTML=Array.from({length:n},(_,i)=>'<i class="'+(i<pinUI.pin.length?'on':'')+'"></i>').join('');
  $('#pinPad').innerHTML=['1','2','3','4','5','6','7','8','9'].map(d=>'<button class="btn" data-pk="'+d+'">'+d+'</button>').join('')+
    '<button class="btn" data-pk="back" aria-label="Delete"'+(pinUI.pin?'':' disabled')+'>&#9003;</button>'+
    '<button class="btn" data-pk="0">0</button>'+
    '<button class="btn go" data-pk="go"'+(pinUI.pin.length>=lo?'':' disabled')+'>'+T[2]+'</button>';
}
function pinKey(k){
  const bt6=pinUI.mode==='bt',lo=bt6?btc.LEN:OPLOCK.MIN,hi=bt6?btc.LEN:OPLOCK.MAX;
  if(k==='back'){if(pinUI.pin){pinUI.pin=pinUI.pin.slice(0,-1);sfx('step_down');}}
  else if(k==='go'){if(pinUI.pin.length>=lo){sfx('confirm');pinGo();return;}}
  else if(pinUI.pin.length<hi){pinUI.pin+=k;sfx('pin_key');}
  pinRender();
}
function pinMiss(){
  pinUI.pin='';
  const left=OPLOCK.waitLeft();
  sfx(left>0?'pin_lockout':'pin_wrong');pinShake();
  pinMsg(left>0?'Wrong PIN. Locked for '+Math.ceil(left/1000)+' s.':'Wrong PIN. '+OPLOCK.triesLeft()+' tries before a wait.');
}
function pinGo(){
  if(pinUI.mode==='bt'){btCodeCheck();if(pinUI.mode)pinRender();return;}
  const wait=OPLOCK.waitLeft();
  if((pinUI.mode==='unlock'||pinUI.mode==='change')&&wait>0){
    pinUI.pin='';pinMsg('Too many wrong PINs. Try again in '+Math.ceil(wait/1000)+' s.');pinRender();return;
  }
  if(pinUI.mode==='unlock'){
    if(OPLOCK.check(pinUI.pin)){
      const first=pinUI.first;pinClose();sfx('op_in');
      setRole('operator');if(first)leaveGate();else toast('Signed in as operator.');
      if(typeof btRender==='function')btRender();
    }else pinMiss();
  }else if(pinUI.mode==='change'){
    if(OPLOCK.check(pinUI.pin)){pinUI.change=true;pinOpen('set',null);}else pinMiss();
  }else{
    if(!OPLOCK.valid(pinUI.pin)){pinUI.pin='';pinMsg('Use '+OPLOCK.MIN+' to '+OPLOCK.MAX+' digits.');}
    else if(!pinUI.pinNew){pinUI.pinNew=pinUI.pin;pinUI.pin='';pinMsg('Type the same PIN again.',true);}
    else if(pinUI.pin!==pinUI.pinNew){pinUI.pin='';pinUI.pinNew='';sfx('pin_wrong');pinShake();pinMsg('The two PINs differ. Start again.');}
    else{
      OPLOCK.set(pinUI.pin);
      const change=pinUI.change,first=pinUI.first;pinUI.change=false;pinClose();
      if(change){sfx('pin_set');toast('Operator PIN changed.');}
      else{sfx('op_in');setRole('operator');if(first)leaveGate();else toast('Operator PIN saved. Signed in as operator.');}
      opRender();
    }
  }
  pinRender();
}
function opRender(){
  const op=state.role==='operator',el=$('#opState');if(!el||!OPLOCK)return;
  el.textContent=op?'Operator':'Collector';
  $('#opChange').classList.toggle('hide',!op||!OPLOCK.has());
  $('#opLockNow').classList.toggle('hide',!op);
  $('#opNote').textContent=op?'':'You are a collector. Press COLLECTOR in the header and enter the operator PIN to unlock operator tools.';
}
$('#pinPad').addEventListener('click',e=>{const k=e.target.closest('[data-pk]');if(k&&!k.disabled)pinKey(k.dataset.pk);});
$('#pinCancel').addEventListener('click',()=>{sfx('back');pinClose();});
$('#pinWrap').addEventListener('click',e=>{if(e.target.id==='pinWrap'){sfx('back');pinClose();}});
```

```
document.addEventListener('keydown',e=>{
  if(!pinUI.mode)return;
  if(/^\d$/.test(e.key))pinKey(e.key);
  else if(e.key==='Backspace')pinKey('back');
  else if(e.key==='Enter')pinKey('go');
  else if(e.key==='Escape'){sfx('back');pinClose();}
  else return;
  e.preventDefault();
});
$('#opChange').addEventListener('click',()=>{pinUI.pinNew='';pinUI.change=false;pinOpen('change',null);});
$('#opLockNow').addEventListener('click',()=>{setRole('collector');toast('Back to collector. Operator needs the PIN again.');});
let opAway=0;
document.addEventListener('visibilitychange',()=>{
  if(document.hidden){opAway=Date.now();return;}
  if(opAway&&state.role==='operator'&&Date.now()-opAway>OPLOCK.AWAY){setRole('collector');toast('Operator locked after 2 min away. Collector mode.');}
  opAway=0;
});
opBooted=true;soundRender();opRender();

/* ---------- v2 logo and sound test ---------- */
let logoSeq=0;
function sbLogo(){
  const n=++logoSeq,body='M17.5 26h29l-2.7 27.6a3.2 3.2 0 0 1-3.2 2.9H23.4a3.2 3.2 0 0 1-3.2-2.9z';
  return '<svg class="sblogo" viewBox="0 0 64 64" fill="none" stroke-linecap="round" stroke-linejoin="round" role="img" aria-label="Smart-Bin">'+
    '<defs><linearGradient id="sbF'+n+'" x1="0" y1="0" x2="0" y2="1"><stop class="g0" offset="0"/><stop class="g1" offset="1"/></linearGradient>'+
    '<linearGradient id="sbG'+n+'" x1="0" y1="0" x2="1" y2="0"><stop class="g0" offset="0" stop-opacity=".05"/><stop class="g0" offset=".5" stop-opacity=".16"/><stop class="g0" offset="1" stop-opacity=".05"/></linearGradient>'+
    '<radialGradient id="sbR'+n+'"><stop class="g0" offset="0" stop-opacity=".45"/><stop class="g0" offset="1" stop-opacity="0"/></radialGradient>'+
    '<clipPath id="sbC'+n+'"><path d="'+body+'"/></clipPath></defs>'+
    '<circle class="sb-glow" cx="32" cy="12" r="12" fill="url(#sbR'+n+')"/>'+
    '<g class="sb-sig"><path class="sb-ac s1" d="M27.8 12.6a6 6 0 0 1 8.4 0" stroke-width="2.6"/>'+
    '<path class="sb-ac s2" d="M24.2 8.9a11 11 0 0 1 15.6 0" stroke-width="2.6" stroke-opacity=".7"/>'+
    '<path class="sb-ac s3" d="M20.6 5.2a16 16 0 0 1 22.8 0" stroke-width="2.6" stroke-opacity=".4"/></g>'+
    '<path d="'+body+'" fill="url(#sbG'+n+')"/>'+
    '<g clip-path="url(#sbC'+n+')"><g class="sb-fill">'+
    '<path class="sb-wave b" d="M0 37q5-3.6 10 0t10 0t10 0t10 0t10 0t10 0v30H0z" fill="url(#sbF'+n+')" opacity=".35"/>'+
    '<path class="sb-wave" d="M-5 39q5-3.6 10 0t10 0t10 0t10 0t10 0v30H-5z" fill="url(#sbF'+n+')"/></g></g>'+
    '<g class="sb-ticks"><path class="sb-ac" d="M22 30.5h3.4M22 34h2.2" stroke-width="1.8" stroke-opacity=".8"/></g>'+
    '<g class="sb-chip"><rect class="sb-chp" x="27.5" y="42" width="9" height="9" rx="1.8" stroke-width="2"/>'+
    '<path class="sb-chp" d="M32 42v-3.2M32 51v2.5M27.5 46.5h-4M36.5 46.5h4" stroke-width="1.8"/>'+
    '<circle class="sb-nd" cx="32" cy="37.6" r="1.5"/><circle class="sb-nd" cx="22.6" cy="46.5" r="1.5"/><circle class="sb-nd" cx="41.4" cy="46.5" r="1.5"/></g>'+
    '<path class="sb-ln" d="'+body+'" stroke-width="3"/>'+
    '<g class="sb-lid"><path class="sb-ln" d="M13 22.5h38" stroke-width="3.4"/><path class="sb-ln" d="M27 22.5v-3.4h10v3.4" stroke-width="3"/>'+
    '<circle class="sb-led" cx="45.5" cy="22.5" r="1.3"/></g></svg>';
}
function playLogo(sel){
  const el=$(sel);if(!el)return;
  el.classList.remove('play');void el.getBoundingClientRect();el.classList.add('play');
}
$('#gateLogo').innerHTML=sbLogo();
$('#brandLogo').innerHTML=sbLogo();
playLogo('#gateLogo .sblogo');
$('#brand').addEventListener('click',()=>{SND.armed=true;sfx('logo');playLogo('#brandLogo .sblogo');});
$('#sndHead').innerHTML=icon('speaker')+'<span>SOUND TEST</span>';
$('#sndGroups').innerHTML=SND_GROUPS.map(g=>'<div class="sndgrp"><h3>'+g[0]+'</h3><div>'+
  g[1].map(x=>'<button type="button" class="btn sm tint" style="--cc:'+x[4]+'" data-snd="'+x[0]+'">'+(x[3]===null?icon(x[2]):ICONS[x[2]](x[3]))+'<span>'+x[1]+'</span></button>').join('')+
  '</div></div>').join('');
</script>
</body>
</html>
```

Part 3. Android App (Kotlin)

Android App/app/src/main/java/smartbin/MainActivity.kt

```
// ================================================================================
// MainActivity.kt - the app's only activity
// Hosts the Compose UI (SmartBinUi), plays the opening sound, routes taps on
// notifications to the Overview, and starts the background WatchService when
// the app leaves the screen with push alerts allowed. Lifecycle problems
// (alerts not arriving in the background, bins read too slowly) usually start
// in onStart/onStop below.
// ================================================================================
package smartbin
```

```

import android.content.ComponentName
import android.content.Intent
import android.content.pm.ActivityInfo
import android.content.pm.PackageManager
import android.content.res.Configuration
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.activity.viewModels
import smartbin.data.Hub
import smartbin.notify.Alerts
import smartbin.notify.Haptics
import smartbin.notify.Sounds
import smartbin.notify.WatchService
import smartbin.ui.AppViewModel
import smartbin.ui.SmartBinUi
import smartbin.ui.ThemeMode

class MainActivity : ComponentActivity() {

    private val vm: AppViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        if (android.os.Build.VERSION.SDK_INT >= 29) window.isNavigationBarContrastEnforced = false
        setContent { SmartBinUi(vm, ::setOrientation) }
        // The opening chime and its rising buzz. Not on a rotation.
        if (savedInstanceState == null) {
            Sounds.init(this)
            Sounds.boot()
            Haptics.init(this)
            Haptics.play(Haptics.Buzz.BOOT)
        }
        handle(intent)
    }

    override fun onNewIntent(intent: Intent) {
        super.onNewIntent(intent)
        handle(intent)
    }

    /** A tap on a notification opens the Overview. */
    private fun handle(intent: Intent?) {
        if (intent?.getStringExtra(EXTRA_OPEN) != null) vm.openFromNotification()
    }

    /**
     * Phones stay upright; the fullscreen chart is the one view drawn sideways.
     * Tablets may be held either way.
     */
    private fun setOrientation(landscape: Boolean) {
        val phone = resources.configuration.smallestScreenWidthDp < 600
        requestedOrientation = when {
            landscape -> ActivityInfo.SCREEN_ORIENTATION_SENSOR_LANDSCAPE
            phone -> ActivityInfo.SCREEN_ORIENTATION_PORTRAIT
            else -> ActivityInfo.SCREEN_ORIENTATION_UNSPECIFIED
        }
    }

    override fun onStart() {
        super.onStart()
        Hub.foreground = true
        vm.foregrounded()
        WatchService.stop(this)
    }

    override fun onResume() {
        super.onResume()
        // Picks up a change made in Android's own notification settings.
        vm.refreshSystemState()
    }

    override fun onStop() {
        Hub.foreground = false
        if (!isChangingConfigurations) vm.backgrounded()
        // With push alerts on, the bins keep being read while the app is in the
        // background, but only once the user is inside the main screens.
        if (Hub.inMain && Alerts.allowed(this) && !isChangingConfigurations) WatchService.start(this)
        syncIcon()
        super.onStop()
    }

    /**
```

```
     * Makes the home-screen logo match the theme the app was left in: the dark
     * logo for dark, the light logo for light. It runs as the app leaves the
     * screen, because some launchers redraw the icon then, and a change made
     * while the app is open can close it.
     */
    private fun syncIcon() {
        val dark = when (vm.theme) {
            ThemeMode.DARK -> true
            ThemeMode.LIGHT -> false
            ThemeMode.AUTO -> (resources.configuration.uiMode and Configuration.UI_MODE_NIGHT_MASK) ==
Configuration.UI_MODE_NIGHT_YES
        }
        fun switch(name: String, on: Boolean) {
            val entry = ComponentName(packageName, "smartbin.$name")
            val want = if (on) PackageManager.COMPONENT_ENABLED_STATE_ENABLED else
PackageManager.COMPONENT_ENABLED_STATE_DISABLED
            if (packageManager.getComponentEnabledSetting(entry) != want) {
                packageManager.setComponentEnabledSetting(entry, want, PackageManager.DONT_KILL_APP)
            }
        }
        runCatching {
            // The new entry goes on before the old one goes off, so there is always one.
            switch(if (dark) "LauncherDark" else "LauncherLight", true)
            switch(if (dark) "LauncherLight" else "LauncherDark", false)
        }
    }

    companion object {
        const val EXTRA_OPEN = "open"
    }
}
```

Android App/app/src/main/java/smartbin/SmartBinApp.kt

```
// ================================================================================
// SmartBinApp.kt - Application class, created once per process
// Starts Hub (the reading loop) and turns every HubEvent into a sound,
// vibration or Android notification. If a bin event happens but nothing is
// heard or posted, check Hub.inMain and Hub.foreground first: nothing plays
// on the start page, and notifications are only posted while off screen.
// ================================================================================
package smartbin

import android.app.Application
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.launch
import smartbin.data.Hub
import smartbin.data.HubEvent
import smartbin.data.Link
import smartbin.notify.Alerts
import smartbin.notify.Feel
import smartbin.notify.Haptics
import smartbin.notify.Sfx
import smartbin.notify.Sounds

/**
 * Starts the reading loop once per process and turns its events into sound,
 * vibration or a notification. On screen an alert plays the app's own sound
 * and pattern; off screen it becomes an Android notification, which carries
 * the same sound through its channel.
 */
class SmartBinApp : Application() {
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.Main.immediate)

    override fun onCreate() {
        super.onCreate()
        Hub.init(this)
        Alerts.ensureChannels(this)
        Sounds.init(this)
        Haptics.init(this)
        val store = Hub.store()
        Feel.sounds = store.bool(KEY_SOUNDS, true)
        Feel.haptics = store.bool(KEY_HAPTICS, true)
        Haptics.strength = runCatching { Haptics.Strength.valueOf(store.string(KEY_STRENGTH,
"NORMAL")) }.getOrDefault(Haptics.Strength.NORMAL)

        scope.launch {
            Hub.events.collect { e ->
                // Nothing sounds or posts on the start page: only once the user is in the main screens.
                if (!Hub.inMain) return@collect
                val onScreen = Hub.foreground
                when (e) {
                    is HubEvent.NewAlert ->
```

```
                        if (onScreen) Feel.event(Alerts.sfxFor(e.type))
                        else if (!e.row.demo) Alerts.post(this@SmartBinApp, e.row, e.type, e.binId)
                    is HubEvent.LinkChanged -> if (onScreen) when (e.to) {
                        Link.LIVE -> Feel.event(Sfx.LINK_UP)
                        Link.CLOUD, Link.THINGSPEAK -> Feel.event(Sfx.CLOUD)
                        Link.OFFLINE -> Feel.event(Sfx.LINK_LOST)
                        Link.DEMO -> {}
                    }
                    is HubEvent.BinLost -> if (onScreen) Feel.event(Sfx.BIN_LOST)
                    is HubEvent.BinBack -> if (onScreen) Feel.event(Sfx.BIN_BACK)
                    is HubEvent.Sound -> if (onScreen) runCatching { Sfx.valueOf(e.name) }.getOrNull()?.let(Feel::event)
                    // The screens tell the user; nothing to post.
                    is HubEvent.CommandResult -> {}
                }
            }
        }
    }

    companion object {
        const val KEY_SOUNDS = "ui_sounds"
        const val KEY_HAPTICS = "ui_haptics"
        const val KEY_STRENGTH = "haptic_strength"
        const val KEY_AWAKE = "keep_awake"
        const val KEY_THEME = "theme"
        const val KEY_ROLE = "role"
        const val KEY_ASKED = "asked_alerts"
    }
}
```

Android App/app/src/main/java/smartbin/data/Models.kt

```
// ============================================================================
// data/Models.kt - plain data types shared by the whole app
// Reading mirrors the firmware's /api/status JSON; Bin is one bin with its
// settings and latest reading; HubState is everything the screens draw. The
// field names and the BinStatus order must stay in step with the firmware
// (state.h) and the web dashboard.
// ============================================================================
package smartbin.data

/** How the app is reaching the bins right now. Local Wi-Fi first, Firebase second, ThingSpeak third. */
enum class Link { DEMO, LIVE, CLOUD, THINGSPEAK, OFFLINE }

/** Where one bin's latest reading came from. */
enum class Source { DEMO, LIVE, CLOUD, THINGSPEAK, STALE }

/** The three ways to the bins, and which of them answer right now. */
data class Routes(val local: Boolean = false, val firebase: Boolean = false, val thingspeak: Boolean = false) {
    val any get() = local || firebase || thingspeak
    val count get() = listOf(local, firebase, thingspeak).count { it }
}

enum class BinStatus { OK, LOCKED, TAMPER, COLLECTING, CALIBRATING }

enum class Role { OPERATOR, COLLECTOR }

/** The app-wide colour mode, the same names as the dashboard. */
enum class Mode { NORMAL, SIM, CALIB, DUAL, ALERT }

enum class Severity { CRITICAL, WARNING, INFO, SUCCESS }

/** The kinds the alert filter bar knows. */
enum class AlertKind(val label: String, val one: String, val many: String) {
    ALL("All", "alert", "alerts"),
    FULL("Full bins", "full-bin alert", "full-bin alerts"),
    SENSOR("Sensor readings", "sensor alert", "sensor alerts"),
    TAMPER("Tamper", "tamper alert", "tamper alerts"),
    LOCK("Lock and unlock", "lock event", "lock events"),
    PICKUP("Pickups", "pickup", "pickups"),
    SETUP("Calibration and settings", "setup event", "setup events"),
}

/** One reading from a bin, the fields of GET /api/status. */
data class Reading(
    val fill: Double = 0.0,
    val distance: Double = 30.0,
    val weight: Double = 0.0,
    val temp: Double = 28.0,
    val tempRaw: Double = 30.0,
    val hum: Double = 60.0,
    val press: Double = 1008.4,
    val gas: Double = 60.0,
    val iaqAcc: Int = 3,
    val tilt: Double = 0.0,
    val motion: Boolean = false,
```

```
    val lidOpen: Boolean = false,
    val status: BinStatus = BinStatus.OK,
    /** When the phone received it. */
    val at: Long = 0L,
    /** Sensors that stopped answering, as the firmware names them: HC-SR04, HX711, BME680, BMI160. */
    val faults: List<String> = emptyList(),
    /** Seconds left in the air sensor's warm-up, sent only while [iaqAcc] is 0. */
    val iaqWarmS: Int? = null,
    /** The raw gas resistance in kOhm, shown while the air sensor warms up. */
    val gasK: Double? = null,
    /** COUNTDOWN (clears by itself) or LATCHED (needs RESET TAMPER), while the bin is in TAMPER. */
    val tamperPhase: String = "",
) {
    fun fault(sensor: String) = sensor in faults
    /** The air score is not settled yet. */
    val warming get() = iaqAcc == 0 && !fault("BME680")
}

/**
 * The calibration values of one bin, kept as named numbers so they map one to
 * one onto the JSON of POST /api/calibrate and onto the Settings steppers.
 */
typealias Cal = Map<String, Double>

// Used until a bin sends its own calibration. Same keys as the firmware's Cal.
val DEFAULT_CAL: Cal = mapOf(
    "d_empty" to 30.0, "d_full" to 5.0, "fill_th" to 80.0, "temp_th" to 40.0, "hum_th" to 85.0,
    "gas_th" to 200.0, "tilt_th" to 30.0, "weight_cap" to 4500.0, "auto_lock" to 0.0,
    "servo_closed" to 0.0, "servo_open" to 90.0, "collect_pin" to 1234.0, "pir_hold_s" to 5.0,
    "buzz_s" to 2.0, "temp_off" to -2.0, "hum_off" to 0.0, "press_off" to 0.0,
)

fun Cal.v(key: String): Double = this[key] ?: DEFAULT_CAL[key] ?: 0.0

/** Simulation: one bit per sensor that is held by hand, and the values it is held at. */
data class Sim(
    val mask: Int = 0,
    val values: Map<String, Double> = DEFAULT_SIM,
) {
    fun on(bit: Int) = (mask shr bit) and 1 == 1
}

// Starting values for the simulation sliders. Bit order of Sim.mask: 0 fill,
// 1 weight, 2 gas, 3 temperature and humidity, 4 tamper, 5 motion, 6 lid.
val DEFAULT_SIM: Map<String, Double> = mapOf(
    "fill" to 60.0, "weight" to 2000.0, "gas" to 120.0, "temp" to 28.0, "humidity" to 60.0,
    "tamper" to 0.0, "motion" to 0.0, "lid" to 0.0,
)

// One bin in the list. Real bins are saved by Store; demo bins are not.
data class Bin(
    /** Stable id inside the app. */
    val id: String,
    /** The bin's node in Firebase, /sb/bins/<cloudId>. */
    val cloudId: String,
    val name: String,
    val place: String,
    val ip: String = "",
    val cal: Cal = DEFAULT_CAL,
    val sim: Sim = Sim(),
    val reading: Reading = Reading(),
    val source: Source = Source.DEMO,
    /** Last time a real reading arrived, over Wi-Fi or the cloud. */
    val lastSeen: Long = 0L,
    /** Demo only: how fast it fills, in percent per hour. */
    val rate: Double = 0.5,
    /** ThingSpeak: the bin's channel and its read key, and the TalkBack queue it takes commands from. */
    val tsChannel: String = "",
    val tsKey: String = "",
    val tbId: String = "",
    val tbKey: String = "",
    /** One of the made-up bins that demo mode adds. They are never saved. */
    val isDemo: Boolean = false,
    /** From /api/status: firmware version, Wi-Fi signal in dBm, seconds since start, touch calibrated on the bin. */
    val fw: String = "",
    val rssi: Int? = null,
    val uptimeS: Long? = null,
    val touchCustom: Boolean? = null,
) {
    val onThingSpeak get() = tsChannel.isNotBlank()
    /** A made-up demo bin. Real bins are read over Wi-Fi, Firebase or ThingSpeak, and show "not answering" instead. */
    val demo get() = isDemo
    val stale get() = source == Source.STALE
    val calibrating get() = reading.status == BinStatus.CALIBRATING
    val simulating get() = sim.mask > 0
    val full get() = reading.fill >= cal.v("fill_th")
```

```
    val sensorOver get() = reading.temp >= cal.v("temp_th") || reading.hum >= cal.v("hum_th") ||
        reading.gas >= cal.v("gas_th") || reading.weight >= cal.v("weight_cap")
    val urgent get() = reading.status == BinStatus.TAMPER || full
    val needsAction get() = urgent || sensorOver
}

/** One point of history for the charts. */
data class Sample(val t: Long, val fill: Float, val temp: Float, val hum: Float, val press: Float, val gas: Float)

// One row of the Alerts page. id grows by one per alert (Hub.alertSeq).
data class AlertRow(
    val id: Long,
    val sev: Severity,
    /** Name of the icon, as in the mockup: fill, gas, tamper, humid, lock, check, truck, calib, temp. */
    val icon: String,
    val kind: AlertKind,
    val title: String,
    val meta: String,
    val at: Long,
    val read: Boolean = false,
    /** Raised by a demo bin: shown while demo mode is on, never saved. */
    val demo: Boolean = false,
)

/** The ThingSpeak server. The real one unless the mock server is used. */
const val THINGSPEAK_URL = "https://api.thingspeak.com"

data class FirebaseConfig(val url: String = "", val key: String = "", val email: String = "", val password: String = "") {
    val filled get() = url.isNotBlank() && key.isNotBlank() && email.isNotBlank() && password.isNotBlank()
}

/**
 * A command sent through Firebase or ThingSpeak that the bin has not confirmed
 * yet. Its key stays locked until the bin's next reading shows it was done,
 * or until [deadline] passes.
 */
data class Pending(val cmd: String, val via: Source, val at: Long, val deadline: Long)

/** Everything the screens draw, in one value. */
data class HubState(
    val bins: List<Bin> = emptyList(),
    val link: Link = Link.DEMO,
    val alerts: List<AlertRow> = emptyList(),
    /** Calibration mode, on one bin at a time. */
    val calibBin: String? = null,
    /** Bytes the cloud reported for this month, when it reports them. */
    val cloudBytes: Long? = null,
    val loading: Boolean = true,
    /** Firebase and ThingSpeak answered within the last minute. Null when they are not set up. */
    val fbUp: Boolean? = null,
    val tsUp: Boolean? = null,
    /** ThingSpeak messages a day for all channels, estimated from how fast their entry numbers rise. */
    val tsPerDay: Double? = null,
    /** Demo options: links to pretend are up, so the link colours can be shown with no hardware. */
    val demoLinks: Routes = Routes(),
    /** Demo options: pretend cloud commands are never confirmed, so their keys time out. */
    val demoNoConfirm: Boolean = false,
    /** Cloud commands waiting for the bin to confirm them, by bin id. */
    val pending: Map<String, Pending> = emptyMap(),
    /** Demo mode, turned on or off only on the start page. It adds the demo bins. */
    val demoOn: Boolean = false,
    /** What the demo tour is acting out right now, for its banner. */
    val tourMsg: String? = null,
) {
    /** Every bin runs on demo data. The demo options can still make it look like any link. */
    val demo get() = bins.isNotEmpty() && bins.all { it.demo }
    val routes: Routes
        get() = if (demo) demoLinks
        else Routes(local = bins.any { it.source == Source.LIVE }, firebase = fbUp == true, thingspeak = tsUp == true)
    val mode: Mode
        get() {
            val cal = calibBin != null
            val sim = bins.any { it.simulating }
            return when {
                cal && sim -> Mode.DUAL
                cal -> Mode.CALIB
                sim -> Mode.SIM
                bins.any { it.urgent } -> Mode.ALERT
                else -> Mode.NORMAL
            }
        }
    val unread get() = alerts.count { !it.read }
}
```

Android App/app/src/main/java/smartbin/data/Store.kt

```
// ================================================================================
// data/Store.kt - everything saved on the phone (SharedPreferences "smartbin")
// Clearing the app's storage in Android settings resets bins, alerts and the
// Firebase account. The Firebase password is AES-GCM encrypted with a key in
// the Android Keystore; if the Keystore key is lost, open() returns "" and the
// user is asked to type the password again.
// ================================================================================
package smartbin.data

import android.content.Context
import android.content.SharedPreferences
import android.security.keystore.KeyGenParameterSpec
import android.security.keystore.KeyProperties
import android.util.Base64
import org.json.JSONArray
import org.json.JSONObject
import smartbin.BuildConfig
import java.security.KeyStore
import javax.crypto.Cipher
import javax.crypto.KeyGenerator
import javax.crypto.SecretKey
import javax.crypto.spec.GCMParameterSpec

/**
 * What the app remembers between runs: the bin list, alert history, the
 * user's choices, and the Firebase account. The Firebase password is sealed
 * with a key in the Android Keystore, so it is never on disk in the clear.
 */
class Store(context: Context) {
    private val prefs: SharedPreferences = context.applicationContext.getSharedPreferences("smartbin", Context.MODE_PRIVATE)

    // ---------- small values ----------

    fun bool(key: String, def: Boolean) = prefs.getBoolean(key, def)
    fun putBool(key: String, v: Boolean) = prefs.edit().putBoolean(key, v).apply()
    fun string(key: String, def: String) = prefs.getString(key, def) ?: def
    fun putString(key: String, v: String) = prefs.edit().putString(key, v).apply()

    // ---------- bins ----------

    // Reads the saved bin list. Only settings and the last fill, weight and
    // gas are saved; every other reading comes fresh from the bins.
    fun bins(): List<Bin>? {
        val raw = prefs.getString("bins", null) ?: return null
        return runCatching {
            val a = JSONArray(raw)
            (0 until a.length()).map { i ->
                val o = a.getJSONObject(i)
                val cal = DEFAULT_CAL.toMutableMap()
                o.optJSONObject("cal")?.let { c -> c.keys().forEach { k -> cal[k] = c.optDouble(k, cal[k] ?: 0.0) } }
                Bin(
                    id = o.getString("id"),
                    cloudId = o.optString("cloudId", "bin%02d".format(i + 1)),
                    name = o.optString("name", "Bin %02d".format(i + 1)),
                    place = o.optString("place", ""),
                    ip = o.optString("ip", ""),
                    cal = cal,
                    rate = o.optDouble("rate", 0.5),
                    tsChannel = o.optString("tsChannel", ""),
                    tsKey = o.optString("tsKey", ""),
                    tbId = o.optString("tbId", ""),
                    tbKey = o.optString("tbKey", ""),
                    reading = Reading(fill = o.optDouble("fill", 0.0), weight = o.optDouble("weight", 0.0), gas =
o.optDouble("gas", 60.0)),
                )
            }
        }.getOrNull()
    }

    fun saveBins(bins: List<Bin>) {
        val a = JSONArray()
        bins.forEach { b ->
            val cal = JSONObject()
            b.cal.forEach { (k, v) -> cal.put(k, v) }
            a.put(
                JSONObject().put("id", b.id).put("cloudId", b.cloudId).put("name", b.name).put("place", b.place)
                    .put("ip", b.ip).put("cal", cal).put("rate", b.rate)
                    .put("tsChannel", b.tsChannel).put("tsKey", b.tsKey).put("tbId", b.tbId).put("tbKey", b.tbKey)
                    .put("fill", b.reading.fill).put("weight", b.reading.weight).put("gas", b.reading.gas)
            )
        }
        prefs.edit().putString("bins", a.toString()).apply()
    }
```

```

    // ---------- alerts ----------

    // Alert history, newest first, at most 200 rows (trimmed by Hub.add).
    fun alerts(): List<AlertRow>? {
        val raw = prefs.getString("alerts", null) ?: return null
        return runCatching {
            val a = JSONArray(raw)
            (0 until a.length()).map { i ->
                val o = a.getJSONObject(i)
                AlertRow(
                    id = o.getLong("id"),
                    sev = Severity.valueOf(o.getString("sev")),
                    icon = o.getString("icon"),
                    kind = AlertKind.valueOf(o.getString("kind")),
                    title = o.getString("title"),
                    meta = o.getString("meta"),
                    at = o.getLong("at"),
                    read = o.optBoolean("read"),
                )
            }
        }.getOrNull()
    }

    fun saveAlerts(rows: List<AlertRow>) {
        val a = JSONArray()
        rows.forEach { r ->
            a.put(
                JSONObject().put("id", r.id).put("sev", r.sev.name).put("icon", r.icon).put("kind", r.kind.name)
                    .put("title", r.title).put("meta", r.meta).put("at", r.at).put("read", r.read)
            )
        }
        prefs.edit().putString("alerts", a.toString()).apply()
    }

    // ---------- Firebase account ----------

    /** The team's account from local.properties at build time. Empty when none was given. */
    fun firebaseDefaults() = FirebaseConfig(BuildConfig.FB_URL, BuildConfig.FB_KEY, BuildConfig.FB_EMAIL,
BuildConfig.FB_PASSWORD)

    /** The saved account, or the built-in one when nothing was saved. */
    fun firebase(): FirebaseConfig = readFb("saved") ?: firebaseDefaults()

    /** The last account that passed a test. */
    fun firebaseWorking(): FirebaseConfig? = readFb("good") ?: firebaseDefaults().takeIf { it.filled }

    fun saveFirebase(cfg: FirebaseConfig, passedTest: Boolean) {
        writeFb("saved", cfg)
        if (passedTest) writeFb("good", cfg)
    }

    private fun readFb(slot: String): FirebaseConfig? {
        val url = prefs.getString("${slot}_url", null) ?: return null
        return FirebaseConfig(
            url = url,
            key = prefs.getString("${slot}_key", "").orEmpty(),
            email = prefs.getString("${slot}_email", "").orEmpty(),
            password = open(prefs.getString("${slot}_pw", "").orEmpty()),
        )
    }

    private fun writeFb(slot: String, cfg: FirebaseConfig) {
        prefs.edit()
            .putString("${slot}_url", cfg.url.trim())
            .putString("${slot}_key", cfg.key.trim())
            .putString("${slot}_email", cfg.email.trim())
            .putString("${slot}_pw", seal(cfg.password))
            .apply()
    }

    private fun key(): SecretKey {
        val store = KeyStore.getInstance("AndroidKeyStore").apply { load(null) }
        (store.getKey(KEY_ALIAS, null) as? SecretKey)?.let { return it }
        val maker = KeyGenerator.getInstance(KeyProperties.KEY_ALGORITHM_AES, "AndroidKeyStore")
        maker.init(
            KeyGenParameterSpec.Builder(KEY_ALIAS, KeyProperties.PURPOSE_ENCRYPT or KeyProperties.PURPOSE_DECRYPT)
                .setBlockModes(KeyProperties.BLOCK_MODE_GCM)
                .setEncryptionPaddings(KeyProperties.ENCRYPTION_PADDING_NONE)
                .build()
        )
        return maker.generateKey()
    }

    private fun seal(plain: String): String = runCatching {
        if (plain.isEmpty()) return ""
```

```
        val cipher = Cipher.getInstance("AES/GCM/NoPadding").apply { init(Cipher.ENCRYPT_MODE, key()) }
        val out = cipher.doFinal(plain.toByteArray())
        Base64.encodeToString(cipher.iv, Base64.NO_WRAP) + ":" + Base64.encodeToString(out, Base64.NO_WRAP)
    }.getOrDefault("")

    private fun open(sealed: String): String = runCatching {
        val parts = sealed.split(':')
        if (parts.size != 2) return ""
        val cipher = Cipher.getInstance("AES/GCM/NoPadding")
        cipher.init(Cipher.DECRYPT_MODE, key(), GCMParameterSpec(128, Base64.decode(parts[0], Base64.NO_WRAP)))
        String(cipher.doFinal(Base64.decode(parts[1], Base64.NO_WRAP)))
    }.getOrDefault("")

    private companion object {
        const val KEY_ALIAS = "smartbin_cloud_key"
    }
}
```

Android App/app/src/main/java/smartbin/data/LocalApi.kt

```
// ================================================================================
// data/LocalApi.kt - HTTP client for the bin's own API (net.cpp in the firmware)
// Debug tip: open http://<bin-ip>/api/status in the phone's browser. If that
// page loads but the app shows the bin as not answering, the IP saved in the
// bin's settings is wrong; if it does not load, the phone is not on the
// bins' Wi-Fi (or the router isolates clients).
// ================================================================================
package smartbin.data

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.io.IOException
import java.net.HttpURLConnection
import java.net.URL

/**
 * The bin's own REST API over the same Wi-Fi (implementation plan, section
 * "Local API"). Every call has a 1.5 s timeout, the same as the dashboard, so
 * a bin that is off is noticed quickly and the app can try the cloud.
 *
 *   GET  /api/status     every reading, the bin status, its calibration and sim mask
 *   POST /api/command    {"cmd": "LOCK"} and the other commands
 *   POST /api/calibrate  the calibration values, stored in EEPROM
 *   POST /api/simulate   {"mask": 5, "values": {...}}
 *   POST /api/cloud      {"url","key","email","pass"}: the Firebase account the bin signs in with
 *
 * A command the bin refuses comes back as HTTP 409 {"ok":false,"why":"..."},
 * thrown as [Refused] so the reason reaches the user and the cloud is not tried.
 */
object LocalApi {
    private const val TIMEOUT_MS = 1500

    /** The parts of /api/status the app uses. */
    class Status(
        val reading: Reading, val cal: Cal?, val simMask: Int?,
        val fw: String = "", val rssi: Int? = null, val uptimeS: Long? = null, val touchCustom: Boolean? = null,
    )

    /** The bin answered and said no, with its reason. */
    class Refused(val why: String) : IOException(why)

    suspend fun status(ip: String): Status = parse(JSONObject(call(ip, "/api/status", null)))

    suspend fun command(ip: String, cmd: String, extra: Map<String, Any> = emptyMap()) {
        val o = JSONObject().put("cmd", cmd)
        extra.forEach { (k, v) -> o.put(k, v) }
        call(ip, "/api/command", o.toString())
    }

    suspend fun calibrate(ip: String, cal: Cal) {
        val o = JSONObject()
        cal.forEach { (k, v) -> o.put(k, if (v == Math.floor(v)) v.toLong() else v) }
        call(ip, "/api/calibrate", o.toString())
    }

    /** Gives the bin the Firebase account it signs in with. */
    suspend fun sendCloud(ip: String, cfg: FirebaseConfig) {
        val o = JSONObject().put("url", cfg.url.trim()).put("key", cfg.key.trim()).put("email", cfg.email.trim()).put("pass",
cfg.password)
        call(ip, "/api/cloud", o.toString())
    }

    suspend fun simulate(ip: String, sim: Sim) {
        val values = JSONObject()
```

```
        sim.values.forEach { (k, v) -> values.put(k, v) }
        call(ip, "/api/simulate", JSONObject().put("mask", sim.mask).put("values", values).toString())
    }

    /** An address the user typed, with or without http:// and a port. */
    fun base(ip: String): String {
        val t = ip.trim().trimEnd('/')
        return if (t.startsWith("http://") || t.startsWith("https://")) t else "http://$t"
    }

    // All calls run on Dispatchers.IO. 400 and 409 become Refused with the
    // bin's own reason; any other non-2xx code is a plain IOException.
    private suspend fun call(ip: String, path: String, body: String?): String = withContext(Dispatchers.IO) {
        if (ip.isBlank()) throw IOException("no address")
        val conn = URL(base(ip) + path).openConnection() as HttpURLConnection
        conn.connectTimeout = TIMEOUT_MS
        conn.readTimeout = TIMEOUT_MS
        conn.useCaches = false
        try {
            if (body != null) {
                val bytes = body.toByteArray()
                conn.requestMethod = "POST"
                conn.doOutput = true
                conn.setRequestProperty("Content-Type", "application/json")
                conn.setFixedLengthStreamingMode(bytes.size)
                conn.outputStream.use { it.write(bytes) }
            }
            val code = conn.responseCode
            if (code == 409 || code == 400) {
                val why = runCatching { JSONObject(conn.errorStream.bufferedReader().use
{ it.readText() }).optString("why") }.getOrNull()
                throw Refused(why?.takeIf { it.isNotBlank() } ?: "The bin refused it")
            }
            if (code !in 200..299) throw IOException("HTTP $code")
            conn.inputStream.bufferedReader().use { it.readText() }.ifBlank { "{}" }
        } finally {
            conn.disconnect()
        }
    }

    /**
     * Reads the status JSON. Field names follow the plan; the older names the
     * dashboard also accepts (fill_percentage, weight_grams and so on) work too.
     */
    fun parse(j: JSONObject): Status {
        fun num(vararg keys: String, def: Double): Double {
            for (k in keys) if (j.has(k) && !j.isNull(k)) {
                val d = j.optDouble(k)
                if (!d.isNaN()) return d
            }
            return def
        }
        val temp = num("temp", "temperature", def = 28.0)
        val lidRaw = j.opt("lid") ?: j.opt("lid_state")
        val lidOpen = lidRaw == 1 || lidRaw.toString().equals("OPEN", true) || lidRaw == true
        val statusRaw = j.opt("status") ?: j.opt("bin_status")
        val names = BinStatus.entries
        val status = when (statusRaw) {
            is Number -> names.getOrElse(statusRaw.toInt()) { BinStatus.OK }
            null -> BinStatus.OK
            else -> names.firstOrNull { it.name.equals(statusRaw.toString(), true) } ?: BinStatus.OK
        }
        val reading = Reading(
            fill = num("fill", "fill_percentage", def = 0.0).coerceIn(0.0, 100.0),
            distance = num("distance_cm", "distance", def = 30.0),
            weight = num("weight_g", "weight_grams", "weight", def = 0.0),
            temp = temp,
            tempRaw = num("temp_raw", def = temp),
            hum = num("humidity", def = 60.0),
            press = num("pressure", "pressure_hpa", def = 1008.0),
            gas = num("iaq", "gas", "gas_level", def = 60.0),
            iaqAcc = num("iaq_acc", def = 3.0).toInt(),
            tilt = num("tilt", "tilt_deg", def = 0.0),
            motion = j.optBoolean("motion", j.optInt("pir", 0) == 1),
            lidOpen = lidOpen,
            status = status,
            at = System.currentTimeMillis(),
            faults = j.optJSONArray("faults")?.let { a -> (0 until a.length()).map { a.optString(it) }.filter
{ it.isNotBlank() } } ?: emptyList(),
            iaqWarmS = if (j.has("iaq_warm_s") && !j.isNull("iaq_warm_s")) j.optInt("iaq_warm_s") else null,
            gasK = if (j.has("gas_k") && !j.isNull("gas_k")) j.optDouble("gas_k").takeUnless { it.isNaN() } else null,
            tamperPhase = j.optString("tamper_phase", ""),
        )
        val cal = j.optJSONObject("cal")?.let { c ->
            val m = DEFAULT_CAL.toMutableMap()
            c.keys().forEach { k -> c.optDouble(k).takeUnless { it.isNaN() }?.let { m[k] = it } }
```

```
            m.toMap()
        }
        val mask = if (j.has("sim_mask")) j.optInt("sim_mask") else null
        return Status(
            reading, cal, mask,
            fw = j.optString("fw", ""),
            rssi = if (j.has("rssi") && !j.isNull("rssi")) j.optInt("rssi") else null,
            uptimeS = if (j.has("uptime_s") && !j.isNull("uptime_s")) j.optLong("uptime_s") else null,
            touchCustom = j.optJSONObject("touch")?.let { t -> if (t.has("custom")) t.optBoolean("custom") else null },
        )
    }
}
```

Android App/app/src/main/java/smartbin/data/Cloud.kt

```
// ================================================================================
// data/Cloud.kt - Firebase Realtime Database client over REST
// Debug tip: Settings > Connection > TEST runs test() below and reports the
// same plain-language errors as friendly(). "Permission denied" almost always
// means the account is missing from /sb_users or the rules were not published.
// ================================================================================
package smartbin.data

import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.io.IOException
import java.net.HttpURLConnection
import java.net.URL
import java.net.URLEncoder

/**
 * Firebase Realtime Database over plain REST, the way Garbage-Guard does it:
 * no SDK, so the URL, key and account can be typed in at run time.
 *
 * Data layout (the firmware writes the same paths):
 *   /sb/bins/<cloudId>/status    the /api/status JSON plus "ts" (ms) and "local_ip"
 *   /sb/commands/<cloudId>       the app pushes {"cmd": "LOCK", "ts": ...}; the bin runs and removes it
 *   /sb/results/<push key>       the bin's answer to that command: {"ok", "cmd", "why", "ts"}
 *   /sb/watchers/<id>            written every 10 s while the app reads bins over Firebase, so they send faster
 *   /sb/alerts/<cloudId>         the bin's own alert list, newest 50
 *   /sb/usage/bytes_month        optional, the month's download total for the meter
 *   /sb_users/<uid>              the accounts allowed in; Test reads this
 *
 * Only LOCK, UNLOCK and COLLECT go over the cloud. Calibration and
 * simulation need live readings, so they stay on the bins' Wi-Fi.
 */
class Cloud(private var config: FirebaseConfig) {
    private val signIn = Mutex()
    private var token: String? = null
    private var tokenFor: FirebaseConfig? = null
    private var tokenExpires = 0L

    fun use(cfg: FirebaseConfig) {
        config = cfg
    }

    val ready get() = config.filled

    class CloudStatus(val status: LocalApi.Status, val ts: Long, val localIp: String?)

    suspend fun status(cloudId: String): CloudStatus? {
        val raw = rest("GET", "sb/bins/$cloudId/status") ?: return null
        val o = JSONObject(raw)
        return CloudStatus(LocalApi.parse(o), o.optLong("ts", 0L), o.optString("local_ip").takeIf { it.isNotBlank() })
    }

    /** Pushes a command and returns its push key, under which the bin writes its answer. */
    suspend fun command(cloudId: String, cmd: String): String? {
        require(cmd in CLOUD_COMMANDS) { "$cmd needs the bins' Wi-Fi" }
        val body = JSONObject().put("cmd", cmd).put("ts", System.currentTimeMillis()).put("from", "app")
        val raw = rest("POST", "sb/commands/$cloudId", body.toString()) ?: return null
        return runCatching { JSONObject(raw).optString("name") }.getOrNull()?.takeIf { it.isNotBlank() }
    }

    /** The bin's answer to a command, or null while it has not answered. */
    suspend fun result(key: String): JSONObject? = rest("GET", "sb/results/$key")?.let { runCatching
{ JSONObject(it) }.getOrNull() }

    suspend fun dropResult(key: String) {
        runCatching { rest("DELETE", "sb/results/$key") }
    }
```

```
70
71      /** Tells the bins someone is watching, so they write every 5 s instead of every minute. */
72      suspend fun watch(id: String, device: String) {
73          val body = JSONObject()
74              .put("ts", JSONObject().put(".sv", "timestamp"))
75              .put("kind", "app").put("device", device).put("mode", "foreground").put("bin", "all")
76          rest("PUT", "sb/watchers/$id", body.toString())
77      }
78
79      suspend fun unwatch(id: String) {
80          runCatching { rest("DELETE", "sb/watchers/$id") }
81      }
82
83      /** The alerts a bin wrote to the cloud, oldest first. */
84      suspend fun alerts(cloudId: String): List<JSONObject> {
85          val raw = rest("GET", "sb/alerts/$cloudId") ?: return emptyList()
86          val o = runCatching { JSONObject(raw) }.getOrNull() ?: return emptyList()
87          return o.keys().asSequence().mapNotNull { o.optJSONObject(it) }.sortedBy { it.optLong("ts") }.toList()
88      }
89
90      suspend fun usageBytes(): Long? = rest("GET", "sb/usage/bytes_month")?.trim()?.toLongOrNull()
91
92      /** True when Firebase answers with the saved account, for the link chip. */
93      suspend fun ping(): Boolean = runCatching { rest("GET", "sb/usage/bytes_month"); true }.getOrDefault(false)
94
95      /** Signs in with [cfg] and reads its line in /sb_users, the same check as the dashboard's TEST. */
96      suspend fun test(cfg: FirebaseConfig): Result<Int> = try {
97          val (idToken, uid) = signInWith(cfg)
98          call(cfg.url, "GET", "sb_users/$uid", idToken)
99              ?: throw IOException("The account is not on the allowed list (sb_users).")
100         val bins = call(cfg.url, "GET", "sb/bins", idToken, query = "&shallow=true")
101         Result.success(bins?.let { runCatching { JSONObject(it).length() }.getOrDefault(0) } ?: 0)
102     } catch (e: CancellationException) {
103         throw e
104     } catch (e: Exception) {
105         Result.failure(IOException(friendly(e.message.orEmpty())))
106     }
107
108     private suspend fun signInWith(cfg: FirebaseConfig): Pair<String, String> {
109         val body = JSONObject().put("email", cfg.email.trim()).put("password", cfg.password).put("returnSecureToken", true)
110         val o = JSONObject(http("POST", authUrl(cfg), body.toString()))
111         tokenExpires = System.currentTimeMillis() + (o.optString("expiresIn", "3600").toLongOrNull() ?: 3600L) * 1000
112         return o.getString("idToken") to o.optString("localId")
113     }
114
115     // The Mutex makes sure only one sign-in runs at a time when several bins
116     // are read in parallel.
117     /** A token for the saved settings, renewed when under five minutes remain or the settings changed. */
118     private suspend fun auth(): String = signIn.withLock {
119         val cfg = config
120         val current = token
121         if (current != null && tokenFor == cfg && System.currentTimeMillis() < tokenExpires - 300_000) return@withLock
    current
122         val (fresh, _) = signInWith(cfg)
123         token = fresh
124         tokenFor = cfg
125         fresh
126     }
127
128     private suspend fun rest(method: String, path: String, body: String? = null): String? {
129         if (!ready) return null
130         return call(config.url, method, path, auth(), body)
131     }
132
133     private suspend fun call(db: String, method: String, path: String, idToken: String, body: String? = null, query: String =
    ""): String? {
134         val text = http(method, "${db.trim().trimEnd('/')}/$path.json?auth=${enc(idToken)}$query", body)
135         return text.takeIf { it.isNotBlank() && it.trim() != "null" }
136     }
137
138     private suspend fun http(method: String, url: String, body: String?): String = withContext(Dispatchers.IO) {
139         val conn = URL(url).openConnection() as HttpURLConnection
140         conn.requestMethod = method
141         conn.connectTimeout = 8000
142         conn.readTimeout = 10000
143         try {
144             if (body != null) {
145                 val bytes = body.toByteArray()
146                 conn.doOutput = true
147                 conn.setRequestProperty("Content-Type", "application/json")
148                 conn.setFixedLengthStreamingMode(bytes.size)
149                 conn.outputStream.use { it.write(bytes) }
150             }
151             val code = conn.responseCode
152             val stream = if (code in 200..299) conn.inputStream else conn.errorStream
153             val text = stream?.bufferedReader()?.use { it.readText() }.orEmpty()
```

```
            if (code !in 200..299) throw IOException(errorOf(text, code))
            text
        } catch (e: IOException) {
            conn.disconnect()
            throw e
        }
    }

    /** Firebase reports errors as {"error": "text"} or {"error": {"message": "CODE"}}. */
    private fun errorOf(text: String, code: Int): String {
        val o = runCatching { JSONObject(text) }.getOrNull()
        val inner = o?.optJSONObject("error")?.optString("message").orEmpty()
        val flat = if (o != null && o.optJSONObject("error") == null) o.optString("error") else ""
        return inner.ifEmpty { flat }.ifEmpty { "HTTP $code" }
    }

    /** A plain-http database address can only be the mock server, so sign-in goes there too. */
    private fun authUrl(cfg: FirebaseConfig): String =
        if (cfg.url.startsWith("http://")) "${cfg.url.trim().trimEnd('/')}/__auth/signInWithPassword?key=${enc(cfg.key)}"
        else "https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword?key=${enc(cfg.key.trim())}"

    private fun enc(s: String) = URLEncoder.encode(s, "UTF-8")

    companion object {
        val CLOUD_COMMANDS = setOf("LOCK", "UNLOCK", "COLLECT")

        /** Firebase's error codes in plain words. */
        fun friendly(message: String): String = when {
            "INVALID_LOGIN_CREDENTIALS" in message || "INVALID_PASSWORD" in message || "EMAIL_NOT_FOUND" in message ||
                "INVALID_EMAIL" in message -> "Wrong email or password."
            "API key not valid" in message || "API_KEY_INVALID" in message -> "The Web API key is wrong. Copy it again from the Firebase console."
            "PASSWORD_LOGIN_DISABLED" in message || "OPERATION_NOT_ALLOWED" in message ->
                "Email/Password sign-in is off in Firebase Authentication."
            "TOO_MANY_ATTEMPTS_TRY_LATER" in message -> "Too many tries, wait a few minutes."
            "USER_DISABLED" in message -> "This account is disabled."
            "Permission denied" in message || "HTTP 401" in message ->
                "The account is not on the allowed list (sb_users), or the rules were not pasted."
            "HTTP 404" in message -> "The database URL is wrong."
            "sb_users" in message -> message
            "Unable to resolve host" in message || "failed to connect" in message || "timeout" in message.lowercase() ->
                "No internet, or the database URL is wrong."
            else -> "Could not sign in. ${message.take(80)}"
        }

        /** The same checks as the dashboard. Null means every field looks right. */
        fun problemWith(cfg: FirebaseConfig): String? {
            val url = cfg.url.trim()
            val local = url.startsWith("http://")
            return when {
                !url.startsWith("https://") && !local -> "The database URL is wrong. It starts with https:// and ends in .firebasedatabase.app."
                url.endsWith("/") -> "Remove the / at the end of the database URL."
                !local && !(url.endsWith(".firebasedatabase.app") || url.endsWith(".firebaseio.com")) ->
                    "The database URL is wrong. It starts with https:// and ends in .firebasedatabase.app."
                !local && (!cfg.key.trim().startsWith("AIza") || cfg.key.trim().length != 39) ->
                    "The Web API key is wrong. Copy it again from the Firebase console."
                !Regex("""^[^@\s]+@[^@\s]+\.[^@\s]+$""").matches(cfg.email.trim()) || cfg.password.length < 6 ->
                    "Wrong email or password."
                else -> null
            }
        }
    }
}
```

Android App/app/src/main/java/smartbin/data/ThingSpeak.kt

```
// ================================================================================
// data/ThingSpeak.kt - ThingSpeak channel reader and TalkBack sender
// Debug tip: open https://api.thingspeak.com/channels/<id>/feeds/last.json?api_key=<read key>
// in a browser. "-1" means the channel ID or key is wrong; an old created_at
// means the bin has stopped writing (see the [thingspeak] lines on its Serial log).
// ================================================================================
package smartbin.data

import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.withContext
import org.json.JSONObject
import java.io.IOException
import java.net.HttpURLConnection
import java.net.URL
import java.net.URLEncoder
import java.text.SimpleDateFormat
import java.util.Locale
import java.util.TimeZone
```

```
19
20 /**
21  * ThingSpeak, the third way to the bins. Each bin has its own channel and
22  * writes to it at most every 15 s (the free licence's limit). The app reads
23  * the newest entry with the channel's read key, and sends LOCK, UNLOCK and
24  * COLLECT through the bin's TalkBack queue, which the bin polls.
25  *
26  * Channel fields (the firmware writes the same ones):
27  *   field1 fill %         field5 air quality (IAQ)
28  *   field2 weight g       field6 pressure hPa
29  *   field3 temperature C  field7 tilt deg
30  *   field4 humidity %     field8 motion + 2 * lid open + 4 * status
31  *   status (text)         the bin's local IP address
32  * Status numbers: 0 OK, 1 LOCKED, 2 TAMPER, 3 COLLECTING, 4 CALIBRATING.
33  */
34 class ThingSpeak(private var server: String = THINGSPEAK_URL) {
35
36     fun use(url: String) {
37         server = url.trim().trimEnd('/').ifBlank { THINGSPEAK_URL }
38     }
39
40     val url get() = server
41
42     /** [entryId] goes up by one with each write, which is how the app estimates the message count. */
43     class Entry(val reading: Reading, val ts: Long, val localIp: String?, val entryId: Long)
44
45     /** The newest entry of a bin's channel, or null when the channel has none yet. */
46     suspend fun last(channel: String, key: String): Entry? {
47         val q = "api_key=${enc(key.trim())}&status=true"
48         val text = http("GET", "$server/channels/${enc(channel.trim())}/feeds/last.json?$q", null)
49         if (text.isBlank() || text.trim() == "-1" || text.trim() == "null") return null
50         val o = JSONObject(text)
51         fun f(n: Int, def: Double) = o.optString("field$n").toDoubleOrNull() ?: def
52         val flags = f(8, 0.0).toInt()
53         val status = BinStatus.entries.getOrElse(flags shr 2) { BinStatus.OK }
54         val fill = f(1, 0.0)
55         val r = Reading(
56             fill = fill, weight = f(2, 0.0), temp = f(3, 28.0), tempRaw = f(3, 28.0), hum = f(4, 60.0),
57             gas = f(5, 60.0), press = f(6, 1008.4), tilt = f(7, 0.0),
58             motion = flags and 1 == 1, lidOpen = flags and 2 == 2, status = status,
59             at = System.currentTimeMillis(),
60         )
61         return Entry(r, parseTime(o.optString("created_at")), o.optString("status").takeIf { it.isNotBlank() && it !=
"null" }, o.optLong("entry_id", 0L))
62     }
63
64     /** Puts a command on the bin's TalkBack queue. Only LOCK, UNLOCK and COLLECT. */
65     suspend fun command(tbId: String, tbKey: String, cmd: String) {
66         require(cmd in Cloud.CLOUD_COMMANDS) { "$cmd needs the bins' Wi-Fi" }
67         val body = "api_key=${enc(tbKey.trim())}&command_string=${enc(cmd)}"
68         http("POST", "$server/talkbacks/${enc(tbId.trim())}/commands.json", body)
69     }
70
71     private suspend fun http(method: String, url: String, form: String?): String = withContext(Dispatchers.IO) {
72         val conn = URL(url).openConnection() as HttpURLConnection
73         conn.requestMethod = method
74         conn.connectTimeout = 8000
75         conn.readTimeout = 10000
76         try {
77             if (form != null) {
78                 val bytes = form.toByteArray()
79                 conn.doOutput = true
80                 conn.setRequestProperty("Content-Type", "application/x-www-form-urlencoded")
81                 conn.setFixedLengthStreamingMode(bytes.size)
82                 conn.outputStream.use { it.write(bytes) }
83             }
84             val code = conn.responseCode
85             val stream = if (code in 200..299) conn.inputStream else conn.errorStream
86             val text = stream?.bufferedReader()?.use { it.readText() }.orEmpty()
87             if (code !in 200..299) throw IOException(if (code == 400 || code == 401) "The channel ID or key is wrong." else
"HTTP $code")
88             text
89         } catch (e: IOException) {
90             conn.disconnect()
91             throw e
92         }
93     }
94
95     private fun parseTime(s: String): Long = runCatching {
96         SimpleDateFormat("yyyy-MM-dd'T'HH:mm:ssX", Locale.US).apply { timeZone =
TimeZone.getTimeZone("UTC") }.parse(s)!!.time
97     }.getOrDefault(0L)
98
99     private fun enc(s: String) = URLEncoder.encode(s, "UTF-8")
100 }
```

Android App/app/src/main/java/smartbin/data/Ble.kt

```kotlin
// ========================================================================
// data/Ble.kt - Bluetooth LE client for the bin's Wi-Fi setup (ble_setup.cpp)
// Android's GATT API allows one operation at a time, so every read, write and
// descriptor write goes through call(), which waits for its callback before
// the next one starts. "Bluetooth error 5, 15 or 137" (GattError.auth) means
// the link was not secured with the passkey; StaleBond means the phone still
// remembers an old pairing. For both, forget the bin in the phone's Bluetooth
// settings (forget() below tries this) and pair again.
// ========================================================================
package smartbin.data

import android.Manifest
import android.annotation.SuppressLint
import android.bluetooth.BluetoothAdapter
import android.bluetooth.BluetoothDevice
import android.bluetooth.BluetoothGatt
import android.bluetooth.BluetoothGattCallback
import android.bluetooth.BluetoothGattCharacteristic
import android.bluetooth.BluetoothGattDescriptor
import android.bluetooth.BluetoothManager
import android.bluetooth.BluetoothProfile
import android.bluetooth.le.ScanCallback
import android.bluetooth.le.ScanResult
import android.bluetooth.le.ScanSettings
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.content.IntentFilter
import android.content.pm.PackageManager
import android.os.Build
import android.os.ParcelUuid
import kotlinx.coroutines.CompletableDeferred
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock
import kotlinx.coroutines.withTimeout
import kotlinx.coroutines.withTimeoutOrNull
import org.json.JSONArray
import org.json.JSONObject
import java.io.IOException
import java.util.UUID

/**
 * Wi-Fi setup over Bluetooth with the bin's own GATT service, as the firmware
 * (ble_setup.cpp) and Gen's INTEGRATION.md describe it:
 *
 *   b1a70001 service     advertised by SmartBin-01 and SmartBin-02
 *   b1a70002 SSID        write, the network name
 *   b1a70003 PASS        write, the password
 *   b1a70004 CTRL        write one byte: 1 joins with SSID and PASS, 2 turns Bluetooth off
 *   b1a70005 STAT        read and notify, {"state":"WAITING|CONNECTING|CONNECTED|FAILED", ...}
 *   b1a70006 SCAN        read, up to 6 networks, strongest first
 *
 * Every characteristic needs an encrypted, passkey-authenticated link, so the
 * phone bonds first and Android shows its own box for the 6-digit code the bin
 * shows. One GATT call runs at a time; each waits for its callback.
 */
@SuppressLint("MissingPermission")
class BinBle(context: Context) {
    private val app = context.applicationContext
    private val manager = app.getSystemService(BluetoothManager::class.java)
    private val adapter: BluetoothAdapter? get() = manager?.adapter

    class Found(val name: String, val address: String, val rssi: Int, val device: BluetoothDevice)
    class Net(val ssid: String, val rssi: Int?)

    /** A link that drops at once: the phone or the bin still remembers an old pairing. */
    class StaleBond : IOException("The bin dropped the link")

    private var gatt: BluetoothGatt? = null
    private var chars = HashMap<UUID, BluetoothGattCharacteristic>()
    private val ops = Mutex()
    private var op: CompletableDeferred<ByteArray>? = null
    private val linkUp = MutableStateFlow(false)
    private val bond = MutableStateFlow(BluetoothDevice.BOND_NONE)
    private var receiver: BroadcastReceiver? = null

    /** Each STAT message the bin sends, whole. */
    var onStat: ((JSONObject) -> Unit)? = null
    /** The link dropped while connected. */
    var onDrop: (() -> Unit)? = null
```

```

    val device: BluetoothDevice? get() = gatt?.device
    val connected get() = linkUp.value

    // ---------- permissions and the adapter ----------

    // Android 12+ needs BLUETOOTH_SCAN and BLUETOOTH_CONNECT; older versions
    // need location for scanning.
    fun permissions(): Array<String> =
        if (Build.VERSION.SDK_INT >= 31) arrayOf(Manifest.permission.BLUETOOTH_SCAN, Manifest.permission.BLUETOOTH_CONNECT)
        else arrayOf(Manifest.permission.ACCESS_FINE_LOCATION)

    fun allowed() = permissions().all { app.checkSelfPermission(it) == PackageManager.PERMISSION_GRANTED }

    val present get() = adapter != null
    val on get() = adapter?.isEnabled == true

    // ---------- scan ----------

    /** Lists bins nearby for [millis]: by the service in the advert, or by a name that starts with SmartBin. */
    suspend fun scan(millis: Long, onFound: (Found) -> Unit) {
        val scanner = adapter?.bluetoothLeScanner ?: throw IOException("Bluetooth is off")
        val seen = HashSet<String>()
        val svc = ParcelUuid(SVC)
        val cb = object : ScanCallback() {
            override fun onScanResult(callbackType: Int, result: ScanResult) {
                val d = result.device
                val name = result.scanRecord?.deviceName ?: runCatching { d.name }.getOrNull() ?: ""
                val ours = result.scanRecord?.serviceUuids?.contains(svc) == true || name.startsWith("SmartBin", ignoreCase =
true)
                if (ours && seen.add(d.address)) onFound(Found(name.ifBlank { "SmartBin" }, d.address, result.rssi, d))
            }
        }
        val settings = ScanSettings.Builder().setScanMode(ScanSettings.SCAN_MODE_LOW_LATENCY).build()
        scanner.startScan(null, settings, cb)
        try {
            delay(millis)
        } finally {
            runCatching { scanner.stopScan(cb) }
        }
    }

    // ---------- connect, pair, read and write ----------

    private val callback = object : BluetoothGattCallback() {
        // GATT callbacks. Each one finishes the waiting call() through done().
        override fun onConnectionStateChange(g: BluetoothGatt, status: Int, newState: Int) {
            if (newState == BluetoothProfile.STATE_CONNECTED && status == BluetoothGatt.GATT_SUCCESS) {
                linkUp.value = true
            } else if (newState == BluetoothProfile.STATE_DISCONNECTED) {
                val was = linkUp.value
                linkUp.value = false
                op?.completeExceptionally(IOException("The Bluetooth link dropped"))
                runCatching { g.close() }
                if (gatt === g) gatt = null
                if (was) onDrop?.invoke()
            }
        }

        override fun onServicesDiscovered(g: BluetoothGatt, status: Int) = done(status, ByteArray(0))

        override fun onMtuChanged(g: BluetoothGatt, mtu: Int, status: Int) = done(BluetoothGatt.GATT_SUCCESS, ByteArray(0))

        @Deprecated("Android 12 and older")
        override fun onCharacteristicRead(g: BluetoothGatt, ch: BluetoothGattCharacteristic, status: Int) {
            @Suppress("DEPRECATION") done(status, ch.value ?: ByteArray(0))
        }

        override fun onCharacteristicRead(g: BluetoothGatt, ch: BluetoothGattCharacteristic, value: ByteArray, status: Int) =
done(status, value)

        override fun onCharacteristicWrite(g: BluetoothGatt, ch: BluetoothGattCharacteristic, status: Int) = done(status,
ByteArray(0))

        override fun onDescriptorWrite(g: BluetoothGatt, d: BluetoothGattDescriptor, status: Int) = done(status,
ByteArray(0))

        @Deprecated("Android 12 and older")
        override fun onCharacteristicChanged(g: BluetoothGatt, ch: BluetoothGattCharacteristic) {
            @Suppress("DEPRECATION") changed(ch, ch.value ?: ByteArray(0))
        }

        override fun onCharacteristicChanged(g: BluetoothGatt, ch: BluetoothGattCharacteristic, value: ByteArray) =
changed(ch, value)
    }

```

```
    private fun done(status: Int, value: ByteArray) {
        val d = op ?: return
        if (status == BluetoothGatt.GATT_SUCCESS) d.complete(value)
        else d.completeExceptionally(GattError(status))
    }

    /** A notification is cut to 20 bytes on a small link; one that does not parse is read in full. */
    private fun changed(ch: BluetoothGattCharacteristic, value: ByteArray) {
        if (ch.uuid != STAT) return
        val j = runCatching { JSONObject(String(value)) }.getOrNull()
        if (j != null) onStat?.invoke(j) else needFullStat = true
    }

    /** Set when a cut-off STAT notification came in; the waiting loop reads STAT then. */
    @Volatile var needFullStat = false

    class GattError(val status: Int) : IOException("Bluetooth error $status") {
        /** 5, 15 and 137: the link is not encrypted with a code the bin accepts. */
        val auth get() = status == 5 || status == 15 || status == 137
    }

    /** Runs one GATT call and waits for its answer. */
    private suspend fun call(timeout: Long = 8_000, start: (BluetoothGatt) -> Boolean): ByteArray = ops.withLock {
        val g = gatt ?: throw IOException("Not connected")
        val d = CompletableDeferred<ByteArray>()
        op = d
        try {
            if (!start(g)) throw IOException("Bluetooth is busy")
            withTimeout(timeout) { d.await() }
        } finally {
            op = null
        }
    }

    /**
     * Connects to the bin and finds its characteristics. A phone that remembers
     * an old pairing often gets dropped at once, so it tries 3 times before
     * saying so with [StaleBond].
     */
    suspend fun connect(device: BluetoothDevice) {
        close()
        watchBond(device)
        repeat(3) { i ->
            val ok = runCatching {
                val g = if (Build.VERSION.SDK_INT >= 26) device.connectGatt(app, false, callback,
BluetoothDevice.TRANSPORT_LE)
                else device.connectGatt(app, false, callback)
                gatt = g
                withTimeout(12_000) { linkUp.first { it } }
                delay(400)
                // A larger packet size, so a whole status message fits in one notification.
                runCatching { call(3_000) { it.requestMtu(247) } }
                call(10_000) { it.discoverServices() }
                val svc = gatt?.getService(SVC) ?: throw IOException("This is not a Smart-Bin")
                chars = HashMap(svc.characteristics.associateBy { it.uuid })
                if (STAT !in chars || SSID !in chars) throw IOException("This bin has no Wi-Fi setup")
            }.isSuccess
            if (ok) return
            close()
            if (i < 2) delay(1200)
        }
        throw StaleBond()
    }

    /**
     * Pairs, so Android asks for the bin's 6-digit code in its own box. Waits as
     * long as the code lasts on the bin (60 s), then reads STAT to prove the
     * link is encrypted. Returns the STAT message.
     */
    suspend fun secure(): JSONObject {
        val d = gatt?.device ?: throw IOException("Not connected")
        if (d.bondState != BluetoothDevice.BOND_BONDED) {
            if (d.bondState != BluetoothDevice.BOND_BONDING && !d.createBond()) throw IOException("Android would not start
pairing")
            val end = withTimeoutOrNull(75_000) { bond.first { it == BluetoothDevice.BOND_BONDED || it == BOND_FAILED } }
            if (end != BluetoothDevice.BOND_BONDED) throw IOException("No code was typed, or it was wrong")
            delay(600)
        }
        var last: Exception? = null
        repeat(3) {
            try {
                return readStat()
            } catch (e: GattError) {
                last = e
                if (e.auth) throw StaleBond()
                delay(1500)
```

```
            } catch (e: Exception) {
                last = e
                delay(1500)
            }
        }
        throw last ?: IOException("The bin did not answer")
    }

    // STAT holds the bin's setup state as JSON: WAITING, CONNECTING,
    // CONNECTED (with ip and ssid) or FAILED (with why).
    suspend fun readStat(): JSONObject = JSONObject(String(read(STAT)))

    /** Turns on STAT notifications. */
    suspend fun listen() {
        val ch = chars[STAT] ?: return
        val g = gatt ?: return
        g.setCharacteristicNotification(ch, true)
        val cccd = ch.getDescriptor(CCCD) ?: return
        val v = BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE
        call {
            if (Build.VERSION.SDK_INT >= 33) it.writeDescriptor(cccd, v) == 0
            else {
                @Suppress("DEPRECATION") cccd.value = v
                @Suppress("DEPRECATION") it.writeDescriptor(cccd)
            }
        }
    }

    /** The networks the bin found, strongest first. Empty while it is still scanning. */
    suspend fun networks(): List<Net> {
        if (SCAN !in chars) return emptyList()
        val a = JSONArray(String(read(SCAN)))
        return (0 until a.length()).mapNotNull { i ->
            when (val o = a.opt(i)) {
                is JSONObject -> o.optString("ssid").takeIf { it.isNotBlank() }?.let { Net(it, if (o.has("rssi"))
o.optInt("rssi") else null) }
                is String -> o.takeIf { it.isNotBlank() }?.let { Net(it, null) }
                else -> null
            }
        }
    }

    /** Hands over the network and asks the bin to join it. The answer comes on STAT. */
    suspend fun join(ssid: String, pass: String) {
        write(SSID, ssid.toByteArray())
        write(PASS, pass.toByteArray())
        write(CTRL, byteArrayOf(1))
    }

    /** Turns Bluetooth off on the bin; it stays on its Wi-Fi. */
    suspend fun off() {
        runCatching { write(CTRL, byteArrayOf(2)) }
        close()
    }

    private suspend fun read(uuid: UUID): ByteArray {
        val ch = chars[uuid] ?: throw IOException("Missing $uuid")
        return call { it.readCharacteristic(ch) }
    }

    private suspend fun write(uuid: UUID, value: ByteArray) {
        val ch = chars[uuid] ?: throw IOException("Missing $uuid")
        val type = BluetoothGattCharacteristic.WRITE_TYPE_DEFAULT
        call {
            if (Build.VERSION.SDK_INT >= 33) it.writeCharacteristic(ch, value, type) == 0
            else {
                @Suppress("DEPRECATION") ch.value = value
                ch.writeType = type
                @Suppress("DEPRECATION") it.writeCharacteristic(ch)
            }
        }
    }

    fun close() {
        val g = gatt
        gatt = null
        linkUp.value = false
        runCatching { g?.disconnect() }
        runCatching { g?.close() }
    }

    /** Ends everything, when the page or the app goes away. */
    fun release() {
        close()
        receiver?.let { runCatching { app.unregisterReceiver(it) } }
        receiver = null
```

```
    }

    // Follows the pairing (bond) state so a refused passkey is noticed.
    private fun watchBond(device: BluetoothDevice) {
        bond.value = device.bondState
        if (receiver != null) return
        val r = object : BroadcastReceiver() {
            override fun onReceive(c: Context, i: Intent) {
                val d: BluetoothDevice? = if (Build.VERSION.SDK_INT >= 33) i.getParcelableExtra(BluetoothDevice.EXTRA_DEVICE,
BluetoothDevice::class.java)
                else @Suppress("DEPRECATION") i.getParcelableExtra(BluetoothDevice.EXTRA_DEVICE)
                if (d == null || d.address != gatt?.device?.address) return
                val now = i.getIntExtra(BluetoothDevice.EXTRA_BOND_STATE, BluetoothDevice.BOND_NONE)
                val before = i.getIntExtra(BluetoothDevice.EXTRA_PREVIOUS_BOND_STATE, BluetoothDevice.BOND_NONE)
                bond.value = if (now == BluetoothDevice.BOND_NONE && before == BluetoothDevice.BOND_BONDING) BOND_FAILED else
now
            }
        }
        val filter = IntentFilter(BluetoothDevice.ACTION_BOND_STATE_CHANGED)
        if (Build.VERSION.SDK_INT >= 33) app.registerReceiver(r, filter, Context.RECEIVER_EXPORTED)
        else app.registerReceiver(r, filter)
        receiver = r
    }

    /**
     * Makes Android forget the bin, for a pairing the bin no longer knows. There
     * is no public call for it, so this uses the hidden one most phones keep;
     * the caller opens Android's Bluetooth settings when it fails.
     */
    fun forget(device: BluetoothDevice): Boolean = runCatching {
        device.javaClass.getMethod("removeBond").invoke(device) as? Boolean ?: false
    }.getOrDefault(false)

    companion object {
        // UUIDs b1a70001..6, the same as BT_*_UUID in the firmware's config.h.
        private fun u(n: String) = UUID.fromString("b1a7000$n-5c2e-4f8a-9d33-0a9c2b6f17c1")
        val SVC: UUID = u("1")
        val SSID: UUID = u("2")
        val PASS: UUID = u("3")
        val CTRL: UUID = u("4")
        val STAT: UUID = u("5")
        val SCAN: UUID = u("6")
        val CCCD: UUID = UUID.fromString("00002902-0000-1000-8000-00805f9b34fb")
        private const val BOND_FAILED = -1
    }
}
```

Android App/app/src/main/java/smartbin/data/OperatorLock.kt

```
// ================================================================================
// data/OperatorLock.kt - PIN that guards the Operator role on this phone
// This is the app's own PIN, separate from the bin's Collect PIN.
// ================================================================================
package smartbin.data

import android.util.Base64
import java.security.MessageDigest
import java.security.SecureRandom
import javax.crypto.SecretKeyFactory
import javax.crypto.spec.PBEKeySpec

/**
 * The operator PIN. Only a salted PBKDF2 hash is kept, never the PIN. Wrong
 * tries are counted on disk, so closing the app does not reset them: after 5
 * the lock waits 30 s, and the wait doubles with every miss after that.
 */
class OperatorLock(private val store: Store) {
    val hasPin get() = store.string(KEY_HASH, "").isNotEmpty()

    /** Milliseconds until another try is allowed, 0 when one is. */
    fun waitLeft(now: Long = System.currentTimeMillis()) = (store.string(KEY_UNTIL, "0").toLongOrNull() ?:
0L).minus(now).coerceAtLeast(0L)

    fun set(pin: String) {
        val salt = ByteArray(16).also { SecureRandom().nextBytes(it) }
        store.putString(KEY_SALT, b64(salt))
        store.putString(KEY_HASH, b64(hash(pin, salt)))
        store.putString(KEY_MISSES, "0")
        store.putString(KEY_UNTIL, "0")
    }

    /** True when the PIN matches. A miss counts toward the wait. */
    fun check(pin: String): Boolean {
        if (!hasPin || waitLeft() > 0) return false
        val salt = Base64.decode(store.string(KEY_SALT, ""), Base64.NO_WRAP)
```

```
        val want = Base64.decode(store.string(KEY_HASH, ""), Base64.NO_WRAP)
        if (MessageDigest.isEqual(hash(pin, salt), want)) {
            store.putString(KEY_MISSES, "0")
            return true
        }
        val misses = (store.string(KEY_MISSES, "0").toIntOrNull() ?: 0) + 1
        store.putString(KEY_MISSES, misses.toString())
        if (misses >= FREE_TRIES) {
            val wait = 30_000L shl (misses - FREE_TRIES).coerceAtMost(6)
            store.putString(KEY_UNTIL, (System.currentTimeMillis() + wait).toString())
        }
        return false
    }

    /** Tries left before the lock starts waiting. */
    fun triesLeft() = (FREE_TRIES - (store.string(KEY_MISSES, "0").toIntOrNull() ?: 0)).coerceAtLeast(0)

    private fun hash(pin: String, salt: ByteArray): ByteArray =
        SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")
            .generateSecret(PBEKeySpec(pin.toCharArray(), salt, 20_000, 256)).encoded

    private fun b64(b: ByteArray) = Base64.encodeToString(b, Base64.NO_WRAP)

    companion object {
        const val MIN = 4
        const val MAX = 8
        private const val FREE_TRIES = 5
        private const val KEY_HASH = "op_pin_hash"
        private const val KEY_SALT = "op_pin_salt"
        private const val KEY_MISSES = "op_pin_misses"
        private const val KEY_UNTIL = "op_pin_until"

        fun valid(pin: String) = pin.length in MIN..MAX && pin.all { it.isDigit() }
    }
}
```

Android App/app/src/main/java/smartbin/data/Demo.kt

```
// ================================================================================
// data/Demo.kt - made-up bins for demonstrations without hardware
// Demo bins exist only while DEMO is switched on at the start page and are
// never saved. step() moves their readings a little every tick so the charts
// and alerts behave like real bins.
// ================================================================================
package smartbin.data

import kotlin.math.floor
import kotlin.math.roundToInt
import kotlin.random.Random

/**
 * Invented readings for the demo bins that DEMO on the start page adds, so the
 * whole app works with no hardware. The numbers move the way the dashboard's demo
 * moves them, and stay inside what the HC-SR04, HX711 and BME680 can report.
 * One demo minute fills a bin as much as one real hour would.
 */
object Demo {
    /** How many times faster than real life the demo bins fill. Demo options can change it; 60 is normal. */
    const val NORMAL_SPEED = 60.0
    var speed = NORMAL_SPEED

    /** The three demo bins, named as on the dashboard. */
    fun bins(): List<Bin> = listOf(
        bin(0, "Demo Bin 1", "Main Hall", fill = 34.0, weight = 1430.0, temp = 28.0, hum = 62.0, gas = 72.0, rate = 0.38, id = "demo1"),
        bin(1, "Demo Bin 2", "Cafeteria", fill = 62.0, weight = 2570.0, temp = 28.1, hum = 63.0, gas = 118.0, rate = 3.1, id = "demo2"),
        bin(2, "Demo Bin 3", "Library", fill = 88.0, weight = 3650.0, temp = 28.4, hum = 61.0, gas = 96.0, rate = 0.9, id = "demo3"),
    ).map { it.copy(isDemo = true, cloudId = "demo%02d".format(it.id.last().digitToInt())) }

    private val PLACES = listOf("Main Hall", "Cafeteria", "Library", "Gym", "Lobby", "Parking", "Admin", "Lab A", "Lab B", "Clinic", "Chapel", "Field")

    /** A new real bin from the Add sheet. It shows "not answering" until it gets an address or a cloud link. */
    fun newBin(index: Int, id: String): Bin =
        bin(index, "Bin %02d".format(index + 1), "New location", fill = 0.0, weight = 0.0, temp = 27.6, hum = 58.0, gas = 40.0, rate = 0.0, id = id)
            .copy(source = Source.STALE)

    fun placeFor(index: Int) = PLACES[index % PLACES.size]

    // Builds one bin with a starting reading and fill rate. Also used by
    // newBin() for real bins added from the Add sheet.
    private fun bin(
```

```
        i: Int, name: String, place: String, fill: Double, weight: Double, temp: Double, hum: Double, gas: Double, rate: Double,
        id: String = "b${i + 1}",
    ): Bin {
        val cal = DEFAULT_CAL
        return Bin(
            id = id, cloudId = "bin%02d".format(i + 1), name = name, place = place, cal = cal, rate = rate,
            reading = Reading(
                fill = fill, weight = weight, temp = temp, tempRaw = temp - cal.v("temp_off"), hum = hum, press = 1008.4, gas = gas,
                distance = distanceFor(fill, cal), at = System.currentTimeMillis(),
            ),
        )
    }

    // Inverse of the firmware's fill formula, so demo distance matches fill.
    fun distanceFor(fill: Double, cal: Cal) = cal.v("d_empty") - fill / 100.0 * (cal.v("d_empty") - cal.v("d_full"))

    /** One second of the demo for one bin. [calibrating] is the app's calibration switch. */
    fun step(b: Bin, calibrating: Boolean, now: Long): Reading {
        val r = b.reading
        val s = b.sim
        val c = b.cal
        val sv = s.values
        // BME680: the air quality index drifts toward a level set by how full the bin is;
        // the raw temperature carries about 2 C of self-heating that the offset removes.
        val gas = if (s.on(2)) sv["gas"] ?: r.gas else (r.gas + ((35 + r.fill * 2.2) - r.gas) * 0.02 + (Random.nextDouble() - 0.5) * 5).coerceIn(10.0, 500.0)
        val temp: Double
        val tempRaw: Double
        val hum: Double
        if (s.on(3)) {
            temp = sv["temp"] ?: r.temp
            tempRaw = temp - c.v("temp_off")
            hum = sv["humidity"] ?: r.hum
        } else {
            tempRaw = (r.tempRaw + (Random.nextDouble() - 0.5) * 0.08).coerceIn(24.0, 34.0)
            temp = tempRaw + c.v("temp_off")
            hum = (r.hum + (Random.nextDouble() - 0.5) * 0.6).coerceIn(35.0, 95.0)
        }
        val press = (r.press + (Random.nextDouble() - 0.5) * 0.06).coerceIn(985.0, 1030.0)
        // After RESET_GAS the air sensor warms up for 20 demo seconds, counting down as the real one does.
        val warm = if (r.iaqAcc == 0) ((r.iaqWarmS ?: 20) - 1).coerceAtLeast(0) else null
        val iaqAcc = when {
            r.iaqAcc == 0 -> if (warm == 0) 1 else 0
            r.iaqAcc < 3 && Random.nextDouble() < 0.05 -> r.iaqAcc + 1
            else -> r.iaqAcc
        }
        val base = r.copy(
            gas = gas, temp = temp, tempRaw = tempRaw, hum = hum, press = press, iaqAcc = iaqAcc, at = now,
            iaqWarmS = if (iaqAcc == 0) warm else null, gasK = if (iaqAcc == 0) 140.0 + Random.nextDouble() * 6 else null,
        )

        if (calibrating) {
            return base.copy(status = BinStatus.CALIBRATING, lidOpen = false, distance = c.v("d_empty") - Random.nextDouble() * 0.4)
        }
        val leaving = r.status == BinStatus.CALIBRATING
        var status = if (leaving) BinStatus.OK else r.status
        val fill = when {
            s.on(0) -> (sv["fill"] ?: r.fill).coerceIn(0.0, 100.0)
            status == BinStatus.COLLECTING -> r.fill
            else -> (r.fill + b.rate * speed / 3600.0 * (0.7 + Random.nextDouble() * 0.6)).coerceIn(0.0, 100.0)
        }
        val weight = if (s.on(1)) sv["weight"] ?: r.weight
        else (fill / 100 * c.v("weight_cap") * 0.92 + Random.nextDouble() * 25).coerceIn(0.0, 5000.0).roundToInt().toDouble()
        val tilt = if (s.on(4) && (sv["tamper"] ?: 0.0) > 0) 38.0 else Random.nextDouble() * 2.2
        val motion = if (s.on(5)) (sv["motion"] ?: 0.0) > 0 else Random.nextDouble() < 0.025
        val distance = floor(distanceFor(fill, c) * 10) / 10
        var lid: Boolean
        when {
            status == BinStatus.COLLECTING -> lid = true
            tilt > c.v("tilt_th") -> { status = BinStatus.TAMPER; lid = false }
            status == BinStatus.TAMPER -> { lid = false }
            status == BinStatus.LOCKED -> lid = false
            c.v("auto_lock") > 0 && fill >= c.v("fill_th") -> { status = BinStatus.LOCKED; lid = false }
            else -> lid = if (s.on(6)) (sv["lid"] ?: 0.0) > 0 else motion
        }
        return base.copy(fill = fill, weight = weight, tilt = tilt, motion = motion, distance = distance, lidOpen = lid, status = status)
    }

    /** A day of made-up history so the charts have something to draw in demo mode. */
    fun history(b: Bin, now: Long): List<Sample> {
        val n = 24 * 60
        val rnd = Random(b.id.hashCode())
```

```
        val out = ArrayList<Sample>(n)
        var fill = b.reading.fill
        var temp = b.reading.temp
        var hum = b.reading.hum
        var press = b.reading.press
        var gas = b.reading.gas
        for (k in 0 until n) {
            out.add(Sample(now - k * 60_000L, fill.toFloat(), temp.toFloat(), hum.toFloat(), press.toFloat(), gas.toFloat()))
            // Backwards in time: the bin was emptier before, and was emptied every few hours.
            fill -= b.rate / 60.0 + (rnd.nextDouble() - 0.5) * 0.25
            if (fill < 2) fill = 60 + rnd.nextDouble() * 25
            temp = (temp + (rnd.nextDouble() - 0.5) * 0.3).coerceIn(22.0, 36.0)
            hum = (hum + (rnd.nextDouble() - 0.5) * 1.1).coerceIn(35.0, 95.0)
            press = (press + (rnd.nextDouble() - 0.5) * 0.18).coerceIn(990.0, 1025.0)
            gas = (gas + (rnd.nextDouble() - 0.5) * 7 - (fill - 30) * 0.004).coerceIn(15.0, 400.0)
        }
        return out.asReversed()
    }

    /** The mockup's example history for the demo bins, so the alert tools have something to act on. */
    fun alerts(bins: List<Bin>, now: Long): List<AlertRow> {
        val a = bins.getOrNull(0)?.name ?: "Bin 01"
        val b = bins.getOrNull(1)?.name ?: a
        val m = 60_000L
        val rows = listOf(
            Triple(2, Severity.CRITICAL, listOf("fill", "Fill threshold passed", "$b is at 81%")),
            Triple(9, Severity.WARNING, listOf("gas", "Air quality warning", "$b index 212, limit 200")),
            Triple(34, Severity.CRITICAL, listOf("tamper", "Tamper detected", "$a was moved or tilted")),
            Triple(41, Severity.WARNING, listOf("humid", "Humidity high", "$b at 88%, limit 85%")),
            Triple(60, Severity.INFO, listOf("lock", "Bin locked", "$a locked from the app")),
            Triple(62, Severity.SUCCESS, listOf("unlock", "Bin unlocked", "$a is open for use again")),
            Triple(180, Severity.SUCCESS, listOf("truck", "Collection complete", "$b is back to normal")),
            Triple(300, Severity.INFO, listOf("calib", "Calibration started", "$a is being calibrated")),
            Triple(302, Severity.SUCCESS, listOf("save", "Settings saved", "$a: empty 30 cm, full 5 cm, alert at 80%")),
            Triple(1260, Severity.CRITICAL, listOf("temp", "Temperature high", "$a at 41.2 °C")),
            Triple(4320, Severity.SUCCESS, listOf("truck", "Collection complete", "$a is back to normal")),
            Triple(17280, Severity.CRITICAL, listOf("fill", "Fill threshold passed", "$a is at 83%")),
        )
        return rows.mapIndexed { i, (mins, sev, x) ->
            val icon = x[0]
            val shown = when (icon) { "unlock" -> "check"; "save" -> "check"; else -> icon }
            AlertRow(
                id = (i + 1).toLong(), sev = sev, icon = shown, kind = kindFor(icon), title = x[1], meta = x[2],
                at = now - mins * m, read = i >= 3, demo = true,
            )
        }
    }

    // Maps an alert icon name to the filter group on the Alerts page.
    fun kindFor(icon: String): AlertKind = when (icon) {
        "fill" -> AlertKind.FULL
        "temp", "humid", "gas" -> AlertKind.SENSOR
        "tamper" -> AlertKind.TAMPER
        "lock", "unlock" -> AlertKind.LOCK
        "truck" -> AlertKind.PICKUP
        else -> AlertKind.SETUP
    }
}
```

Android App/app/src/main/java/smartbin/data/Hub.kt

```
// ============================================================================
// data/Hub.kt - the app's data engine
// tick() runs once a second (every 5 s in the background): it polls every bin
// in parallel, merges the answers into HubState, settles cloud commands,
// raises alerts and samples history for the charts. Screens never call the
// network themselves; they read Hub.state and call the functions below.
// Debug tip: a bin's Source (LIVE, CLOUD, THINGSPEAK, STALE) shows which route
// answered last; the link chip on screen shows the same thing.
// ============================================================================
package smartbin.data

import android.content.Context
import android.net.ConnectivityManager
import android.net.Network
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.Job
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.async
import kotlinx.coroutines.awaitAll
import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableSharedFlow
```

```
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharedFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch
import kotlin.math.max

/** The kinds of alert, one notification channel and one sound each. */
enum class AlertType { FULL, TAMPER, SENSOR, LOCK, PICKUP, INFO }

// One-off events for sounds and notifications (collected in SmartBinApp).
sealed interface HubEvent {
    data class NewAlert(val row: AlertRow, val type: AlertType, val binId: String) : HubEvent
    data class LinkChanged(val from: Link, val to: Link) : HubEvent
    data class BinLost(val name: String) : HubEvent
    data class BinBack(val name: String) : HubEvent
    /** A cloud command was confirmed by the bin, refused ([why]), or ran out of time. */
    data class CommandResult(val binId: String, val name: String, val cmd: String, val ok: Boolean, val why: String? =
null) : HubEvent
    /** A sound for something the demo tour did, named as in the Sfx list (LOCK, PICKUP, ...). */
    data class Sound(val name: String) : HubEvent
}

/**
 * The one place that talks to the bins. It reads every bin once a second
 * (every five while the app is closed), local Wi-Fi first and Firebase second,
 * keeps a day of history for the charts, turns changes into alerts, and runs
 * the commands. Screens only read [state] and call these functions.
 *
 * Everything runs on the main thread; only the network calls leave it.
 */
object Hub {
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.Main.immediate)
    private lateinit var store: Store
    private var loop: Job? = null
    private var started = false

    private val _state = MutableStateFlow(HubState())
    val state: StateFlow<HubState> = _state

    private val _history = MutableStateFlow<Map<String, List<Sample>>>(emptyMap())
    val history: StateFlow<Map<String, List<Sample>>> = _history

    private val _events = MutableSharedFlow<HubEvent>(extraBufferCapacity = 32)
    val events: SharedFlow<HubEvent> = _events

    lateinit var cloud: Cloud
        private set
    lateinit var ts: ThingSpeak
        private set

    /** True while the app is on screen. Off screen the bins are read less often. */
    @Volatile var foreground = true

    /**
     * True once the user is past the start page and the role picker in this
     * run of the app. Alerts reach the notification shade only then, and only
     * while the app is in the background: never from the start page, and
     * never after the app is swiped away or closed.
     */
    @Volatile var inMain = false

    private val nextLocal = HashMap<String, Long>()
    private val localFails = HashMap<String, Int>()
    private val nextCloud = HashMap<String, Long>()
    private val nextTs = HashMap<String, Long>()
    private var fbOk = 0L
    private var tsOk = 0L
    private var lastCheck = 0L
    /** Per channel: the first and the latest (time, entry number) seen, for the message estimate. */
    private val tsSeen = HashMap<String, Pair<Pair<Long, Long>, Pair<Long, Long>>>()
    /** Demo options: bins pretending to have gone quiet. */
    private val demoQuiet = HashSet<String>()
    /** Demo options: when each pretend cloud command is carried out, by bin. */
    private val demoAnswer = HashMap<String, Pair<Long, String>>()
    private val simDirty = HashSet<String>()
    private val prev = HashMap<String, Flags>()
    private var lastSample = 0L
    private var lastUsage = 0L
    private var alertSeq = 0L

    // The alert-worthy facts of a bin from the last tick. watch() compares the
    // new Flags with these to raise an alert only when something changes.
    private data class Flags(
        val full: Boolean, val hot: Boolean, val humid: Boolean, val gassy: Boolean, val status: BinStatus, val source:
Source,
```

```
        val faults: String, val acc: Int,
    )

    /** Demo tour: readings held on a demo bin while the tour acts something out. */
    private val holds = HashMap<String, (Reading) -> Reading>()
    private var tourStep = 0
    private var tourNext = 0L
    private var tourPauseUntil = 0L

    /** Firebase watchers: this phone's id, and when it last told the bins it is watching. */
    private var watcherId = ""
    private var watchedAt = 0L
    private var watching = false
    /** Firebase alerts written by the bins while the app was away, read once per bin per run. */
    private val alertsRead = HashSet<String>()
    private var alertsCheck = 0L
    private var startedAt = 0L

    // Runs once per process. Loads saved bins (marked STALE until they answer),
    // listens for network changes, and starts the polling loop.
    fun init(context: Context) {
        if (started) return
        started = true
        store = Store(context)
        cloud = Cloud(store.firebase())
        ts = ThingSpeak().apply { use(store.string(KEY_TS_URL, THINGSPEAK_URL)) }
        val now = System.currentTimeMillis()
        startedAt = now
        watcherId = store.string(KEY_WATCHER, "").ifBlank { "app-" + java.util.UUID.randomUUID().toString().take(10) }.also
{ store.putString(KEY_WATCHER, it) }
        var bins = store.bins() ?: emptyList()
        var alerts = store.alerts() ?: emptyList()
        // Version 1 kept its demo bins in the saved list, as bins with no address. Version 2 makes
        // demo bins only when DEMO is on at the start page, so those old ones go, once.
        if (!store.bool(KEY_V2, false)) {
            // b1 and b2 were the demo bins; a bin with no address, channel or cloud account was demo data too.
            val real = bins.filter { it.ip.isNotBlank() || it.onThingSpeak || (cloud.ready && it.id !in setOf("b1", "b2")) }
            if (real.isEmpty()) alerts = emptyList()
            bins = real
            store.saveBins(bins)
            store.saveAlerts(alerts)
            store.putBool(KEY_V2, true)
        }
        bins = bins.map { it.copy(source = Source.STALE) }
        alertSeq = alerts.maxOfOrNull { it.id } ?: 0L
        _state.value = HubState(bins = bins, alerts = alerts, link = linkOf(bins), loading = true)
        _history.value = emptyMap()

        // A new network (joining the bins' Wi-Fi, or leaving it) means try Wi-Fi again at once.
        runCatching {
            val cm = context.getSystemService(ConnectivityManager::class.java)
            cm?.registerDefaultNetworkCallback(object : ConnectivityManager.NetworkCallback() {
                override fun onAvailable(network: Network) {
                    scope.launch { nextLocal.clear(); localFails.clear(); nextCloud.clear(); nextTs.clear(); lastCheck = 0L }
                }
            })
        }
        loop = scope.launch {
            while (isActive) {
                val began = System.currentTimeMillis()
                try {
                    tick()
                } catch (e: CancellationException) {
                    throw e
                } catch (_: Exception) {
                    // One bad round never stops the loop.
                }
                val period = if (foreground) 1000L else 5000L
                delay(max(200L, period - (System.currentTimeMillis() - began)))
            }
        }
    }

    // ---------- the loop ----------

    // One round: poll all bins at once, merge, then derived work. The merge
    // keeps edits the user made while the network calls were in flight.
    private suspend fun tick() {
        val now = System.currentTimeMillis()
        val before = _state.value
        val polled = coroutineScope { before.bins.map { b -> async { poll(b, now) } }.awaitAll() }
        // Bins may have been added, edited or removed while the calls were out: keep those edits.
        val byId = polled.associateBy { it.id }
        val current = _state.value
        val merged = current.bins.map { b ->
            val p = byId[b.id] ?: return@map b
```

```
            val was = before.bins.firstOrNull { it.id == b.id } ?: return@map b
            // An edit made while the calls were out wins over the bin's answer.
            if (b.ip != was.ip) return@map b
            b.copy(
                reading = p.reading, source = p.source, lastSeen = p.lastSeen,
                cal = if (p.cal !== was.cal && b.cal === was.cal) p.cal else b.cal,
                ip = p.ip,
                sim = if (b.id in simDirty) b.sim else p.sim,
            )
        }
        val link = linkOf(merged)
        if (link != current.link && !current.loading) _events.tryEmit(HubEvent.LinkChanged(current.link, link))
        checkRoutes(merged, now)
        val pending = settle(_state.value.pending, merged, now)
        _state.value = current.copy(
            pending = pending,
            bins = merged, link = link, loading = false,
            fbUp = if (cloud.ready) now - fbOk < 60_000 else null,
            tsUp = if (merged.any { it.onThingSpeak }) now - tsOk < 60_000 else null,
            tsPerDay = tsPerDay(merged),
        )
        if (merged.any { it.ip != (current.bins.firstOrNull { c -> c.id == it.id }?.ip) }) persist(merged)
        watch(merged, now)
        tour(now)
        watchers(merged, now)
        cloudAlerts(merged, now)
        if (now - lastSample >= 10_000) {
            lastSample = now
            sample(merged, now)
        }
        if (current.fbUp == true && now - lastUsage > 60_000) {
            lastUsage = now
            runCatching { cloud.usageBytes() }.getOrNull()?.let { _state.value = _state.value.copy(cloudBytes = it) }
        }
    }

    // Reads one bin by the first route that answers: demo data, local Wi-Fi,
    // Firebase, then ThingSpeak. Cloud readings older than 90 s are ignored so
    // a dead bin is not shown as live.
    private suspend fun poll(b: Bin, now: Long): Bin {
        if (b.demo) {
            // Demo options can make a bin go quiet; it keeps its last reading until it comes back.
            if (b.id in demoQuiet) {
                demoAnswer.remove(b.id)
                return b.copy(source = Source.STALE)
            }
            // A pretend cloud command is carried out a few seconds after it was sent, as a real bin would.
            var from = b
            demoAnswer[b.id]?.let { (at, cmd) ->
                if (now >= at) {
                    demoAnswer.remove(b.id)
                    from = b.copy(reading = effect(b.reading, cmd))
                }
            }
            var r = Demo.step(from, _state.value.calibBin == b.id, now)
            holds[b.id]?.let { r = it(r) }
            return from.copy(reading = r, source = demoSource(_state.value.demoLinks), lastSeen = now)
        }
        // Local Wi-Fi first: every second while it answers. After a miss it is tried
        // again in 2 s, and after three misses in a row every 30 s, as on the dashboard.
        if (b.ip.isNotBlank() && now >= (nextLocal[b.id] ?: 0L)) {
            try {
                val s = LocalApi.status(b.ip)
                nextLocal.remove(b.id)
                localFails.remove(b.id)
                return b.copy(
                    reading = s.reading, source = Source.LIVE, lastSeen = now,
                    cal = s.cal ?: b.cal,
                    sim = if (s.simMask != null && b.id !in simDirty) b.sim.copy(mask = s.simMask) else b.sim,
                    fw = s.fw.ifBlank { b.fw }, rssi = s.rssi ?: b.rssi, uptimeS = s.uptimeS ?: b.uptimeS, touchCustom =
s.touchCustom ?: b.touchCustom,
                )
            } catch (e: CancellationException) {
                throw e
            } catch (_: Exception) {
                val fails = (localFails[b.id] ?: 0) + 1
                localFails[b.id] = fails
                nextLocal[b.id] = now + if (fails < 3) 2_000 else 30_000
            }
        }
        // Then Firebase, every 5 s, when an account is set up.
        if (cloud.ready) {
            if (now >= (nextCloud[b.id] ?: 0L)) {
                nextCloud[b.id] = now + 5_000
                try {
                    val c = cloud.status(b.cloudId)
```

```
                    fbOk = now
                    val age = if (c != null && c.ts > 0) now - c.ts else 0L
                    if (c != null && age < 90_000) {
                        return b.copy(
                            reading = c.status.reading, source = Source.CLOUD, lastSeen = now,
                            cal = c.status.cal ?: b.cal,
                            // Each bin writes its own address, so a new one is picked up without typing it.
                            ip = c.localIp ?: b.ip,
                            fw = c.status.fw.ifBlank { b.fw }, rssi = c.status.rssi ?: b.rssi, uptimeS = c.status.uptimeS ?: b.uptimeS,
                        )
                    }
                } catch (e: CancellationException) {
                    throw e
                } catch (_: Exception) {
                }
            } else if (b.source == Source.CLOUD && now - b.lastSeen < 90_000) {
                return b
            }
        }
        // Then ThingSpeak, every 15 s, the fastest its free licence lets a bin write.
        if (b.onThingSpeak) {
            if (now >= (nextTs[b.id] ?: 0L)) {
                nextTs[b.id] = now + 15_000
                try {
                    val e = ts.last(b.tsChannel, b.tsKey)
                    tsOk = now
                    e?.let { noteEntry(b.tsChannel, it.entryId, now) }
                    val age = if (e != null && e.ts > 0) now - e.ts else 0L
                    if (e != null && age < 90_000) {
                        return b.copy(reading = e.reading, source = Source.THINGSPEAK, lastSeen = now, ip = e.localIp ?: b.ip)
                    }
                } catch (e: CancellationException) {
                    throw e
                } catch (_: Exception) {
                }
            } else if (b.source == Source.THINGSPEAK && now - b.lastSeen < 90_000) {
                return b
            }
        }
        // A bin is called lost only after 90 s with no reading at all; until then its last one stays.
        if ((b.source == Source.LIVE || b.source == Source.CLOUD || b.source == Source.THINGSPEAK) && now - b.lastSeen < 90_000) return b
        return b.copy(source = Source.STALE)
    }

    // The overall link shown in the header: the best route any bin is using.
    private fun linkOf(bins: List<Bin>, pretend: Routes = _state.value.demoLinks): Link = when {
        bins.isEmpty() -> Link.OFFLINE
        bins.all { it.demo } -> when (demoSource(pretend)) {
            Source.LIVE -> Link.LIVE
            Source.CLOUD -> Link.CLOUD
            Source.THINGSPEAK -> Link.THINGSPEAK
            else -> Link.DEMO
        }
        bins.any { it.source == Source.LIVE } -> Link.LIVE
        bins.any { it.source == Source.CLOUD } -> Link.CLOUD
        bins.any { it.source == Source.THINGSPEAK } -> Link.THINGSPEAK
        else -> Link.OFFLINE
    }

    /**
     * Firebase and ThingSpeak are only read for data when Wi-Fi fails, so every
     * 30 s the one not in use gets a small call, and the link chip can show each
     * route that answers.
     */
    private suspend fun checkRoutes(bins: List<Bin>, now: Long) {
        if (now - lastCheck < 30_000) return
        lastCheck = now
        if (cloud.ready && now - fbOk > 30_000 && cloud.ping()) fbOk = now
        // Every channel is read here too, so the message estimate covers all of them.
        bins.filter { it.onThingSpeak && it.source != Source.THINGSPEAK }.distinctBy { it.tsChannel }.forEach { t ->
            runCatching { ts.last(t.tsChannel, t.tsKey) }.onSuccess { e ->
                tsOk = now
                e?.let { noteEntry(t.tsChannel, it.entryId, now) }
            }
        }
    }

    /** Drops each cloud command the bin has now carried out, or that waited too long. */
    private fun settle(pending: Map<String, Pending>, bins: List<Bin>, now: Long): Map<String, Pending> {
        if (pending.isEmpty()) return pending
        val out = HashMap<String, Pending>()
        pending.forEach { (id, p) ->
            val b = bins.firstOrNull { it.id == id } ?: return@forEach
```

```
            val fresh = b.lastSeen > p.at
            val done = fresh && when (p.cmd) {
                "LOCK" -> b.reading.status == BinStatus.LOCKED
                "UNLOCK" -> b.reading.status != BinStatus.LOCKED
                "COLLECT" -> b.reading.status == BinStatus.COLLECTING
                else -> true
            }
            when {
                done -> _events.tryEmit(HubEvent.CommandResult(id, b.name, p.cmd, true))
                now > p.deadline -> _events.tryEmit(HubEvent.CommandResult(id, b.name, p.cmd, false))
                else -> out[id] = p
            }
        }
        return out
    }

    // Remembers the first and latest ThingSpeak entry number seen per channel,
    // for the messages-per-day estimate.
    private fun noteEntry(channel: String, entry: Long, now: Long) {
        if (entry <= 0) return
        val seen = tsSeen[channel]
        tsSeen[channel] = if (seen == null || entry < seen.second.second) (now to entry) to (now to entry) else seen.first to
(now to entry)
    }

    /** Sums each channel's writes per day, once it has been watched for two minutes. */
    private fun tsPerDay(bins: List<Bin>): Double? {
        val channels = bins.filter { it.onThingSpeak }.map { it.tsChannel }.toSet()
        var total = 0.0
        var any = false
        channels.forEach { ch ->
            val (a, b) = tsSeen[ch] ?: return@forEach
            val ms = b.first - a.first
            if (ms < 120_000) return@forEach
            any = true
            total += (b.second - a.second).toDouble() / ms * 86_400_000.0
        }
        return if (any) total else null
    }

    // Every 10 s, adds one history point per bin and drops points older than
    // 24 h. The charts on the Analytics page draw these lists.
    private fun sample(bins: List<Bin>, now: Long) {
        val cut = now - 24 * 3600_000L
        val h = _history.value.toMutableMap()
        bins.forEach { b ->
            if (b.source == Source.STALE) return@forEach
            val r = b.reading
            val list = (h[b.id] ?: emptyList()).dropWhile { it.t < cut } +
                Sample(now, r.fill.toFloat(), r.temp.toFloat(), r.hum.toFloat(), r.press.toFloat(), r.gas.toFloat())
            h[b.id] = list
        }
        h.keys.retainAll(bins.map { it.id }.toSet())
        _history.value = h
    }

    /** Turns changes into alerts. The first reading of a bin only sets the starting point. */
    private fun watch(bins: List<Bin>, now: Long) {
        bins.forEach { b ->
            val r = b.reading
            val c = b.cal
            val f = Flags(
                full = r.fill >= c.v("fill_th") && !r.fault("HC-SR04"), hot = r.temp >= c.v("temp_th") && !r.fault("BME680"),
                humid = r.hum >= c.v("hum_th") && !r.fault("BME680"), gassy = r.gas >= c.v("gas_th") && !r.warming
&& !r.fault("BME680"),
                status = r.status, source = b.source, faults = r.faults.joinToString(", "), acc = r.iaqAcc,
            )
            val p = prev[b.id]
            prev[b.id] = f
            if (p == null) return@forEach
            val n = b.name
            if (f.full && !p.full) add(b, Severity.CRITICAL, "fill", "Fill threshold passed", "$n is at ${r.fill.toInt()}%",
AlertType.FULL)
            if (f.hot && !p.hot) add(b, Severity.CRITICAL, "temp", "Temperature high", "$n at ${"%.1f".format(r.temp)} °C",
AlertType.SENSOR)
            if (f.humid && !p.humid) add(b, Severity.WARNING, "humid", "Humidity high", "$n at ${r.hum.toInt()}%, limit
${c.v("hum_th").toInt()}%", AlertType.SENSOR)
            if (f.gassy && !p.gassy) add(b, Severity.WARNING, "gas", "Air quality warning", "$n index ${r.gas.toInt()}, limit
${c.v("gas_th").toInt()}", AlertType.SENSOR)
            if (f.status != p.status) when (f.status) {
                BinStatus.TAMPER -> add(b, Severity.CRITICAL, "tamper", "Tamper detected", "$n was moved or tilted",
AlertType.TAMPER)
                BinStatus.LOCKED -> add(b, Severity.INFO, "lock", "Bin locked", "$n locked from the app", AlertType.LOCK)
                BinStatus.CALIBRATING -> add(b, Severity.INFO, "calib", "Calibration started", "$n is being calibrated",
AlertType.INFO, kind = AlertKind.SETUP)
                BinStatus.COLLECTING -> {}
```

```
                BinStatus.OK -> when (p.status) {
                    BinStatus.LOCKED -> add(b, Severity.SUCCESS, "check", "Bin unlocked", "$n is open for use again",
AlertType.LOCK, kind = AlertKind.LOCK)
                    BinStatus.COLLECTING -> add(b, Severity.SUCCESS, "truck", "Collection complete", "$n is back to normal",
AlertType.PICKUP)
                    else -> {}
                }
            }
            if (f.faults.isNotEmpty() && f.faults != p.faults) add(b, Severity.WARNING, "gear", "Sensor not answering", "$n:
${f.faults}", AlertType.SENSOR)
            if (f.faults.isEmpty() && p.faults.isNotEmpty()) add(b, Severity.SUCCESS, "check", "Sensor back", "$n reads every
sensor again", AlertType.INFO, kind = AlertKind.SENSOR)
            if (p.acc == 0 && f.acc > 0) add(b, Severity.SUCCESS, "gas", "Air sensor ready", "$n air quality readings are
settled", AlertType.INFO, kind = AlertKind.SENSOR)
            run {
                if (f.source == Source.STALE && p.source != Source.STALE) _events.tryEmit(HubEvent.BinLost(n))
                if (f.source != Source.STALE && p.source == Source.STALE) _events.tryEmit(HubEvent.BinBack(n))
            }
        }
        prev.keys.retainAll(bins.map { it.id }.toSet())
    }

    // Adds one alert row, keeps the newest 200, saves them and emits NewAlert.
    private fun add(b: Bin, sev: Severity, icon: String, title: String, meta: String, type: AlertType, kind: AlertKind =
Demo.kindFor(icon)) {
        val row = AlertRow(++alertSeq, sev, icon, kind, title, meta, System.currentTimeMillis(), demo = b.demo)
        val list = (listOf(row) + _state.value.alerts).take(200)
        _state.value = _state.value.copy(alerts = list)
        saveAlerts(list)
        _events.tryEmit(HubEvent.NewAlert(row, type, b.id))
    }

    // ---------- commands ----------

    fun bin(id: String) = _state.value.bins.firstOrNull { it.id == id }

    // Changes one bin in the state, optionally saving the list.
    private fun update(id: String, persist: Boolean = false, f: (Bin) -> Bin) {
        val s = _state.value
        val bins = s.bins.map { if (it.id == id) f(it) else it }
        _state.value = s.copy(bins = bins, link = linkOf(bins))
        if (persist) persist(bins)
    }

    /** Only real bins and their alerts are saved. Demo ones live while demo mode is on. */
    private fun persist(bins: List<Bin>) = store.saveBins(bins.filterNot { it.demo })
    private fun saveAlerts(list: List<AlertRow>) = store.saveAlerts(list.filterNot { it.demo })

    /**
     * Sends one command. Over Wi-Fi every command works; over the cloud only
     * LOCK, UNLOCK and COLLECT. Returns null when it went through, or what to
     * tell the user when it did not. The card changes at once either way, the
     * same as the dashboard, and the next reading shows what the bin really did.
     */
    suspend fun command(id: String, cmd: String, extra: Map<String, Any> = emptyMap()): String? {
        val b = bin(id) ?: return null
        _state.value.pending[id]?.let { return "Waiting for ${b.name} to confirm ${it.cmd}." }
        var problem: String? = null
        // Set when the command went through Firebase or ThingSpeak, so its key waits for the bin.
        var via: Source? = null
        // Demo bins act out the route the demo options pretend to use.
        if (b.demo) when (b.source) {
            Source.STALE -> return "${b.name} is not answering."
            Source.CLOUD, Source.THINGSPEAK -> {
                if (cmd !in Cloud.CLOUD_COMMANDS) return "That needs the bins' Wi-Fi."
                val now = System.currentTimeMillis()
                _state.value = _state.value.copy(pending = _state.value.pending + (id to Pending(cmd, b.source, now, now +
DEMO_TIMEOUT)))
                if (!_state.value.demoNoConfirm) demoAnswer[id] = (now + DEMO_CONFIRM) to cmd
                return null
            }
            else -> {}
        } else when (b.source) {
            Source.DEMO -> {}
            Source.LIVE -> try {
                LocalApi.command(b.ip, cmd, extra)
            } catch (e: CancellationException) {
                throw e
            } catch (e: LocalApi.Refused) {
                // The bin answered and said no: say why, and do not try the cloud.
                return "${b.name}: ${e.why}."
            } catch (_: Exception) {
                problem = if (cmd in Cloud.CLOUD_COMMANDS && (cloud.ready || b.tbId.isNotBlank())) {
                    val viaFb = cloud.ready && runCatching { cloud.command(b.cloudId, cmd).also { k -> k?.let { confirm(id,
b.name, cmd, it) } } }.isSuccess
                    val viaTs = !viaFb && b.tbId.isNotBlank() && runCatching { ts.command(b.tbId, b.tbKey, cmd) }.isSuccess
```

```
                    if (viaFb) via = Source.CLOUD
                    if (viaTs) via = Source.THINGSPEAK
                    if (viaFb || viaTs) null else "${b.name} did not answer on Wi-Fi or the cloud."
                } else "${b.name} did not answer. Check its power and Wi-Fi."
            }
            Source.CLOUD -> problem = if (cmd !in Cloud.CLOUD_COMMANDS) "That needs the bins' Wi-Fi."
            else runCatching { cloud.command(b.cloudId, cmd).also { k -> k?.let { confirm(id, b.name, cmd, it) } } }
                .exceptionOrNull()?.let { "The cloud did not take it. ${it.message.orEmpty()}" }
                .also { if (it == null) via = Source.CLOUD }
            Source.THINGSPEAK -> problem = when {
                cmd !in Cloud.CLOUD_COMMANDS -> "That needs the bins' Wi-Fi."
                b.tbId.isBlank() -> "Add ${b.name}'s TalkBack ID and key in SET to send commands through ThingSpeak."
                else -> runCatching { ts.command(b.tbId, b.tbKey, cmd) }.exceptionOrNull()?.let { "ThingSpeak did not take it. ${it.message.orEmpty()}" }
                    .also { if (it == null) via = Source.THINGSPEAK }
            }
            Source.STALE -> problem = "${b.name} is not answering."
        }
        if (problem != null && b.source != Source.LIVE) return problem
        // Over the cloud the card does not change until the bin confirms; the key stays locked meanwhile.
        via?.let { v ->
            if (problem != null) return problem
            val now = System.currentTimeMillis()
            val wait = if (v == Source.THINGSPEAK) 120_000L else 60_000L
            _state.value = _state.value.copy(pending = _state.value.pending + (id to Pending(cmd, v, now, now + wait)))
            return null
        }
        update(id) { x ->
            x.copy(
                reading = effect(x.reading, cmd),
                sim = if (cmd == "CLEAR_TAMPER") x.sim.copy(values = x.sim.values + ("tamper" to 0.0)) else x.sim,
            )
        }
        return problem
    }

    /**
     * The bin writes its answer to a Firebase command under the push key. It is
     * read every second for 8 s; with no answer the key keeps waiting for the
     * bin's status instead, as before.
     */
    private fun confirm(id: String, name: String, cmd: String, key: String) {
        scope.launch {
            repeat(8) {
                delay(1000)
                val r = runCatching { cloud.result(key) }.getOrNull()
                if (r != null && (r.optString("cmd").isBlank() || r.optString("cmd") == cmd)) {
                    cloud.dropResult(key)
                    if (_state.value.pending[id]?.cmd != cmd) return@launch
                    _state.value = _state.value.copy(pending = _state.value.pending - id)
                    val ok = r.optBoolean("ok", false)
                    _events.tryEmit(HubEvent.CommandResult(id, name, cmd, ok, r.optString("why").takeIf { !ok && it.isNotBlank() }))
                    if (ok) update(id) { it.copy(reading = effect(it.reading, cmd)) }
                    return@launch
                }
            }
        }
    }

    /** What a command does to a bin's reading, as the card shows it. */
    private fun effect(r: Reading, cmd: String): Reading = when (cmd) {
        "LOCK" -> r.copy(status = BinStatus.LOCKED, lidOpen = false)
        "UNLOCK" -> r.copy(status = BinStatus.OK)
        "CLEAR_TAMPER" -> r.copy(status = BinStatus.OK, tilt = 0.0)
        "COLLECT" -> r.copy(status = BinStatus.COLLECTING, lidOpen = true)
        "COLLECT_DONE" -> r.copy(status = BinStatus.OK, lidOpen = false, fill = 0.0, weight = 0.0, gas = minOf(r.gas, 48.0))
        "TARE" -> r.copy(weight = 0.0)
        "RESET_GAS" -> r.copy(iaqAcc = 0, iaqWarmS = 20)
        else -> r
    }

    /** Calibration mode, on one bin at a time. The bin shows its teal calibration screen while it is on. */
    suspend fun setCalibration(id: String?) {
        val old = _state.value.calibBin
        if (old == id) return
        _state.value = _state.value.copy(calibBin = id)
        old?.let { o -> bin(o)?.takeIf { it.source == Source.LIVE && !it.demo }?.let { runCatching { LocalApi.command(it.ip, "CALIBRATE_END") } } }
        old?.let { o -> update(o) { b -> if (b.reading.status == BinStatus.CALIBRATING) b.copy(reading = b.reading.copy(status = BinStatus.OK)) else b } }
        id?.let { n -> bin(n)?.takeIf { it.source == Source.LIVE && !it.demo }?.let { runCatching { LocalApi.command(it.ip, "CALIBRATE_START") } } }
        id?.let { n -> update(n) { b -> b.copy(reading = b.reading.copy(status = BinStatus.CALIBRATING)) } }
    }

```

```
    /** Stores new calibration values on the bin (EEPROM over Wi-Fi) and in the app. */
    suspend fun saveCal(id: String, cal: Cal): String? {
        val b = bin(id) ?: return null
        if (b.source == Source.LIVE && !b.demo) {
            try {
                LocalApi.calibrate(b.ip, cal)
            } catch (e: CancellationException) {
                throw e
            } catch (_: Exception) {
                return "${b.name} did not answer, so nothing was saved."
            }
        }
        update(id, persist = true) { it.copy(cal = cal) }
        bin(id)?.let {
            add(it, Severity.SUCCESS, "check", "Settings saved",
                "${it.name}: empty ${fmt(cal.v("d_empty"))} cm, full ${fmt(cal.v("d_full"))} cm, alert at
${cal.v("fill_th").toInt()}%",
                AlertType.INFO, kind = AlertKind.SETUP)
        }
        return null
    }

    /** Re-reads the values a bin is running on. */
    suspend fun reload(id: String): String? {
        val b = bin(id) ?: return null
        if (b.ip.isBlank()) return null
        return try {
            val s = LocalApi.status(b.ip)
            update(id, persist = true) { it.copy(cal = s.cal ?: it.cal, reading = s.reading, source = Source.LIVE, lastSeen =
System.currentTimeMillis()) }
            null
        } catch (e: CancellationException) {
            throw e
        } catch (_: Exception) {
            "${b.name} did not answer on Wi-Fi."
        }
    }

    /** Changes the simulation rows on screen. [apply] sends them to the bin. */
    fun editSim(id: String, sim: Sim) {
        simDirty.add(id)
        update(id) { it.copy(sim = sim) }
    }

    // Sends the simulation rows to the bin. Simulation only works over local
    // Wi-Fi because the bin needs to reply with the simulated values at once.
    suspend fun applySim(id: String): String? {
        val b = bin(id) ?: return null
        if (b.source == Source.LIVE && !b.demo) {
            try {
                LocalApi.simulate(b.ip, b.sim)
            } catch (e: CancellationException) {
                throw e
            } catch (_: Exception) {
                return "${b.name} did not answer. Simulation needs the bins' Wi-Fi."
            }
        }
        simDirty.remove(id)
        return null
    }

    // ---------- bins ----------

    // Adds n empty bins numbered after the existing real ones (bin03, bin04...).
    fun addBins(n: Int) {
        val s = _state.value
        val now = System.currentTimeMillis()
        val added = (0 until n).map { k ->
            val index = s.bins.count { !it.demo } + k
            Demo.newBin(index, "b$now-$k").copy(cloudId = "bin%02d".format(index + 1))
        }
        val bins = s.bins + added
        _state.value = s.copy(bins = bins, link = linkOf(bins))
        _history.value = _history.value + added.associate { it.id to emptyList() }
        persist(bins)
    }

    /**
     * A bin that just joined Wi-Fi over Bluetooth and is not in the list yet.
     * Named after its Bluetooth name (SmartBin-01 is Bin 01, cloud name bin01).
     */
    fun addRealBin(number: String, ip: String): Bin {
        val s = _state.value
        s.bins.firstOrNull { !it.demo && it.ip == ip }?.let { return it }
        val index = s.bins.count { !it.demo }
        val num = number.ifBlank { "%02d".format(index + 1) }
```

```
        val b = Demo.newBin(index, "b${System.currentTimeMillis()}").copy(name = "Bin $num", place = "", cloudId = "bin$num", ip = ip)
        val bins = listOf(b) + s.bins
        _state.value = s.copy(bins = bins, link = linkOf(bins))
        _history.value = _history.value + (b.id to emptyList())
        persist(bins)
        return b
    }

    /** Sends the saved Firebase account to every real bin on Wi-Fi. Returns how many took it and how many were tried. */
    suspend fun sendCloudToBins(): Pair<Int, Int> {
        val cfg = store.firebase()
        val bins = _state.value.bins.filter { !it.demo && it.ip.isNotBlank() }
        var ok = 0
        bins.forEach { b -> if (runCatching { LocalApi.sendCloud(b.ip, cfg) }.isSuccess) ok++ }
        return ok to bins.size
    }

    // Saves a bin's settings. A new IP or ThingSpeak channel marks it STALE
    // and resets its retry timers so the new address is tried at once.
    fun editBin(
        id: String, name: String, place: String, ip: String, cloudId: String,
        tsChannel: String? = null, tsKey: String? = null, tbId: String? = null, tbKey: String? = null,
    ) {
        val clean = ip.trim()
        update(id, persist = true) {
            val next = it.copy(
                name = name.trim().ifBlank { it.name }, place = place.trim(), ip = clean,
                cloudId = cloudId.trim().ifBlank { it.cloudId },
                tsChannel = tsChannel?.trim() ?: it.tsChannel, tsKey = tsKey?.trim() ?: it.tsKey,
                tbId = tbId?.trim() ?: it.tbId, tbKey = tbKey?.trim() ?: it.tbKey,
            )
            val moved = next.ip != it.ip || next.tsChannel != it.tsChannel
            next.copy(source = if (moved) (if (next.demo) Source.DEMO else Source.STALE) else it.source)
        }
        nextLocal.remove(id)
        nextCloud.remove(id)
        nextTs.remove(id)
        demoQuiet.remove(id)
    }

    fun removeBin(id: String) {
        val s = _state.value
        val bins = s.bins.filterNot { it.id == id }
        _state.value = s.copy(bins = bins, link = linkOf(bins), calibBin = s.calibBin.takeIf { it != id })
        persist(bins)
    }

    // ---------- alerts ----------

    // Alert list edits from the Alerts page.
    fun markRead(ids: Set<Long>, read: Boolean = true) = editAlerts { list -> list.map { if (it.id in ids) it.copy(read = read) else it } }

    fun deleteAlerts(ids: Set<Long>) = editAlerts { list -> list.filterNot { it.id in ids } }

    private fun editAlerts(f: (List<AlertRow>) -> List<AlertRow>) {
        val list = f(_state.value.alerts)
        _state.value = _state.value.copy(alerts = list)
        saveAlerts(list)
    }

    // ---------- demo options ----------

    /** Demo options change only bins with no address. Left alone, the demo runs as normal. */
    val demoInUse get() = Demo.speed != Demo.NORMAL_SPEED || _state.value.demoLinks.any || _state.value.demoNoConfirm || demoQuiet.isNotEmpty()

    // Demo options: how fast the demo bins fill.
    fun demoSpeed(x: Double) {
        Demo.speed = x
        _state.value = _state.value.copy()
    }

    /**
     * Pretend links. The first one on, in the app's own order (Wi-Fi, Firebase,
     * ThingSpeak), is the one the demo bins answer on, so the cards, banners,
     * link sounds and command waits behave as they would on that route.
     */
    fun demoLinks(r: Routes) {
        val s = _state.value
        val via = demoSource(r)
        val bins = s.bins.map { if (it.demo && it.source != Source.STALE) it.copy(source = via) else it }
        val link = linkOf(bins, r)
        if (link != s.link) _events.tryEmit(HubEvent.LinkChanged(s.link, link))
        _state.value = s.copy(bins = bins, demoLinks = r, link = link)
```

```
    }

    /** Pretend cloud commands time out instead of being confirmed. */
    fun setDemoNoConfirm(no: Boolean) {
        if (no) demoAnswer.clear()
        _state.value = _state.value.copy(demoNoConfirm = no)
    }

    private fun demoSource(r: Routes) = when {
        r.local -> Source.LIVE
        r.firebase -> Source.CLOUD
        r.thingspeak -> Source.THINGSPEAK
        else -> Source.DEMO
    }

    // Demo options: make a demo bin stop answering, to show the "lost" state.
    fun demoQuiet(id: String) = id in demoQuiet

    fun setDemoQuiet(id: String, quiet: Boolean) {
        if (quiet) demoQuiet.add(id) else demoQuiet.remove(id)
        if (quiet) update(id) { it.copy(lastSeen = System.currentTimeMillis()) }
        if (!quiet) update(id) { it.copy(source = demoSource(_state.value.demoLinks)) }
    }

    // Demo options: jump a demo bin straight into one of these situations.
    enum class DemoEvent { FULL, TAMPER, AIR, EMPTY }

    fun demoEvent(id: String, e: DemoEvent) {
        update(id) { b ->
            val r = b.reading
            val c = b.cal
            b.copy(
                reading = when (e) {
                    DemoEvent.FULL -> r.copy(fill = minOf(100.0, c.v("fill_th") + 3), distance =
Demo.distanceFor(c.v("fill_th") + 3, c))
                    DemoEvent.TAMPER -> r.copy(status = BinStatus.TAMPER, tilt = 38.0, lidOpen = false)
                    DemoEvent.AIR -> r.copy(gas = c.v("gas_th") + 60)
                    DemoEvent.EMPTY -> r.copy(fill = 0.0, weight = 0.0, gas = minOf(r.gas, 48.0), status = BinStatus.OK,
distance = c.v("d_empty"))
                },
            )
        }
    }

    /** Puts every demo option back, so the demo runs as it does on a fresh start. */
    fun resetDemo() {
        Demo.speed = Demo.NORMAL_SPEED
        demoQuiet.clear()
        demoAnswer.clear()
        val now = System.currentTimeMillis()
        val start = Demo.bins().associateBy { it.id }
        val s = _state.value
        val bins = s.bins.map { b -> if (!b.demo) b else start[b.id]?.let { o -> b.copy(reading = o.reading, rate = o.rate,
source = Source.DEMO) } ?: b.copy(source = Source.DEMO) }
        val link = linkOf(bins, Routes())
        if (link != s.link) _events.tryEmit(HubEvent.LinkChanged(s.link, link))
        _state.value = s.copy(bins = bins, link = link, demoLinks = Routes(), demoNoConfirm = false, pending =
s.pending.filterKeys { k -> bins.none { it.id == k && it.demo } })
        _history.value = _history.value + bins.filter { it.demo }.associate { it.id to Demo.history(it, now) }
    }

    // ---------- demo mode and the demo tour ----------

    /**
     * Demo mode, from the start page only. On adds the three demo bins with a
     * day of made-up history and alerts; off takes them away with everything
     * the demo options changed.
     */
    fun setDemo(on: Boolean) {
        val s = _state.value
        if (on == s.demoOn) return
        val now = System.currentTimeMillis()
        holds.clear()
        tourStep = 0
        tourNext = now + 6_000
        tourPauseUntil = 0L
        if (on) {
            val demo = Demo.bins()
            val alerts = (Demo.alerts(demo, now).map { it.copy(id = ++alertSeq) } + s.alerts).sortedByDescending { it.at }
            val bins = s.bins.filterNot { it.demo } + demo
            _state.value = s.copy(bins = bins, alerts = alerts, demoOn = true, link = linkOf(bins), tourMsg = null)
            _history.value = _history.value + demo.associate { it.id to Demo.history(it, now) }
        } else {
            Demo.speed = Demo.NORMAL_SPEED
            demoQuiet.clear()
            demoAnswer.clear()
```

```
            val gone = s.bins.filter { it.demo }.map { it.id }.toSet()
            val bins = s.bins.filterNot { it.demo }
            _state.value = s.copy(
                bins = bins, alerts = s.alerts.filterNot { it.demo }, demoOn = false, link = linkOf(bins, Routes()),
                demoLinks = Routes(), demoNoConfirm = false, tourMsg = null,
                pending = s.pending - gone, calibBin = s.calibBin.takeIf { it !in gone },
            )
            _history.value = _history.value - gone
            gone.forEach { prev.remove(it) }
        }
    }

    /** A key pressed on a demo bin, or a demo option changed: the tour waits a minute. */
    fun pauseTour() {
        tourPauseUntil = System.currentTimeMillis() + 60_000
        if (_state.value.tourMsg != null) _state.value = _state.value.copy(tourMsg = null)
    }

    private fun hold(id: String, f: ((Reading) -> Reading)?) {
        if (f == null) holds.remove(id) else holds[id] = f
    }

    /** A command acted out by the tour: the card changes and its sound plays, as if a key was pressed. */
    private fun tourCmd(b: Bin, cmd: String) {
        update(b.id) { it.copy(reading = effect(it.reading, cmd)) }
        TOUR_SFX[cmd]?.let { _events.tryEmit(HubEvent.Sound(it)) }
    }

    /**
     * With demo mode on and nobody pressing anything on a demo bin, the demo
     * bins act out every outcome in turn, about every 10 s: full, pickup,
     * tamper, lock, bad air, heat, humidity, a sensor fault, warm-up, the
     * links changing, a bin going quiet, and calibration.
     */
    private fun tour(now: Long) {
        val s = _state.value
        val demo = s.bins.filter { it.demo }
        if (!s.demoOn || demo.isEmpty() || !inMain || !foreground) {
            tourNext = maxOf(tourNext, now + 6_000)
            return
        }
        if (now < tourPauseUntil || now < tourNext) return
        tourNext = now + 10_000
        val a = demo[0]
        val b = demo[1 % demo.size]
        val c = demo[2 % demo.size]
        val step = tourStep
        tourStep = (tourStep + 1) % TOUR_STEPS
        val msg: String = when (step) {
            0 -> { demoEvent(a.id, DemoEvent.FULL); "${a.name} fills past its ${a.cal.v("fill_th").toInt()}% limit." }
            1 -> { if (bin(a.id)?.reading?.status == BinStatus.LOCKED) tourCmd(a, "UNLOCK"); tourCmd(a, "COLLECT"); "A collector starts the pickup at ${a.name}. The lid opens." }
            2 -> { tourCmd(a, "COLLECT_DONE"); "${a.name} is emptied and back to normal." }
            3 -> { hold(b.id) { r -> r.copy(tilt = 40.0, status = BinStatus.TAMPER, lidOpen = false, tamperPhase = "LATCHED") }; "${b.name} is tipped over. Tamper alert." }
            4 -> { hold(b.id, null); tourCmd(b, "CLEAR_TAMPER"); "The tamper alert on ${b.name} is reset." }
            5 -> { tourCmd(c, "LOCK"); "${c.name} is locked from the app." }
            6 -> { tourCmd(c, "UNLOCK"); "${c.name} is unlocked again." }
            7 -> { val g = c.cal.v("gas_th") + 45; hold(c.id) { r -> r.copy(gas = g) }; "The air in ${c.name} goes bad." }
            8 -> {
                hold(c.id, null); update(c.id) { it.copy(reading = it.reading.copy(gas = 60.0)) }
                val t = b.cal.v("temp_th") + 3.5
                hold(b.id) { r -> r.copy(temp = t, tempRaw = t - b.cal.v("temp_off")) }
                "${b.name} gets too hot."
            }
            9 -> {
                val h = b.cal.v("hum_th") + 5
                hold(b.id) { r -> r.copy(hum = h) }
                update(b.id) { it.copy(reading = it.reading.copy(temp = 29.0, tempRaw = 29.0 - it.cal.v("temp_off"))) }
                "${b.name} gets too humid."
            }
            10 -> {
                hold(b.id, null); update(b.id) { it.copy(reading = it.reading.copy(hum = 62.0)) }
                hold(a.id) { r -> r.copy(faults = listOf("HX711"), weight = 0.0) }
                "The weight sensor of ${a.name} stops answering."
            }
            11 -> { hold(a.id, null); "The weight sensor of ${a.name} answers again." }
            12 -> { tourCmd(c, "RESET_GAS"); "The air sensor of ${c.name} warms up. Its score is held until it settles." }
            13 -> { demoLinks(Routes(firebase = true)); "${b.name} and the others lose Wi-Fi, so they are read through Firebase." }
            14 -> { demoLinks(Routes(thingspeak = true)); "Firebase is down too, so the bins are read through ThingSpeak." }
            15 -> { setDemoQuiet(b.id, true); "${b.name} stops answering on every link." }
            16 -> { setDemoQuiet(b.id, false); demoLinks(Routes(local = true)); "${b.name} is back, and the bins answer on Wi-Fi again." }
            17 -> { demoLinks(Routes()); scope.launch { setCalibration(b.id) }; TOUR_SFX["CALIBRATE_START"]?.let { _events.tryEmit(HubEvent.Sound(it)) }; "${b.name} is being calibrated." }
```

```
            else -> { scope.launch { setCalibration(null) }; "Calibration of ${b.name} is done. The tour starts again." }
        }
        _state.value = _state.value.copy(tourMsg = msg)
    }

    // ---------- Firebase watchers and alerts ----------

    /**
     * While the app is open and reads a bin through Firebase, it tells the bins
     * every 10 s, so they write every 5 s instead of every minute. Going to
     * the background takes the note away.
     */
    private suspend fun watchers(bins: List<Bin>, now: Long) {
        if (!cloud.ready) return
        val want = foreground && bins.any { !it.demo && it.source == Source.CLOUD }
        if (want && now - watchedAt >= 10_000) {
            watchedAt = now
            if (runCatching { cloud.watch(watcherId, android.os.Build.MODEL ?: "phone") }.isSuccess) watching = true
        } else if (!want && watching) {
            watching = false
            watchedAt = 0L
            cloud.unwatch(watcherId)
        }
    }

    /**
     * The alerts each bin wrote to Firebase while the app was closed, read once
     * per bin after the app starts. Alerts while the app is open come from the
     * readings themselves, so they are not read twice.
     */
    private suspend fun cloudAlerts(bins: List<Bin>, now: Long) {
        if (!cloud.ready || !foreground || now - alertsCheck < 30_000) return
        alertsCheck = now
        bins.filter { !it.demo && it.cloudId.isNotBlank() && it.cloudId !in alertsRead }.forEach { b ->
            val list = runCatching { cloud.alerts(b.cloudId) }.getOrNull() ?: return@forEach
            alertsRead.add(b.cloudId)
            val seenKey = "fb_alerts_" + b.cloudId
            val seen = store.string(seenKey, "0").toLongOrNull() ?: 0L
            val fresh = list.filter { it.optLong("ts") in (seen + 1) until startedAt }
            if (fresh.isNotEmpty()) {
                val rows = fresh.map { o ->
                    val kind = when (o.optString("kind")) {
                        "full" -> AlertKind.FULL; "sensor" -> AlertKind.SENSOR; "tamper" -> AlertKind.TAMPER
                        "lock" -> AlertKind.LOCK; "pickup" -> AlertKind.PICKUP; else -> AlertKind.SETUP
                    }
                    val sev = when (o.optString("sev")) {
                        "critical" -> Severity.CRITICAL; "warning" -> Severity.WARNING; "success" -> Severity.SUCCESS; else -
> Severity.INFO
                    }
                    val icon = when (kind) {
                        AlertKind.FULL -> "fill"; AlertKind.SENSOR -> "gas"; AlertKind.TAMPER -> "tamper"
                        AlertKind.LOCK -> "lock"; AlertKind.PICKUP -> "truck"; else -> "calib"
                    }
                    AlertRow(++alertSeq, sev, icon, kind, o.optString("title", "Bin alert"), o.optString("meta", b.name),
o.optLong("ts"))
                }
                val all = (rows + _state.value.alerts).sortedByDescending { it.at }.take(200)
                _state.value = _state.value.copy(alerts = all)
                saveAlerts(all)
            }
            list.maxOfOrNull { it.optLong("ts") }?.let { store.putString(seenKey, it.toString()) }
        }
    }

    // ---------- ThingSpeak ----------

    // ---------- ThingSpeak and Firebase settings ----------
    fun thingSpeakUrl() = ts.url

    fun saveThingSpeak(url: String) {
        ts.use(url)
        store.putString(KEY_TS_URL, ts.url)
        nextTs.clear()
        lastCheck = 0L
    }

    /** Reads each bin's channel once. Returns how many answered and the first problem. */
    suspend fun testThingSpeak(): Pair<Int, String?> {
        val bins = _state.value.bins.filter { it.onThingSpeak }
        if (bins.isEmpty()) return 0 to "No bin has a ThingSpeak channel yet. Add one in its SET sheet."
        var ok = 0
        var problem: String? = null
        bins.forEach { b ->
            try {
                ts.last(b.tsChannel, b.tsKey)
                ok++
```

```
            } catch (e: CancellationException) {
                throw e
            } catch (e: Exception) {
                if (problem == null) problem = "${b.name}: ${e.message ?: "no answer"}"
            }
        }
        if (ok > 0) tsOk = System.currentTimeMillis()
        return ok to problem
    }

    // ---------- Firebase ----------

    fun firebase() = store.firebase()
    fun firebaseWorking() = store.firebaseWorking()

    fun saveFirebase(cfg: FirebaseConfig) {
        store.saveFirebase(cfg, passedTest = true)
        cloud.use(cfg)
        nextCloud.clear()
    }

    /** Estimated fill speed in percent per hour, from the last hour of history; demo bins use their own rate. */
    fun ratePerHour(b: Bin): Double {
        if (b.demo) return b.rate
        val h = _history.value[b.id] ?: return 0.0
        val now = System.currentTimeMillis()
        val recent = h.filter { it.t >= now - 3600_000L }
        if (recent.size < 6) return 0.0
        val hours = (recent.last().t - recent.first().t) / 3600_000.0
        if (hours <= 0.05) return 0.0
        return ((recent.last().fill - recent.first().fill) / hours).coerceAtLeast(0.0)
    }

    private const val KEY_TS_URL = "ts_url"
    private const val KEY_V2 = "v2_bins"
    private const val KEY_WATCHER = "watcher_id"
    private const val TOUR_STEPS = 19
    /** The sound each tour command plays, the same as its key in the app. */
    private val TOUR_SFX = mapOf(
        "LOCK" to "LOCK", "UNLOCK" to "UNLOCK", "COLLECT" to "PICKUP", "COLLECT_DONE" to "DONE",
        "CLEAR_TAMPER" to "CONFIRM", "RESET_GAS" to "CONFIRM", "CALIBRATE_START" to "CALIB",
    )
    /** Pretend cloud commands: confirmed after 4 s, or given up after 8 s, as in Garbage-Guard's demo. */
    private const val DEMO_CONFIRM = 4_000L
    private const val DEMO_TIMEOUT = 8_000L

    private fun fmt(v: Double) = if (v == Math.floor(v)) v.toInt().toString() else "%.1f".format(v)

    fun store() = store
}
```

Android App/app/src/main/java/smartbin/notify/Sounds.kt

```
// ============================================================================
// notify/Sounds.kt - in-app sound effects
// Every Sfx entry is a WAV in res/raw. SoundPool loads them all at start-up
// and can mix up to 6 at once. A sound asked for before it has loaded is
// dropped (except the boot chime), so a missing first sound is not a bug.
// ============================================================================
package smartbin.notify

import android.content.Context
import android.media.AudioAttributes
import android.media.SoundPool
import android.os.SystemClock
import smartbin.R

/**
 * The app's own sound for each kind of action, built by tools/make_sounds.py.
 * Rising pitch means forward, on, open or OK; falling means back, off, close
 * or no. Alerts that must sound while the app is closed go through the
 * notification channels in [Alerts], because only Android can play those.
 */
enum class Sfx(val res: Int, val volume: Float = 0.6f) {
    TAP(R.raw.sb_tap),
    CHOICE(R.raw.sb_choice),
    SWITCH_ON(R.raw.sb_switch_on),
    SWITCH_OFF(R.raw.sb_switch_off),
    STEP_UP(R.raw.sb_step_up),
    STEP_DOWN(R.raw.sb_step_down),
    TAB(R.raw.sb_tab),
    OPEN(R.raw.sb_open),
    BACK(R.raw.sb_back),
    CONFIRM(R.raw.sb_confirm),
    DANGER(R.raw.sb_danger),
```

```
    DENIED(R.raw.sb_denied),
    LOGO(R.raw.sb_logo),
    THEME_LIGHT(R.raw.sb_theme_light),
    THEME_DARK(R.raw.sb_theme_dark),

    BOOT(R.raw.sb_boot, 0.8f),
    SAVED(R.raw.sb_saved, 0.7f),
    LINK_UP(R.raw.sb_link_up, 0.7f),
    LINK_LOST(R.raw.sb_link_lost, 0.7f),
    CLOUD(R.raw.sb_cloud, 0.7f),
    LOCK(R.raw.sb_lock, 0.7f),
    UNLOCK(R.raw.sb_unlock, 0.7f),
    PICKUP(R.raw.sb_pickup, 0.7f),
    DONE(R.raw.sb_done, 0.75f),
    CALIB(R.raw.sb_calib, 0.7f),
    SIM(R.raw.sb_sim, 0.7f),
    BIN_LOST(R.raw.sb_bin_lost, 0.7f),
    BIN_BACK(R.raw.sb_bin_back, 0.7f),
    PIN_KEY(R.raw.sb_pin_key, 0.5f),
    PIN_WRONG(R.raw.sb_pin_wrong, 0.7f),
    PIN_LOCKOUT(R.raw.sb_pin_lockout, 0.75f),
    OP_IN(R.raw.sb_op_in, 0.7f),
    OP_OUT(R.raw.sb_op_out, 0.7f),
    CMD_OK(R.raw.sb_cmd_ok, 0.7f),
    CMD_FAIL(R.raw.sb_cmd_fail, 0.7f),
    BT_PAIRED(R.raw.sb_bt_paired, 0.7f),
    BT_WRONG(R.raw.sb_bt_wrong, 0.7f),
    BT_BLOCKED(R.raw.sb_bt_blocked, 0.75f),
    WIFI_JOINED(R.raw.sb_wifi_joined, 0.7f),
    PIN_SET(R.raw.sb_pin_set, 0.7f),
    WIFI_FAILED(R.raw.sb_wifi_failed, 0.7f),
    BT_OFF(R.raw.sb_bt_off, 0.7f),
    DEMO(R.raw.sb_demo, 0.7f),

    ALERT_FULL(R.raw.sb_alert_full, 0.8f),
    ALERT_TAMPER(R.raw.sb_alert_tamper, 0.8f),
    ALERT_SENSOR(R.raw.sb_alert_sensor, 0.8f),
    ALERT_LOCK(R.raw.sb_alert_lock, 0.8f),
    ALERT_PICKUP(R.raw.sb_alert_pickup, 0.8f),
    ALERT_INFO(R.raw.sb_alert_info, 0.8f),
}

object Sounds {
    private var pool: SoundPool? = null
    private val loaded = HashMap<Sfx, Int>()
    private val ready = HashSet<Int>()
    private var bootWanted = false
    @Volatile private var last = 0L

    fun init(context: Context) {
        if (pool != null) return
        val app = context.applicationContext
        val made = SoundPool.Builder()
            .setMaxStreams(6)
            .setAudioAttributes(
                AudioAttributes.Builder()
                    .setUsage(AudioAttributes.USAGE_ASSISTANCE_SONIFICATION)
                    .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
                    .build()
            )
            .build()
        pool = made
        made.setOnLoadCompleteListener { p, sample, status ->
            if (status != 0) return@setOnLoadCompleteListener
            ready.add(sample)
            // The opening chime is asked for before the sounds have loaded; it plays as soon as it can.
            if (bootWanted && sample == loaded[Sfx.BOOT]) {
                bootWanted = false
                if (Feel.sounds) p.play(sample, Sfx.BOOT.volume, Sfx.BOOT.volume, 1, 0, 1f)
            }
        }
        Sfx.entries.forEach { loaded[it] = made.load(app, it.res, 1) }
    }

    fun play(sfx: Sfx, force: Boolean = false) {
        if (!Feel.sounds && !force) return
        val id = loaded[sfx] ?: return
        if (id !in ready) return
        last = SystemClock.uptimeMillis()
        pool?.play(id, sfx.volume, sfx.volume, 1, 0, 1f)
    }

    /** For sounds that follow from another action: skipped when a key has just made its own sound. */
    fun playIfQuiet(sfx: Sfx) {
        if (SystemClock.uptimeMillis() - last > 250) play(sfx)
    }
```

```
119
120     /** The opening chime. Waits for the sound to load if it has not yet. */
121     fun boot() {
122         val id = loaded[Sfx.BOOT]
123         if (id != null && id in ready) play(Sfx.BOOT) else bootWanted = true
124     }
125 }
```

Android App/app/src/main/java/smartbin/notify/Haptics.kt

```
1  // ================================================================================
2  // notify/Haptics.kt - custom vibration patterns
3  // Strength scales every amplitude: LIGHT 0.45, NORMAL 0.75, STRONG 1.0.
4  // Vibration off, or a phone with no motor, skips the pattern silently.
5  // ================================================================================
6  package smartbin.notify
7
8  import android.content.Context
9  import android.os.Build
10 import android.os.VibrationEffect
11 import android.os.Vibrator
12 import android.os.VibratorManager
13 import kotlin.math.roundToInt
14
15 /**
16  * The app's own vibration patterns, for moments that deserve more than a
17  * plain click: a full bin, tamper, a lock, a pickup, the link dropping. Each
18  * has its own rhythm, so the phone can be told apart in a pocket. Plain keys
19  * keep Android's touch clicks in [Feel]; these patterns follow the Vibration
20  * switch and the strength chosen in Settings.
21  */
22 object Haptics {
23     enum class Strength(val label: String, val scale: Float) {
24         LIGHT("Light", 0.45f), NORMAL("Normal", 0.75f), STRONG("Strong", 1f),
25     }
26
27     @Volatile var strength = Strength.NORMAL
28
29     /** One pattern, as (milliseconds, amplitude) steps. Amplitude runs 1 to 255; 0 is a pause. */
30     enum class Buzz(vararg val steps: Pair<Int, Int>) {
31         // Alerts: long and hard. Routine events: short and soft.
32         FULL(300 to 255, 120 to 0, 300 to 255, 120 to 0, 300 to 255),
33         TAMPER(90 to 255, 60 to 0, 90 to 255, 60 to 0, 90 to 255, 60 to 0, 400 to 255),
34         SENSOR(150 to 200, 100 to 0, 300 to 200),
35         LOCK(60 to 200, 50 to 0, 160 to 150),
36         UNLOCK(160 to 150, 50 to 0, 60 to 200),
37         PICKUP(40 to 120, 60 to 0, 40 to 160, 60 to 0, 120 to 220),
38         DONE(30 to 120, 40 to 0, 30 to 160, 40 to 0, 30 to 200, 40 to 0, 140 to 255),
39         INFO(40 to 120, 80 to 0, 60 to 170),
40
41         // In the app only.
42         DANGER(60 to 255, 70 to 0, 90 to 255),
43         LOGO(40 to 80, 40 to 0, 40 to 150, 40 to 0, 70 to 255),
44         BOOT(30 to 40, 30 to 80, 30 to 120, 30 to 160, 40 to 200),
45         SAVED(20 to 170, 60 to 0, 20 to 170),
46         DENIED(50 to 200, 50 to 0, 50 to 200),
47         CALIB(20 to 140, 80 to 0, 20 to 140, 80 to 0, 20 to 140, 80 to 0, 80 to 220),
48         SIM(60 to 90, 40 to 130, 40 to 170, 60 to 210),
49         LINK_LOST(120 to 220, 100 to 150, 120 to 80, 140 to 40),
50         LINK_UP(40 to 120, 60 to 0, 70 to 230),
51         CLOUD(50 to 140, 60 to 0, 50 to 140, 60 to 0, 120 to 90),
52         BIN_LOST(80 to 150, 80 to 60),
53         BIN_BACK(40 to 70, 50 to 0, 50 to 160),
54         PIN_WRONG(70 to 230, 50 to 0, 70 to 230, 50 to 0, 140 to 160),
55         PIN_LOCKOUT(400 to 255, 120 to 0, 250 to 120),
56         OP_IN(30 to 100, 40 to 0, 30 to 160, 40 to 0, 110 to 240),
57         OP_OUT(110 to 200, 40 to 0, 30 to 90),
58         CMD_OK(25 to 140, 40 to 0, 70 to 210),
59         CMD_FAIL(90 to 200, 50 to 0, 90 to 160, 50 to 0, 160 to 90),
60         BT_PAIRED(30 to 110, 40 to 0, 30 to 110, 40 to 0, 100 to 230),
61         BT_WRONG(40 to 220, 40 to 0, 40 to 220, 40 to 0, 40 to 220),
62         BT_BLOCKED(300 to 230, 150 to 0, 300 to 120),
63         WIFI_JOINED(40 to 120, 30 to 0, 40 to 170, 30 to 0, 120 to 240),
64         PIN_SET(20 to 150, 50 to 0, 20 to 150, 50 to 0, 60 to 200),
65         WIFI_FAILED(120 to 220, 60 to 0, 120 to 160, 60 to 0, 200 to 90),
66         BT_OFF(40 to 160, 40 to 110, 40 to 70, 120 to 40),
67         DEMO(30 to 100, 40 to 0, 30 to 140, 40 to 0, 30 to 180, 40 to 0, 90 to 230),
68         ;
69
70         /** The same rhythm as a notification channel wants it: a pause first, then buzz and pause in turn. */
71         fun channelPattern(): LongArray {
72             val out = mutableListOf(0L)
73             var on = false
74             steps.forEach { (ms, amp) ->
75                 if ((amp > 0) == on) out[out.lastIndex] = out.last() + ms
```

```
                else { out.add(ms.toLong()); on = amp > 0 }
            }
            return out.toLongArray()
        }
    }

    private var vibrator: Vibrator? = null

    fun init(context: Context) {
        if (vibrator != null) return
        val app = context.applicationContext
        vibrator = runCatching {
            if (Build.VERSION.SDK_INT >= 31) app.getSystemService(VibratorManager::class.java)?.defaultVibrator
            else @Suppress("DEPRECATION") (app.getSystemService(Context.VIBRATOR_SERVICE) as? Vibrator)
        }.getOrNull()
    }

    /** Plays one pattern, unless vibration is switched off in Settings. */
    fun play(buzz: Buzz) {
        if (!Feel.haptics) return
        val v = vibrator ?: return
        if (!v.hasVibrator()) return
        val scale = strength.scale
        val timings = LongArray(buzz.steps.size) { buzz.steps[it].first.toLong() }
        val amps = IntArray(buzz.steps.size) {
            val a = buzz.steps[it].second
            if (a == 0) 0 else (a * scale).roundToInt().coerceIn(1, 255)
        }
        try {
            v.vibrate(VibrationEffect.createWaveform(timings, amps, -1))
        } catch (_: Exception) {
            // No motor: the sound still plays.
        }
    }

    /** The pattern that goes with a sound, for the sounds that have one. */
    fun forSfx(sfx: Sfx): Buzz? = when (sfx) {
        Sfx.DANGER -> Buzz.DANGER
        Sfx.LOGO -> Buzz.LOGO
        Sfx.DENIED -> Buzz.DENIED
        Sfx.SAVED -> Buzz.SAVED
        Sfx.BOOT -> Buzz.BOOT
        Sfx.LINK_LOST -> Buzz.LINK_LOST
        Sfx.LINK_UP -> Buzz.LINK_UP
        Sfx.CLOUD -> Buzz.CLOUD
        Sfx.LOCK -> Buzz.LOCK
        Sfx.UNLOCK -> Buzz.UNLOCK
        Sfx.PICKUP -> Buzz.PICKUP
        Sfx.DONE -> Buzz.DONE
        Sfx.CALIB -> Buzz.CALIB
        Sfx.SIM -> Buzz.SIM
        Sfx.BIN_LOST -> Buzz.BIN_LOST
        Sfx.BIN_BACK -> Buzz.BIN_BACK
        Sfx.PIN_WRONG -> Buzz.PIN_WRONG
        Sfx.PIN_LOCKOUT -> Buzz.PIN_LOCKOUT
        Sfx.OP_IN -> Buzz.OP_IN
        Sfx.OP_OUT -> Buzz.OP_OUT
        Sfx.CMD_OK -> Buzz.CMD_OK
        Sfx.CMD_FAIL -> Buzz.CMD_FAIL
        Sfx.BT_PAIRED -> Buzz.BT_PAIRED
        Sfx.BT_WRONG -> Buzz.BT_WRONG
        Sfx.BT_BLOCKED -> Buzz.BT_BLOCKED
        Sfx.WIFI_JOINED -> Buzz.WIFI_JOINED
        Sfx.PIN_SET -> Buzz.PIN_SET
        Sfx.WIFI_FAILED -> Buzz.WIFI_FAILED
        Sfx.BT_OFF -> Buzz.BT_OFF
        Sfx.DEMO -> Buzz.DEMO
        Sfx.ALERT_FULL -> Buzz.FULL
        Sfx.ALERT_TAMPER -> Buzz.TAMPER
        Sfx.ALERT_SENSOR -> Buzz.SENSOR
        Sfx.ALERT_LOCK -> Buzz.LOCK
        Sfx.ALERT_PICKUP -> Buzz.DONE
        Sfx.ALERT_INFO -> Buzz.INFO
        else -> null
    }
}
```

Android App/app/src/main/java/smartbin/notify/Feel.kt

```
// ============================================================================
// notify/Feel.kt - touch feedback (key sounds and haptic clicks)
// sounds and haptics are the Sound and Vibration switches on the App settings
// page. Sound off also mutes in-app event sounds, but not notification sounds,
// which follow the phone's own notification volume.
// ============================================================================
```

```
package smartbin.notify

import android.os.Build
import android.view.HapticFeedbackConstants
import android.view.View

/**
 * How the app feels under the finger: a small vibration and a soft sound on
 * keys and switches. Both can be turned off in Settings. Alert sounds are
 * separate; those belong to Android's notification settings.
 */
object Feel {
    @Volatile var sounds = true
    @Volatile var haptics = true

    /** Any control: its own sound and a vibration that fits it. */
    fun play(view: View, sfx: Sfx) {
        // Events have patterns of their own; plain keys use Android's touch clicks.
        val buzz = Haptics.forSfx(sfx)
        if (buzz != null) Haptics.play(buzz)
        else if (haptics) view.performHapticFeedback(hapticFor(sfx))
        Sounds.play(sfx)
    }

    /** A sound and its vibration with no key involved, such as a bin going full. */
    fun event(sfx: Sfx) {
        Haptics.forSfx(sfx)?.let(Haptics::play)
        Sounds.play(sfx)
    }

    private fun hapticFor(sfx: Sfx): Int {
        val v = Build.VERSION.SDK_INT
        val key = HapticFeedbackConstants.VIRTUAL_KEY
        return when (sfx) {
            Sfx.CONFIRM -> if (v >= 30) HapticFeedbackConstants.CONFIRM else key
            Sfx.SWITCH_ON -> if (v >= 34) HapticFeedbackConstants.TOGGLE_ON else HapticFeedbackConstants.KEYBOARD_TAP
            Sfx.SWITCH_OFF -> if (v >= 34) HapticFeedbackConstants.TOGGLE_OFF else HapticFeedbackConstants.KEYBOARD_TAP
            Sfx.STEP_UP, Sfx.STEP_DOWN, Sfx.CHOICE -> if (v >= 30) HapticFeedbackConstants.SEGMENT_TICK else
HapticFeedbackConstants.KEYBOARD_TAP
            Sfx.BACK -> HapticFeedbackConstants.CLOCK_TICK
            Sfx.PIN_KEY, Sfx.TAB, Sfx.THEME_LIGHT, Sfx.THEME_DARK, Sfx.OPEN -> HapticFeedbackConstants.KEYBOARD_TAP
            else -> key
        }
    }

    /** Something weighty, such as a long hold being recognised. */
    fun thump(view: View) {
        if (haptics) view.performHapticFeedback(HapticFeedbackConstants.LONG_PRESS)
    }
}
```

Android App/app/src/main/java/smartbin/notify/Alerts.kt

```
// ============================================================================
// notify/Alerts.kt - Android notification channels and posting
// Channel ids end in _v1 because Android never lets an app change a channel's
// sound after it is created. To ship a new alert sound, bump the id (_v2);
// otherwise phones that already have the channel keep the old sound.
// Debug tip: if an alert shows in the app but no notification appears, check
// Android Settings > Apps > SmartBin > Notifications for that channel first.
// ============================================================================
package smartbin.notify

import android.annotation.SuppressLint
import android.app.NotificationChannel
import android.app.NotificationManager
import android.app.PendingIntent
import android.content.Context
import android.content.Intent
import android.media.AudioAttributes
import android.net.Uri
import android.os.Build
import androidx.core.app.NotificationCompat
import androidx.core.app.NotificationManagerCompat
import smartbin.MainActivity
import smartbin.R
import smartbin.data.AlertRow
import smartbin.data.AlertType

/**
 * Android notifications for alerts, each kind with its own sound and its own
 * vibration. On Android a notification's sound belongs to its channel, so
 * every kind has a channel. That is also what lets the right sound play while
 * the app is closed, and lets each kind be changed or silenced in Android's
 * settings.
```

```
 */
object Alerts {
    const val CHANNEL_WATCH = "sb_watch"
    const val ACTION_COLLECT = "smartbin.START_PICKUP"
    const val EXTRA_BIN = "bin"
    const val EXTRA_NOTIFICATION = "notification"

    private class Kind(val channel: String, val name: String, val about: String, val sfx: Sfx, val importance: Int, val buzz: Haptics.Buzz)

    // One channel per alert type: id, name shown in Android settings, its
    // description, the in-app sound, importance (HIGH makes a heads-up
    // banner) and the vibration rhythm.
    private val kinds = mapOf(
        AlertType.FULL to Kind("sb_full_v1", "Bin full", "A bin passed its fill limit. Has a Start pickup button.", Sfx.ALERT_FULL, NotificationManager.IMPORTANCE_HIGH, Haptics.Buzz.FULL),
        AlertType.TAMPER to Kind("sb_tamper_v1", "Tamper", "A bin was moved or tilted.", Sfx.ALERT_TAMPER, NotificationManager.IMPORTANCE_HIGH, Haptics.Buzz.TAMPER),
        AlertType.SENSOR to Kind("sb_sensor_v1", "Sensor over its limit", "Temperature, humidity or air quality passed its limit.", Sfx.ALERT_SENSOR, NotificationManager.IMPORTANCE_HIGH, Haptics.Buzz.SENSOR),
        AlertType.LOCK to Kind("sb_lock_v1", "Lock and unlock", "A bin was locked or unlocked.", Sfx.ALERT_LOCK, NotificationManager.IMPORTANCE_DEFAULT, Haptics.Buzz.LOCK),
        AlertType.PICKUP to Kind("sb_pickup_v1", "Pickups", "A bin was emptied.", Sfx.ALERT_PICKUP, NotificationManager.IMPORTANCE_DEFAULT, Haptics.Buzz.DONE),
        AlertType.INFO to Kind("sb_info_v1", "Calibration and settings", "A bin was calibrated or its settings were saved.", Sfx.ALERT_INFO, NotificationManager.IMPORTANCE_LOW, Haptics.Buzz.INFO),
    )

    /** The sound the app plays for a kind of alert while it is on screen. */
    fun sfxFor(type: AlertType): Sfx = kinds.getValue(type).sfx

    fun ensureChannels(context: Context) {
        if (Build.VERSION.SDK_INT < 26) return
        val manager = context.getSystemService(NotificationManager::class.java) ?: return
        val audio = AudioAttributes.Builder()
            .setUsage(AudioAttributes.USAGE_NOTIFICATION)
            .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
            .build()
        kinds.values.forEach { k ->
            manager.createNotificationChannel(
                NotificationChannel(k.channel, k.name, k.importance).apply {
                    description = k.about
                    setSound(Uri.parse("android.resource://${context.packageName}/${k.sfx.res}"), audio)
                    enableVibration(true)
                    vibrationPattern = k.buzz.channelPattern()
                }
            )
        }
        manager.createNotificationChannel(
            NotificationChannel(CHANNEL_WATCH, "Watching the bins", NotificationManager.IMPORTANCE_MIN).apply {
                description = "Keeps reading the bins so alerts arrive while the app is closed."
                setShowBadge(false)
            }
        )
    }

    fun allowed(context: Context) = NotificationManagerCompat.from(context).areNotificationsEnabled()

    // Posts one alert. The alert row id doubles as the notification id, so a
    // later alert never replaces an earlier one. FULL alerts also get a
    // "Start pickup" action handled by ActionReceiver.
    @SuppressLint("MissingPermission")
    fun post(context: Context, row: AlertRow, type: AlertType, binId: String) {
        if (!allowed(context)) return
        val k = kinds.getValue(type)
        val id = (row.id % Int.MAX_VALUE).toInt()
        val open = PendingIntent.getActivity(
            context, id,
            Intent(context, MainActivity::class.java).addFlags(Intent.FLAG_ACTIVITY_SINGLE_TOP or Intent.FLAG_ACTIVITY_CLEAR_TOP)
                .putExtra(MainActivity.EXTRA_OPEN, "overview"),
            PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
        )
        val n = NotificationCompat.Builder(context, k.channel)
            .setSmallIcon(R.drawable.ic_stat_bin)
            .setContentTitle(row.title)
            .setContentText(row.meta)
            .setStyle(NotificationCompat.BigTextStyle().bigText(row.meta))
            .setContentIntent(open)
            .setAutoCancel(true)
            .setWhen(row.at)
            .setShowWhen(true)
            .setPriority(if (k.importance >= NotificationManager.IMPORTANCE_HIGH) NotificationCompat.PRIORITY_HIGH else NotificationCompat.PRIORITY_DEFAULT)
            .setCategory(NotificationCompat.CATEGORY_ALARM.takeIf { type == AlertType.TAMPER } ?: NotificationCompat.CATEGORY_STATUS)
```

```
            // Phones older than Android 8 have no channels, so the sound goes on the notification itself.
            .setSound(Uri.parse("android.resource://${context.packageName}/${k.sfx.res}"))
            .setVibrate(k.buzz.channelPattern())
        if (type == AlertType.FULL) {
            val pickup = PendingIntent.getBroadcast(
                context, id,
                Intent(context, ActionReceiver::class.java).setAction(ACTION_COLLECT).putExtra(EXTRA_BIN,
binId).putExtra(EXTRA_NOTIFICATION, id),
                PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
            )
            n.addAction(0, "Start pickup", pickup)
        }
        n.addAction(0, "Open", open)
        runCatching { NotificationManagerCompat.from(context).notify(id, n.build()) }
    }

    /** The quiet notification a foreground service must show. */
    fun watching(context: Context) = NotificationCompat.Builder(context, CHANNEL_WATCH)
        .setSmallIcon(R.drawable.ic_stat_bin)
        .setContentTitle("Watching your bins")
        .setContentText("Alerts arrive while the app is closed.")
        .setOngoing(true)
        .setPriority(NotificationCompat.PRIORITY_MIN)
        .setContentIntent(
            PendingIntent.getActivity(
                context, 0, Intent(context, MainActivity::class.java),
                PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE,
            )
        )
        .build()
}
```

Android App/app/src/main/java/smartbin/notify/WatchService.kt

```
// ================================================================================
// notify/WatchService.kt - foreground service that keeps Hub polling
// Shows the quiet "watching" notification (id 7001, channel sb_watch). Without
// it, Android may stop the process a few minutes after the app leaves the
// screen and background alerts stop. Battery savers on some phones still kill
// it; setting the app's battery use to "Unrestricted" fixes that.
// ================================================================================
package smartbin.notify

import android.app.Service
import android.content.Context
import android.content.Intent
import android.content.pm.ServiceInfo
import android.os.Build
import android.os.IBinder
import androidx.core.content.ContextCompat
import smartbin.data.Hub

/**
 * Keeps the app's process alive while it is in the background, so [Hub] goes
 * on reading the bins every few seconds and alerts still arrive. It runs only
 * while push alerts are on, the app is off screen, and the user had reached
 * the main screens. Swiping the app away ends it, and with it the alerts.
 */
class WatchService : Service() {
    override fun onBind(intent: Intent?): IBinder? = null

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        // Android restarting a killed app's service is not the user opening it.
        if (!Hub.inMain) {
            stopSelf()
            return START_NOT_STICKY
        }
        Hub.init(this)
        Alerts.ensureChannels(this)
        val note = Alerts.watching(this)
        try {
            if (Build.VERSION.SDK_INT >= 29) startForeground(ID, note, ServiceInfo.FOREGROUND_SERVICE_TYPE_CONNECTED_DEVICE)
            else startForeground(ID, note)
        } catch (_: Exception) {
            // Android refused a background start; alerts then arrive only while the app is open.
            stopSelf()
        }
        return START_NOT_STICKY
    }

    /** Swiped away from recent apps: the app counts as closed, so no more alerts. */
    override fun onTaskRemoved(rootIntent: Intent?) {
        Hub.inMain = false
        stopSelf()
        super.onTaskRemoved(rootIntent)
```

```
    }

    companion object {
        private const val ID = 7001

        fun start(context: Context) {
            runCatching { ContextCompat.startForegroundService(context, Intent(context, WatchService::class.java)) }
        }

        fun stop(context: Context) {
            runCatching { context.stopService(Intent(context, WatchService::class.java)) }
        }
    }
}
```

Android App/app/src/main/java/smartbin/notify/ActionReceiver.kt

```
// ================================================================================
// notify/ActionReceiver.kt - handles the "Start pickup" button on a notification
// Sends COLLECT through Hub.command (local Wi-Fi first, then Firebase or
// ThingSpeak TalkBack if the bin is off the Wi-Fi) without opening
// the app, then shows a Toast with the result. goAsync() keeps the receiver
// alive while the network call runs. If the Toast says the bin did not answer,
// the same command from inside the app will fail the same way.
// ================================================================================
package smartbin.notify

import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.widget.Toast
import androidx.core.app.NotificationManagerCompat
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import smartbin.data.Hub

/** "Start pickup" on a full-bin notification: puts the bin in pickup mode without opening the app. */
class ActionReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        if (intent.action != Alerts.ACTION_COLLECT) return
        val bin = intent.getStringExtra(Alerts.EXTRA_BIN) ?: return
        val note = intent.getIntExtra(Alerts.EXTRA_NOTIFICATION, 0)
        val pending = goAsync()
        Hub.init(context)
        CoroutineScope(Dispatchers.Main).launch {
            try {
                val problem = Hub.command(bin, "COLLECT")
                val name = Hub.bin(bin)?.name ?: "The bin"
                Toast.makeText(context, problem ?: "Pickup started on $name.", Toast.LENGTH_SHORT).show()
                if (problem == null) {
                    NotificationManagerCompat.from(context).cancel(note)
                    Feel.event(Sfx.PICKUP)
                }
            } finally {
                pending.finish()
            }
        }
    }
}
```

Android App/app/src/main/java/smartbin/ui/Theme.kt

```
// ================================================================================
// ui/Theme.kt - colors, fonts and the light and dark palettes
// SbColors holds every color token, named after the mockup's CSS variables.
// The palette also changes with the system mode (NORMAL, SIM, CALIB, DUAL for
// both, ALERT), which tints the page so an operator can tell modes apart.
// ================================================================================
package smartbin.ui

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.tween
import androidx.compose.runtime.Composable
import androidx.compose.runtime.Immutable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.staticCompositionLocalOf
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.Font
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import smartbin.R
import smartbin.data.Mode
```

```

enum class ThemeMode { AUTO, LIGHT, DARK }

/** ThingSpeak's colour on the link chip and in Connection: blue, next to Wi-Fi green and Firebase orange. */
val SbColors.ts: Color get() = humid

/**
 * The design tokens of the mockup, the .ph[data-th] and .ph[data-mode]
 * blocks of smartbin-app-mockup v3.html. The accent and the page tint follow
 * the app mode: green for normal, violet for simulation, teal for
 * calibration, blue for both, red for an alert.
 */
@Immutable
data class SbColors(
    val isDark: Boolean,
    val base: Color,
    val tint: Color,
    val shL: Color,
    val shD: Color,
    val ink: Color,
    val ink2: Color,
    val well: Color,
    val ok: Color,
    val warn: Color,
    val bad: Color,
    val sim: Color,
    val cal: Color,
    val accent: Color,
    val accentSoft: Color,
    val onAccent: Color,
    val simSoft: Color,
    val calSoft: Color,
    val temp: Color,
    val humid: Color,
    val weight: Color,
    val gas: Color,
    val press: Color,
    val fill: Color,
    val bt: Color,
    val lid: Color,
    val motion: Color,
    val link: Color,
    val cloud: Color,
    val tamper: Color,
    val lvCool: Color,
    val lvLow: Color,
    val lvMid: Color,
    val lvHigh: Color,
    val lvCrit: Color,
    /** The accent bar and mode chip: one colour, or violet to blue to teal in SIM + CAL. */
    val bar: List<Color>,
) {
    val barBrush: Brush get() = if (bar.size == 1) Brush.linearGradient(listOf(bar[0], bar[0])) else
Brush.horizontalGradient(bar)
    /** Text on the brutal coloured banners: white on light, near black on dark. */
    val onBanner: Color get() = if (isDark) Color(0xFF13100D) else Color.White
}

private val Light = SbColors(
    isDark = false,
    base = Color(0xFFE4DCD2), tint = Color(0xFFE4DCD2), shL = Color(0xFFFFFBF5), shD = Color(0xFFBEB2A4),
    ink = Color(0xFF2A241E), ink2 = Color(0xFF7C7265), well = Color(0xFFDED5CA),
    ok = Color(0xFF1E8E3E), warn = Color(0xFFD98A1F), bad = Color(0xFFD93025), sim = Color(0xFF7A45B5), cal =
Color(0xFF0D8A8A),
    accent = Color(0xFF1E8E3E), accentSoft = Color(0x381E8E3E), onAccent = Color.White,
    simSoft = Color(0x4D7A45B5), calSoft = Color(0x4D0D8A8A),
    temp = Color(0xFFC2591B), humid = Color(0xFF1F6FA8), weight = Color(0xFF7A5A2E), gas = Color(0xFF6B4FB8), press =
Color(0xFFB03A6F),
    fill = Color(0xFF1E7A52), bt = Color(0xFF1A62C4), lid = Color(0xFFB07415), motion = Color(0xFF0E7F9E), link =
Color(0xFF1E8E3E),
    cloud = Color(0xFFD9631E), tamper = Color(0xFFD93025),
    lvCool = Color(0xFF1F6FA8), lvLow = Color(0xFF1E8E3E), lvMid = Color(0xFFB88410), lvHigh = Color(0xFFD9641F), lvCrit =
Color(0xFFD93025),
    bar = listOf(Color(0xFF1E8E3E)),
)

private val Dark = SbColors(
    isDark = true,
    base = Color(0xFF2B2622), tint = Color(0xFF2B2622), shL = Color(0xFF3B342E), shD = Color(0xFF1A1613),
    ink = Color(0xFFEFE7DC), ink2 = Color(0xFFA2978B), well = Color(0xFF262220),
    ok = Color(0xFF4FBF6E), warn = Color(0xFFE8A94A), bad = Color(0xFFFF6B5E), sim = Color(0xFFB98BE8), cal =
Color(0xFF3FC7C2),
    accent = Color(0xFF4FBF6E), accentSoft = Color(0x384FBF6E), onAccent = Color(0xFF13100D),
    simSoft = Color(0x57B98BE8), calSoft = Color(0x523FC7C2),
    temp = Color(0xFFF0A368), humid = Color(0xFF6CB8E8), weight = Color(0xFFD6B98C), gas = Color(0xFFB596F0), press =
Color(0xFFE58AB4),
```

```
    fill = Color(0xFF4FC48D), bt = Color(0xFF79AEF5), lid = Color(0xFFE8B45C), motion = Color(0xFF5FD0E8), link = Color(0xFF4FBF6E),
    cloud = Color(0xFFF29A5C), tamper = Color(0xFFFF6B5E),
    lvCool = Color(0xFF6CB8E8), lvLow = Color(0xFF4FBF6E), lvMid = Color(0xFFE8C45A), lvHigh = Color(0xFFF0A368), lvCrit = Color(0xFFFF6B5E),
    bar = listOf(Color(0xFF4FBF6E)),
)

/** The tokens for one theme and one app mode. */
fun sbColors(dark: Boolean, mode: Mode): SbColors {
    val c = if (dark) Dark else Light
    return when (mode) {
        Mode.NORMAL -> c
        Mode.SIM -> c.copy(
            accent = c.sim, bar = listOf(c.sim),
            accentSoft = if (dark) Color(0x38B98BE8) else Color(0x387A45B5),
            tint = if (dark) Color(0xFF2C2530) else Color(0xFFE2DAE1),
        )
        Mode.CALIB -> c.copy(
            accent = c.cal, bar = listOf(c.cal),
            accentSoft = if (dark) Color(0x333FC7C2) else Color(0x330D8A8A),
            tint = if (dark) Color(0xFF222B2A) else Color(0xFFDBE3E0),
        )
        Mode.DUAL -> {
            val mid = if (dark) Color(0xFF7CA9D5) else Color(0xFF43679F)
            c.copy(
                accent = mid, bar = listOf(c.sim, mid, c.cal),
                accentSoft = if (dark) Color(0x3D7CA9D5) else Color(0x3D43679F),
                tint = if (dark) Color(0xFF262A33) else Color(0xFFDEDDE6),
            )
        }
        Mode.ALERT -> c.copy(
            accent = c.bad, bar = listOf(c.bad),
            accentSoft = if (dark) Color(0x33FF6B5E) else Color(0x33D93025),
            tint = if (dark) Color(0xFF302422) else Color(0xFFEADCD5),
        )
    }
}

/** Colours that change with the mode ease across, as the mockup's background does in 0.5 s. */
@Composable
fun animated(c: SbColors): SbColors {
    val spec = tween<Color>(500)
    val accent by animateColorAsState(c.accent, spec, label = "accent")
    val soft by animateColorAsState(c.accentSoft, spec, label = "soft")
    val tint by animateColorAsState(c.tint, spec, label = "tint")
    return c.copy(accent = accent, accentSoft = soft, tint = tint)
}

val LocalSb = staticCompositionLocalOf { Light }

object Sb {
    val c: SbColors
        @Composable get() = LocalSb.current
}

val Inter = FontFamily(
    Font(R.font.inter_regular, FontWeight.Normal),
    Font(R.font.inter_regular, FontWeight.Medium),
    Font(R.font.inter_semibold, FontWeight.SemiBold),
    Font(R.font.inter_bold, FontWeight.Bold),
    Font(R.font.inter_extrabold, FontWeight.ExtraBold),
    Font(R.font.inter_extrabold, FontWeight.Black),
)

/**
 * Each bin has its own colour. Twelve hues spread round the wheel, red left
 * out because red means a limit, ordered so neighbours differ. Bins 13 to 24
 * reuse them one step darker and 25 to 36 one step lighter, the same formula
 * as the mockup.
 */
private val HUES = intArrayOf(153, 287, 47, 207, 100, 260, 20, 180, 313, 73, 233, 127)
private val ODD = intArrayOf(0, 1, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1)

fun binColor(i: Int, dark: Boolean): Color {
    val t = (i / 12) % 3
    val r = i / 36
    val h = (HUES[i % 12] + r * 11) % 360
    val odd = ODD[i % 12] == 1
    val l = (if (dark) intArrayOf(64, 78, 50)[t] else intArrayOf(40, 27, 54)[t]) + if (odd) (if (dark) -9 else 8) else 0
    val s = if (dark) intArrayOf(70, 60, 72)[t] else intArrayOf(66, 62, 72)[t]
    return Color.hsl(h.toFloat(), s / 100f, l / 100f)
}

/** color-mix(in srgb, x p%, transparent) */
fun Color.soft(p: Float) = copy(alpha = alpha * p)
```

```

/** color-mix(in srgb, a p%, b) */
fun mix(a: Color, p: Float, b: Color) = Color(
    red = a.red * p + b.red * (1 - p),
    green = a.green * p + b.green * (1 - p),
    blue = a.blue * p + b.blue * (1 - p),
    alpha = 1f,
)
```

Android App/app/src/main/java/smartbin/ui/Neu.kt

```
// ================================================================================
// ui/Neu.kt - the neumorphic and neo-brutalist surfaces
// Modifiers that draw the soft raised and sunk shadows (raised, sunk, nm,
// bump, well) and the hard offset block (brutal). Shadow sizes match the
// mockup's --o1/--o2/--i1/--i2 values so the app and dashboard look alike.
// ================================================================================
package smartbin.ui

import android.graphics.BlurMaskFilter
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.drawBehind
import androidx.compose.ui.draw.drawWithCache
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.RoundRect
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.Paint
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.drawscope.clipPath
import androidx.compose.ui.graphics.drawscope.drawIntoCanvas
import androidx.compose.ui.graphics.nativeCanvas
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp

/*
 * The mockup's two shadow systems. The soft pair (light above left, dark
 * below right) is the Neumorphism depth: --o1 and --o2 raise a surface,
 * --i1 and --i2 sink it. The hard offset block with a 2 dp ink edge is the
 * Neo-Brutalism part, used for banners, alarms and danger keys.
 */

/** --o1 and --o2: offset and blur in px. */
enum class Out(val off: Dp, val blur: Dp) { O1(4.dp, 9.dp), O2(7.dp, 15.dp), POP(3.dp, 8.dp) }

/** --i1 and --i2 */
enum class In(val off: Dp, val blur: Dp) { I1(4.dp, 8.dp), I2(6.dp, 12.dp) }

// A CSS blur of b px is a Gaussian with sigma b / 2. BlurMaskFilter takes a
// radius where sigma is about 0.577 * radius, so the factor is close to 0.85.
private const val BLUR_K = 0.85f

private fun blurPaint(color: Color, blurPx: Float, stroke: Float = 0f) =
    Paint().asFrameworkPaint().apply {
        isAntiAlias = true
        this.color = color.toArgb()
        if (stroke > 0f) {
            style = android.graphics.Paint.Style.STROKE
            strokeWidth = stroke
        }
        if (blurPx > 0f) maskFilter = BlurMaskFilter(blurPx, BlurMaskFilter.Blur.NORMAL)
    }

/**
 * Raised: the soft pair, plus an optional ring (box-shadow 0 0 0 Npx) drawn
 * outside the edge. Put it before the clip and the background.
 */
fun Modifier.raised(c: SbColors, radius: Dp, out: Out = Out.O1, ring: Color? = null, ringWidth: Dp = 2.dp): Modifier =
drawWithCache {
    val r = radius.toPx().coerceAtMost(size.minDimension / 2f)
    val off = out.off.toPx()
    val light = blurPaint(c.shL, out.blur.toPx() * BLUR_K)
    val dark = blurPaint(c.shD, out.blur.toPx() * BLUR_K)
    val rw = ringWidth.toPx()
    onDrawBehind {
        drawIntoCanvas { canvas ->
            val n = canvas.nativeCanvas
```

```
            n.drawRoundRect(-off, -off, size.width - off, size.height - off, r, r, light)
            n.drawRoundRect(off, off, size.width + off, size.height + off, r, r, dark)
        }
        if (ring != null && ring.alpha > 0f) {
            drawRoundRect(
                ring, topLeft = Offset(-rw / 2, -rw / 2), size = Size(size.width + rw, size.height + rw),
                cornerRadius = CornerRadius(r + rw / 2, r + rw / 2), style = Stroke(rw),
            )
        }
    }
}

/** Sunk: the inner soft pair. Put it after the background. */
fun Modifier.sunk(c: SbColors, radius: Dp, inset: In = In.I1): Modifier = drawWithCache {
    val r = radius.toPx().coerceAtMost(size.minDimension / 2f)
    val off = inset.off.toPx()
    val clip = Path().apply { addRoundRect(RoundRect(0f, 0f, size.width, size.height, CornerRadius(r, r))) }
    val dark = blurPaint(c.shD, inset.blur.toPx() * BLUR_K, stroke = off * 2)
    val light = blurPaint(c.shL, inset.blur.toPx() * BLUR_K, stroke = off * 2)
    onDrawBehind {
        clipPath(clip) {
            drawIntoCanvas { canvas ->
                val n = canvas.nativeCanvas
                n.drawRoundRect(off, off, size.width + off, size.height + off, r, r, dark)
                n.drawRoundRect(-off, -off, size.width - off, size.height - off, r, r, light)
            }
        }
    }
}

/** A ring outside the edge with nothing else, for a surface that is already sunk. */
fun Modifier.ring(color: Color, radius: Dp, width: Dp = 2.dp): Modifier = drawBehind {
    val rw = width.toPx()
    val r = radius.toPx().coerceAtMost(size.minDimension / 2f)
    drawRoundRect(
        color, topLeft = Offset(-rw / 2, -rw / 2), size = Size(size.width + rw, size.height + rw),
        cornerRadius = CornerRadius(r + rw / 2, r + rw / 2), style = Stroke(rw),
    )
}

/** The brutal block: a hard offset shadow and a 2 dp ink edge. */
fun Modifier.brutal(c: SbColors, radius: Dp, fill: Color, off: Dp = 4.dp, edge: Color = c.ink): Modifier {
    val shape = RoundedCornerShape(radius)
    return this
        .drawBehind {
            val r = radius.toPx()
            drawRoundRect(edge, topLeft = Offset(off.toPx(), off.toPx()), size = size, cornerRadius = CornerRadius(r, r))
        }
        .clip(shape)
        .background(fill)
        .border(2.dp, edge, shape)
}

/** .nm: a raised card, a shade lighter toward the light. */
fun Modifier.nm(c: SbColors, radius: Dp = 18.dp, out: Out = Out.O2, ring: Color? = null, ringWidth: Dp = 2.dp): Modifier {
    val shape = RoundedCornerShape(radius)
    return this
        .raised(c, radius, out, ring, ringWidth)
        .clip(shape)
        .background(c.base)
        .background(
            Brush.linearGradient(
                0f to c.shL.copy(alpha = 0.42f), 0.58f to Color.Transparent,
                start = Offset(0f, 0f), end = Offset(Float.POSITIVE_INFINITY, Float.POSITIVE_INFINITY),
            )
        )
}

/** A flat raised surface: --o1 on the page colour. */
fun Modifier.bump(c: SbColors, radius: Dp, out: Out = Out.O1, ring: Color? = null, fill: Color = c.base): Modifier =
    this.raised(c, radius, out, ring).clip(RoundedCornerShape(radius)).background(fill)

/** A well: the page sunk in, for inputs, steppers and readouts. */
fun Modifier.well(c: SbColors, radius: Dp, inset: In = In.I1, fill: Color = c.well): Modifier =
    this.clip(RoundedCornerShape(radius)).background(fill).sunk(c, radius, inset)

/** The page: two soft blooms of the accent over the mode tint. */
fun Modifier.page(c: SbColors): Modifier = drawBehind {
    drawRect(c.tint)
    drawRect(
        Brush.radialGradient(
            0f to c.accent.copy(alpha = 0.09f), 0.7f to Color.Transparent,
            center = Offset(size.width * 0.10f, -size.height * 0.06f), radius = size.width * 1.3f,
        )
    )
    drawRect(
```

```
        Brush.radialGradient(
            0f to c.accent.copy(alpha = 0.06f), 0.72f to Color.Transparent,
            center = Offset(size.width * 1.08f, size.height * 0.06f), radius = size.width * 1.1f,
        )
    )
}
```

Android App/app/src/main/java/smartbin/ui/Icons.kt

```
// ========================================================================
// ui/Icons.kt - line icons and small animations
// svg() turns a tiny SVG string (path, circle, rect, line) into a Compose
// ImageVector, so each icon is written as the same SVG markup a web page uses.
// Ic holds every icon by name; shake, spin and breathe are the animations.
// ========================================================================
package smartbin.ui

import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.tween
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Icon
import androidx.compose.material3.LocalContentColor
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.StrokeJoin
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.drawscope.clipPath
import androidx.compose.ui.graphics.drawscope.scale
import androidx.compose.ui.graphics.drawscope.translate
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.graphics.vector.PathParser
import androidx.compose.ui.graphics.vector.addPathNodes
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import kotlin.math.PI
import kotlin.math.cos
import kotlin.math.sin

/*
 * The icon set of the mockup (copied there from dashboard v9), kept as the
 * same SVG markup and turned into vectors by a small reader. Only what the
 * mockup uses is understood: path, circle and rect, fill="currentColor" for
 * a solid shape, opacity, stroke-width, and groups that turn or move.
 */

private val TAG = Regex("""<(/?)(\w+)([^>]*?)/?>""")
private val ATTR = Regex("""([\w-]+)="([^"]*)"""")

private fun circleD(cx: Float, cy: Float, r: Float) = "M${cx - r} ${cy}a$r $r 0 1 0 ${2 * r} 0a$r $r 0 1 0 ${-2 * r} 0z"

private fun rectD(x: Float, y: Float, w: Float, h: Float, rx: Float): String {
    if (rx <= 0f) return "M$x ${y}h${w}v${h}h${-w}z"
    val r = minOf(rx, w / 2, h / 2)
    val iw = w - 2 * r
    val ih = h - 2 * r
    return "M${x + r} ${y}h${iw}a$r $r 0 0 1 $r ${r}v${ih}a$r $r 0 0 1 ${-r} ${r}h${-iw}a$r $r 0 0 1 ${-r} ${-r}v${-ih}a$r $r
0 0 1 $r ${-r}z"
}

fun svg(name: String, body: String, sw: Float = 2f): ImageVector {
    val b = ImageVector.Builder(name, 24.dp, 24.dp, 24f, 24f)
    val ink = SolidColor(Color.Black)
    val alphas = ArrayList<Float>().apply { add(1f) }
    for (m in TAG.findAll(body)) {
        val closing = m.groupValues[1] == "/"
        val tag = m.groupValues[2]
        val a = ATTR.findAll(m.groupValues[3]).associate { it.groupValues[1] to it.groupValues[2] }
        fun f(k: String, d: Float = 0f) = a[k]?.toFloatOrNull() ?: d
        if (tag == "g") {
            if (closing) {
                b.clearGroup()
                if (alphas.size > 1) alphas.removeAt(alphas.lastIndex)
            } else {
```

```
                b.addGroup(
                    rotate = f("rot"), pivotX = f("px"), pivotY = f("py"),
                    translationX = f("tx"), translationY = f("ty"),
                )
                alphas.add(alphas.last() * f("opacity", 1f))
            }
            continue
        }
        if (closing) continue
        val d = when (tag) {
            "path" -> a["d"] ?: continue
            "circle" -> circleD(f("cx"), f("cy"), f("r"))
            "rect" -> rectD(f("x"), f("y"), f("width"), f("height"), f("rx"))
            else -> continue
        }
        val alpha = alphas.last() * f("opacity", 1f)
        if (a["fill"] == "currentColor") {
            b.addPath(addPathNodes(d), fill = ink, fillAlpha = alpha)
        } else {
            b.addPath(
                addPathNodes(d), fill = null, stroke = ink, strokeAlpha = alpha, strokeLineWidth = f("stroke-width", sw),
                strokeLineCap = StrokeCap.Round, strokeLineJoin = StrokeJoin.Round,
            )
        }
    }
    return b.build()
}

/** The calibration ring is dashed 7 on, 6 off; vectors have no dashes, so the dashes are arcs. */
private fun dashedRing(cx: Float, cy: Float, r: Float, on: Float, off: Float): String {
    val circ = 2 * PI.toFloat() * r
    val n = (circ / (on + off)).toInt().coerceAtLeast(1)
    val step = 360f / n
    val sweep = step * on / (on + off)
    val sb = StringBuilder()
    for (k in 0 until n) {
        val a0 = Math.toRadians((k * step - 90).toDouble())
        val a1 = Math.toRadians((k * step + sweep - 90).toDouble())
        val x0 = cx + r * cos(a0).toFloat()
        val y0 = cy + r * sin(a0).toFloat()
        val x1 = cx + r * cos(a1).toFloat()
        val y1 = cy + r * sin(a1).toFloat()
        sb.append("M$x0 ${y0}A$r $r 0 0 1 $x1 $y1")
    }
    return sb.toString()
}

object Ic {
    private val cache = HashMap<String, ImageVector>()
    private fun k(key: String, body: () -> String): ImageVector = cache.getOrPut(key) { svg(key, body()) }

    val grid get() = k("grid") { """<rect x="3" y="3" width="7.5" height="7.5" rx="1.6"/><rect x="13.5" y="3" width="7.5" height="7.5" rx="1.6"/><rect x="3" y="13.5" width="7.5" height="7.5" rx="1.6"/><rect x="13.5" y="13.5" width="7.5" height="7.5" rx="1.6"/>""" }
    val chart get() = k("chart") { """<path d="M3 20h18"/><path d="M5 15l4.5-5 3.5 3.5L19 6"/>""" }
    val sliders get() = k("sliders") { """<path d="M4 7h10M18 7h2M4 17h4M12 17h8"/><circle cx="16" cy="7" r="2.2"/><circle cx="10" cy="17" r="2.2"/>""" }
    val flask get() = k("flask") { """<path d="M9.5 3v6.2L4.6 18a2 2 0 0 0 1.7 3h11.4a2 2 0 0 0 1.7-3L14.5 9.2V3"/><path d="M8 3h8M7.4 15h9.2"/>""" }
    val bell get() = k("bell") { """<path d="M18 9a6 6 0 1 0-12 0c0 6-2 7-2 7h16s-2-1-2-7"/><path d="M10.4 20.5a2 2 0 0 0 3.2 0"/>""" }
    val plus get() = k("plus") { """<path d="M12 5v14M5 12h14"/>""" }
    val gear get() = k("gear") { """<circle cx="12" cy="12" r="3"/><path d="M12 2.8v2.6M12 18.6v2.6M2.8 12h2.6M18.6 12h2.6M5.5 5.5l1.8 1.8M16.7 16.7l1.8 1.8M18.5 5.5l-1.8 1.8M7.3 16.7l-1.8 1.8"/>""" }
    val truck get() = k("truck") { """<path d="M2 7h11v9H2zM13 10h4.5l3.5 3.5V16h-8z"/><circle cx="6.5" cy="18.5" r="1.8"/><circle cx="16.5" cy="18.5" r="1.8"/>""" }
    val check get() = k("check") { """<path d="M4 12.5l5 5L20 6.5"/>""" }
    val humid get() = k("humid") { """<path d="M12 3.2s6.2 6.6 6.2 10.3a6.2 6.2 0 1 1-12.4 0C5.8 9.8 12 3.2 12 3.2z"/><circle cx="12" cy="14.4" r="3" fill="currentColor" opacity=".45"/>""" }
    val gas get() = k("gas") { """<path d="M6.5 17c0-2.6 2.6-2.6 2.6-5.2S6.5 9.2 6.5 6.6"/><path d="M12 18c0-2.6 2.6-2.6 2.6-5.2S12 10.2 12 7.6"/><path d="M17.5 17c0-2.6 2.6-2.6 2.6-5.2s-2.6-2.6-2.6-5.2"/>""" }
    val press get() = k("press") { """<path d="M4.6 17.6a8.6 8.6 0 1 1 14.8 0"/><path d="M12 14l3.6-4.4"/><circle cx="12" cy="14.2" r="1.5" fill="currentColor"/><path d="M7.2 17.6h9.6"/>""" }
    val tamper get() = k("tamper") { """<path d="M12 3.4L22 20H2z"/><path d="M12 9.6v4.2M12 17h.01"/>""" }
    val calib get() = k("calib") { """<path d="${dashedRing(12f, 12f, 8.4f, 7f, 6f)}"/><circle cx="12" cy="12" r="2.4" fill="currentColor"/><path d="M12 1.6v2.4M12 20v2.4M1.6 12h2.4M20 12h2.4"/>""" }
    val fill get() = k("fill") { """<path d="M5.4 4.4h13.2l-1.3 15.2a2 2 0 0 1-2 1.8H8.7a2 2 0 0 1-2-1.8z"/><path d="M6.9 12.6c1.6-1.1 3.3-1.1 4.9 0s3.3 1.1 4.9 0"/>""" }
    val bins get() = k("bins") { """<path d="M3.2 7.6h7l-.8 11.8H4zM13.8 7.6h7l-.8 11.8h-5.4z"/><path d="M2.4 7.6h8.6M13 7.6h8.6M5.4 5h2.6M16 5h2.6"/>""" }
    val clock get() = k("clock") { """<circle cx="12" cy="12" r="8.4"/><path d="M12 7.2V12l3.2 2"/>""" }
    val trash get() = k("trash") { """<path d="M4.4 6.8h15.2M9.6 6.8V4.6h4.8v2.2M6.8 6.8l1 12.6h8.4l1-12.6M10.2 10.4v5.4M13.8 10.4v5.4"/>""" }
    val stopwatch get() = k("stopwatch") { """<circle cx="12" cy="13.6" r="7.4"/><path d="M12 9.8v3.8l2.6 1.6M9.6 2.8h4.8M19.4 7.2l1.4-1.4"/>""" }
    val layers get() = k("layers") { """<path d="M12 3.4l8.6 4.6L12 12.6 3.4 8z"/><path d="M3.4 12.4L12 17l8.6-4.6M3.4
```

```
16.6L12 21.2l8.6-4.6"/>""" }
    val rank get() = k("rank") { """<path d="M9.6 6.6h10.8M9.6 12h10.8M9.6 17.4h10.8"/><path d="M4.8 5v3.6M3.6 5h1.2M3.4 10.4h2.8M3.4 15.4h2.6v1.6h-2.6v1.8h2.6"/>""" }
    val eye get() = k("eye") { """<path d="M2.4 12s3.6-6.4 9.6-6.4S21.6 12 21.6 12s-3.6 6.4-9.6 6.4S2.4 12 2.4 12z"/><circle cx="12" cy="12" r="2.8"/>""" }
    val eyeoff get() = k("eyeoff") { """<path d="M4 4l16 16"/><path d="M9.6 9.7a2.8 2.8 0 0 0 3.9 3.9"/><path d="M6.4 6.8C3.9 8.5 2.4 12 2.4 12s3.6 6.4 9.6 6.4c1.6 0 3-.4 4.2-1M18.3 15.1c1.9-1.6 3.3-3.1 3.3-3.1S18 5.6 12 5.6c-.8 0-1.5.1-2.2.3"/>""" }
    val bluetooth get() = k("bluetooth") { """<path d="M7.4 7.4l9.2 9.2L12 20.6V3.4l4.6 4L7.4 16.6"/>""" }
    val radar get() = k("radar") { """<path d="M12 12l5.6-5.6"/><circle cx="12" cy="12" r="1.6" fill="currentColor"/><path d="M8.4 15.6a5.1 5.1 0 0 1 0-7.2M5.6 18.4a9 9 0 0 1 0-12.8"/><path d="M15.6 8.4a5.1 5.1 0 0 1 0 7.2"/>""" }
    val key get() = k("key") { """<circle cx="8.2" cy="12" r="3.6"/><path d="M11.8 12h9v3M17.6 12v2.6"/>""" }
    val ruler get() = k("ruler") { """<rect x="2.6" y="8" width="18.8" height="8" rx="1.8"/><path d="M7 8v3M11 8v4M15 8v3M19 8v4"/>""" }
    val percent get() = k("percent") { """<path d="M18.5 5.5l-13 13"/><circle cx="7.6" cy="7.6" r="2.3"/><circle cx="16.4" cy="16.4" r="2.3"/>""" }
    val save get() = k("save") { """<path d="M4.5 4.5h11l4 4v11h-15z"/><path d="M8 4.5v5h6v-5M8 19.5v-5.5h8v5.5"/>""" }
    val reload get() = k("reload") { """<path d="M20.5 12a8.5 8.5 0 0 1 1-2.6-6.1"/><path d="M20.8 4.4v4.8h-4.8"/>""" }
    val download get() = k("download") { """<path d="M12 3.6v11.2M7.6 10.6l4.4 4.4 4.4-4.4"/><path d="M4.4 19.4h15.2"/>""" }
    val undo get() = k("undo") { """<path d="M3.6 12a8.4 8.4 0 1 0 2.6-6.1"/><path d="M3.3 4.4v4.8h4.8"/>""" }
    val capture get() = k("capture") { """<circle cx="12" cy="12" r="7.6"/><circle cx="12" cy="12" r="1.8" fill="currentColor"/><path d="M12 1.8v2.8M12 19.4v2.8M1.8 12h2.8M19.4 12h2.8"/>""" }
    val pin get() = k("pin") { """<rect x="3.4" y="4.6" width="17.2" height="14.8" rx="2.2"/><path d="M7.4 9.4h.01M12 9.4h.01M16.6 9.4h.01M7.4 14.6h9.2"/>""" }
    val tilt get() = k("tilt") { """<path d="M3.4 20.2h17.2"/><path d="M6.6 20.2L15 6.4"/><path d="M11.4 20.2a7.5 7.5 0 0 0-1.6-4.4"/>""" }
    val servo get() = k("servo") { """<circle cx="12" cy="13.6" r="3.2"/><path d="M12 10.4V4.6"/><path d="M4.6 13.6a7.4 7.4 0 0 1 3.2-6.1M19.4 13.6a7.4 7.4 0 0 0-3.2-6.1"/>""" }
    val cross get() = k("cross") { """<path d="M6.4 6.4l11.2 11.2M17.6 6.4L6.4 17.6"/>""" }
    val power get() = k("power") { """<path d="M12 3.4v8.2"/><path d="M6.8 6.6a7.6 7.6 0 1 0 10.4 0"/>""" }
    val chev get() = k("chev") { """<path d="M6 9l6 6 6-6"/>""" }
    val cloud get() = k("cloud") { """<path d="M7 18.6h10.4a4.2 4.2 0 0 0 .5-8.4 6 6 0 0 0-11.5-1A4.8 4.8 0 0 0 7 18.6z"/>""" }
    val back get() = k("back") { """<path d="M15 5l-7 7 7 7"/>""" }
    val expand get() = k("expand") { """<path d="M9.5 3.5H3.5v6M14.5 3.5h6v6M20.5 14.5v6h-6M3.5 14.5v6h6"/>""" }
    val shrink get() = k("shrink") { """<path d="M3.5 9.5h6v-6M20.5 9.5h-6v-6M14.5 20.5v-6h6M9.5 20.5v-6h-6"/>""" }
    val user get() = k("user") { """<circle cx="12" cy="8.2" r="3.8"/><path d="M4.6 20.4a7.4 7.4 0 0 1 14.8 0"/>""" }
    val palette get() = k("palette") { """<path d="M12 3.4a8.6 8.6 0 1 0 0 17.2c1.2 0 1.9-.8 1.9-1.8 0-1-.9-1.5-.9-2.4s.7-1.6 1.7-1.6h1.3a4.6 4.6 0 0 0 4.6-4.6c0-3.8-3.9-6.8-8.6-6.8z"/><circle cx="7.8" cy="11.6" r="1.1" fill="currentColor"/><circle cx="10.6" cy="7.8" r="1.1" fill="currentColor"/><circle cx="15" cy="8.4" r="1.1" fill="currentColor"/>""" }
    val speaker get() = k("speaker") { """<path d="M4 9.5h3.6L12.4 5v14l-4.8-4.5H4z"/><path d="M15.6 9a4.2 4.2 0 0 1 0 6M18.4 6.4a8 8 0 0 1 0 11.2"/>""" }
    val vibrate get() = k("vibrate") { """<rect x="7.4" y="3.6" width="9.2" height="16.8" rx="2"/><path d="M3.6 8.4v7.2M20.4 8.4v7.2M11 17.2h2"/>""" }
    val sun get() = k("sun") { """<circle cx="12" cy="12" r="3.8"/><path d="M12 2.6v2.4M12 19v2.4M2.6 12h2.4M19 12h2.4M5.4 5.4l1.7 1.7M16.9 16.9l1.7 1.7M5.4l-1.7 1.7M7.1 16.9l-1.7 1.7"/>""" }

    /** The thermometer fills with the temperature, 10 to 50 C. */
    fun temp(t: Double?): ImageVector {
        val f = (((t ?: 28.0) - 10) / 40).coerceIn(0.05, 1.0)
        val q = (f * 20).toInt() / 20.0
        return k("temp$q") {
            val h = 6 * q + 1.4
            val y = 15.4 - h
            """<path d="M14.2 14.4V5.2a2.2 2.2 0 1 0-4.4 0v9.2a4.6 4.6 0 1 0 4.4 0z"/><rect x="10.6" y="$y" width="2.8" height="$h" rx="1.4" fill="currentColor"/><circle cx="12" cy="17.2" r="2.6" fill="currentColor"/>"""
        }
    }

    /** The scale needle swings with the load, a share of capacity. */
    fun weight(f: Double?): ImageVector {
        val deg = (((f ?: 0.3).coerceIn(0.0, 1.1) * 150 - 75) / 5).toInt() * 5
        return k("weight$deg") {
            """<path d="M4.6 20.5h14.8l-1.9-9.6H6.5z"/><path d="M9 10.9a3 3 0 1 1 6 0"/><g rot="$deg" px="12" py="17.6"><path d="M12 17.6v-3.1" stroke-width="2.2"/></g>"""
        }
    }

    fun lid(open: Boolean) = k("lid$open") {
        """<path d="M6.6 9.5l1.1 11h8.6l1.1-11"/><g rot="${if (open) -38 else 0}" px="6" py="9"><path d="M4.5 9.5h15"/><path d="M9.6 7.1h4.8"/></g>"""
    }

    fun motion(on: Boolean) = k("motion$on") {
        """<circle cx="5.5" cy="12" r="1.8" fill="currentColor"/><g opacity="${if (on) 1 else 0.4}"><path d="M10 7.5a7 7 0 0 1 0 9"/><path d="M13.6 5a11 11 0 0 1 0 14"/><path d="M17.2 2.6a15 15 0 0 1 0 18.8"/></g>"""
    }

    fun lock(locked: Boolean) = k("lock$locked") {
        """<rect x="4.5" y="10.5" width="15" height="10.5" rx="2.4"/><g ty="${if (locked) 0 else -3}"><path d="M8 10.5V7.4a4 4 0 0 1 8 0v3.1"/></g><circle cx="12" cy="15.7" r="1.5" fill="currentColor"/>"""
    }

    fun sunmoon(dark: Boolean) = k("sunmoon$dark") {
        val sun = if (dark) 1.0 else 0.3
        val moon = if (dark) 0.3 else 1.0
        """<g opacity="$sun"><circle cx="8.2" cy="12" r="3"/><path d="M8.2 5.4v1.6M8.2 17v1.6M3.4 7.2l1.2 1.2M11.8 15.6l1.2
```

```
1.2M3.4 16.8l1.2-1.2M1.2 12h1.6"/></g><g opacity="$moon"><path d="M19.4 7.6a5.4 5.4 0 1 0 0 8.8 6.2 6.2 0 0 1 0-8.8z"/></g>"""
    }

    fun wifi(live: Boolean) = k("wifi$live") {
        """<path d="M8.5 15.2a5 5 0 0 1 7 0" stroke-width="2.2"/><path d="M5.5 11.8a9.4 9.4 0 0 1 13 0" stroke-width="2.2"/><path d="M2.4 8.4a14 14 0 0 1 19.2 0" stroke-width="2.2"/><circle cx="12" cy="18.6" r="1.4" fill="currentColor"/>"""
    }

    /** The icon names alert rows carry. */
    fun named(n: String): ImageVector = when (n) {
        "fill" -> fill; "gas" -> gas; "tamper" -> tamper; "humid" -> humid; "lock" -> lock(true)
        "check" -> check; "truck" -> truck; "calib" -> calib; "temp" -> temp(41.0); "save" -> save; "gear" -> gear
        else -> bell
    }
}

/** An icon from [Ic] in the colour of the text around it, or [tint]. */
@Composable
fun Ico(icon: ImageVector, size: Dp = 21.dp, tint: Color = LocalContentColor.current, modifier: Modifier = Modifier) {
    Icon(icon, contentDescription = null, modifier = modifier.size(size), tint = tint)
}

/** @keyframes shake: the tamper triangle rocks while it is shown. */
@Composable
fun Modifier.shake(on: Boolean = true): Modifier {
    if (!on) return this
    val t = rememberInfiniteTransition(label = "shake")
    val r by t.animateFloat(-6f, 6f, infiniteRepeatable(tween(225), RepeatMode.Reverse), label = "shake")
    return graphicsLayer { rotationZ = r }
}

@Composable
fun Modifier.spin(on: Boolean = true): Modifier {
    if (!on) return this
    val t = rememberInfiniteTransition(label = "spin")
    val r by t.animateFloat(0f, 360f, infiniteRepeatable(tween(1100, easing = LinearEasing)), label = "spin")
    return graphicsLayer { rotationZ = r }
}

/** @keyframes breathe, 0.82 to 1.12 in scale and 0.45 to 1 in opacity. */
@Composable
fun Modifier.breathe(on: Boolean = true, millis: Int = 2800): Modifier {
    if (!on) return this
    val t = rememberInfiniteTransition(label = "breathe")
    val f by t.animateFloat(0f, 1f, infiniteRepeatable(tween(millis / 2), RepeatMode.Reverse), label = "breathe")
    return graphicsLayer {
        val s = 0.82f + 0.30f * f
        scaleX = s
        scaleY = s
        alpha = 0.45f + 0.55f * f
    }
}

private val BIN_BODY = PathParser().parsePathString("M11.5 14h25l-2.6 26.2a2.4 2.4 0 0 1-2.4 2.1H16.5a2.4 2.4 0 0 1-2.4-2.1z").toPath()
private val BIN_LID = PathParser().parsePathString("M8 13h32M19 9.4h10").toPath()
private val WAVE_A = PathParser().parsePathString("M-24 4 q6 -5 12 0 t12 0 t12 0 t12 0 t12 0 t12 0 v36 h-72z").toPath()
private val WAVE_B = PathParser().parsePathString("M-24 7 q6 -5 12 0 t12 0 t12 0 t12 0 t12 0 t12 0 v36 h-72z").toPath()

/**
 * The bin with its fill level as a moving wave, the hero of every bin card
 * and tile. The waves drift sideways the whole time and rise or fall to a
 * new level over 0.6 s.
 */
@Composable
fun BinIcon(pct: Double, color: Color, size: Dp = 46.dp, modifier: Modifier = Modifier) {
    val level by animateFloatAsState(pct.coerceIn(0.0, 100.0).toFloat(), tween(600), label = "level")
    val t = rememberInfiniteTransition(label = "wave")
    val a by t.animateFloat(0f, 24f, infiniteRepeatable(tween(3400, easing = LinearEasing)), label = "a")
    val b by t.animateFloat(0f, 24f, infiniteRepeatable(tween(5000, easing = LinearEasing)), label = "b")
    Canvas(modifier.size(size)) {
        scale(this.size.width / 48f, pivot = Offset.Zero) {
            val y = 42f - level / 100f * 30f
            clipPath(BIN_BODY) {
                translate(a, y) { drawPath(WAVE_A, color, alpha = 0.85f) }
                translate(b, y) { drawPath(WAVE_B, color, alpha = 0.35f) }
            }
            drawPath(BIN_BODY, color, style = Stroke(2.6f, join = StrokeJoin.Round))
            drawPath(BIN_LID, color, style = Stroke(2.6f, cap = StrokeCap.Round))
        }
    }
}
```

Android App/app/src/main/java/smartbin/ui/Logo.kt

```
// ========================================================================
// ui/Logo.kt - the animated Smart-Bin logo on the start page and header
// The bin, waves, signal arcs and chip are vector paths on a 64 x 64 grid.
// play counts up to replay the build-up animation.
// ========================================================================
package smartbin.ui

import androidx.compose.animation.core.Animatable
import androidx.compose.animation.core.CubicBezierEasing
import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.tween
import androidx.compose.foundation.Canvas
import androidx.compose.foundation.layout.size
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Modifier
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.StrokeJoin
import androidx.compose.ui.graphics.drawscope.DrawScope
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.drawscope.clipPath
import androidx.compose.ui.graphics.drawscope.rotate
import androidx.compose.ui.graphics.drawscope.scale
import androidx.compose.ui.graphics.drawscope.translate
import androidx.compose.ui.graphics.vector.PathParser
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import kotlinx.coroutines.coroutineScope
import kotlinx.coroutines.launch

/*
 * The v2 Smart-Bin logo, the same drawing as the dashboard's: a bin with a fill
 * wave, a controller chip on its front, a status light on the lid and a Wi-Fi
 * signal above it. It plays a short build-up (the lid lifts, the bin fills, the
 * chip lights, the signal goes out) and then holds its still frame, which is
 * also the home-screen icon. Drawn on a 64 by 64 grid.
 */
private fun p(d: String) = PathParser().parsePathString(d).toPath()

private val L_BODY = p("M17.5 26h29l-2.7 27.6a3.2 3.2 0 0 1-3.2 2.9H23.4a3.2 3.2 0 0 1-3.2-2.9z")
private val L_WAVE_BACK = p("M0 37q5-3.6 10 0t10 0t10 0t10 0t10 0t10 0t10 0v30H0z")
private val L_WAVE_FRONT = p("M-5 39q5-3.6 10 0t10 0t10 0t10 0t10 0t10 0t10 0v30H-5z")
private val L_SIG = listOf(
    p("M27.8 12.6a6 6 0 0 1 8.4 0"),
    p("M24.2 8.9a11 11 0 0 1 15.6 0"),
    p("M20.6 5.2a16 16 0 0 1 22.8 0"),
)
private val L_TICKS = p("M22 30.5h3.4M22 34h2.2")
private val L_PINS = p("M32 42v-3.2M32 51v2.5M27.5 46.5h-4M36.5 46.5h4")
private val L_LID = p("M13 22.5h38")
private val L_HANDLE = p("M27 22.5v-3.4h10v3.4")
private val SPRING = CubicBezierEasing(0.2f, 0.9f, 0.3f, 1.4f)
private val SOFT = CubicBezierEasing(0.2f, 0.8f, 0.2f, 1f)

/**
 * The logo at [size]. Each new value of [play] plays the build-up again, as a
 * tap on it does; 0 shows the still frame at once.
 */
@Composable
fun SmartBinLogo(size: Dp, play: Int = 0, modifier: Modifier = Modifier) {
    val c = Sb.c
    val ok = c.ok
    val teal = c.cal
    val ink = c.ink
    val onBrand = if (c.isDark) Color(0xFF13100D) else Color.White

    // 0 to 1 for each part of the build-up. The still frame is every part at 1.
    val lid = remember { Animatable(1f) }
    val fill = remember { Animatable(1f) }
    val chip = remember { Animatable(1f) }
    val sig = remember { Animatable(1f) }
    LaunchedEffect(play) {
        if (play == 0) return@LaunchedEffect
        lid.snapTo(0f); fill.snapTo(0f); chip.snapTo(0f); sig.snapTo(0f)
```

```
        coroutineScope {
            launch { lid.animateTo(1f, tween(1100)) }
            launch { kotlinx.coroutines.delay(250); fill.animateTo(1f, tween(1500, easing = SOFT)) }
            launch { kotlinx.coroutines.delay(1000); chip.animateTo(1f, tween(600, easing = SPRING)) }
            launch { kotlinx.coroutines.delay(1300); sig.animateTo(1f, tween(800)) }
        }
    }
    val t = rememberInfiniteTransition(label = "logo")
    val wave by t.animateFloat(0f, -10f, infiniteRepeatable(tween(3200, easing = LinearEasing)), label = "wave")
    val wave2 by t.animateFloat(-10f, 0f, infiniteRepeatable(tween(4600, easing = LinearEasing)), label = "wave2")
    val ping by t.animateFloat(0f, 1f, infiniteRepeatable(tween(3600, easing = LinearEasing)), label = "ping")
    val led by t.animateFloat(0.35f, 1f, infiniteRepeatable(tween(1200), RepeatMode.Reverse), label = "led")

    Canvas(modifier.size(size)) {
        scale(this.size.width / 64f, pivot = Offset.Zero) {
            drawLogo(
                ink, ok, teal, onBrand,
                lid = lid.value, fill = fill.value, chip = chip.value, sig = sig.value,
                wave = wave, wave2 = wave2, ping = ping, led = led,
            )
        }
    }
}

private fun DrawScope.drawLogo(
    ink: Color, ok: Color, teal: Color, onBrand: Color,
    lid: Float, fill: Float, chip: Float, sig: Float,
    wave: Float, wave2: Float, ping: Float, led: Float,
) {
    // The glow behind the signal grows in with it.
    if (sig > 0f) {
        drawCircle(
            Brush.radialGradient(listOf(ok.copy(alpha = 0.45f * sig), ok.copy(alpha = 0f)), center = Offset(32f, 12f), radius = 12f * (0.3f + 0.7f * sig)),
            radius = 12f, center = Offset(32f, 12f),
        )
    }
    // Three arcs go out one after another, then pulse gently.
    val base = floatArrayOf(1f, 0.7f, 0.4f)
    L_SIG.forEachIndexed { i, path ->
        val appear = ((sig * 3f) - i).coerceIn(0f, 1f)
        val phase = (ping - i * 0.07f + 1f) % 1f
        val pulse = 1f - 0.65f * kotlin.math.sin(phase * Math.PI.toFloat()).let { it * it } * 0.6f
        if (appear > 0f) {
            translate(0f, 3f * (1f - appear)) {
                drawPath(path, ok, alpha = base[i] * appear * pulse, style = Stroke(2.6f, cap = StrokeCap.Round))
            }
        }
    }
    // A soft glass tint inside the bin.
    drawPath(
        L_BODY,
        Brush.horizontalGradient(listOf(ok.copy(alpha = 0.05f), ok.copy(alpha = 0.16f), ok.copy(alpha = 0.05f)), startX = 17.5f, endX = 46.5f),
    )
    // The fill rises from the bottom, its two waves drifting the whole time.
    val gradient = Brush.verticalGradient(listOf(ok, teal), startY = 34f, endY = 57f)
    clipPath(L_BODY) {
        translate(wave2, 17f * (1f - fill)) { drawPath(L_WAVE_BACK, gradient, alpha = 0.35f) }
        translate(wave, 17f * (1f - fill)) { drawPath(L_WAVE_FRONT, gradient) }
    }
    drawPath(L_TICKS, ok, alpha = 0.8f * chip, style = Stroke(1.8f, cap = StrokeCap.Round))
    // The controller chip pops in.
    if (chip > 0f) {
        scale(chip, pivot = Offset(32f, 46.5f)) {
            val a = chip.coerceIn(0f, 1f)
            drawRoundRect(onBrand, topLeft = Offset(27.5f, 42f), size = androidx.compose.ui.geometry.Size(9f, 9f),
                cornerRadius = androidx.compose.ui.geometry.CornerRadius(1.8f), alpha = a, style = Stroke(2f))
            drawPath(L_PINS, onBrand, alpha = a, style = Stroke(1.8f, cap = StrokeCap.Round))
            listOf(Offset(32f, 37.6f), Offset(22.6f, 46.5f), Offset(41.4f, 46.5f)).forEach { drawCircle(onBrand, 1.5f, it, alpha = a) }
        }
    }
    drawPath(L_BODY, ink, style = Stroke(3f, join = StrokeJoin.Round))
    // The lid lifts on its hinge at the left and settles back.
    val swing = when {
        lid < 0.35f -> lid / 0.35f
        lid < 0.7f -> 1f - (lid - 0.35f) / 0.35f * 0.62f
        else -> 0.38f * (1f - (lid - 0.7f) / 0.3f)
    }
    translate(0f, -3f * swing) {
        rotate(-16f * swing, pivot = Offset(13f, 22.5f)) {
            drawPath(L_LID, ink, style = Stroke(3.4f, cap = StrokeCap.Round))
            drawPath(L_HANDLE, ink, style = Stroke(3f, cap = StrokeCap.Round, join = StrokeJoin.Round))
            drawCircle(ok, 1.3f, Offset(45.5f, 22.5f), alpha = led)
        }
```

```
    }
}
```

Android App/app/src/main/java/smartbin/ui/Kit.kt

```
// ================================================================================
// ui/Kit.kt - shared building blocks for every page
// Text (T, Hint, Sh), keys (Btn, IconKey, Toggle, Stepper), inputs, banners,
// cards, sheets and the toast. Each one names the mockup CSS class it copies
// (.hint, .tgl, .sheet ...). Sizes use em() on a 14 sp base, like the mockup.
// ================================================================================
package smartbin.ui

import androidx.activity.compose.BackHandler
import androidx.compose.animation.core.Animatable
import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.tween
import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.BoxScope
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.ColumnScope
import androidx.compose.foundation.layout.PaddingValues
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.RowScope
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.defaultMinSize
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.heightIn
import androidx.compose.foundation.layout.imePadding
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.offset
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.BasicTextField
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.LocalContentColor
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.CompositionLocalProvider
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableFloatStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.alpha
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.drawBehind
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.SolidColor
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.platform.LocalView
import androidx.compose.ui.text.AnnotatedString
import androidx.compose.ui.text.SpanStyle
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.buildAnnotatedString
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.text.withStyle
import androidx.compose.ui.unit.Dp
```

```
import androidx.compose.ui.unit.IntOffset
import androidx.compose.ui.unit.TextUnit
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.zIndex
import smartbin.notify.Feel
import smartbin.notify.Sfx
import smartbin.notify.Sounds

/** The mockup sizes text in em on a 14 px base. */
fun em(x: Float): TextUnit = (14f * x).sp

/** Text in the mockup's type scale. **bold** marks the parts .hint b makes darker. */
@Composable
fun T(
    text: String,
    modifier: Modifier = Modifier,
    size: Float = 1f,
    weight: FontWeight = FontWeight.ExtraBold,
    color: Color = LocalContentColor.current,
    align: TextAlign? = null,
    maxLines: Int = Int.MAX_VALUE,
    tnum: Boolean = false,
    spacing: Float = 0f,
    lineHeight: Float = 1.35f,
    bold: Color? = null,
    /** Steps the size down until it fits one line, the mockup's container-query labels. */
    fit: Boolean = false,
) {
    var scale by remember(text.length, size) { mutableFloatStateOf(1f) }
    val shown = if (fit) size * scale else size
    val body: AnnotatedString = if ("**" in text) marked(text, bold ?: color) else AnnotatedString(text)
    Text(
        text = body,
        modifier = modifier,
        color = color,
        textAlign = align,
        maxLines = if (fit) 1 else maxLines,
        softWrap = !fit && maxLines != 1,
        overflow = TextOverflow.Ellipsis,
        onTextLayout = { if (fit && it.didOverflowWidth && scale > 0.6f) scale = (scale * 0.92f).coerceAtLeast(0.6f) },
        style = TextStyle(
            fontFamily = Inter,
            fontSize = em(shown),
            fontWeight = weight,
            lineHeight = em(shown * lineHeight),
            letterSpacing = (spacing * 14f * shown).sp,
            fontFeatureSettings = if (tnum) "tnum" else null,
        ),
    )
}

private fun marked(text: String, bold: Color): AnnotatedString = buildAnnotatedString {
    text.split("**").forEachIndexed { i, part ->
        if (i % 2 == 1) withStyle(SpanStyle(color = bold, fontWeight = FontWeight.ExtraBold)) { append(part) } else
append(part)
    }
}

/** .hint */
@Composable
fun Hint(text: String, modifier: Modifier = Modifier, align: TextAlign? = null) {
    val c = Sb.c
    T(text, modifier, size = 0.8f, weight = FontWeight.Medium, color = c.ink2, bold = c.ink, align = align, lineHeight =
1.45f)
}

/** .sh, the small uppercase section heading. */
@Composable
fun Sh(icon: ImageVector, text: String, cc: Color = Sb.c.ink2, modifier: Modifier = Modifier) {
    Row(modifier, verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        Ico(icon, 16.dp, cc)
        T(text.uppercase(), size = 0.76f, spacing = 0.1f, color = cc, maxLines = 1)
    }
}

/** A press with no ripple and the key's own sound and tick. */
@Composable
fun Modifier.press(enabled: Boolean = true, sound: Sfx = Sfx.TAP, src: MutableInteractionSource? = null, onClick: () ->
Unit): Modifier {
    val view = LocalView.current
    return clickable(interactionSource = src ?: remember { MutableInteractionSource() }, indication = null, enabled =
enabled) {
        Feel.play(view, sound)
        onClick()
    }
```

```
}

/** Swallows taps, so a sheet does not pass them to the page behind it. */
@Composable
fun Modifier.swallow(): Modifier = clickable(interactionSource = remember { MutableInteractionSource() }, indication = null)
{}

enum class Look { PLAIN, TINT, SOLID, ACCENT, DANGER }

/**
 * .btn and its kinds. PLAIN is a raised key on the page colour; TINT adds a
 * ring and colours the label; SOLID and ACCENT fill it; DANGER is the brutal
 * red key. Pressed, a key sinks into the page. [stack] puts the icon above
 * the label, as on the bin card's three action keys.
 */
@Composable
fun Btn(
    text: String?,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    icon: ImageVector? = null,
    look: Look = Look.PLAIN,
    cc: Color = Sb.c.ink2,
    small: Boolean = false,
    enabled: Boolean = true,
    stack: Boolean = false,
    sound: Sfx = Sfx.TAP,
    /** For a key that stays pushed in, such as a picked bin. */
    pressedIn: Boolean = false,
    iconMod: Modifier = Modifier,
    textSize: Float? = null,
) {
    val c = Sb.c
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    val down = (pressed && enabled) || pressedIn
    val radius = if (look == Look.DANGER) 7.dp else if (small) 11.dp else 13.dp
    val shape = RoundedCornerShape(radius)
    val kind = if (enabled) look else Look.PLAIN
    val fg = when {
        !enabled -> c.ink2
        kind == Look.SOLID || kind == Look.ACCENT -> c.onAccent
        kind == Look.DANGER -> Color.White
        kind == Look.TINT || pressedIn -> cc
        else -> c.ink
    }
    val iconTint = when {
        !enabled -> c.ink2
        kind == Look.PLAIN && !pressedIn -> cc.takeIf { it != c.ink2 } ?: c.ink
        else -> fg
    }
    val fill = when (kind) {
        Look.SOLID -> cc
        Look.ACCENT -> c.accent
        Look.DANGER -> c.bad
        else -> c.base
    }
    val ring = if (enabled && (kind == Look.TINT || pressedIn)) cc.soft(0.3f) else null
    val surface = if (kind == Look.DANGER) {
        Modifier
            .offset { if (down) IntOffset(3.dp.roundToPx(), 3.dp.roundToPx()) else IntOffset.Zero }
            .brutal(c, radius, fill, if (down) 1.dp else 4.dp)
    } else if (down) {
        Modifier
            .offset { IntOffset(0, if (pressed) 1.dp.roundToPx() else 0) }
            .then(if (ring != null) Modifier.ring(ring, radius) else Modifier)
            .clip(shape).background(fill).sunk(c, radius)
    } else {
        Modifier.raised(c, radius, Out.O1, ring).clip(shape).background(fill)
    }
    val minH = if (stack) 56.dp else if (small) 38.dp else 44.dp
    val size = textSize ?: if (stack) 0.76f else if (small) 0.8f else 0.88f
    val content: @Composable () -> Unit = {
        CompositionLocalProvider(LocalContentColor provides fg) {
            if (icon != null) Ico(icon, if (stack) 22.dp else if (small) 15.dp else 18.dp, iconTint, iconMod)
            if (text != null) T(text, size = size, weight = FontWeight.Bold, spacing = 0.02f, maxLines = if (stack) 2 else 1,
align = TextAlign.Center, lineHeight = 1.1f)
        }
    }
    val base = modifier
        .alpha(if (enabled) 1f else 0.4f)
        .then(surface)
        .press(enabled, sound, src, onClick)
        .defaultMinSize(minHeight = minH)
    if (stack) {
        Column(
```

```
            base.padding(horizontal = 4.dp, vertical = 6.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(3.dp, Alignment.CenterVertically),
        ) { content() }
    } else {
        Row(
            base.padding(horizontal = if (small) 12.dp else 14.dp, vertical = if (small) 5.dp else 8.dp),
            horizontalArrangement = Arrangement.spacedBy(7.dp, Alignment.CenterHorizontally),
            verticalAlignment = Alignment.CenterVertically,
        ) { content() }
    }
}

/** A square key with one icon: .iconbtn, .cardbtn, .gearbtn. */
@Composable
fun IconKey(
    icon: ImageVector,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,
    tint: Color = Sb.c.accent,
    size: Dp = 44.dp,
    iconSize: Dp = 21.dp,
    radius: Dp = 13.dp,
    enabled: Boolean = true,
    sound: Sfx = Sfx.TAP,
    label: String? = null,
    iconMod: Modifier = Modifier,
) {
    val c = Sb.c
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    val shape = RoundedCornerShape(radius)
    Column(
        modifier
            .size(size)
            .alpha(if (enabled) 1f else 0.35f)
            .then(if (pressed) Modifier.clip(shape).background(c.base).sunk(c, radius) else Modifier.bump(c, radius))
            .press(enabled, sound, src, onClick),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center,
    ) {
        val t = if (pressed && label != null) c.accent else tint
        Ico(icon, iconSize, t, iconMod)
        if (label != null) T(label, size = 8f / 14f, spacing = 0.05f, color = t, maxLines = 1)
    }
}

/** .tag, the small white-on-colour label. */
@Composable
fun Tag(text: String, bg: Color, modifier: Modifier = Modifier, edge: Boolean = true) {
    val c = Sb.c
    val shape = RoundedCornerShape(6.dp)
    Box(
        modifier
            .clip(shape)
            .background(bg)
            .then(if (edge) Modifier.drawBehind { drawRoundRect(c.ink, cornerRadius = CornerRadius(6.dp.toPx()), style =
androidx.compose.ui.graphics.drawscope.Stroke(2.dp.toPx())) } else Modifier)
            .padding(horizontal = 9.dp, vertical = 4.dp),
    ) { T(text, size = 0.7f, spacing = 0.05f, color = Color.White, maxLines = 1) }
}

/** .tgl, the switch. */
@Composable
fun Toggle(on: Boolean, onChange: (Boolean) -> Unit, enabled: Boolean = true) {
    val c = Sb.c
    val x by animateFloatAsState(if (on) 1f else 0f, tween(220), label = "tgl")
    Box(
        Modifier
            .size(54.dp, 32.dp)
            .alpha(if (enabled) 1f else 0.42f)
            .clip(RoundedCornerShape(16.dp))
            .background(if (on) c.accent else c.well)
            .sunk(c, 16.dp)
            .press(enabled, if (on) Sfx.SWITCH_OFF else Sfx.SWITCH_ON) { onChange(!on) },
    ) {
        Box(
            Modifier
                .padding(4.dp)
                .offset { IntOffset((22.dp.toPx() * x).toInt(), 0) }
                .size(24.dp)
                .bump(c, 12.dp),
        )
    }
}

```

```
/** .twoway: a sunk tray of choices. Use [Tw] inside. */
@Composable
fun TwoWay(modifier: Modifier = Modifier, full: Boolean = false, content: @Composable RowScope.() -> Unit) {
    val c = Sb.c
    Row(
        (if (full) modifier.fillMaxWidth() else modifier).well(c, 13.dp).padding(5.dp),
        horizontalArrangement = Arrangement.spacedBy(5.dp),
        verticalAlignment = Alignment.CenterVertically,
        content = content,
    )
}

/** .tw, one choice in a [TwoWay]. */
@Composable
fun RowScope.Tw(
    on: Boolean,
    onClick: () -> Unit,
    label: String? = null,
    icon: ImageVector? = null,
    cc: Color = Sb.c.accent,
    enabled: Boolean = true,
    grow: Float? = 1f,
    stack: Boolean = false,
    minHeight: Dp = 38.dp,
    textSize: Float = 0.78f,
) {
    val c = Sb.c
    val shape = RoundedCornerShape(9.dp)
    val fg = if (on) c.onAccent else c.ink2
    val m = Modifier
        .then(if (grow != null) Modifier.weight(grow) else Modifier)
        .defaultMinSize(minHeight = minHeight)
        .alpha(if (enabled) 1f else 0.42f)
        .then(if (on) Modifier.bump(c, 9.dp, fill = cc) else Modifier.clip(shape))
        .press(enabled && !on, Sfx.CHOICE, onClick = onClick)
        .padding(horizontal = if (stack) 2.dp else 9.dp, vertical = if (stack) 5.dp else 0.dp)
    val inner: @Composable () -> Unit = {
        if (icon != null) Ico(icon, if (stack) 26.dp else 17.dp, fg)
        if (label != null) T(label, size = textSize, weight = FontWeight.Bold, color = fg, fit = true, align =
TextAlign.Center)
    }
    if (stack) Column(m, horizontalAlignment = Alignment.CenterHorizontally, verticalArrangement = Arrangement.spacedBy(2.dp,
Alignment.CenterVertically)) { inner() }
    else Row(m, horizontalArrangement = Arrangement.spacedBy(6.dp, Alignment.CenterHorizontally), verticalAlignment =
Alignment.CenterVertically) { inner() }
}

/** .chip on the bin card. */
@Composable
fun Chip(icon: ImageVector, text: String, on: Boolean, cc: Color, modifier: Modifier = Modifier, iconMod: Modifier =
Modifier) {
    val c = Sb.c
    Row(
        modifier
            .height(30.dp)
            .alpha(if (on) 1f else 0.38f)
            .bump(c, 9.dp, ring = if (on) cc.soft(0.3f) else null)
            .padding(horizontal = 6.dp),
        horizontalArrangement = Arrangement.spacedBy(5.dp, Alignment.CenterHorizontally),
        verticalAlignment = Alignment.CenterVertically,
    ) {
        val t = if (on) cc else c.ink2
        Ico(icon, 13.dp, t, iconMod)
        T(text, size = 0.72f, spacing = 0.02f, color = t, maxLines = 1)
    }
}

/** .field: a label line over its control. */
@Composable
fun Field(label: String, icon: ImageVector? = null, cc: Color = Sb.c.ink2, content: @Composable ColumnScope.() -> Unit) {
    Column(verticalArrangement = Arrangement.spacedBy(7.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(6.dp)) {
            if (icon != null) Ico(icon, 15.dp, cc)
            T(label, size = 0.76f, spacing = 0.03f, color = cc)
        }
        content()
    }
}

/** .well with − and + keys: the stepper. */
@Composable
fun Stepper(value: String, unit: String, onStep: (Int) -> Unit, modifier: Modifier = Modifier, enabled: Boolean = true, cc:
Color = Sb.c.accent) {
    val c = Sb.c
    Row(
        modifier.fillMaxWidth().well(c, 13.dp).padding(4.dp),
```

```
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(6.dp),
    ) {
        StepKey("−", enabled, Sfx.STEP_DOWN, cc) { onStep(-1) }
        T(value, Modifier.weight(1f), size = 1.25f, tnum = true, align = TextAlign.Center, maxLines = 1)
        T(unit, Modifier.padding(end = 2.dp), size = 0.76f, color = c.ink2, maxLines = 1)
        StepKey("+", enabled, Sfx.STEP_UP, cc) { onStep(1) }
    }
}

@Composable
private fun StepKey(label: String, enabled: Boolean, sound: Sfx, cc: Color, onClick: () -> Unit) {
    val c = Sb.c
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    Box(
        Modifier
            .size(44.dp)
            .then(if (pressed) Modifier.clip(RoundedCornerShape(11.dp)).background(c.base).sunk(c, 11.dp) else
Modifier.bump(c, 11.dp))
            .press(enabled, sound, src, onClick),
        contentAlignment = Alignment.Center,
    ) { T(label, size = 1.3f, color = if (pressed) cc else c.ink2, lineHeight = 1f) }
}

/** .readout, a value with no control. */
@Composable
fun Readout(text: String) {
    val c = Sb.c
    Box(Modifier.fillMaxWidth().well(c, 12.dp).padding(horizontal = 14.dp, vertical = 11.dp)) { T(text, tnum = true) }
}

/** .inp, a text field sunk into the page. */
@Composable
fun Inp(
    value: String,
    onChange: (String) -> Unit,
    modifier: Modifier = Modifier,
    placeholder: String = "",
    password: Boolean = false,
    keyboard: KeyboardType = KeyboardType.Text,
    enabled: Boolean = true,
) {
    val c = Sb.c
    val style = TextStyle(fontFamily = Inter, fontSize = em(1.02f), fontWeight = FontWeight.Bold, color = c.ink)
    BasicTextField(
        value = value,
        onValueChange = onChange,
        enabled = enabled,
        singleLine = true,
        textStyle = style,
        cursorBrush = SolidColor(c.accent),
        keyboardOptions = KeyboardOptions(keyboardType = if (password) KeyboardType.Password else keyboard,
autoCorrectEnabled = false),
        visualTransformation = if (password) PasswordVisualTransformation() else VisualTransformation.None,
        modifier = modifier
            .fillMaxWidth()
            .heightIn(min = 50.dp)
            .alpha(if (enabled) 1f else 0.5f)
            .well(c, 12.dp),
        decorationBox = { inner ->
            Box(Modifier.padding(horizontal = 13.dp, vertical = 14.dp), contentAlignment = Alignment.CenterStart) {
                if (value.isEmpty() && placeholder.isNotEmpty()) T(placeholder, size = 1.02f, weight = FontWeight.Bold, color
= c.ink2.soft(0.7f), maxLines = 1)
                inner()
            }
        },
    )
}

enum class Tone { RED, PURPLE, TEAL, GREEN, AMBER, ORANGE, BLUE }

/** .banner: the brutal message block. */
@Composable
fun Banner(tone: Tone, icon: ImageVector, title: String, small: String? = null, modifier: Modifier = Modifier, iconMod:
Modifier = Modifier) {
    val c = Sb.c
    val bg = when (tone) {
        Tone.RED -> c.bad; Tone.PURPLE -> c.sim; Tone.TEAL -> c.cal; Tone.GREEN -> c.ok; Tone.AMBER -> c.warn; Tone.ORANGE ->
c.cloud; Tone.BLUE -> c.ts
    }
    val fg = if (c.isDark || tone == Tone.AMBER) (if (c.isDark) Color(0xFF13100D) else Color(0xFF241C10)) else Color.White
    Row(
        modifier.fillMaxWidth().brutal(c, 8.dp, bg).padding(horizontal = 13.dp, vertical = 11.dp),
        horizontalArrangement = Arrangement.spacedBy(10.dp),
    ) {
```

```
        Ico(icon, 20.dp, fg, iconMod)
        Column(Modifier.weight(1f)) {
            T(title, size = 0.88f, weight = FontWeight.Bold, color = fg)
            if (!small.isNullOrBlank()) T(small, Modifier.padding(top = 2.dp).alpha(0.92f), size = 0.79f, weight = FontWeight.SemiBold, color = fg)
        }
    }
}

/** .pcard.nm: the plain raised card. */
@Composable
fun Card(modifier: Modifier = Modifier, padding: Dp = 14.dp, gap: Dp = 12.dp, content: @Composable ColumnScope.() -> Unit) {
    val c = Sb.c
    Column(modifier.fillMaxWidth().nm(c).padding(padding), verticalArrangement = Arrangement.spacedBy(gap), content = content)
}

/** .lrow: a row of the App section. */
@Composable
fun LRow(
    icon: ImageVector,
    title: String,
    sub: String? = null,
    cc: Color = Sb.c.ink2,
    onClick: (() -> Unit)? = null,
    chevron: Boolean = onClick != null,
    sound: Sfx = Sfx.OPEN,
    iconMod: Modifier = Modifier,
    trailing: @Composable (RowScope.() -> Unit)? = null,
) {
    val c = Sb.c
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    Row(
        Modifier
            .fillMaxWidth()
            .defaultMinSize(minHeight = 58.dp)
            .then(if (pressed) Modifier.clip(RoundedCornerShape(14.dp)).background(c.base).sunk(c, 14.dp) else Modifier.bump(c, 14.dp))
            .then(if (onClick != null) Modifier.press(true, sound, src, onClick) else Modifier)
            .padding(horizontal = 13.dp, vertical = 9.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(12.dp),
    ) {
        Ico(icon, 21.dp, cc, iconMod)
        Column(Modifier.weight(1f)) {
            T(title, weight = FontWeight.Bold)
            if (sub != null) T(sub, Modifier.padding(top = 1.dp), size = 0.8f, weight = FontWeight.SemiBold, color = c.ink2)
        }
        trailing?.invoke(this)
        if (chevron) Ico(Ic.chev, 18.dp, c.ink2, Modifier.graphicsLayer { rotationZ = -90f })
    }
}

/** .opt: one line of a pick list, with its radio. */
@Composable
fun Opt(on: Boolean, onClick: () -> Unit, modifier: Modifier = Modifier, content: @Composable RowScope.() -> Unit) {
    val c = Sb.c
    Row(
        modifier
            .fillMaxWidth()
            .defaultMinSize(minHeight = 50.dp)
            .then(if (on) Modifier.clip(RoundedCornerShape(13.dp)).background(c.base).sunk(c, 13.dp) else Modifier.bump(c, 13.dp))
            .press(true, Sfx.CHOICE, onClick = onClick)
            .padding(horizontal = 12.dp, vertical = 6.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(12.dp),
    ) {
        Box(Modifier.size(22.dp).well(c, 11.dp), contentAlignment = Alignment.Center) {
            if (on) Box(Modifier.size(10.dp).clip(CircleShape).background(c.accent))
        }
        CompositionLocalProvider(LocalContentColor provides if (on) c.accent else c.ink) { content() }
    }
}

/** .scrim and .sheet: slides up from the bottom; a tap outside or Back closes it. */
@Composable
fun Sheet(onDismiss: () -> Unit, content: @Composable ColumnScope.() -> Unit) {
    val c = Sb.c
    val slide = remember { Animatable(1f) }
    val fade = remember { Animatable(0f) }
    LaunchedEffect(Unit) { fade.animateTo(1f, tween(180)) }
    LaunchedEffect(Unit) { slide.animateTo(0f, tween(240)) }
    LaunchedEffect(Unit) { Sounds.playIfQuiet(Sfx.OPEN) }
    val view = LocalView.current
```

```
    BackHandler { Feel.play(view, Sfx.BACK); onDismiss() }
    androidx.compose.foundation.layout.BoxWithConstraints(Modifier.fillMaxSize().zIndex(40f)) {
        val maxH = maxHeight * 0.82f
        Box(
            Modifier
                .fillMaxSize()
                .alpha(fade.value)
                .background(Color(0x800A0806))
                .press(true, Sfx.BACK, onClick = onDismiss),
        )
        val shape = RoundedCornerShape(topStart = 24.dp, topEnd = 24.dp)
        Column(
            Modifier
                .align(Alignment.BottomCenter)
                .widthIn(max = 520.dp)
                .fillMaxWidth()
                .heightIn(max = maxH)
                .graphicsLayer { translationY = slide.value * size.height }
                .drawBehind {
                    drawRoundRect(Color(0x47000000), topLeft = Offset(0f, -6.dp.toPx()), size = size, cornerRadius =
CornerRadius(24.dp.toPx()))
                }
                .clip(shape)
                .background(c.base)
                .swallow()
                .verticalScroll(rememberScrollState())
                .navigationBarsPadding()
                .imePadding()
                .padding(start = 16.dp, end = 16.dp, top = 8.dp, bottom = 26.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            Box(Modifier.align(Alignment.CenterHorizontally).size(42.dp,
5.dp).clip(CircleShape).background(c.ink2.soft(0.4f)))
            CompositionLocalProvider(LocalContentColor provides c.ink) { content() }
        }
    }
}

@Composable
fun SheetTitle(icon: ImageVector?, text: String, center: Boolean = false, trailing: (@Composable RowScope.() -> Unit)? =
null) {
    val c = Sb.c
    Row(
        if (center) Modifier.fillMaxWidth() else Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(8.dp, if (center) Alignment.CenterHorizontally else Alignment.Start),
    ) {
        if (icon != null) Ico(icon, 21.dp, c.accent)
        T(text, size = 1.1f, align = if (center) TextAlign.Center else null)
        if (trailing != null) {
            Spacer(Modifier.weight(1f))
            trailing()
        }
    }
}

@Composable
fun SheetText(text: String, center: Boolean = false) {
    T(text, if (center) Modifier.fillMaxWidth() else Modifier, size = 0.9f, weight = FontWeight.Medium, align = if (center)
TextAlign.Center else null)
}

/** .acts: two keys sharing a row. */
@Composable
fun Acts(content: @Composable RowScope.() -> Unit) {
    Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(10.dp), content = content)
}

/** .toast: an amber banner that rises over the bottom of the page. */
@Composable
fun BoxScope.Toast(message: String, bottom: Dp) {
    val rise = remember(message) { Animatable(0f) }
    LaunchedEffect(message) { rise.animateTo(1f, tween(250)) }
    Box(
        Modifier
            .align(Alignment.BottomCenter)
            .zIndex(60f)
            .navigationBarsPadding()
            .padding(start = 14.dp, end = 14.dp, bottom = bottom)
            .widthIn(max = 520.dp)
            .graphicsLayer {
                alpha = rise.value
                translationY = (1f - rise.value) * 8.dp.toPx()
            },
    ) { Banner(Tone.AMBER, Ic.bell, message) }
}
```

```

/** .sk: a breathing block while the first reading comes in. */
@Composable
fun Skeleton(height: Dp, modifier: Modifier = Modifier, radius: Dp = 18.dp) {
    val c = Sb.c
    val t = rememberInfiniteTransition(label = "sk")
    val a by t.animateFloat(0.55f, 1f, infiniteRepeatable(tween(600), RepeatMode.Reverse), label = "sk")
    Box(modifier.fillMaxWidth().height(height).alpha(a).well(c, radius))
}

/** The page's scrolling column, .scroll. */
@Composable
fun ScrollPage(modifier: Modifier = Modifier, state: androidx.compose.foundation.ScrollState = rememberScrollState(),
bottomExtra: Dp = 0.dp, content: @Composable ColumnScope.() -> Unit) {
    Column(
        modifier
            .fillMaxSize()
            .verticalScroll(state)
            .padding(PaddingValues(start = 14.dp, end = 14.dp, top = 16.dp, bottom = 20.dp + bottomExtra)),
        verticalArrangement = Arrangement.spacedBy(18.dp),
        content = content,
    )
}

/** .backbar: the back key and a title, for the pages under Settings. */
@Composable
fun BackBar(title: String, onBack: () -> Unit, trailing: @Composable RowScope.() -> Unit = {}) {
    Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(10.dp)) {
        IconKey(Ic.back, onBack, sound = Sfx.BACK)
        T(title, Modifier.weight(1f), size = 1.12f)
        trailing()
    }
}

/** .btstate: the small status pill in a back bar. */
@Composable
fun StatePill(icon: ImageVector, text: String, cc: Color, on: Boolean) {
    val c = Sb.c
    Row(
        Modifier.height(34.dp).bump(c, 17.dp, ring = if (on) cc.soft(0.32f) else null).padding(horizontal = 12.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(7.dp),
    ) {
        val t = if (on) cc else c.ink2
        Ico(icon, 18.dp, t)
        T(text.uppercase(), size = 0.76f, spacing = 0.03f, color = t, maxLines = 1)
    }
}

/** A swatch for chart legends: solid, or dashed like the line it stands for. */
@Composable
fun Swatch(color: Color, dash: Int = 0) {
    androidx.compose.foundation.Canvas(Modifier.size(16.dp, 4.dp)) {
        val h = size.height
        if (dash == 0) {
            drawRoundRect(color, cornerRadius = CornerRadius(h / 2))
        } else {
            val on = (if (dash == 1) 5f else 2.5f) * density
            val gap = (if (dash == 1) 4f else 4f) * density
            var x = 0f
            while (x < size.width) {
                drawRect(color, topLeft = Offset(x, 0f), size = androidx.compose.ui.geometry.Size(minOf(on, size.width - x),
h))
                x += on + gap
            }
        }
    }
}

@Composable
fun Gap(h: Dp) = Spacer(Modifier.height(h))

@Composable
fun RowScope.Grow() = Spacer(Modifier.weight(1f))

@Composable
fun HGap(w: Dp) = Spacer(Modifier.width(w))
```

Android App/app/src/main/java/smartbin/ui/AppViewModel.kt

```
// ============================================================================
// ui/AppViewModel.kt - screen state and user actions
// Holds what the screens need beyond Hub.state: the current page, role, PIN
// entry, sheet contents, chart choices, the calibration edit copy and the
// Bluetooth setup steps. Each button calls one function here; the function
```

```
// calls Hub (or BinBle) and reports back with say(), which shows the toast.
// Debug tip: almost every failure reaches the user as a toast string built in
// this file, so searching for the toast text finds the code path.
// ============================================================================
package smartbin.ui

import android.app.Application
import android.content.ContentValues
import android.content.Intent
import android.net.Uri
import android.os.Build
import android.os.PowerManager
import android.provider.Settings
import android.provider.MediaStore
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableLongStateOf
import androidx.compose.runtime.mutableStateMapOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import org.json.JSONArray
import org.json.JSONObject
import smartbin.SmartBinApp
import smartbin.data.AlertKind
import smartbin.data.AlertRow
import smartbin.data.Bin
import smartbin.data.BinBle
import smartbin.data.Cal
import smartbin.data.Cloud
import smartbin.data.DEFAULT_SIM
import smartbin.data.FirebaseConfig
import smartbin.data.Demo
import smartbin.data.Hub
import smartbin.data.OperatorLock
import smartbin.data.HubEvent
import smartbin.data.Routes
import smartbin.data.Role
import smartbin.data.Source
import smartbin.data.v
import smartbin.notify.Alerts
import smartbin.notify.Feel
import smartbin.notify.Haptics
import smartbin.notify.Sfx
import java.util.Calendar

/** The five tabs, then the pages opened from the gear key: app settings, Bluetooth setup and Connection. */
enum class Screen { OVERVIEW, CHARTS, SETTINGS, SIM, ALERTS, APP, BT, NET }

enum class BinView { AUTO, GRID, CARDS }
enum class BinFilter { ALL, ACTION, MODES }

/** Which list a bin picker sheet chooses for. */
enum class PickFor(val title: String) { CHART("Fill level chart"), ENV("Environment chart"), SET("Settings"),
SIM("Simulation") }

enum class EnvMode { ALL, TEMP, HUMID, GAS, PRESS }

enum class AlertRange(val label: String) { TODAY("Today"), WEEK("7 days"), MONTH("30 days"), ANY("Any") }

enum class Perm { ON, OFF, BLOCKED }

sealed interface Overlay {
    data object Attention : Overlay
    data class BinEdit(val id: String) : Overlay
    data class BinDetail(val id: String) : Overlay
    data class Pick(val forWhat: PickFor) : Overlay
    data object AddBins : Overlay
    data object ClearAlerts : Overlay
    data object Wifi : Overlay
    data object NotifPre : Overlay
    data class PinUnlock(val first: Boolean) : Overlay
    data class PinSet(val first: Boolean, val change: Boolean) : Overlay
    data object PinChange : Overlay
    data class BtCode(val id: String) : Overlay
    /** Bins found over Bluetooth, to pick the one to set up. */
    data object BtDevices : Overlay
}

/**
 * What the screens show besides the bins themselves: the tab, the open sheet,
 * the role, the theme, the choices on each page and the edits not saved yet.
```

```
 * The bins, their readings and the alerts live in [Hub].
 */
/** A cloud command's answer: confirmed ([ok]) or timed out, and when. */
data class Settled(val cmd: String, val ok: Boolean, val at: Long)

/** How long a key flashes after its cloud command is answered. */
const val SETTLE_MILLIS = 1100L

// Survives screen rotation; recreated when the process restarts.
class AppViewModel(app: Application) : AndroidViewModel(app) {
    private val store = Hub.store()

    /** Cloud commands just confirmed or timed out, by bin, while their key flashes. */
    val settled = mutableStateMapOf<String, Settled>()

    init {
        // A cloud command's key unlocks when the bin confirms it, or when it waited too long.
        viewModelScope.launch {
            Hub.events.collect { e ->
                if (e is HubEvent.CommandResult) {
                    // The key flashes green or red for a moment, then lets go.
                    val s = Settled(e.cmd, e.ok, System.currentTimeMillis())
                    settled[e.binId] = s
                    launch {
                        delay(SETTLE_MILLIS)
                        if (settled[e.binId] == s) settled.remove(e.binId)
                    }
                    if (e.ok) {
                        Feel.event(Sfx.CMD_OK)
                        say("${e.name} confirmed ${e.cmd}.")
                    } else {
                        Feel.event(Sfx.CMD_FAIL)
                        say(e.why?.let { "${e.name} refused ${e.cmd}: $it." } ?: "${e.name} did not confirm ${e.cmd}. Check
the bin, then try again.")
                    }
                }
            }
        }
    }

    // ---------- whole app ----------

    // Every start opens on the start page with no role picked. Operator is never
    // remembered, and a return after a while away drops it to collector.
    var role by mutableStateOf<Role?>(null)
        private set
    val lock = OperatorLock(store)
    val operator get() = role != Role.COLLECTOR
    var screen by mutableStateOf(Screen.OVERVIEW)
    /** The tab the gear key was pressed on, so back returns there. */
    private var beforeApp = Screen.OVERVIEW
    /**
     * False on the start page. A new view model means the app was closed or
     * Android ended it, so the start page shows again. Coming back from the
     * background keeps this one, and the page the user left.
     */
    var started by mutableStateOf(false)

    /** Back to the start page, where demo mode is turned on or off. Operator needs the PIN again after. */
    fun toStart() {
        if (role == Role.OPERATOR) Feel.event(Sfx.OP_OUT)
        if (editing) viewModelScope.launch { Hub.setCalibration(null) }
        edit = null
        dirty = false
        overlay = null
        fullscreen = null
        screen = Screen.OVERVIEW
        role = null
        started = false
        syncInMain()
    }

    /** The DEMO key on the start page. */
    fun toggleDemo() {
        val on = !Hub.state.value.demoOn
        Hub.setDemo(on)
        demoSpeed = Demo.speed
        demoTick++
        Feel.event(if (on) Sfx.DEMO else Sfx.SWITCH_OFF)
    }

    /** Tells [Hub] whether the user is inside the main screens, which is when alerts may be posted. */
    fun syncInMain() {
        Hub.inMain = started && role != null
    }

```

```
    override fun onCleared() {
        // The app was closed for good (back out of it, or the activity finished).
        Hub.inMain = false
        ble.release()
        super.onCleared()
    }
    var overlay by mutableStateOf<Overlay?>(null)
    var theme by mutableStateOf(runCatching { ThemeMode.valueOf(store.string(SmartBinApp.KEY_THEME,
"AUTO")) }.getOrDefault(ThemeMode.AUTO))
        private set
    var toast by mutableStateOf<String?>(null)
        private set
    var toastSeq by mutableIntStateOf(0)
        private set
    private var toastJob: Job? = null

    fun say(message: String) {
        toast = message
        toastSeq++
        toastJob?.cancel()
        toastJob = viewModelScope.launch {
            delay(2200)
            toast = null
        }
    }

    /** Any touch on the screen clears the notice, so it never sits over a key. */
    fun hideToast() {
        if (toast == null) return
        toastJob?.cancel()
        toast = null
    }

    fun go(s: Screen) {
        overlay = null
        fullscreen = null
        if (s == Screen.APP && screen.ordinal < Screen.APP.ordinal) beforeApp = screen
        screen = if (s == Screen.SIM && !operator) Screen.OVERVIEW else s
    }

    /** Back from a page: Bluetooth and Connection go to the gear page, the gear page to its tab, a tab to Overview. */
    fun back() = when (screen) {
        Screen.BT, Screen.NET -> go(Screen.APP)
        Screen.APP -> go(beforeApp)
        else -> go(Screen.OVERVIEW)
    }

    fun toggleTheme(nowDark: Boolean) = setThemeMode(if (nowDark) ThemeMode.LIGHT else ThemeMode.DARK)

    fun setThemeMode(m: ThemeMode) {
        theme = m
        store.putString(SmartBinApp.KEY_THEME, m.name)
    }

    /** Asking for operator opens the PIN sheet. Collector needs nothing. */
    fun pickRole(r: Role, first: Boolean = false) {
        if (r == role && !first) return
        if (r == Role.OPERATOR) {
            opPin = ""; pinNew = ""; pinMsg = null
            overlay = if (lock.hasPin) Overlay.PinUnlock(first) else Overlay.PinSet(first, change = false)
            return
        }
        setRole(r, first)
    }

    private fun setRole(r: Role, first: Boolean) {
        val had = role != null
        if (role == Role.OPERATOR && r == Role.COLLECTOR) Feel.event(Sfx.OP_OUT)
        role = r
        started = true
        syncInMain()
        // Only "a role was picked" is kept; the next start is always collector.
        store.putString(SmartBinApp.KEY_ROLE, Role.COLLECTOR.name)
        if (r == Role.COLLECTOR) {
            if (editing) viewModelScope.launch { Hub.setCalibration(null) }
            edit = null
            dirty = false
            if (screen == Screen.SIM) screen = Screen.OVERVIEW
        }
        if (first || !had) {
            screen = Screen.OVERVIEW
            if (notifications != Perm.ON && !store.bool(SmartBinApp.KEY_ASKED, false)) overlay = Overlay.NotifPre
        } else {
            say("Signed in as ${if (r == Role.OPERATOR) "operator" else "collector"}.")
        }
    }
```

```
261
262     // ---------- operator PIN ----------
263
264     /** The digits typed so far, the first entry of a new PIN, and the line under the dots. */
265     var opPin by mutableStateOf("")
266     var pinNew by mutableStateOf("")
267     var pinMsg by mutableStateOf<Pair<Boolean, String>?>(null)
268     /** Set when the app went to the background, to drop operator after AUTO_LOCK. */
269     private var awaySince = 0L
270
271     fun pinKey(d: Char) {
272         if (opPin.length < OperatorLock.MAX) opPin += d
273     }
274
275     fun pinBack() {
276         opPin = opPin.dropLast(1)
277     }
278
279     fun pinUnlock(first: Boolean) {
280         val wait = lock.waitLeft()
281         if (wait > 0) {
282             pinMsg = false to "Too many wrong PINs. Try again in ${(wait + 999) / 1000} s."
283             return
284         }
285         if (lock.check(opPin)) {
286             overlay = null
287             opPin = ""
288             Feel.event(Sfx.OP_IN)
289             setRole(Role.OPERATOR, first)
290         } else {
291             opPin = ""
292             val left = lock.waitLeft()
293             Feel.event(if (left > 0) Sfx.PIN_LOCKOUT else Sfx.PIN_WRONG)
294             pinMsg = false to if (left > 0) "Wrong PIN. Locked for ${(left + 999) / 1000} s." else "Wrong PIN.
    ${lock.triesLeft()} tries before a wait."
295         }
296     }
297
298     /** Making or changing the PIN: type it, then type it again. */
299     fun pinSave(first: Boolean, change: Boolean) {
300         if (!OperatorLock.valid(opPin)) {
301             pinMsg = false to "Use ${OperatorLock.MIN} to ${OperatorLock.MAX} digits."
302             return
303         }
304         if (pinNew.isEmpty()) {
305             pinNew = opPin
306             opPin = ""
307             pinMsg = true to "Type the same PIN again."
308             return
309         }
310         if (opPin != pinNew) {
311             opPin = ""; pinNew = ""
312             Feel.event(Sfx.PIN_WRONG)
313             pinMsg = false to "The two PINs differ. Start again."
314             return
315         }
316         lock.set(opPin)
317         opPin = ""; pinNew = ""; pinMsg = null
318         overlay = null
319         Feel.event(if (change) Sfx.PIN_SET else Sfx.OP_IN)
320         if (change) say("Operator PIN changed.") else setRole(Role.OPERATOR, first)
321     }
322
323     /** Changing the PIN asks for the old one first. */
324     fun changePin() {
325         opPin = ""; pinNew = ""; pinMsg = null
326         overlay = Overlay.PinChange
327     }
328
329     fun pinChangeCheck() {
330         val wait = lock.waitLeft()
331         if (wait > 0) {
332             pinMsg = false to "Too many wrong PINs. Try again in ${(wait + 999) / 1000} s."
333             return
334         }
335         if (!lock.check(opPin)) {
336             opPin = ""
337             val left = lock.waitLeft()
338             Feel.event(if (left > 0) Sfx.PIN_LOCKOUT else Sfx.PIN_WRONG)
339             pinMsg = false to if (left > 0) "Wrong PIN. Locked for ${(left + 999) / 1000} s." else "Wrong PIN.
    ${lock.triesLeft()} tries before a wait."
340             return
341         }
342         opPin = ""; pinMsg = null
343         overlay = Overlay.PinSet(first = false, change = true)
344     }
```

```
345
346     fun backgrounded() {
347         awaySince = System.currentTimeMillis()
348     }
349
350     /** Back after more than AUTO_LOCK away, operator drops to collector. */
351     fun foregrounded() {
352         val away = awaySince
353         awaySince = 0L
354         if (away > 0 && role == Role.OPERATOR && System.currentTimeMillis() - away > AUTO_LOCK) {
355             setRole(Role.COLLECTOR, first = false)
356             say("Operator locked after 2 min away. Collector mode.")
357         }
358     }
359
360     fun openFromNotification() {
361         if (role == null) return
362         started = true
363         overlay = null
364         fullscreen = null
365         screen = Screen.OVERVIEW
366     }
367
368     // ---------- sound and feel, from Garbage-Guard ----------
369
370     var sounds by mutableStateOf(Feel.sounds)
371         private set
372     var haptics by mutableStateOf(Feel.haptics)
373         private set
374     var strength by mutableStateOf(Haptics.strength)
375         private set
376     var keepAwake by mutableStateOf(store.bool(SmartBinApp.KEY_AWAKE, false))
377         private set
378
379     fun toggleSounds() {
380         sounds = !sounds
381         Feel.sounds = sounds
382         store.putBool(SmartBinApp.KEY_SOUNDS, sounds)
383     }
384
385     fun toggleHaptics() {
386         haptics = !haptics
387         Feel.haptics = haptics
388         store.putBool(SmartBinApp.KEY_HAPTICS, haptics)
389         if (haptics) Haptics.play(Haptics.Buzz.SAVED)
390     }
391
392     fun chooseStrength(s: Haptics.Strength) {
393         strength = s
394         Haptics.strength = s
395         store.putString(SmartBinApp.KEY_STRENGTH, s.name)
396         Haptics.play(Haptics.Buzz.SAVED)
397     }
398
399     fun toggleAwake() {
400         keepAwake = !keepAwake
401         store.putBool(SmartBinApp.KEY_AWAKE, keepAwake)
402     }
403
404     // ---------- battery ----------
405
406     /** True when Android lets the app run in the background with no battery limits. */
407     var unrestricted by mutableStateOf(batteryFree())
408         private set
409
410     private fun batteryFree(): Boolean {
411         val ctx = getApplication<Application>()
412         return ctx.getSystemService(PowerManager::class.java)?.isIgnoringBatteryOptimizations(ctx.packageName) == true
413     }
414
415     /** Asks Android for unrestricted battery, or opens the list when the phone has no such prompt. */
416     fun askUnrestricted() {
417         val ctx = getApplication<Application>()
418         val ask = Intent(Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS, Uri.parse("package:${ctx.packageName}"))
419         val list = Intent(Settings.ACTION_IGNORE_BATTERY_OPTIMIZATION_SETTINGS)
420         val ok = runCatching { ctx.startActivity(ask.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)) }.isSuccess ||
421             runCatching { ctx.startActivity(list.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)) }.isSuccess
422         if (!ok) say("This phone has no battery setting for apps.")
423     }
424
425     /** Opens the app's own page in Android settings, where some phones keep the battery choice. */
426     fun openAppInfo() {
427         val ctx = getApplication<Application>()
428         runCatching {
429             ctx.startActivity(Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS,
    Uri.parse("package:${ctx.packageName}")).addFlags(Intent.FLAG_ACTIVITY_NEW_TASK))
```

```
        }
    }

    // ---------- push alerts ----------

    var notifications by mutableStateOf(if (Alerts.allowed(app)) Perm.ON else Perm.OFF)
        private set

    fun refreshSystemState() {
        unrestricted = batteryFree()
        val on = Alerts.allowed(getApplication())
        notifications = when {
            on -> Perm.ON
            store.bool(SmartBinApp.KEY_ASKED, false) -> Perm.BLOCKED
            else -> Perm.OFF
        }
    }

    fun permissionAnswered(granted: Boolean) {
        store.putBool(SmartBinApp.KEY_ASKED, true)
        notifications = if (granted) Perm.ON else Perm.BLOCKED
        say(if (granted) "Push alerts are on." else "Push alerts are off. You can change this in Settings.")
    }

    fun notNow() {
        store.putBool(SmartBinApp.KEY_ASKED, true)
        overlay = null
    }

    // ---------- overview ----------

    var binView by mutableStateOf(BinView.AUTO)
    var binFilter by mutableStateOf(BinFilter.ALL)
    var prioAll by mutableStateOf(false)
    var addN by mutableIntStateOf(1)

    // LOCK, UNLOCK, COLLECT, COLLECT_DONE and CLEAR_TAMPER from the bin card.
    // Over local Wi-Fi the result is immediate. Over Firebase or ThingSpeak the
    // command waits in hub.pending until the bin confirms it or times out; the
    // key then flashes green or red for SETTLE_MILLIS (see init above).
    fun command(id: String, cmd: String) {
        viewModelScope.launch {
            val b = Hub.bin(id) ?: return@launch
            if (b.demo) Hub.pauseTour()
            val problem = Hub.command(id, cmd)
            if (problem != null) {
                Feel.event(Sfx.DENIED)
                say(problem)
                return@launch
            }
            val n = b.name
            val msg = when (cmd) {
                "LOCK" -> "$n is locked."
                "UNLOCK" -> "$n is unlocked."
                "CLEAR_TAMPER" -> "Tamper reset on $n."
                "COLLECT" -> "Pickup started on $n. The lid is open."
                "COLLECT_DONE" -> "$n is empty and back to normal."
                else -> "Sent to $n."
            }
            val waiting = Hub.state.value.pending[id]
            if (waiting != null) {
                val via = if (waiting.via == Source.THINGSPEAK) "ThingSpeak" else "Firebase"
                Feel.event(Sfx.CLOUD)
                say("${waiting.cmd} sent to $n via $via. Waiting for the bin.")
            } else say(msg)
        }
    }

    // Bin edit sheet
    var formName by mutableStateOf("")
    var formPlace by mutableStateOf("")
    var formIp by mutableStateOf("")
    var formCloud by mutableStateOf("")
    var formTsCh by mutableStateOf("")
    var formTsKey by mutableStateOf("")
    var formTbId by mutableStateOf("")
    var formTbKey by mutableStateOf("")

    fun openBinEdit(b: Bin) {
        formName = b.name
        formPlace = b.place
        formIp = b.ip
        formCloud = b.cloudId
        formTsCh = b.tsChannel
        formTsKey = b.tsKey
        formTbId = b.tbId
```

```
        formTbKey = b.tbKey
        overlay = Overlay.BinEdit(b.id)
    }

    fun saveBinEdit(id: String) {
        Hub.editBin(id, formName, formPlace, formIp, formCloud, formTsCh, formTsKey, formTbId, formTbKey)
        overlay = null
        say(
            "Saved ${Hub.bin(id)?.name ?: "the bin"}." + when {
                formIp.isNotBlank() -> " Reading it over Wi-Fi now."
                formTsCh.isNotBlank() -> " Reading it from ThingSpeak now."
                else -> ""
            }
        )
    }

    fun removeBin(id: String) {
        val name = Hub.bin(id)?.name ?: "The bin"
        if (calibBin() == id) {
            edit = null
            dirty = false
        }
        Hub.removeBin(id)
        overlay = null
        say("$name removed.")
    }

    fun addBins() {
        val n = addN
        Hub.addBins(n)
        overlay = null
        say("Added $n bin${if (n > 1) "s" else ""}. Tap SET to give each one an address.")
    }

    // ---------- charts ----------

    /** null means all bins. */
    var chartBin by mutableStateOf<String?>(null)
    var envBin by mutableStateOf<String?>(null)
    var envMode by mutableStateOf(EnvMode.ALL)
    var rangeMin by mutableIntStateOf(60)
    /** "fill" or "env" while the sideways chart is open. */
    var fullscreen by mutableStateOf<String?>(null)

    // ---------- settings ----------

    var selSet by mutableStateOf<String?>(null)
    var selSim by mutableStateOf<String?>(null)
    var edit by mutableStateOf<Cal?>(null)
        private set
    var dirty by mutableStateOf(false)
        private set
    /** Settings groups that are open. All start closed, so the one needed is a short scroll away. */
    var open by mutableStateOf(emptySet<String>())
    var pinShown by mutableStateOf(false)
    var tref by mutableStateOf(27.5)
    var known by mutableStateOf(1000.0)

    private fun calibBin() = Hub.state.value.calibBin
    val editing get() = edit != null && calibBin() != null

    fun binFor(id: String?, bins: List<Bin>): Bin? = bins.firstOrNull { it.id == id } ?: bins.firstOrNull()

    fun toggleSection(k: String) {
        open = if (k in open) open - k else open + k
    }

    // Calibration mode on one bin at a time. edit is a local copy of the bin's
    // Cal; nothing reaches the bin until save(). Turning calibration off drops
    // unsaved edits.
    fun setCalibration(on: Boolean, b: Bin) {
        viewModelScope.launch {
            if (on) {
                Hub.setCalibration(b.id)
                edit = b.cal
                dirty = false
                Feel.event(Sfx.CALIB)
                say("Calibration mode on for ${b.name}.")
            } else {
                val lost = dirty
                Hub.setCalibration(null)
                edit = null
                dirty = false
                say(if (lost) "Calibration off. Unsaved changes were dropped." else "Calibration off. ${b.name} is back to
normal.")
            }
```

```
        }
    }

    fun step(key: String, dir: Int) {
        val f = FIELDS.getValue(key)
        when (key) {
            "tref" -> tref = round2((tref + f.step * dir).coerceIn(f.min, f.max))
            "known" -> known = round2((known + f.step * dir).coerceIn(f.min, f.max))
            else -> {
                val e = edit ?: return
                edit = e + (key to round2((e.v(key) + f.step * dir).coerceIn(f.min, f.max)))
                dirty = true
            }
        }
    }

    fun setPin(text: String) {
        val e = edit ?: return
        val digits = text.filter { it.isDigit() }.take(4)
        edit = e + ("collect_pin" to (digits.toIntOrNull() ?: 0).toDouble())
        dirty = true
    }

    fun toggleAutoLock() {
        val e = edit ?: return
        edit = e + ("auto_lock" to if (e.v("auto_lock") > 0) 0.0 else 1.0)
        dirty = true
    }

    fun useNow(key: String, b: Bin) {
        val e = edit ?: return
        var v = Math.round(b.reading.distance * 10) / 10.0
        if (key == "d_full") v = minOf(v, e.v("d_empty") - 1)
        edit = e + (key to v)
        dirty = true
        say("Took ${fmt(v, 1)} cm from the sensor.")
    }

    fun setOffset(b: Bin) {
        val e = edit ?: return
        val off = Math.round((tref - b.reading.tempRaw).coerceIn(-10.0, 10.0) * 10) / 10.0
        edit = e + ("temp_off" to off)
        dirty = true
        say("Offset set to ${fmt(off, 1)} C. Press SAVE to keep it.")
    }

    fun binCommand(b: Bin, cmd: String, extra: Map<String, Any>, ok: String) {
        viewModelScope.launch {
            if (b.demo) Hub.pauseTour()
            val problem = Hub.command(b.id, cmd, extra)
            if (problem != null) Feel.event(Sfx.DENIED)
            say(problem ?: ok)
        }
    }

    /** The bin's touchscreen: four crosses to press on the bin, or back to the firmware's values. Wi-Fi only. */
    fun touchCal(b: Bin, reset: Boolean) {
        if (!b.demo && b.source != Source.LIVE) {
            Feel.event(Sfx.DENIED)
            say("Touch calibration needs ${b.name} on the same Wi-Fi as the phone.")
            return
        }
        Feel.event(if (reset) Sfx.CONFIRM else Sfx.CALIB)
        binCommand(
            b, if (reset) "TOUCH_CAL_RESET" else "TOUCH_CAL", emptyMap(),
            if (reset) "${b.name}'s touchscreen is back to the default values."
            else "${b.name} shows 4 crosses. Press the centre of each one on the bin.",
        )
    }

    /** Gives every bin on Wi-Fi the saved Firebase account, so they write to the same database. */
    fun sendCloudToBins() {
        if (!operator) {
            Feel.event(Sfx.DENIED)
            say("Only an operator can send the cloud account to the bins.")
            return
        }
        viewModelScope.launch {
            val (ok, tried) = Hub.sendCloudToBins()
            when {
                tried == 0 -> { Feel.event(Sfx.DENIED); say("No bin is on Wi-Fi with an address, so nothing was sent.") }
                ok == tried -> { Feel.event(Sfx.SAVED); say("Sent the cloud account to $ok bin${if (ok == 1) "" else
"s"}.") }
                else -> { Feel.event(Sfx.CMD_FAIL); say("$ok of $tried bins took the cloud account. Check the others are
on.") }
            }
```

```
        }
    }

    // Sends the edited calibration to the bin. The full distance must be
    // shorter than the empty distance, the same check the firmware makes.
    fun save(b: Bin) {
        val e = edit ?: return
        if (e.v("d_full") >= e.v("d_empty")) {
            Feel.event(Sfx.DENIED)
            say("Distance full must be smaller than distance empty.")
            return
        }
        viewModelScope.launch {
            val problem = Hub.saveCal(b.id, e)
            if (problem != null) {
                Feel.event(Sfx.DENIED)
                say(problem)
            } else {
                dirty = false
                Feel.event(Sfx.SAVED)
                say("Saved to ${b.name}.")
            }
        }
    }

    fun cancel(b: Bin) {
        edit = b.cal
        dirty = false
        say("Changes dropped.")
    }

    fun reload(b: Bin) {
        viewModelScope.launch {
            if (b.ip.isBlank()) {
                say(if (b.demo) "${b.name} is a demo bin." else "Give ${b.name} an address first.")
                return@launch
            }
            val problem = Hub.reload(b.id)
            if (problem == null && editing && calibBin() == b.id) {
                edit = Hub.bin(b.id)?.cal
                dirty = false
            }
            say(problem ?: "Read the latest values from ${b.name}.")
        }
    }

    /** Writes the bins' names, places and addresses to Downloads, or offers to share them on older phones. */
    fun exportBins(bins: List<Bin>) {
        val a = JSONArray()
        bins.forEach { b -> a.put(JSONObject().put("name", b.name).put("place", b.place).put("ip", b.ip).put("cloud_id",
b.cloudId)) }
        val text = a.toString(2)
        val ctx = getApplication<Application>()
        if (Build.VERSION.SDK_INT >= 29) {
            val ok = runCatching {
                val values = ContentValues().apply {
                    put(MediaStore.Downloads.DISPLAY_NAME, "smartbin-bins.json")
                    put(MediaStore.Downloads.MIME_TYPE, "application/json")
                }
                val uri = ctx.contentResolver.insert(MediaStore.Downloads.EXTERNAL_CONTENT_URI, values) ?: error("no file")
                ctx.contentResolver.openOutputStream(uri)?.use { it.write(text.toByteArray()) }
            }.isSuccess
            say(if (ok) "Bin list saved to Downloads." else "The bin list could not be saved.")
        } else {
            val send = Intent(Intent.ACTION_SEND).setType("application/json").putExtra(Intent.EXTRA_TEXT, text)
            runCatching { ctx.startActivity(Intent.createChooser(send, "Export bin
list").addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)) }
        }
    }

    // ---------- Bluetooth Wi-Fi setup ----------

    /** 0 not paired, 1 paired, 2 joining, 3 joined, 4 Bluetooth off on the bin. */
    var bt by mutableIntStateOf(0)
    var ssid by mutableStateOf("")
    var pass by mutableStateOf("")
    var showPass by mutableStateOf(false)
    /** What is happening now, under the keys. */
    var btNote by mutableStateOf<String?>(null)
    var btBusy by mutableStateOf(false)
    /** The bins found nearby, and whether the search still runs. */
    val btFound = androidx.compose.runtime.mutableStateListOf<BinBle.Found>()
    var btScanning by mutableStateOf(false)
    /** The networks the bin can see. */
    var btNets by mutableStateOf<List<BinBle.Net>>(emptyList())
    /** The address and network the bin joined. */
```

```
    var btIp by mutableStateOf("")
    var btJoined by mutableStateOf("")
    /** The bin being set up: its Bluetooth name, and whether it is the demo stand-in. */
    var btName by mutableStateOf("")
    var btDemo by mutableStateOf(false)
    /** Set when the phone and the bin disagree about an old pairing. */
    var btStale by mutableStateOf(false)

    private val ble by lazy { BinBle(getApplication()) }
    private var btJob: Job? = null
    private var btWait: Job? = null

    /**
     * Pairing a demo bin uses a stand-in for its screen: a fresh random code
     * each time, gone after 60 s, and 3 wrong codes stop pairing for 60 s, the
     * same rules the bin's firmware keeps.
     */
    var btCode by mutableStateOf("")
        private set
    var btCodeAt by mutableLongStateOf(0L)
        private set
    var btEntry by mutableStateOf("")
    var btMsg by mutableStateOf<Pair<Boolean, String>?>(null)
    private var btFails = 0
    private var btWaitUntil = 0L

    fun btPermissions() = ble.permissions()
    fun btAllowed() = ble.allowed()

    private fun btReset() {
        btWait?.cancel()
        bt = 0
        btIp = ""
        btJoined = ""
        btNets = emptyList()
        btStale = false
    }

    /** PAIR A BIN. A demo bin walks through the stand-in; anything else looks for real bins nearby. */
    fun btPair(b: Bin?) {
        btReset()
        if (b != null && b.demo) {
            val wait = btWaitUntil - System.currentTimeMillis()
            if (wait > 0) {
                Feel.event(Sfx.DENIED)
                say("${b.name} stopped pairing after 3 wrong codes. Try again in ${(wait + 999) / 1000} s.")
                return
            }
            btDemo = true
            btName = b.name
            btFails = 0
            btEntry = ""
            btMsg = null
            btNewCode()
            overlay = Overlay.BtCode(b.id)
            return
        }
        btDemo = false
        when {
            !ble.present -> { Feel.event(Sfx.DENIED); btNote = "This phone has no Bluetooth." }
            !ble.allowed() -> { Feel.event(Sfx.DENIED); btNote = "Allow Nearby devices (Bluetooth) for Smart-Bin, then press PAIR A BIN again." }
            !ble.on -> { Feel.event(Sfx.DENIED); btNote = "Turn Bluetooth on, then press PAIR A BIN again." }
            else -> btSearch()
        }
    }

    /** Looks for bins for 8 s and lists them in a sheet. */
    fun btSearch() {
        btFound.clear()
        btScanning = true
        overlay = Overlay.BtDevices
        btNote = "Looking for bins nearby. On the bin, tap i, then WI-FI SETUP, and type the Collect PIN."
        btJob?.cancel()
        btJob = viewModelScope.launch {
            try {
                ble.scan(8_000) { f -> if (btFound.none { it.address == f.address }) btFound.add(f) }
            } catch (e: Exception) {
                if (e is kotlinx.coroutines.CancellationException) throw e
                btNote = e.message ?: "Could not look for bins."
            } finally {
                btScanning = false
            }
            if (btFound.isEmpty()) btNote = "No bin found. Check that the bin shows WI-FI SETUP and stands close by" +
                (if (Build.VERSION.SDK_INT < 31) ", and that Location is on." else ".")
        }
```

```
    }

    /** A bin picked in the sheet: connect, pair with the code Android asks for, then read its networks. */
    fun btConnect(f: BinBle.Found) {
        overlay = null
        btJob?.cancel()
        btName = f.name
        btBusy = true
        btNote = "Connecting to ${f.name}..."
        btJob = viewModelScope.launch {
            try {
                ble.onDrop = { viewModelScope.launch { btDropped() } }
                ble.onStat = { j -> viewModelScope.launch { btStatus(j) } }
                ble.connect(f.device)
                btNote = "${f.name} shows a 6-digit code now. Type it in the box Android opens. The code lasts 60 s."
                ble.secure()
                ble.listen()
                Feel.event(Sfx.BT_PAIRED)
                bt = 1
                btNote = "Paired with ${f.name}. The link is encrypted. Pick the network next."
                btLoadNets()
            } catch (e: kotlinx.coroutines.CancellationException) {
                throw e
            } catch (e: BinBle.StaleBond) {
                ble.close()
                btStale = true
                Feel.event(Sfx.BT_WRONG)
                btNote = "${f.name} keeps dropping the link. The phone most likely remembers it from an earlier pairing.
Press FORGET AND PAIR AGAIN, then on the bin open WI-FI SETUP again."
            } catch (e: Exception) {
                ble.close()
                Feel.event(Sfx.BT_WRONG)
                btNote = "Pairing did not finish (${e.message}). If the code was wrong, the bin shows a new one; after 3
wrong codes it waits 60 s."
            } finally {
                btBusy = false
            }
        }
    }

    /** Makes the phone forget the bin, then searches again. Opens Android's Bluetooth settings if that is not allowed. */
    fun btForget() {
        val d = ble.device ?: btFound.firstOrNull { it.name == btName }?.device
        btStale = false
        if (d != null && ble.forget(d)) {
            Feel.event(Sfx.CONFIRM)
            say("The phone forgot $btName. Pair it again.")
            btSearch()
        } else {
            say("Remove $btName in Android's Bluetooth settings, then press PAIR A BIN.")
            runCatching {

getApplication<Application>().startActivity(Intent(Settings.ACTION_BLUETOOTH_SETTINGS).addFlags(Intent.FLAG_ACTIVITY_NEW_TASK
))
            }
        }
    }

    /** The bin scans when WI-FI SETUP opens; an empty list means it is not done, so it asks again after 5 s. */
    private suspend fun btLoadNets() {
        repeat(3) { k ->
            val nets = runCatching { ble.networks() }.getOrDefault(emptyList())
            if (nets.isNotEmpty()) {
                btNets = nets
                if (ssid.isBlank()) ssid = nets.first().ssid
                return
            }
            if (k < 2) {
                btNote = "The bin is still looking for networks. Asking again in 5 s. You can also type the name."
                delay(5_000)
                if (bt != 1) return
            }
        }
        btNote = "The bin found no networks. Check the router is on 2.4 GHz and close by, press SCAN, or type the name."
    }

    fun btScanNets() {
        if (btDemo) {
            overlay = Overlay.Wifi
            return
        }
        viewModelScope.launch {
            btBusy = true
            btLoadNets()
            btBusy = false
            if (btNets.isNotEmpty()) overlay = Overlay.Wifi
```

```
        }
    }

    fun btNewCode() {
        btCode = "%06d".format(java.security.SecureRandom().nextInt(1_000_000))
        btCodeAt = System.currentTimeMillis()
    }

    fun btKey(d: Char) {
        if (btEntry.length < BT_CODE_LEN) btEntry += d
    }

    fun btBack() {
        btEntry = btEntry.dropLast(1)
    }

    fun btCheck(b: Bin) {
        if (System.currentTimeMillis() - btCodeAt > BT_CODE_LIFE) {
            btNewCode()
            btEntry = ""
            Feel.event(Sfx.BT_WRONG)
            btMsg = false to "That code expired. The bin shows a new one."
            return
        }
        if (btEntry == btCode) {
            overlay = null
            btEntry = ""
            btCode = ""
            bt = 1
            btNets = DEMO_NETS
            Feel.event(Sfx.BT_PAIRED)
            btNote = "Paired with ${b.name} (demo). Pick a network. A name with \"fail\" in it shows what a failed join looks like."
            return
        }
        btEntry = ""
        btFails++
        if (btFails >= 3) {
            overlay = null
            btCode = ""
            btWaitUntil = System.currentTimeMillis() + 60_000L
            Feel.event(Sfx.BT_BLOCKED)
            say("3 wrong codes. ${b.name} stops pairing for 60 s.")
        } else {
            val left = 3 - btFails
            Feel.event(Sfx.BT_WRONG)
            btMsg = false to "Wrong code. $left ${if (left == 1) "try" else "tries"} left."
        }
    }

    fun btCancel() {
        overlay = null
        btEntry = ""
        btCode = ""
    }

    /** SEND WI-FI: the bin tries the network for up to 15 s and keeps its old one if it fails. */
    fun btSend() {
        val name = ssid.trim()
        when {
            name.isBlank() -> { Feel.event(Sfx.DENIED); say("Pick or type the Wi-Fi name first."); return }
            name.length > 32 || pass.length > 63 -> { Feel.event(Sfx.DENIED); say("The name can be 32 characters at most and the password 63."); return }
            pass.isNotEmpty() && pass.length < 8 -> { Feel.event(Sfx.DENIED); say("A Wi-Fi password has at least 8 characters. Leave it empty only for an open network."); return }
        }
        bt = 2
        btNote = "Sending the network to $btName and waiting for it to join..."
        btWait?.cancel()
        if (btDemo) {
            btWait = viewModelScope.launch {
                delay(1800)
                if (name.contains("fail", true)) btStatus(JSONObject().put("state", "FAILED").put("why", "Network not found"))
                else btStatus(JSONObject().put("state", "CONNECTED").put("ip", "192.168.1.${101 + (Hub.state.value.bins.indexOfFirst { it.id == selSet }.coerceAtLeast(0))}").put("ssid", name))
            }
            return
        }
        btWait = viewModelScope.launch {
            try {
                ble.join(name, pass)
            } catch (e: kotlinx.coroutines.CancellationException) {
                throw e
            } catch (e: Exception) {
                bt = 1
```

```
                Feel.event(Sfx.CMD_FAIL)
                btNote = "Could not send (${e.message}). Press SEND WI-FI again."
                return@launch
            }
            // Notifications can be missed or cut short, so the status is read every 2 s too.
            val end = System.currentTimeMillis() + 25_000
            while (bt == 2 && System.currentTimeMillis() < end) {
                delay(2_000)
                if (bt == 2 && ble.connected) runCatching { ble.readStat() }.getOrNull()?.let { btStatus(it) }
            }
            if (bt == 2) {
                bt = 1
                Feel.event(Sfx.WIFI_FAILED)
                btNote = "No answer from the bin after 25 s. Its touchscreen shows whether it joined. Press SEND WI-FI again, or PAIR A BIN."
            }
        }
    }

    /** One STAT message from the bin. */
    private fun btStatus(j: JSONObject) {
        if (bt != 2) return
        when (j.optString("state").uppercase()) {
            "CONNECTING" -> btNote = "$btName is joining ${ssid.trim()}. This takes up to 15 s."
            "CONNECTED" -> {
                val ip = j.optString("ip")
                if (ip.isBlank()) return
                btWait?.cancel()
                btIp = ip
                btJoined = j.optString("ssid").ifBlank { ssid.trim() }
                bt = 3
                Feel.event(Sfx.WIFI_JOINED)
                btNote = "Press ${btUseLabel()} to keep this address, then TURN BT OFF."
            }
            "FAILED" -> {
                btWait?.cancel()
                bt = 1
                Feel.event(Sfx.WIFI_FAILED)
                btNote = "$btName could not join: ${j.optString("why").ifBlank { "no reason given" }}. It went back to its old Wi-Fi. Check the name and password, then SEND WI-FI again."
            }
        }
    }

    /** The link dropped. While joining, the answer is read again after reconnecting. */
    private suspend fun btDropped() {
        when (bt) {
            1 -> {
                bt = 0
                Feel.event(Sfx.BIN_LOST)
                btNote = "$btName closed the Bluetooth link. On the bin, open WI-FI SETUP again, then PAIR A BIN."
            }
            2 -> {
                btNote = "The link dropped while $btName joins. Reconnecting to read the result..."
                val d = btFound.firstOrNull { it.name == btName }?.device ?: return
                runCatching {
                    ble.connect(d)
                    ble.listen()
                    btStatus(ble.readStat())
                }
            }
        }
    }

    /** The bin being set up is the picked one if it is real; otherwise it becomes a new bin. */
    private fun btTarget(): Bin? = Hub.bin(selSet ?: "")?.takeIf { !it.demo }

    fun btUseLabel(): String = when {
        btDemo -> "USE"
        btTarget() != null -> "USE FOR ${btTarget()!!.name.uppercase()}"
        else -> "ADD AS BIN ${btNumber()}".trim()
    }

    private fun btNumber() = Regex("(\\d+)\\s*$").find(btName)?.groupValues?.get(1) ?: ""

    fun btUse() {
        if (btIp.isBlank()) return
        if (btDemo) {
            Feel.event(Sfx.SAVED)
            say("$btName is a demo bin, so nothing was stored.")
            return
        }
        val t = btTarget()
        if (t != null) {
            Hub.editBin(t.id, t.name, t.place, btIp, t.cloudId)
            say("${t.name} will now be read at $btIp.")
```

```
        } else {
            val b = Hub.addRealBin(btNumber().padStart(2, '0').takeIf { btNumber().isNotEmpty() } ?: "", btIp)
            selSet = b.id
            say("Added ${b.name} at $btIp.")
        }
        Feel.event(Sfx.SAVED)
    }

    fun btOff() {
        btWait?.cancel()
        viewModelScope.launch {
            if (!btDemo) ble.off()
            bt = 4
            Feel.event(Sfx.BT_OFF)
            btNote = "Bluetooth is off on $btName. It stays on its Wi-Fi until someone opens WI-FI SETUP again."
        }
    }

    // ---------- simulation ----------

    fun simToggle(b: Bin, bit: Int) {
        val s = b.sim
        Hub.editSim(b.id, s.copy(mask = s.mask xor (1 shl bit)))
    }

    fun simValue(b: Bin, key: String, v: Double) {
        val s = b.sim
        Hub.editSim(b.id, s.copy(values = s.values + (key to v)))
    }

    fun simStep(b: Bin, key: String, dir: Int) {
        val f = SIM_STEP.getValue(key)
        val v = (b.sim.values[key] ?: DEFAULT_SIM[key] ?: 0.0) + f.step * dir
        simValue(b, key, Math.round(v.coerceIn(f.min, f.max) * 10) / 10.0)
    }

    fun applySim(b: Bin) {
        viewModelScope.launch {
            val problem = Hub.applySim(b.id)
            if (problem != null) {
                Feel.event(Sfx.DENIED)
                say(problem)
            } else {
                if (b.sim.mask > 0) Feel.event(Sfx.SIM)
                say(if (b.sim.mask > 0) "Simulation applied to ${b.name}." else "${b.name} is reading real sensors again.")
            }
        }
    }

    fun simAllAuto(b: Bin) {
        Hub.editSim(b.id, b.sim.copy(mask = 0))
        viewModelScope.launch {
            Hub.applySim(b.id)
            say("All rows back to AUTO.")
        }
    }

    // ---------- alerts ----------

    var akind by mutableStateOf(AlertKind.ALL)
    var arange by mutableStateOf(AlertRange.ANY)
    var alertShown by mutableIntStateOf(6)

    fun shown(all: List<AlertRow>): List<AlertRow> {
        val now = System.currentTimeMillis()
        val since = when (arange) {
            AlertRange.TODAY -> Calendar.getInstance().apply {
                set(Calendar.HOUR_OF_DAY, 0); set(Calendar.MINUTE, 0); set(Calendar.SECOND, 0); set(Calendar.MILLISECOND, 0)
            }.timeInMillis
            AlertRange.WEEK -> now - 7 * DAY
            AlertRange.MONTH -> now - 30 * DAY
            AlertRange.ANY -> Long.MIN_VALUE
        }
        return all.filter { (akind == AlertKind.ALL || it.kind == akind) && it.at >= since }
    }

    fun scope(n: Int): String {
        val w = if (n == 1) akind.one else akind.many
        val r = when (arange) {
            AlertRange.TODAY -> " from today"; AlertRange.WEEK -> " from the last 7 days"
            AlertRange.MONTH -> " from the last 30 days"; AlertRange.ANY -> ""
        }
        return (if (akind == AlertKind.ALL && arange == AlertRange.ANY) "all $n $w" else "$n $w") + r
    }

    fun markShown(all: List<AlertRow>) {
```

```
        val s = shown(all)
        Hub.markRead(s.map { it.id }.toSet())
        say("Marked ${scope(s.size)} as read.")
    }

    fun markAll() {
        Hub.markRead(Hub.state.value.alerts.map { it.id }.toSet())
        overlay = null
        say("All alerts marked read.")
    }

    fun deleteShown(all: List<AlertRow>) {
        val s = shown(all)
        val t = scope(s.size)
        Hub.deleteAlerts(s.map { it.id }.toSet())
        overlay = null
        Feel.event(Sfx.DANGER)
        say("Deleted $t.")
    }

    // ---------- demo options ----------

    /** Demo fill speed as a multiple of real life. 60 is the normal demo. */
    var demoSpeed by mutableStateOf(Demo.speed)
        private set
    /** The pretend links live in [Hub], because the demo tour changes them too. */
    val demoLinks: Routes get() = Hub.state.value.demoLinks

    fun changeDemoSpeed(x: Double) {
        Hub.pauseTour()
        demoSpeed = x
        Hub.demoSpeed(x)
    }

    fun toggleDemoLink(which: Int) {
        Hub.pauseTour()
        val r = demoLinks
        val next = when (which) {
            0 -> r.copy(local = !r.local)
            1 -> r.copy(firebase = !r.firebase)
            else -> r.copy(thingspeak = !r.thingspeak)
        }
        Hub.demoLinks(next)
        val main = when {
            next.local -> "Demo bins now answer on Wi-Fi."
            next.firebase -> "Demo bins now answer through Firebase. LOCK, UNLOCK and COLLECT wait for the bin."
            next.thingspeak -> "Demo bins now answer through ThingSpeak. LOCK, UNLOCK and COLLECT wait for the bin."
            else -> "Demo bins show plain demo data again."
        }
        say(main)
    }

    /** Pretend cloud commands: false confirms them after 4 s, true lets them time out after 8 s. */
    fun setDemoNoConfirm(no: Boolean) {
        Hub.pauseTour()
        Hub.setDemoNoConfirm(no)
        say(if (no) "Cloud commands on demo bins will not be confirmed. Their keys give up after 8 s." else "Cloud commands
on demo bins are confirmed after 4 s.")
    }

    fun demoEvent(b: Bin, e: Hub.DemoEvent) {
        if (!b.demo) {
            Feel.event(Sfx.DENIED)
            say("${b.name} is a real bin. Demo options only change demo bins.")
            return
        }
        Hub.pauseTour()
        Hub.demoEvent(b.id, e)
        say(
            when (e) {
                Hub.DemoEvent.FULL -> "${b.name} is now over its fill limit."
                Hub.DemoEvent.TAMPER -> "${b.name} was tipped. Reset it from its card."
                Hub.DemoEvent.AIR -> "Air quality spiked in ${b.name}. It settles by itself."
                Hub.DemoEvent.EMPTY -> "${b.name} is empty."
            }
        )
    }

    fun toggleDemoQuiet(b: Bin) {
        if (!b.demo) {
            Feel.event(Sfx.DENIED)
            say("${b.name} is a real bin. Demo options only change demo bins.")
            return
        }
        Hub.pauseTour()
        val quiet = !Hub.demoQuiet(b.id)
```

```
        Hub.setDemoQuiet(b.id, quiet)
        demoTick++
        say(if (quiet) "${b.name} stops answering. It shows as lost after 90 s." else "${b.name} answers again.")
    }

    /** Bumped when a demo choice kept in [Hub] changes, so the page redraws. */
    var demoTick by mutableIntStateOf(0)
        private set

    fun resetDemo() {
        Hub.resetDemo()
        demoSpeed = Demo.speed
        demoTick++
        Feel.event(Sfx.SAVED)
        say("Demo options are back to normal.")
    }

    // ---------- ThingSpeak ----------

    var tsUrl by mutableStateOf(Hub.thingSpeakUrl())
    var tsMsg by mutableStateOf<Pair<Boolean, String>?>(null)
    var tsBusy by mutableStateOf(false)

    fun tsTest() {
        val url = tsUrl.trim()
        if (!url.startsWith("https://") && !url.startsWith("http://")) {
            Feel.event(Sfx.DENIED)
            tsMsg = false to "The server address starts with https://."
            return
        }
        Hub.saveThingSpeak(url)
        tsBusy = true
        viewModelScope.launch {
            val (ok, problem) = Hub.testThingSpeak()
            tsBusy = false
            if (ok > 0 && problem == null) {
                Feel.event(Sfx.CLOUD)
                tsMsg = true to "$ok channel${if (ok == 1) "" else "s"} answered. Saved."
            } else {
                Feel.event(Sfx.DENIED)
                tsMsg = false to (if (ok > 0) "$ok answered. " else "") + (problem ?: "No channel answered.")
            }
        }
    }

    // ---------- Firebase ----------

    var fb by mutableStateOf(Hub.firebase())
    var fbShow by mutableStateOf(false)
    /** (passed, message) */
    var fbMsg by mutableStateOf<Pair<Boolean, String>?>(null)
    var fbBusy by mutableStateOf(false)

    fun fbTest(save: Boolean) {
        val cfg = fb
        Cloud.problemWith(cfg)?.let {
            Feel.event(Sfx.DENIED)
            fbMsg = false to (it + if (save) " Nothing was saved." else "")
            return
        }
        fbBusy = true
        viewModelScope.launch {
            val r = Hub.cloud.test(cfg)
            fbBusy = false
            r.onSuccess { n ->
                if (save) Hub.saveFirebase(cfg)
                Feel.event(if (save) Sfx.SAVED else Sfx.CLOUD)
                fbMsg = true to "Signed in as ${cfg.email.trim()}. $n bin${if (n == 1) " is" else "s are"} in the database."
+ if (save) " Saved as the working set." else ""
            }.onFailure {
                Feel.event(Sfx.DENIED)
                fbMsg = false to (it.message.orEmpty() + if (save) " Nothing was saved." else "")
            }
        }
    }

    fun fbRestore() {
        val w = Hub.firebaseWorking()
        if (w == null) {
            fbMsg = false to "No details have passed a test on this phone yet."
            return
        }
        fb = w
        Hub.saveFirebase(w)
        fbMsg = true to "Put back the last details that passed a test."
    }
```

```

    companion object {
        private const val DAY = 24 * 3600_000L
        private const val AUTO_LOCK = 2 * 60_000L
        const val BT_CODE_LEN = 6
        const val BT_CODE_LIFE = 60_000L
        val DEMO_NETS = listOf(BinBle.Net("HomeWiFi-2G", -48), BinBle.Net("TUP-Manila", -61), BinBle.Net("Smart_Bin_Lab", -70), BinBle.Net("PLDTHOMEFIBR24", -79))
        private fun round2(v: Double) = Math.round(v * 100) / 100.0
    }
}

/** One setting stepper: the mockup's FIELDS table. */
class FieldSpec(val label: String, val icon: () -> androidx.compose.ui.graphics.vector.ImageVector, val unit: String, val step: Double, val min: Double, val max: Double, val dec: Int)

val FIELDS: Map<String, FieldSpec> = mapOf(
    "d_empty" to FieldSpec("Distance empty", { Ic.ruler }, "cm", 0.5, 1.0, 400.0, 1),
    "d_full" to FieldSpec("Distance full", { Ic.ruler }, "cm", 0.5, 1.0, 400.0, 1),
    "fill_th" to FieldSpec("Fill threshold", { Ic.percent }, "%", 1.0, 10.0, 100.0, 0),
    "temp_th" to FieldSpec("Temperature threshold", { Ic.temp(40.0) }, "°C", 1.0, 10.0, 85.0, 0),
    "hum_th" to FieldSpec("Humidity threshold", { Ic.humid }, "%RH", 1.0, 30.0, 100.0, 0),
    "gas_th" to FieldSpec("Air quality limit", { Ic.gas }, "IAQ", 5.0, 50.0, 500.0, 0),
    "tilt_th" to FieldSpec("Tilt threshold", { Ic.tilt }, "deg", 1.0, 5.0, 90.0, 0),
    "weight_cap" to FieldSpec("Weight capacity", { Ic.weight(0.6) }, "g", 50.0, 100.0, 5000.0, 0),
    "temp_off" to FieldSpec("Temperature offset", { Ic.temp(28.0) }, "°C", 0.1, -10.0, 10.0, 1),
    "hum_off" to FieldSpec("Humidity offset", { Ic.humid }, "%RH", 0.5, -20.0, 20.0, 1),
    "press_off" to FieldSpec("Pressure offset", { Ic.press }, "hPa", 0.1, -50.0, 50.0, 1),
    "servo_closed" to FieldSpec("Servo closed", { Ic.servo }, "deg", 1.0, 0.0, 180.0, 0),
    "servo_open" to FieldSpec("Servo open", { Ic.servo }, "deg", 1.0, 0.0, 180.0, 0),
    "pir_hold_s" to FieldSpec("Lid stays open", { Ic.clock }, "s", 1.0, 1.0, 60.0, 0),
    "buzz_s" to FieldSpec("Buzzer minimum", { Ic.bell }, "s", 1.0, 1.0, 30.0, 0),
    "tref" to FieldSpec("Reference thermometer", { Ic.temp(28.0) }, "°C", 0.1, -40.0, 85.0, 1),
    "known" to FieldSpec("Known weight", { Ic.weight(0.3) }, "g", 50.0, 10.0, 5000.0, 0),
)

class SimStep(val unit: String, val step: Double, val min: Double, val max: Double)

val SIM_STEP = mapOf(
    "weight" to SimStep("g", 100.0, 0.0, 5000.0),
    "temp" to SimStep("°C", 0.5, -40.0, 85.0),
    "humidity" to SimStep("%", 1.0, 0.0, 100.0),
)

fun fmt(v: Double, dec: Int): String = if (dec == 0) Math.round(v).toString() else "%.${dec}f".format(v)

/** "2 min ago", as the alert rows say it. */
fun ago(at: Long, now: Long = System.currentTimeMillis()): String {
    val m = ((now - at) / 60_000L).coerceAtLeast(0)
    return when {
        m < 1 -> "now"
        m < 60 -> "$m min ago"
        m < 24 * 60 -> "${m / 60} h ago"
        m < 48 * 60 -> "1 day ago"
        else -> "${m / (24 * 60)} days ago"
    }
}
```

Android App/app/src/main/java/smartbin/ui/SmartBinUi.kt

```
// ============================================================================
// ui/SmartBinUi.kt - the root composable
// Draws the start page (Gate) until a role is picked, then the header, the
// current page, the bottom tabs, the overlays and the toast. Every page reads
// Hub.state through collectAsState, so a screen that does not update usually
// means Hub did not change the value, not that the UI missed it.
// The layout is drawn for a 384 dp wide phone and scaled (0.8x to 1.6x), and
// the phone's font size is clamped to 0.9..1.1 so labels are not cut off.
// ============================================================================
package smartbin.ui

import android.Manifest
import android.content.Intent
import android.os.Build
import android.provider.Settings
import androidx.activity.ComponentActivity
import androidx.activity.SystemBarStyle
import androidx.activity.compose.BackHandler
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.enableEdgeToEdge
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.AnimatedContent
import androidx.compose.animation.core.tween
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
```

```
import androidx.compose.animation.slideInHorizontally
import androidx.compose.animation.slideOutHorizontally
import androidx.compose.animation.togetherWith
import androidx.compose.foundation.background
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.RowScope
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.defaultMinSize
import androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.offset
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.LocalContentColor
import androidx.compose.runtime.Composable
import androidx.compose.runtime.CompositionLocalProvider
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.foundation.gestures.awaitEachGesture
import androidx.compose.foundation.gestures.awaitFirstDown
import androidx.compose.ui.input.pointer.PointerEventPass
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.setValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.drawBehind
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.platform.LocalConfiguration
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.platform.LocalView
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.Density
import androidx.compose.ui.unit.IntOffset
import androidx.compose.ui.unit.dp
import androidx.compose.ui.zIndex
import smartbin.data.Bin
import smartbin.data.BinStatus
import smartbin.data.Hub
import smartbin.data.HubState
import smartbin.data.Link
import smartbin.data.Mode
import smartbin.data.Role
import smartbin.data.Severity
import smartbin.data.v
import smartbin.notify.Feel
import smartbin.notify.Sfx

/** The whole app: the gate on first launch, then header, page, tabs and sheets, as the mockup's phone frame. */
@Composable
fun SmartBinUi(vm: AppViewModel, setLandscape: (Boolean) -> Unit) {
    val hub by Hub.state.collectAsState()
    val dark = when (vm.theme) {
        ThemeMode.AUTO -> isSystemInDarkTheme()
        ThemeMode.LIGHT -> false
        ThemeMode.DARK -> true
    }
    val mode = if (vm.role == null || !vm.started) Mode.NORMAL else hub.mode
    LaunchedEffect(vm.started, vm.role) { vm.syncInMain() }
    val c = animated(sbColors(dark, mode))
    val context = LocalContext.current

    LaunchedEffect(dark) {
        val clear = android.graphics.Color.TRANSPARENT
        val style = if (dark) SystemBarStyle.dark(clear) else SystemBarStyle.light(clear, clear)
        (context as? ComponentActivity)?.enableEdgeToEdge(statusBarStyle = style, navigationBarStyle = style)
    }
    LaunchedEffect(vm.fullscreen) { setLandscape(vm.fullscreen != null) }
    val root = LocalView.current
```

```
    LaunchedEffect(vm.keepAwake) { root.keepScreenOn = vm.keepAwake }

    val askAlerts = rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission())
{ vm.permissionAnswered(it) }
    val allowAlerts: () -> Unit = {
        vm.overlay = null
        if (Build.VERSION.SDK_INT >= 33) askAlerts.launch(Manifest.permission.POST_NOTIFICATIONS)
        else openAlertSettings(context)
    }

    // The mockup is drawn for a phone 384 dp wide. Other phones scale it, so
    // sizes, spacing and text keep their proportions, and the phone's own font
    // size is held to a small range so no label is cut off.
    val cfg = LocalConfiguration.current
    val base = LocalDensity.current
    val land = vm.fullscreen != null
    val byWidth = cfg.smallestScreenWidthDp / 384f
    val fit = (if (land) byWidth else minOf(byWidth, cfg.screenHeightDp / 700f)).coerceIn(0.8f, 1.6f)
    val scaled = remember(base.density, base.fontScale, fit) { Density(base.density * fit, base.fontScale.coerceIn(0.9f,
1.1f)) }

    CompositionLocalProvider(LocalSb provides c, LocalContentColor provides c.ink, LocalDensity provides scaled) {
        Box(
            Modifier.fillMaxSize().page(c).pointerInput(Unit) {
                // Watches every touch before the keys get it, without taking it:
                // the notice goes away and the key under the finger still works.
                awaitEachGesture {
                    awaitFirstDown(requireUnconsumed = false, pass = PointerEventPass.Initial)
                    vm.hideToast()
                }
            },
        ) {
            when {
                vm.role == null || !vm.started -> Column(Modifier.fillMaxSize().statusBarsPadding().navigationBarsPadding())
{ Gate(vm, hub) }
                land -> FullChart(vm, hub)
                else -> Column(
                    Modifier.align(Alignment.TopCenter).fillMaxHeight().widthIn(max =
520.dp).fillMaxWidth().statusBarsPadding(),
                ) {
                    Header(vm, hub, dark)
                    Box(Modifier.weight(1f)) {
                        if (hub.loading) Loading()
                        else AnimatedContent(
                            targetState = vm.screen,
                            transitionSpec = {
                                val dir = if (targetState.ordinal > initialState.ordinal) 1 else -1
                                (slideInHorizontally(tween(220)) { dir * it / 8 } + fadeIn(tween(220))) togetherWith
                                    (slideOutHorizontally(tween(160)) { -dir * it / 8 } + fadeOut(tween(110)))
                            },
                            label = "tab",
                        ) { s ->
                            when (s) {
                                Screen.OVERVIEW -> Overview(vm, hub)
                                Screen.CHARTS -> Charts(vm, hub)
                                Screen.SETTINGS -> SettingsPage(vm, hub)
                                Screen.SIM -> if (vm.operator) Simulation(vm, hub) else Overview(vm, hub)
                                Screen.ALERTS -> AlertsPage(vm, hub)
                                Screen.APP -> AppPage(vm, hub)
                                Screen.BT -> Bluetooth(vm, hub)
                                Screen.NET -> Connection(vm, hub)
                            }
                        }
                    }
                    BottomNav(vm, hub)
                }
            }
            // Back from a page under Settings, or from a tab, goes one step toward Overview.
            val view = LocalView.current
            BackHandler(enabled = vm.overlay == null && vm.role != null && vm.started && (land || vm.screen !=
Screen.OVERVIEW)) {
                Feel.play(view, Sfx.BACK)
                if (land) vm.fullscreen = null else vm.back()
            }
            Overlays(vm, hub, allowAlerts)
            vm.toast?.let { Toast(it, bottom = if (land || vm.role == null || !vm.started) 24.dp else 92.dp) }
        }
    }
}

fun openAlertSettings(context: android.content.Context) {
    runCatching {
        context.startActivity(
            Intent(Settings.ACTION_APP_NOTIFICATION_SETTINGS)
                .putExtra(Settings.EXTRA_APP_PACKAGE, context.packageName)
                .addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
```

```
        )
    }
}

@Composable
private fun Loading() {
    ScrollPage {
        Skeleton(40.dp, Modifier.fillMaxWidth(0.5f), radius = 12.dp)
        Skeleton(430.dp)
        Skeleton(180.dp)
    }
}

// ---------- header ----------

private val MODES = mapOf(
    Mode.NORMAL to "NORMAL", Mode.SIM to "SIMULATION", Mode.CALIB to "CALIBRATING", Mode.DUAL to "SIM + CAL", Mode.ALERT to "ALERT",
)

private fun modeIcon(m: Mode): ImageVector = when (m) {
    Mode.NORMAL -> Ic.check; Mode.SIM -> Ic.flask; Mode.CALIB, Mode.DUAL -> Ic.calib; Mode.ALERT -> Ic.bell
}

@Composable
private fun Header(vm: AppViewModel, hub: HubState, dark: Boolean) {
    val c = Sb.c
    val view = LocalView.current
    // Brand, mode chip, theme key
    Row(
        Modifier.fillMaxWidth().padding(horizontal = 14.dp, vertical = 4.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        // A tap on the logo plays its chime and its build-up again.
        var logoPlay by remember { androidx.compose.runtime.mutableIntStateOf(1) }
        Row(
            Modifier.press(true, Sfx.LOGO) { logoPlay++ },
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(6.dp),
        ) {
            SmartBinLogo(30.dp, logoPlay)
            T("SMART-BIN", size = 1.12f, spacing = 0.04f, maxLines = 1)
        }
        Spacer(Modifier.weight(1f))
        // DEMO shows only while demo mode is on. It goes back to the start page, where demo is turned off.
        if (hub.demoOn) {
            Box(
                Modifier.defaultMinSize(minHeight = 36.dp).bump(c, 11.dp).press(true, Sfx.BACK) { vm.toStart() }.padding(horizontal = 10.dp),
                contentAlignment = Alignment.Center,
            ) { T("DEMO", size = 0.74f, spacing = 0.04f, color = c.warn, maxLines = 1) }
        }
        val m = hub.mode
        Row(
            Modifier
                .raised(c, 999.dp)
                .clip(RoundedCornerShape(999.dp))
                .background(c.barBrush)
                .padding(horizontal = 12.dp, vertical = 8.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(6.dp),
        ) {
            Ico(modeIcon(m), 15.dp, c.onAccent)
            T(MODES.getValue(m), size = 0.74f, spacing = 0.04f, color = c.onAccent, maxLines = 1)
        }
        IconKey(Ic.sunmoon(dark), {
            vm.toggleTheme(dark)
        }, size = 40.dp, iconSize = 19.dp, sound = if (dark) Sfx.THEME_LIGHT else Sfx.THEME_DARK)
        // The gear opens the app's own settings: sound, vibration, battery, links and demo options.
        val onApp = vm.screen == Screen.APP || vm.screen == Screen.NET || vm.screen == Screen.BT
        IconKey(Ic.gear, { if (onApp) vm.back() else vm.go(Screen.APP) }, size = 40.dp, iconSize = 19.dp, tint = if (onApp) c.accent else c.ink2, sound = if (onApp) Sfx.BACK else Sfx.OPEN)
    }
    // The live strip: what needs attention, the link and the role
    Row(
        Modifier.fillMaxWidth().padding(horizontal = 14.dp, vertical = 6.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        Row(Modifier.weight(1f), verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            StatusPill(vm, hub)
            LinkChip(hub, Modifier.weight(1f, fill = false)) { vm.go(Screen.NET) }
        }
        val src = remember { MutableInteractionSource() }
```

```
        val pressed by src.collectIsPressedAsState()
        Box(
            Modifier
                .defaultMinSize(minHeight = 36.dp)
                .then(if (pressed) Modifier.clip(RoundedCornerShape(11.dp)).background(c.base).sunk(c, 11.dp) else Modifier.bump(c, 11.dp))
                .press(true, Sfx.CHOICE, src) { vm.pickRole(if (vm.operator) Role.COLLECTOR else Role.OPERATOR) }
                .padding(horizontal = 13.dp),
            contentAlignment = Alignment.Center,
        ) { T(if (vm.operator) "OPERATOR" else "COLLECTOR", size = 0.76f, spacing = 0.04f, maxLines = 1) }
    }
    Box(Modifier.padding(start = 14.dp, end = 14.dp, top = 4.dp).fillMaxWidth().height(3.dp).clip(RoundedCornerShape(3.dp)).background(c.barBrush))
}

/** One active condition: the banner kind, its icon, title and line under it. */
class Live(val tone: Tone, val icon: ImageVector, val title: String, val small: String? = null, val pill: Pair<ImageVector, Color>)

// Builds the attention banners (calibration, tamper, full, simulation, lost
// bins, sensor faults) shown in the header pill and the Attention sheet.
/** The conditions the mockup shows as banners, in its order. */
@Composable
fun liveBanners(hub: HubState): List<Live> {
    val c = Sb.c
    val b = hub.bins
    val out = ArrayList<Live>()
    val cal = b.filter { it.calibrating }
    val tam = b.filter { it.reading.status == BinStatus.TAMPER }
    val full = b.filter { it.full }
    val sim = b.filter { it.simulating }
    val lost = b.filter { it.stale }
    if (cal.isNotEmpty()) out += Live(Tone.TEAL, Ic.calib, "CALIBRATING: " + cal.joinToString { it.name }, "Keep the bin empty and do not touch it.", Ic.calib to c.cal)
    if (tam.isNotEmpty()) out += Live(Tone.RED, Ic.tamper, "TAMPER: " + tam.joinToString { it.name }, "Check the bin before resetting the alert.", Ic.tamper to c.tamper)
    if (full.isNotEmpty()) out += Live(Tone.RED, Ic.truck, "COLLECT NOW: " + full.joinToString { "${it.name} at ${Math.round(it.reading.fill)}%" }, null, Ic.truck to c.tamper)
    if (sim.isNotEmpty()) out += Live(Tone.PURPLE, Ic.flask, "SIMULATION MODE", "Some readings are set by hand, not by the sensors.", Ic.flask to c.sim)
    // Pretend links from the demo options say so, so nobody mistakes them for real ones.
    hub.tourMsg?.let { out += Live(Tone.PURPLE, Ic.flask, "Demo tour: $it", "The tour runs by itself. Press a key on a demo bin and it waits a minute for you.", Ic.flask to c.sim) }
    val broken = b.filter { it.reading.faults.isNotEmpty() }
    if (broken.isNotEmpty()) out += Live(
        Tone.AMBER, Ic.gear, "SENSOR NOT ANSWERING: " + broken.joinToString { "${it.name} (${it.reading.faults.joinToString()})" },
        "Its readings show -- until the sensor answers again. Check the wiring.", Ic.gear to c.warn,
    )
    if (b.isEmpty()) {
        out += Live(Tone.AMBER, Ic.wifi(false), "No bins linked yet", "Pair a bin over Bluetooth in App settings, or add one with its IP address as operator. Turn DEMO on at the start page to look around.", Ic.wifi(false) to c.warn)
        return out
    }
    if (hub.demoOn) out += Live(Tone.AMBER, Ic.flask, "Demo mode", "Bins named Demo show made-up readings. Press DEMO at the top to go back to the start page and turn it off.", Ic.flask to c.warn)
    val pretend = if (hub.demo) "Pretend link from the demo options. " else ""
    when (hub.link) {
        Link.DEMO -> {}
        Link.OFFLINE -> out += Live(Tone.RED, Ic.wifi(false), "Bins are not answering", "No bin answers on Wi-Fi and the cloud cannot be reached. Check the phone has internet.", Ic.wifi(false) to c.tamper)
        Link.CLOUD -> out += Live(
            Tone.ORANGE, Ic.cloud, "Cloud mode",
            pretend + "No bin answers on this Wi-Fi, so readings come through Firebase every 5 s. LOCK, UNLOCK and COLLECT still work. Calibration and simulation need the bins' Wi-Fi.",
            Ic.cloud to c.cloud,
        )
        Link.THINGSPEAK -> out += Live(
            Tone.BLUE, Ic.chart, "ThingSpeak mode",
            pretend + "Wi-Fi and Firebase do not answer, so readings come from ThingSpeak every 15 s. LOCK, UNLOCK and COLLECT go through TalkBack.",
            Ic.chart to c.ts,
        )
        Link.LIVE -> {}
    }
    if (lost.isNotEmpty() && hub.link != Link.OFFLINE) {
        out += Live(Tone.RED, Ic.wifi(false), "Not answering: " + lost.joinToString { it.name }, "No reading for more than 90 s. Check its power and Wi-Fi.", Ic.wifi(false) to c.tamper)
    }
    return out
}

@Composable
private fun StatusPill(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val live = liveBanners(hub)
```

```
    val unread = hub.alerts.filter { !it.read }
    if (unread.isEmpty() && live.isEmpty()) return
    val (icon, cc, text) = if (unread.isNotEmpty()) {
        val worst = Severity.entries.first { s -> unread.any { it.sev == s } }
        Triple(Ic.bell, sevColor(worst, c), "${unread.size} unread")
    } else {
        val p = live.first().pill
        Triple(p.first, p.second, "${live.size} active")
    }
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    Row(
        Modifier
            .defaultMinSize(minHeight = 36.dp)
            .then(if (pressed) Modifier.ring(cc.soft(0.34f),
18.dp).clip(RoundedCornerShape(18.dp)).background(c.base).sunk(c, 18.dp) else Modifier.bump(c, 18.dp, ring = cc.soft(0.34f)))
            .press(true, Sfx.OPEN, src) { vm.overlay = Overlay.Attention }
            .padding(horizontal = 13.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(7.dp),
    ) {
        Ico(icon, 18.dp, cc)
        T(text, size = 0.8f, spacing = 0.03f, color = cc, maxLines = 1)
    }
}

fun sevColor(s: Severity, c: SbColors) = when (s) {
    Severity.CRITICAL -> c.bad; Severity.WARNING -> c.warn; Severity.INFO -> c.sim; Severity.SUCCESS -> c.ok
}

fun sevTone(s: Severity) = when (s) {
    Severity.CRITICAL -> Tone.RED; Severity.WARNING -> Tone.AMBER; Severity.INFO -> Tone.PURPLE; Severity.SUCCESS ->
Tone.GREEN
}

/** One way to the bins, as the link chip and Connection draw it. */
class RouteLook(val name: String, val icon: ImageVector, val color: Color)

/** The routes that answer, in their order: Wi-Fi green, Firebase orange, ThingSpeak blue. */
fun routeLooks(r: smartbin.data.Routes, c: SbColors): List<RouteLook> = buildList {
    if (r.local) add(RouteLook("Wi-Fi", Ic.wifi(true), c.link))
    if (r.firebase) add(RouteLook("Firebase", Ic.cloud, c.cloud))
    if (r.thingspeak) add(RouteLook("ThingSpeak", Ic.chart, c.ts))
}

// The header chip: which route is reaching the bins. Tapping it opens the
// Connection page.
/**
 * Which links answer. Each route has its own colour and icon, so every mix
 * looks different: one icon per route that is up, a ring in the colour of the
 * one the readings come from, and a strip of all their colours underneath.
 */
@Composable
private fun LinkChip(hub: HubState, modifier: Modifier = Modifier, onClick: () -> Unit) {
    val c = Sb.c
    val link = hub.link
    val parts = routeLooks(hub.routes, c)
    val main = when (link) {
        Link.LIVE -> c.link; Link.CLOUD -> c.cloud; Link.THINGSPEAK -> c.ts
        Link.OFFLINE -> c.bad; Link.DEMO -> parts.firstOrNull()?.color ?: c.warn
    }
    val text = when {
        parts.size == 3 -> "All 3 links"
        parts.isNotEmpty() -> parts.joinToString(" + ") { it.name }
        link == Link.OFFLINE -> "Not connected"
        link == Link.DEMO -> "Demo data"
        else -> "Connecting"
    }
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    Row(
        modifier
            .defaultMinSize(minHeight = 36.dp)
            .then(if (pressed) Modifier.clip(RoundedCornerShape(18.dp)).background(c.base).sunk(c, 18.dp) else
Modifier.bump(c, 18.dp, ring = if (parts.isNotEmpty()) main.soft(0.34f) else null))
            .press(true, Sfx.OPEN, src, onClick)
            .drawBehind {
                // The colour strip: one segment per route that answers.
                if (parts.size > 1) {
                    val h = 3.dp.toPx()
                    val w = (size.width - 26.dp.toPx()) / parts.size
                    parts.forEachIndexed { k, p ->
                        drawRoundRect(
                            p.color, topLeft = Offset(13.dp.toPx() + k * w, size.height - h - 3.dp.toPx()),
                            size = androidx.compose.ui.geometry.Size(w - 2.dp.toPx(), h),
                            cornerRadius = androidx.compose.ui.geometry.CornerRadius(h / 2),
```

```
                        )
                    }
                }
            }
            .padding(horizontal = 13.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(5.dp),
    ) {
        if (parts.isEmpty()) Ico(Ic.wifi(false), 17.dp, if (link == Link.OFFLINE) c.bad else c.warn)
        parts.forEach { p -> Ico(p.icon, 16.dp, p.color, Modifier.breathe(p.color == main, 2200)) }
        T(text, Modifier.padding(start = 2.dp), size = 0.8f, weight = FontWeight.Bold, color = if (parts.size == 1) main else c.ink, fit = true)
    }
}

// ---------- bottom nav ----------

private class TabDef(val screen: Screen, val label: String, val icon: ImageVector, val cc: (SbColors) -> Color)

private val TABS = listOf(
    TabDef(Screen.OVERVIEW, "Overview", Ic.grid) { it.fill },
    TabDef(Screen.CHARTS, "Charts", Ic.chart) { it.humid },
    TabDef(Screen.SETTINGS, "Settings", Ic.sliders) { it.cal },
    TabDef(Screen.SIM, "Simulation", Ic.flask) { it.sim },
    TabDef(Screen.ALERTS, "Alerts", Ic.bell) { it.tamper },
)

// Five tabs. A red badge on Alerts counts unread alert rows.
@Composable
private fun BottomNav(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    // The gear pages belong to no tab.
    val current = vm.screen.takeIf { it.ordinal < Screen.APP.ordinal }
    val unread = hub.unread
    Row(
        Modifier
            .fillMaxWidth()
            .drawBehind {
                drawRect(
                    androidx.compose.ui.graphics.Brush.verticalGradient(listOf(Color.Transparent, c.shD.soft(0.6f))),
                    topLeft = Offset(0f, -14.dp.toPx()), size = androidx.compose.ui.geometry.Size(size.width, 14.dp.toPx()),
                )
            }
            .background(c.base)
            .navigationBarsPadding()
            .padding(start = 8.dp, end = 8.dp, top = 8.dp, bottom = 4.dp),
        horizontalArrangement = Arrangement.spacedBy(5.dp),
    ) {
        TABS.filter { it.screen != Screen.SIM || vm.operator }.forEach { t ->
            NavItem(t.icon, t.label, current == t.screen, t.cc(c), if (t.screen == Screen.ALERTS) unread else 0) {
                vm.go(t.screen)
            }
        }
    }
}

@Composable
private fun RowScope.NavItem(icon: ImageVector, label: String, selected: Boolean, cc: Color, badge: Int, onClick: () -> Unit) {
    val c = Sb.c
    Box(Modifier.weight(1f)) {
        Column(
            Modifier
                .fillMaxWidth()
                .defaultMinSize(minHeight = 56.dp)
                .then(if (selected) Modifier.ring(cc.soft(0.26f), 14.dp).clip(RoundedCornerShape(14.dp)).background(c.base).sunk(c, 14.dp) else Modifier.clip(RoundedCornerShape(14.dp)))
                .press(!selected, Sfx.TAB, onClick = onClick),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(3.dp, Alignment.CenterVertically),
        ) {
            val t = if (selected) cc else c.ink2
            Ico(icon, 22.dp, t)
            T(label, size = 9.8f / 14f, weight = FontWeight.Bold, color = t, maxLines = 1)
        }
        if (badge > 0) {
            Box(
                Modifier
                    .align(Alignment.TopCenter)
                    .offset { IntOffset(15.dp.roundToPx(), 3.dp.roundToPx()) }
                    .defaultMinSize(minWidth = 20.dp)
                    .height(20.dp)
                    .brutal(c, 6.dp, c.tamper, off = 2.dp)
                    .padding(horizontal = 4.dp),
                contentAlignment = Alignment.Center,
            ) { T(if (badge > 99) "99+" else badge.toString(), size = 11f / 14f, color = Color.White, maxLines = 1,
```

```
    lineHeight = 1f) }
512         }
513     }
514 }
515 
516 // ---------- start page ----------
517 
518 // The start page: role cards (Collector, Operator), the DEMO switch and the
519 // state of each route. Demo mode can only be turned on or off here.
520 /**
521  * The start page, on every start of the app and from App settings: the logo,
522  * the two roles, demo mode, and how the bins answer. Nothing sounds or posts
523  * from here; alerts start once a role is picked.
524  */
525 @Composable
526 private fun Gate(vm: AppViewModel, hub: HubState) {
527     val c = Sb.c
528     var play by remember { androidx.compose.runtime.mutableIntStateOf(1) }
529     ScrollPage {
530         Column(Modifier.fillMaxWidth().padding(top = 14.dp, bottom = 4.dp), horizontalAlignment =
    Alignment.CenterHorizontally, verticalArrangement = Arrangement.spacedBy(10.dp)) {
531             Box(Modifier.size(124.dp).well(c, 62.dp).press(true, Sfx.LOGO) { play++ }, contentAlignment = Alignment.Center) {
532                 SmartBinLogo(96.dp, play)
533             }
534             T("SMART-BIN", size = 1.65f, spacing = 0.04f)
535             Hint("Pick how you are using the system today. Operator asks for a PIN. Collector is the default.",
    Modifier.widthIn(max = 290.dp), align = androidx.compose.ui.text.style.TextAlign.Center)
536         }
537         RoleCard(Ic.sliders, "Admin / Operator", listOf("Charts and history", "Lock, unlock, reset tamper", "Calibration",
    "Simulation controls", "Add and edit bins")) {
538             vm.pickRole(Role.OPERATOR, first = true)
539         }
540         RoleCard(Ic.truck, "Collector", listOf("Which bin to empty first", "Charts, alerts and the collect PIN", "Start and
    finish a pickup", "Lock and unlock a bin", "Pair a bin to Wi-Fi over Bluetooth", "Settings are read only")) {
541             vm.pickRole(Role.COLLECTOR, first = true)
542         }
543         // Demo mode is turned on or off only here.
544         Row(
545             Modifier.fillMaxWidth().well(c, 18.dp).padding(14.dp),
546             verticalAlignment = Alignment.CenterVertically,
547             horizontalArrangement = Arrangement.spacedBy(12.dp),
548         ) {
549             Btn(
550                 if (hub.demoOn) "DEMO ON" else "DEMO OFF", { vm.toggleDemo() }, icon = Ic.flask, look = Look.TINT,
551                 cc = if (hub.demoOn) c.warn else c.ink2, small = true, pressedIn = hub.demoOn, sound = Sfx.TAP,
552             )
553             T(
554                 if (hub.demoOn) "Three made-up bins sit next to your real ones and act out every alert by themselves."
555                 else "Turn DEMO on to add three made-up bins that show every alert by themselves.",
556                 Modifier.weight(1f), size = 0.8f, weight = FontWeight.SemiBold, color = c.ink2,
557             )
558         }
559         val real = hub.bins.filter { !it.demo }
560         if (real.isNotEmpty()) {
561             Column(Modifier.fillMaxWidth().well(c, 16.dp).padding(14.dp), verticalArrangement = Arrangement.spacedBy(10.dp))
    {
562                 val anyIp = real.any { it.ip.isNotBlank() }
563                 StartLine(
564                     Ic.wifi(true), "Local Wi-Fi", c.link,
565                     when { real.any { it.source == smartbin.data.Source.LIVE } -> true; !anyIp -> null; else -> false },
566                     if (anyIp) "Trying" else "No address",
567                 )
568                 StartLine(Ic.cloud, "Firebase", c.cloud, hub.fbUp, if (hub.fbUp == null) "Not set up" else "Trying")
569                 StartLine(Ic.chart, "ThingSpeak", c.ts, hub.tsUp, if (hub.tsUp == null) "Not set up" else "Trying")
570             }
571         }
572     }
573 }
574 
575 @Composable
576 private fun StartLine(icon: ImageVector, name: String, cc: Color, up: Boolean?, waiting: String) {
577     val c = Sb.c
578     Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(10.dp)) {
579         Ico(icon, 19.dp, if (up == true) cc else c.ink2, Modifier.breathe(up == false, 1200))
580         T(name, Modifier.weight(1f), size = 0.9f, weight = FontWeight.Bold)
581         when (up) {
582             true -> Tag("CONNECTED", cc)
583             false -> T(waiting, size = 0.78f, weight = FontWeight.Bold, color = c.ink2)
584             null -> T(waiting, size = 0.78f, weight = FontWeight.SemiBold, color = c.ink2)
585         }
586     }
587 }
588 
589 @Composable
590 private fun RoleCard(icon: ImageVector, title: String, points: List<String>, onClick: () -> Unit) {
591     val c = Sb.c
```

```
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    Row(
        Modifier
            .fillMaxWidth()
            .then(if (pressed) Modifier.ring(c.accentSoft, 18.dp).clip(RoundedCornerShape(18.dp)).background(c.base).sunk(c, 18.dp) else Modifier.bump(c, 18.dp))
            .press(true, Sfx.CONFIRM, src, onClick)
            .padding(16.dp),
        horizontalArrangement = Arrangement.spacedBy(13.dp),
    ) {
        Ico(icon, 34.dp, c.accent)
        Column(verticalArrangement = Arrangement.spacedBy(6.dp)) {
            T(title, size = 1.05f)
            Column {
                points.forEach { p -> T("•  $p", size = 0.84f, weight = FontWeight.Medium, color = c.ink2, lineHeight = 1.6f) }
            }
        }
    }
}

/** The bin's colour on the card and in the tiles: the status colour, or how full it is. */
fun statusColor(b: Bin, c: SbColors): Color = when (b.reading.status) {
    BinStatus.CALIBRATING, BinStatus.COLLECTING -> c.cal
    BinStatus.TAMPER -> c.bad
    BinStatus.LOCKED -> c.sim
    BinStatus.OK -> levelColor(b, c)
}

fun levelColor(b: Bin, c: SbColors): Color {
    val p = b.reading.fill
    return when {
        p >= b.cal.v("fill_th") -> c.bad
        p >= 50 -> c.warn
        else -> c.ok
    }
}

/** A short line for a stale bin: when its last reading came. */
fun lastSeen(b: Bin): String = if (b.lastSeen <= 0) "No reading yet" else
    "Last reading " + java.text.SimpleDateFormat("H:mm", java.util.Locale.getDefault()).format(java.util.Date(b.lastSeen))
```

Android App/app/src/main/java/smartbin/ui/Overview.kt

```
// ============================================================================
// ui/Overview.kt - the first tab: bin cards, the tile grid and collection priority
// Cards show fill, sensor tiles, status chips and the command keys for one
// bin. In the AUTO view, more than 2 bins switch to a compact tile grid. A key
// that is waiting for a cloud command shows a progress line along its edge
// (WaitKey) until the bin answers.
// ============================================================================
package smartbin.ui

import androidx.compose.runtime.setValue
import androidx.compose.runtime.collectAsState
import androidx.compose.foundation.Canvas
import androidx.compose.animation.core.Animatable
import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.RepeatMode
import androidx.compose.animation.core.animateFloat
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.infiniteRepeatable
import androidx.compose.animation.core.rememberInfiniteTransition
import androidx.compose.animation.core.tween
import androidx.compose.foundation.background
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.BoxWithConstraints
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.defaultMinSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.offset
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.runtime.Composable
```

```
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.State
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableLongStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.withFrameMillis
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.alpha
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.drawBehind
import androidx.compose.ui.draw.drawWithContent
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.RoundRect
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.graphics.PathMeasure
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.drawscope.DrawScope
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.IntOffset
import androidx.compose.ui.unit.dp
import smartbin.data.Bin
import smartbin.data.BinStatus
import smartbin.data.Hub
import smartbin.data.HubState
import smartbin.data.Source
import smartbin.data.v
import smartbin.notify.Sfx

@Composable
fun Overview(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val bins = hub.bins
    val n = bins.size
    val grid = vm.binView == BinView.GRID || (vm.binView == BinView.AUTO && n > 2)
    val list = bins.filter {
        when (vm.binFilter) {
            BinFilter.ALL -> true
            BinFilter.ACTION -> it.needsAction
            BinFilter.MODES -> it.simulating || it.calibrating
        }
    }
    ScrollPage {
        liveBanners(hub).forEach { Banner(it.tone, it.icon, it.title, it.small, iconMod = Modifier.shake(it.icon ===
Ic.tamper)) }
        val head: @Composable () -> Unit = {
            Row(Modifier.fillMaxWidth().defaultMinSize(minHeight = 40.dp), verticalAlignment = Alignment.CenterVertically,
horizontalArrangement = Arrangement.spacedBy(10.dp)) {
                Sh(Ic.bins, "Bins · $n", c.fill)
                Spacer(Modifier.weight(1f))
                if (n > 0) TwoWay {
                    Tw(grid, { vm.binView = BinView.GRID }, icon = Ic.grid, grow = null, minHeight = 36.dp)
                    Tw(!grid, { vm.binView = BinView.CARDS }, icon = Ic.layers, grow = null, minHeight = 36.dp)
                }
                if (vm.operator) AddMini { vm.addN = 1; vm.overlay = Overlay.AddBins }
            }
        }
        if (n == 0) {
            head()
            Card { Empty(Ic.bins, "No bins yet. Tap Add.") }
            return@ScrollPage
        }
        if (grid) {
            CollectNext(vm, bins)
            head()
            if (n > 2) Row(Modifier.fillMaxWidth().horizontalScroll(rememberScrollState()).padding(vertical = 2.dp),
horizontalArrangement = Arrangement.spacedBy(9.dp)) {
                FilterKey(Ic.layers, "All $n", vm.binFilter == BinFilter.ALL) { vm.binFilter = BinFilter.ALL }
                FilterKey(Ic.tamper, "Needs action ${bins.count { it.needsAction }}", vm.binFilter == BinFilter.ACTION,
Sb.c.tamper) { vm.binFilter = BinFilter.ACTION }
                FilterKey(Ic.flask, "Sim / Cal ${bins.count { it.simulating || it.calibrating }}", vm.binFilter ==
BinFilter.MODES, Sb.c.sim) { vm.binFilter = BinFilter.MODES }
            }
            if (list.isEmpty()) Card { Empty(Ic.check, "No bins match this filter.") }
            else TileGrid(list) { b -> Tile(b) { vm.overlay = Overlay.BinDetail(b.id) } }
            TileLegend()
        } else {
            head()
            bins.forEach { BinCard(vm, it, hub) }
```

```
            CollectNext(vm, bins)
        }
    }
}

@Composable
private fun AddMini(onClick: () -> Unit) {
    val c = Sb.c
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    Row(
        Modifier
            .height(40.dp)
            .then(if (pressed) Modifier.clip(RoundedCornerShape(20.dp)).background(c.base).sunk(c, 20.dp) else
Modifier.bump(c, 20.dp))
            .press(true, Sfx.OPEN, src, onClick)
            .padding(horizontal = 14.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(7.dp),
    ) {
        val t = if (pressed) c.accent else c.ink2
        Ico(Ic.plus, 17.dp, t)
        T("Add", size = 0.86f, weight = FontWeight.Bold, color = t)
    }
}

/** .btn.sm.binbtn: a key that stays pushed in while it is the chosen one. */
@Composable
fun FilterKey(icon: ImageVector?, text: String, on: Boolean, cc: Color = Sb.c.accent, simRing: Boolean = false, enabled:
Boolean = true, onClick: () -> Unit) {
    val c = Sb.c
    Box(if (simRing && !on) Modifier.ring(c.simSoft, 11.dp) else Modifier) {
        Btn(
            text, onClick, icon = icon, small = true, pressedIn = on, cc = if (on) cc else c.ink2,
            sound = Sfx.CHOICE, enabled = enabled,
        )
    }
}

@Composable
fun Empty(icon: ImageVector, text: String) {
    val c = Sb.c
    Column(Modifier.fillMaxWidth().padding(vertical = 30.dp, horizontal = 14.dp), horizontalAlignment =
Alignment.CenterHorizontally, verticalArrangement = Arrangement.spacedBy(10.dp)) {
        Ico(icon, 34.dp, c.ink2.soft(0.55f))
        T(text, size = 0.92f, weight = FontWeight.Bold, color = c.ink2, align = TextAlign.Center)
    }
}

// ---------- the bin card ----------

private fun band(f: Double) = when { f >= 1 -> 4; f >= .8 -> 3; f >= .5 -> 2; else -> 1 }

private fun lvColor(lv: Int, c: SbColors) = when (lv) { 0 -> c.lvCool; 1 -> c.lvLow; 2 -> c.lvMid; 3 -> c.lvHigh; else ->
c.lvCrit }

@Composable
fun BinCard(vm: AppViewModel, b: Bin, hub: HubState, inSheet: Boolean = false) {
    val c = Sb.c
    val r = b.reading
    val cal = b.cal
    val op = vm.operator
    val fillTh = cal.v("fill_th")
    val calOn = b.calibrating
    val simOn = b.simulating
    val hot = r.status != BinStatus.OK || r.fill >= fillTh
    val urgent = b.urgent
    val stale = b.stale
    val haze = if (urgent && !inSheet) {
        val t = rememberInfiniteTransition(label = "haze")
        t.animateFloat(2f, 7f, infiniteRepeatable(tween(1300), RepeatMode.Reverse), label = "haze").value
    } else 2f
    val outline = when {
        calOn && simOn -> listOf(c.simSoft to 4.dp, c.calSoft to 9.dp)
        calOn -> listOf(c.calSoft to 4.dp)
        simOn -> listOf(c.simSoft to 4.dp)
        else -> emptyList()
    }
    Column(
        Modifier
            .fillMaxWidth()
            .drawBehind {
                outline.forEach { (col, out) ->
                    val o = out.toPx()
                    drawRoundRect(
                        col, topLeft = Offset(-o + 1.5.dp.toPx(), -o + 1.5.dp.toPx()),
```

```
                        size = Size(size.width + 2 * o - 3.dp.toPx(), size.height + 2 * o - 3.dp.toPx()),
                        cornerRadius = CornerRadius(18.dp.toPx() + o), style = Stroke(3.dp.toPx()),
                    )
                }
            }
            .nm(c, out = if (inSheet) Out.O1 else Out.O2, ring = if (hot && !inSheet) c.accentSoft else null, ringWidth =
haze.dp)
            .padding(14.dp),
    ) {
        // Top: hero, name, fill
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            Box(Modifier.size(62.dp).well(c, 31.dp), contentAlignment = Alignment.Center) {
                if (calOn) Ico(Ic.calib, 46.dp, c.cal, Modifier.spin())
                else BinIcon(r.fill, statusColor(b, c), 46.dp)
            }
            Column(Modifier.weight(1f).padding(top = 4.dp)) {
                T(b.name, size = 1.2f, spacing = 0.02f, maxLines = 1)
                T(b.place, size = 0.86f, weight = FontWeight.Medium, color = c.ink2, maxLines = 1)
                if (stale) Row(Modifier.padding(top = 3.dp), verticalAlignment = Alignment.CenterVertically,
horizontalArrangement = Arrangement.spacedBy(5.dp)) {
                    Ico(Ic.clock, 13.dp, c.bad)
                    T(lastSeen(b), size = 0.74f, color = c.bad, maxLines = 1)
                }
            }
            Column(horizontalAlignment = Alignment.End, verticalArrangement = Arrangement.spacedBy(8.dp)) {
                Row(Modifier.alpha(if (stale) 0.42f else 1f), verticalAlignment = Alignment.Bottom) {
                    T(Math.round(r.fill).toString(), size = 2.45f, color = c.accent, tnum = true, lineHeight = 1f)
                    T("%", Modifier.padding(bottom = 3.dp), size = 2.45f * 0.42f, color = c.accent, lineHeight = 1f)
                }
                if (op) IconKey(Ic.gear, { vm.openBinEdit(b) }, tint = c.ink2, iconSize = 16.dp, label = "SET", sound =
Sfx.OPEN)
            }
        }
        // Six chips
        val live = b.source == Source.LIVE
        val cloud = b.source == Source.CLOUD
        val tsp = b.source == Source.THINGSPEAK
        val stKey = if (calOn) BinStatus.OK else r.status
        val stIcon = when (stKey) {
            BinStatus.TAMPER -> Ic.tamper; BinStatus.LOCKED -> Ic.lock(true); BinStatus.COLLECTING -> Ic.truck; else ->
Ic.check
        }
        val stColor = when (stKey) { BinStatus.LOCKED -> c.sim; BinStatus.TAMPER -> c.tamper; BinStatus.COLLECTING -> c.cal;
else -> c.ok }
        // While the bin is tipped but not yet latched, the firmware counts down before calling it tamper.
        val tilting = !calOn && r.tamperPhase == "COUNTDOWN" && stKey != BinStatus.TAMPER
        val stText = if (tilting) "TILTED" else when (stKey) { BinStatus.LOCKED -> "LOCKED"; BinStatus.TAMPER -> "TAMPER";
BinStatus.COLLECTING -> "PICKUP"; else -> "OK" }
        Column(Modifier.padding(top = 13.dp), verticalArrangement = Arrangement.spacedBy(7.dp)) {
            Row(horizontalArrangement = Arrangement.spacedBy(7.dp)) {
                Chip(Ic.lid(r.lidOpen), if (r.lidOpen) "OPEN" else "CLOSED", r.lidOpen, c.lid, Modifier.weight(1f))
                Chip(if (tilting) Ic.tamper else stIcon, stText, true, if (tilting) c.warn else stColor, Modifier.weight(1f),
Modifier.shake(stKey == BinStatus.TAMPER || tilting))
                Chip(
                    if (cloud) Ic.cloud else if (tsp) Ic.chart else Ic.wifi(live),
                    when { live -> "WI-FI"; cloud -> "CLOUD"; tsp -> "TSPEAK"; stale -> "OFFLINE"; else -> "DEMO" }, true,
                    when { live -> c.link; cloud -> c.cloud; tsp -> c.ts; stale -> c.tamper; else -> c.warn },
Modifier.weight(1f),
                )
            }
            Row(horizontalArrangement = Arrangement.spacedBy(7.dp)) {
                Chip(Ic.flask, "SIM", simOn, c.sim, Modifier.weight(1f))
                Chip(Ic.calib, "CAL", calOn, c.cal, Modifier.weight(1f))
                Chip(Ic.motion(r.motion), "MOTION", r.motion, c.motion, Modifier.weight(1f))
            }
        }
        // The fill bar with ticks at 50 and the limit
        val shown by animateFloatAsState(r.fill.coerceIn(0.0, 100.0).toFloat(), tween(600), label = "bar")
        val barColor = levelColor(b, c)
        Column(Modifier.alpha(if (stale) 0.42f else 1f)) {
            Box(
                Modifier
                    .padding(top = 16.dp, bottom = 6.dp)
                    .fillMaxWidth()
                    .height(24.dp)
                    .well(c, 12.dp, In.I2)
                    .drawBehind {
                        val p = 3.dp.toPx()
                        val w = (size.width * shown / 100f - 2 * p).coerceAtLeast(0f)
                        if (w > 0) drawRoundRect(barColor, topLeft = Offset(p, p), size = Size(w, size.height - 2 * p),
cornerRadius = CornerRadius(10.dp.toPx()))
                        listOf(50.0, fillTh).forEach { at ->
                            val x = (size.width * at / 100).toFloat()
                            drawRoundRect(
                                c.ink.soft(0.35f), topLeft = Offset(x - 1.5.dp.toPx(), 4.dp.toPx()),
                                size = Size(3.dp.toPx(), size.height - 8.dp.toPx()), cornerRadius =
```

```
    CornerRadius(2.dp.toPx()),
281                         )
282                     }
283                 },
284             )
285             Ticks(listOf(0.0, 50.0, fillTh, 100.0))
286             if (r.fault("HC-SR04")) T("Fill sensor not answering. The bar shows the last reading.", Modifier.padding(top =
    4.dp), size = 0.74f, weight = FontWeight.Bold, color = c.bad)
287             CardFoot(b, barColor)
288         }
289         // Four sensor tiles
290         Row(Modifier.padding(top = 12.dp).alpha(if (stale) 0.42f else 1f), horizontalArrangement =
    Arrangement.spacedBy(8.dp)) {
291             val tTh = cal.v("temp_th")
292             val hTh = cal.v("hum_th")
293             val wCap = cal.v("weight_cap")
294             val gTh = cal.v("gas_th")
295             // A sensor that stops answering shows -- and says so, instead of an old number.
296             val bme = r.fault("BME680")
297             val hx = r.fault("HX711")
298             val warm = r.warming && (b.sim.mask and 4) == 0
299             if (bme) SensorTile(Ic.temp(r.temp), c.temp, "--", "Not answering", true, 3, Modifier.weight(1f))
300             else SensorTile(Ic.temp(r.temp), c.temp, "${fmt(r.temp, 1)}°", "Temperature", r.temp >= tTh, if (r.temp < 22) 0
    else band(r.temp / tTh), Modifier.weight(1f))
301             if (bme) SensorTile(Ic.humid, c.humid, "--", "Not answering", true, 3, Modifier.weight(1f))
302             else SensorTile(Ic.humid, c.humid, "${Math.round(r.hum)}%", "Humidity", r.hum >= hTh,
303                 when { r.hum >= hTh - 10 -> 3; r.hum >= 60 -> 2; r.hum <= 35 -> 0; else -> 1 }, Modifier.weight(1f))
304             if (hx) SensorTile(Ic.weight(0.0), c.weight, "--", "Not answering", true, 3, Modifier.weight(1f))
305             else SensorTile(Ic.weight(r.weight / wCap), c.weight, "%.2fkg".format(r.weight / 1000), "Weight", r.weight >=
    wCap, band(r.weight / wCap), Modifier.weight(1f))
306             when {
307                 bme -> SensorTile(Ic.gas, c.gas, "--", "Not answering", true, 3, Modifier.weight(1f))
308                 warm -> SensorTile(Ic.gas, c.gas, r.iaqWarmS?.takeIf { it > 0 }?.let { "${it / 60}:%02d".format(it % 60) } ?:
    "...", "Air warming up", false, 0, Modifier.weight(1f))
309                 else -> SensorTile(Ic.gas, c.gas, Math.round(r.gas).toString(), "Air quality", r.gas >= gTh, band(r.gas /
    gTh), Modifier.weight(1f))
310             }
311         }
312         // LOCK or UNLOCK, RESET TAMPER (operator), COLLECT or DONE
313         // A command sent through Firebase or ThingSpeak holds every key until the bin confirms it.
314         val wait = hub.pending[b.id]
315         val busy = calOn || stale || wait != null
316         val locked = r.status == BinStatus.LOCKED
317         val tamper = r.status == BinStatus.TAMPER
318         val coll = r.status == BinStatus.COLLECTING
319         // Once the bin answers, the key that sent the command flashes green (confirmed) or red (timed out).
320         val done = vm.settled[b.id]
321         val fade = settleFade(done)
322         fun flashFor(vararg cmds: String) = if (done != null && done.cmd in cmds) Modifier.settleFlash(done.ok, fade, c) else
    Modifier
323         Row(Modifier.padding(top = 14.dp), horizontalArrangement = Arrangement.spacedBy(9.dp)) {
324             val m = Modifier.weight(1f)
325             val lm = m.then(flashFor("LOCK", "UNLOCK"))
326             val cm = m.then(flashFor("COLLECT", "COLLECT_DONE"))
327             if (wait != null && (wait.cmd == "LOCK" || wait.cmd == "UNLOCK")) WaitKey(wait, if (wait.cmd == "LOCK") "LOCKING"
    else "UNLOCKING", c.sim, m)
328             else if (locked) Btn("UNLOCK", { vm.command(b.id, "UNLOCK") }, lm, Ic.lock(false), Look.SOLID, c.sim, enabled
    = !busy, stack = true, sound = Sfx.UNLOCK)
329             else Btn("LOCK", { vm.command(b.id, "LOCK") }, lm, Ic.lock(true), Look.TINT, c.sim, enabled = !busy && !tamper,
    stack = true, sound = Sfx.LOCK)
330             if (op) Btn(
331                 "RESET TAMPER", { vm.command(b.id, "CLEAR_TAMPER") }, m.then(flashFor("CLEAR_TAMPER")), Ic.tamper, if
    (tamper) Look.DANGER else Look.TINT, c.tamper,
332                 enabled = tamper && !stale && wait == null, stack = true, sound = Sfx.CONFIRM,
333             )
334             if (wait != null && wait.cmd == "COLLECT") WaitKey(wait, "STARTING", c.link, m)
335             else if (coll) Btn("DONE", { vm.command(b.id, "COLLECT_DONE") }, cm, Ic.check, Look.SOLID, c.cal, enabled
    = !stale && wait == null, stack = true, sound = Sfx.DONE)
336             else Btn("COLLECT", { vm.command(b.id, "COLLECT") }, cm, Ic.truck, Look.TINT, c.link, enabled = !busy, stack =
    true, sound = Sfx.PICKUP)
337         }
338         if (wait != null) {
339             val via = if (wait.via == Source.THINGSPEAK) "ThingSpeak" else "Firebase"
340             T(
341                 "Sent through $via. The keys stay locked until ${b.name} confirms.", Modifier.padding(top =
    8.dp).fillMaxWidth(),
342                 size = 0.74f, weight = FontWeight.Bold, color = if (wait.via == Source.THINGSPEAK) c.ts else c.cloud, align =
    TextAlign.Center,
343             )
344         }
345     }
346 }
347
348 /** Corner radius of a stacked key, for the lines drawn round it. */
349 private val KEY_RADIUS = 13.dp
350
```

```
/**
 * A key held down while a cloud command waits for the bin: pushed in under
 * a light veil, its icon turning, and a line round its edge running down to
 * the deadline, moved every frame.
 */
@Composable
private fun WaitKey(p: smartbin.data.Pending, label: String, cc: Color, modifier: Modifier) {
    val c = Sb.c
    val clock = remember { mutableLongStateOf(System.currentTimeMillis()) }
    LaunchedEffect(p) {
        while (true) withFrameMillis { clock.longValue = System.currentTimeMillis() }
    }
    Btn(
        label, {}, modifier.drawWithContent {
            drawContent()
            val left = ((p.deadline - clock.longValue).toFloat() / (p.deadline - p.at).coerceAtLeast(1)).coerceIn(0f, 1f)
            drawRoundRect(c.base.soft(0.35f), cornerRadius = CornerRadius(KEY_RADIUS.toPx()))
            edgeLine(1f, cc.soft(0.22f))
            edgeLine(left, cc)
        },
        Ic.reload, Look.TINT, cc, stack = true, pressedIn = true, iconMod = Modifier.spin(true), sound = Sfx.DENIED,
    )
}

/** The part [part] (0 to 1) of a line round a key's edge, from its top-left corner. */
private fun DrawScope.edgeLine(part: Float, color: Color) {
    if (part <= 0f) return
    val w = 3.dp.toPx()
    val r = (KEY_RADIUS.toPx() - w / 2f).coerceAtLeast(0f)
    val whole = Path().apply { addRoundRect(RoundRect(w / 2f, w / 2f, size.width - w / 2f, size.height - w / 2f,
CornerRadius(r, r))) }
    val measure = PathMeasure().apply { setPath(whole, false) }
    val piece = Path()
    measure.getSegment(0f, measure.length * part, piece, true)
    drawPath(piece, color, style = Stroke(width = w, cap = StrokeCap.Round))
}

/** Runs from 1 to 0 over [SETTLE_MILLIS] each time a new answer arrives. */
@Composable
private fun settleFade(s: Settled?): State<Float> {
    val fade = remember(s) { Animatable(if (s == null) 0f else 1f) }
    LaunchedEffect(s) {
        if (s != null) fade.animateTo(0f, tween(SETTLE_MILLIS.toInt(), easing = LinearEasing))
    }
    return fade.asState()
}

/** A green or red wash and edge over a key, fading out as [fade] runs down. */
private fun Modifier.settleFlash(ok: Boolean, fade: State<Float>, c: SbColors): Modifier = drawWithContent {
    drawContent()
    val f = fade.value
    if (f <= 0f) return@drawWithContent
    val tone = if (ok) c.ok else c.bad
    drawRoundRect(tone.soft(0.28f * f), cornerRadius = CornerRadius(KEY_RADIUS.toPx()))
    edgeLine(1f, tone.soft(f))
}

/** .ticks: 0, 50, the limit and 100 under the bar, each centred on its mark. */
@Composable
private fun Ticks(at: List<Double>) {
    val c = Sb.c
    BoxWithConstraints(Modifier.fillMaxWidth().height(18.dp)) {
        val w = maxWidth
        at.forEachIndexed { i, v ->
            val label = Math.round(v).toString()
            val x = w * (v / 100).toFloat()
            val align = when (i) { 0 -> 0f; at.lastIndex -> 1f; else -> 0.5f }
            Box(Modifier.offset { IntOffset((x.toPx() - 30.dp.toPx() * align * 2).toInt(), 0) }.width(60.dp)) {
                T(
                    label, Modifier.fillMaxWidth(), size = 0.76f, weight = FontWeight.Bold, color = c.ink2, maxLines = 1,
                    align = when (i) { 0 -> TextAlign.Start; at.lastIndex -> TextAlign.End; else -> TextAlign.Center },
                )
            }
        }
    }
}

@Composable
private fun SensorTile(icon: ImageVector, cc: Color, value: String, label: String, alarm: Boolean, level: Int, modifier:
Modifier) {
    val c = Sb.c
    val surface = if (alarm) Modifier.brutal(c, 10.dp, c.bad, off = 3.dp) else Modifier.well(c, 13.dp)
    Column(
        modifier.then(surface).padding(top = 10.dp, bottom = 8.dp, start = 2.dp, end = 2.dp),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.spacedBy(4.dp),
```

```
    ) {
        Box(contentAlignment = Alignment.Center) {
            if (alarm) Box(Modifier.size(21.dp).breathe(true,
2600).clip(RoundedCornerShape(50)).background(Color.White.soft(0.38f)))
            Ico(icon, 21.dp, if (alarm) Color.White else cc)
        }
        T(value, size = 1.04f, color = if (alarm) Color.White else lvColor(level, c), tnum = true, maxLines = 1, lineHeight =
1.05f, fit = true)
        T(label, size = 0.66f, weight = FontWeight.Bold, color = if (alarm) Color.White else c.ink2, fit = true)
    }
}

// ---------- collection priority ----------

// Sort key for Collection priority. Full bins sort first (negative values),
// then by hours left to the fill threshold, then bins that are not filling.
// The fill rate comes from Hub.ratePerHour: the change over the last hour,
// needing at least 6 history points, so a new bin shows "Not filling" at first.
/** Lower is more urgent: a full bin, then hours to full, then bins that are not filling. */
private fun urgency(b: Bin): Double {
    val p = b.reading.fill
    val th = b.cal.v("fill_th")
    if (p >= th) return -1000 + p
    val rate = if (b.reading.status == BinStatus.COLLECTING) 0.0 else Hub.ratePerHour(b)
    return if (rate > 0.05) (th - p) / rate else 9999 - p
}

/** The last hour of fill as a small line, and how long ago the last reading came. */
@Composable
private fun CardFoot(b: Bin, cc: Color) {
    val c = Sb.c
    val history by Hub.history.collectAsState()
    var now by remember { mutableLongStateOf(System.currentTimeMillis()) }
    LaunchedEffect(Unit) {
        while (true) {
            kotlinx.coroutines.delay(1000)
            now = System.currentTimeMillis()
        }
    }
    val hour = 3_600_000L
    val pts = history[b.id].orEmpty().filter { now - it.t <= hour }
    Row(
        Modifier.fillMaxWidth().padding(top = 8.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(10.dp),
    ) {
        T("1 h", size = 0.68f, weight = FontWeight.Bold, color = c.ink2)
        Canvas(Modifier.weight(1f).height(22.dp)) {
            if (pts.size >= 2) {
                val line = Path()
                pts.forEachIndexed { k, s ->
                    val x = ((s.t - (now - hour)).toFloat() / hour * size.width).coerceIn(0f, size.width)
                    val y = size.height - (s.fill.coerceIn(0f, 100f) / 100f) * size.height
                    if (k == 0) line.moveTo(x, y) else line.lineTo(x, y)
                }
                drawPath(line, cc, style = Stroke(2.dp.toPx(), cap = StrokeCap.Round))
            }
        }
        val age = (now - b.lastSeen) / 1000
        T(
            when {
                b.lastSeen <= 0 -> "No reading yet"
                age < 5 -> "Updated just now"
                age < 60 -> "Updated $age s ago"
                age < 3600 -> "Updated ${age / 60} min ago"
                else -> "Updated ${age / 3600} h ago"
            },
            size = 0.7f, weight = FontWeight.SemiBold, color = if (b.stale) c.bad else c.ink2, maxLines = 1,
        )
    }
}

// The tags follow from this: under 2 h SOON, under 8 h TODAY, longer LATER,
// no rate IDLE, full NOW. The dashboard uses the same limits.
/** Time to full from how fast the bin has been filling. Returns the text and how near it is: 0 near, 1 mid, 2 far, 3 none.
*/
fun etaOf(b: Bin): Pair<String, Int> {
    val left = b.cal.v("fill_th") - b.reading.fill
    val rate = if (b.reading.status == BinStatus.COLLECTING) 0.0 else Hub.ratePerHour(b)
    if (left <= 0) return "Full now" to 0
    if (rate <= 0.05) return "Not filling" to 3
    val h = left / rate
    val t = if (h < 1) "~${Math.round(h * 60)} min" else "~${"%.1f".format(h)} h"
    return "$t to full" to when { h < 2 -> 0; h < 8 -> 1; else -> 2 }
}

```

```
@Composable
private fun CollectNext(vm: AppViewModel, bins: List<Bin>) {
    val c = Sb.c
    // The same order and tags as the dashboard: full bins, then the soonest to fill.
    val rows = bins.sortedBy { urgency(it) }
    val show = if (vm.prioAll) rows else rows.take(3)
    Card {
        Sh(Ic.rank, "Collection priority", c.fill)
        Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
            show.forEachIndexed { k, b ->
                val p = Math.round(b.reading.fill)
                val th = b.cal.v("fill_th")
                val (eta, near) = etaOf(b)
                val (tag, tagColor) = when {
                    p >= th -> "NOW" to c.bad
                    near == 3 -> "IDLE" to c.ink2
                    near == 0 -> "SOON" to c.bad
                    near == 1 -> "TODAY" to c.warn
                    else -> "LATER" to c.ok
                }
                val edge = when (near) { 0 -> c.bad; 1 -> c.warn; 2 -> c.ok; else -> c.ink2 }
                val src = remember { MutableInteractionSource() }
                val pressed by src.collectIsPressedAsState()
                Row(
                    Modifier
                        .fillMaxWidth()
                        .then(if (pressed) Modifier.clip(RoundedCornerShape(14.dp)).background(c.base).sunk(c, 14.dp) else
Modifier.bump(c, 14.dp))
                        .drawBehind { drawRect(edge, size = Size(5.dp.toPx(), size.height)) }
                        .press(true, Sfx.OPEN, src) { vm.overlay = Overlay.BinDetail(b.id) }
                        .padding(start = 16.dp, end = 13.dp, top = 11.dp, bottom = 11.dp),
                    verticalAlignment = Alignment.CenterVertically,
                    horizontalArrangement = Arrangement.spacedBy(11.dp),
                ) {
                    Box(Modifier.size(28.dp).well(c, 14.dp), contentAlignment = Alignment.Center) { T("${k + 1}", size =
0.9f) }
                    Column(Modifier.weight(1f)) {
                        T(b.name, weight = FontWeight.Bold, maxLines = 1)
                        Row(Modifier.padding(top = 2.dp), verticalAlignment = Alignment.CenterVertically,
horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                            Ico(Ic.stopwatch, 12.dp, c.ink2)
                            T(eta, size = 0.76f, weight = FontWeight.SemiBold, color = c.ink2, maxLines = 1)
                        }
                    }
                    T("$p%", tnum = true)
                    Tag(tag, tagColor)
                }
            }
        }
        if (rows.size > 3) Btn(
            if (vm.prioAll) "Show top 3" else "Show all ${rows.size} bins", { vm.prioAll = !vm.prioAll },
            Modifier.fillMaxWidth(), if (vm.prioAll) Ic.chev else Ic.rank, small = true,
        )
        Hint("Full bins first, then the ones that will fill soonest. Time to full comes from how fast each bin filled over
the last hour. Tap a row to open that bin.")
    }
}

// ---------- many bins: the grid ----------

@Composable
private fun TileGrid(bins: List<Bin>, tile: @Composable (Bin) -> Unit) {
    val cols = 3
    Column(verticalArrangement = Arrangement.spacedBy(12.dp)) {
        bins.chunked(cols).forEach { row ->
            Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
                row.forEach { b -> Box(Modifier.weight(1f)) { tile(b) } }
                repeat(cols - row.size) { Spacer(Modifier.weight(1f)) }
            }
        }
    }
}

@Composable
private fun Tile(b: Bin, onClick: () -> Unit) {
    val c = Sb.c
    val r = b.reading
    val calOn = b.calibrating
    val simOn = b.simulating
    val lc = levelColor(b, c)
    val stale = b.stale
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    val ringW = if (b.urgent) {
        val t = rememberInfiniteTransition(label = "thaze")
        t.animateFloat(2.5f, 5f, infiniteRepeatable(tween(1100), RepeatMode.Reverse), label = "thaze").value
```

```
    } else 0f
    val fill: Brush = when {
        calOn && simOn -> Brush.linearGradient(
            0f to mix(c.sim, 0.22f, c.base), 0.5f to mix(c.sim, 0.22f, c.base), 0.5f to mix(c.cal, 0.22f, c.base), 1f to mix(c.cal, 0.22f, c.base),
        )
        calOn -> Brush.linearGradient(listOf(mix(c.cal, 0.18f, c.base), mix(c.cal, 0.18f, c.base)))
        simOn -> Brush.linearGradient(listOf(mix(c.sim, 0.18f, c.base), mix(c.sim, 0.18f, c.base)))
        else -> Brush.linearGradient(listOf(c.base, c.base))
    }
    Box(Modifier.fillMaxWidth().alpha(if (stale) 0.5f else 1f)) {
        Column(
            Modifier
                .fillMaxWidth()
                .then(if (pressed) Modifier else Modifier.raised(c, 16.dp, ring = if (b.urgent) c.bad.soft(1f - (ringW - 2.5f) / 2.5f * 0.55f) else null, ringWidth = ringW.dp))
                .clip(RoundedCornerShape(16.dp))
                .background(fill)
                .then(if (pressed) Modifier.sunk(c, 16.dp) else Modifier)
                .press(true, Sfx.OPEN, src, onClick)
                .padding(start = 6.dp, end = 6.dp, top = 10.dp, bottom = 9.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.spacedBy(3.dp),
        ) {
            BinIcon(r.fill, lc, 42.dp)
            T(b.name, size = 0.84f, maxLines = 1)
            T("${Math.round(r.fill)}%", size = 1.22f, color = lc, tnum = true, lineHeight = 1.05f)
            Row(Modifier.height(15.dp), horizontalArrangement = Arrangement.spacedBy(4.dp, Alignment.CenterHorizontally)) {
                if (r.status == BinStatus.TAMPER) Ico(Ic.tamper, 15.dp, c.tamper, Modifier.shake())
                if (r.status == BinStatus.LOCKED) Ico(Ic.lock(true), 15.dp, c.sim)
                if (r.status == BinStatus.COLLECTING) Ico(Ic.truck, 15.dp, c.cal)
                if (simOn) Ico(Ic.flask, 15.dp, c.sim)
                if (calOn) Ico(Ic.calib, 15.dp, c.cal)
                if (r.lidOpen && r.status != BinStatus.COLLECTING) Ico(Ic.lid(true), 15.dp, c.lid)
                if (stale) Ico(Ic.wifi(false), 15.dp, c.tamper)
            }
        }
        if (b.sensorOver) Adot(Modifier.align(Alignment.TopEnd).padding(6.dp))
    }
}

/** .adot: the brutal "!" for a sensor over its limit. */
@Composable
fun Adot(modifier: Modifier = Modifier, small: Boolean = false) {
    val c = Sb.c
    Box(modifier.size(if (small) 16.dp else 19.dp).brutal(c, 5.dp, c.bad, off = 2.dp), contentAlignment = Alignment.Center) {
        T("!", size = (if (small) 10f else 11f) / 14f, weight = FontWeight.Black, color = Color.White, lineHeight = 1f)
    }
}

@OptIn(androidx.compose.foundation.layout.ExperimentalLayoutApi::class)
@Composable
private fun TileLegend() {
    val c = Sb.c
    @Composable
    fun key(fill: Brush, text: String, ring: Color? = null) {
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(5.dp)) {
            Box(Modifier.size(14.dp).then(if (ring != null) Modifier.ring(ring, 4.dp) else Modifier).clip(RoundedCornerShape(4.dp)).background(fill))
            T(text, size = 0.72f, weight = FontWeight.Bold, color = c.ink2)
        }
    }
    val solid = { col: Color -> Brush.linearGradient(listOf(col, col)) }
    androidx.compose.foundation.layout.FlowRow(
        horizontalArrangement = Arrangement.spacedBy(13.dp),
        verticalArrangement = Arrangement.spacedBy(7.dp),
    ) {
        key(solid(mix(c.sim, 0.3f, c.base)), "Simulation")
        key(solid(mix(c.cal, 0.3f, c.base)), "Calibration")
        key(
            Brush.linearGradient(0f to mix(c.sim, 0.34f, c.base), 0.5f to mix(c.sim, 0.34f, c.base), 0.5f to mix(c.cal, 0.34f, c.base), 1f to mix(c.cal, 0.34f, c.base)),
            "Both",
        )
        key(solid(c.base), "Full or tamper", ring = c.bad)
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(5.dp)) {
            Adot(small = true)
            T("Sensor over limit", size = 0.72f, weight = FontWeight.Bold, color = c.ink2)
        }
    }
}
```

Android App/app/src/main/java/smartbin/ui/Charts.kt

```
// ============================================================================
```

```kotlin
// ui/Charts.kt - the Analytics tab: fill level and environment line charts
// Data comes from Hub.history (one sample every 10 s, kept for 24 h). Each
// series is cut to the chosen time range and averaged into at most 60 points
// so long ranges stay fast to draw. "Collecting readings for this chart"
// means fewer than 2 points fall inside the range yet.
// ============================================================================
package smartbin.ui

import androidx.compose.foundation.Canvas
import androidx.compose.foundation.background
import androidx.compose.foundation.horizontalScroll
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.ExperimentalLayoutApi
import androidx.compose.foundation.layout.FlowRow
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.defaultMinSize
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.navigationBarsPadding
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.statusBarsPadding
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.remember
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.Path
import androidx.compose.ui.graphics.PathEffect
import androidx.compose.ui.graphics.StrokeCap
import androidx.compose.ui.graphics.StrokeJoin
import androidx.compose.ui.graphics.drawscope.DrawScope
import androidx.compose.ui.graphics.drawscope.Stroke
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.text.TextMeasurer
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.drawText
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.rememberTextMeasurer
import androidx.compose.ui.unit.Dp
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import smartbin.data.Bin
import smartbin.data.Hub
import smartbin.data.HubState
import smartbin.data.Sample
import smartbin.data.v
import smartbin.notify.Sfx
import java.text.SimpleDateFormat
import java.util.Date
import java.util.Locale

class Series(val color: Color, val pts: List<Pair<Long, Float>>, val dash: Int = 0, val area: Boolean = false)
class Mark(val v: Float, val text: String)

/** One chart: its scale, lines, limit lines, legend and the line of text under it. */
class ChartSpec(
    val min: Float, val max: Float, val ticks: List<Float>, val series: List<Series>, val marks: List<Mark>,
    val legend: List<Triple<Color, Int, String>>, val note: String? = null,
)

private const val POINTS = 60

/** History of one bin over the window, averaged into at most 60 points so the line stays smooth. */
private fun window(h: List<Sample>, from: Long, to: Long, pick: (Sample) -> Float): List<Pair<Long, Float>> {
    val inside = h.filter { it.t in from..to }
    if (inside.size <= POINTS) return inside.map { it.t to pick(it) }
    val span = (to - from).toDouble() / POINTS
    return inside.groupBy { ((it.t - from) / span).toInt().coerceIn(0, POINTS - 1) }
        .toSortedMap()
        .map { (_, g) -> g.map { it.t }.average().toLong() to g.map(pick).average().toFloat() }
```

```
}

private val TEMP = Triple(0f, 85f, listOf(0f, 20f, 40f, 60f, 85f))
private val HUM = Triple(0f, 100f, listOf(0f, 25f, 50f, 75f, 100f))
private val PRESS = Triple(300f, 1100f, listOf(300f, 500f, 700f, 900f, 1100f))
private val GAS = Triple(0f, 500f, listOf(0f, 100f, 200f, 300f, 400f, 500f))

@Composable
fun fillSpec(vm: AppViewModel, hub: HubState, history: Map<String, List<Sample>>): ChartSpec {
    val c = Sb.c
    val now = System.currentTimeMillis()
    val from = now - vm.rangeMin * 60_000L
    val bins = hub.bins
    val all = vm.chartBin == null || bins.none { it.id == vm.chartBin }
    val ids = if (all) bins.sortedByDescending { it.reading.fill }.take(5) else bins.filter { it.id == vm.chartBin }
    val th = ids.firstOrNull()?.cal?.v("fill_th")?.toFloat() ?: 80f
    val series = ids.mapIndexed { k, b ->
        Series(binColor(bins.indexOf(b), c.isDark), window(history[b.id].orEmpty(), from, now) { it.fill }, if (all) k % 3 else 0, k == 0)
    }
    val legend = ids.mapIndexed { k, b -> Triple(binColor(bins.indexOf(b), c.isDark), if (all) k % 3 else 0, b.name) } +
        Triple(c.bad, 1, "Threshold")
    val note = if (all && bins.size > 5) "The 5 fullest of ${bins.size}. Pick a bin to see any other." else null
    return ChartSpec(0f, 100f, listOf(0f, 25f, 50f, 75f, 100f), series, listOf(Mark(th, "ALERT LIMIT ${th.toInt()}%")), legend, note)
}

@Composable
fun envSpec(vm: AppViewModel, hub: HubState, history: Map<String, List<Sample>>): ChartSpec? {
    val c = Sb.c
    val b = vm.binFor(vm.envBin, hub.bins) ?: return null
    val now = System.currentTimeMillis()
    val from = now - vm.rangeMin * 60_000L
    val h = history[b.id].orEmpty()
    val tTh = b.cal.v("temp_th").toFloat()
    val hTh = b.cal.v("hum_th").toFloat()
    val gTh = b.cal.v("gas_th").toFloat()
    val r = b.reading
    fun one(scale: Triple<Float, Float, List<Float>>, col: Color, name: String, unit: String, cur: String, lim: Float?, pick: (Sample) -> Float, note: String): ChartSpec {
        val legend = mutableListOf(Triple(col, 0, "$name, now **$cur $unit**"))
        if (lim != null) legend += Triple(c.bad, 1, "Alert limit ${lim.toInt()} $unit")
        legend += Triple(Color.Transparent, -1, "Scale ${scale.first.toInt()} to ${scale.second.toInt()} $unit")
        return ChartSpec(
            scale.first, scale.second, scale.third, listOf(Series(col, window(h, from, now, pick), area = true)),
            if (lim == null) emptyList() else listOf(Mark(lim, "ALERT LIMIT ${lim.toInt()} $unit")), legend, note,
        )
    }
    return when (vm.envMode) {
        EnvMode.TEMP -> one(TEMP, c.temp, "Temperature", "C", fmt(r.temp, 1), tTh, { it.temp },
            "Temperature of ${b.name} in degrees C, after the offset from Settings. The scale runs to 85 C, the most the BME680 can read.")
        EnvMode.HUMID -> one(HUM, c.humid, "Humidity", "%", fmt(r.hum, 0), hTh, { it.hum },
            "Relative humidity of ${b.name}, 0 to 100 percent, the full BME680 range. The dashed line is its alert limit, set in Settings.")
        EnvMode.GAS -> one(GAS, c.gas, "Air quality", "IAQ", fmt(r.gas, 0), gTh, { it.gas },
            "Air quality index of ${b.name} from the BME680, 0 to 500, lower is cleaner. The dashed line is its alert limit.")
        EnvMode.PRESS -> one(PRESS, c.press, "Pressure", "hPa", fmt(r.press, 1), null, { it.press },
            "Air pressure around ${b.name} on the BME680 range of 300 to 1100 hPa, so normal weather looks flat. Charted here only, with no tile and no alert.")
        EnvMode.ALL -> ChartSpec(
            0f, 200f, listOf(0f, 50f, 100f, 150f, 200f),
            listOf(
                Series(c.temp, window(h, from, now) { it.temp / tTh * 100 }, area = true),
                Series(c.gas, window(h, from, now) { it.gas / gTh * 100 }, dash = 1),
                Series(c.humid, window(h, from, now) { it.hum / hTh * 100 }, dash = 2),
            ),
            listOf(Mark(100f, "ALERT LIMIT (100%)")),
            listOf(
                Triple(c.temp, 0, "Temperature, limit ${tTh.toInt()} C"),
                Triple(c.humid, 2, "Humidity, limit ${hTh.toInt()} %RH"),
                Triple(c.gas, 1, "Air quality, limit ${gTh.toInt()} IAQ"),
                Triple(c.bad, 1, "Alert limit (100%)"),
            ),
            "Temperature, humidity and air quality are each drawn as a share of their own limit, so the dashed line at 100% is the limit for all three. The scale runs to 200%, which keeps that line in the middle. Pick one reading for real units.",
        )
    }
}

/**
 * The mockup's lineChart(): grid lines with their values, the lines with a
 * soft fill under the first, a dot at the newest point, the dashed alert
 * limit with its label, and the start time and "now" along the bottom.
 */
```

```
@Composable
fun LineChart(spec: ChartSpec, from: Long, modifier: Modifier = Modifier, height: Dp = 190.dp) {
    val c = Sb.c
    val measurer = rememberTextMeasurer()
    val start = remember(from / 60_000) { SimpleDateFormat("HH:mm", Locale.getDefault()).format(Date(from)) }
    Box(modifier.fillMaxWidth().then(if (height > 0.dp) Modifier.height(height) else Modifier.fillMaxSize()).well(c, 14.dp))
{
        Canvas(Modifier.fillMaxSize()) { chart(spec, from, start, measurer, c) }
        if (spec.series.all { it.pts.size < 2 }) {
            Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                T("Collecting readings for this chart", size = 0.8f, weight = FontWeight.Bold, color = c.ink2)
            }
        }
    }
}

private fun DrawScope.chart(spec: ChartSpec, from: Long, start: String, tm: TextMeasurer, c: SbColors) {
    val to = System.currentTimeMillis()
    val l = 32.dp.toPx()
    val r = 12.dp.toPx()
    val t = 12.dp.toPx()
    val b = 20.dp.toPx()
    val pw = size.width - l - r
    val ph = size.height - t - b
    fun y(v: Float) = t + ph - ((v.coerceIn(spec.min, spec.max) - spec.min) / (spec.max - spec.min)) * ph
    fun x(time: Long) = l + ((time - from).toFloat() / (to - from).toFloat()).coerceIn(0f, 1f) * pw
    val label = TextStyle(fontFamily = Inter, fontSize = 9.5.sp, fontWeight = FontWeight.Bold, color = c.ink2)
    spec.ticks.forEach { v ->
        val yy = y(v)
        drawLine(c.ink2.soft(0.22f), Offset(l, yy), Offset(size.width - r, yy), 1.dp.toPx())
        val m = tm.measure(Math.round(v).toString(), label)
        drawText(m, topLeft = Offset(l - 7.dp.toPx() - m.size.width, yy - m.size.height / 2f))
    }
    val dashes = listOf(null, floatArrayOf(7f, 5f), floatArrayOf(2.5f, 4.5f))
    spec.series.forEach { s ->
        if (s.pts.size < 2) return@forEach
        val line = Path()
        s.pts.forEachIndexed { k, (time, v) -> if (k == 0) line.moveTo(x(time), y(v)) else line.lineTo(x(time), y(v)) }
        if (s.area) {
            val area = Path().apply {
                addPath(line)
                lineTo(x(s.pts.last().first), t + ph)
                lineTo(x(s.pts.first().first), t + ph)
                close()
            }
            drawPath(area, Brush.verticalGradient(listOf(s.color.soft(0.24f), s.color.copy(alpha = 0f)), startY = t, endY = t
+ ph))
        }
        val d = dashes[s.dash.coerceIn(0, 2)]
        drawPath(
            line, s.color,
            style = Stroke(
                2.4.dp.toPx(), cap = StrokeCap.Round, join = StrokeJoin.Round,
                pathEffect = d?.let { PathEffect.dashPathEffect(floatArrayOf(it[0].dp.toPx(), it[1].dp.toPx())) },
            ),
        )
        val end = Offset(x(s.pts.last().first), y(s.pts.last().second))
        drawCircle(s.color.soft(0.22f), 7.5.dp.toPx(), end)
        drawCircle(s.color, 3.8.dp.toPx(), end)
    }
    val markStyle = TextStyle(fontFamily = Inter, fontSize = 8.6.sp, fontWeight = FontWeight.ExtraBold, color = c.bad)
    spec.marks.forEach { m ->
        val my = y(m.v)
        drawLine(c.bad, Offset(l, my), Offset(size.width - r, my), 1.6.dp.toPx(), pathEffect =
PathEffect.dashPathEffect(floatArrayOf(7.dp.toPx(), 6.dp.toPx())))
        val txt = tm.measure(m.text, markStyle)
        val pad = 7.dp.toPx()
        val box = Size(txt.size.width + pad * 2, 14.dp.toPx())
        val tl = Offset(l + 8.dp.toPx(), my - 17.dp.toPx())
        drawRoundRect(c.well, tl, box, CornerRadius(4.dp.toPx()))
        drawRoundRect(c.bad, tl, box, CornerRadius(4.dp.toPx()), style = Stroke(1.2.dp.toPx()))
        drawText(txt, topLeft = Offset(tl.x + pad, tl.y + (box.height - txt.size.height) / 2f))
    }
    val bottom = TextStyle(fontFamily = Inter, fontSize = 9.sp, fontWeight = FontWeight.Bold, color = c.ink2)
    val st = tm.measure(start, bottom)
    drawText(st, topLeft = Offset(l, size.height - st.size.height - 2.dp.toPx()))
    val nw = tm.measure("now", bottom.copy(color = c.ok))
    drawText(nw, topLeft = Offset(size.width - r - nw.size.width, size.height - nw.size.height - 2.dp.toPx()))
}

@OptIn(ExperimentalLayoutApi::class)
@Composable
fun Legend(items: List<Triple<Color, Int, String>>, modifier: Modifier = Modifier) {
    val c = Sb.c
    FlowRow(modifier, horizontalArrangement = Arrangement.spacedBy(14.dp), verticalArrangement = Arrangement.spacedBy(6.dp))
{
```

```
        items.forEach { (col, dash, text) ->
            Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                if (dash >= 0) Swatch(col, dash)
                T(text, size = 0.76f, weight = FontWeight.Bold, color = c.ink2, bold = c.ink)
            }
        }
    }
}

@Composable
fun RangeSeg(vm: AppViewModel) {
    val c = Sb.c
    TwoWay(full = true) {
        listOf(15 to "15 m", 60 to "1 h", 360 to "6 h", 1440 to "24 h").forEach { (m, label) ->
            Tw(vm.rangeMin == m, { vm.rangeMin = m }, label, cc = c.fill)
        }
    }
}

@Composable
fun EnvSeg(vm: AppViewModel) {
    val c = Sb.c
    TwoWay(full = true) {
        listOf(
            Triple(EnvMode.ALL, "ALL", Ic.layers to c.ink),
            Triple(EnvMode.TEMP, "TEMPERATURE", Ic.temp(30.0) to c.temp),
            Triple(EnvMode.HUMID, "HUMIDITY", Ic.humid to c.humid),
            Triple(EnvMode.GAS, "AIR QUALITY", Ic.gas to c.gas),
            Triple(EnvMode.PRESS, "PRESSURE", Ic.press to c.press),
        ).forEach { (m, label, ic) ->
            Tw(
                vm.envMode == m, { vm.envMode = m }, label, ic.first, ic.second,
                grow = if (m == EnvMode.ALL) 0.75f else 1f, stack = true, minHeight = 52.dp, textSize = 0.62f,
            )
        }
    }
}

/**
 * The bin selector: one row of keys while the bins fit, otherwise a picker
 * that opens a sheet. [allowAll] adds ALL BINS for the fill chart.
 */
@Composable
fun BinSeg(vm: AppViewModel, hub: HubState, forWhat: PickFor, selected: String?, allowAll: Boolean, locked: Boolean = false,
onPick: (String?) -> Unit) {
    val c = Sb.c
    val bins = hub.bins
    val sel = if (allowAll) selected?.takeIf { id -> bins.any { it.id == id } } else (vm.binFor(selected, bins)?.id)
    if (bins.size <= (if (allowAll) 3 else 4)) {
        Row(Modifier.fillMaxWidth().horizontalScroll(rememberScrollState()).padding(vertical = 4.dp, horizontal = 2.dp),
horizontalArrangement = Arrangement.spacedBy(9.dp)) {
            if (allowAll) FilterKey(Ic.layers, "ALL BINS", sel == null) { onPick(null) }
            bins.forEachIndexed { i, b ->
                FilterKey(Ic.fill, b.name, sel == b.id, binColor(i, c.isDark), simRing = b.simulating, enabled = !locked ||
sel == b.id) { onPick(b.id) }
            }
        }
        return
    }
    val b = bins.firstOrNull { it.id == sel }
    val i = bins.indexOf(b)
    val src = remember { MutableInteractionSource() }
    val pressed by src.collectIsPressedAsState()
    Row(
        Modifier
            .fillMaxWidth()
            .defaultMinSize(minHeight = 54.dp)
            .graphicsLayer { alpha = if (locked) 0.6f else 1f }
            .then(if (pressed) Modifier.clip(RoundedCornerShape(14.dp)).background(c.base).sunk(c, 14.dp) else
Modifier.bump(c, 14.dp, ring = c.accentSoft))
            .press(!locked, Sfx.OPEN, src) { vm.overlay = Overlay.Pick(forWhat) }
            .padding(start = 8.dp, end = 12.dp, top = 6.dp, bottom = 6.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(10.dp),
    ) {
        if (b != null) BinIcon(b.reading.fill, binColor(i, c.isDark), 34.dp)
        else Box(Modifier.size(34.dp), contentAlignment = Alignment.Center) { Ico(Ic.layers, 21.dp, c.ink2) }
        Column(Modifier.weight(1f)) {
            T(b?.name ?: "All bins", size = 0.95f, maxLines = 1)
            T(
                if (b != null) "${b.place} · ${Math.round(b.reading.fill)}%" + modeTxt(b) else if (bins.size > 5) "The 5
fullest of ${bins.size} are drawn" else "${bins.size} bins",
                size = 0.76f, weight = FontWeight.SemiBold, color = c.ink2, maxLines = 1,
            )
        }
        T((if (b != null) "${i + 1} of ${bins.size}" else "") + if (locked) "\nlocked" else "", size = 0.72f, color = c.ink2,
```

```
    align = androidx.compose.ui.text.style.TextAlign.End)
        Ico(Ic.chev, 18.dp, c.ink2)
    }
}

fun modeTxt(b: Bin): String {
    val cal = b.calibrating
    val sim = b.simulating
    return when { cal && sim -> " · sim + cal"; cal -> " · calibrating"; sim -> " · simulation"; else -> "" }
}

@Composable
fun Charts(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val history by Hub.history.collectAsState()
    val from = System.currentTimeMillis() - vm.rangeMin * 60_000L
    ScrollPage {
        Card {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Sh(Ic.fill, "Fill level", c.fill, Modifier.weight(1f))
                IconKey(Ic.expand, { vm.fullscreen = "fill" }, tint = c.ink2, size = 40.dp, iconSize = 18.dp, radius = 12.dp,
    sound = Sfx.OPEN)
            }
            RangeSeg(vm)
            BinSeg(vm, hub, PickFor.CHART, vm.chartBin, allowAll = true) { vm.chartBin = it }
            val f = fillSpec(vm, hub, history)
            LineChart(f, from)
            Legend(f.legend)
            f.note?.let { Hint(it) }
        }
        val e = envSpec(vm, hub, history)
        if (e != null) Card {
            Row(verticalAlignment = Alignment.CenterVertically) {
                Sh(Ic.temp(30.0), "Environment", c.temp, Modifier.weight(1f))
                IconKey(Ic.expand, { vm.fullscreen = "env" }, tint = c.ink2, size = 40.dp, iconSize = 18.dp, radius = 12.dp,
    sound = Sfx.OPEN)
            }
            EnvSeg(vm)
            BinSeg(vm, hub, PickFor.ENV, vm.envBin, allowAll = false) { vm.envBin = it }
            LineChart(e, from)
            Legend(e.legend)
            e.note?.let { Hint(it) }
        }
    }
}

/** The sideways chart: a bar with the title, its picker and controls, then the chart over the whole width. */
@Composable
fun FullChart(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val history by Hub.history.collectAsState()
    val env = vm.fullscreen == "env"
    val from = System.currentTimeMillis() - vm.rangeMin * 60_000L
    val spec = if (env) envSpec(vm, hub, history) else fillSpec(vm, hub, history)
    Column(
        Modifier.fillMaxSize().statusBarsPadding().navigationBarsPadding().padding(horizontal = 12.dp, vertical = 8.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        Row(
            Modifier.fillMaxWidth().bump(c, 14.dp).padding(start = 6.dp, end = 6.dp, top = 5.dp, bottom = 5.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(10.dp),
        ) {
            val name = if (env) vm.binFor(vm.envBin, hub.bins)?.name.orEmpty() else hub.bins.firstOrNull { it.id ==
    vm.chartBin }?.name ?: "All bins"
            Row(
                Modifier.defaultMinSize(minHeight = 44.dp).clip(RoundedCornerShape(12.dp))
                    .press(true, Sfx.OPEN) { vm.overlay = Overlay.Pick(if (env) PickFor.ENV else
    PickFor.CHART) }.padding(horizontal = 8.dp),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(10.dp),
            ) {
                Sh(if (env) Ic.temp(30.0) else Ic.fill, if (env) "Environment" else "Fill level", if (env) c.temp else
    c.fill)
                T(name, size = 0.86f, maxLines = 1)
                Ico(Ic.chev, 16.dp, c.ink2)
            }
            Box(Modifier.weight(1f)) { if (env) EnvSeg(vm) else RangeSeg(vm) }
            IconKey(Ic.shrink, { vm.fullscreen = null }, tint = c.ink2, size = 40.dp, iconSize = 18.dp, radius = 12.dp, sound
    = Sfx.BACK)
        }
        if (spec != null) {
            Box(Modifier.weight(1f).fillMaxWidth(), contentAlignment = Alignment.Center) {
                LineChart(spec, from, Modifier.widthIn(max = 1400.dp), height = 0.dp)
            }
            Legend(spec.legend, Modifier.padding(horizontal = 6.dp))
```

```
        }
    }
}
```

Android App/app/src/main/java/smartbin/ui/Settings.kt

```
// ========================================================================
// ui/Settings.kt - the Settings tab, plus the Connection and Bluetooth pages
// Settings edits one bin's calibration (Cal keys such as d_empty, fill_th,
// temp_off). Fields are read-only until an operator turns calibration on,
// and the Save bar appears only when something changed.
// Connection sets up the Firebase account and ThingSpeak keys and shows which
// of the three routes is answering. Bluetooth walks through the Wi-Fi setup
// of a bin (pair, pick network, send, confirm).
// ========================================================================
package smartbin.ui

import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.defaultMinSize
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.alpha
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.drawBehind
import androidx.compose.ui.geometry.CornerRadius
import androidx.compose.ui.geometry.Offset
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import smartbin.data.Bin
import smartbin.data.Cal
import smartbin.data.HubState
import smartbin.data.Link
import smartbin.data.Source
import smartbin.data.v
import smartbin.notify.Sfx

@Composable
fun SettingsPage(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val b = vm.binFor(vm.selSet, hub.bins)
    val scroll = rememberScrollState()
    val calHere = b != null && hub.calibBin == b.id
    val on = vm.operator && calHere && vm.edit != null
    val showSave = on && vm.dirty
    Box(Modifier.fillMaxSize()) {
        ScrollPage(state = scroll, bottomExtra = if (showSave) 72.dp else 0.dp) {
            Sh(Ic.sliders, "Settings", c.cal)
            if (b == null) {
                Card { Empty(Ic.bins, "No bins yet.") }
            } else {
                // Calibration runs on one bin at a time, so the picker is held while it is on.
                BinSeg(vm, hub, PickFor.SET, vm.selSet, allowAll = false, locked = hub.calibBin != null) {
                    vm.selSet = it
                    vm.pinShown = false
                }
                // The Collect PIN sits at the top for both roles, as on the dashboard.
                PinCard(vm, b)
                if (vm.operator) CalBar(vm, b, on, hub) else ReadOnlyNote()
                val v: Cal = if (on) vm.edit!! else b.cal
                val keys = SECTION_KEYS + if (vm.operator) listOf("touch") else emptyList()
                val allOpen = keys.all { it in vm.open }
```

```
                Btn(
                    if (allOpen) "HIDE ALL" else "SHOW ALL", { vm.open = if (allOpen) emptySet() else keys.toSet() },
                    Modifier.fillMaxWidth(), Ic.layers, Look.TINT, small = true, sound = if (allOpen) Sfx.BACK else Sfx.OPEN,
                )
                Sections(vm, b, v, on)
            }
            if (b != null) {
                Row(Modifier.padding(top = 4.dp)) { Sh(Ic.reload, "Bin tools") }
                LRow(Ic.reload, "Reload from bin", "Read the values ${b.name} is running on", onClick = { vm.reload(b) },
chevron = false, sound = Sfx.TAP)
            }
            // The app's own settings live behind the gear key in the header.
            LRow(Ic.gear, "App settings", "Sound, vibration, battery, links and demo options. Also the gear at the top
right.", onClick = { vm.go(Screen.APP) })
        }
        if (showSave) {
            SaveBar(
                Modifier.align(Alignment.BottomCenter).padding(horizontal = 14.dp, vertical = 8.dp),
                onCancel = { vm.cancel(b) }, onSave = { vm.save(b) },
            )
        }
    }
}

@Composable
private fun SaveBar(modifier: Modifier, onCancel: () -> Unit, onSave: () -> Unit) {
    val c = Sb.c
    Row(
        modifier.widthIn(max = 520.dp).fillMaxWidth().bump(c, 16.dp, Out.O2, ring = c.accentSoft).padding(10.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(8.dp),
    ) {
        Row(Modifier.weight(1f), verticalAlignment = Alignment.CenterVertically, horizontalArrangement =
Arrangement.spacedBy(6.dp)) {
            Ico(Ic.save, 18.dp, c.accent)
            T("Unsaved changes", size = 0.8f, color = c.accent, maxLines = 1)
        }
        Btn("CANCEL", onCancel, small = true, sound = Sfx.BACK)
        Btn("SAVE", onSave, icon = Ic.save, look = Look.ACCENT, small = true, sound = Sfx.CONFIRM)
    }
}

@Composable
private fun CalBar(vm: AppViewModel, b: Bin, on: Boolean, hub: HubState) {
    val c = Sb.c
    val cloud = b.source == Source.CLOUD
    val elsewhere = hub.calibBin != null && hub.calibBin != b.id
    Column(
        Modifier.fillMaxWidth().then(if (on) Modifier.ring(c.accentSoft, 16.dp) else Modifier).well(c, 16.dp).padding(14.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp),
    ) {
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp), verticalAlignment = Alignment.CenterVertically) {
            Box(Modifier.size(42.dp).bump(c, 21.dp), contentAlignment = Alignment.Center) {
                Ico(if (on) Ic.calib else if (cloud) Ic.cloud else Ic.lock(true), 21.dp, if (on) c.accent else c.ink2,
Modifier.spin(on))
            }
            Column(Modifier.weight(1f)) {
                T(if (on) "Calibration mode is on for ${b.name}" else "Settings are locked", size = 0.98f)
                T(
                    when {
                        on -> "Change what you need, then SAVE. Switching off without saving leaves the bin as it was."
                        cloud -> "Calibration needs live readings, so it only runs on the bins’ Wi-Fi. The app is on cloud
now. Every value can still be read."
                        elsewhere -> "Calibration is on for another bin. Switch it off there first."
                        else -> "Switch on calibration mode to change anything. The bin shows its teal calibration screen
while it is on."
                    },
                    Modifier.padding(top = 2.dp), size = 0.8f, weight = FontWeight.Medium, color = c.ink2,
                )
            }
        }
        Row(
            Modifier.fillMaxWidth().bump(c, 14.dp).padding(horizontal = 12.dp, vertical = 10.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            T("Calibration mode", Modifier.weight(1f), size = 0.9f, weight = FontWeight.Bold, color = if (on) c.accent else
c.ink)
            Toggle(on, { vm.setCalibration(it, b) }, enabled = on || (!cloud && !elsewhere && b.source != Source.STALE))
        }
    }
}

@Composable
private fun PinCard(vm: AppViewModel, b: Bin) {
    val c = Sb.c
```

```
    Column(
        Modifier.fillMaxWidth().well(c, 16.dp).padding(14.dp),
        verticalArrangement = Arrangement.spacedBy(10.dp),
    ) {
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            Box(Modifier.size(44.dp).bump(c, 22.dp), contentAlignment = Alignment.Center) { Ico(Ic.pin, 21.dp, c.cal) }
            Column(Modifier.weight(1f)) {
                T("Collect PIN for ${b.name}", size = 0.98f)
                T(
                    if (vm.operator) "Collectors type this on the bin for COLLECT and WI-FI SETUP. Change it under Lid and lock."
                    else "Type this on the bin touchscreen when you tap COLLECT.",
                    size = 0.8f, weight = FontWeight.Medium, color = c.ink2,
                )
            }
        }
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            T(
                if (vm.pinShown) "%04d".format(b.cal.v("collect_pin").toInt()) else "••••", Modifier.weight(1f).padding(start = 4.dp),
                size = 2f, color = c.cal, tnum = true, spacing = if (vm.pinShown) 0.21f else 0.14f, lineHeight = 1.1f,
            )
            Btn(if (vm.pinShown) "HIDE" else "SHOW", { vm.pinShown = !vm.pinShown }, icon = if (vm.pinShown) Ic.eyeoff else Ic.eye, look = Look.TINT, cc = c.cal, small = true, sound = Sfx.CHOICE)
        }
    }
}

@Composable
private fun ReadOnlyNote() {
    val c = Sb.c
    Row(
        Modifier.fillMaxWidth().well(c, 16.dp).padding(14.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(12.dp),
    ) {
        Box(Modifier.size(42.dp).bump(c, 21.dp), contentAlignment = Alignment.Center) { Ico(Ic.eye, 21.dp, c.ink2) }
        Column(Modifier.weight(1f)) {
            T("These settings are read only", size = 0.98f)
            T("You can see every value, but only the operator can change them or run a calibration.", size = 0.8f, weight = FontWeight.Medium, color = c.ink2)
        }
    }
}

/** The fold-out groups, in order. Touchscreen is added for operators. */
private val SECTION_KEYS = listOf("range", "limits", "air", "lid", "weight")

/** .sect: a fold-out group of settings with its summary on the right. */
@Composable
private fun Sect(vm: AppViewModel, key: String, title: String, summary: String, cc: Color, on: Boolean, body: @Composable () -> Unit) {
    val c = Sb.c
    val open = key in vm.open
    val turn by animateFloatAsState(if (open) 180f else 0f, label = "chev")
    Column(Modifier.fillMaxWidth().bump(c, 16.dp)) {
        Row(
            Modifier.fillMaxWidth().defaultMinSize(minHeight = 54.dp).press(true, if (open) Sfx.BACK else Sfx.OPEN) { vm.toggleSection(key) }.padding(horizontal = 14.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(10.dp),
        ) {
            Box(Modifier.size(4.dp, 15.dp).clip(RoundedCornerShape(2.dp)).background(cc))
            T(title.uppercase(), size = 0.76f, spacing = 0.09f, color = cc, maxLines = 1)
            T(summary, Modifier.weight(1f), size = 0.76f, weight = FontWeight.Bold, color = c.ink2, maxLines = 1, align = TextAlign.End)
            Ico(Ic.chev, 18.dp, c.ink2, Modifier.graphicsLayer { rotationZ = turn })
        }
        AnimatedVisibility(open) {
            Column(
                Modifier.fillMaxWidth().padding(start = 14.dp, end = 14.dp, top = 2.dp, bottom = 16.dp).alpha(if (on) 1f else 0.5f),
                verticalArrangement = Arrangement.spacedBy(14.dp),
            ) { body() }
        }
    }
}

@Composable
private fun Fld(vm: AppViewModel, key: String, value: Double, on: Boolean, cc: Color, extra: (@Composable () -> Unit)? = null) {
    val f = FIELDS.getValue(key)
    Field("${f.label} (${f.unit})", f.icon(), cc) {
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            Stepper(fmt(value, f.dec), f.unit, { vm.step(key, it) }, Modifier.weight(1f), enabled = on, cc = cc)
            extra?.invoke()
```

```
        }
    }
}

// The folding groups: fill range, alert limits, air sensor, lid and weight.
// USE NOW copies the bin's live reading into the field (for d_empty, d_full).
@Composable
private fun Sections(vm: AppViewModel, b: Bin, v: Cal, on: Boolean) {
    val c = Sb.c
    val r = b.reading
    Sect(vm, "range", "Fill range", "${fmt(v.v("d_empty"), 1)} to ${fmt(v.v("d_full"), 1)} cm", c.cal, on) {
        Fld(vm, "d_empty", v.v("d_empty"), on, c.cal) { Btn("USE NOW", { vm.useNow("d_empty", b) }, look = Look.TINT, cc = c.cal, small = true, enabled = on, sound = Sfx.CONFIRM) }
        Fld(vm, "d_full", v.v("d_full"), on, c.cal) { Btn("USE NOW", { vm.useNow("d_full", b) }, look = Look.TINT, cc = c.cal, small = true, enabled = on, sound = Sfx.CONFIRM) }
        Hint("Empty the bin and press USE NOW for the first one. Hold a board at your full line and press it for the second. Sensor reads **${fmt(r.distance, 1)} cm** right now.")
    }
    Sect(
        vm, "limits", "Alert limits",
        "${fmt(v.v("fill_th"), 0)}% · ${fmt(v.v("temp_th"), 0)}°C · ${fmt(v.v("hum_th"), 0)}%RH · ${fmt(v.v("gas_th"), 0)} IAQ", c.bad, on,
    ) {
        listOf("fill_th", "temp_th", "hum_th", "gas_th", "tilt_th", "weight_cap").forEach { Fld(vm, it, v.v(it), on, c.bad) }
        Hint("A reading at or above its threshold turns its tile red, raises an alert and sounds the bin buzzer. Humidity uses 85 %RH by default, the same limit as the bin touchscreen.")
    }
    Sect(vm, "air", "Air sensor (BME680)", "offset ${fmt(v.v("temp_off"), 1)}°C", c.gas, on) {
        Field("Temperature raw / corrected", Ic.temp(r.temp), c.gas) { Readout("${fmt(r.tempRaw, 1)} / ${fmt(r.tempRaw + v.v("temp_off"), 1)} °C") }
        Fld(vm, "tref", vm.tref, on, c.gas) { Btn("SET OFFSET", { vm.setOffset(b) }, look = Look.TINT, cc = c.gas, small = true, enabled = on, sound = Sfx.CONFIRM) }
        Fld(vm, "temp_off", v.v("temp_off"), on, c.gas)
        val acc = listOf("Stabilizing", "Low accuracy", "Medium accuracy", "Calibrated")[r.iaqAcc.coerceIn(0, 3)]
        Field("Gas sensor state", Ic.gas, c.gas) {
            Readout(
                when {
                    r.fault("BME680") -> "Not answering"
                    r.warming -> "Warming up" + (r.iaqWarmS?.let { " (${it / 60}:%02d)".format(it % 60) } ?: "") + (r.gasK?.let { ", gas ${Math.round(it)}kΩ" } ?: "")
                    else -> "$acc, IAQ ${Math.round(r.gas)}"
                },
            )
        }
        Fld(vm, "hum_off", v.v("hum_off"), on, c.gas)
        Fld(vm, "press_off", v.v("press_off"), on, c.gas)
        Btn(
            "RESET GAS BASELINE", { vm.binCommand(b, "RESET_GAS", emptyMap(), "Gas baseline cleared. It needs time in normal air to be accurate again.") },
            Modifier.fillMaxWidth(), Ic.reload, Look.TINT, c.gas, enabled = on, sound = Sfx.CONFIRM,
        )
        Hint("The BME680 warms itself, so it reads a little hot and its humidity a little low. Put a reference thermometer next to the bin for a few minutes, enter its reading, press SET OFFSET, then SAVE. The gas reading learns its own baseline in normal air, and RESET GAS BASELINE restarts that.")
    }
    Sect(vm, "lid", "Lid and lock", "PIN •••• · ${fmt(v.v("pir_hold_s"), 0)} s", c.sim, on) {
        Fld(vm, "servo_closed", v.v("servo_closed"), on, c.sim)
        Fld(vm, "servo_open", v.v("servo_open"), on, c.sim)
        Field("Collect PIN", Ic.pin, c.sim) {
            Inp(
                if (vm.operator) v.v("collect_pin").toInt().takeIf { it > 0 }?.toString().orEmpty() else "••••", vm::setPin,
                keyboard = KeyboardType.NumberPassword, enabled = on,
            )
        }
        Fld(vm, "pir_hold_s", v.v("pir_hold_s"), on, c.sim)
        Fld(vm, "buzz_s", v.v("buzz_s"), on, c.sim)
        Row(
            Modifier.fillMaxWidth().bump(c, 14.dp).padding(horizontal = 12.dp, vertical = 10.dp),
            verticalAlignment = Alignment.CenterVertically,
            horizontalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            T("Lock the lid when the bin is full", Modifier.weight(1f), size = 0.9f, weight = FontWeight.Bold)
            Toggle(v.v("auto_lock") > 0, { vm.toggleAutoLock() }, enabled = on)
        }
    }
    Sect(vm, "weight", "Weight", "%.2f kg now".format(r.weight / 1000), c.weight, on) {
        Field("Reading now", Ic.weight(r.weight / v.v("weight_cap")), c.weight) { Readout("%.2f kg".format(r.weight / 1000)) }
        Fld(vm, "known", vm.known, on, c.weight)
        Acts {
            Btn("TARE / ZERO", { vm.binCommand(b, "TARE", emptyMap(), "Scale zeroed on ${b.name}.") }, Modifier.weight(1f), Ic.undo, Look.TINT, c.weight, enabled = on, sound = Sfx.CONFIRM)
            Btn(
                "CALIBRATE", { vm.binCommand(b, "CAL_WEIGHT", mapOf("grams" to vm.known), "Calibrated with ${fmt(vm.known, 0)} g.") },
                Modifier.weight(1f), Ic.capture, Look.TINT, c.weight, enabled = on, sound = Sfx.CONFIRM,
```

```
            )
        }
        Hint("Tare with the bin empty. Then put the known weight in and press CALIBRATE.")
    }
    if (vm.operator) Sect(
        vm, "touch", "Touchscreen",
        when (b.touchCustom) { true -> "calibrated"; false -> "default values"; null -> "not reported" }, c.accent, true,
    ) {
        Acts {
            Btn("CALIBRATE TOUCH", { vm.touchCal(b, false) }, Modifier.weight(1f), Ic.capture, Look.TINT, c.accent, sound = Sfx.TAP)
            Btn("RESET", { vm.touchCal(b, true) }, Modifier.weight(1f), Ic.undo, Look.TINT, c.accent, sound = Sfx.TAP)
        }
        Hint("If taps on the bin land in the wrong place, press CALIBRATE TOUCH and press the centre of each of the 4 crosses the bin shows. RESET goes back to the firmware's own values. Needs the bin on the same Wi-Fi as the phone.")
    }
}

// ---------- connection ----------

@Composable
fun Connection(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val link = hub.link
    val (stateText, stateCc) = when (link) {
        Link.LIVE -> "Local Wi-Fi" to c.link; Link.CLOUD -> "Firebase" to c.cloud; Link.THINGSPEAK -> "ThingSpeak" to c.ts
        Link.OFFLINE -> "Not connected" to c.bad; Link.DEMO -> "Demo data" to c.warn
    }
    val r = hub.routes
    val demo = hub.demo
    ScrollPage {
        BackBar("Connection", { vm.back() }) {
            StatePill(when (link) { Link.CLOUD -> Ic.cloud; Link.THINGSPEAK -> Ic.chart; else -> Ic.wifi(link == Link.LIVE) }, stateText, stateCc, true)
        }
        Card {
            Sh(Ic.layers, "How the app reaches the bins", c.link)
            Route(1, Ic.wifi(true), c.link, link == Link.LIVE, if (demo) r.local else r.local.takeIf { hub.bins.any { it.ip.isNotBlank() } }, "Local Wi-Fi", "Always tried first. Readings every second and every control.")
            Route(2, Ic.cloud, c.cloud, link == Link.CLOUD, if (demo) r.firebase else hub.fbUp, "Firebase", "When no bin answers on Wi-Fi. Readings every 5 s. LOCK, UNLOCK and COLLECT only.")
            Route(3, Ic.chart, c.ts, link == Link.THINGSPEAK, if (demo) r.thingspeak else hub.tsUp, "ThingSpeak", "When Wi-Fi and Firebase do not answer. Readings every 15 s. LOCK, UNLOCK and COLLECT through TalkBack.")
            if (demo && r.any) Hint("These are pretend links from the demo options, to show the colours.")
            Column(verticalArrangement = Arrangement.spacedBy(7.dp)) {
                hub.bins.take(4).forEach { b ->
                    val (cc, txt) = when (b.source) {
                        Source.DEMO -> c.warn to "demo data"
                        Source.STALE -> c.bad to "not answering"
                        Source.LIVE -> c.link to "answers on Wi-Fi"
                        Source.CLOUD -> c.cloud to "via Firebase"
                        Source.THINGSPEAK -> c.ts to "via ThingSpeak"
                    }
                    Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(9.dp)) {
                        Box(Modifier.size(10.dp).ring(cc.soft(0.22f), 5.dp, 3.dp).clip(RoundedCornerShape(5.dp)).background(cc))
                        Row(Modifier.weight(1f), verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(6.dp)) {
                            T(b.name, size = 0.84f, weight = FontWeight.Bold, maxLines = 1)
                            T(b.ip.ifBlank { "no address" }, size = 0.84f, weight = FontWeight.SemiBold, color = c.ink2, maxLines = 1)
                        }
                        T(txt, size = 0.72f, weight = FontWeight.Bold, color = cc, maxLines = 1)
                    }
                    if (!b.isDemo && (b.fw.isNotBlank() || b.rssi != null)) {
                        val up = b.uptimeS?.let { u -> if (u >= 86_400) "up ${u / 86_400} d" else if (u >= 3600) "up ${u / 3600} h" else "up ${u / 60} min" }
                        T(
                            listOfNotNull(b.fw.takeIf { it.isNotBlank() }?.let { "firmware $it" }, b.rssi?.let { "signal $it dBm" }, up).joinToString(" · "),
                            Modifier.padding(start = 19.dp), size = 0.72f, weight = FontWeight.SemiBold, color = c.ink2, maxLines = 1,
                        )
                    }
                }
                if (hub.bins.size > 4) T("and ${hub.bins.size - 4} more", size = 0.84f, weight = FontWeight.SemiBold, color = c.ink2)
            }
            Hint("The app calls every bin address with a 1.5 s timeout when it opens, when the phone changes network, and every 30 s while it is on cloud, and goes back to Wi-Fi by itself. Each bin also writes its own address to the cloud, so a new address is picked up without typing it.")
        }
        Card {
            Sh(Ic.cloud, "Firebase account", c.cloud)
            Field("Database URL", Ic.cloud) { Inp(vm.fb.url, { vm.fb = vm.fb.copy(url = it.trim()) }, placeholder = "https://...firebasedatabase.app", keyboard = KeyboardType.Uri) }
```

```
            Field("Web API key", Ic.key) { Inp(vm.fb.key, { vm.fb = vm.fb.copy(key = it.trim()) }, placeholder = "AIza...") }
            Field("Email", Ic.user) { Inp(vm.fb.email, { vm.fb = vm.fb.copy(email = it.trim()) }, placeholder = "name@example.com", keyboard = KeyboardType.Email) }
            Field("Password", Ic.key) {
                Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Inp(vm.fb.password, { vm.fb = vm.fb.copy(password = it) }, Modifier.weight(1f), password = !vm.fbShow)
                    Btn(if (vm.fbShow) "HIDE" else "SHOW", { vm.fbShow = !vm.fbShow }, icon = if (vm.fbShow) Ic.eyeoff else Ic.eye, look = Look.TINT, small = true, sound = Sfx.CHOICE)
                }
            }
            vm.fbMsg?.let { (ok, msg) ->
                Row(
                    Modifier.fillMaxWidth().brutal(c, 12.dp, if (ok) c.ok else c.bad, off = 3.dp).padding(horizontal = 12.dp, vertical = 10.dp),
                    horizontalArrangement = Arrangement.spacedBy(8.dp),
                ) {
                    Ico(if (ok) Ic.check else Ic.tamper, 18.dp, c.onBanner)
                    T(msg, size = 0.82f, weight = FontWeight.Bold, color = c.onBanner)
                }
            }
            Acts {
                Btn(if (vm.fbBusy) "TESTING" else "TEST", { vm.fbTest(false) }, Modifier.weight(1f), Ic.radar, Look.TINT, c.cloud, enabled = !vm.fbBusy)
                Btn("TEST AND SAVE", { vm.fbTest(true) }, Modifier.weight(1f), Ic.save, Look.SOLID, c.cloud, enabled = !vm.fbBusy)
            }
            Btn("RESTORE WORKING", { vm.fbRestore() }, Modifier.fillMaxWidth(), Ic.undo, Look.TINT)
            if (vm.operator) {
                Btn("SEND TO BINS", { vm.sendCloudToBins() }, Modifier.fillMaxWidth(), Ic.bins, Look.TINT, c.cloud, sound = Sfx.TAP)
                Hint("SEND TO BINS gives every bin on Wi-Fi this account, so they write to the same database. Save it with TEST AND SAVE first.")
            }
            Hint("TEST signs in and reads the list of bins. TEST AND SAVE keeps the details only when the test passes. RESTORE WORKING puts back the last details that passed. The password is stored encrypted on this phone.")
        }
        Card {
            Sh(Ic.chart, "ThingSpeak", c.ts)
            Field("Server", Ic.chart, c.ts) { Inp(vm.tsUrl, { vm.tsUrl = it.trim() }, placeholder = "https://api.thingspeak.com", keyboard = KeyboardType.Uri) }
            vm.tsMsg?.let { (ok, msg) ->
                Row(
                    Modifier.fillMaxWidth().brutal(c, 12.dp, if (ok) c.ok else c.bad, off = 3.dp).padding(horizontal = 12.dp, vertical = 10.dp),
                    horizontalArrangement = Arrangement.spacedBy(8.dp),
                ) {
                    Ico(if (ok) Ic.check else Ic.tamper, 18.dp, c.onBanner)
                    T(msg, size = 0.82f, weight = FontWeight.Bold, color = c.onBanner)
                }
            }
            Btn(if (vm.tsBusy) "TESTING" else "TEST AND SAVE", { vm.tsTest() }, Modifier.fillMaxWidth(), Ic.radar, Look.SOLID, c.ts, enabled = !vm.tsBusy)
            // ThingSpeak has no call that reports the message count, so the app estimates it.
            val perDay = hub.tsPerDay
            val year = (perDay ?: 0.0) * 365
            val yearShare = (year / 3_000_000.0).coerceIn(0.0, 1.0)
            Box(
                Modifier.fillMaxWidth().height(14.dp).well(c, 7.dp).drawBehind {
                    if (yearShare > 0) drawRoundRect(
                        if (year > 3_000_000) c.bad else c.ts,
                        size = Size((size.width * yearShare).toFloat().coerceAtLeast(size.height), size.height), cornerRadius = CornerRadius(7.dp.toPx()),
                    )
                },
            )
            Hint(
                if (perDay == null) "No estimate yet. Once a channel has been read for two minutes, the app counts how fast its entries rise and works out the year. The free licence allows 3 million messages a year. The exact count is on your ThingSpeak account page."
                else "About **${"%,d".format(Math.round(perDay))}** messages a day, so about **${"%,d".format(Math.round(year))}** a year of the 3,000,000 the free licence allows" +
                    (if (year > 3_000_000) ". That is over the limit, so make the bins write less often." else ".") + " This is an estimate. The exact count is on your ThingSpeak account page.",
            )
            Hint(
                "Each bin needs its own channel. Put the channel ID and read API key in the bin's SET sheet, and its TalkBack ID and key for LOCK, UNLOCK and COLLECT. " +
                    "The write API key stays in the bin's firmware. Fields: **1** fill %, **2** weight g, **3** temperature, **4** humidity, **5** air quality, **6** pressure, **7** tilt, " +
                    "**8** motion + 2 × lid + 4 × status, and the status text holds the bin's IP address. The free licence allows a write every 15 s per channel and 3 million a year in all.",
            )
        }
        Card {
            Sh(Ic.download, "Data use this month", c.cloud)
            val bytes = hub.cloudBytes
```

```
            val limit = 10L * 1024 * 1024 * 1024
            val share = ((bytes ?: 0L).toDouble() / limit).coerceIn(0.0, 1.0)
            Box(
                Modifier.fillMaxWidth().height(14.dp).well(c, 7.dp).drawBehind {
                    if (share > 0) drawRoundRect(c.cloud, size = Size((size.width *
share).toFloat().coerceAtLeast(size.height), size.height), cornerRadius = CornerRadius(7.dp.toPx()))
                },
            )
            Hint(
                if (bytes == null) "No figure yet. The bins report how much they sent once the app has used the cloud. The
free Firebase plan allows 10 GB each month."
                else "**${bytes / (1024 * 1024)} MB** of the 10 GB the free Firebase plan allows each month. The bins write
every 5 s while a phone watches over cloud and every minute otherwise, and at once when an alert starts."
            )
        }
    }
}

@Composable
private fun Route(n: Int, icon: ImageVector, cc: Color, on: Boolean, up: Boolean?, title: String, sub: String) {
    val c = Sb.c
    Row(
        Modifier.fillMaxWidth().then(if (on) Modifier.bump(c, 14.dp, ring = cc.soft(0.34f)) else Modifier.well(c,
14.dp)).padding(horizontal = 12.dp, vertical = 11.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(11.dp),
    ) {
        Box(Modifier.size(26.dp).bump(c, 13.dp), contentAlignment = Alignment.Center) { T("$n", size = 0.8f, color = cc) }
        Ico(icon, 21.dp, cc)
        Column(Modifier.weight(1f)) {
            T(title, size = 0.92f)
            T(sub, Modifier.padding(top = 2.dp), size = 0.77f, weight = FontWeight.SemiBold, color = c.ink2)
        }
        when {
            on -> Tag("IN USE", cc)
            up == true -> Tag("ANSWERS", cc.soft(0.85f))
            up == false -> T("NO ANSWER", size = 0.66f, spacing = 0.05f, color = c.bad, maxLines = 1)
            else -> T("NOT SET UP", size = 0.66f, spacing = 0.05f, color = c.ink2, maxLines = 1)
        }
    }
}

// ---------- Bluetooth setup ----------

@Composable
fun Bluetooth(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val b = vm.binFor(vm.selSet, hub.bins)
    val paired = vm.bt in 1..3
    val name = vm.btName.ifBlank { b?.takeIf { it.isDemo }?.name ?: "the bin" }
    // Android asks for Nearby devices (or Location before Android 12) the first time.
    val ask = rememberLauncherForActivityResult(ActivityResultContracts.RequestMultiplePermissions()) { vm.btPair(b) }
    val pair = { if (b?.isDemo != true && !vm.btAllowed()) ask.launch(vm.btPermissions()) else vm.btPair(b) }
    ScrollPage {
        BackBar("Bluetooth setup", { vm.back() }) {
            StatePill(Ic.bluetooth, if (vm.bt >= 4) "Off" else if (paired) "Paired" else "Not paired", c.bt, paired)
        }
        Card {
            Hint("Hand the Wi-Fi name and password to a bin over Bluetooth, read back the address it gets, then switch its
Bluetooth off so only Wi-Fi stays on.")
            Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                listOf(Ic.bluetooth to "PAIR + CODE", Ic.radar to "PICK NETWORK", Ic.key to "SEND", Ic.power to "BT
OFF").forEachIndexed { k, (ic, label) ->
                    val done = vm.bt > k
                    val now = vm.bt == k
                    val cc = when { done -> c.ok; now -> c.bt; else -> c.ink2 }
                    Column(
                        Modifier.weight(1f).then(if (now) Modifier.bump(c, 13.dp, ring = c.bt.soft(0.3f)) else
Modifier.well(c, 13.dp)).padding(vertical = 11.dp, horizontal = 4.dp),
                        horizontalAlignment = Alignment.CenterHorizontally,
                        verticalArrangement = Arrangement.spacedBy(6.dp),
                    ) {
                        Ico(if (done) Ic.check else ic, 19.dp, cc)
                        T(label, size = 0.64f, color = cc, align = TextAlign.Center, lineHeight = 1.2f)
                    }
                }
            }
            vm.btNote?.let { note ->
                Row(
                    Modifier.fillMaxWidth().well(c, 12.dp).padding(horizontal = 12.dp, vertical = 10.dp),
                    verticalAlignment = Alignment.CenterVertically,
                    horizontalArrangement = Arrangement.spacedBy(9.dp),
                ) {
                    Ico(if (vm.btBusy || vm.bt == 2) Ic.reload else Ic.bluetooth, 18.dp, c.bt, Modifier.spin(vm.btBusy ||
vm.bt == 2))
                    T(note, Modifier.weight(1f), size = 0.82f, weight = FontWeight.SemiBold)
```

```
                }
            }
            val canEdit = vm.bt == 1
            Field("Wi-Fi name (2.4 GHz)", Ic.wifi(false)) {
                Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Inp(vm.ssid, { vm.ssid = it }, Modifier.weight(1f), placeholder = "HomeWiFi-2G", enabled = canEdit)
                    Btn("SCAN", { vm.btScanNets() }, icon = Ic.radar, look = Look.TINT, cc = c.bt, small = true, enabled =
canEdit && !vm.btBusy, sound = Sfx.OPEN)
                }
            }
            Field("Password", Ic.key) {
                Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Inp(vm.pass, { vm.pass = it }, Modifier.weight(1f), password = !vm.showPass, enabled = canEdit)
                    Btn(if (vm.showPass) "HIDE" else "SHOW", { vm.showPass = !vm.showPass }, icon = if (vm.showPass)
Ic.eyeoff else Ic.eye, look = Look.TINT, small = true, sound = Sfx.CHOICE)
                }
            }
            if (vm.bt >= 3 && vm.btIp.isNotBlank()) {
                Column(Modifier.fillMaxWidth().well(c, 14.dp).padding(14.dp), verticalArrangement =
Arrangement.spacedBy(12.dp)) {
                    Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(12.dp))
{
                        Ico(Ic.wifi(true), 21.dp, c.ok)
                        Column(Modifier.weight(1f)) {
                            T("$name joined ${vm.btJoined}", size = 0.86f, weight = FontWeight.Bold)
                            T(vm.btIp, size = 1.3f, color = c.ok, tnum = true)
                        }
                    }
                    Btn(vm.btUseLabel(), { vm.btUse() }, Modifier.fillMaxWidth(), Ic.check, Look.SOLID, c.ok, small = true,
sound = Sfx.TAP)
                }
            }
            if (vm.btStale) {
                Btn("FORGET AND PAIR AGAIN", { vm.btForget() }, Modifier.fillMaxWidth(), Ic.undo, Look.TINT, c.warn, sound =
Sfx.TAP)
            }
            Btn(
                if (vm.btBusy && vm.bt == 0) "PAIRING" else "PAIR A BIN", { pair() }, Modifier.fillMaxWidth(), Ic.bluetooth,
Look.TINT, c.bt,
                enabled = (vm.bt == 0 || vm.bt >= 4) && !vm.btBusy, sound = Sfx.OPEN,
            )
            Btn(
                "SEND WI-FI", { vm.btSend() }, Modifier.fillMaxWidth(), if (vm.bt == 2) Ic.reload else Ic.key, Look.SOLID,
c.bt,
                enabled = vm.bt == 1, iconMod = Modifier.spin(vm.bt == 2), sound = Sfx.CONFIRM,
            )
            Btn("TURN BT OFF", { vm.btOff() }, Modifier.fillMaxWidth(), Ic.power, Look.TINT, c.warn, enabled = vm.bt in 1..3,
sound = Sfx.TAP)
            Hint(
                when (vm.bt) {
                    0 -> "On the bin, tap i, then WI-FI SETUP, and type the Collect PIN. Then press PAIR A BIN, pick the bin,
and type the 6-digit code its screen shows."
                    1 -> "Pick the 2.4 GHz network the bin should join, then SEND WI-FI."
                    2 -> "The bin tries the network for up to 15 s. If it fails it goes back to its old Wi-Fi."
                    3 -> "Press ${vm.btUseLabel()} to keep this address, then TURN BT OFF."
                    else -> "Bluetooth is off on the bin. It stays on Wi-Fi until someone opens WI-FI SETUP on it again."
                }
            )
            if (b?.isDemo == true) Hint("${b.name} is a demo bin, so pairing uses a stand-in code here. Pick a real bin, or
none, to set up a real one.")
            Hint("The code keeps strangers from pairing and encrypts the link, so the Wi-Fi password cannot be overheard. It
is new each time, lasts 60 s, and 3 wrong codes stop pairing for 60 s.")
        }
    }
}
```

Android App/app/src/main/java/smartbin/ui/Pages.kt

```
// ================================================================================
// ui/Pages.kt - the Simulation and Alerts tabs
// Simulation needs the bin on local Wi-Fi: each MANUAL row sets one bit of the
// firmware's Sim mask and its value, and APPLY sends them in one POST to
// /api/simulate. Alerts lists Hub's alert history with filters by kind and
// time range.
// ================================================================================
package smartbin.ui

import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.defaultMinSize
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
```

```
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Slider
import androidx.compose.material3.SliderDefaults
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.alpha
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.drawBehind
import androidx.compose.ui.geometry.Size
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import smartbin.data.AlertKind
import smartbin.data.AlertRow
import smartbin.data.Bin
import smartbin.data.DEFAULT_SIM
import smartbin.data.Hub
import smartbin.data.HubState
import smartbin.data.Severity
import smartbin.data.Source
import smartbin.notify.Sfx

// ---------- simulation ----------

private enum class SimKind { RANGE, NUM, PAIR, TOGGLE, LID }

private class SimRowDef(val key: String, val bit: Int, val label: String, val kind: SimKind, val min: Float = 0f, val max: Float = 100f, val step: Float = 1f, val unit: String = "")

private val SENSORS = listOf(
    SimRowDef("fill", 0, "Fill level", SimKind.RANGE, 0f, 100f, 1f, "%"),
    SimRowDef("weight", 1, "Weight", SimKind.NUM),
    SimRowDef("gas", 2, "Air quality", SimKind.RANGE, 0f, 500f, 5f, "IAQ"),
    SimRowDef("temp", 3, "Temperature and humidity", SimKind.PAIR),
    SimRowDef("tamper", 4, "Tamper", SimKind.TOGGLE),
    SimRowDef("motion", 5, "Motion", SimKind.TOGGLE),
    SimRowDef("lid", 6, "Lid", SimKind.LID),
)

@Composable
fun Simulation(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val b = vm.binFor(vm.selSim, hub.bins)
    Box(Modifier.fillMaxSize()) {
        ScrollPage(bottomExtra = 80.dp) {
            Sh(Ic.flask, "Simulation", c.sim)
            if (b == null) {
                Card { Empty(Ic.bins, "No bins yet.") }
                return@ScrollPage
            }
            BinSeg(vm, hub, PickFor.SIM, vm.selSim, allowAll = false) { vm.selSim = it }
            Hint("AUTO reads the real hardware and lets the firmware drive the outputs. MANUAL holds the value you set, one row at a time.")
            if (b.source == Source.CLOUD || b.source == Source.THINGSPEAK) Row(
                Modifier.fillMaxWidth().well(c, 16.dp).padding(14.dp),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(12.dp),
            ) {
                Box(Modifier.size(42.dp).bump(c, 21.dp), contentAlignment = Alignment.Center) { Ico(Ic.cloud, 21.dp, c.cloud) }
                Column(Modifier.weight(1f)) {
                    T("Simulation needs the bins’ Wi-Fi", size = 0.98f)
                    T("The app is on cloud now, so APPLY rests until a bin answers on Wi-Fi again.", size = 0.8f, weight = FontWeight.Medium, color = c.ink2)
                }
            }
            SENSORS.forEach { SimRow(vm, b, it) }
            Hint("The whole app turns violet while any sensor is manual.", Modifier.fillMaxWidth(), align = TextAlign.Center)
        }
        if (b != null) Row(
            Modifier.align(Alignment.BottomCenter).padding(horizontal = 14.dp, vertical = 8.dp).widthIn(max = 520.dp).fillMaxWidth()
                .bump(c, 16.dp, Out.O2).padding(10.dp),
            horizontalArrangement = Arrangement.spacedBy(8.dp),
        ) {
            Btn("APPLY", { vm.applySim(b) }, Modifier.weight(1f), Ic.flask, Look.ACCENT, enabled = b.source != Source.CLOUD && b.source != Source.THINGSPEAK, sound = Sfx.CONFIRM)
            Btn("ALL TO AUTO", { vm.simAllAuto(b) }, Modifier.weight(1f), Ic.reload, Look.TINT, c.ok, enabled = b.sim.mask >
```

```
    0, sound = Sfx.SWITCH_OFF)
        }
    }
}

@Composable
private fun SimRow(vm: AppViewModel, b: Bin, s: SimRowDef) {
    val c = Sb.c
    val cc = when (s.key) {
        "fill" -> c.fill; "weight" -> c.weight; "gas" -> c.gas; "temp" -> c.temp; "tamper" -> c.tamper; "motion" -> c.motion;
else -> c.lid
    }
    val values = b.sim.values
    fun value(k: String) = values[k] ?: DEFAULT_SIM[k] ?: 0.0
    val manual = b.sim.on(s.bit)
    val icon: ImageVector = when (s.key) {
        "fill" -> Ic.fill; "weight" -> Ic.weight(value("weight") / 4500); "gas" -> Ic.gas; "temp" -> Ic.temp(value("temp"))
        "tamper" -> Ic.tamper; "motion" -> Ic.motion(value("motion") > 0); else -> Ic.lid(value("lid") > 0)
    }
    Column(
        Modifier
            .fillMaxWidth()
            .bump(c, 16.dp)
            .drawBehind { if (manual) drawRect(cc, size = Size(4.dp.toPx(), size.height)) }
            .padding(horizontal = 13.dp, vertical = 12.dp),
        verticalArrangement = Arrangement.spacedBy(10.dp),
    ) {
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(10.dp)) {
            Ico(icon, 21.dp, cc)
            T(s.label, Modifier.weight(1f), weight = FontWeight.Bold)
            Btn(
                if (manual) "MANUAL" else "AUTO", { vm.simToggle(b, s.bit) }, Modifier.widthIn(min = 104.dp),
                if (manual) Ic.flask else Ic.check, if (manual) Look.SOLID else Look.TINT, if (manual) c.sim else c.ok,
                small = true, sound = if (manual) Sfx.SWITCH_OFF else Sfx.SWITCH_ON,
            )
        }
        val on = manual
        Box(Modifier.alpha(if (on) 1f else 0.45f)) {
            when (s.kind) {
                SimKind.RANGE -> Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement =
Arrangement.spacedBy(10.dp)) {
                    Slider(
                        value = value(s.key).toFloat().coerceIn(s.min, s.max),
                        onValueChange = { vm.simValue(b, s.key, (Math.round(it / s.step) * s.step).toDouble()) },
                        valueRange = s.min..s.max,
                        enabled = on,
                        modifier = Modifier.weight(1f).height(28.dp),
                        colors = SliderDefaults.colors(
                            thumbColor = c.sim, activeTrackColor = c.sim, inactiveTrackColor = c.well,
                            disabledThumbColor = c.ink2, disabledActiveTrackColor = c.ink2, disabledInactiveTrackColor =
c.well,
                        ),
                    )
                    Row(Modifier.defaultMinSize(minWidth = 66.dp).well(c, 11.dp).padding(horizontal = 10.dp, vertical =
8.dp), verticalAlignment = Alignment.Bottom) {
                        T(Math.round(value(s.key)).toString(), tnum = true)
                        T(" ${s.unit}", size = 0.7f, color = c.ink2)
                    }
                }
                SimKind.NUM -> SimStepper(vm, b, "weight", value("weight"), on)
                SimKind.PAIR -> Column(verticalArrangement = Arrangement.spacedBy(10.dp)) {
                    SimStepper(vm, b, "temp", value("temp"), on)
                    SimStepper(vm, b, "humidity", value("humidity"), on)
                }
                SimKind.TOGGLE -> TwoWay(full = true) {
                    Tw(value(s.key) <= 0, { vm.simValue(b, s.key, 0.0) }, "OFF", Ic.cross, cc, enabled = on)
                    Tw(value(s.key) > 0, { vm.simValue(b, s.key, 1.0) }, "TRIGGERED", if (s.key == "motion") Ic.motion(true)
else Ic.tamper, cc, enabled = on)
                }
                SimKind.LID -> TwoWay(full = true) {
                    Tw(value("lid") <= 0, { vm.simValue(b, "lid", 0.0) }, "CLOSED", Ic.lock(true), c.lid, enabled = on)
                    Tw(value("lid") > 0, { vm.simValue(b, "lid", 1.0) }, "OPEN", Ic.lid(true), c.lid, enabled = on)
                }
            }
        }
    }
}

@Composable
private fun SimStepper(vm: AppViewModel, b: Bin, key: String, v: Double, on: Boolean) {
    val f = SIM_STEP.getValue(key)
    Stepper(fmt(v, if (key == "temp") 1 else 0), f.unit, { vm.simStep(b, key, it) }, enabled = on, cc = Sb.c.sim)
}

// ---------- alerts ----------

```

```
private val KIND_ICONS: List<Pair<AlertKind, ImageVector>>
    get() = listOf(
        AlertKind.ALL to Ic.layers, AlertKind.FULL to Ic.fill, AlertKind.SENSOR to Ic.temp(30.0), AlertKind.TAMPER to Ic.tamper,
        AlertKind.LOCK to Ic.lock(true), AlertKind.PICKUP to Ic.truck, AlertKind.SETUP to Ic.calib,
    )

@Composable
fun AlertsPage(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val every = hub.alerts
    val unread = hub.unread
    val filtered = vm.shown(every)
    val unShown = filtered.count { !it.read }
    val list = filtered.take(vm.alertShown)
    ScrollPage {
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(10.dp)) {
            Sh(Ic.clock, "Alert history", c.tamper, Modifier.weight(1f))
            if (unread > 0) Box(Modifier.clip(RoundedCornerShape(5.dp)).background(c.tamper).padding(horizontal = 8.dp, vertical = 4.dp)) {
                T("$unread UNREAD", size = 0.68f, spacing = 0.05f, color = Color.White)
            }
        }
        TwoWay(full = true) {
            KIND_ICONS.forEach { (k, ic) ->
                Tw(vm.akind == k, { vm.akind = k; vm.alertShown = 6 }, icon = ic, cc = if (k == AlertKind.ALL) c.ink else c.tamper, minHeight = 40.dp)
            }
        }
        TwoWay(full = true) {
            AlertRange.entries.forEach { r -> Tw(vm.arange == r, { vm.arange = r; vm.alertShown = 6 }, r.label, cc = c.tamper, minHeight = 36.dp) }
        }
        Column {
            T(if (filtered.isNotEmpty()) "Showing ${vm.scope(filtered.size)}" else "No alerts match.", size = 0.82f)
            T(
                if (filtered.isNotEmpty()) vm.akind.label + if (unShown > 0) ", $unShown unread" else ", all read" else "Change the kind or the dates.",
                Modifier.padding(top = 2.dp), size = 0.82f, weight = FontWeight.SemiBold, color = c.ink2,
            )
        }
        Acts {
            Btn("MARK READ", { vm.markShown(every) }, Modifier.weight(1f), Ic.check, Look.TINT, c.ok, small = true, enabled = unShown > 0, sound = Sfx.CONFIRM)
            Btn("DELETE", { vm.overlay = Overlay.ClearAlerts }, Modifier.weight(1f), Ic.trash, Look.TINT, c.tamper, small = true, enabled = filtered.isNotEmpty(), sound = Sfx.OPEN)
        }
        if (list.isEmpty()) {
            Card { Empty(Ic.bell, if (every.isNotEmpty()) "No alerts match. Change the kind or the dates." else "No alerts yet. This fills up as bins reach their limits.") }
        } else {
            Column(verticalArrangement = Arrangement.spacedBy(11.dp)) { list.forEach { AlertLine(it) } }
        }
        if (filtered.size > list.size) Btn("Show older (${filtered.size - list.size} more)", { vm.alertShown += 6 }, Modifier.fillMaxWidth(), Ic.clock)
        if (list.isNotEmpty()) Hint("MARK READ and DELETE act on exactly the alerts shown. Tap the dot to mark one read or unread. The cross removes one alert.", Modifier.fillMaxWidth(), align = TextAlign.Center)
    }
}

@Composable
private fun AlertLine(a: AlertRow) {
    val c = Sb.c
    val cc = sevColor(a.sev, c)
    val critical = a.sev == Severity.CRITICAL
    val surface = if (critical) Modifier.brutal(c, 8.dp, c.bad)
    else Modifier.bump(c, 14.dp).drawBehind { drawRect(cc, size = Size(5.dp.toPx(), size.height)) }
    val fg = if (critical) Color.White else c.ink
    val sub = if (critical) Color.White.soft(0.88f) else c.ink2
    Row(
        Modifier.fillMaxWidth().alpha(if (a.read) 0.66f else 1f).then(surface).padding(start = 8.dp, end = 10.dp, top = 11.dp, bottom = 11.dp),
        verticalAlignment = Alignment.CenterVertically,
        horizontalArrangement = Arrangement.spacedBy(10.dp),
    ) {
        // The dot: full while unread, a ring once read. A tap flips it.
        Box(
            Modifier.size(30.dp, 44.dp).press(true, Sfx.CHOICE) { Hub.markRead(setOf(a.id), !a.read) },
            contentAlignment = Alignment.Center,
        ) {
            val dot = if (critical) Color.White else cc
            if (a.read) Box(Modifier.size(11.dp).alpha(0.5f).ring(dot, 5.5.dp, 2.dp).padding(2.dp))
            else Box(Modifier.size(11.dp).ring(dot.soft(0.24f), 5.5.dp, 3.dp).clip(CircleShape).background(dot))
        }
        Box(
            Modifier.size(38.dp).then(if (critical)
```

```
    Modifier.clip(RoundedCornerShape(11.dp)).background(Color.White.soft(0.17f)) else Modifier.well(c, 11.dp)),
253             contentAlignment = Alignment.Center,
254         ) { Ico(Ic.named(a.icon), 20.dp, if (critical) Color.White else cc) }
255         Column(Modifier.weight(1f)) {
256             T(a.title, size = 0.95f, weight = if (a.read) FontWeight.Bold else FontWeight.ExtraBold, color = fg)
257             T("${a.meta} · ${ago(a.at)}", Modifier.padding(top = 2.dp), size = 0.78f, weight = FontWeight.Medium, color =
    sub)
258         }
259         Box(
260             Modifier.size(34.dp).then(if (critical)
    Modifier.clip(RoundedCornerShape(10.dp)).background(Color.White.soft(0.18f)) else Modifier.bump(c, 10.dp))
261                 .press(true, Sfx.DANGER) { Hub.deleteAlerts(setOf(a.id)) },
262             contentAlignment = Alignment.Center,
263         ) { Ico(Ic.cross, 15.dp, if (critical) Color.White else c.ink2) }
264     }
265 }
```

Android App/app/src/main/java/smartbin/ui/Overlays.kt

```
1  // ================================================================================
2  // ui/Overlays.kt - sheets and pop-ups drawn over the current page
3  // vm.overlay picks one: bin edit, bin detail, add bins, clear alerts, bin
4  // picker, Wi-Fi help, operator PIN pads, Bluetooth device list and pairing
5  // code, the Attention list and the push-alert prompt. Only one shows at a time; a
6  // tap outside or Back closes it (vm.overlay = null).
7  // ================================================================================
8  package smartbin.ui
9
10 import smartbin.notify.Feel
11 import androidx.compose.foundation.gestures.detectHorizontalDragGestures
12 import androidx.compose.ui.input.pointer.pointerInput
13 import androidx.compose.foundation.layout.Arrangement
14 import androidx.compose.foundation.layout.Box
15 import androidx.compose.foundation.layout.Column
16 import androidx.compose.foundation.layout.Row
17 import androidx.compose.foundation.layout.fillMaxWidth
18 import androidx.compose.foundation.layout.height
19 import androidx.compose.foundation.layout.padding
20 import androidx.compose.foundation.layout.size
21 import androidx.compose.foundation.shape.RoundedCornerShape
22 import androidx.compose.foundation.background
23 import androidx.compose.runtime.Composable
24 import androidx.compose.runtime.LaunchedEffect
25 import androidx.compose.runtime.getValue
26 import androidx.compose.runtime.mutableStateOf
27 import androidx.compose.runtime.remember
28 import androidx.compose.runtime.setValue
29 import androidx.compose.ui.Alignment
30 import androidx.compose.ui.Modifier
31 import androidx.compose.ui.draw.clip
32 import androidx.compose.ui.graphics.graphicsLayer
33 import androidx.compose.ui.text.font.FontWeight
34 import androidx.compose.ui.text.input.KeyboardType
35 import androidx.compose.ui.unit.dp
36 import smartbin.data.HubState
37 import smartbin.data.OperatorLock
38 import smartbin.notify.Sfx
39
40 /** Every sheet, drawn over the page. Only one is open at a time. */
41 @Composable
42 fun Overlays(vm: AppViewModel, hub: HubState, allowAlerts: () -> Unit) {
43     val o = vm.overlay ?: return
44     val close = { vm.overlay = null }
45     when (o) {
46         Overlay.Attention -> Sheet(close) { Attention(vm, hub) }
47         is Overlay.BinEdit -> {
48             val b = hub.bins.firstOrNull { it.id == o.id } ?: return Gone(vm)
49             Sheet(close) {
50                 SheetTitle(Ic.gear, "${b.name} settings")
51                 Field("Name") { Inp(vm.formName, { vm.formName = it }) }
52                 Field("Location") { Inp(vm.formPlace, { vm.formPlace = it }) }
53                 Field("IP address") { Inp(vm.formIp, { vm.formIp = it.trim() }, placeholder = "192.168.1.101", keyboard =
    KeyboardType.Uri) }
54                 Field("Cloud name (Firebase)") { Inp(vm.formCloud, { vm.formCloud = it.trim() }, placeholder = "bin01") }
55                 Field("ThingSpeak channel ID", Ic.chart, Sb.c.ts) { Inp(vm.formTsCh, { vm.formTsCh = it.trim() }, placeholder
    = "2345678", keyboard = KeyboardType.Number) }
56                 Field("ThingSpeak read API key", Ic.key, Sb.c.ts) { Inp(vm.formTsKey, { vm.formTsKey = it.trim() },
    placeholder = "Only for a private channel") }
57                 Field("TalkBack ID and API key", Ic.key, Sb.c.ts) {
58                     Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
59                         Inp(vm.formTbId, { vm.formTbId = it.trim() }, Modifier.weight(0.42f), placeholder = "ID", keyboard =
    KeyboardType.Number)
60                         Inp(vm.formTbKey, { vm.formTbKey = it.trim() }, Modifier.weight(0.58f), placeholder = "API key")
61                     }
62                 }
```

```
                Hint("With an IP address the app reads this bin live over Wi-Fi. With none of the three it shows demo data. Bluetooth setup can fill the address in. The cloud name is the bin's node in Firebase, tried when Wi-Fi does not answer. The ThingSpeak channel is tried after that, and TalkBack carries LOCK, UNLOCK and COLLECT to the bin.")
                Acts {
                    Btn("Remove", { vm.removeBin(b.id) }, Modifier.weight(1f), Ic.trash, Look.DANGER, sound = Sfx.DANGER)
                    Btn("Save", { vm.saveBinEdit(b.id) }, Modifier.weight(1f), Ic.save, Look.ACCENT, sound = Sfx.SAVED)
                }
            }
        }
        is Overlay.BinDetail -> {
            val bins = hub.bins
            val i = bins.indexOfFirst { it.id == o.id }
            if (i < 0) return Gone(vm)
            val b = bins[i]
            Sheet(close) {
                Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    IconKey(Ic.back, { vm.overlay = Overlay.BinDetail(bins[i - 1].id) }, tint = muted(), enabled = i > 0, sound = Sfx.STEP_DOWN)
                    Column(Modifier.weight(1f), horizontalAlignment = Alignment.CenterHorizontally) {
                        T(b.name, size = 1.02f, maxLines = 1)
                        T("${i + 1} of ${bins.size} · swipe or use the arrows", size = 0.74f, weight = FontWeight.Bold, color = Sb.c.ink2, maxLines = 1)
                    }
                    IconKey(
                        Ic.back, { vm.overlay = Overlay.BinDetail(bins[i + 1].id) }, tint = muted(), enabled = i < bins.size - 1, sound = Sfx.STEP_UP,
                        iconMod = Modifier.rotate180(),
                    )
                    IconKey(Ic.cross, close, tint = muted(), sound = Sfx.BACK)
                }
                // A swipe across the card moves to the next or previous bin, like the arrows.
                var drag by remember { mutableStateOf(0f) }
                Box(
                    Modifier.pointerInput(i, bins.size) {
                        detectHorizontalDragGestures(
                            onDragStart = { drag = 0f },
                            onDragEnd = {
                                val far = 80.dp.toPx()
                                when {
                                    drag < -far && i < bins.size - 1 -> { Feel.event(Sfx.STEP_UP); vm.overlay = Overlay.BinDetail(bins[i + 1].id) }
                                    drag > far && i > 0 -> { Feel.event(Sfx.STEP_DOWN); vm.overlay = Overlay.BinDetail(bins[i - 1].id) }
                                }
                                drag = 0f
                            },
                        ) { _, dx -> drag += dx }
                    },
                ) { BinCard(vm, b, hub, inSheet = true) }
            }
        }
        is Overlay.Pick -> Sheet(close) { Picker(vm, hub, o.forWhat) }
        Overlay.AddBins -> Sheet(close) {
            SheetTitle(Ic.plus, "Add bins")
            SheetText("How many bins are you adding? Each one gets a card with demo data until you give it an address.")
            val c = Sb.c
            Row(
                Modifier.align(Alignment.CenterHorizontally).well(c, 999.dp).padding(5.dp),
                verticalAlignment = Alignment.CenterVertically,
                horizontalArrangement = Arrangement.spacedBy(8.dp),
            ) {
                RoundKey("−", vm.addN > 1, Sfx.STEP_DOWN) { vm.addN-- }
                T("${vm.addN}", Modifier.size(40.dp, 30.dp), size = 1.4f, tnum = true, align = androidx.compose.ui.text.style.TextAlign.Center)
                RoundKey("+", vm.addN < 3, Sfx.STEP_UP) { vm.addN++ }
            }
            Acts {
                Btn("Cancel", close, Modifier.weight(1f), sound = Sfx.BACK)
                Btn("Add ${vm.addN} bin${if (vm.addN > 1) "s" else ""}", { vm.addBins() }, Modifier.weight(1f), Ic.plus, Look.ACCENT, sound = Sfx.CONFIRM)
            }
        }
        Overlay.ClearAlerts -> Sheet(close) {
            val n = vm.shown(hub.alerts).size
            SheetTitle(Ic.trash, "Delete alerts")
            SheetText("Delete ${vm.scope(n)}? This cannot be undone.")
            Acts {
                Btn("Cancel", close, Modifier.weight(1f), sound = Sfx.BACK)
                Btn("Delete $n", { vm.deleteShown(hub.alerts) }, Modifier.weight(1f), Ic.trash, Look.DANGER, sound = Sfx.DANGER)
            }
        }
        Overlay.Wifi -> Sheet(close) {
            SheetTitle(Ic.radar, "Networks the bin can see")
            Hint("Only 2.4 GHz networks are listed. The bin cannot join 5 GHz.")
            val nets = vm.btNets.ifEmpty { if (vm.btDemo) AppViewModel.DEMO_NETS else emptyList() }
```

```
            if (nets.isEmpty()) Hint("The bin has not listed any networks yet. Close this and press SCAN again, or type the name.")
            nets.forEach { n ->
                Opt(vm.ssid == n.ssid, { vm.ssid = n.ssid; vm.overlay = null }) {
                    T(n.ssid, Modifier.weight(1f), weight = FontWeight.Bold, maxLines = 1)
                    T(
                        n.rssi?.let { r -> (if (r >= -55) "Strong" else if (r >= -67) "Good" else if (r >= -75) "Fair" else "Weak") + " · $r dBm" } ?: "",
                        size = 0.8f, weight = FontWeight.Bold, color = Sb.c.ink2,
                    )
                }
            }
        }
        Overlay.BtDevices -> Sheet(close) {
            SheetTitle(Ic.bluetooth, "Bins nearby")
            Hint(if (vm.btScanning) "Looking for bins with WI-FI SETUP open..." else vm.btNote ?: "Pick the bin to pair with.")
            vm.btFound.forEach { f ->
                Opt(false, { vm.btConnect(f) }) {
                    Ico(Ic.bluetooth, 19.dp, Sb.c.bt)
                    T(f.name, Modifier.weight(1f), weight = FontWeight.Bold, maxLines = 1)
                    T("${f.rssi} dBm", size = 0.8f, weight = FontWeight.Bold, color = Sb.c.ink2)
                }
            }
            Acts {
                Btn("Cancel", close, Modifier.weight(1f), sound = Sfx.BACK)
                Btn(
                    if (vm.btScanning) "SEARCHING" else "SEARCH AGAIN", { vm.btSearch() }, Modifier.weight(1f), Ic.radar, Look.TINT, Sb.c.bt,
                    enabled = !vm.btScanning, iconMod = Modifier.spin(vm.btScanning),
                )
            }
        }
        is Overlay.PinUnlock -> Sheet(close) {
            OpPad(vm, Ic.lock(true), "Operator PIN", "Collector is the default. Enter the operator PIN to calibrate, simulate and edit bins.", "UNLOCK") { vm.pinUnlock(o.first) }
        }
        is Overlay.PinSet -> Sheet(close) {
            val again = vm.pinNew.isNotEmpty()
            OpPad(
                vm, Ic.key, if (o.change) "New operator PIN" else "Create the operator PIN",
                if (again) "Type it once more to be sure."
                else "${OperatorLock.MIN} to ${OperatorLock.MAX} digits. Anyone without it stays a collector. Only an operator can change it later.",
                if (again) "SAVE" else "NEXT",
            ) { vm.pinSave(o.first, o.change) }
        }
        Overlay.PinChange -> Sheet(close) {
            OpPad(vm, Ic.lock(true), "Change operator PIN", "Enter the current PIN first.", "NEXT") { vm.pinChangeCheck() }
        }
        is Overlay.BtCode -> {
            val b = hub.bins.firstOrNull { it.id == o.id } ?: return Gone(vm)
            Sheet({ vm.btCancel() }) {
                BinScreenCode(vm, b.name)
                PinPad(
                    Ic.bluetooth, "Type the pairing code", "On ${b.name}, open WI-FI SETUP, then type the 6 digits it shows. The code changes every time.",
                    "PAIR", vm.btEntry, AppViewModel.BT_CODE_LEN, AppViewModel.BT_CODE_LEN, vm.btMsg,
                    { vm.btKey(it) }, { vm.btBack() }, { vm.btCheck(b) },
                )
            }
        }
        Overlay.NotifPre -> Sheet({ vm.notNow() }) {
            val c = Sb.c
            Box(Modifier.fillMaxWidth(), contentAlignment = Alignment.Center) { Ico(Ic.bell, 40.dp, c.tamper) }
            SheetTitle(null, "Get alerts when the app is closed", center = true)
            SheetText("Smart-Bin tells you when a bin is full, tilted, too hot, too humid or its air turns bad, with a Start pickup button on the alert.", center = true)
            Acts {
                Btn("Not now", { vm.notNow() }, Modifier.weight(1f), sound = Sfx.BACK)
                Btn("Allow alerts", allowAlerts, Modifier.weight(1f), look = Look.ACCENT, sound = Sfx.CONFIRM)
            }
        }
    }
}

@Composable
private fun muted() = Sb.c.ink2

private fun Modifier.rotate180(): Modifier = this.graphicsLayer { rotationZ = 180f }

/** A sheet whose bin has gone, removed elsewhere: it closes itself. */
@Composable
private fun Gone(vm: AppViewModel) {
    LaunchedEffect(Unit) { vm.overlay = null }
}
```

```
217
218 @Composable
219 private fun RoundKey(label: String, enabled: Boolean, sound: Sfx, onClick: () -> Unit) {
220     val c = Sb.c
221     Box(
222         Modifier.size(44.dp).then(if (enabled) Modifier else Modifier.padding(0.dp)).bump(c, 22.dp)
223             .press(enabled, sound, onClick = onClick),
224         contentAlignment = Alignment.Center,
225     ) { T(label, size = 1.3f, color = if (enabled) c.ink2 else c.ink2.soft(0.35f), lineHeight = 1f) }
226 }
227
228 /** The operator PIN sheet. */
229 @Composable
230 private fun OpPad(vm: AppViewModel, icon: androidx.compose.ui.graphics.vector.ImageVector, title: String, text: String, go:
    String, onGo: () -> Unit) =
231     PinPad(icon, title, text, go, vm.opPin, OperatorLock.MIN, OperatorLock.MAX, vm.pinMsg, { vm.pinKey(it) },
    { vm.pinBack() }, onGo)
232
233 /** A code sheet: dots for the digits, a line for what went wrong, and a keypad. */
234 @Composable
235 private fun PinPad(
236     icon: androidx.compose.ui.graphics.vector.ImageVector, title: String, text: String, go: String,
237     value: String, min: Int, max: Int, msg: Pair<Boolean, String>?,
238     onKey: (Char) -> Unit, onBack: () -> Unit, onGo: () -> Unit,
239 ) {
240     val c = Sb.c
241     Box(Modifier.fillMaxWidth(), contentAlignment = Alignment.Center) { Ico(icon, 38.dp, c.accent) }
242     SheetTitle(null, title, center = true)
243     SheetText(text, center = true)
244     Row(Modifier.fillMaxWidth().padding(vertical = 6.dp), horizontalArrangement = Arrangement.spacedBy(10.dp,
    Alignment.CenterHorizontally)) {
245         repeat(maxOf(min, value.length)) { i ->
246             Box(Modifier.size(14.dp).clip(RoundedCornerShape(7.dp)).background(if (i < value.length) c.accent else
    c.ink2.soft(0.25f)))
247         }
248     }
249     msg?.let { (ok, m) -> T(m, Modifier.fillMaxWidth(), size = 0.82f, weight = FontWeight.Bold, color = if (ok) c.ok else
    c.bad, align = androidx.compose.ui.text.style.TextAlign.Center) }
250     val rows = listOf("123", "456", "789")
251     Column(Modifier.fillMaxWidth().padding(top = 4.dp), verticalArrangement = Arrangement.spacedBy(10.dp)) {
252         rows.forEach { r ->
253             Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(10.dp)) {
254                 r.forEach { d -> PinKey(d.toString(), Modifier.weight(1f), value.length < max) { onKey(d) } }
255             }
256         }
257         Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(10.dp)) {
258             PinKey("⌫", Modifier.weight(1f), value.isNotEmpty(), Sfx.STEP_DOWN, onClick = onBack)
259             PinKey("0", Modifier.weight(1f), value.length < max) { onKey('0') }
260             PinKey(go, Modifier.weight(1f), value.length >= min, Sfx.CONFIRM, accent = true, onClick = onGo)
261         }
262     }
263 }
264
265 /**
266  * The stand-in for the bin's touchscreen while it pairs: the dark ILI9341
267  * panel with the code in big digits and how long it lasts. A real bin shows
268  * this on its own screen, never on the phone.
269  */
270 @Composable
271 private fun BinScreenCode(vm: AppViewModel, name: String) {
272     var left by remember { mutableStateOf(60) }
273     LaunchedEffect(vm.btCodeAt) {
274         while (true) {
275             left = ((AppViewModel.BT_CODE_LIFE - (System.currentTimeMillis() - vm.btCodeAt)) / 1000).toInt().coerceAtLeast(0)
276             if (left == 0) {
277                 vm.btNewCode()
278                 break
279             }
280             kotlinx.coroutines.delay(250)
281         }
282     }
283     val dim = androidx.compose.ui.graphics.Color(0xFF8A96A3)
284     Column(
285
    Modifier.fillMaxWidth().clip(RoundedCornerShape(14.dp)).background(androidx.compose.ui.graphics.Color(0xFF14181C)).padding(ho
    rizontal = 14.dp, vertical = 12.dp),
286         horizontalAlignment = Alignment.CenterHorizontally,
287         verticalArrangement = Arrangement.spacedBy(4.dp),
288     ) {
289         T("STAND-IN FOR ${name.uppercase()}'S SCREEN", size = 0.6f, spacing = 0.08f, color = dim)
290         T("PAIRING CODE", size = 0.72f, spacing = 0.12f, color = androidx.compose.ui.graphics.Color(0xFF5FD0E8))
291         T(vm.btCode.chunked(3).joinToString(" "), size = 2.1f, spacing = 0.08f, color =
    androidx.compose.ui.graphics.Color.White, tnum = true, maxLines = 1)
292         T("Expires in $left s", size = 0.72f, weight = FontWeight.Bold, color = if (left <= 10)
    androidx.compose.ui.graphics.Color(0xFFFF8A7A) else dim, tnum = true)
293     }
```

```
}

@Composable
private fun PinKey(label: String, modifier: Modifier, enabled: Boolean = true, sound: Sfx = Sfx.PIN_KEY, accent: Boolean = false, onClick: () -> Unit) {
    val c = Sb.c
    Box(
        modifier.height(54.dp).bump(c, 14.dp).press(enabled, sound, onClick = onClick),
        contentAlignment = Alignment.Center,
    ) {
        val ink = when { !enabled -> c.ink2.soft(0.35f); accent -> c.accent; else -> c.ink }
        T(label, size = if (label.length > 1 && label != "⌫") 0.86f else 1.35f, spacing = if (label.length > 1) 0.05f else 0f, color = ink, tnum = true, maxLines = 1)
    }
}

/** The Needs attention sheet: unread alerts first, then what is going on right now. */
@Composable
private fun Attention(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    SheetTitle(Ic.bell, "Needs attention")
    val pending = hub.alerts.filter { !it.read }
    val live = liveBanners(hub)
    if (pending.isNotEmpty()) {
        PopSec("Unread alerts")
        pending.take(4).forEach { a -> Banner(sevTone(a.sev), Ic.named(a.icon), a.title, "${a.meta} · ${ago(a.at)}") }
        if (pending.size > 4) PopSec("and ${pending.size - 4} more in the Alerts tab")
        Row(Modifier.padding(horizontal = 4.dp, vertical = 6.dp), horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            Btn("MARK ALL READ", { vm.markAll() }, Modifier.weight(1f), Ic.check, Look.TINT, c.ok, small = true, sound = Sfx.CONFIRM)
            Btn("OPEN ALERTS", { vm.go(Screen.ALERTS) }, Modifier.weight(1f), Ic.bell, Look.TINT, c.tamper, small = true, sound = Sfx.OPEN)
        }
    }
    if (live.isNotEmpty()) {
        PopSec("Right now")
        live.forEach { Banner(it.tone, it.icon, it.title, it.small) }
    }
    if (pending.isEmpty() && live.isEmpty()) Empty(Ic.check, "Nothing needs attention.")
}

@Composable
private fun PopSec(text: String) {
    T(text.uppercase(), Modifier.padding(start = 2.dp, top = 4.dp), size = 0.7f, spacing = 0.08f, color = Sb.c.ink2)
}

/** Pick a bin, with a search box when there are many. */
@Composable
private fun Picker(vm: AppViewModel, hub: HubState, forWhat: PickFor) {
    val c = Sb.c
    val bins = hub.bins
    var q by remember { mutableStateOf("") }
    val sel: String? = when (forWhat) {
        PickFor.CHART -> vm.chartBin
        PickFor.ENV -> vm.binFor(vm.envBin, bins)?.id
        PickFor.SET -> vm.binFor(vm.selSet, bins)?.id
        PickFor.SIM -> vm.binFor(vm.selSim, bins)?.id
    }
    val choose = { id: String? ->
        when (forWhat) {
            PickFor.CHART -> vm.chartBin = id
            PickFor.ENV -> vm.envBin = id
            PickFor.SET -> { vm.selSet = id; vm.pinShown = false }
            PickFor.SIM -> vm.selSim = id
        }
        vm.overlay = null
    }
    SheetTitle(Ic.bins, "Pick a bin") { T(forWhat.title, size = 0.68f, weight = FontWeight.Bold, color = c.ink2) }
    if (bins.size > 8) Inp(q, { q = it }, placeholder = "Search by name or place")
    val shown = bins.withIndex().filter { (_, b) -> q.isBlank() || "${b.name} ${b.place}".contains(q.trim(), ignoreCase = true) }
    Column(verticalArrangement = Arrangement.spacedBy(9.dp)) {
        if (forWhat == PickFor.CHART && q.isBlank()) Opt(sel == null || bins.none { it.id == sel }, { choose(null) }) {
            Box(Modifier.size(34.dp), contentAlignment = Alignment.Center) { Ico(Ic.layers, 21.dp, c.ink2) }
            Column(Modifier.weight(1f)) {
                T("All bins", size = 0.95f)
                T(if (bins.size > 5) "The 5 fullest are drawn" else "Every bin on one chart", size = 0.76f, weight = FontWeight.SemiBold, color = c.ink2)
            }
        }
        shown.forEach { (i, b) ->
            Opt(sel == b.id, { choose(b.id) }) {
                BinIcon(b.reading.fill, binColor(i, c.isDark), 34.dp)
                Column(Modifier.weight(1f)) {
                    T(b.name, size = 0.95f, maxLines = 1)
                    T(b.place, size = 0.76f, weight = FontWeight.SemiBold, color = c.ink2, maxLines = 1)
```

```
                }
                if (b.simulating) MTag("SIM", c.sim)
                if (b.calibrating) MTag("CAL", c.cal)
                T("${Math.round(b.reading.fill)}%", size = 0.9f, color = levelColor(b, c))
            }
        }
        if (shown.isEmpty()) Empty(Ic.bins, "No bin matches.")
    }
}

@Composable
private fun MTag(text: String, bg: androidx.compose.ui.graphics.Color) {
    Box(Modifier.clip(RoundedCornerShape(5.dp)).background(bg).padding(horizontal = 6.dp, vertical = 3.dp)) {
        T(text, size = 0.6f, spacing = 0.04f, color = androidx.compose.ui.graphics.Color.White)
    }
}
```

Android App/app/src/main/java/smartbin/ui/AppPage.kt

```
// ================================================================================
// ui/AppPage.kt - App settings (the gear key)
// Sound and vibration switches, vibration strength, keep screen on, battery
// and push-alert permissions, the sound test and the demo options. Nothing
// here talks to the bins.
// ================================================================================
package smartbin.ui

import androidx.compose.foundation.background
import androidx.compose.foundation.interaction.MutableInteractionSource
import androidx.compose.foundation.interaction.collectIsPressedAsState
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.RowScope
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.widthIn
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import kotlinx.coroutines.delay
import smartbin.BuildConfig
import smartbin.data.AlertType
import smartbin.data.Hub
import smartbin.data.HubState
import smartbin.data.Link
import smartbin.data.Role
import smartbin.data.Source
import smartbin.notify.Alerts
import smartbin.notify.Feel
import smartbin.notify.Haptics
import smartbin.notify.Sfx
import smartbin.notify.Sounds

/**
 * The gear page: everything about the app itself, kept apart from the bins'
 * own settings on the Settings tab. Sound and vibration come first, then the
 * look, the links, running in the background, and the demo options.
 */
@Composable
fun AppPage(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    val context = LocalContext.current
    ScrollPage {
        BackBar("App settings", { vm.back() })

        // Sound and vibration, the two keys the gear is mostly opened for.
        Row(horizontalArrangement = Arrangement.spacedBy(10.dp)) {
```

```
            BigSwitch(Ic.speaker, "Sound", vm.sounds, c.accent) { vm.toggleSounds() }
            BigSwitch(Ic.vibrate, "Vibration", vm.haptics, c.accent) { vm.toggleHaptics() }
        }
        Column(
            Modifier.fillMaxWidth().bump(c, 14.dp).padding(horizontal = 13.dp, vertical = 11.dp),
            verticalArrangement = Arrangement.spacedBy(9.dp),
        ) {
            T("Vibration strength", weight = FontWeight.Bold)
            TwoWay(full = true) {
                Haptics.Strength.entries.forEach { s ->
                    Tw(vm.strength == s, { vm.chooseStrength(s) }, s.label, enabled = vm.haptics)
                }
            }
        }
        SoundTest()
        LRow(Ic.sun, "Keep screen on", "Stops the phone dimming while the app is open", chevron = false) {
            Toggle(vm.keepAwake, { vm.toggleAwake() })
        }

        Row(Modifier.padding(top = 4.dp)) { Sh(Ic.palette, "Look and role") }
        LRow(Ic.palette, "Appearance", chevron = false) {
            TwoWay {
                ThemeMode.entries.forEach { m ->
                    Tw(vm.theme == m, { vm.setThemeMode(m) }, m.name.lowercase().replaceFirstChar { it.uppercase() }, grow =
null)
                }
            }
        }
        LRow(Ic.user, "Signed in as", chevron = false) {
            TwoWay {
                Tw(vm.operator, { vm.pickRole(Role.OPERATOR) }, "Operator", grow = null)
                Tw(!vm.operator, { vm.pickRole(Role.COLLECTOR) }, "Collector", grow = null)
            }
        }
        if (vm.operator) LRow(Ic.key, "Change operator PIN", "Asks for the current PIN first", onClick = { vm.changePin() })
        Hint("The app opens on the start page each time, where you pick a role. Operator needs the PIN, and drops back to
collector after 2 min in the background. After 5 wrong PINs it waits 30 s, longer each time.")

        Row(Modifier.padding(top = 4.dp)) { Sh(Ic.layers, "Links to the bins") }
        val looks = routeLooks(hub.routes, c)
        LRow(
            looks.firstOrNull()?.icon ?: Ic.wifi(false), "Connection",
            when {
                looks.isNotEmpty() -> looks.joinToString(" + ") { it.name } + if (hub.demo) " (demo)" else " answering"
                hub.link == Link.DEMO -> "Demo bins only. Pair a bin over Bluetooth to go live."
                else -> "Nothing answers. Check Wi-Fi and the cloud accounts."
            },
            looks.firstOrNull()?.color ?: if (hub.link == Link.OFFLINE) c.bad else c.warn, onClick = { vm.go(Screen.NET) },
        )
        LRow(
            Ic.bluetooth, "Bluetooth setup",
            when { vm.bt in 1..3 -> "Paired, step ${vm.bt + 1} of 4"; vm.bt >= 4 -> "Done, Bluetooth is off"; else -> "Send
Wi-Fi to a bin" },
            c.bt, onClick = { vm.go(Screen.BT) },
        )

        Row(Modifier.padding(top = 4.dp)) { Sh(Ic.bell, "Alerts in the background") }
        LRow(
            Ic.bell, "Push alerts",
            when (vm.notifications) {
                Perm.ON -> "On. Alerts arrive with the app closed."
                Perm.BLOCKED -> "Blocked in Android settings."
                Perm.OFF -> "Not turned on yet."
            },
            c.tamper, chevron = false,
        ) {
            when (vm.notifications) {
                Perm.ON -> Tag("ON", c.ok)
                Perm.OFF -> Btn("TURN ON", { vm.overlay = Overlay.NotifPre }, look = Look.TINT, cc = c.tamper, small = true)
                Perm.BLOCKED -> Btn("OPEN", { openAlertSettings(context) }, look = Look.TINT, cc = c.tamper, small = true,
sound = Sfx.OPEN)
            }
        }
        LRow(
            Ic.power, "Unrestricted battery",
            if (vm.unrestricted) "On. Android will not pause the app to save battery."
            else "Optimized. Android may pause the app after a while in the background.",
            c.warn, chevron = false,
        ) {
            if (vm.unrestricted) Tag("ON", c.ok)
            else Btn("ALLOW", { vm.askUnrestricted() }, look = Look.TINT, cc = c.warn, small = true, sound = Sfx.OPEN)
        }
        LRow(Ic.bell, "Android alert settings", "Change or silence each kind of alert while the app is closed", onClick =
{ openAlertSettings(context) })
        Hint(
            "With push alerts on, the app keeps reading the bins after you close it, every 5 s, and a quiet **Watching bins**
```

```
    notice shows while it does. " +
148                 "Unrestricted battery stops Android from pausing that. Some phones also keep their own list in the app's page
    of Android settings.",
149         )
150         if (!vm.unrestricted) Btn("OPEN APP INFO", { vm.openAppInfo() }, Modifier.fillMaxWidth(), Ic.gear, Look.TINT, sound =
    Sfx.OPEN)
151
152         Row(Modifier.padding(top = 4.dp)) { Sh(Ic.flask, "Demo", c.sim) }
153         LRow(
154             Ic.undo, "Start page",
155             if (hub.demoOn) "Demo is on. Turn it off or on from the start page." else "Pick a role again, or turn on the
    demo",
156             c.sim, onClick = { vm.toStart() },
157         )
158
159         if (vm.operator) {
160             Row(Modifier.padding(top = 4.dp)) { Sh(Ic.download, "Bin list") }
161             LRow(Ic.download, "Export bin list", "Names, places and addresses as a file", onClick =
    { vm.exportBins(hub.bins) }, chevron = false, sound = Sfx.SAVED)
162         }
163
164         if (hub.demoOn) DemoOptions(vm, hub)
165         Hint("Smart-Bin app ${BuildConfig.VERSION_NAME}, matches dashboard v2 and firmware ${hub.bins.firstOrNull
    { !it.isDemo && it.fw.isNotBlank() }?.fw ?: "v2"}", Modifier.fillMaxWidth(), align = TextAlign.Center)
166     }
167 }
168
169 /** A large on and off key for the two things people change most. */
170 @Composable
171 private fun RowScope.BigSwitch(icon: ImageVector, label: String, on: Boolean, cc: Color, onClick: () -> Unit) {
172     val c = Sb.c
173     val src = remember { MutableInteractionSource() }
174     val pressed by src.collectIsPressedAsState()
175     Column(
176         Modifier
177             .weight(1f)
178             .then(
179                 if (on || pressed) Modifier.ring(cc.soft(0.3f),
    16.dp).clip(RoundedCornerShape(16.dp)).background(c.base).sunk(c, 16.dp)
180                 else Modifier.bump(c, 16.dp)
181             )
182             .press(true, if (on) Sfx.SWITCH_OFF else Sfx.SWITCH_ON, src, onClick)
183             .padding(vertical = 14.dp),
184         horizontalAlignment = Alignment.CenterHorizontally,
185         verticalArrangement = Arrangement.spacedBy(6.dp),
186     ) {
187         Ico(icon, 28.dp, if (on) cc else c.ink2)
188         T(label, size = 0.9f, color = if (on) c.ink else c.ink2)
189         T(if (on) "ON" else "OFF", size = 0.72f, spacing = 0.08f, color = if (on) cc else c.ink2)
190     }
191 }
192
193 /** One key of the sound test: the sound, its name, and an icon and colour of its own. */
194 private class Snd(val sfx: Sfx, val label: String, val icon: ImageVector, val color: (SbColors) -> Color)
195
196 /**
197  * The sound test, the same groups, names, icons and colours as the dashboard's.
198  * Each key plays its sound and vibration even with sound off, so people can learn them.
199  */
200 private val SOUND_GROUPS: List<Pair<String, List<Snd>>> = listOf(
201     "Alerts" to listOf(
202         Snd(Sfx.ALERT_FULL, "Bin full", Ic.fill) { it.bad },
203         Snd(Sfx.ALERT_TAMPER, "Tamper", Ic.tamper) { it.tamper },
204         Snd(Sfx.ALERT_SENSOR, "Sensor limit", Ic.temp(41.0)) { it.temp },
205         Snd(Sfx.ALERT_LOCK, "Bin locked", Ic.lock(true)) { it.sim },
206         Snd(Sfx.ALERT_PICKUP, "Pickup alert", Ic.truck) { it.cal },
207         Snd(Sfx.ALERT_INFO, "Info", Ic.bell) { it.bt },
208     ),
209     "Bins and links" to listOf(
210         Snd(Sfx.LINK_UP, "Wi-Fi link", Ic.wifi(true)) { it.link },
211         Snd(Sfx.CLOUD, "Cloud link", Ic.cloud) { it.cloud },
212         Snd(Sfx.LINK_LOST, "Link lost", Ic.wifi(false)) { it.bad },
213         Snd(Sfx.BIN_LOST, "Bin quiet", Ic.radar) { it.warn },
214         Snd(Sfx.BIN_BACK, "Bin back", Ic.bins) { it.ok },
215         Snd(Sfx.CMD_OK, "Command OK", Ic.check) { it.ok },
216         Snd(Sfx.CMD_FAIL, "Command failed", Ic.cross) { it.bad },
217     ),
218     "Lid and pickup" to listOf(
219         Snd(Sfx.LOCK, "Lock", Ic.lock(true)) { it.sim },
220         Snd(Sfx.UNLOCK, "Unlock", Ic.lock(false)) { it.lid },
221         Snd(Sfx.PICKUP, "Pickup start", Ic.truck) { it.cal },
222         Snd(Sfx.DONE, "Pickup finished", Ic.check) { it.fill },
223     ),
224     "Setup" to listOf(
225         Snd(Sfx.CALIB, "Calibrate", Ic.calib) { it.cal },
226         Snd(Sfx.SIM, "Simulation", Ic.flask) { it.sim },
```

```
        Snd(Sfx.SAVED, "Saved", Ic.save) { it.ok },
        Snd(Sfx.DENIED, "Not allowed", Ic.cross) { it.warn },
    ),
    "Bluetooth Wi-Fi setup" to listOf(
        Snd(Sfx.BT_PAIRED, "Paired", Ic.bluetooth) { it.bt },
        Snd(Sfx.BT_WRONG, "Wrong code", Ic.key) { it.warn },
        Snd(Sfx.BT_BLOCKED, "Pairing stopped", Ic.lock(true)) { it.bad },
        Snd(Sfx.WIFI_JOINED, "Wi-Fi joined", Ic.wifi(true)) { it.ok },
        Snd(Sfx.WIFI_FAILED, "Wi-Fi failed", Ic.wifi(false)) { it.tamper },
        Snd(Sfx.BT_OFF, "Bluetooth off", Ic.power) { it.motion },
    ),
    "Operator PIN" to listOf(
        Snd(Sfx.OP_IN, "Operator in", Ic.sliders) { it.accent },
        Snd(Sfx.OP_OUT, "Operator out", Ic.undo) { it.weight },
        Snd(Sfx.PIN_KEY, "PIN key", Ic.pin) { it.humid },
        Snd(Sfx.PIN_WRONG, "Wrong PIN", Ic.key) { it.bad },
        Snd(Sfx.PIN_LOCKOUT, "PIN lockout", Ic.stopwatch) { it.tamper },
        Snd(Sfx.PIN_SET, "PIN saved", Ic.save) { it.gas },
    ),
    "App" to listOf(
        Snd(Sfx.BOOT, "Start", Ic.power) { it.ok },
        Snd(Sfx.LOGO, "Logo", Ic.bins) { it.gas },
        Snd(Sfx.DEMO, "Demo on", Ic.flask) { it.warn },
        Snd(Sfx.THEME_LIGHT, "Light theme", Ic.sunmoon(false)) { it.lid },
        Snd(Sfx.THEME_DARK, "Dark theme", Ic.sunmoon(true)) { it.humid },
    ),
    "Keys" to listOf(
        Snd(Sfx.TAP, "Tap", Ic.capture) { it.humid },
        Snd(Sfx.CHOICE, "Choice", Ic.sliders) { it.gas },
        Snd(Sfx.TAB, "Tab", Ic.layers) { it.motion },
        Snd(Sfx.OPEN, "Open", Ic.eye) { it.fill },
        Snd(Sfx.BACK, "Back", Ic.undo) { it.weight },
        Snd(Sfx.CONFIRM, "Confirm", Ic.check) { it.ok },
        Snd(Sfx.DANGER, "Danger", Ic.trash) { it.bad },
        Snd(Sfx.SWITCH_ON, "Switch on", Ic.power) { it.link },
        Snd(Sfx.SWITCH_OFF, "Switch off", Ic.power) { it.ink2 },
        Snd(Sfx.STEP_UP, "Step up", Ic.plus) { it.temp },
        Snd(Sfx.STEP_DOWN, "Step down", Ic.percent) { it.press },
    ),
)

@Composable
private fun SoundTest() {
    val c = Sb.c
    var open by remember { mutableStateOf(false) }
    Column(
        Modifier.fillMaxWidth().bump(c, 14.dp).padding(horizontal = 13.dp, vertical = 11.dp),
        verticalArrangement = Arrangement.spacedBy(9.dp),
    ) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            T("Sound test", Modifier.weight(1f), weight = FontWeight.Bold)
            Btn(if (open) "HIDE" else "SHOW", { open = !open }, icon = Ic.speaker, look = Look.TINT, small = true, sound =
Sfx.OPEN)
        }
        T("Every sound the app and dashboard make, in the same groups. Each one plays with its vibration, even with sound
off, so you can learn them.", size = 0.8f, weight = FontWeight.SemiBold, color = c.ink2)
        if (open) SOUND_GROUPS.forEach { (group, sounds) ->
            T(group, size = 0.78f, spacing = 0.06f, weight = FontWeight.Bold, color = c.ink2)
            sounds.chunked(3).forEach { row ->
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    row.forEach { k ->
                        // The key itself plays the sound and its buzz; with sound off, play it anyway.
                        Btn(k.label, { if (!Feel.sounds) Sounds.play(k.sfx, force = true) }, Modifier.weight(1f), k.icon,
Look.TINT, k.color(c), small = true, sound = k.sfx, textSize = 0.7f)
                    }
                    repeat(3 - row.size) { Box(Modifier.weight(1f)) }
                }
            }
        }
    }
}

/**
 * Demo options act out what simulation cannot: a faster or paused demo, links
 * coming and going, a bin going quiet, and one-tap events. They change only
 * bins with no address, are forgotten when the app closes, and while none is
 * used the demo runs exactly as before.
 */
@Composable
private fun DemoOptions(vm: AppViewModel, hub: HubState) {
    val c = Sb.c
    @Suppress("UNUSED_VARIABLE") val tick = vm.demoTick
    Row(Modifier.padding(top = 4.dp)) { Sh(Ic.flask, "Demo options", c.sim) }
    val inUse = Hub.demoInUse
    Card {
        Row(verticalAlignment = Alignment.CenterVertically, horizontalArrangement = Arrangement.spacedBy(10.dp)) {
```

```
            Ico(Ic.flask, 20.dp, if (inUse) c.sim else c.ink2)
            T(if (inUse) "Demo options are in use" else "Not in use. The demo runs as normal.", Modifier.weight(1f), size = 0.9f, weight = FontWeight.Bold)
            if (inUse) Btn("RESET", { vm.resetDemo() }, icon = Ic.undo, look = Look.TINT, cc = c.sim, small = true, sound = Sfx.CONFIRM)
        }

        Field("Fill speed", Ic.stopwatch, c.sim) {
            TwoWay(full = true) {
                listOf(0.0 to "Pause", 1.0 to "Real", 60.0 to "Normal", 600.0 to "Fast").forEach { (x, label) ->
                    Tw(vm.demoSpeed == x, { vm.changeDemoSpeed(x) }, label, cc = c.sim)
                }
            }
        }
        Hint("Normal fills a bin in one demo minute as much as in one real hour. Fast is ten times that, for showing a bin fill up in class.")

        Field("Pretend these links answer", Ic.layers, c.sim) {
            Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                val r = hub.demoLinks
                Btn("WI-FI", { vm.toggleDemoLink(0) }, Modifier.weight(1f), Ic.wifi(true), Look.TINT, c.link, small = true, pressedIn = r.local, sound = Sfx.CHOICE)
                Btn("FIREBASE", { vm.toggleDemoLink(1) }, Modifier.weight(1f), Ic.cloud, Look.TINT, c.cloud, small = true, pressedIn = r.firebase, sound = Sfx.CHOICE, textSize = 0.7f)
                Btn("TSPEAK", { vm.toggleDemoLink(2) }, Modifier.weight(1f), Ic.chart, Look.TINT, c.ts, small = true, pressedIn = r.thingspeak, sound = Sfx.CHOICE, textSize = 0.7f)
            }
        }
        Hint("The first one on, in the app's order (Wi-Fi, then Firebase, then ThingSpeak), is the one the demo bins answer on. The cards, banners and link chip change to match, in each link's colour: Wi-Fi green, Firebase orange, ThingSpeak blue.")

        Field("Cloud commands", Ic.cloud, c.sim) {
            TwoWay(full = true) {
                Tw(!hub.demoNoConfirm, { vm.setDemoNoConfirm(false) }, "Confirm in 4 s", cc = c.ok)
                Tw(hub.demoNoConfirm, { vm.setDemoNoConfirm(true) }, "Time out at 8 s", cc = c.bad)
            }
        }
        Hint("With Firebase or ThingSpeak on, LOCK, UNLOCK and COLLECT stay locked while the bin is asked. The bin either confirms after 4 s and the key flashes green, or never answers and the key flashes red at 8 s.")

        Field("Act out on a bin", Ic.bins, c.sim) {
            BinSeg(vm, hub, PickFor.SET, vm.selSet, allAll = false) { vm.selSet = it }
        }
        val b = vm.binFor(vm.selSet, hub.bins)
        if (b != null) {
            Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                Btn("FULL", { vm.demoEvent(b, Hub.DemoEvent.FULL) }, Modifier.weight(1f), Ic.fill, Look.TINT, c.bad, small = true, sound = Sfx.CONFIRM)
                Btn("TAMPER", { vm.demoEvent(b, Hub.DemoEvent.TAMPER) }, Modifier.weight(1f), Ic.tamper, Look.TINT, c.tamper, small = true, sound = Sfx.CONFIRM)
            }
            Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                Btn("BAD AIR", { vm.demoEvent(b, Hub.DemoEvent.AIR) }, Modifier.weight(1f), Ic.gas, Look.TINT, c.gas, small = true, sound = Sfx.CONFIRM)
                Btn("EMPTY", { vm.demoEvent(b, Hub.DemoEvent.EMPTY) }, Modifier.weight(1f), Ic.truck, Look.TINT, c.cal, small = true, sound = Sfx.CONFIRM)
            }
            val quiet = Hub.demoQuiet(b.id)
            Btn(
                if (quiet) "ANSWER AGAIN" else "GO QUIET", { vm.toggleDemoQuiet(b) }, Modifier.fillMaxWidth(),
                Ic.wifi(quiet), Look.TINT, if (quiet) c.ok else c.warn, small = true, sound = if (quiet) Sfx.LINK_UP else Sfx.LINK_LOST,
            )
        }
        Hint("Simulation holds sensor values by hand. These act out events instead: a bin filling past its limit, being tipped, a burst of bad air, a pickup, or a bin that stops answering. They change only the demo bins and are forgotten when the app closes.")
    }
}
```

Part 4. Android App (XML resources)

Android App/app/src/main/AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!--
================================================================================
AndroidManifest.xml - what the app is and what it may do
Lists the permissions, the single activity (MainActivity), the two launcher
aliases (light and dark logo), the background WatchService and the
notification action receiver. A missing permission here fails silently at
runtime: for example, without POST_NOTIFICATIONS no alert is ever shown on
```

```
Android 13 and newer.
================================================================================
-->
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <!-- Network: the bins' local API, Firebase and ThingSpeak. -->
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <!-- The app's own vibration patterns for alerts, warnings and the link to the bins. -->
    <uses-permission android:name="android.permission.VIBRATE" />
    <!-- Keeps reading the bins for alerts while the app is closed. -->
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_CONNECTED_DEVICE" />
    <uses-permission android:name="android.permission.CHANGE_NETWORK_STATE" />
    <!-- The Unrestricted battery option in App settings asks for this. -->
    <uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS" />
    <!-- Wi-Fi setup over Bluetooth: find the bin, pair with its code, hand over the network. -->
    <uses-permission android:name="android.permission.BLUETOOTH" android:maxSdkVersion="30" />
    <uses-permission android:name="android.permission.BLUETOOTH_ADMIN" android:maxSdkVersion="30" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" android:maxSdkVersion="30" />
    <uses-permission android:name="android.permission.BLUETOOTH_SCAN" android:usesPermissionFlags="neverForLocation"
tools:targetApi="s" />
    <uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
    <uses-feature android:name="android.hardware.bluetooth_le" android:required="false" />

    <!-- usesCleartextTraffic="true" below: the bins answer on plain http://
         inside the local network, so Android must allow non-TLS traffic.
         Without it every local call fails and the app falls back to the clouds.
         allowBackup="false" and the two rules files keep the saved settings
         (with the sealed Firebase password) out of backups. -->
    <application
        android:name=".SmartBinApp"
        android:allowBackup="false"
        android:usesCleartextTraffic="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.SmartBin">
        <!-- configChanges: rotation, screen size and dark-mode changes do not
             recreate the activity; Compose redraws for the new size or theme. -->
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:configChanges="orientation|screenSize|screenLayout|smallestScreenSize|keyboardHidden|uiMode"
            android:label="@string/app_name"
            android:launchMode="singleTop"
            android:windowSoftInputMode="adjustResize" />
        <!-- The home-screen entry. There are two, one per logo, and only one is
             switched on at a time: the light logo or the dark one, following the
             theme the app was last left in. MainActivity switches them. -->
        <activity-alias
            android:name=".LauncherLight"
            android:enabled="true"
            android:exported="true"
            android:icon="@mipmap/ic_launcher"
            android:roundIcon="@mipmap/ic_launcher_round"
            android:label="@string/app_name"
            android:targetActivity=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity-alias>
        <activity-alias
            android:name=".LauncherDark"
            android:enabled="false"
            android:exported="true"
            android:icon="@mipmap/ic_launcher_dark"
            android:roundIcon="@mipmap/ic_launcher_dark_round"
            android:label="@string/app_name"
            android:targetActivity=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity-alias>
        <!-- stopWithTask: swiping the app away from recent apps stops the
             service, so alerts end when the user closes the app. -->
        <service
            android:name=".notify.WatchService"
            android:exported="false"
```

```
            android:stopWithTask="true"
            android:foregroundServiceType="connectedDevice" />
        <receiver
            android:name=".notify.ActionReceiver"
            android:exported="false" />
    </application>

</manifest>
```

Android App/app/src/main/res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Colors Android itself needs before Compose starts: icon backgrounds and
     the splash background. The app's own palette lives in ui/Theme.kt. -->
<resources>
    <!-- The page colours of the mockup's light and dark themes. -->
    <color name="ic_launcher_bg_light">#FFE4DCD2</color>
    <color name="ic_launcher_bg_dark">#FF2B2622</color>
    <color name="splash_light">#FFE4DCD2</color>
</resources>
```

Android App/app/src/main/res/values/strings.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- The app name under the launcher icon. All other text is in the Kotlin code. -->
<resources>
    <string name="app_name">Smart-Bin</string>
</resources>
```

Android App/app/src/main/res/values/themes.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Window theme shown for the instant before Compose draws: no action bar,
     and the mockup's light page color so the start has no white flash. -->
<resources>
    <style name="Theme.SmartBin" parent="android:Theme.Material.Light.NoActionBar">
        <item name="android:windowBackground">@color/splash_light</item>
    </style>
</resources>
```

Android App/app/src/main/res/xml/backup_rules.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Nothing is backed up: the Firebase password is sealed with a key that never leaves this phone. -->
<full-backup-content>
    <exclude domain="sharedpref" path="." />
</full-backup-content>
```

Android App/app/src/main/res/xml/data_extraction_rules.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Android 12+ version of backup_rules.xml: the saved settings
     (SharedPreferences) are left out of cloud backup and phone-to-phone
     transfer, for the same reason. -->
<data-extraction-rules>
    <cloud-backup>
        <exclude domain="sharedpref" path="." />
    </cloud-backup>
    <device-transfer>
        <exclude domain="sharedpref" path="." />
    </device-transfer>
</data-extraction-rules>
```

Android App/app/src/main/res/mipmap-anydpi-v26/ic_launcher.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Adaptive launcher icon, light theme: background color, foreground logo and
     the one-color version Android 13+ uses for themed icons. -->
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_bg_light" />
    <foreground android:drawable="@drawable/ic_launcher_fg_light" />
    <monochrome android:drawable="@drawable/ic_launcher_mono" />
</adaptive-icon>
```

Android App/app/src/main/res/mipmap-anydpi-v26/ic_launcher_round.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Same icon for launchers that ask for a round one. -->
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
```

```
    <background android:drawable="@color/ic_launcher_bg_light" />
    <foreground android:drawable="@drawable/ic_launcher_fg_light" />
    <monochrome android:drawable="@drawable/ic_launcher_mono" />
</adaptive-icon>
```

Android App/app/src/main/res/mipmap-anydpi-v26/ic_launcher_dark.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Adaptive launcher icon, dark theme (used by the LauncherDark alias). -->
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_bg_dark" />
    <foreground android:drawable="@drawable/ic_launcher_fg_dark" />
    <monochrome android:drawable="@drawable/ic_launcher_mono" />
</adaptive-icon>
```

Android App/app/src/main/res/mipmap-anydpi-v26/ic_launcher_dark_round.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Dark icon for launchers that ask for a round one. -->
<adaptive-icon xmlns:android="http://schemas.android.com/apk/res/android">
    <background android:drawable="@color/ic_launcher_bg_dark" />
    <foreground android:drawable="@drawable/ic_launcher_fg_dark" />
    <monochrome android:drawable="@drawable/ic_launcher_mono" />
</adaptive-icon>
```

Android App/app/src/main/res/drawable/ic_launcher_fg_light.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Home-screen logo, light: the still frame of the app's v2 logo (ui/Logo.kt), a bin with a
     fill wave, a controller chip, a status light on the lid and a Wi-Fi signal. -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:aapt="http://schemas.android.com/aapt"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <group android:scaleX="1.1" android:scaleY="1.1" android:translateX="18.8" android:translateY="21">
        <path android:pathData="M32,0a12,12 0,1 1,0 24a12,12 0,1 1,0 -24z">
            <aapt:attr name="android:fillColor">
                <gradient android:type="radial" android:centerX="32" android:centerY="12" android:gradientRadius="12"
                    android:startColor="#731E8E3E" android:endColor="#001E8E3E" />
            </aapt:attr>
        </path>
        <path android:pathData="M27.8,12.6a6,6 0,0 1,8.4 0" android:strokeColor="#FF1E8E3E" android:strokeWidth="2.6"
android:strokeLineCap="round" />
        <path android:pathData="M24.2,8.9a11,11 0,0 1,15.6 0" android:strokeColor="#B31E8E3E" android:strokeWidth="2.6"
android:strokeLineCap="round" />
        <path android:pathData="M20.6,5.2a16,16 0,0 1,22.8 0" android:strokeColor="#661E8E3E" android:strokeWidth="2.6"
android:strokeLineCap="round" />
        <path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z">
            <aapt:attr name="android:fillColor">
                <gradient android:type="linear" android:startX="17.5" android:startY="0" android:endX="46.5" android:endY="0"
                    android:startColor="#0D1E8E3E" android:centerColor="#291E8E3E" android:endColor="#0D1E8E3E" />
            </aapt:attr>
        </path>
        <group>
            <clip-path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z" />
            <path android:pathData="M0,37q5,-3.6 10,0t10,0t10,0t10,0t10,0t10,0t10,0v30H0z" android:fillAlpha="0.35">
                <aapt:attr name="android:fillColor">
                    <gradient android:type="linear" android:startX="0" android:startY="34" android:endX="0" android:endY="57"
                        android:startColor="#FF1E8E3E" android:endColor="#FF0D8A8A" />
                </aapt:attr>
            </path>
            <path android:pathData="M-5,39q5,-3.6 10,0t10,0t10,0t10,0t10,0t10,0t10,0v30H-5z">
                <aapt:attr name="android:fillColor">
                    <gradient android:type="linear" android:startX="0" android:startY="34" android:endX="0" android:endY="57"
                        android:startColor="#FF1E8E3E" android:endColor="#FF0D8A8A" />
                </aapt:attr>
            </path>
        </group>
        <path android:pathData="M22,30.5h3.4M22,34h2.2" android:strokeColor="#CC1E8E3E" android:strokeWidth="1.8"
android:strokeLineCap="round" />
        <path android:pathData="M29.3,42h5.4a1.8,1.8 0,0 1,1.8 1.8v5.4a1.8,1.8 0,0 1,-1.8 1.8h-5.4a1.8,1.8 0,0 1,-1.8 -1.8v-
5.4a1.8,1.8 0,0 1,1.8 -1.8z"
            android:strokeColor="#FFFFFFFF" android:strokeWidth="2" />
        <path android:pathData="M32,42v-3.2M32,51v2.5M27.5,46.5h-4M36.5,46.5h4" android:strokeColor="#FFFFFFFF"
android:strokeWidth="1.8" android:strokeLineCap="round" />
        <path android:pathData="M30.5,37.6a1.5,1.5 0,1 0,3 0a1.5,1.5 0,1 0,-3 0zM21.1,46.5a1.5,1.5 0,1 0,3 0a1.5,1.5 0,1 0,-3
0zM39.9,46.5a1.5,1.5 0,1 0,3 0a1.5,1.5 0,1 0,-3 0z"
            android:fillColor="#FFFFFFFF" />
        <path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z"
android:strokeColor="#FF2A241E" android:strokeWidth="3" android:strokeLineJoin="round" />
        <path android:pathData="M13,22.5h38" android:strokeColor="#FF2A241E" android:strokeWidth="3.4"
```

```
android:strokeLineCap="round" />
        <path android:pathData="M27,22.5v-3.4h10v3.4" android:strokeColor="#FF2A241E" android:strokeWidth="3"
android:strokeLineCap="round" android:strokeLineJoin="round" />
        <path android:pathData="M44.2,22.5a1.3,1.3 0,1 0,2.6 0a1.3,1.3 0,1 0,-2.6 0z" android:fillColor="#FF1E8E3E" />
    </group>
</vector>
```

Android App/app/src/main/res/drawable/ic_launcher_fg_dark.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Home-screen logo, dark: the still frame of the app's v2 logo (ui/Logo.kt), a bin with a
     fill wave, a controller chip, a status light on the lid and a Wi-Fi signal. -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:aapt="http://schemas.android.com/aapt"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <group android:scaleX="1.1" android:scaleY="1.1" android:translateX="18.8" android:translateY="21">
        <path android:pathData="M32,0a12,12 0,1 1,0 24a12,12 0,1 1,0 -24z">
            <aapt:attr name="android:fillColor">
                <gradient android:type="radial" android:centerX="32" android:centerY="12" android:gradientRadius="12"
                    android:startColor="#734FBF6E" android:endColor="#004FBF6E" />
            </aapt:attr>
        </path>
        <path android:pathData="M27.8,12.6a6,6 0,0 1,8.4 0" android:strokeColor="#FF4FBF6E" android:strokeWidth="2.6"
android:strokeLineCap="round" />
        <path android:pathData="M24.2,8.9a11,11 0,0 1,15.6 0" android:strokeColor="#B34FBF6E" android:strokeWidth="2.6"
android:strokeLineCap="round" />
        <path android:pathData="M20.6,5.2a16,16 0,0 1,22.8 0" android:strokeColor="#664FBF6E" android:strokeWidth="2.6"
android:strokeLineCap="round" />
        <path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z">
            <aapt:attr name="android:fillColor">
                <gradient android:type="linear" android:startX="17.5" android:startY="0" android:endX="46.5" android:endY="0"
                    android:startColor="#0D4FBF6E" android:centerColor="#294FBF6E" android:endColor="#0D4FBF6E" />
            </aapt:attr>
        </path>
        <group>
            <clip-path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z" />
            <path android:pathData="M0,37q5,-3.6 10,0t10,0t10,0t10,0t10,0t10,0t10,0v30H0z" android:fillAlpha="0.35">
                <aapt:attr name="android:fillColor">
                    <gradient android:type="linear" android:startX="0" android:startY="34" android:endX="0" android:endY="57"
                        android:startColor="#FF4FBF6E" android:endColor="#FF3FC7C2" />
                </aapt:attr>
            </path>
            <path android:pathData="M-5,39q5,-3.6 10,0t10,0t10,0t10,0t10,0t10,0t10,0v30H-5z">
                <aapt:attr name="android:fillColor">
                    <gradient android:type="linear" android:startX="0" android:startY="34" android:endX="0" android:endY="57"
                        android:startColor="#FF4FBF6E" android:endColor="#FF3FC7C2" />
                </aapt:attr>
            </path>
        </group>
        <path android:pathData="M22,30.5h3.4M22,34h2.2" android:strokeColor="#CC4FBF6E" android:strokeWidth="1.8"
android:strokeLineCap="round" />
        <path android:pathData="M29.3,42h5.4a1.8,1.8 0,0 1,1.8 1.8v5.4a1.8,1.8 0,0 1,-1.8 1.8h-5.4a1.8,1.8 0,0 1,-1.8 -1.8v-
5.4a1.8,1.8 0,0 1,1.8 -1.8z"
            android:strokeColor="#FF13100D" android:strokeWidth="2" />
        <path android:pathData="M32,42v-3.2M32,51v2.5M27.5,46.5h-4M36.5,46.5h4" android:strokeColor="#FF13100D"
android:strokeWidth="1.8" android:strokeLineCap="round" />
        <path android:pathData="M30.5,37.6a1.5,1.5 0,1 0,3 0a1.5,1.5 0,1 0,-3 0zM21.1,46.5a1.5,1.5 0,1 0,3 0a1.5,1.5 0,1 0,-3
0zM39.9,46.5a1.5,1.5 0,1 0,3 0a1.5,1.5 0,1 0,-3 0z"
            android:fillColor="#FF13100D" />
        <path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z"
android:strokeColor="#FFEFE7DC" android:strokeWidth="3" android:strokeLineJoin="round" />
        <path android:pathData="M13,22.5h38" android:strokeColor="#FFEFE7DC" android:strokeWidth="3.4"
android:strokeLineCap="round" />
        <path android:pathData="M27,22.5v-3.4h10v3.4" android:strokeColor="#FFEFE7DC" android:strokeWidth="3"
android:strokeLineCap="round" android:strokeLineJoin="round" />
        <path android:pathData="M44.2,22.5a1.3,1.3 0,1 0,2.6 0a1.3,1.3 0,1 0,-2.6 0z" android:fillColor="#FF4FBF6E" />
    </group>
</vector>
```

Android App/app/src/main/res/drawable/ic_launcher_mono.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Themed-icon version of the logo: Android tints it, so it is one colour with the chip cut out. -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="108dp"
    android:height="108dp"
    android:viewportWidth="108"
    android:viewportHeight="108">
    <group android:scaleX="1.1" android:scaleY="1.1" android:translateX="18.8" android:translateY="21">
        <path android:pathData="M27.8,12.6a6,6 0,0 1,8.4 0M24.2,8.9a11,11 0,0 1,15.6 0M20.6,5.2a16,16 0,0 1,22.8 0"
android:strokeColor="#FF000000" android:strokeWidth="2.6" android:strokeLineCap="round" />
```

```
        <path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z"
android:strokeColor="#FF000000" android:strokeWidth="3" android:strokeLineJoin="round" />
        <group>
            <clip-path android:pathData="M17.5,26h29l-2.7,27.6a3.2,3.2 0,0 1,-3.2 2.9H23.4a3.2,3.2 0,0 1,-3.2 -2.9z" />
            <path android:pathData="M-5,39q5,-3.6 10,0t10,0t10,0t10,0t10,0t10,0t10,0v30H-5z" android:fillColor="#FF000000"
android:fillAlpha="0.8" />
        </group>
        <path android:pathData="M13,22.5h38" android:strokeColor="#FF000000" android:strokeWidth="3.4"
android:strokeLineCap="round" />
        <path android:pathData="M27,22.5v-3.4h10v3.4" android:strokeColor="#FF000000" android:strokeWidth="3"
android:strokeLineCap="round" android:strokeLineJoin="round" />
    </group>
</vector>
```

Android App/app/src/main/res/drawable/ic_stat_bin.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!-- Status bar icon: the bin with its fill line, one colour as Android requires. -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp"
    android:height="24dp"
    android:viewportWidth="24"
    android:viewportHeight="24">
    <path
        android:pathData="M6.6,9.5l1.1,11h8.6l1.1,-11"
        android:strokeColor="#FFFFFFFF"
        android:strokeWidth="2"
        android:strokeLineCap="round"
        android:strokeLineJoin="round" />
    <path
        android:pathData="M4.5,9.5h15M9.6,7.1h4.8"
        android:strokeColor="#FFFFFFFF"
        android:strokeWidth="2"
        android:strokeLineCap="round" />
    <path
        android:pathData="M8,15.2c1.3,-0.9 2.7,-0.9 4,0s2.7,0.9 4,0"
        android:strokeColor="#FFFFFFFF"
        android:strokeWidth="1.8"
        android:strokeLineCap="round" />
</vector>
```

Part 5. Build and Test Tools

Android App/settings.gradle.kts

```
// ================================================================================
// settings.gradle.kts - names the Gradle project ("SmartBin") and its one
// module (:app), and says where plugins and libraries are downloaded from.
// ================================================================================
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.android.*")
                includeGroupByRegex("com\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
    }
}
plugins {
    id("org.gradle.toolchains.foojay-resolver-convention") version "1.0.0"
}
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
    }
}

rootProject.name = "SmartBin"
include(":app")
```

Android App/build.gradle.kts

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
// Root build file: declares the Android and Compose plugins once (apply false)
```

```
// so the :app module can use them without repeating versions.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.compose) apply false
}
```

Android App/app/build.gradle.kts

```
// ============================================================================
// app/build.gradle.kts - how the APK is built
// applicationId edu.tup.smartbin, Android 8.0 (API 26) and newer. Bump
// versionCode and versionName for every release; the GitHub Actions workflow
// names the APK after versionName ("Smart-Bin v2.2.apk").
// Build from the "Android App" folder with:  ./gradlew :app:assembleDebug
// ============================================================================
import java.util.Properties

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
}

android {
    // The code lives in the short package "smartbin" so the source folders stay shallow.
    namespace = "smartbin"
    compileSdk {
        version = release(37)
    }

    defaultConfig {
        applicationId = "edu.tup.smartbin"
        minSdk = 26
        targetSdk = 37
        versionCode = 6
        versionName = "2.2"

        // The team's Firebase account, read from local.properties (git-ignored) so the
        // password never reaches the public repository. Missing values become empty.
        val local = Properties().apply {
            rootProject.file("local.properties").takeIf { it.exists() }?.inputStream()?.use(::load)
        }
        fun prop(key: String) = "\"" + (local.getProperty(key) ?: "").replace("\\", "\\\\").replace("\"", "\\\"") + "\""
        buildConfigField("String", "FB_URL", prop("sb.fb.url"))
        buildConfigField("String", "FB_KEY", prop("sb.fb.key"))
        buildConfigField("String", "FB_EMAIL", prop("sb.fb.email"))
        buildConfigField("String", "FB_PASSWORD", prop("sb.fb.password"))
    }

    buildTypes {
        release {
            optimization {
                enable = true
                packageScope = setOf("androidx.**", "kotlin.**", "kotlinx.**")
            }
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    buildFeatures {
        compose = true
        buildConfig = true
    }
}

dependencies {
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.activity.compose)
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.compose.ui.graphics)
    implementation(libs.androidx.compose.ui.tooling.preview)
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.7")
    debugImplementation(libs.androidx.compose.ui.tooling)
}
```

Android App/gradle.properties

```
# ------------------------------------------------------------------------
# gradle.properties - Gradle settings shared by every build of the app.
# If a build runs out of memory, raise -Xmx below.
```

```
# ------------------------------------------------------------------------
# Project-wide Gradle settings.
# IDE (e.g. Android Studio) users:
# Gradle settings configured through the IDE *will override*
# any settings specified in this file.
# For more details on how to configure your build environment visit
# http://www.gradle.org/docs/current/userguide/build_environment.html
# Specifies the JVM arguments used for the daemon process.
# The setting is particularly useful for tweaking memory settings.
org.gradle.jvmargs=-Xmx2048m -Dfile.encoding=UTF-8
# When configured, Gradle will run in incubating parallel mode.
# This option should only be used with decoupled projects. For more details, visit
# https://developer.android.com/r/tools/gradle-multi-project-decoupled-projects
# org.gradle.parallel=true
# When enabled, the Configuration Cache allows Gradle to skip the configuration
# phase entirely if nothing that affects the build configuration (such as build scripts)
# has changed. Additionally, Gradle applies performance optimizations to task execution.
org.gradle.configuration-cache=true
# Kotlin code style for this project: "official" or "obsolete":
kotlin.code.style=official
```

Android App/gradle/libs.versions.toml

```
# ------------------------------------------------------------------------
# libs.versions.toml - Gradle version catalog: one place for the version of
# every library and plugin. build.gradle.kts refers to them as libs.<name>.
# ------------------------------------------------------------------------
[versions]
agp = "9.4.1"
coreKtx = "1.10.1"
junit = "4.13.2"
junitVersion = "1.1.5"
espressoCore = "3.5.1"
lifecycleRuntimeKtx = "2.6.1"
activityCompose = "1.8.0"
kotlin = "2.2.10"
composeBom = "2026.02.01"

[libraries]
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
junit = { group = "junit", name = "junit", version.ref = "junit" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitVersion" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref = "espressoCore" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx", version.ref =
"lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose", version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref = "composeBom" }
androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-compose-ui-tooling = { group = "androidx.compose.ui", name = "ui-tooling" }
androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name = "ui-tooling-preview" }
androidx-compose-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-compose-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

Android App/gradle/wrapper/gradle-wrapper.properties

```
# gradle-wrapper.properties - the Gradle version gradlew downloads (9.6.0) and
# the checksum it verifies before running it.
#Sat Oct 10 09:56:41 PST 2026
distributionBase=GRADLE_USER_HOME
distributionPath=wrapper/dists
distributionSha256Sum=bbaeb2fef8710818cf0e261201dab964c572f92b942812df0c3620d62a529a01
distributionUrl=https\://services.gradle.org/distributions/gradle-9.6.0-bin.zip
networkTimeout=10000
validateDistributionUrl=true
zipStoreBase=GRADLE_USER_HOME
zipStorePath=wrapper/dists
```

Android App/gradle/gradle-daemon-jvm.properties

```
# gradle-daemon-jvm.properties - the Java version (25) Gradle runs on, with a
# download link per operating system. Generated; do not edit by hand.
#This file is generated by updateDaemonJvm
toolchainUrl.FREE_BSD.AARCH64=https\://api.foojay.io/disco/v3.0/ids/cf726b4a1c84b50457225f9bba6d7650/redirect
toolchainUrl.FREE_BSD.X86_64=https\://api.foojay.io/disco/v3.0/ids/fa1e318c287360478e3c83a9a3ef1007/redirect
toolchainUrl.LINUX.AARCH64=https\://api.foojay.io/disco/v3.0/ids/cf726b4a1c84b50457225f9bba6d7650/redirect
toolchainUrl.LINUX.X86_64=https\://api.foojay.io/disco/v3.0/ids/fa1e318c287360478e3c83a9a3ef1007/redirect
toolchainUrl.MAC_OS.AARCH64=https\://api.foojay.io/disco/v3.0/ids/c2dd35c9d0aaf0ba6ad0791320f99dfc/redirect
```

```
toolchainUrl.MAC_OS.X86_64=https\://api.foojay.io/disco/v3.0/ids/e5810bd7fd1f8a586644409d395a7e55/redirect
toolchainUrl.UNIX.AARCH64=https\://api.foojay.io/disco/v3.0/ids/cf726b4a1c84b50457225f9bba6d7650/redirect
toolchainUrl.UNIX.X86_64=https\://api.foojay.io/disco/v3.0/ids/fa1e318c287360478e3c83a9a3ef1007/redirect
toolchainUrl.WINDOWS.AARCH64=https\://api.foojay.io/disco/v3.0/ids/7b3c4877c0749019e6805bb61e421497/redirect
toolchainUrl.WINDOWS.X86_64=https\://api.foojay.io/disco/v3.0/ids/d76df094a9cbbabd3b08251f9e61444a/redirect
toolchainVersion=25
```

Android App/gradlew

```
#!/bin/sh

#
# gradlew - Gradle wrapper start script for Linux and macOS, generated by Gradle
# (Apache License 2.0). It downloads the Gradle version named in
# gradle-wrapper.properties on first use, then runs it.
# Copyright © 2015 the original authors.
#
# Licensed under the Apache License, Version 2.0 (the "License");
# you may not use this file except in compliance with the License.
# You may obtain a copy of the License at
#
#      https://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing, software
# distributed under the License is distributed on an "AS IS" BASIS,
# WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
# See the License for the specific language governing permissions and
# limitations under the License.
#
# SPDX-License-Identifier: Apache-2.0
#

##############################################################################
#
#   Gradle start up script for POSIX generated by Gradle.
#
#   Important for running:
#
#   (1) You need a POSIX-compliant shell to run this script. If your /bin/sh is
#       noncompliant, but you have some other compliant shell such as ksh or
#       bash, then to run this script, type that shell name before the whole
#       command line, like:
#
#           ksh Gradle
#
#       Busybox and similar reduced shells will NOT work, because this script
#       requires all of these POSIX shell features:
#         * functions;
#         * expansions «$var», «${var}», «${var:-default}», «${var+SET}»,
#           «${var#prefix}», «${var%suffix}», and «$( cmd )»;
#         * compound commands having a testable exit status, especially «case»;
#         * various built-in commands including «command», «set», and «ulimit».
#
#   Important for patching:
#
#   (2) This script targets any POSIX shell, so it avoids extensions provided
#       by Bash, Ksh, etc; in particular arrays are avoided.
#
#       The "traditional" practice of packing multiple parameters into a
#       space-separated string is a well documented source of bugs and security
#       problems, so this is (mostly) avoided, by progressively accumulating
#       options in "$@", and eventually passing that to Java.
#
#       Where the inherited environment variables (DEFAULT_JVM_OPTS, JAVA_OPTS,
#       and GRADLE_OPTS) rely on word-splitting, this is performed explicitly;
#       see the in-line comments for details.
#
#       There are tweaks for specific operating systems such as AIX, CygWin,
#       Darwin, MinGW, and NonStop.
#
#   (3) This script is generated from the Groovy template
#       https://github.com/gradle/gradle/blob/HEAD/platforms/jvm/plugins-
application/src/main/resources/org/gradle/api/internal/plugins/unixStartScript.txt
#       within the Gradle project.
#
#       You can find Gradle at https://github.com/gradle/gradle/.
#
##############################################################################

# Attempt to set APP_HOME

# Resolve links: $0 may be a link
app_path=$0

# Need this for daisy-chained symlinks.
```

```
while
    APP_HOME=${app_path%"${app_path##*/}"}  # leaves a trailing /; empty if no leading path
    [ -h "$app_path" ]
do
    ls=$( ls -ld "$app_path" )
    link=${ls#*' -> '}
    case $link in             #(
      /*)   app_path=$link ;; #(
      *)    app_path=$APP_HOME$link ;;
    esac
done

# This is normally unused
# shellcheck disable=SC2034
APP_BASE_NAME=${0##*/}
# Discard cd standard output in case $CDPATH is set (https://github.com/gradle/gradle/issues/25036)
APP_HOME=$( cd -P "${APP_HOME:-./}" > /dev/null && printf '%s\n' "$PWD" ) || exit

# Use the maximum available, or set MAX_FD != -1 to use that value.
MAX_FD=maximum

warn () {
    echo "$*"
} >&2

die () {
    echo
    echo "$*"
    echo
    exit 1
} >&2

# OS specific support (must be 'true' or 'false').
cygwin=false
msys=false
darwin=false
nonstop=false
case "$( uname )" in                #(
  CYGWIN* )         cygwin=true  ;; #(
  Darwin* )         darwin=true  ;; #(
  MSYS* | MINGW* )  msys=true    ;; #(
  NONSTOP* )        nonstop=true ;;
esac

CLASSPATH="\\\"\\\""


# Determine the Java command to use to start the JVM.
if [ -n "$JAVA_HOME" ] ; then
    if [ -x "$JAVA_HOME/jre/sh/java" ] ; then
        # IBM's JDK on AIX uses strange locations for the executables
        JAVACMD=$JAVA_HOME/jre/sh/java
    else
        JAVACMD=$JAVA_HOME/bin/java
    fi
    if [ ! -x "$JAVACMD" ] ; then
        die "ERROR: JAVA_HOME is set to an invalid directory: $JAVA_HOME

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
else
    JAVACMD=java
    if ! command -v java >/dev/null 2>&1
    then
        die "ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH.

Please set the JAVA_HOME variable in your environment to match the
location of your Java installation."
    fi
fi

# Increase the maximum file descriptors if we can.
if ! "$cygwin" && ! "$darwin" && ! "$nonstop" ; then
    case $MAX_FD in #(
      max*)
        # In POSIX sh, ulimit -H is undefined. That's why the result is checked to see if it worked.
        # shellcheck disable=SC2039,SC3045
        MAX_FD=$( ulimit -H -n ) ||
            warn "Could not query maximum file descriptor limit"
    esac
    case $MAX_FD in  #(
      '' | soft) :;; #(
      *)
        # In POSIX sh, ulimit -n is undefined. That's why the result is checked to see if it worked.
        # shellcheck disable=SC2039,SC3045
```

```
        ulimit -n "$MAX_FD" ||
            warn "Could not set maximum file descriptor limit to $MAX_FD"
    esac
fi

# Collect all arguments for the java command, stacking in reverse order:
#   * args from the command line
#   * the main class name
#   * -classpath
#   * -D...appname settings
#   * --module-path (only if needed)
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, and GRADLE_OPTS environment variables.

# For Cygwin or MSYS, switch paths to Windows format before running java
if "$cygwin" || "$msys" ; then
    APP_HOME=$( cygpath --path --mixed "$APP_HOME" )
    CLASSPATH=$( cygpath --path --mixed "$CLASSPATH" )

    JAVACMD=$( cygpath --unix "$JAVACMD" )

    # Now convert the arguments - kludge to limit ourselves to /bin/sh
    for arg do
        if
            case $arg in                                #(
              -*)   false ;;                            # don't mess with options #(
              /?*)  t=${arg#/} t=/${t%%/*}              # looks like a POSIX filepath
                    [ -e "$t" ] ;;                      #(
              *)    false ;;
            esac
        then
            arg=$( cygpath --path --ignore --mixed "$arg" )
        fi
        # Roll the args list around exactly as many times as the number of
        # args, so each arg winds up back in the position where it started, but
        # possibly modified.
        #
        # NB: a `for` loop captures its iteration list before it begins, so
        # changing the positional parameters here affects neither the number of
        # iterations, nor the values presented in `arg`.
        shift                   # remove old arg
        set -- "$@" "$arg"      # push replacement arg
    done
fi


# Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to this script.
DEFAULT_JVM_OPTS='"-Xmx64m" "-Xms64m"'

# Collect all arguments for the java command:
#   * DEFAULT_JVM_OPTS, JAVA_OPTS, and optsEnvironmentVar are not allowed to contain shell fragments,
#     and any embedded shellness will be escaped.
#   * For example: A user cannot expect ${Hostname} to be expanded, as it is an environment variable and will be
#     treated as '${Hostname}' itself on the command line.

set -- \
        "-Dorg.gradle.appname=$APP_BASE_NAME" \
        -classpath "$CLASSPATH" \
        -jar "$APP_HOME/gradle/wrapper/gradle-wrapper.jar" \
        "$@"

# Stop when "xargs" is not available.
if ! command -v xargs >/dev/null 2>&1
then
    die "xargs is not available"
fi

# Use "xargs" to parse quoted args.
#
# With -n1 it outputs one arg per line, with the quotes and backslashes removed.
#
# In Bash we could simply go:
#
#   readarray ARGS < <( xargs -n1 <<<"$var" ) &&
#   set -- "${ARGS[@]}" "$@"
#
# but POSIX shell has neither arrays nor command substitution, so instead we
# post-process each arg (as a line of input to sed) to backslash-escape any
# character that might be a shell metacharacter, then use eval to reverse
# that process (while maintaining the separation between arguments), and wrap
# the whole thing up as a single "set" statement.
#
# This will of course break if any of these variables contains a newline or
# an unmatched quote.
#

eval "set -- $(
```

```
        printf '%s\n' "$DEFAULT_JVM_OPTS $JAVA_OPTS $GRADLE_OPTS" |
        xargs -n1 |
        sed ' s~[^-[:alnum:]+,./:=@_]~\\&~g; ' |
        tr '\n' ' '
    )" '"$@"'

exec "$JAVACMD" "$@"
```

Android App/gradlew.bat

```
@rem gradlew.bat - the same Gradle wrapper start script for Windows, generated by Gradle.
@rem
@rem Copyright 2015 the original author or authors.
@rem
@rem Licensed under the Apache License, Version 2.0 (the "License");
@rem you may not use this file except in compliance with the License.
@rem You may obtain a copy of the License at
@rem
@rem      https://www.apache.org/licenses/LICENSE-2.0
@rem
@rem Unless required by applicable law or agreed to in writing, software
@rem distributed under the License is distributed on an "AS IS" BASIS,
@rem WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
@rem See the License for the specific language governing permissions and
@rem limitations under the License.
@rem
@rem SPDX-License-Identifier: Apache-2.0
@rem

@if "%DEBUG%"=="" @echo off
@rem ##########################################################################
@rem
@rem  Gradle startup script for Windows
@rem
@rem ##########################################################################

@rem Set local scope for the variables with windows NT shell
if "%OS%"=="Windows_NT" setlocal

set DIRNAME=%~dp0
if "%DIRNAME%"=="" set DIRNAME=.
@rem This is normally unused
set APP_BASE_NAME=%~n0
set APP_HOME=%DIRNAME%

@rem Resolve any "." and ".." in APP_HOME to make it shorter.
for %%i in ("%APP_HOME%") do set APP_HOME=%%~fi

@rem Add default JVM options here. You can also use JAVA_OPTS and GRADLE_OPTS to pass JVM options to this script.
set DEFAULT_JVM_OPTS="-Xmx64m" "-Xms64m"

@rem Find java.exe
if defined JAVA_HOME goto findJavaFromJavaHome

set JAVA_EXE=java.exe
%JAVA_EXE% -version >NUL 2>&1
if %ERRORLEVEL% equ 0 goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is not set and no 'java' command could be found in your PATH. 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:findJavaFromJavaHome
set JAVA_HOME=%JAVA_HOME:"=%
set JAVA_EXE=%JAVA_HOME%/bin/java.exe

if exist "%JAVA_EXE%" goto execute

echo. 1>&2
echo ERROR: JAVA_HOME is set to an invalid directory: %JAVA_HOME% 1>&2
echo. 1>&2
echo Please set the JAVA_HOME variable in your environment to match the 1>&2
echo location of your Java installation. 1>&2

goto fail

:execute
@rem Setup the command line

set CLASSPATH=

```

```
76
77 @rem Execute Gradle
78 "%JAVA_EXE%" %DEFAULT_JVM_OPTS% %JAVA_OPTS% %GRADLE_OPTS% "-Dorg.gradle.appname=%APP_BASE_NAME%" -classpath "%CLASSPATH%" -
   jar "%APP_HOME%\gradle\wrapper\gradle-wrapper.jar" %*
79
80 :end
81 @rem End local scope for the variables with windows NT shell
82 if %ERRORLEVEL% equ 0 goto mainEnd
83
84 :fail
85 rem Set variable GRADLE_EXIT_CONSOLE if you need the _script_ return code instead of
86 rem the _cmd.exe /c_ return code!
87 set EXIT_CODE=%ERRORLEVEL%
88 if %EXIT_CODE% equ 0 set EXIT_CODE=1
89 if not ""=="%GRADLE_EXIT_CONSOLE%" exit %EXIT_CODE%
90 exit /b %EXIT_CODE%
91
92 :mainEnd
93 if "%OS%"=="Windows_NT" endlocal
94
95 :omega
```

.github/workflows/android-apk.yml

```
1  # Builds the Android app and puts the debug APK in "Android App/Smart-Bin vX.Y.apk",
2  # replacing the older one, so the APK can be made without a laptop.
3  # The Firebase account is never baked in: local.properties holds only sdk.dir here.
4  # Debug tip: if the APK is not updated after a push, open the Actions tab on
5  # GitHub; a failed "Build" step shows the Gradle error in its log.
6  name: Android APK
7
8  on:
9    workflow_dispatch:
10   push:
11     branches: ['claude/**']
12     paths: ['Android App/app/**', 'Android App/*.kts', 'Android App/gradle/**', '.github/workflows/android-apk.yml']
13
14 permissions:
15   contents: write
16
17 concurrency:
18   group: apk-${{ github.ref }}
19   cancel-in-progress: true
20
21 jobs:
22   build:
23     runs-on: ubuntu-latest
24     defaults:
25       run:
26         working-directory: Android App
27     steps:
28       - uses: actions/checkout@v4
29       - uses: actions/setup-java@v4
30         with:
31           distribution: temurin
32           java-version: '21'
33       # The runner image already has the Android SDK; AGP fetches any missing platform.
34       - name: Point Gradle at the SDK
35         run: |
36           sdk="${ANDROID_HOME:-/usr/local/lib/android/sdk}"
37           yes | "$sdk/cmdline-tools/latest/bin/sdkmanager" --licenses > /dev/null || true
38           echo "sdk.dir=$sdk" > local.properties
39       - name: Build
40         run: |
41           chmod +x gradlew
42           ./gradlew :app:assembleDebug --no-configuration-cache --stacktrace
43       - name: Put the APK in the repo
44         run: |
45           ver=$(sed -n 's/.*versionName = "\(.*\)".*/\1/p' app/build.gradle.kts | head -1)
46           name="Smart-Bin v$ver.apk"
47           git rm -q --ignore-unmatch Smart-Bin\ v*.apk
48           cp app/build/outputs/apk/debug/app-debug.apk "$name"
49           git add -f "$name"
50           git config user.name "github-actions[bot]"
51           git config user.email "41898282+github-actions[bot]@users.noreply.github.com"
52           if git diff --cached --quiet; then echo "No change"; exit 0; fi
53           git commit -q -m "Build $name"
54           git push origin HEAD:${GITHUB_REF_NAME}
55       - uses: actions/upload-artifact@v4
56         with:
57           name: smart-bin-apk
58           path: Android App/app/build/outputs/apk/debug/app-debug.apk
```

Android App/tools/make_sounds.py

```
# ============================================================================
# make_sounds.py - generates every WAV in app/src/main/res/raw (development tool)
# Each sound is a list of notes rendered as sine waves with a 2nd and 3rd
# harmonic (the "timbre"), a short attack and release, and an exponential
# fade. Output is 22,050 Hz, 16-bit mono. Rerun it after editing SOUNDS, then
# rebuild the app so the new files are packed into the APK.
# ============================================================================
"""Builds the Smart-Bin app's sounds into app/src/main/res/raw.

Every action in the app has its own short tune, and every kind of alert has
a longer one, so the phone can be told apart by ear. The rules follow the
Garbage-Guard app: rising pitch means forward, on, open or OK; falling pitch
means back, off, close or no. Button sounds are quiet, events louder, and
alerts loudest with a buzzier tone.

The SOUNDS table is plain data, so the ESP32 buzzer can later play the same
tunes for the same actions and the bin and the phone will sound alike.

Run from anywhere after changing the table:
    python3 "Android App/tools/make_sounds.py"
"""
import math
import struct
import wave
from pathlib import Path

OUT = Path(__file__).resolve().parents[1] / "app" / "src" / "main" / "res" / "raw"
RATE = 22050
ATTACK = 0.004    # seconds
RELEASE = 0.014   # seconds

# name: (group, notes). A note is (hz, ms), (start_hz, end_hz, ms) for a sweep, or (0, ms) for a rest.
SOUNDS = {
    # Keys and switches.
    "tap":         ("ui", [(1320, 22)]),
    "choice":      ("ui", [(1180, 16), (1480, 18)]),
    "switch_on":   ("ui", [(880, 40), (1320, 55)]),
    "switch_off":  ("ui", [(1320, 40), (880, 55)]),
    "step_up":     ("ui", [(1400, 24)]),
    "step_down":   ("ui", [(1100, 24)]),
    "tab":         ("ui", [(990, 26), (0, 10), (1245, 30)]),
    "open":        ("ui", [(740, 35), (990, 50)]),
    "back":        ("ui", [(990, 35), (740, 50)]),
    "confirm":     ("ui", [(880, 50), (1175, 50), (1760, 90)]),
    "danger":      ("ui", [(440, 90), (0, 30), (440, 90)]),
    "denied":      ("ui", [(392, 70), (0, 40), (330, 120)]),
    "logo":        ("ui", [(784, 70), (988, 70), (1175, 70), (1568, 170)]),
    "theme_light": ("ui", [(1175, 45), (1568, 75)]),
    "theme_dark":  ("ui", [(784, 45), (587, 75)]),
    # Things that happen to a bin or to the link.
    "boot":        ("event", [(523, 80), (659, 80), (784, 80), (1047, 220)]),
    "saved":       ("event", [(1047, 60), (1319, 100)]),
    "link_up":     ("event", [(659, 70), (880, 70), (1319, 150)]),
    "link_lost":   ("event", [(880, 90), (659, 90), (440, 190)]),
    "cloud":       ("event", [(880, 60), (0, 30), (1047, 60), (0, 30), (784, 130)]),
    "lock":        ("event", [(660, 40), (0, 20), (440, 120)]),
    "unlock":      ("event", [(440, 40), (0, 20), (660, 120)]),
    "pickup":      ("event", [(523, 70), (659, 70), (784, 70), (659, 70), (784, 150)]),
    "done":        ("event", [(784, 80), (1047, 80), (1319, 80), (1568, 230)]),
    "calib":       ("event", [(988, 40), (0, 50), (988, 40), (0, 50), (988, 40), (0, 50), (1319, 130)]),
    "sim":         ("event", [(523, 60), (622, 60), (740, 60), (880, 140)]),
    "bin_lost":    ("event", [(740, 100), (0, 60), (554, 210)]),
    "bin_back":    ("event", [(554, 80), (740, 150)]),
    # The operator PIN: a dry tick per digit, a key-turn into operator, and a
    # falling bolt back to collector. A wrong PIN buzzes low; a lockout drops further.
    "pin_key":     ("ui", [(1660, 14)]),
    "pin_wrong":   ("event", [(349, 60), (0, 30), (349, 60), (0, 30), (262, 160)]),
    "pin_lockout": ("event", [(523, 260, 260), (0, 60), (196, 300)]),
    "op_in":       ("event", [(587, 50), (0, 25), (880, 50), (1175, 60), (1760, 160)]),
    "op_out":      ("event", [(1175, 50), (880, 50), (0, 25), (587, 140)]),
    # A bin confirming a command or not, Bluetooth pairing, the bin joining
    # Wi-Fi, and a new operator PIN: each its own, so none borrows another's.
    "cmd_ok":      ("event", [(1319, 50), (0, 25), (1760, 110)]),
    "cmd_fail":    ("event", [(659, 80), (0, 30), (523, 80), (0, 30), (392, 200)]),
    "bt_paired":   ("event", [(1175, 40), (1568, 40), (1175, 40), (1976, 140)]),
    "bt_wrong":    ("event", [(466, 50), (0, 25), (466, 50), (370, 140)]),
    "bt_blocked":  ("event", [(622, 180), (0, 50), (311, 320)]),
    "wifi_joined": ("event", [(880, 60), (1175, 60), (1480, 60), (1760, 180)]),
    "pin_set":     ("event", [(1047, 50), (1397, 50), (1047, 50), (1397, 120)]),
    # v2: the bin could not join the Wi-Fi, Bluetooth switched off on the bin,
    # and demo mode turned on at the start page.
    "wifi_failed": ("event", [(740, 60), (0, 25), (554, 60), (0, 25), (440, 90), (370, 180)]),
    "bt_off":      ("event", [(1568, 35), (1175, 35), (784, 35), (523, 150)]),
```

```
    "demo":        ("event", [(659, 60), (784, 60), (988, 60), (784, 60), (1319, 160)]),
    # Notification sounds, one per kind of alert. They also play inside the app.
    "alert_full":   ("alert", [(880, 180), (0, 80), (880, 180), (0, 80), (880, 180), (0, 200), (1175, 420)]),
    "alert_tamper": ("alert", [(700, 1400, 340), (1400, 700, 340), (700, 1400, 340), (1400, 700, 340)]),
    "alert_sensor": ("alert", [(1047, 120), (784, 120), (1047, 120), (784, 260)]),
    "alert_lock":   ("alert", [(660, 70), (0, 30), (440, 220)]),
    "alert_pickup": ("alert", [(523, 90), (659, 90), (784, 90), (1047, 260)]),
    "alert_info":   ("alert", [(988, 90), (1319, 170)]),
}

LEVEL = {"ui": 0.5, "event": 0.72, "alert": 0.85}
# How much of the 2nd and 3rd harmonic each group carries. Alerts buzz a little, keys stay soft.
TIMBRE = {"ui": (0.18, 0.0), "event": (0.22, 0.05), "alert": (0.35, 0.22)}


# One note or sweep (hz0 to hz1) as a list of samples between -1 and 1.
def note(hz0, hz1, secs, level, timbre):
    n = int(RATE * secs)
    h2, h3 = timbre
    norm = 1 + h2 + h3
    out = []
    phase = 0.0
    for i in range(n):
        t = i / RATE
        env = min(1.0, t / ATTACK) * min(1.0, (secs - t) / RELEASE)
        env *= math.exp(-1.6 * t / max(secs, 0.05))
        hz = hz0 + (hz1 - hz0) * (i / max(n - 1, 1))
        phase += 2 * math.pi * hz / RATE
        s = math.sin(phase) + h2 * math.sin(2 * phase) + h3 * math.sin(3 * phase)
        out.append(level * env * s / norm)
    return out


# A whole tune: notes, sweeps and rests (hz 0) joined, plus 12 ms of silence.
def render(group, notes):
    level, timbre = LEVEL[group], TIMBRE[group]
    samples = []
    for n in notes:
        if len(n) == 3:
            samples += note(n[0], n[1], n[2] / 1000.0, level, timbre)
        elif n[0] == 0:
            samples += [0.0] * int(RATE * n[1] / 1000.0)
        else:
            samples += note(n[0], n[0], n[1] / 1000.0, level, timbre)
    return samples + [0.0] * int(RATE * 0.012)


def write(path, samples):
    with wave.open(str(path), "wb") as w:
        w.setnchannels(1)
        w.setsampwidth(2)
        w.setframerate(RATE)
        w.writeframes(b"".join(struct.pack("<h", int(max(-1, min(1, s)) * 32767)) for s in samples))


def main():
    OUT.mkdir(parents=True, exist_ok=True)
    for name, (group, notes) in SOUNDS.items():
        write(OUT / f"sb_{name}.wav", render(group, notes))
    print(f"wrote {len(SOUNDS)} sounds to {OUT}")


if __name__ == "__main__":
    main()
```

Android App/tools/smartbin_mock_server.py

```
#!/usr/bin/env python3
# ============================================================================
# smartbin_mock_server.py - fake bins, Firebase and ThingSpeak for testing
# (development tool, not used in the finished system)
# Lets the app and dashboard be tested with no ESP32. Each fake bin follows
# the same rules as the firmware (fill rises, tilt over tilt_th means TAMPER,
# simulation mask bits override readings), so most app bugs can be reproduced
# here first.
# ============================================================================
"""
Stand-in Smart-Bin bins for testing the Android app with no hardware.

Each bin answers the same local REST API the ESP32 firmware will serve:

    GET  /api/status     readings, status, calibration and simulation mask
    POST /api/command    {"cmd": "LOCK"} and the other commands
    POST /api/calibrate  calibration values
    POST /api/simulate   {"mask": 5, "values": {...}}
```

```
19
20 Optionally it also plays a small Firebase Realtime Database on another port,
21 with the same paths the app uses for cloud mode:
22
23     /sb/bins/<id>/status.json    GET   the bin's last status, ts and local_ip
24     /sb/commands/<id>.json       POST  LOCK, UNLOCK or COLLECT
25     /sb/usage/bytes_month.json   GET
26     /sb_users/<uid>.json         GET   the allowed-user check
27     /__auth/signInWithPassword   POST  sign-in, any email and a password of 6+ characters
28
29 and a small ThingSpeak on a third port, with --ts PORT:
30
31     /channels/<id>/feeds/last.json   GET   the newest entry, fields 1 to 8 and the status text
32     /talkbacks/<id>/commands.json    POST  command_string=LOCK, UNLOCK or COLLECT
33     Bin 1 is channel 1001 and TalkBack 2001, bin 2 is 1002 and 2002, and so on. Any key works.
34
35 Plain Python 3, no packages. Run it on a computer on the same Wi-Fi as the phone:
36
37     python smartbin_mock_server.py                two bins on ports 8081 and 8082
38     python smartbin_mock_server.py --bins 3       three bins, 8081 to 8083
39     python smartbin_mock_server.py --cloud 8090   also the fake Firebase on 8090
40     python smartbin_mock_server.py --ts 8095      also the fake ThingSpeak on 8095
41
42 In the app, open a bin with SET and type <computer IP>:8081 as its address.
43 For cloud mode, use http://<computer IP>:8090 as the database URL, any API
44 key, any email and a 6+ character password, then stop the bins with
45 --no-local so only the cloud answers. For ThingSpeak, put http://<computer IP>:8095
46 as the server on the Connection page, and channel 1001, TalkBack 2001 (any keys)
47 in bin 1's SET sheet.
48 """
49
50 import argparse
51 import json
52 import random
53 import socket
54 import threading
55 import time
56 from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer
57 from urllib.parse import parse_qs, urlparse
58
59 DEFAULT_CAL = {
60     "d_empty": 30, "d_full": 5, "fill_th": 80, "temp_th": 40, "hum_th": 85, "gas_th": 200,
61     "tilt_th": 30, "weight_cap": 4500, "auto_lock": 0, "servo_closed": 0, "servo_open": 90,
62     "collect_pin": 1234, "pir_hold_s": 5, "buzz_s": 2, "temp_off": -2, "hum_off": 0, "press_off": 0,
63 }
64 DEFAULT_SIM = {"fill": 60, "weight": 2000, "gas": 120, "temp": 28, "humidity": 60, "tamper": 0, "motion": 0, "lid": 0}
65
66
67 # One fake bin with its own port, readings, calibration and simulation.
68 class Bin:
69     """One bin, stepped once a second the same way the app's demo mode steps it."""
70
71     def __init__(self, index, port):
72         self.id = "bin%02d" % (index + 1)
73         self.port = port
74         self.lock = threading.Lock()
75         self.cal = dict(DEFAULT_CAL)
76         self.sim_mask = 0
77         self.sim = dict(DEFAULT_SIM)
78         self.fill = [34.0, 62.0, 18.0][index % 3]
79         self.rate = [0.38, 3.1, 0.9][index % 3] * 60  # percent per hour, sped up 60 times
80         self.temp_raw = 30.0
81         self.hum = 62.0
82         self.press = 1008.4
83         self.gas = 60 + self.fill
84         self.iaq_acc = 3
85         self.tilt = 0.0
86         self.motion = False
87         self.lid_open = False
88         self.status = "OK"
89         self.calibrating = False
90
91     def on(self, bit):
92         return (self.sim_mask >> bit) & 1 == 1
93
94     # Called about once a second: moves the readings and works out the
95     # status the same way control.cpp does.
96     def step(self):
97         with self.lock:
98             c, s = self.cal, self.sim
99             if self.calibrating:
100                 self.status = "CALIBRATING"
101                 self.lid_open = False
102                 return
103             if self.status == "CALIBRATING":
104                 self.status = "OK"
```

```
            if self.on(0):
                self.fill = float(s["fill"])
            elif self.status != "COLLECTING":
                self.fill = min(100.0, self.fill + self.rate / 3600 * random.uniform(0.7, 1.3))
            if self.on(2):
                self.gas = float(s["gas"])
            else:
                self.gas += ((35 + self.fill * 2.2) - self.gas) * 0.02 + random.uniform(-2.5, 2.5)
            if not self.on(3):
                self.temp_raw = min(34, max(24, self.temp_raw + random.uniform(-0.04, 0.04)))
                self.hum = min(95, max(35, self.hum + random.uniform(-0.3, 0.3)))
            self.press += random.uniform(-0.03, 0.03)
            self.tilt = 38.0 if self.on(4) and s["tamper"] else random.uniform(0, 2.2)
            self.motion = bool(s["motion"]) if self.on(5) else random.random() < 0.025
            if self.status == "COLLECTING":
                self.lid_open = True
            elif self.tilt > c["tilt_th"]:
                self.status, self.lid_open = "TAMPER", False
            elif self.status in ("TAMPER", "LOCKED"):
                self.lid_open = False
            elif c["auto_lock"] and self.fill >= c["fill_th"]:
                self.status, self.lid_open = "LOCKED", False
            else:
                self.lid_open = bool(s["lid"]) if self.on(6) else self.motion

    def status_json(self):
        with self.lock:
            c, s = self.cal, self.sim
            temp = float(s["temp"]) if self.on(3) else self.temp_raw + c["temp_off"]
            hum = float(s["humidity"]) if self.on(3) else self.hum
            weight = float(s["weight"]) if self.on(1) else round(self.fill / 100 * c["weight_cap"] * 0.92 + random.uniform(0,
25))
            distance = c["d_empty"] - self.fill / 100 * (c["d_empty"] - c["d_full"])
            return {
                "fill": round(self.fill, 1), "distance_cm": round(distance, 1), "weight_g": weight,
                "temp": round(temp, 2), "temp_raw": round(temp - c["temp_off"], 2), "humidity": round(hum, 1),
                "pressure": round(self.press, 1), "iaq": round(self.gas), "iaq_acc": self.iaq_acc,
                "tilt": round(self.tilt, 1), "motion": self.motion, "lid": "OPEN" if self.lid_open else "CLOSED",
                "status": self.status, "cal": c, "sim_mask": self.sim_mask,
            }

    def command(self, cmd, body):
        with self.lock:
            if cmd == "LOCK":
                self.status, self.lid_open = "LOCKED", False
            elif cmd == "UNLOCK":
                self.status = "OK"
            elif cmd == "CLEAR_TAMPER":
                self.status, self.tilt = "OK", 0.0
                self.sim["tamper"] = 0
            elif cmd == "COLLECT":
                self.status, self.lid_open = "COLLECTING", True
            elif cmd == "COLLECT_DONE":
                self.status, self.fill, self.lid_open = "OK", 0.0, False
                self.gas = min(self.gas, 48)
            elif cmd == "CALIBRATE_START":
                self.calibrating = True
            elif cmd == "CALIBRATE_END":
                self.calibrating = False
            elif cmd == "RESET_GAS":
                self.iaq_acc = 0
            elif cmd in ("TARE", "CAL_WEIGHT"):
                pass
            else:
                return False
            print("  %s: %s" % (self.id, cmd), flush=True)
            return True


def local_handler(b):
    class H(BaseHTTPRequestHandler):
        def log_message(self, *a):
            pass

        def send(self, code, obj):
            data = json.dumps(obj).encode()
            self.send_response(code)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(data)))
            self.send_header("Access-Control-Allow-Origin", "*")
            self.send_header("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS")
            self.send_header("Access-Control-Allow-Headers", "Content-Type, X-SB-Op")
            self.end_headers()
            self.wfile.write(data)

        def body(self):
```

```
            n = int(self.headers.get("Content-Length") or 0)
            try:
                return json.loads(self.rfile.read(n) or b"{}")
            except ValueError:
                return {}


        def do_OPTIONS(self):
            self.send_response(204)
            self.send_header("Access-Control-Allow-Origin", "*")
            self.send_header("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS")
            self.send_header("Access-Control-Allow-Headers", "Content-Type, X-SB-Op")
            self.send_header("Content-Length", "0")
            self.end_headers()

        def do_GET(self):
            if urlparse(self.path).path == "/api/status":
                self.send(200, b.status_json())
            else:
                self.send(404, {"error": "not found"})

        def do_POST(self):
            p = urlparse(self.path).path
            j = self.body()
            if p == "/api/command":
                ok = b.command(str(j.get("cmd", "")), j)
                self.send(200 if ok else 400, {"ok": ok})
            elif p == "/api/calibrate":
                with b.lock:
                    for k, v in j.items():
                        if k in b.cal:
                            b.cal[k] = v
                print("  %s: calibration saved" % b.id, flush=True)
                self.send(200, {"ok": True})
            elif p == "/api/simulate":
                with b.lock:
                    b.sim_mask = int(j.get("mask", 0))
                    b.sim.update(j.get("values", {}))
                print("  %s: simulation mask %d" % (b.id, b.sim_mask), flush=True)
                self.send(200, {"ok": True})
            else:
                self.send(404, {"error": "not found"})

    return H


# Fake Firebase: an in-memory JSON tree with GET, PUT, PATCH, DELETE and POST.
def cloud_handler(bins, ip):
    by_id = {b.id: b for b in bins}
    sent = {"bytes": 0}
    tree = {}

    def tree_get(parts):
        node = tree
        for k in parts:
            if not isinstance(node, dict) or k not in node:
                return None
            node = node[k]
        return node

    def tree_set(parts, value, merge=False):
        node = tree
        for k in parts[:-1]:
            node = node.setdefault(k, {})
        if value is None:
            node.pop(parts[-1], None)
        elif merge and isinstance(node.get(parts[-1]), dict) and isinstance(value, dict):
            node[parts[-1]].update(value)
        else:
            node[parts[-1]] = value

    class H(BaseHTTPRequestHandler):
        def log_message(self, *a):
            pass

        def send(self, code, obj):
            data = json.dumps(obj).encode()
            sent["bytes"] += len(data)
            self.send_response(code)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(data)))
            self.send_header("Access-Control-Allow-Origin", "*")
            self.send_header("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS")
            self.send_header("Access-Control-Allow-Headers", "Content-Type, X-SB-Op")
            self.end_headers()
            self.wfile.write(data)
```

```
276
277         def body(self):
278             n = int(self.headers.get("Content-Length") or 0)
279             try:
280                 return json.loads(self.rfile.read(n) or b"{}")
281             except ValueError:
282                 return {}
283
284         def do_GET(self):
285             u = urlparse(self.path)
286             parts = u.path.strip("/").removesuffix(".json").split("/")
287             if parts[:2] == ["sb", "bins"] and len(parts) == 2:
288                 self.send(200, {k: True for k in by_id})
289             elif parts[:2] == ["sb", "bins"] and len(parts) == 4 and parts[3] == "status" and parts[2] in by_id:
290                 b = by_id[parts[2]]
291                 s = b.status_json()
292                 s["ts"] = int(time.time() * 1000)
293                 s["local_ip"] = "%s:%d" % (ip, b.port)
294                 self.send(200, s)
295             elif parts == ["sb", "usage", "bytes_month"]:
296                 self.send(200, sent["bytes"])
297             elif parts == ["sb", "usage"]:
298                 self.send(200, {"bytes_month": sent["bytes"], "limit_bytes": 10737418240})
299             elif parts[0] == "sb_users":
300                 self.send(200, {"role": "collector"})
301             else:
302                 self.send(200, tree_get(parts))
303
304         def do_OPTIONS(self):
305             self.send_response(204)
306             self.send_header("Access-Control-Allow-Origin", "*")
307             self.send_header("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS")
308             self.send_header("Access-Control-Allow-Headers", "Content-Type, X-SB-Op")
309             self.send_header("Content-Length", "0")
310             self.end_headers()
311
312         def do_PUT(self, merge=False):
313             parts = urlparse(self.path).path.strip("/").removesuffix(".json").split("/")
314             j = self.body()
315             if isinstance(j, dict) and isinstance(j.get("ts"), dict):
316                 j["ts"] = int(time.time() * 1000)
317             tree_set(parts, j, merge)
318             self.send(200, j)
319
320         def do_PATCH(self):
321             self.do_PUT(merge=True)
322
323         def do_DELETE(self):
324             parts = urlparse(self.path).path.strip("/").removesuffix(".json").split("/")
325             tree_set(parts, None)
326             self.send(200, None)
327
328         def do_POST(self):
329             u = urlparse(self.path)
330             if u.path.startswith("/__auth/signInWithPassword"):
331                 j = self.body()
332                 if "@" not in str(j.get("email", "")) or len(str(j.get("password", ""))) < 6:
333                     self.send(400, {"error": {"message": "INVALID_LOGIN_CREDENTIALS"}})
334                 else:
335                     self.send(200, {"idToken": "mock-token", "localId": "mock-user", "expiresIn": "3600"})
336                 return
337             parts = u.path.strip("/").removesuffix(".json").split("/")
338             if parts[:2] == ["sb", "commands"] and len(parts) == 3 and parts[2] in by_id:
339                 cmd = str(self.body().get("cmd", ""))
340                 ok = cmd in ("LOCK", "UNLOCK", "COLLECT") and by_id[parts[2]].command(cmd, {})
341                 self.send(200 if ok else 400, {"name": "-mock%d" % int(time.time())} if ok else {"error": "Permission
    denied"})
342             else:
343                 self.send(404, {"error": "not found"})
344
345     return H
346
347
348 STATUS_CODES = ["OK", "LOCKED", "TAMPER", "COLLECTING", "CALIBRATING"]
349
350
351 # Fake ThingSpeak: last feed per channel and a TalkBack command queue.
352 def ts_handler(bins, ip):
353     """ThingSpeak's read and TalkBack calls, laid out as the app reads them."""
354
355     class H(BaseHTTPRequestHandler):
356         def log_message(self, *a):
357             pass
358
359         def send(self, code, obj):
360             data = json.dumps(obj).encode()
```

```
            self.send_response(code)
            self.send_header("Content-Type", "application/json")
            self.send_header("Content-Length", str(len(data)))
            self.send_header("Access-Control-Allow-Origin", "*")
            self.send_header("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS")
            self.send_header("Access-Control-Allow-Headers", "Content-Type, X-SB-Op")
            self.end_headers()
            self.wfile.write(data)

        def bin_for(self, n, base):
            k = n - base
            return bins[k] if 0 <= k < len(bins) else None

        def do_GET(self):
            parts = urlparse(self.path).path.strip("/").split("/")
            if len(parts) == 4 and parts[0] == "channels" and parts[2:] == ["feeds", "last.json"] and parts[1].isdigit():
                b = self.bin_for(int(parts[1]), 1001)
                if b is None:
                    self.send(404, -1)
                    return
                s = b.status_json()
                code = STATUS_CODES.index(s["status"]) if s["status"] in STATUS_CODES else 0
                flags = (1 if s["motion"] else 0) + (2 if s["lid"] == "OPEN" else 0) + 4 * code
                self.send(200, {
                    "created_at": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime()), "entry_id": int(time.time()),
                    "field1": str(s["fill"]), "field2": str(s["weight_g"]), "field3": str(s["temp"]), "field4":
str(s["humidity"]),
                    "field5": str(s["iaq"]), "field6": str(s["pressure"]), "field7": str(s["tilt"]), "field8": str(flags),
                    "status": "%s:%d" % (ip, b.port),
                })
            elif len(parts) == 3 and parts[0] == "channels" and parts[2] == "feeds.json" and parts[1].isdigit():
                b = self.bin_for(int(parts[1]), 1001)
                if b is None:
                    self.send(404, -1)
                    return
                s = b.status_json()
                now = time.time()
                feeds = []
                for i in range(48, -1, -1):
                    f = max(0.0, s["fill"] - i * 0.6)
                    feeds.append({"created_at": time.strftime("%Y-%m-%dT%H:%M:%SZ", time.gmtime(now - i * 1800)), "entry_id":
1000 - i,
                                  "field1": str(round(f, 1)), "field2": str(round(f * 40)), "field3": str(s["temp"]),
"field4": str(s["humidity"]),
                                  "field5": str(s["iaq"]), "field6": str(s["pressure"]), "field7": "0", "field8": "0"})
                self.send(200, {"channel": {"id": int(parts[1])}, "feeds": feeds})
            else:
                self.send(404, -1)

        def do_OPTIONS(self):
            self.send_response(204)
            self.send_header("Access-Control-Allow-Origin", "*")
            self.send_header("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS")
            self.send_header("Access-Control-Allow-Headers", "Content-Type, X-SB-Op")
            self.send_header("Content-Length", "0")
            self.end_headers()

        def do_POST(self):
            parts = urlparse(self.path).path.strip("/").split("/")
            n = int(self.headers.get("Content-Length") or 0)
            form = parse_qs((self.rfile.read(n) or b"").decode())
            if len(parts) == 3 and parts[0] == "talkbacks" and parts[2] == "commands.json" and parts[1].isdigit():
                b = self.bin_for(int(parts[1]), 2001)
                cmd = (form.get("command_string") or [""])[0]
                if b is not None and cmd in ("LOCK", "UNLOCK", "COLLECT") and b.command(cmd, {}):
                    self.send(200, {"id": int(time.time()), "command_string": cmd, "position": 1})
                else:
                    self.send(400, {"status": "400", "error": "bad command"})
            else:
                self.send(404, -1)

    return H


def my_ip():
    s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    try:
        s.connect(("10.255.255.255", 1))
        return s.getsockname()[0]
    except OSError:
        return "127.0.0.1"
    finally:
        s.close()


def main():
```

```
    ap = argparse.ArgumentParser(description="Stand-in Smart-Bin bins for the Android app.")
    ap.add_argument("--bins", type=int, default=2, help="how many bins (default 2)")
    ap.add_argument("--port", type=int, default=8081, help="port of the first bin (default 8081)")
    ap.add_argument("--cloud", type=int, default=0, help="also serve a fake Firebase on this port")
    ap.add_argument("--ts", type=int, default=0, help="also serve a fake ThingSpeak on this port")
    ap.add_argument("--no-local", action="store_true", help="do not answer on the bin ports, to test cloud mode")
    a = ap.parse_args()

    ip = my_ip()
    bins = [Bin(i, a.port + i) for i in range(max(1, a.bins))]
    servers = []
    if not a.no_local:
        for b in bins:
            servers.append(ThreadingHTTPServer(("0.0.0.0", b.port), local_handler(b)))
            print("%s answers at %s:%d" % (b.id, ip, b.port))
    if a.cloud:
        servers.append(ThreadingHTTPServer(("0.0.0.0", a.cloud), cloud_handler(bins, ip)))
        print("Fake Firebase at http://%s:%d" % (ip, a.cloud))
    if a.ts:
        servers.append(ThreadingHTTPServer(("0.0.0.0", a.ts), ts_handler(bins, ip)))
        print("Fake ThingSpeak at http://%s:%d (channels %s)" % (ip, a.ts, ", ".join(str(1001 + i) for i in
range(len(bins)))))
    for s in servers:
        threading.Thread(target=s.serve_forever, daemon=True).start()
    print("Press Ctrl+C to stop.", flush=True)
    try:
        while True:
            for b in bins:
                b.step()
            time.sleep(1)
    except KeyboardInterrupt:
        for s in servers:
            s.shutdown()


if __name__ == "__main__":
    main()
```